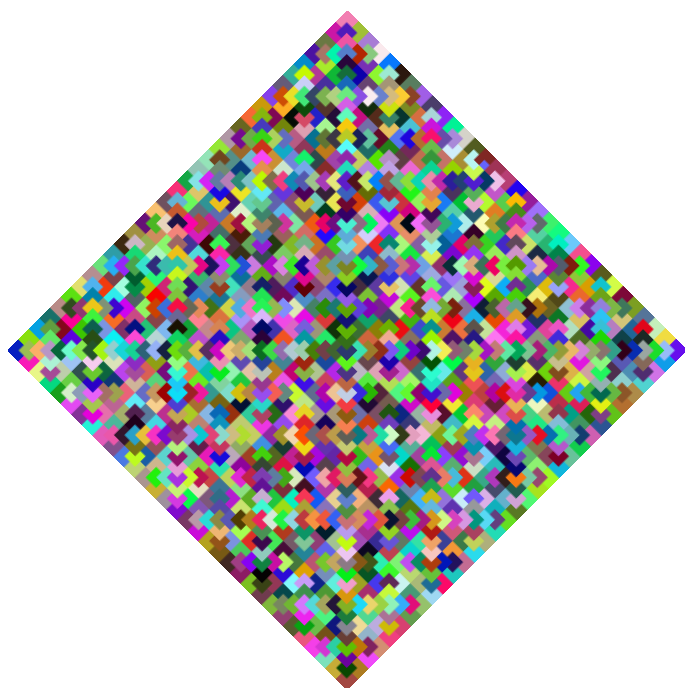




Diseño y Análisis de Algoritmos

Camilo Rocha

Email address: me@camilorochoa.info

Para Laura.

© Derechos de autor 2019-2026 Camilo Rocha.
Última actualización 7 de septiembre de 2026.
Versión 0.0



Esta obra está bajo una licencia de Creative Commons
Reconocimiento-NoComercial-CompartirIgual 4.0 Internacional.
Este trabajo puede ser copiado y distribuido libremente, como copia electrónica o en
papel. No puede ser vendido por un valor mayor a su costo actual de reproducción,
almacenamiento o transmisión.

Índice general

Capítulo 0. Preliminares	1
0.1. Arreglos: notación y convenciones	1
0.2. Problemas algorítmicos y su especificación	3
0.3. El concepto de algoritmo	11
0.4. Especificación de algoritmos	16
Notas del capítulo y referencias	18
Capítulo 1. Análisis asintótico	19
1.1. Medidas para problemas y algoritmos	20
1.2. Notación asintótica	23
1.3. Propiedades de la notación asintótica	29
1.4. El Teorema Maestro	33
Notas del capítulo y referencias	39
Capítulo 2. Dividir, conquistar y combinar	41
2.1. Teselación de tableros de ajedrez	41
2.2. Ordenamiento de arreglos	47
2.3. Una versión iterativa del ordenamiento	53
2.4. <i>Mergesort</i> : un ordenamiento de arreglos más eficiente	58
2.5. Búsqueda binaria	65
Notas del capítulo y referencias	72
Capítulo 3. Programación dinámica	73
3.1. Ideas clave	75

3.2. Cálculo de los números de Fibonacci	81
3.3. Una metodología	93
3.4. Suma máxima de un subarreglo	99
3.5. Subsecuencia creciente más larga	109
3.6. El problema del morral	119
3.7. Subsecuencia común más larga	134
3.8. Caminos mínimos entre todos los pares	146
3.9. El problema del agente viajero	161
Notas del capítulo y referencias	173
Capítulo 4. Algoritmos voraces	177
4.1. Ideas clave	179
4.2. Agendamiento de actividades	189
4.3. Una metodología	203
4.4. Cubrimiento de intervalos	208
4.5. Tiempo en el sistema	221
4.6. El morral fraccionario	230
4.7. Árboles de cubrimiento mínimo	239
4.8. Caminos más cortos	273
Notas del capítulo y referencias	286
Capítulo 5. Reintento	289
5.1. Ideas clave	290
5.2. Generación de objetos combinatorios	299
5.3. Una metodología	306
5.4. El problema de las N reinas	310
5.5. Suma exacta de un subconjunto	321
5.6. Conteo de segmentaciones de texto	331
5.7. Ramificación y poda: el problema del morral	345
5.8. El agente viajero, revisitado	356
Notas del capítulo y referencias	363
Capítulo 6. Problemas y algoritmos de decisión	367
6.1. Problemas de decisión	368
6.2. Un marco universal basado en lenguajes	370
6.3. Aceptación y decisión	374

6.4. Computabilidad	378
6.5. Indecidibilidad	383
Notas del capítulo y referencias	391
Capítulo 7. NP-completitud: P vs. NP	393
7.1. La clase P	395
7.2. La clase NP	400
7.3. Reducciones polinomiales	405
7.4. La clase NPC	410
Bibliografía	415

Preliminares

El propósito de un algoritmo es resolver mecánicamente un problema –o, equivalentemente, todas las instancias de un problema. Contar con una noción de algoritmo para este propósito requiere antes precisar qué se entiende por y cómo se define un problema algorítmico. La finalidad de este capítulo es presentar un marco general para la definición de problemas algorítmicos, e introducir convenciones para su posterior diseño y análisis.

0.1. Arreglos: notación y convenciones

Los problemas algorítmicos que se estudian en este texto, en su gran mayoría, son definidos y estudiados con base en arreglos. Un arreglo pueden contener números, caracteres, valores Booleanos, entre otros. También se pueden componer entre sí para formar matrices, hipermatrices como cubos, etc.

Definición 0.1.1

Un *arreglo* de tamaño N es una estructura de datos que almacena una colección de elementos (o valores), cada uno identificado por un *índice* correspondiente a un número natural en el rango $0 .. N - 1$.

No es de interés en este texto estudiar detalles de implementación de arreglos en lenguajes de programación en general o en un lenguaje de programación particular. Más bien, se supondrá que dado un arreglo, el acceso a cada uno de sus elementos se hace de manera eficiente, como sucede realmente en la práctica de la programación.

Nota 0.1.1

Se usa una notación similar a la de rangos en los números reales, con paréntesis circulares $()$ y llaves cuadradas $[]$, para especificar arreglos y definir operaciones (de indexación y proyección) sobre ellos.

- La expresión $A[0..N)$ indica que A es un arreglo de N elementos con índices en el rango $0..N - 1$.
- Con $A[0..N)$ y n un número natural, la *indexación* $A[n]$ denota el valor de A en la posición n siempre y cuando $0 \leq n < N$. Si $n \geq N$, entonces la expresión es un error.

Por ejemplo, con $A = [1, 0, 25, -1, 8]$ se tiene $A[3] = -1$ y $A[6]$ es un error.

- Un arreglo sin índices se llama *vacío*.
- Con $A[0..N)$ y a, b números naturales, la expresión $A[a..b)$ denota la *sección* (o el *subarreglo*) de A , la cual es también un arreglo que:
 - cuando $0 \leq a < b \leq N$, re-indexa desde 0 y hasta $b - (a + 1)$, respetando su orden, los elementos de A entre a y $b - 1$ (i.e., b no se incluye), y
 - de lo contrario, el arreglo vacío.

Por ejemplo, $A[1..3) = [0, 25]$ con $A = [1, 0, 25, -1, 8]$; $A[2..2)$ denota el arreglo vacío.

- El *tamaño* (o, equivalentemente, *la cantidad de índices*) de un arreglo o una sección de un arreglo se denota como $|-|$.

Adoptando la notación introducida para arreglos en la Nota 0.1.1, una matriz A de tamaño $N \times M$ se especifica como $A[0..N)[0..M)$, y así sucesivamente. Una cadena de tamaño N puede representarse como un arreglo de tamaño N . Abusando un poco de la notación, las expresiones $A[0..N)$ y $A[0..N - 1]$ denotan el mismo arreglo. Es importante tener en cuenta que cualquier operación de indexación en el arreglo vacío resulta en un error.

Los arreglos también se pueden concebir como objetos matemáticos. De esta forma es posible referirse a ellos de manera abstracta y formal (e.g., no atada a su implementación en un lenguaje de programación particular), posibilitando así un diseño y análisis algorítmico más enfocado en las ideas que una implementación en particular, cuando sea conveniente.

Nota 0.1.2

Matemáticamente, un arreglo $A[0..N)$ es una función A con dominio

$$\{0, 1, \dots, N - 1\}$$

y co-dominio

$$\{A[0], A[1], \dots, A[N-1]\}.$$

El conjunto vacío es el dominio y co-dominio del arreglo vacío.

Note que no se menciona el tipo de elementos que almacena un arreglo. Esta información deberá entenderse del contexto dentro del cual se usan los arreglos en cada caso o se suministrará cuando sea necesario.

Ejercicios

1. Considere dos arreglos $A[0..M)$ y $B[0..N)$.
 - a) Describa e implemente un algoritmo que permita determinar si los dos arreglos son iguales.
 - b) Describa e implemente un algoritmo que permita determinar si B es sub-arreglo de A .
2. Investigue sobre árboles binarios casi llenos (e.g., aquellos que se utilizan en la implementación de *HeapSort* como implementación de colas de prioridad). Explique brevemente cómo representar un árbol binario casi lleno (e.g., de números) en un arreglo. ¿Qué relación hay entre los índices del arreglo y la profundidad de los elementos del árbol?
3. En el lenguaje de programación Python, los arreglos son en realidad listas indexadas. Investigue sobre cuál es la principal diferencia entre los arreglos de Python y los arreglos de C/C++/Java. ¿Qué ventajas tienen los arreglos de Python sobre los arreglos en estos otros lenguajes? ¿Tienen alguna desventaja?
4. Cuando están disponibles en un lenguaje de programación imperativo, los arreglos se pueden pasar como parámetros de funciones o procedimientos. Hay diferentes formas de pasar un arreglo como parámetro: por ejemplo, por valor o por referencia. Considere las siguientes preguntas para los lenguajes de programación Python, C, C++, Java:
 - a) ¿Cuál es el mecanismo por defecto al pasar un arreglo como parámetro?
 - b) ¿Se puede modificar el comportamiento por defecto? De ser posible, ¿cómo?
 - c) ¿Cuáles son las principales ventajas y desventajas de cada uno de los mecanismos identificados anteriormente?

0.2. Problemas algorítmicos y su especificación

Un computador puede ser utilizado para asistir en la mecanización de un sinnúmero de tareas, de diferentes envergaduras y con propósitos muy diversos.

Por ejemplo, un computador puede ser utilizado para encontrar la ruta más rápida (o corta) para conducir de un lugar a otro en una ciudad. También puede ser utilizado para almacenar y analizar grandes volúmenes de datos como los asociados al genoma humano. Hoy en día, los computadores son la herramienta de trabajo en muchas profesiones y se requiere para una gran cantidad de actividades.

Generalmente, es necesario diseñar piezas algorítmicas pequeñas y con propósitos muy específicos que, al componerse con otras piezas, permitan abordar tareas de mayor dificultad, como las mencionadas anteriormente. En algunos lenguajes de programación estas piezas reciben el nombre de funciones, procedimientos o módulos.

Nota 0.2.1

El propósito de este manuscrito es presentar técnicas de diseño y análisis de algoritmos para construir satisfactoriamente piezas algorítmicas fundamentales, que junto con otras piezas pueden conformar grandes sistemas de cómputo, información o procesamiento.

Conceptualmente, se puede aceptar que un problema es una pregunta formulada y a la cual se desea encontrar respuesta. De esta forma, se puede concebir un problema algorítmico como un problema cuya respuesta puede ser encontrada con la asistencia de un computador que ejecute mecánicamente unas instrucciones.

Definición 0.2.1

La *especificación* de un problema algorítmico está conformada por dos partes: la descripción de la *entrada* y la descripción de la *salida*.

En un problema algorítmico, la descripción de la entrada identifica cuáles son los datos relevantes que hacen parte del planteamiento del problema. La descripción de la salida identifica la pregunta que se desea responder con base en los datos disponibles, previamente identificados en la descripción de la entrada.

Nota 0.2.2

Se usa el siguiente formato para presentar problemas algorítmicos:

Entrada: ...

Salida: ...

en donde a la derecha de “Entrada:” se describe la entrada del problema y a la derecha de “Salida:” la salida del problema.

El Ejemplo 0.2.1 presenta la especificación del problema de ordenamiento de un arreglo de números.

Ejemplo 0.2.1

Considere el problema de ordenar un arreglo de números. El problema algorítmico asociado puede definirse de la siguiente manera:

Entrada: Un arreglo $A[0..N)$, con $N \geq 0$, de números enteros.

Salida: El arreglo $A[0..N)$ ordenado ascendentemente.

Observe que en el proceso de especificar el problema algorítmico de ordenar ascendentemente un arreglo de números en el Ejemplo 0.2.1 se tomaron algunas decisiones. Primero, se indica concretamente la estructura de datos que representa dicha lista y cómo se llama: en este caso, la lista está dada como un arreglo llamado A y que contiene N números enteros, indexados desde 0. Segundo, se identifican qué casos de arreglos de números enteros son considerados: la entrada indica que se considera cualquier arreglo de números enteros, incluyendo el arreglo vacío (¿por qué?). Tercero, se establece unívocamente que el objetivo es ordenar el arreglo dado y no una copia de este, es decir, se plantea hacer un ordenamiento *in situ* del arreglo dado.

Un problema algorítmico puede contener “variables” a las cuales no han sido asignados valores concretos en su especificación. En el Ejemplo 0.2.1, este es el caso de A y N : indican que la entrada del problema puede ser *cualquier* arreglo (llamado A por conveniencia o por falta de imaginación) de *cualquier* tamaño. La intención de emplear variables en la especificación de un problema algorítmico, como A y N en el Ejemplo 0.2.1, es transmitir el mensaje de que su formulación es de carácter general y no particular.

Definición 0.2.2

Un *parámetro* es una variable que aparece en la especificación de un problema algorítmico y no tiene un valor concreto.

Dado que la especificación de un problema puede tener parámetros, un problema algorítmico en realidad representa una colección de problemas concretos: uno por cada asignación de valores concretos a sus parámetros y que cumple con las

condiciones dadas. Es decir, un problema algorítmico con parámetros es una expresión simbólica que representa una colección de problemas concretos (o instancias), aquellos que específicamente se desea resolver en la práctica.

Definición 0.2.3

Una *instancia* de un problema algorítmico es una asignación de valores concretos a los parámetros de su especificación (y que cumplen con las condiciones dadas).

Un problema algorítmico, en general, tiene tantas instancias como asignaciones tengan sus parámetros. De esta forma, hay problemas algorítmicos que representan una cantidad finita de instancias y otros una cantidad infinita de ellas.

Ejemplo 0.2.2

Considere el problema algorítmico presentado en el Ejemplo 0.2.1. Las siguientes son algunas de sus instancias:

- $A = []$ y $N = 0$.
- $A = [1, 2, 3]$ y $N = 3$.
- $A = [2, 7, 1, 9, 8]$ y $N = 5$.

Ejemplo 0.2.3

La noción de instancia ha sido útil desde la formación básica en matemáticas. Por ejemplo, al establecer la ecuación $(a + b)^2 = a^2 + 2ab + b^2$, con a y b números reales, lo que realmente se está indicando es que para cualquier par de números reales (i.e., instancias de a y b) se puede calcular su suma al cuadrado calculando independientemente sus cuadrados y sumando el doble de su producto.

Aún no se ha definido claramente qué se entiende por responder una pregunta algorítmica. ¿Acaso se refiere a resolver una instancia de un problema algorítmico? ¿O a resolver algunas instancias? ¿O más bien se refiere a resolver todas las instancias? A continuación se propone una noción precisa de qué significa responder una pregunta algorítmica con base en las nociones presentadas anteriormente.

Definición 0.2.4

Una *solución de una instancia* de un problema algorítmico es la respuesta a la pregunta representada por dicha instancia. Una solución de un problema algorítmico es la respuesta a todas las instancias del problema.

En un problema algorítmico, puede haber una distancia abismal entre la dificultad que presenta resolver *una* de sus instancias específicas y resolver *todas* sus instancias. Existe el caso en el cual el tamaño de una instancia es pequeño y, como tal, podría ser fácil resolverla sin ingenio alguno. Por ejemplo, ordenar un arreglo con cinco números es fácil, mientras que ordenar otro con un millón puede ser más demandante en recursos. También está el caso en el cual ciertas instancias pueden estar sesgadas y “escondan”, de alguna manera, la complejidad real de muchas otras instancias del problema. Por ejemplo, ordenar un arreglo que ya está ordenado –sin importar su tamaño– es trivial, mientras que resolver el problema de ordenamiento no es tan directo en el caso más general. En una situación más dramática, puede suceder que ciertas instancias de un problema algorítmico no tengan solución.

El estudio de algoritmos es interesante cuando su propósito es resolver una gran cantidad de instancias de un problema algorítmico, si no todas. Es decir, el reto de diseño algorítmico está en definir un algoritmo (e.g., unas instrucciones que un computador pueda ejecutar) que resuelva (todas las instancias de) un problema algorítmico dado.

Ejemplo 0.2.4

Para el problema de ordenamiento presentando en el Ejemplo 0.2.1, las siguientes son soluciones a las instancias en el Ejemplo 0.2.2:

- $A = []$.
- $A = [1, 2, 3]$.
- $A = [1, 2, 7, 8, 9]$.

En los dos primeros casos, el arreglo A estaba originalmente ordenado ascendentemente.

Vale la pena llamar la atención del lector acerca de un detalle que puede ser importante en la Definición 0.2.4. Allí se hace explícito el hecho de que pueden existir varias soluciones diferentes para una misma instancia de un problema algorítmico dado y, por ende, para un problema algorítmico dado. A continuación se presenta un problema algorítmico en el cual su única instancia puede tener más de una solución.

Ejemplo 0.2.5

Considere la siguiente especificación:

Entrada: Un tablero de ajedrez de 8×8 vacío.

Salida: Ocho celdas del tablero dado en las cuales se pueden ubicar reinas sin que se ataquen entre ellas.

Se propone como ejercicio al lector mostrar que la única instancia de este problema algorítmico, conocido como el “problema de las 8 reinas”, tiene más de una solución.

Hay una diferencia en los niveles de abstracción empleados en las especificaciones de los problemas en los ejemplos 0.2.1 y 0.2.5. Para el ordenamiento, la entrada es un arreglo que puede ser considerado como un elemento primitivo en un lenguaje de programación. Sin embargo, la entrada en el problema de las 8 reinas es un tablero de ajedrez, que difícilmente es parte de los elementos primitivos en un lenguaje de programación. Esta diferencia no indica que alguno de los dos estilos de especificación esté mal. Lo que sugiere esta diferencia es, más bien, que en algunas ocasiones es conveniente especificar sin detallar tan finamente la forma de los parámetros porque se hacerlo así se considera algo sin importancia en ese momento o porque hay una razón de fondo para mantenerlo a ese nivel de abstracción alto. En el caso del problema de las 8 reinas, el tablero de entrada puede representarse fácilmente con una matriz de números enteros con dimensiones 8×8 , llena de ceros indicando que el tablero está vacío. Sin embargo, la especificación dada asume que no hay necesidad de entrar en este tipo de detalles para entender la esencia del problema y pensar en una forma de resolverlo.

Especificar problemas algorítmicos requiere experiencia que se construye con la práctica. Puede ser retador adquirir esta disciplina, pero merece la pena: un problema bien especificado, es un problema que está más cerca de ser resuelto. En la Nota 0.2.3 se presentan algunas sugerencias para la especificación de problemas algorítmicos.

Nota 0.2.3: Regla de la mano derecha para especificar problemas

Un problema bien especificado formula cómo determinar si una salida dada es correcta para una entrada dada. Para ello, es recomendable:

- usar notación y términos claros y concisos, evitando argot especializado ajeno a la computación;

- si es necesario incluir argot especializado, este debe ser definido formalmente (y preferiblemente antes de ser usado);
- la especificación *no* debe mencionar
 - ni detalles de la fuente de datos de la cual proviene la entrada (e.g., base de datos, archivo de texto o un microservicio en la nube),
 - ni una estrategia de solución o detalles de su implementación,
- si la especificación *puede* ser simplificada, entonces *debe* ser simplificada.

Toda especificación debe ser precisa, corta y elegante.

El problema especificado en el Ejemplo 0.2.1 está bien especificado de acuerdo con la regla de la mano derecha en la Nota 0.2.3.

Esta sección concluye con ejemplos de buenas y malas especificaciones de problemas algorítmicos.

Ejemplo 0.2.6

A continuación se presentan buenas especificaciones de problemas algorítmicos.

Entrada: un grafo G dirigido.

Salida: un orden topológico de G , si existe; de lo contrario, una lista vacía.

Entrada: un grafo $G = (V, E)$ y un conjunto $C \subseteq V$.

Salida: ¿el subgrafo de G inducido por C es un clique (i.e., un grafo en donde cualquier par de vértices está conectado)?

Entrada: una colección P de coordenadas en el plano Cartesiano de dos dimensiones.

Salida: mínima distancia entre cualquier par de puntos (distintos) en P .

Ejemplo 0.2.7

A continuación se presentan malas especificaciones de problemas algorítmicos; las razones por las cuales son consideradas malas especificaciones, en cada caso, se proponen como ejercicio para el lector.

Entrada: una secuencia de líneas de texto; en la primera un número x y en la segunda un arreglo A .

Salida: ¿está x en A ?

Entrada: un grafo G con peso en los arcos.

Salida: el mejor camino para ir de u a v .

Entrada: una colección de cadenas.

Salida: la supercadena maximal de todas las cadenas de la entrada.

Ejercicios

1. Considere el problema de encontrar un número en un arreglo de números. Especifique los siguientes problemas asociados e identifique cuáles son sus parámetros:
 - a) Determinar si el número está en el arreglo.
 - b) Suponiendo que el número está en el arreglo, determinar un índice en donde está el número en el arreglo.
 - c) Si el número está en el arreglo, determinar un índice en el arreglo en donde está el número; de lo contrario, identificar que no aparece.
 - d) Suponiendo que el número está en el arreglo, determinar el índice más pequeño en donde está el número en el arreglo.
 - e) Suponiendo que el número está en el arreglo, determinar el índice más grande en donde está el número en el arreglo.
2. Investigue y especifique el problema de buscar un valor numérico en un arreglo de números *ordenados* ascendentemente. ¿Qué diferencia fundamental hay entre este problema y los especificados en el Ejercicio 1?
3. Especifique el problema de calcular en $A[0..N)$ el valor de los primeros N números factoriales.
4. Especifique el problema de multiplicar dos matrices de números. ¿Qué restricciones deben cumplir las dimensiones de las matrices?
5. ¿Cuántas instancias tiene el problema algorítmico en el Ejemplo 0.2.1? Justifique su respuesta.
6. Investigue acerca de cada uno de los siguientes problemas y especifíquelos como problemas algorítmicos:
 - a) El problema de primalidad de números.
 - b) El Teorema de Fermat.
 - c) La Conjetura de Goldbach.
 - d) El problema de satisfacción proposicional.
 - e) El problema de la parada.

7. Clasifique los problemas indicados en el Ejercicio 1 de acuerdo a si tienen instancias con solución única o no. Ilustre su respuesta con ejemplos.
 8. Clasifique los problemas indicados en el Ejercicio 6 de acuerdo a si tienen instancias con solución única o no. Ilustre su respuesta con ejemplos.
 9. ¿Cuántas soluciones tiene el problema de las 8 reinas en el Ejemplo 0.2.5? Investigue y formule tres soluciones.
 10. Considere la siguiente problema:
Entrada: Un tablero de Sudoku parcialmente lleno.
Salida: Una solución del tablero de Sudoku dado.
 - a) ¿Cuántas instancias tiene el problema? Detalle los cálculos hechos.
 - b) Especifique una instancia del problema que no tiene solución.
 - c) ¿Es un problema para el cual cada instancia tiene a lo sumo una solución? Justifique su respuesta.
 11. Investigue y especifique tres problemas algorítmicos que tengan una cantidad finita de instancias.
 12. Investigue y especifique tres problemas algorítmicos que tengan una cantidad infinita de instancias.
 13. ¿Existe un problema algorítmico con múltiples instancias, pero para el cual hay una única solución (i.e., independientemente de la entrada, la salida siempre es la misma)? Justifique su respuesta.
 14. Explique brevemente por qué las especificaciones en el Ejemplo 0.2.7 no son consideradas como buenas especificaciones de problemas algorítmicos. En cada caso justifique su respuesta.
-

0.3. El concepto de algoritmo

De acuerdo con Donald Knuth, uno de los científicos más destacados y prolíficos de la informática, los algoritmos son los hilos que permiten relacionar y asociar diferentes disciplinas de las ciencias de la computación. Dada su importancia, definir qué se entiende por algoritmo ha sido una de las primeras tareas abordadas desde las matemáticas y desde las ciencias de la computación. Esta sección presenta una definición de algoritmo, los asocia a problemas algorítmicos e identifica algunas de sus propiedades principales.

Nota 0.3.1

La palabra *algoritmo* es de origen árabe y se asocia al nombre del matemático Mohamed ibn Musa, cuyo apodo era Al-Khwarismi y quien vivió entre los años

770 y 840 A.D. Se cree que, inicialmente, se usaban las palabras *alguarismo* y *guarismo*, y que estas palabras evolucionaron con influencia de la palabra griega *aríthmō* (i.e., número) a algoritmo. Sin embargo, la Real Academia Española indica que esta palabra proviene del latín *algobarismus*, para abreviar en árabe la acción de calcular mediante cifras arábigas. De cualquier forma, Al-Khwarismi –además de desarrollar algoritmos para resolver ecuaciones de primer y segundo grado– explicó cómo usar ábacos en lugar de las manos para calcular a partir de una especificación clara y concisa. Esta es considerada una de las primeras expresiones de cálculo asistido mecánicamente en la humanidad. Así parece que surge la popularidad de la palabra ‘algoritmo’ como término para referirse a una secuencia de reglas para manipular números arábigos. Posteriormente, esta palabra fue generalizada para dar cuenta de cálculos con otros objetos matemáticos, no necesariamente numéricos. El primer caso de un algoritmo escrito para una máquina se encontró en las notas de Ada Lovelace hacia 1840, quien propuso cómo calcular los números de Bernoulli en la Máquina Analítica de Charles Babbage. A pesar de que la Máquina Analítica nunca fue terminada y, en consecuencia, el algoritmo propuesto por Ada Lovelace no pudo ser ejecutado mecánicamente en dicha máquina, a ella se atribuye el título de primer programador de la humanidad.

El propósito intuitivo de un algoritmo es resolver un problema algorítmico a partir de una secuencia de pasos. Antes de que las ciencias de la computación existieran, los matemáticos se preocuparon por definir formalmente la noción de “computar”, la cual está estrechamente ligada a la de algoritmo. Varias nociones fueron propuestas para finalmente llegar a un consenso.

Definición 0.3.1

Un *algoritmo* es un conjunto de instrucciones que pueden ser ejecutadas por una máquina de Turing.

Posiblemente, esta sea la definición más formal de lo que signifique computar, en cuanto a que en este contexto computar se refiere a las operaciones que puede realizar una máquina ideal, dada una secuencia de instrucciones (i.e., un algoritmo).

Nota 0.3.2

Es importante resaltar que la noción de algoritmo relativa a una *máquina de Turing* puede no solo ser la más formal, sino también la más general posible de acuerdo con la tesis de Church-Turing (en inglés, *Church-Turing thesis*).

Este postulado afirma que la formalización de algoritmo en una máquina de Turing es *posiblemente* la noción más poderosa posible. Note que este postulado, como tal, es más una creencia que un hecho demostrado a causa de su formulación coloquial (i.e., no matemática). A pesar de ello, hay consenso en las comunidades académica y científica para aceptar esta afirmación.

Los computadores que se usan hoy en día están inspirados en el modelo de computación asociado a una máquina de Turing y, por ende, los cálculos que hacen están basados en lo que formalmente se entiende por algoritmo. Para el propósito de diseñar y analizar algoritmos, convenientemente se puede abstraer el concepto de “conjunto de instrucciones” de una máquina de Turing para obtener una definición de más alto nivel (pero sin perder de vista su definición formal) de lo que se entiende por algoritmo.

Definición 0.3.2

Un *algoritmo* es una colección finita y ordenada de pasos sin ambigüedad que produce un resultado y termina en una cantidad finita de pasos (y de tiempo).

La descripción de un algoritmo debe ser finita por naturaleza, al igual que sucede con la descripción y la entrada de una máquina de Turing. El orden y la inexistencia de ambigüedad en los pasos que definen un algoritmo se refiere a que debe ser claro qué paso sigue en una “ejecución” de un algoritmo a partir de un estado parcial de su ejecución. Por ejemplo, si un paso es sumar dos números enteros, ambos números enteros deben estar definidos y la operación de adición estar identificada. No se puede confundir el operador de suma con otro operador, ni la acción de sumar con otro paso como almacenar un valor en una base de datos. El objetivo de un algoritmo es calcular algo como resultado; de lo contrario sería inoficioso seguir sus pasos. El resultado de un algoritmo se entiende como la salida de un proceso mecánico que obtiene un valor o modifica una cantidad. Finalmente, un algoritmo debe terminar después de una cantidad finita de tiempo o de pasos. De lo contrario, aquello definido *no* sería un algoritmo (esta restricción puede ser polémica en general, pero es conveniente y suficiente para el propósito de estudio en este texto).

Adicionalmente a las propiedades asociadas a un algoritmo y enunciadas en la Definición 0.3.2, un algoritmo puede tener cero, una o más entradas, dependiendo de lo que se desea calcular, y también producir uno o más resultados, cada uno de ellos asociados unívocamente con la entrada dada. Las operaciones usadas en los pasos de un algoritmo para producir los valores de retorno a partir de la entrada deben

ser suficientemente básicas de tal forma que cada una de ellas se pueda procesar en tiempo finito.

La noción de algoritmo en la Definición 0.3.2 no está relacionada de ninguna manera con problemas algorítmicos, a pesar de que la finalidad de un algoritmo (al menos en este manuscrito) es resolverlos. De acuerdo con la Definición 0.2.4, una solución a un problema algorítmico necesariamente debe resolver todas sus instancias.

Definición 0.3.3

Sean P un problema algorítmico y A un algoritmo. Se dice que A *resuelve* (o *es una solución* de) P si y solo si A calcula una respuesta correcta para cada una de las instancias de P .

El concepto de solución algorítmica planteado en la Definición 0.3.3 indica que un algoritmo puede ser visto como solución a un problema algorítmico. Note que no hay excepción en cuanto a que algunas instancias del problema dado pueden quedar sin respuesta. En este sentido, considerar un algoritmo como solución a un problema algorítmico es una propiedad categórica de los problemas: debe resolver *todas* sus instancias. Note también que esta noción es agnóstica del lenguaje de especificación con que se describe el algoritmo y del lenguaje de programación con que se vaya a implementar. Como se verá a lo largo del texto, garantizar cuándo un algoritmo resuelve un problema algorítmico es una de las principales preocupaciones (si no la más importante) asociada a su diseño.

Ejemplo 0.3.1

Considere el problema de ordenamiento de un arreglo $A[0..N)$ de números enteros, especificado en el Ejemplo 0.2.1. A continuación se presenta un algoritmo especificado en lenguaje natural que “permite” resolver el problema.

1. Si $N \neq 0$, entonces repita, para $n = 0, 1, \dots, N - 1$, la siguiente secuencia de pasos:
 - a) sea i el índice del mínimo valor en $A[n..N)$
 - b) intercambie $A[n]$ y $A[i]$
2. Retorne $A[0..N)$.

Aparentemente, el algoritmo propuesto en el Ejemplo 0.3.1 ordena un arreglo de números. Sin embargo, dado que no hay una certeza matemática de ello, se usa la palabra *permite* en comillas: no es suficiente con creer, es necesario demostrar que en realidad funciona. Este será uno de los temas de énfasis en el texto.

Ejercicios

1. Elabore un pequeño resumen de las principales contribuciones de Donald Knuth a la informática y las matemáticas, identificando al menos cinco de sus principales manuscritos (e.g., libros y artículos).
 2. Investigue sobre la vida de Mohamed ibn Musa y explique por qué (se cree) que lo apodaron Al-Khwarismi. Además, liste al menos cinco de sus principales aportes a la ciencia.
 3. Investigue sobre el algoritmo diseñado por Ada Lovelace para Máquina Analítica de Charles Babbage y explique, a un alto nivel, su diseño.
 4. Explique cuáles son las partes de una máquina de Turing y cómo se ejecutan algoritmos allí. ¿Puede proponer un corto algoritmo para una máquina de Turing? Justifique su respuesta.
 5. Considere el algoritmo propuesto en el Ejemplo 0.3.1 para ordenar ascendentemente un arreglo de números. Identifique al menos una fuente de ambigüedad en su descripción y proponga cómo corregir este defecto.
 6. Considere el problema de ordenamiento de un arreglo $A[0..N)$ de números enteros, especificado en el Ejemplo 0.2.1. A continuación se presenta un algoritmo especificado en lenguaje natural que “permite” resolver el problema.
 - a) Si $N \geq 2$, entonces repita, para $n = 0, 1, \dots, N - 1$, la siguiente secuencia de pasos:
 - 1) sea i el índice del mínimo valor en $A[n..N)$
 - 2) intercambie $A[n]$ y $A[i]$
 - b) Retorne $A[0..N)$.Aparentemente, este algoritmo es una alternativa al presentado en el Ejemplo 0.3.1 en cuanto ordena el arreglo $A[0..N)$. Proponga una forma de demostrar que las dos soluciones presentadas al problema algorítmico de ordenamiento son equivalentes, es decir, las dos solucionan correctamente todas las instancias del problema.
 7. Considere el problema de ordenamiento de un arreglo $A[0..N)$ de números enteros, especificado en el Ejemplo 0.2.1. Suponga que hay un algoritmo que dado $A[0..N)$ ordena su contenido en un arreglo $B[0..N)$, el cual retorna como resultado. ¿Resuelve dicho algoritmo el problema formulado en el Ejemplo 0.2.1? Justifique su respuesta.
-

0.4. Especificación de algoritmos

En este texto se presentan técnicas de solución de problemas que resultan en algoritmos, recurrentes o iterativos. Independientemente de la forma que tome una solución, se debe contar con un lenguaje suficientemente expresivo para escribirla y conciso para entenderla. Para ello existen numerosas opciones, desde lenguaje natural (e.g., el que usamos para comunicarnos verbalmente) hasta lenguajes matemáticos o de programación con una semántica muy precisa y clara. Cada opción viene con beneficios e inconvenientes. Por ejemplo, una ventaja del lenguaje natural es su fácil descripción; el precio que se paga es, como tal, la posibilidad de involuntariamente llegar a una descripción ambigua dada la inherente ambigüedad del lenguaje natural. Una opción que contrarrestaría esa posible ambigüedad sería optar por un lenguaje de programación en lugar del lenguaje natural. Sin embargo, cada lenguaje de programación tiene su propia semántica (e.g., el comando **for** en Python y en Java puede ser interpretado de forma diferente) y esto obliga al lector a aprenderlo. Como estos, hay más ejemplos de beneficios e inconvenientes, animados por un sinnúmero de discusiones. En conclusión, en este texto se reconoce que no hay una solución perfecta, que convenga a cualquier tipo de lector, para el problema de cómo especificar algoritmos.

Por las razones expuestas anteriormente, se usarán varios *lenguajes* de especificación de algoritmos. Dependiendo del contexto, se podrá usar uno u otro. El compromiso del autor con los lectores es el siguiente: en ningún caso habrán ambigüedades ni especificaciones incompletas, sin importar el lenguaje que se use. Eso sí, el espectro de posibilidades se restringe a tres opciones como se anuncia en la Nota 0.4.1.

Nota 0.4.1

En orden de preferencia, en este texto se usaran los siguientes lenguajes para especificar un algoritmo:

- Enumeraciones estructuradas, como en el Ejemplo 0.3.1.
- Pseudo-código/código en el lenguaje de programación Python3, como en el Ejemplo 2.2.1
- Pseudocódigo con notación matemática, similar a lo que se emplea en otros textos de algoritmos, usualmente.

En las enumeraciones estructuradas se usa lenguaje natural (e.g., Castellano) en combinación con notación matemática sencilla, si es necesario. En estos casos, los pasos de un algoritmo se describen con frases de palabras, que algunas veces contienen fórmulas. A pesar de ser una opción muy intuitiva, se ha de propender

por ser preciso con las palabras porque de lo contrario describir un algoritmo podría fácilmente resultar en pasos ambiguos, como usualmente sucede en las recetas de cocina. Usar pseudo-código o código puede ayudar a eliminar fuentes de ambigüedad gracias a la sintaxis técnica de los lenguajes de programación, como se explicó anteriormente. De esta forma, un algoritmo es o está muy cerca de ser un programa de computador. La relativa desventaja con esta opción es que la utilidad del algoritmo está supeditada a la semántica del lenguaje de programación elegido y se puede perder –entre tanto detalle técnico– la idea primordial detrás de la solución que encarna. Por ello, a lo largo de este texto se usará esta opción siempre y cuando la especificación del algoritmo no resulte en demasiado detalle técnico o sea extremadamente dependiente de la semántica de Python. Finalmente, al usar notación matemática se elimina de raíz el problema de la ambigüedad, pero la especificación puede resultar extraña a los ojos poco entrenados o desconocedores de la notación elegida.

Ejercicios

1. Investigue sobre el formalismo conocido como *diagramas de flujo* y úselo para describir el algoritmo en el Ejemplo 0.3.1.
2. Investigue sobre el formalismo conocido como *lenguaje de comandos guardados* (en inglés, *guarded command language*) y úselo para describir el algoritmo en el Ejemplo 0.3.1.
3. Recuerde el problema de buscar un número en un arreglo de números ordenado ascendente (Ejercicio 0.2.2). Para este problema se conoce una solución llamada “búsqueda binaria”. Investigue sobre esta solución y:
 - a) Especifique el algoritmo de búsqueda binaria en lenguaje natural para resolver el problema dado.
 - b) Especifique el algoritmo de búsqueda binaria en diagramas de flujo para resolver el problema dado (ver Ejercicio 1).
 - c) Especifique el algoritmo de búsqueda binaria en el lenguaje de comandos guardados para resolver el problema dado (ver Ejercicio 2).
 - d) Especifique el algoritmo de búsqueda binaria en el lenguaje de programación Python para resolver el problema dado.
 - e) Especifique el algoritmo de búsqueda binaria en el lenguaje de programación Ada para resolver el problema dado.
 - f) Especifique el algoritmo de búsqueda binaria en el lenguaje de programación Ruby para resolver el problema dado.
 - g) Especifique el algoritmo de búsqueda binaria en el lenguaje de programación Julia para resolver el problema dado.

- h) Especifique el algoritmo de búsqueda binaria en el lenguaje de programación Scala para resolver el problema dado.
 - i) Especifique el algoritmo de búsqueda binaria en el lenguaje de programación Maude para resolver el problema dado.
 - j) Especifique el algoritmo de búsqueda binaria en el lenguaje de programación Brainfuck para resolver el problema dado.
4. Considere el problema de calcular la descomposición en factores primos de un número natural. Una aproximación para resolver el problema es la construcción de la Criba de Eratóstenes, para luego factorizar el número dado.
- a) Investigue sobre la Criba de Eratóstenes. ¿Cuáles son las operaciones básicas en su construcción? Explique su respuesta.
 - b) Especifique el problema de construir la Criba de Eratóstenes en un arreglo $A[0..N)$ para los primeros N números naturales.
 - c) Proponga un algoritmo para resolver el problema dado, utilizando el lenguaje de especificación de su preferencia.
 - d) ¿Cómo puede garantizar que la solución propuesta es un algoritmo y función?

Notas del capítulo y referencias

En numerosos textos, los arreglos son considerados como conjuntos de variables elementales e indexadas consecutivamente. Esta concepción coincide con el tratamiento presentado en este capítulo. Autores como E. Dijkstra [Dij76] proponen un tratamiento alejado de la tradición y más cercano a la lógica para los arreglos, introduciendo el concepto de “variable arreglo” y adoptando una notación distinta para su indexación. Un tratamiento más extenso de la relación entre arreglos y funciones es propuesta por A. Kaldewaij [Kal90], incluyendo las nociones de sección e indexación, y notación para operar con ellos.

La especificación de problemas a partir de una relación entre la entrada dada y la salida esperada, y la concepción de un problema algorítmico como una colección de instancias, son usuales en la literatura; ver, e.g., [CLRS22, Bha15]. Autores como J. Kleinberg y É. Tardos [KET06] usan enumeraciones estructuradas para especificar algoritmos, mientras que autores como T. Cormen et al. [CLRS22] J. Erickson [Eri19] usan pseudo-código. Autores como H. Bhasin [Bha15] y S. Skiena [Ski08] usan complementariamente pseudo-código y un lenguaje de programación para especificar algoritmos. D. Gries [Gri81] usa el lenguaje de comandos guardados (en inglés, *guarded command language*) para especificar algoritmos iterativos; este puede ser considerado un lenguaje matemático para la especificación de algoritmos.

Análisis asintótico

Considere por un momento que cuenta con dos algoritmos, digamos A_0 y A_1 , que resuelven un problema algorítmico dado. Al ser soluciones del mismo problema, se está suponiendo que tanto A_0 como A_1 resuelven correctamente todas las instancias del problema. Sin más información y con poca reflexión, cualquiera de las dos soluciones podría adoptarse como “la” solución del problema algorítmico. Sin embargo, en la práctica es importante entender cómo se comportarían A_0 y A_1 para decir objetivamente cuál de las dos soluciones es “mejor” o “más conveniente”. Por ello, entender cuáles son las características que hacen que un algoritmo sea mejor que el otro es clave para diseñar algoritmos.

El *análisis asintótico* es una herramienta matemática que permite comparar algoritmos con base en algunas de sus medidas. En el caso puntual de este manuscrito, el interés es las medidas que permiten identificar cuántas instrucciones, o cuántas unidades de tiempo o memoria –haciendo suposiciones mínimas sobre el lenguaje de programación en el cual se implemente y la arquitectura de la máquina en donde se despliegue dicha implementación– se requieren para que un algoritmo resuelva las instancias del problema para el cual fue diseñado. Al uso de estos recursos (i.e., tiempo y espacio) que hace un algoritmo se le denomina *eficiencia algorítmica*: es mejor usar menos instrucciones, o menos unidades de tiempo o memoria cuando sea posible.

Un reto fundamental al cual se enfrenta el análisis asintótico tiene que ver con el hecho de que un problema algorítmico puede contar con múltiples parámetros e infinitas instancias. Es decir, el análisis asintótico debe ser capaz de clasificar la eficiencia algorítmica de manera tal que refleje lo que en realidad sucede con cada instancia, pero de manera suficientemente general como para no entrar a distinguir casos muy específicos para cada una de ellas.

Al final de este capítulo, el lector estará familiarizado con notación asintótica, complejidad temporal y espacial asociada a un algoritmo, y con nociones fundamentales para poder comparar algoritmos con base en esta notación.

1.1. Medidas para problemas y algoritmos

Como se aprenderá más adelante, la cantidad de problemas algorítmicos es inmensa. Por ello, es prácticamente imposible construir un marco matemático lo suficientemente general como para albergar un análisis asintótico de cualquier algoritmo. Por ejemplo, ¿cómo analizar soluciones a un problema algorítmico que tiene diez parámetros de la misma forma como se analiza para otro par de un problema algorítmico con un solo parámetro? Así como hay heterogeneidad en la cantidad de parámetros, también puede haber heterogeneidad en el tipo de operaciones que los algoritmos realizan. En algunos casos, por ejemplo, estas operaciones pueden tener naturaleza aritmética (e.g., multiplicar dos números enteros o calcular la raíz cuadrada de un número real no negativo), o –en otros casos– permitir manipular y construir cadenas de caracteres. En general, es necesario asumir algunas suposiciones acerca de la forma de los problemas algorítmicos y sobre el tipo de instrucciones que un algoritmo puede utilizar como instrucciones básicas.

1.1.1. El tamaño de la entrada. Para muchos problemas algorítmicos, es fácil identificar una medida razonable del tamaño de la entrada. En la especificación del Ejemplo 0.2.1, en la cual el problema consiste en ordenar ascendentemente un arreglo $A[0..N)$ dado, la cantidad N de elementos del arreglo parece ser una buena elección para identificar el tamaño de la entrada (tampoco hay muchas más opciones). En el caso del tablero de ajedrez en el Ejemplo 0.2.5, el tamaño de la entrada está dado por las dimensiones del tablero; en particular, hay 8×8 filas y columnas o, de manera equivalente, 64 casillas.

Nota 1.1.1

Dado un problema algorítmico P , se denomina *tamaño de la entrada de P* a alguna medida razonable sobre los parámetros de P .

La noción de *tamaño de la entrada* presentada en la Nota 1.1.1 está lejos de ser un concepto matemáticamente preciso, además que no indica las unidades con las cuales se medirá. Algunas veces se usará la cantidad de elementos de un arreglo como unidad de medida y otras veces la cantidad de bits que conforman un número; sin embargo, no hay una sola unidad preferida para medir el tamaño de las entradas de los problemas algorítmicos. Otra “mala” noticia es que, en la práctica, no existe una forma automática (i.e., algorítmica) de asignar el tamaño de la entrada para cualquier problema algorítmico. Por ello, es necesario desarrollar rápidamente la

destreza de especificar claramente los problemas algorítmicos, para luego identificar cuáles son los parámetros clave que permiten definir el tamaño de su entrada.

Ejemplo 1.1.1

Considere el problema de sumar los elementos de una matriz de números:

Entrada: Matriz $A[0..M][0..N)$, con $M, N \geq 0$, de números enteros.

Salida: Suma de los elementos en $A[0..M][0..N)$.

En este caso, el tamaño de la entrada es la cantidad de filas y columnas de la matriz dada, es decir, el tamaño del problema es una función de M y N .

Aún en los problemas sencillos especificados en los ejemplos 0.2.1 y 1.1.1, se está ignorando el tamaño de los números que se están ordenando o sumando. El tamaño de los números puede llegar a ser un factor importante a la hora de entender cómo se comporta un algoritmo: no cuesta lo mismo comparar un par de números de pocos dígitos o comparar otro par con miles de dígitos. Sin embargo, este nivel de detalle resulta innecesario en la mayoría de los casos dado que se opera con datos cuyos tamaños básicos no son exageradamente grandes.

Nota 1.1.2

Para evitar un detalle excesivo al definir el tamaño de la entrada de un problema, y en la medida de las posibilidades, se supondrá que el tamaño de los números que conforman la entrada es despreciable.

El acuerdo en la Nota 1.1.2 será utilizado en la mayoría de los problemas abordados en este manuscrito. Este acuerdo tiene una justificación en la práctica de implementar algoritmos. Por ejemplo, es común emplear procesadores de 64 bits para ejecutar algoritmos implementados en un lenguaje de programación. Esto último quiere decir que las operaciones aritméticas de comparación, suma o multiplicación entre números de 64 bits se pueden relizar directamente con operaciones del procesador, sin incurrir en “costos” adicionales para realizarlas externamente. En general, y aunque no siempre es posible, el acuerdo establecido en la Nota 1.1.2 eliminará dificultades técnicas y facilitará la tarea de analizar asintóticamente un algoritmo.

1.1.2. Asociando medidas a un algoritmo. Es indispensable modelar matemáticamente la cantidad de instrucciones, o de unidades de tiempo o memoria que un algoritmo emplea para resolver las instancias de un problema algorítmico. La intuición es que dicha cantidad de operaciones o de unidades de tiempo/memoria

pueden depender del tamaño de cada instancia. Matemáticamente, esto se puede modelar directamente con el concepto de función.

Nota 1.1.3

La cantidad de operaciones, tiempo o memoria requeridos por un algoritmo A con $k \in \mathbb{N}$ parámetros, se pueden representar por medio de una función

$$T_A : \mathbb{N}^k \rightarrow \mathbb{R}_{\geq 0}.$$

De esta forma, si A tiene un único parámetro, la expresión $T_A(n)$ representaría la cantidad de operaciones, tiempo o memoria empleadas por el algoritmo A al resolver una instancia de tamaño n .

Note que las funciones T_A presentadas en la Nota 1.1.3 tienen como dominio los números naturales y rango los números reales no negativos. Esto obedece a que el tamaño de una instancia se mide, generalmente, en unidades enteras (e.g., cantidad de elementos, cantidad de bits), y a que la cantidad de operaciones, tiempo o memoria nunca es negativa (por cuestiones técnicas, se prefiere que sea un número real y no uno natural).

Abstraer la cantidad de recursos empleados por un algoritmo con una función matemática tiene varios beneficios. Primero, la noción de función es un concepto básico no solo en matemáticas sino en cualquier ciencia. Segundo, hay herramientas disponibles desde las matemáticas para analizar funciones, i.e., para clasificar algoritmos en función de la cantidad de recursos que requieren. Tercero, esta brinda un nivel de abstracción conveniente para comparar algoritmos, dejando un poco de lado la velocidad del procesador o de la memoria de una máquina concreta en la cual se implanten.

Ejercicios

1. Investigue sobre el problema de determinar la ruta más corta de un vértice a otro en un grafo dirigido *sin* pesos en los arcos. Especifique este problema y determine el tamaño de la entrada.
 2. Investigue sobre el problema de determinar la ruta más corta de un vértice a otro en un grafo dirigido *con* pesos en los arcos. Especifique este problema y determine el tamaño de la entrada.
 3. Investigue sobre el problema del morral (en inglés, *knapsack*). Especifique este problema y determine el tamaño de la entrada. ¿Se puede expresar el tamaño de la entrada en función de solo una de sus variables?
-

1.2. Notación asintótica

Esta sección presenta la definición formal de la notación O (en inglés, *big Oh notation*), y algunas notaciones relacionadas. También se incluyen ejemplos de cómo utilizarlas para clasificar y comparar funciones (matemáticas).

Definición 1.2.1

Sea $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. El *orden de f* , denotado $O(f)$, es el conjunto definido de la siguiente manera:

$$O(f) = \{g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0} \mid (\exists n_0 \in \mathbb{N})(\exists c \in \mathbb{R}_{>0})(\forall n \in \mathbb{N}) n \geq n_0 \Rightarrow g(n) \leq cf(n)\}.$$

El orden $O(f)$ de una función f es el conjunto de todas las funciones que en el límite están acotadas (por encima) por un factor constante de f . En Castellano, se dice que una función g está en $O(f)$ (i.e., g es del orden de f) si eventualmente g está acotada por encima por un múltiplo constante de f . En este caso, se escribe $g \in O(f)$. Las variables n_0 y c cuantificadas existencialmente en la Definición 1.2.1 son usadas para denotar “eventualmente” y “factor constante”, respectivamente.

La notación O puede también definirse sobre funciones con otro tipo. Por ejemplo, se pueden considerar funciones con dominio en los reales o de tuplas de números naturales. Para el propósito de este manuscrito, basta con enfocarse en aquellas funciones que tienen dominio en los naturales y codominio en los reales no negativos. La definición de O , tal y como se presenta con dominio en los naturales y rango en los reales no negativos, es suficiente para clasificar y comparar una gran cantidad de funciones asociadas a algoritmos.

Nota 1.2.1

Por conveniencia, en algunas ocasiones se abusa la notación de orden permitiendo funciones más generales. Por ejemplo, de vez en cuando, se indica que $g \in O(f)$ aún si $g(n)$ es negativo o indefinido para algunos valores $n < n_0$. También se permite que f sea negativa o indefinida para alguna cantidad finita de valores en su dominio. En estos casos, es necesario escoger un n_0 lo suficientemente grande para que estos comportamientos sean excluidos para cualquier $n \geq n_0$.

La Figura 1 ilustra cuándo una función g está en $O(f)$. La constante c indica cuál es el múltiplo de f con el que se acota a g y la constante n_0 indica a partir de qué punto cf permanece estable como acotamiento de g .

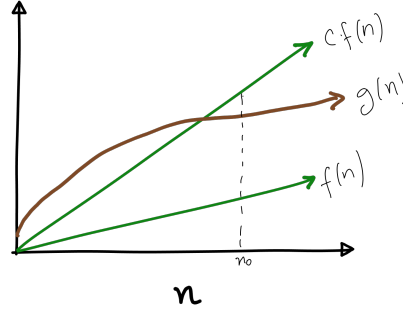


Figura 1. La función g está acotada por un factor constante positivo c de f en el límite (i.e., $g \in O(f)$).

La necesidad de introducir una definición formal de la notación O es para establecer observaciones precisas sobre el comportamiento asintótico de algoritmos. En estos casos, se requieren demostraciones rigurosas que permitan clasificarlos. Para establecer $g \in O(f)$ se debe demostrar la existencia de dos valores $n_0 \in \mathbb{N}$ y $c \in \mathbb{R}_{>0}$ de tal forma que $g(n) \leq cf(n)$ sea cierto para cualquier $n \geq n_0$. Una forma alternativa de ver este tipo de demostraciones es a modo de juego contra un oponente que siempre juega de forma perfecta (i.e., si hay forma de ganar, el oponente escoge una opción que le garantiza ganar).

Nota 1.2.2

El jugador con el primer turno es quien tiene el objetivo de establecer $g \in O(f)$. En el primer turno, dicho jugador escoge valores para n_0 y c . En el segundo turno, el oponente “todopoderoso” escoge un valor $n \geq n_0$. El juego lo gana el primer jugador si $g(n) \leq cf(n)$; de lo contrario, gana el oponente.

La justificación de ver una posible demostración de $g \in O(f)$ a modo de juego, como lo propone la Nota 1.2.2, es la siguiente. Si $g \in O(f)$, entonces existen constantes n_0 y c tal que $g(n) \leq cf(n)$ para $n \geq n_0$, y de esta forma el primer jugador tiene una estrategia ganadora. De lo contrario, sin importar cómo el primer jugador escoja n_0 y c , el oponente encontrará (dado que siempre juega de forma perfecta) un $n \geq n_0$ que falsifique la desigualdad y así ganará el juego. Es importante entender que en ocasiones se dice que las variables n_0 y c son constantes. En realidad, este abuso del lenguaje tiene el propósito de dejar claro que estos valores no dependen de n .

Ejemplo 1.2.1

Se desea demostrar que $5n + 10$ está en $O(n)$. Para ello, es necesario escoger valores $n_0 \in \mathbb{N}$ y $c \in \mathbb{R}_{>0}$ tales que para cualquier $n \geq n_0$ se dé $5n + 10 < cn$.

El primer jugador puede tomar $n_0 = 3$ y $c = 10$, y se observa que $5n + 10 < 10n$ siempre y cuando $2 < n$. Como $n_0 = 3$, la desigualdad $5n + 10 < 10n$ es cierta cuando $n \geq n_0$. Es decir, no importa qué valor escoja el oponente en su turno, aquel que siempre juega perfecto, pues cualquier escogencia hace que la desigualdad sea cierta. En conclusión,

$$(5n + 10) \in O(n).$$

Un ejercicio común en la comparación asintótica de funciones es el de establecer cuándo una función *no* está en el orden de otra. Para este propósito es indispensable entender muy bien la definición de O y, en particular, la alternancia de cuantificadores en ella. Intuitivamente, $g \notin O(f)$ cuando sin importar con qué factor constante se amplifique a f es imposible que dicho factor constante de f domine a g en el límite. Estas demostraciones tienden a ser posiblemente más retadoras que las de pertenencia dado que se debe demostrar que las constantes n_0 y c no existen. Dicho de otra forma, que sin importar cómo se escojan n_0 y c , al menos un valor de $n \geq n_0$ satisface $g(n) > cf(n)$. En estos casos, resulta conveniente usar la técnica de demostración por contradicción: suponer que dichos valores existen para luego llegar a un sinsentido o absurdo lógico.

Ejemplo 1.2.2

Se desea demostrar que n^2 no está en $O(10000n)$. Para ello, es necesario garantizar que es imposible escoger valores $n_0 \in \mathbb{N}$ y $c \in \mathbb{R}_{>0}$ tales que si $n \geq n_0$ se dé $n^2 \leq 10000cn$.

Hacia una contradicción, suponga que dichos valores existen. Sin pérdida de generalidad, también suponga que $n_0 \geq 1$. Entonces, para $n \geq n_0$ se tiene

$$n^2 < 10000cn \quad \text{sii} \quad \frac{n}{10000} < c.$$

Es decir, se debe dar que la constante c acote la función $\frac{n}{10000}$, lo cual es imposible (¿por qué?). En conclusión,

$$n^2 \notin O(10000n).$$

La notación asintótica es suficientemente robusta y de alto nivel como para obviar detalles que se desean ignorar y que dependen –por ejemplo– de la arquitectura de máquina o del lenguaje de programación en el cual se implementa un

algoritmo. También, es precisa como para poder establecer comparaciones entre diferentes alternativas de alto nivel que resuelvan un mismo problema algorítmico, especialmente cuando el tamaño de la entrada es grande y se requiere ingenio para plantear un algoritmo que sea práctico.

La notación O es útil para estimar una cota *superior* del uso de recursos que un algoritmo requiere para resolver un problema. De forma dual, también puede ser interesante estimar una cota *inferior* del uso de estos recursos. A continuación se introduce nueva notación para tal fin.

Definición 1.2.2

Sea $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. El conjunto $\Omega(f)$ se define para cualquier $g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ como:

$$g \in \Omega(f) \quad \text{sii} \quad f \in O(g).$$

El conjunto $\Omega(f)$ (leído, ‘ómega de f ’) es el dual de $O(f)$, pues es el conjunto de todas aquellas funciones que están acotadas por debajo (en el límite) por un múltiplo positivo de f .

Nota 1.2.3

La definición de Ω en la Definición 1.2.2 no es la habitual. En este texto se opta por esta formulación dado que directamente depende de O . De este modo, basta con recordar solamente la definición de O , y una equivalencia sencilla para utilizar y razonar sobre Ω . El Ejercicio 14 de esta sección presenta la definición usual de Ω (para cualquier $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$)

$$\Omega(f) = \{g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0} \mid (\exists n_0 \in \mathbb{N})(\exists c \in \mathbb{R}_{>0})(\forall n \in \mathbb{N}) n \geq n_0 \Rightarrow g(n) \geq cf(n)\}.$$

y plantea la equivalencia entre las dos definiciones.

En adición a O y Ω , existe otra noción asintótica para clasificar funciones que cuentan con una función que, simultáneamente en el límite, sirve como cota superior e inferior con factores que pueden ser distintos en cada caso.

Definición 1.2.3

Sea $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. El conjunto $\Theta(f)$ se define como:

$$\Theta(f) = O(f) \cap \Omega(f).$$

El conjunto $\Theta(f)$ se denomina el *exacto orden de f* y, como se puede deducir de su definición, es más poderoso (en relación a que brinda más información) que $O(f)$ o $\Omega(f)$ individualmente. Al igual que con la definición de Ω , hay una definición distinta que se usa habitualmente (Ejercicio 15 de esta sección). Por los mismos motivos explicados anteriormente para Ω , se prefiere la caracterización de Θ en la 1.2.3.

Ejemplo 1.2.3

Se desea demostrar, para $n \in \mathbb{N}$:

$$5n \in \Theta(n + \log n^2).$$

Considere el siguiente razonamiento:

- $5n \in \Theta(n + \log n^2)$
- sii $5n \in \Theta(n + 2 \log n)$ (aritmética)
- sii $5n \in O(n + 2 \log n) \cap \Omega(n + 2 \log n)$ (def. de Θ)
- sii $5n \in O(n + 2 \log n) \wedge 5n \in \Omega(n + 2 \log n)$ (def. de $\cap$)
- sii $5n \in O(n + 2 \log n) \wedge (n + 2 \log n) \in O(5n)$ ($O(5n) = O(n + 2 \log n)$)
- sii $(n + 2 \log n) \in O(n + 2 \log n) \wedge 5n \in O(5n)$ ($f \in O(f)$)
- sii *true*.

La demostración de la justificación dada en el quinto paso del razonamiento se propone como ejercicio al lector.

Ejercicios

- Justificando su respuesta, clasifique ascendentemente por orden asintótico las siguientes funciones:

$$2^{2^n} \quad \log \sqrt{n} \quad n^2 \log n \quad \log(n!) \quad n! \quad n^{2,5}$$

- Justificando su respuesta, clasifique ascendentemente por orden asintótico las siguientes funciones:

$$\sqrt{2n} \quad \left(\frac{3}{2}\right)^n \quad (\log n)^2 \quad 2^{\sqrt{2 \log n}} \quad n \log n \quad \log(n^2)$$

- En la Nota 1.2.1 se indica que en ocasiones se pueden considerar funciones negativas o indefinidas en algunos puntos. Explique por qué los abusos identificados en la Definición 1.2.1 no afectan el uso formal de la notación O .
- Considere la demostración del Ejemplo 1.2.1. Escoja valores diferentes para n_0 y c de tal forma que se obtenga una nueva demostración.

5. En la demostración por contradicción en el Ejemplo 1.2.1 se indica que sin pérdida de generalidad se puede suponer $n_0 \geq 1$. Explique por qué esta afirmación es correcta y no limita la generalidad de la demostración.
6. Suponga que un algoritmo A realiza $T_A : \mathbb{N} \rightarrow \mathbb{R}_{>0}$ operaciones básicas para ordenar un arreglo (i.e., $T_A(n)$ es la cantidad de operaciones que realiza A para ordenar un arreglo de tamaño n). Como parte de una tarea, Pedrito demostró que $T_A \in O(n^2)$, Juanita que $T_A \in O(n^{3/2})$ y el profesor publicó una solución con una demostración de $T_A \in O(n \log n)$.
 - a) ¿Es posible que las tres demostraciones sean correctas? Justifique su respuesta.
 - b) Si las tres demostraciones fueran correctas, ¿cuál de las tres afirmaciones brinda “más” información sobre el comportamiento de A ? Explique brevemente su respuesta.
7. En el Ejemplo 1.2.2 se usa el argumento de que la función $\frac{n}{100000}$ no puede ser acotada por la constante c . Demuestre formalmente que esta afirmación es cierta.
8. Para $n \in \mathbb{N}$, demuestre que $O(5n) = O(n + 2 \log n)$.
9. Para $n \in \mathbb{N}$, demuestre que $2n^2 \in O(n^3)$ y que $n^3 \notin 2n^2$.
10. Para $n \in \mathbb{N}$, demuestre o refute: $2^{n+1} \in O(2^n)$.
11. Para $n \in \mathbb{N}$, demuestre o refute: $(n+1)! \in O(n!)$, en donde $!$ es la función factorial.
12. Para $n \in \mathbb{N}$ y $f : \mathbb{N} \rightarrow \mathbb{R}_{>0}$, demuestre o refute:
 - a) Si $f \in O(n)$, entonces $f^2 \in O(n^2)$.
 - b) Si $f \in O(n)$, entonces $2^f \in O(2^n)$.
13. Considere la siguiente definición para una función $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$:

$$O^*(f) = \{g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0} \mid (\exists c \in \mathbb{R}_{>0})(\forall n \in \mathbb{N})g(n) \leq cf(n)\}.$$

Demuestre: si $f : \mathbb{N} \rightarrow \mathbb{R}_{>0}$, entonces $O(f) = O^*(f)$.

14. Sea $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. El conjunto $\Omega^*(f)$ se define de la siguiente manera:

$$\Omega^*(f) = \{g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0} \mid (\exists n_0 \in \mathbb{N})(\exists c \in \mathbb{R}_{>0})(\forall n \in \mathbb{N})n \geq n_0 \Rightarrow g(n) \geq cf(n)\}.$$

Demuestre que $\Omega(f) = \Omega^*(f)$.

15. Sea $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. El conjunto $\Theta^*(f)$ se define de la siguiente manera:

$$\Theta^*(f) = \{g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0} \mid (\exists n_0 \in \mathbb{N})(\exists c, d \in \mathbb{R}_{>0})(\forall n \in \mathbb{N}) \\ n \geq n_0 \Rightarrow cf(n) \leq g(n) \leq df(n)\}.$$

Demuestre que $\Theta(f) = \Theta^*(f)$.

1.3. Propiedades de la notación asintótica

El razonamiento para clasificar y comparar funciones con las notaciones O , Ω , Θ se hace más práctico explotando sus propiedades matemáticas. Esta sección presenta algunas de estas propiedades, enfocándose principalmente en O , y las ilustra con ejemplos. Los ejercicios de la sección incluyen más propiedades de la notación asintótica.

Los factores constantes en una expresión que determina la cantidad de recursos requeridos para resolver un problema tienden a ser extremadamente dependientes del sistema en donde se desplieguen los algoritmos. Por ello, si no se desean cálculos que estén comprometidos con un lenguaje de programación específico o un entorno de ejecución particular, tiene todo el sentido ignorar los términos constantes al clasificar y comprar asintóticamente las funciones asociadas a los algoritmos.

Teorema 1.3.1: Regla de constantes

Sea $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Si $c \in \mathbb{R}_{>0}$, entonces $O(f) = O(cf)$.

Demostración

Se supone $c \in \mathbb{R}_{>0}$ y se procede por doble inclusión:

- El objetivo es demostrar $O(f) \subseteq O(cf)$. Si $g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ es tal que $g \in O(f)$, basta con demostrar $g \in O(cf)$. Si $g \in O(f)$, entonces hay $n_0 \in \mathbb{N}$ y $c_0 \in \mathbb{R}_{>0}$ tales que $g(n) \leq c_0 f$ para $n \geq n_0$. Tome $n_1 = n_0$ y $c_1 = \frac{c_0}{c}$, y note que para $n \geq n_1$ se tiene:

$$\begin{aligned} g(n) &\leq c_0 f(n) && \text{(por suposición)} \\ &= c_1 c f(n) && \text{(por definición de } c_1). \end{aligned}$$

Luego, $g \in O(cf)$ con testigos n_1 y c_1 .

- El objetivo es demostrar $O(cf) \subseteq O(f)$. Si $g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ es tal que $g \in O(cf)$, basta con demostrar $g \in O(f)$. Si $g \in O(cf)$, entonces hay $n_0 \in \mathbb{N}$ y $c_0 \in \mathbb{R}_{>0}$ tales que $g(n) \leq c_0 c f$ para $n \geq n_0$. Tome $n_1 = n_0$ y $c_1 = c_0 c$, y note que para $n \geq n_1$ se tiene:

$$\begin{aligned} g(n) &\leq c_0 c f(n) && \text{(por suposición)} \\ &= c_1 f(n) && \text{(por definición de } c_1). \end{aligned}$$

Luego, $g \in O(f)$ con testigos n_1 y c_1 .

Es importante aclarar que los factores constantes pueden ser determinantes en el diseño de un algoritmo. Puede pensarse que la notación O es una herramienta

que permite comparar algoritmos con enfoques fundamentalmente distintos que resuelven un problema dado y determinar, entre ellos, cuál tiene el mejor potencial para escalar a entradas de gran tamaño. Una vez el algoritmo de alto nivel esté claro, es importante esforzarse por reducir el factor constante asociado al algoritmo. De cualquier forma, si el éxito de la solución algorítmica depende de qué tan rápido pueda resolver un problema, claramente este debe funcionar tan rápido como sea posible.

Los términos que tienen orden bajo, en relación con los demás términos que definen una función, tienden a ser irrelevantes a medida que el tamaño del problema a resolver aumenta.

Teorema 1.3.2: Subsunción por suma

Sea $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Si $g \in O(f)$, entonces $O(f + g) = O(f)$, en donde $f + g$ denota la suma punto a punto de f y g (i.e., $(f + g)(n) = f(n) + g(n)$).

Demostración

Se supone $g \in O(f)$ y se procede por doble inclusión.

- El objetivo es demostrar $O(f + g) \subseteq O(f)$. Si $h : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ es tal que $h \in O(f + g)$, basta con demostrar $h \in O(f)$. Primero, por la suposición $g \in O(f)$, hay $n_0 \in \mathbb{N}$ y $c_0 \in \mathbb{R}_{>0}$ tales que $g(n) \leq c_0 f(n)$ para $n \geq n_0$. Segundo, por la suposición $h \in O(f + g)$, hay $n_1 \in \mathbb{N}$ y $c_1 \in \mathbb{R}_{>0}$ tales que $h(n) \leq c_1(f + g)(n)$ para $n \geq n_1$. Observe que:

$$\begin{aligned}
 h(n) &\leq c_1(f + g)(n) && \text{(por suposición si } n \geq n_1) \\
 &= c_1 f(n) + c_1 g(n) && \text{(por aritmética)} \\
 &\leq c_1 f(n) + c_1 c_0 f(n) && \text{(por suposición si } n \geq n_0) \\
 &\leq c_1(c_0 + 1)f(n) + c_1(c_0 + 1)f(n) && \text{(por aritmética)} \\
 &= (c_1(c_0 + 1))2f(n) && \text{(por aritmética).}
 \end{aligned}$$

Luego, con testigos $n_2 = n_0 \uparrow n_1$ y $c_2 = c_1(c_0 + 1)$, se concluye $h \in O(f)$.

- El caso $O(f) \subseteq O(f + g)$ se propone como ejercicio al lector.

El Ejemplo 1.3.1 presenta un caso en el cual se pueden emplear los teoremas 1.3.1 y 1.3.2 para simplificar la clasificación asintótica de funciones. En conjunto, estas dos propiedades son de gran utilidad práctica pues permiten eliminar factores constantes e ignorar términos irrelevantes de una función.

Ejemplo 1.3.1

Una cota superior para la cantidad de operaciones que toma el algoritmo MergeSort es aproximadamente

$$6n \log n + 6n,$$

en donde n es la cantidad de elementos a ordenar. Entonces, la complejidad temporal de este algoritmo puede abstraerse asintóticamente así:

$$\begin{aligned} O(6n \log n + 6n) &= O(6n \log n) && \text{(por el Teorema 1.3.2)} \\ &= O(n \log n) && \text{(por el Teorema 1.3.1).} \end{aligned}$$

Los conjuntos contruídos a partir de la notación O satisfacen una especie de transitividad.

Teorema 1.3.3: Transitividad

Sean $f, g, h : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Si $h \in O(g)$ y $g \in O(f)$, entonces $h \in O(f)$.

Demostración

Se propone como ejercicio al lector.

También es posible usar la notación O para establecer una relación de orden parcial entre funciones y, consecuentemente, comparar indirectamente la eficiencia relativa de diferentes algoritmos para resolver un problema dado.

Teorema 1.3.4

Sean $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$:

1. $O(f) = O(g)$ sii $f \in O(g)$ y $g \in O(f)$.
2. $O(f) \subset O(g)$ sii $f \in O(g)$ y $g \notin O(f)$.

Demostración

Se proponen como ejercicio al lector.

Finalmente, se presentan propiedades de la notación O que son especialmente útiles para analizar la complejidad temporal de algoritmos secuenciales e iterativos.

Teorema 1.3.5: Reglas de suma y producto

Sean $f, f_1, g, g_1 : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$:

1. $O(f + g) = O(f \uparrow g)$, en donde la suma y el máximo se interpretan punto a punto.
2. Si $f_1 \in O(f)$ y $g_1 \in O(g)$, entonces $f_1 g_1 \in O(fg)$, en donde el producto se interpreta punto a punto.

La Propiedad 1 en el Teorema 1.3.5 se conoce como la *regla de la suma*, mientras que la Propiedad 2 como la *regla del producto*.

Ejercicios

1. Sea $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Demuestre que, si $c \in \mathbb{R}_{> 0}$, entonces:
 - a) $\Omega(f) = \Omega(cf)$.
 - b) $\Theta(f) = \Theta(cf)$.
2. Complete la demostración del Teorema 1.3.2 con el caso $O(f) \subseteq O(f + g)$.
3. Sean $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ tales que $g \in O(f)$. Demuestre o refute:
 - a) $\Omega(f + g) = \Omega(f)$.
 - b) $\Theta(f + g) = \Theta(f)$.
4. Demuestre el Teorema 1.3.3.
5. Sean $f, g, h : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ tales que $g \in O(f)$. Demuestre o refute:
 - a) Si $h \in \Omega(g)$ y $g \in \Omega(f)$, entonces $h \in \Omega(f)$.
 - b) Si $h \in \Theta(g)$ y $g \in \Theta(f)$, entonces $h \in \Theta(f)$.
6. Demuestre la Propiedad 1 del Teorema 1.3.4.
7. Demuestre la Propiedad 2 del Teorema 1.3.4.
8. Sean $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Demuestre o refute:
 - a) $\Omega(f) = \Omega(g)$ sii $f \in \Omega(g)$ y $g \in \Omega(f)$.
 - b) $\Omega(f) \subset \Omega(g)$ sii $f \in \Omega(g)$ y $g \notin \Omega(f)$.
 - c) $\Theta(f) = \Theta(g)$ sii $f \in \Theta(g)$ y $g \in \Theta(f)$.
 - d) $\Theta(f) \subset \Theta(g)$ sii $f \in \Theta(g)$ y $g \notin \Theta(f)$.
9. Demuestre la Propiedad 1 del Teorema 1.3.5.
10. Demuestre la Propiedad 2 del Teorema 1.3.5.

11. Formule duales para Ω y Θ de las propiedades de suma y producto en el 1.3.5. Justifique su respuesta con demostraciones.
12. Proponga funciones $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ tales que $f \notin O(g)$ y $g \notin O(f)$. Justifique su respuesta.
13. Sean $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Demuestre que las siguientes propiedades son equivalentes:
 - a) $O(f) = O(g)$.
 - b) $\Theta(f) = \Theta(g)$.
 - c) $f \in O(g)$.
14. Considere la siguiente argumentación:

$$O(n^2) = O(n^3 + (n^2 - n^3)) \quad (\text{por aritmética})$$

$$= O(n^3) \quad (\text{por Teorema 1.3.2 con } n^2 - n^3 \in O(n^3)).$$
 Claramente $O(n^2) \neq O(n^3)$. Entonces, ¿cuál es el error en la argumentación?
15. La noción de límite es una herramienta versátil y útil para comparar funciones. Para $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$, demuestre:
 - a) Si $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \in \mathbb{R}_{>0}$, entonces $O(f) = O(g)$.
 - b) Si $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, entonces $O(f) \subset O(g)$.
 - c) Si $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \in \mathbb{R}_{>0}$, entonces $f \in \Theta(g)$.
 - d) Si $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, entonces $f \in O(g) \wedge f \notin \Theta(g)$.
16. Encuentre funciones $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ tales que $O(f) = O(g)$, pero para las cuales $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ no exista.

1.4. El Teorema Maestro

El Teorema Maestro es un resultado del análisis de algoritmos que permite estimar una cota asintótica para una clase de funciones recurrentes. Esta sección presenta una versión del Teorema Maestro que es útil a lo largo del texto, caracteriza el tipo de función recurrente que puede ser resuelto con su aplicación y lo ilustra con ejemplos. Como tal, la presentación del Teorema Maestro se hace sin acompañamiento de una demostración formal de su validez, pues esta está fuera del alcance del texto.

El Teorema Maestro es de gran utilidad para analizar la eficiencia de algoritmos recurrentes como los que resultan de aplicar técnicas como, e.g., dividir y conquistar. Básicamente, este teorema brinda un método que permite tomar como entrada una función definida recurrentemente (e.g., determinando la cantidad de operaciones o de recursos que un algoritmo necesita para resolver un problema) para calcular una cota superior (asintóticamente definida) para dicha función. Por ejemplo, si se tiene una función $T(-)$ que define recurrentemente la cantidad de operaciones que efectúa

MergeSort para ordenar un arreglo en función de su tamaño, entonces usando el Teorema Maestro se puede obtener una función $f(\cdot)$ que depende de dicho tamaño y tal que $T \in O(f)$. Como se verá en el desarrollo de esta sección, este proceso requiere únicamente de pocos cálculos aritméticos.

Dado que el Teorema Maestro no puede ser usado con cualquier función recurrente, es importante caracterizar el tipo de función que puede ser analizado con el teorema.

Definición 1.4.1: Función simple

Se dice que la función $T : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ es *simple* si satisface las siguientes condiciones para $n \in \mathbb{N}$:

- Caso base: para n suficientemente pequeño,

$$T(n) \in O(1).$$

- Caso recurrente: para los demás valores de n y constantes $a, b, k \in \mathbb{R}_{\geq 0}$,

$$T(n) \leq a \cdot T(n/b) + O(n^k)$$

tales que:

- $a \geq 1$ representa la cantidad de llamados recurrentes hechos por el algoritmo asociado a T y
- $b > 1$ representa el factor por el cual se reduce el espacio en cada llamado recurrente de T .

El caso base en una función simple indica que una vez el tamaño de la entrada es suficientemente pequeño como para no hacer llamados recurrentes, entonces el problema se puede resolver en tiempo/espacio constante (i.e., $O(1)$). El caso recurrente asume que se recurre sobre a instancias similares a la instancia dada, pero de tamaño reducido en una fracción b , y que estos llamados requieren $O(n^k)$ tiempo/espacio para ser consolidados como respuesta de la instancia inicial.

Ejemplo 1.4.1

Sea $T(n)$ la función que determina la cantidad de operaciones realizadas por *MergeSort* para ordenar un arreglo de tamaño n . Como los casos base, que corresponden a ordenar un arreglo vacío o de un solo elemento, toman tiempo constante, se tiene:

$$T(n) \in O(1), \text{ para } n \leq 1.$$

En el algoritmo hay dos llamados recurrentes (i.e., $a = 2$), cada uno sobre una mitad del arreglo dado (i.e., $b = 2$), y se requiere un cálculo lineal en el

tamaño de la entrada (i.e., $k = 1$) para consolidar los resultados obtenidos de los llamados recurrentes. Es decir, para el caso recurrente, se tiene:

$$T(n) \leq 2 \cdot T(n/2) + O(n), \text{ para } n > 1.$$

Luego, $T(\cdot)$ es una función simple.

En la práctica se puede trabajar con aproximaciones de a, b, k , que son los parámetros asociados a una función simple. También es importante entender que cuando una función ha sido definida/declarada como simple, no es necesario fijarse en el valor exacto de su(s) caso(s) base (¿por qué?). Estas funciones, a su vez, cuentan con algunas restricciones. Por ejemplo, cada llamado recurrente debe hacerse sobre subproblemas del mismo tamaño. Aunque este no siempre es el caso, los algoritmos que se diseñan en este texto usando dividir y conquistar permiten ser asociados a funciones simples, similares a la asociada a *MergeSort*.

Se puede precisar ahora el enunciado del Teorema Maestro con base en la noción de función simple.

Teorema 1.4.1: Teorema Maestro

Si $T : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ es una función simple con parámetros $a \geq 1$, $b > 1$ y $k \geq 0$, entonces para $n \in \mathbb{N}$:

$$T(n) \in \begin{cases} O(n^k \log n) & , \text{ si } a = b^k \\ O(n^k) & , \text{ si } a < b^k \\ O(n^{\log_b a}) & , \text{ si } a > b^k. \end{cases}$$

El Teorema Maestro identifica tres casos que provienen de la comparación de dos cantidades: a y b^k . Intuitivamente, a representa la tasa con la que se recurre sobre subproblemas de un problema dado, mientras que b^k representa la tasa con la que se reduce “la complejidad” para resolver estos subproblemas. En este contexto, hay menos operaciones/recursos requeridos por un algoritmo cuando $a < b^k$; lo contrario sucede cuando $a > b^k$ porque hay demasiada proliferación de subproblemas. Cuando $a = b^k$, la intuición indica que el algoritmo lleva a cabo la misma cantidad de trabajo en cada uno de los niveles de la recurrencia: $O(n^k)$ operaciones en cada uno de los $O(\log n)$ niveles de la recurrencia, es decir, $O(n^k \log n)$ operaciones en total. Cuando $a < b^k$, la cantidad de trabajo va decreciendo a medida que se profundiza la recurrencia: es decir, la cantidad de trabajo $O(n^k)$ al tope de la recurrencia domina la cantidad de operaciones totales. Cuando $a > b^k$, la cantidad de operaciones en cada nivel de la recurrencia aumenta con su profundidad: la cantidad total de operaciones es dominada por la cantidad de operaciones en el

último nivel de la recurrencia, es decir, está acotada por $O(n^{\log_b a})$ (sabiendo que $n^{\log_b a} = a^{\log_b n}$, lo cual se propone como ejercicio para el lector).

A continuación se presentan ejemplos del uso del Teorema Maestro.

Ejemplo 1.4.2: Complejidad temporal de MergeSort

La cantidad de operaciones básicas que lleva a cabo el algoritmo *MergeSort* está definida por la función simple T cuyo caso recurrente corresponde a:

$$T(n) = 2 \cdot T(n/2) + O(n).$$

Luego, $a = 2$, $b = 2$ y $k = 1$. Note que:

$$a = 2 = 2^1 = b^k,$$

lo cual corresponde al primer caso del Teorema Maestro. En consecuencia,

$$T(n) \in O(n \log n).$$

Ejemplo 1.4.3: Complejidad temporal de búsqueda binaria

La cantidad de operaciones básicas que lleva a cabo el algoritmo de búsqueda binaria está definida por la función simple T cuyo caso recurrente corresponde a:

$$T(n) = T(n/2) + O(1).$$

Luego, $a = 1$, $b = 2$ y $k = 0$. Note que:

$$a = 1 = 2^0 = b^k,$$

lo cual corresponde al primer caso del Teorema Maestro. En consecuencia,

$$T(n) \in O(\log n).$$

Ejemplo 1.4.4

Considere una función simple cuyo caso recurrente está dado por:

$$T(n) = 2 \cdot T(n/3) + O(n).$$

Luego, $a = 2$, $b = 3$ y $k = 1$. Note que:

$$a = 2 < 3 = 3^1 = b^k,$$

lo cual corresponde al segundo caso del Teorema Maestro. En consecuencia,

$$T(n) \in O(n).$$

Ejemplo 1.4.5: Complejidad temporal de Strassen

El algoritmo de Strassen permite multiplicar dos matrices cuadradas de tamaño n . De una forma ingeniosa, este algoritmo hace 7 llamados recurrentes sobre subproblemas reduciendo en mitades los tamaños de los subproblemas. Las respuestas obtenidas de los llamados recurrentes se consolidan con $O(n^2)$ operaciones:

$$T(n) = 7 \cdot T(n/2) + O(n^2).$$

Luego, $a = 7$, $b = 2$ y $k = 2$. Note que:

$$a = 7 > 4 = 2^2 = b^k$$

lo cual corresponde al tercer caso del Teorema Maestro. En consecuencia, como $\log_2 7 = 2,81$, se tiene

$$T(n) \in O(n^{2,81}).$$

Existen herramientas que permiten analizar el comportamiento de funciones recurrentes más generales que las caracterizadas en esta sección como funciones simples. Este es el caso de los árboles de recurrencia, los cuales permiten analizar visualmente funciones recurrentes muy generales. Por ejemplo, un árbol de recurrencia se puede utilizar para analizar una función que resulta de dividir y conquistar en la cual los llamados recurrentes no se hacen sobre subproblemas del mismo tamaño. Algunos de los ejercicios de esta sección están propuestos para familiarizarse con esta técnica y requieren consultar material no contenido en este manuscrito; referencias apropiadas se indican como de consulta complementaria.

Ejercicios

1. Use el Teorema Maestro para calcular $T(n)$ en cada uno de los siguientes casos:
 - a) $T(n) = T(n/2) + 1$
 - b) $T(n) = 3T(n/3) + 8n$
 - c) $T(n) = 2T(4n/5) + n^3$
2. Use el Teorema Maestro para calcular $T(n)$ en cada uno de los siguientes casos:
 - a) $T(n) = 2T(n/3) + n^3$
 - b) $T(n) = 3T(n/3) + 5n$
 - c) $T(n) = 6T(4n/5) + 4n^2$
3. Justifique por qué, para analizar con ayuda del Teorema Maestro el orden de la cantidad de recursos que emplea un algoritmo, no es necesario contar con los valores exactos de los casos base de la función simple asociada.
4. Sean $a, b \in \mathbb{R}$ tales que $a \geq 1$ y $b > 1$. Demuestre, para $n \in \mathbb{N}$, que $n^{\log_b a} = a^{\log_b n}$.
5. Investigue acerca del Algoritmo de Strassen para la multiplicación de matrices. Explique brevemente en qué consiste, cómo reduce la cantidad de llamados recurrentes de 8 a 7 e ilustre su funcionamiento con un ejemplo.
6. Existe una versión más general del Teorema Maestro en la cual se pueden analizar funciones recurrentes $T : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ cuyos casos bases son $O(1)$ y cuyos casos recurrentes son de la forma

$$T(n) \leq a \cdot T(n/b) + \Theta(n^k),$$
 para $a, b, k \in \mathbb{R}_{\geq 0}$ tales que $a \geq 1$ y $b > 1$. Investigue acerca de esta versión del Teorema Maestro y enúncielo.
7. Los árboles de recurrencia son una herramienta visual para resolver recurrencias más generales que las admitidas para funciones simples. Investigue acerca de los árboles de recurrencia, elabore una breve descripción y presente un ejemplo de cómo usarlos para analizar funciones recurrentes más generales que las simples.
8. Use árboles de recurrencia (ver Ejercicio 7) para resolver cada una de las siguientes recurrencias:
 - a) $T(n) = 2T(n/4) + \sqrt{n}$
 - b) $T(n) = 2T(n/4) + n$
 - c) $T(n) = 2T(n/4) + n^2$
9. Use árboles de recurrencia (ver Ejercicio 7) para resolver cada una de las siguientes recurrencias:
 - a) $T(n) = T(n/2) + T(n/3) + T(n/6) + n$
 - b) $T(n) = T(n/2) + 2T(n/3) + 3T(n/4) + n^2$
 - c) 🌈 $T(n) = 2T(n/2) + O(n \log n)$

$$d) \text{ ☀ } T(n) = 2T(n/2) + O(n/\log n)$$

$$e) \text{ ☀ } T(n) = \sqrt{n}T(\sqrt{n}) + n$$

10. Use árboles de recurrencia (ver Ejercicio 7) para resolver cada una de las siguientes recurrencias:

$$a) T(n) = T(n-1) + n$$

$$b) T(n) = 2T(n-1) + n \log n$$

$$c) T(n) = 2T(n-1) + n^2$$

Notas del capítulo y referencias

Asociar funciones que miden diferentes aspectos de un algoritmo es una práctica común en el análisis de algoritmos. Las notaciones O , Ω y Θ son también muy comunes. Sin embargo, la presentación en este texto de Ω y Θ no es la habitual. Presentaciones convencionales de estas nociones se encuentran en, e.g., [CLRS22, KET06]. Los ejercicios de la Sección 1.2 que establecen las respectivas equivalencias entre las definiciones habituales de Ω y Θ con las presentadas en este texto, están inspirados por la presentación de la notación asintótica de G. Brassard y P. Bratley en [BB88]. Cormen et al. en [CLRS22] hacen una presentación sucinta de los análogos o , ω y θ de las notaciones presentadas en la Sección 1.2.

Las propiedades para el análisis asintótico en la Sección 1.3 son bien conocidas y se pueden consultar en varios de los textos de referencia citados aquí. Algunos ejercicios de esta sección están inspirados o aparacen en [BB88], especialmente los relacionados con límites que fueron transcritos literalmente en algunos casos. Los textos [CLRS22, KET06, BB88, Eri19] son una fuente extensa de más propiedades de O , Ω y Θ , y ejemplos de su uso.

El tratamiento del Teorema Maestro en la Sección 1.4 sigue la propuesta de T. Roughgarden [Rou17]; en este texto se encuentra una demostración del teorema, al igual que en [CLRS22]. Los ejercicios de la sección relacionados con árboles de recurrencia están inspirados en el tratamiento del tema y algunos ejercicios en [Eri19]. Cormen et al. en [CLRS22] incluyen técnicas para resolver recurrencias, complementarias al Teorema Maestro y a los árboles de recurrencia.

El texto [SF13] de R. Sedgwick y P. Flajolet es una fuente de profundización y temas especializados en el área de análisis de algoritmos; se recomienda como lectura obligatoria (y selectiva) al lector decididamente interesado en el tema.

Dividir, conquistar y combinar

Dividir y conquistar (acá llamada, *dividir, conquistar y combinar*) es la técnica por excelencia para resolver problemas algorítmicos. La idea está basada en un principio que la humanidad conoce y usa desde hace muchos años para construir demostraciones: el principio de *inducción matemática*. Como su nombre lo indica, la técnica permite resolver un problema tratando de dividirlo en subcasos y siendo oportunista para resolverlos directamente cuando no es necesario/posible reducirlos más. Técnicamente, una instancia de un problema se divide en subcasos que a su vez son instancias del mismo problema, pero más pequeñas (i.e., dividir), y estas soluciones se combinan para resolver la instancia dada (i.e., combinar); si la subdivisión no es necesaria, entonces la instancia del problema se resuelve directamente (i.e., conquistar).

Este capítulo hace una presentación de esta técnica de diseño algorítmico, la relaciona con el principio de inducción matemática y presenta varios ejemplos de cómo se utiliza en algoritmos sobre arreglos que han sido ampliamente estudiados. Algunos problemas y demostraciones se proponen como ejercicios para el lector.

2.1. Teselación de tableros de ajedrez

Esta sección tiene dos propósitos. Uno es mostrar cómo la técnica de dividir, conquistar y combinar permite resolver el problema de la teselación de tableros de ajedrez. El otro es establecer (de manera informal) la relación que existe entre esta técnica algorítmica y la demostración por inducción matemática.

Nota 2.1.1

A continuación se indican las tres partes que forman parte de una demostración por inducción matemática:

- **Casos base:** son aquellos casos que no dependen de otros casos y que se pueden resolver directamente.
- **Hipótesis inductiva:** son aquellos casos que se pueden suponer resueltos (con base en un ordenamiento de las instancias del problema).
- **Casos inductivos:** son aquellos casos que se apoyan en la hipótesis inductiva para poder llegar al objetivo de la demostración.

Esta es una sobresimplificación (un poco abusiva) del principio de inducción matemática. Sin embargo, es útil para el propósito de explicar la técnica de dividir, conquistar y combinar.

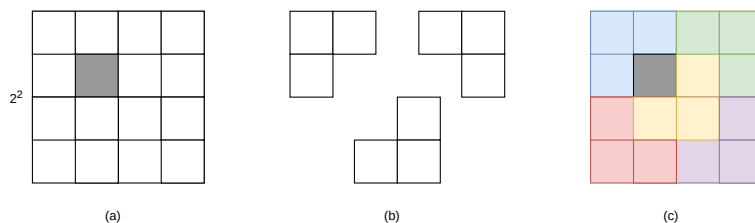
2.1.1. Especificación del problema. A continuación se especifica el problema de teselación de los tableros de ajedrez generalizados.

Problema 2.1.1: Teselación de un tablero generalizado

Entrada: un tablero cuadrado de lado 2^N , $N \geq 1$, con una celda faltante.

Salida: ¿es posible teselar el tablero con fichas en forma de L que comprenden exactamente tres celdas del tablero?

A continuación se muestran: (a) un tablero generalizado de lado 4; (b) fichas con las cuales se teselan los tableros (de las cuales siempre hay suficientes); y (c) una posible teselación de un tablero de lado 4.



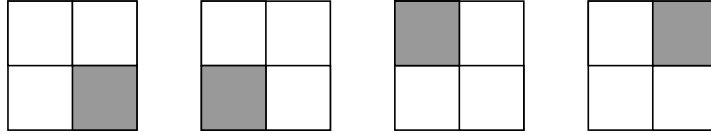
El problema se plantea de manera general, i.e., para cualquier tablero que cumpla con las indicaciones de la entrada. Como N no está acotado, esto significa que la cantidad de tableros es infinita y que no es posible tratar de construir teselaciones enumerando explícitamente todos los tableros. Si se formulase el problema directamente a modo de una fórmula matemática (e.g., $\forall N. N \geq 1 \dots$), sería claro que una opción es proceder por inducción matemática.

2.1.2. Diseño de una solución. Se propone un algoritmo recurrente que está basado en análisis de casos, siguiendo las ideas de una demostración por inducción. Los tableros que pueden ser teselados directamente se tratan como casos base, mientras que los tableros que no son lo suficientemente pequeños como para ser teselados

directamente se teselan al componer las teselaciones de tableros generalizados más pequeños.

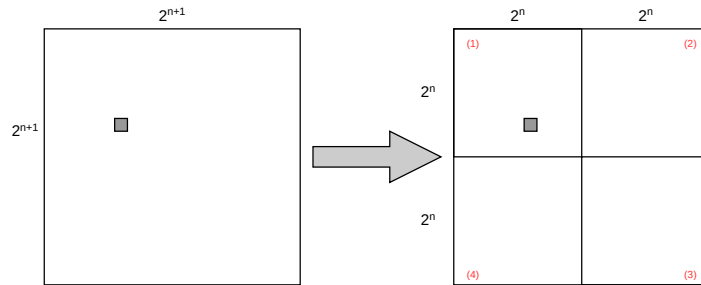
Algoritmo 2.1.1

- Los tableros más pequeños se pueden enumerar; estos corresponden a $N = 1$ y son 4:

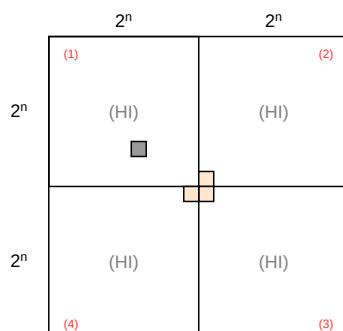


Cada uno de estos tableros se puede teselar con una ficha en forma de L . Esto concluye el *caso base* o, más bien, los *casos base*.

- Los demás tableros tienen lado 2^N , con $N \geq 2$. Se supondrá, a modo de *hipótesis inductiva*, que todo tablero de lado 2^n , con $n \geq 1$, se puede teselar (tal cual como se procede por inducción matemática sobre N). Tenga en cuenta que cada uno de estos tableros tiene exactamente un hueco. La idea es entonces encontrar una forma de teselar un tablero de lado 2^{n+1} que tiene exactamente un hueco. Note también que si un tablero de lado 2^{n+1} se divide en 4 partes iguales, resultan 4 tableros de lado 2^n (enumerados de 1 a 4 en la gráfica a continuación) y se acerca a algo parecido a la hipótesis inductiva. Estos dos hechos se representan gráficamente a continuación:



Como el subtablero 1 tiene lado 2^n y un hueco, se puede teselar por la hipótesis inductiva. Sin embargo, los otros 3 subtableros no tienen hueco y por ello no se puede usar la hipótesis inductiva directamente. Pero, el hecho de que no tengan hueco no quiere decir que no se pueda suponer *convenientemente* que tienen un hueco: uno que resulta de ubicar una de las fichas en forma de L en sus tres esquinas concéntricas:



Ahora, los subtableros 2-4 se pueden tratar como si tuvieran un hueco en una de sus esquinas y consecuentemente cumplen con la condición de la hipótesis inductiva (HI). Es decir, cada uno de ellos se puede teselar al tener lado 2^n y exactamente un hueco. Por consiguiente, todo el tablero de lado 2^{n+1} se puede teselar.

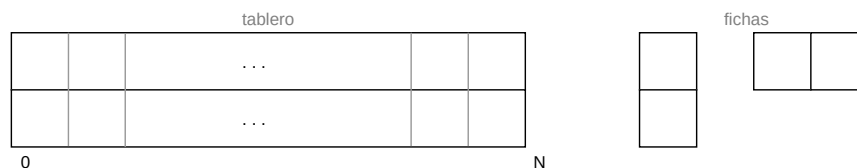
La solución dada en el problema anterior muestra cómo un tablero de ajedrez generalizado y con un hueco puede ser teselado. En realidad, brinda más que eso, pues demuestra constructivamente la posibilidad de encontrar una teselación con ayuda de un algoritmo. En los casos base, el problema se resuelve directamente teselando con una ficha (i.e., conquistar). Mientras que en los casos inductivos: (i) se “recurre” sobre el subtablero en donde está el hueco dado que es una subinstancia del problema inicial (i.e., dividir); y (ii) en los otros tres subtableros se marcan las esquinas concéntricas como si tuvieran un hueco, se recurre en cada subtablero (i.e., dividir) y finalmente se ubica una ficha en forma de L sobre las tres esquinas concéntricas de los subtableros teselados para completar la teselación (i.e., combinar). Este algoritmo puede ser usado en cualquier tablero por el estilo: está garantizada su correctitud por una demostración por inducción matemática.

La siguiente tabla hace explícita la relación que hay entre el principio de inducción matemática, la técnica dividir, conquistar y combinar, y un algoritmo recurrente.

Inducción mat.	DCC	Alg. recurrente
Caso base	Conquistar	Solución directa
Hipótesis inductiva	Dividir	Recurrencia
Caso inductivo	Combinar	Consolidación de soluciones a subproblemas

Ejercicios

1. Con base en la demostración que prueba la existencia de una teselación de un tablero de ajedrez generalizado:
 - a) Diseñe un algoritmo en el lenguaje de programación Python3 que tesele un tablero dado.
 - b) Investigue sobre el lenguaje de programación `dita`. Intente traducir el algoritmo diseñado en Python para el numeral (a) a `dita`.
2. Un tablero de ajedrez *doblemente* generalizado es un tablero cuadrado de lado 2^N , $N \geq 1$, al cual le hacen falta dos celdas de colores opuestos (asumiendo que está coloreado como un tablero de ajedrez convencional; por ejemplo, una celda blanca y una negra).
 - a) ¿Es posible teselar dicho tablero con (suficientes) fichas de dominó, en donde cada ficha cubre exactamente dos celdas adyacentes del tablero? Si su respuesta es negativa, suministre un contra-ejemplo. De lo contrario, construya una demostración siguiendo la estructura propuesta en esta sección.
 - b) 🌈 Considere la misma pregunta del numeral anterior para tableros cuadrados de lado $N \geq 1$ (a los cuales le faltan dos celdas de colores opuestos). Tenga en cuenta que esta es una generalización del problema inicial. ¿Es posible teselar cualquiera de estos tableros?
3. Considere un tablero de ajedrez de 8×8 celdas que ha sido “mutilado” removiendo exactamente dos celdas de esquinas opuestas.
 - a) Especifique el problema de determinar si dichos tableros se pueden teselar con (suficientes) fichas de dominó, en donde cada ficha de dominó cubre exactamente dos celdas adyacentes.
 - b) Resuelva el problema especificado en el numeral anterior.
4. Considere tableros de $2 \times N$ celdas y fichas de dominó que cubren exactamente dos celdas adyacentes.



Proponga una función recurrente que determine la *cantidad* de formas de teselar el tablero con (suficientes) fichas de dominó. Justifique su respuesta con una demostración.

5. De acuerdo con Wikipedia,

Las *Torres de Hanói* es un rompecabezas o juego matemático inventado en 1883 por el matemático francés Édouard Lucas. Este juego de mesa individual consiste en un número de discos perforados de radio creciente que se apilan intertándose en uno de los tres postes fijados a un tablero (...) Para realizar este objetivo, es necesario seguir tres simples reglas:

- Solo se puede mover un disco cada vez y para mover otro los demás tienen que estar en postes.
- Un disco de mayor tamaño no puede estar sobre uno más pequeño que él mismo.
- Solo se puede desplazar el disco que se encuentre arriba en cada poste.

En este ejercicio tiene dos objetivos: demostrar que el juego tiene solución y que hay un algoritmo que lo resuelve con una cantidad de movimientos dado.

- a) Especifique el problema de las Torres de Hanói para $N \geq 1$ discos perforados.
 - b) Demuestre que el problema especificado es soluble para cualquier N .
 - c) Proponga un algoritmo que lleve a cabo exactamente $2^N - 1$ movimientos de discos para resolver el problema. Demuestre que su algoritmo resuelve el problema dado.
6. Sean k y l números naturales. Se dice que k es *divisible* por l si y solo si existe un $p \in \mathbb{N}$ tal que $k = p \cdot l$ (en este caso, p se llama el *testigo* de la divisibilidad de k entre l). Por ejemplo, 15 es divisible por 3 con testigo 5 porque $15 = 5 \cdot 3$. Use el principio de inducción matemática para demostrar que $11^n - 4^n$ es divisible por 7 para todo $n \in \mathbb{N}$.
7. Demuestre que $x^n - 1$ es divisible por $x - 1$ para todos $n, x \in \mathbb{N}$.
8. La función $F : \mathbb{N} \rightarrow \mathbb{N}$ de Fibonacci se define inductivamente de la siguiente manera:

$$F(0) = 0 \quad F(1) = 1 \quad F(n) = F(n-2) + F(n-1), \text{ para } n \geq 2.$$

Demuestre que la función F de Fibonacci satisface las siguientes igualdades:

- a) $F(1) + F(3) + \cdots + F(2(n-1) + 1) = F(2n)$, para $n \geq 1$.
 - b) $F(0) + F(2) + \cdots + F(2n) = F(2n+1) - 1$, para $n \geq 0$.
9. Demuestre que la función F de Fibonacci satisface, para $n \in \mathbb{N}$, la siguiente igualdad:

$$F(0)^2 + F(1)^2 + \cdots + F(n)^2 = F(n)F(n+1).$$

10. Demuestre que la función F de Fibonacci satisface, para $n \in \mathbb{N}$, la siguiente igualdad:

$$F(n)^2 - F(n+1)F(n-1) = (-1)^{n+1}.$$

11. Demuestre que la función F de Fibonacci, para $n \geq 1$, satisface

$$F(n) \geq \left(\frac{3}{2}\right)^{n-2}.$$

12. El producto entre dos matrices de dimensión 2×2 se define como:

$$\begin{pmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{pmatrix} \times \begin{pmatrix} b_{00} & b_{01} \\ b_{10} & b_{11} \end{pmatrix} = \begin{pmatrix} a_{00}b_{00} + a_{01}b_{10} & a_{00}b_{01} + a_{01}b_{11} \\ a_{10}b_{00} + a_{11}b_{10} & a_{10}b_{01} + a_{11}b_{11} \end{pmatrix}.$$

Dada una matriz A de 2×2 , se define A^n para $n \geq 1$ de la siguiente manera:

$$A^1 = A$$

$$A^{n+1} = A \times A^n, \quad n \geq 2.$$

Usando el principio de inducción matemática demuestre, para $n \geq 1$, la siguiente igualdad relacionada con la función F de Fibonacci:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F(n+1) & F(n) \\ F(n) & F(n-1) \end{pmatrix}.$$

2.2. Ordenamiento de arreglos

Ordenar un arreglo de, por ejemplo, números es un problema fundamental en computación.

Problema 2.2.1: Ordenamiento de un arreglo de números

Entrada: Un arreglo $A[0..N)$, con $N \geq 0$, de números.

Salida: $A[0..N)$ ordenado ascendentemente.

A pesar de la simplicidad de su planteamiento y de contar con soluciones muy intuitivas, es un problema que ha sido estudiado extensivamente. La principal razón, pareciera, es la dificultad para resolverlo eficientemente por medio de un algoritmo cuando no se cuenta con información adicional sobre el arreglo dado.

El objetivo ahora es derivar un algoritmo que ordene un arreglo de números a partir de la técnica dividir, conquistar y combinar. Para ello, es deseable identificar rápidamente los subcasos que generalmente surgen al tratar de dividir el problema en suproblemas similares. En este sentido, es importante fijarse en qué casos se pueden resolver directamente sin necesidad de un ordenamiento explícito y los demás casos que parece no se puede resolver directamente. Los primeros casos corresponderán, muy seguramente, a los casos base de una demostración por inducción y al *conquistar* de la técnica algorítmica. Los segundos, serán los casos inductivos

(i.e., *combinar*), que a su vez nos permitirán identificar cuál debería ser la hipótesis inductiva (i.e., *dividir*). Por supuesto, dependiendo de cómo se haga este análisis, diferentes algoritmos resultarán.

Algoritmo 2.2.1

1. Si $n = N$, entonces no se hace nada.
2. Si $n \neq N$, entonces hay al menos un elemento en $A[n..N]$:
 - a) identificar el índice $n \leq m < N$ con el mínimo valor en $A[n..N]$,
 - b) intercambiar $A[n]$ y $A[m]$, y
 - c) recurrir sobre $A[n + 1..N]$.

Note que el caso $n = N$, identifica la situación en la cual $A[n..N]$ es vacío (y está ordenado). En el caso $n \neq N$ es posible que el arreglo no esté ordenado, por lo cual se ubica en $A[n]$ el elemento más pequeño en $A[n..N]$; de esta forma y se recurre ordenando $A[n + 1..N]$. La intuición es que en este último caso, dado que $A[n]$ es el mínimo valor en $A[n..N]$ y $A[n + 1..N]$ está ordenado ascendentemente, necesariamente $A[n..N]$ queda ordenado ascendentemente.

Implementación 2.2.1: Ordenamiento basado en el Algoritmo 2.2.1

```

1 def sort(A, n, N):
2     if n==N: pass
3     else:
4         m = n
5         for i in range(n+1, N):
6             if A[i] < A[m]: m = i
7         A[n],A[m] = A[m],A[n]
8         sort(A, n+1, N)
```

Ejemplo 2.2.1

La función `sort` se usa a continuación con algunos ejemplos:

```

1 A = [8, -10, -2, 3, -50, 2, 10, 8, 4 ]
2 sort(A, 0, len(A))
3 print(A)
4
5 A = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9 ]
6 sort(A, 0, len(A))
```

```
7 print(A)
8
9 A = [ ]
10 sort(A, 0, len(A))
11 print(A)
12
13 A = [ 5 ]
14 sort(A, 0, len(A))
15 print(A)
16
```

El resultado de esta ejecución es el siguiente:

```
1 [-50, -10, -2, 2, 3, 4, 8, 8, 10]
2 [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
3 []
4 [5]
```

Se han mostrado ejemplos que sugieren indicar que funciona bien para algunas instancias del problema, mas no una demostración de que el algoritmo funciona correctamente (i.e., resuelve adecuadamente todas las instancias del problema). Para ello es necesario demostrar que en realidad el código hace lo que debe hacer, es decir, ordenar un arreglo ascendentemente. Dicho de otra forma: se desea demostrar que `sort` ordena ascendentemente cualquier arreglo de números.

Teorema 2.2.1: Correctitud de sort

Para $N = \text{len}(A)$ y $0 \leq n \leq N$, el llamado `sort(A, n, N)` ordena ascendentemente el arreglo $A[n..N)$. En particular, `sort(A, 0, N)` ordena ascendentemente a $A[0..N)$.

Demostración

Se procede por inducción sobre n :

- Caso base ($n = N$): en este caso, el algoritmo no hace ninguna operación. Note que $A[n..N)$ es el arreglo vacío, el cual por definición está ordenado (pues no tiene un par de elementos en desorden).
- Caso inductivo ($n \neq N$): en este caso, n es un índice dentro del arreglo y $A[n]$ representa el valor en esa posición. El ciclo **for** y la asignación posterior al ciclo ubican en $A[n]$ el mínimo valor en $A[n..N)$. Por la hipótesis

inductiva, $A[n+1..N]$ está ordenado ascendentemente (este es el llamado recurrente de `sort`). Como ningún elemento en $A[n+1..N]$ es menor que $A[n]$ y $A[n+1..N]$ está ordenado ascendentemente, necesariamente $A[n..N]$ está ordenado ascendentemente.

Finalmente, como el resultado es cierto para cualquier n que satisface $0 \leq n \leq N$, en particular vale para $n = 0$ y de esa forma se tiene que `sort(A, 0, N)` ordena ascendentemente a $A[0..N]$.

La complejidad temporal del algoritmo `sort` se establece asintóticamente a continuación.

Teorema 2.2.2: Complejidad temporal de `sort`

Para $N = \text{len}(A)$ y $0 \leq n \leq N$, el llamado `sort(A, n, N)` toma tiempo $O((N-n)^2)$. En particular, el llamado `sort(A, 0, N)` toma tiempo $O(N^2)$.

Demostración

Sin pérdida de generalidad, se supondrá que las operaciones aritméticas, de asignación y de control toman exactamente una unidad de tiempo. Además, se define $k = N - n$ para simplificar la manipulación aritmética en los cálculos. Note que en cada llamado recurrente se realizan $N - n = k$ iteraciones del ciclo `for` y cada una de ellas toma tiempo constante. Entonces, la cantidad de operaciones está dada por:

$$\begin{aligned} & (N - n) + (N - (n + 1)) + \cdots + (N - (N - 1)) + (N - N) \\ &= k + (k - 1) + \cdots + 1 + 0 \\ &= \frac{k(k + 1)}{2} \\ &= \frac{(N - n)(N - n + 1)}{2} \in O((N - n)^2). \end{aligned}$$

En particular, para $n = 0$, se tiene que la cantidad de operaciones está dada por $O(N^2)$.

La complejidad espacial se debe calcular con cuidado, en especial bajo la ausencia de información acerca de detalles de implementación de la recurrencia. A continuación, se presenta un estimado que supone que la creación del ambiente de ejecución de cada uno de los llamados recurrentes es de orden constante.

Teorema 2.2.3: Complejidad espacial de sort

Para $N = \text{len}(A)$ y $0 \leq n \leq N$, el llamado `sort(A, n, N)` usa espacio $O(N - n)$. En particular, el llamado `sort(A, 0, N)` usa espacio $O(N)$.

Demostración

En cada llamado recurrente de `sort` se crean a lo sumo dos variables, llamadas `m` e `i`. Como hay a lo sumo $N - n$ llamados recurrentes en donde se crean estas dos variables (únicamente se evita su creación en el caso base), la complejidad espacial de `sort(A, n, N)` es $O(N - n)$. Luego, la complejidad espacial de `sort(A, 0, N)` es $O(N)$.

Es importante aclarar que en el Teorema 2.2.3 se está suponiendo que las variables que se crean en cada llamado recurrente son independientes entre los llamados. Sin embargo, si se pudieran reutilizar de tal forma que fuera una cantidad constante para todos los llamados recurrentes, la complejidad espacial del algoritmo sería de orden constante.

Ejercicios

1. El algoritmo de ordenamiento diseñado e implementado en esta sección es muy conocido y tiene un nombre. ¿Cuál es?
2. Demuestre la siguiente versión, más general, del teorema de corrección de la función `sort`

Teorema. Para $N = \text{len}(A)$ y $0 \leq n \leq m \leq N$, el llamado `sort(A, n, m)` ordena ascendentemente el arreglo $A[n..m)$.

3. Se propone el siguiente algoritmo para ordenar un arreglo $A[0..N)$ de números: mientras que $A[0..N)$ no esté ordenado, permutarlo.
 - a) ¿Puede estimar la complejidad temporal de este algoritmo?
 - b) ¿Puede demostrar que funciona?

Si es necesario contar con algunos supuestos, indique cuáles son. Justifique sus respuestas.

4. Considere la siguiente variación del Algoritmo 2.2.1:
 - Si $n = N$, entonces no se hace nada.
 - Si $n = N - 1$, entonces no se hace nada.
 - Si $n \neq N$, entonces hay al menos un elemento en $A[n..N)$:
 - identificar el índice $n \leq m < N$ con el mínimo valor en $A[n..N)$,

- intercambiar $A[n]$ y $A[m]$, y
- recurrir sobre $A[n + 1..N]$.

Con base en este algoritmo:

- a) Modifique la función `sort` en la Implementación 2.2.1 para que considere el segundo caso base planteado anteriormente.
 - b) Formule el teorema de corrección y demuestre que es correcto para esta nueva versión del algoritmo.
 - c) ¿Cuál es la complejidad temporal y espacial del algoritmo? Justifique su respuesta.
5. En el caso recurrente del Algoritmo 2.2.1, para ordenar el arreglo $A[n..N]$ se usa la estrategia de identificar el índice $n \leq m < N$ con el mínimo valor e intercambiarlo con $A[n]$ antes de recurrir sobre $A[n + 1..N]$.
- Otra estrategia para resolver el Problema 2.2.1 resulta de identificar valores *máximos* y no mínimos.
- a) Diseñe un algoritmo recurrente que, usando dividir, conquistar y combinar, aplique esta estrategia.
 - b) Implemente su diseño en una función del lenguaje de programación Python.
 - c) Enuncie y demuestre el teorema de corrección de su algoritmo.
 - d) ¿Cuál es la complejidad temporal del algoritmo?
6. Considere el problema de ordenar *descendentemente* un arreglo de números.
- a) Especifique el problema dado.
 - b) Usando dividir, conquistar y combinar, diseñe un algoritmo recurrente que resuelva el problema especificado anteriormente.
 - c) Demuestre que el algoritmo es correcto con respecto a la especificación dada.
 - d) Calcule la complejidad temporal del algoritmo diseñado.
7. Considere la siguiente colección de problemas algorítmicos sobre un arreglo de números:
- a) Calcular la suma de los elementos del arreglo.
 - b) Determinar el mínimo valor en el arreglo.
 - c) Determinar el máximo valor en el arreglo.
 - d) Calcular la cantidad de apariciones de un número dado en el arreglo.

Para cada uno de estos problemas:

- Especifique el problema dado.
- Usando dividir, conquistar y combinar, diseñe un algoritmo recurrente que resuelva el problema especificado.
- Demuestre que el algoritmo es correcto con respecto a la especificación.
- Calcule la complejidad temporal del algoritmo diseñado.
- Implemente el algoritmo en el lenguaje de programación Python.

2.3. Una versión iterativa del ordenamiento

En la Sección 2.2 se diseñó un algoritmo recurrente que ordena ascendente-mente un arreglo de números. Su diseño se basó en la técnica dividir, conquistar y combinar, con la cual se obtuvo la implementación de la función `sort` en el lenguaje de programación Python. También es posible obtener una versión iterativa del Algoritmo 2.2.1 de ordenamiento. Para ello es necesario usar la noción de *invariante* como guía para diseñar un ciclo iterativo. Esta sección presenta la noción de invariante asociada a un ciclo iterativo y la ilustra con el ejemplo de una versión iterativa del algoritmo de ordenamiento ascendente de un arreglo de números.

Nota 2.3.1

Un *invariante* (de ciclo) es una fórmula lógica asociada a un ciclo iterativo que cumple las siguientes dos condiciones:

- ser cierta antes de que inicien las iteraciones del ciclo
- ser cierta después de cualquier iteración del ciclo.

El diseño de algoritmos con invariantes requiere de disciplina, práctica y un entendimiento claro de la estrategia de solución que materializa el ciclo al cual está asociado. Cuando los invariantes están bien planteados y son suficientemente completos, es relativamente fácil obtener un ciclo iterativo correcto. La estrategia en estas notas es diseñar ciclos iterativos a partir de invariantes.

Se utilizarán los siguientes invariantes para representar el avance del algoritmo hacia el ordenamiento del arreglo dado. Para evitar confusiones porque el ordenamiento se hará en el mismo arreglo (i.e., es un ordenamiento in-situ) se identificará con $\bar{A}$ al estado inicial del arreglo, antes de que el algoritmo a ser diseñado lo manipule para lograr su ordenamiento. Es decir, el estado inicial del arreglo A se llama $\bar{A}$.

P_0 : $A[0..N)$ es una permutación de $\bar{A}[0..N)$.

P_1 : $A[0..n)$ tiene los n elementos de $\bar{A}[0..N)$ más pequeños y ordenados ascendente-mente.

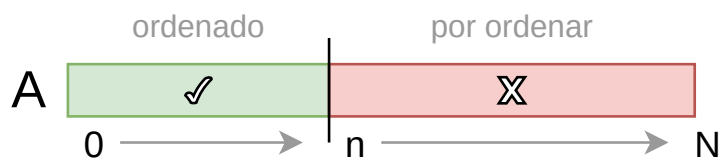
P_2 : $0 \leq n \leq N$.

Hay tres invariantes, llamados P_0, P_1, P_2 . El invariante P_0 indica que los elementos de A son los mismos que los de $\bar{A}$, posiblemente en diferente orden. El invariante P_1 indica no solo que $A[0..N)$ está parcialmente ordenado en las posiciones $0..n-1$, sino que los elementos allí corresponden a los más pequeños en $\bar{A}$. El invariante P_2 indica que el valor de n varía entre 0 y N , inclusive.

Antes de continuar con el diseño de los casos del algoritmo, es clave pensar en cuál sería la situación ideal que –con base en los invariantes dados– garantizará que el arreglo A está ordenado. En este sentido, el invariante P_1 es clave, pues cuando n sea N (lo cual es permitido por el invariante P_2), se tendría la siguiente situación:

$A[0..N)$ tiene los n elementos de $\bar{A}[0..N)$ más pequeños y ordenados ascendentemente.

Es decir, $A[0..N)$ tendría sus elementos ordenados ascendentemente; no faltaría ni sobraría elemento alguno gracias al invariante P_0 . De esta forma, se sugiere que la estrategia es ir avanzando n , que puede iniciar desde 0, hasta que su valor sea N , mientras se mantienen los invariantes formulados. Un representación gráfica de los invariantes P_1 y P_2 , y de esta idea de solución algorítmica, se muestra a continuación:



Con base en los invariantes y en la idea gráfica de diseño, se propone el Algoritmo 2.3.1 para ordenar iterativa y ascendentemente un arreglo de números.

Algoritmo 2.3.1

Para $n = 0, 1, \dots, N - 1$:

1. identificar el índice $n \leq m < N$ con el mínimo valor en $A[n..N)$ e
2. intercambiar $A[n]$ y $A[m]$.

La propuesta del Algoritmo 2.3.1 es similar a la del Algoritmo 2.2.1 recurrente, diseñado en la Sección 2.2. La gran diferencia es que después del paso (2) no se recurre sino que se continúa con las iteraciones del ciclo. Este diseño resulta en la siguiente implementación.

Implementación 2.3.1: Ordenamiento basado en el Algoritmo 2.3.1

```

1 def sortiter(A):
2     N = len(A)
3     #  $P_0 \wedge P_1 \wedge P_2$ 
4     for n in range(0, N):
5         m = n

```

```

6   for i in range(n+1, N):
7       if A[i] < A[m]: m = i
8       A[n],A[m] = A[m],A[n]

```

Note que el código que selecciona el valor mínimo en $A[n..N)$ es el mismo de la versión recurrente del algoritmo.

La demostración de que un algoritmo iterativo de correcto está directamente ligada a la noción de invariante.

Nota 2.3.2

Para demostrar que un ciclo iterativo C con condición de terminación B es correcto con respecto a un invariante P es necesario demostrar que:

1. [Iniciación] P es cierto antes de la primera iteración de C y
2. [Estabilidad] Si P es cierto antes de una iteración de C , entonces P es cierto después de una iteración de C (i.e., que $P \wedge B$ implica P después de la terminación de una iteración de C).

Adicionalmente, una vez terminen las iteraciones de C , el invariante P debe suministrar información importante acerca del objetivo de C (i.e., $P \wedge \neg B$ deben proveer información sobre la finalidad de C).

A manera de ejemplo de cómo establecer la correctitud del ciclo principal de Algoritmo 2.3.1 con respecto sus invariantes, se presentan demostraciones de iniciación y estabilidad. Además, se muestra cómo los invariantes ayudan a determinar qué logra la ejecución del algoritmo. Dado que hay un ciclo anidado dentro del ciclo principal, se supondrá que dicho ciclo cumple su objetivo: calcular en la variable m el índice del valor mínimo en $A[n..N)$.

Teorema 2.3.1

Los invariantes P_0, P_1, P_2 son ciertos antes de la ejecución del ciclo (principal) de la Implementación 2.3.1.

Demostración

Antes de la primera iteración del ciclo, las variables n y N tienen los valores 0 y $\text{len}(A)$, respectivamente. Note que:

- Dado que el arreglo A no ha sido modificado, claramente P_0 es cierto.

- Como $n = 0$, el arreglo $A[0..n)$ es vacío y por tanto no tiene elementos. Esto concuerda con el invariante P_1 .
- Con $n = 0$ y $N = \text{len}(A)$ se establecen trivialmente las desigualdades $0 \leq n \leq N$.

Teorema 2.3.2

Si los invariantes P_0, P_1, P_2 son ciertos y se itera una vez el ciclo (principal) de la Implementación 2.3.1, entonces P_0, P_1, P_2 siguen siendo ciertos.

Demostración

Lo que cambia en una iteración del ciclo son la variable n y (posiblemente) el arreglo $A[n..N)$. Para proceder, se supone que P_0, P_1, P_2 son ciertos antes de que estos cambios surtan efecto. Además, como es necesario que la condición $0 \leq n < N$ (esta es la guarda del ciclo) se cumpla para que dicha iteración se dé, se supondrá también que es cierta. Observe:

- Como $A[0..N)$ es una permutación de $\bar{A}[0..N)$ (suposición de que P_0 es cierto antes de la iteración) y como lo único que afecta a A en una iteración es el intercambio $A[n], A[m] = A[m], A[n]$, el invariante P_0 es cierto después de una iteración del ciclo.
- Por P_1 , se tiene que $A[0..n)$ está ordenado ascendentemente con los n elementos más pequeños de $\bar{A}[0..N)$. En una iteración, $A[n]$ es actualizado con el elemento más pequeño de $A[n..N)$. Luego, $A[0..n+1)$ está ordenado ascendentemente con los $n+1$ elementos más pequeños de $\bar{A}[0..N)$ (este es el invariante P_1 después de una iteración del ciclo).
- Como $0 \leq n < N$ y n aumenta en uno con una iteración del ciclo, entonces claramente $0 \leq n+1 \leq N$, lo cual corresponde al invariante P_2 después de incrementar n .

Finalmente, note que el ciclo termina cuando $n = N$. Como el ciclo es correcto con respecto a los invariantes P_0, P_1, P_2 , se tiene que $A[0..N)$ tiene los N elementos de $\bar{A}[0..N)$ más pequeños y ordenados. Es decir, `sortiter(A)` ordena ascendentemente el arreglo A . El análisis de las complejidades temporal y espacial, al igual que la corrección del ciclo interno de Implementación 2.3.1 se proponen como ejercicio al lector.

Ejercicios

1. Demuestre que el ciclo interno en Implementación 2.3.1 termina.
2. Proponga invariantes para el ciclo interno en Implementación 2.3.1 y demuestre que el diseño del ciclo es correcto.
3. Considere el problema del ordenamiento de un arreglo de números. Suponga que únicamente se condieran arreglos cuyos valores están entre 0 y 10000.
 - a) Especifique el problema.
 - b) Diseñe un algoritmo que resuelva el problema dado, y cuyas complejidades temporal y espacial sean $O(N)$, en donde N es el tamaño del arreglo.
4. Considere la siguiente colección de problemas algorítmicos sobre un arreglo de números:
 - a) Calcular la suma de los elementos del arreglo.
 - b) Determinar el mínimo valor en el arreglo.
 - c) Determinar el máximo valor en el arreglo.

Para cada uno de estos problemas:

- Especifique el problema dado.
- Diseñe un algoritmo iterativo, incluyendo los invariantes correspondientes, que resuelva el problema especificado.
- Demuestre que el algoritmo es correcto con respecto a la especificación.
- Calcule la complejidad temporal del algoritmo diseñado.
- Implemente el algoritmo en el lenguaje de programación Python.

5. Considere la siguiente función `sortitergen`:

```

1 def sortitergen(A, low, hi):
2     for n in range(low, hi):
3         m = n
4         for i in range(n+1, hi):
5             if A[i] < A[m]: m = i
6         A[n], A[m] = A[m], A[n]
```

Esta es una versión generalizada de la función `sortiter` en la Implementación 2.3.1 que –dados $A[0..N)$, low , hi , con $0 \leq low \leq hi \leq N$ – ordena ascendentemente el subarreglo $A[low..hi)$.

Proponga invariantes para el ciclo externo y demuestre la correctitud de la función (suponiendo que el ciclo interno es correcto).

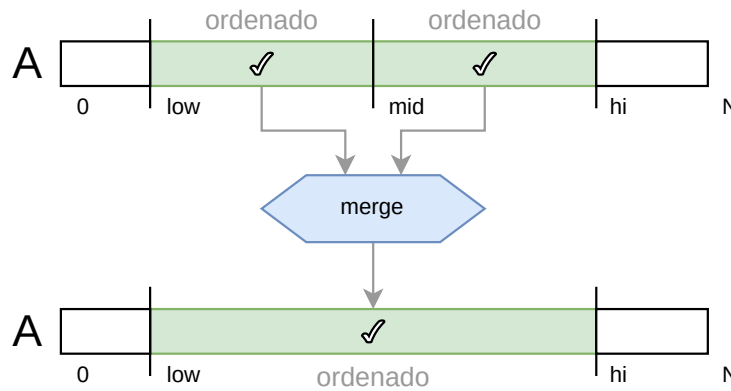
6. La siguiente es una enumeración de algoritmos de ordenamiento iterativos ampliamente conocidos:
 - a) *BubbleSort*
 - b) *InsertionSort*
 - c) *ShellSort*

Para cada uno de estos algoritmos en el caso de ordenamiento de un arreglo de números:

- Plasme el diseño del algoritmo al estilo del Algoritmo 2.3.1.
- Proponga invariantes para los ciclos que materializan los algoritmos iterativos.
- Demuestre que son correctos con respecto a la especificación propuesta.
- Determine la complejidad temporal y espacial del algoritmo.

2.4. Mergesort: un ordenamiento de arreglos más eficiente

La idea principal detrás del algoritmo *MergeSort* no es muy distinta a la de los algoritmos de ordenamiento presentado en las secciones 2.2 y 2.3, pues su diseño se puede también explicar con base en dividir, conquistar y combinar. La principal característica de *MergeSort* es que usa la hipótesis inductiva dos veces en cada paso del “combinar”: el subarreglo se divide en dos “mitades” que, siendo problemas más pequeños que el original, se puede suponer que están ordenadas ascendentemente (i.e., se hace un llamado recurrente para cada una de ellas), para posteriormente combinarlas a modo de cremallera, manteniendo el orden de los elementos al ser consolidados en el arreglo original.



A cada momento, el algoritmo mantiene un “cerco” para el ordenamiento de un subarreglo de A con base en las variables low y hi : el objetivo es ordenar $A[low..hi]$. Si esto es posible para cualesquiera low y hi , en particular se tiene resuelto el ordenamiento de todo el arreglo $A[0..N]$.

Antes de continuar explicando la imagen anterior que resume la idea de *MergeSort*, se introducen las siguientes condiciones:

C_0 : $A[0..N]$ es una permutación de $\overline{A}[0..N]$.

C_1 : $0 \leq low \leq hi \leq N$.

Note que la idea de ordenar el arreglo dado /in-situ/ se captura con la condición C_0 . La condición C_1 brinda pistas de cómo se puede avanzar con low y hi ordenando por partes:

- Si $A[low..hi]$ tiene a lo sumo un elemento, entonces no hay mucho que hacer pues el arreglo vacío y cualquier arreglo unitario están ordenados por definición. Estos casos suceden cuando $low + 1 \geq hi$.
- Si $A[low..hi]$ tiene al menos dos elementos, entonces se puede “partir” en dos mitades (aproximadas) $A[low..mid]$ y $A[mid..hi]$ ordenando cada una de ellas recurrentemente y posteriormente combinándolas. Estos casos suceden cuando $low + 1 < hi$.

La idea principal detrás de la función `mergesort` en el Algoritmo 2.4.1 debería estar clara en este punto. Por ende, el foco estará en entender cómo la función `merge` combina las dos partes del arreglo que están ordenadas garantizando el ordenamiento general del arreglo $A[low..hi]$.

Implementación 2.4.1: MergeSort en Python

```

1 def mergesort(A, low, hi):
2     if low+1<hi:
3         mid = low+((hi-low)>>1) # mid = (low+hi)//2
4         mergesort(A, low, mid) # induction hypothesis on the first half
5         mergesort(A, mid, hi)  # induction hypothesis on the second half
6         merge(A, low, mid, hi) # combine the two halves preserving the order
7
8 def merge(A, low, mid, hi):
9     global tmp # a global array at least the size of A
10    for i in range(low, hi): tmp[i] = A[i] # copy A[low..hi) to tmp[low..hi)
11    l,r = low,mid
12    for n in range(low, hi):
13        if l==mid: A[n],r = tmp[r],r+1 # only process the right half
14        elif r==hi: A[n],l = tmp[l],l+1 # only process the left half
15        else:
16            # the first pending element of each half needs to be compared
17            if tmp[l]<=tmp[r]: A[n],l = tmp[l],l+1 # choose the one on the left
18            else: A[n],r = tmp[r],r+1 # choose the one on the right

```

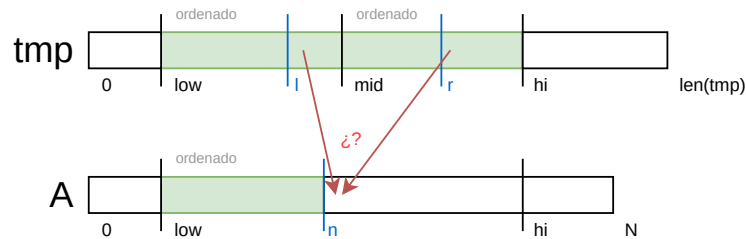
Para entender cómo encajan las piezas del algoritmo, se proponen los siguientes invariantes para el segundo ciclo de la función `merge`:

P_0 : $A[low..n]$ es un ordenamiento de $tmp[low..l]$ y $tmp[mid..r]$.

P_1 : $low \leq l \leq mid \leq r \leq hi$.

P_2 : $low \leq n \leq hi$.

El invariante P_0 indica que en $A[low..n]$ se van almacenando, en orden, los elementos de $tmp[low..l]$ y $tmp[mid..r]$ que han sido procesados. Los invariantes P_1 y P_2 indican entre qué rangos pueden tomar valores las variables l , r y n . Intuitivamente, las variables l y r pueden entenderse como dos (dedos) índices que señalan el siguiente elemento de la mitad izquierda y de la derecha, respectivamente, que debe ser procesado. Cuando $l = mid$ se tiene que la mitad izquierda ha sido agotada; de la misma forma, cuando $r = hi$ se tiene que la mitad derecha ha sido agotada. En cualquiera de estos dos casos no hay que comparar nada y basta con copiar a A los elementos pendientes en la mitad correspondiente. Si hay al menos un elemento en cada una de las mitades, entonces es necesario compararlos para determinar cuál de los dos se copiará a $A[n]$: el menor de los dos o, en caso de igualdad, el de la mitad izquierda. El siguiente diagrama resume visualmente el rol de cada una de las variables y de los arreglos involucrados en `merge`:



Antes de proceder a demostrar que `merge` y `mergesort` son correctos con respecto a su especificación, se presentan ejemplos de cómo el algoritmo ordena algunos arreglos de números.

Ejemplo 2.4.1

La función `mergesort` se usa a continuación con algunos ejemplos:

```
1 tmp = [ None for _ in range(10) ]
2 A = [ 8, -10, -2, 3, -50, 2, 10, 8, 4 ]
3 mergesort(A, 0, len(A))
4 print(A)
5
6 A = [ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 ]
```

```

7 mergesort(A, 0, len(A))
8 print(A)
9
10 A = [ ]
11 mergesort(A, 0, len(A))
12 print(A)
13
14 A = [ 5 ]
15 mergesort(A, 0, len(A))
16 print(A)

```

El resultado de esta ejecución es el siguiente:

```

1 [-50, -10, -2, 2, 3, 4, 8, 8, 10]
2 [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
3 []
4 [5]

```

A continuación se presenta y demuestra el teorema de correctitud de `mergesort`.

Teorema 2.4.1

Si $A[low..mid)$ y $A[mid..hi)$ están ordenados ascendentemente, entonces el llamado `merge(A, low, mid, hi)` garantiza el ordenamiento ascendente de $tmp[low..hi)$ en $A[low..hi)$.

Demostración

Se procede por inducción sobre n .

- Caso base ($n = hi$): en este caso no debe hacerse nada pues las dos mitades tmp han sido procesadas. Para observar esto con mayor claridad, a partir del código se puede deducir la siguiente igualdad (esto hay que demostrarlo y se propone como ejercicio al lector):

$$n - low = (l - low) + (r - mid)$$

Con $n = hi$ y con base en el invariante P_1 , se tiene que las igualdades $l = mid$ y $r = hi$ son ciertas (esto también se propone como ejercicio al lector); es decir, no hacen falta elementos en $tmp[low..hi)$ por ser procesados.

- Caso inductivo ($n \neq hi$): esto indica que hay al menos un valor por ser procesado, y que al menos uno de los arreglos $tmp[l..mid)$ y $tmp[r..hi)$ no es vacío. Se procede por casos:

- Si $l = mid$, entonces la mitad izquierda no tiene elementos pendientes por ser procesados y el siguiente a ser almacenado en $A[n]$ es $tmp[r]$; se “recurre” ordenando $tmp[r + 1..hi]$ en $A[n + 1..hi]$.
- Si $r = hi$, el caso es similar al anterior, pero se procesa el siguiente valor en la mitad izquierda pues la derecha no tiene elementos por ser procesados; se “recurre” ordenando $tmp[l + 1..mid]$ en $A[n + 1..hi]$.
- De lo contrario, $l \neq mid$ y $r \neq hi$ y se debe decidir entre $tmp[l]$ y $tmp[r]$ para actualizar $A[n]$. Si $tmp[l] \leq tmp[r]$ entonces se actualiza $A[n]$ con $tmp[l]$, y se “recurre” ordenando $tmp[l + 1..mid]$ y $tmp[r..hi]$ en $A[n + 1..hi]$. De lo contrario (i.e., $tmp[l] > tmp[r]$) se actualiza $A[n]$ con $tmp[r]$, y se “recurre” ordenando $tmp[l..mid]$ y $tmp[r + 1..hi]$ en $A[n + 1..hi]$.

Por la hipótesis inductiva, dado que hay menos elementos que ordenar en los casos recurrentes, se tiene que $A[low..hi]$ es un ordenamiento ascendente de $tmp[low..hi]$.

En consecuencia, como $tmp[low..hi]$ es una copia de la versión inicial de $A[low..hi]$, se concluye que $merge(A, low, mid, hi)$ ordena ascendentemente a $A[low..hi]$.

Con base en la correctitud (o corrección) de $merge$, se puede establecer la correctitud de $mergesort$.

Teorema 2.4.2

Para $N = len(A)$ y $0 \leq low \leq hi \leq N$, el llamado $mergesort(A, low, hi)$ ordena ascendentemente $A[low..hi]$.

Demostración

Se procede por inducción sobre el tamaño de $A[low..hi]$:

- Caso base ($low + 1 \geq hi$): en este caso, $mergesort$ no modifica a A . Note que cuando $low + 1 \geq hi$, $A[low..hi]$ es vacío o tiene exactamente un elemento, por lo cual está ordenado ascendentemente.
- Caso inductivo ($low + 1 < hi$): en este caso, el problema se divide en dos subproblemas: ordenar $A[low..mid]$ y $A[mid..hi]$. Bajo la condición dada, el código garantiza que la desigualdad $low < mid < hi$ (esto se debe demostrar y se propone como ejercicio para el lector). Esto garantiza que cada uno de los sub-arreglos tiene al menos un elemento y ambos tienen menor tamaño que $A[low..mid]$: como son más pequeños,

por hipótesis inductiva, se puede suponer que después de los llamados recurrentes `mergesort(A, low, mid)` y `mergesort(A, mid, hi)` ordenan $A[low..mid]$ y $A[mid..hi]$, respectivamente. Dado que `merge` es correcto (Teorema 2.4.1), el llamado `merge(A, low, mid, hi)` ordena ascendentemente a $A[low..hi]$.

En cualquiera de los dos casos, $A[low..hi]$ resulta ordenado ascendentemente por `mergesort(A, low, hi)`.

La ventaja que tiene el algoritmo `mergesort` con respecto al primer algoritmo de ordenamiento estudiado, es que su comportamiento asintótico en función de la cantidad de operaciones que lleva a cabo para ordenar un arreglo es mejor (i.e., crece menos rápido).

Teorema 2.4.3

Para $N = \text{len}(A)$ y $0 \leq low \leq hi \leq N$, el llamado `mergesort(A, low, hi)` toma tiempo $O((hi - low) \log(hi - low))$. En consecuencia, `mergesort(A, 0, N)` toma tiempo $O(N \log N)$.

Demostración

Sea $n = hi - low$ (note que $n \geq 0$). A modo de ejercicio para el lector, se puede establecer que `merge(A, low, mid, hi)` toma tiempo $O(hi - low) = O(n)$ (esto es lo que cuesta combinar las soluciones a los subproblemas de `mergesort`). Entonces, la ecuación de recurrencia que define la cantidad de operaciones básicas está dada por

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

pues se recurre una vez en cada una de las dos mitades y cuesta $O(n)$ combinar estas soluciones. De acuerdo con el Teorema Maestro, se tiene que

$$T(n) \in O(n \log n).$$

Note que con $low = 0$ y $hi = N$, que corresponden al llamado `mergesort(A, 0, N)`, se tiene $n = N$ y consecuentemente $T(N) \in O(N \log N)$.

Finalmente, se analiza la complejidad espacial de `mergesort`.

Teorema 2.4.4

Para $N = \text{len}(A)$ y suponiendo que la variable `tmp` es suministrada para `merge`, el llamado `mergesort(A, 0, N)` usa $O(\lceil \log N \rceil)$ espacio, en donde $\lceil - \rceil$ es la función techo.

Demostración

Se propone como ejercicio para el lector.

Ejercicios

1. En la demostración del Teorema 2.4.1 se usa la siguiente igualdad:

$$n - \text{low} = (l - \text{low}) + (r - \text{mid})$$

Demuestre que esta igualdad es cierta. Concluya que, cuando $n = \text{hi}$ y con base en el invariante P_1 , $l = \text{mid}$ y $r = \text{hi}$.

2. Demuestre que la desigualdad $\text{low} < \text{mid} < \text{hi}$ es cierta en `mergesort`.
3. Demuestre el Teorema 2.4.3.
4. Demuestre el Teorema 2.4.4.
5. Considere la función `merge` que es llamada desde la función `mergesort` en la Implementación 2.4.1. Allí se usa la variable global `tmp` para almacenar una copia de partes de `A` en cada llamado. Suponiendo que dicha variable no es global y que en cada llamado recurrente se crea un arreglo de tamaño $\text{hi} - \text{low}$ para la copia temporal, demuestre que `mergesort(A, 0, N)` usa espacio $O(N \log N)$. Justifique su respuesta.
6. La cantidad de inversiones de un arreglo es un indicador de qué tan desordenado está: si este valor es 0, entonces el arreglo está completamente ordenado. Si el arreglo está ordenado descendientemente, entonces este valor es el máximo. Formalmente, una *inversión* en un arreglo $A[0..N)$, $N \geq 0$, es una pareja de índices $0 \leq i < j < N$ tales que $A[i] > A[j]$. Por ejemplo, en el arreglo $[3, 1, 2]$ hay dos inversiones: $(0, 1)$ y $(0, 2)$.
 - a) Diseñe un algoritmo que calcule la cantidad de inversiones en $A[0..N)$ en tiempo $O(N \log N)$.
 - b) Demuestre que el algoritmo propuesto es correcto con respecto a la especificación.

2.5. Búsqueda binaria

El problema de búsqueda binaria consiste en encontrar un valor en un arreglo ordenado.

Problema 2.5.1: Búsqueda binaria

Entrada: Un arreglo $A[0..N)$ de números ordenado ascendentemente, $N \geq 0$, y un número x .

Salida: ¿Está x en $A[0..N)$?

Esta versión del problema corresponde al problema de decisión, es decir, el que requiere responder *sí* o *no*. Hay otras versiones en las cuales se desea averiguar más información como, por ejemplo, un índice en el arreglo en el cual aparece el valor dado (si aparece) o la cantidad de ocurrencias de este. También hay variantes en las cuales el arreglo dado no está ordenado ascendentemente sino descendentemente. Cualquiera de estas variantes se puede resolver con modificaciones (en su mayoría sencillas) del algoritmo que se presenta en esta sección.

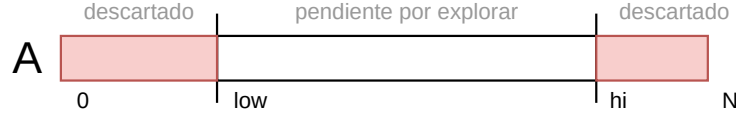
La idea general es explotar la información que se tiene sobre el arreglo $A[0..N)$. En particular, el hecho de que esté ordenado (ascendentemente, aunque –como se explica anteriormente– puede estar ordenado de otra forma) brinda la posibilidad de evitar hacer comparaciones y operaciones innecesarias. La idea general es la siguiente: suponga que se compara el valor dado x con el valor $A[n]$ del arreglo ($0 \leq n < N$). Allí hay tres opciones:

- Si $A[n] = x$, entonces x está en $A[0..N)$ y se responde afirmativamente.
- Si $A[n] < x$, entonces x necesariamente NO está en $A[0..n)$ dado que el arreglo está ordenado; en este caso, solo tiene sentido buscar a partir del índice $n + 1$. Es decir, se puede recurrir con un problema de tamaño menor.
- Si $A[n] > x$, entonces x necesariamente NO está en $A[n..N)$ dado que el arreglo está ordenado; en este caso, solo tiene sentido buscar antes del índice n . Es decir, se puede recurrir con un problema de tamaño menor.

Cualquier caso, bajo las suposiciones hechas en la especificación del problema, cae en una de las tres opciones enumeradas anteriormente. Así, se puede explicar de manera más precisa la idea central detrás del algoritmo de búsqueda: descartar rápidamente porciones del arreglo de entrada durante el proceso de búsqueda teniendo la certeza de que la porción del espacio de búsqueda que se preserva tiene suficiente información para responder la pregunta. La razón por la cual el algoritmo tiene la palabra “binaria” en su nombre es por la forma en cual el espacio restante

para la búsqueda se divide en dos “mitades”, de las cuales una se mantiene y la otra se descarta.

El diseño general del algoritmo se presenta a continuación:



El algoritmo mantiene dos *centinelas* *low* y *hi* que definen las fronteras del espacio de búsqueda activo $A[low..hi]$ para el valor x . Las otras dos porciones $A[0..low]$ y $A[hi..N]$ han sido descartadas y se tiene certeza de que no es necesario seguir buscando allí. Este diseño usa las siguientes condiciones:

$$C_0 : (\exists i \mid 0 \leq i < N : A[i] = x) \equiv (\exists i \mid low \leq i < hi : A[i] = x).$$

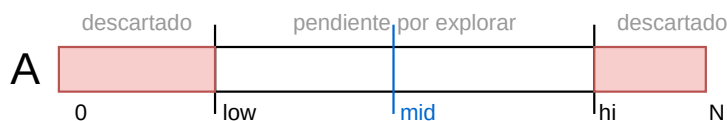
$$C_1 : 0 \leq low \leq hi \leq N.$$

La lectura de la condición C_0 se puede facilitar si la equivalencia lógica se interpreta como una doble implicación: si el valor está en el arreglo, entonces está en el espacio de búsqueda pendiente por explorar (implicación de izquierda a derecha); si el valor está en el espacio pendiente por explorar, entonces (claramente) está en el arreglo (derecha a izquierda). De manera alternativa y apelando a la contraposición proposicional, esta segunda implicación se puede interpretar de la siguiente forma: si el valor NO está en el arreglo, entonces NO está en el espacio de búsqueda pendiente por explorar. La condición C_1 es auxiliar y limita los valores que las variables *low* y *hi* pueden tomar. Es importante hacer dos observaciones sobre estas variables. Primero, *low* nunca debe sobrepasar a *hi*. Segundo, bajo estas restricciones necesariamente *low* es un índice de $A[0..N]$ cuando el arreglo no es vacío (i.e., cuando $N \neq 0$), pero este no es el caso con *hi* (¿por qué?). Estas propiedades son importantes pues la corrección y terminación del algoritmo dependen, en gran medida, de que sean ciertas.

La idea del algoritmo de búsqueda binaria es estrechar incrementalmente el espacio pendiente por explorar. Es decir, la idea es que las variables *low* y *hi* se acerquen de tal manera que la cantidad de valores pendientes por explorar $A[low..hi]$ sea cada vez más pequeña. ¿En qué casos indican estas variables que se ha estrechado suficientemente el cerco? De acuerdo con la condición C_1 , estos casos están dados por $low = hi$ y $low + 1 = hi$: o no hay espacio por explorar o hay exactamente una posición de *A* por ser explorada. En el primer caso, claramente la respuesta es negativa pues un arreglo vacío no contiene valor alguno. Como se verá en los algoritmos que se diseñarán, este caso base solo es posible cuando el arreglo $A[0..N]$ es vacío. En el segundo caso, esto quiere decir que la máxima reducción que se puede hacer del espacio de búsqueda, de acuerdo con los invariantes propuestos, es a $A[low..low + 1]$ que corresponde a $A[low]$. Si se combina esta observación con la

condición C_0 , entonces se habría logrado reducir la búsqueda en todo el arreglo a solo un punto: para determinar si x está en $A[0..N)$, basta con determinar si x es el valor en $A[low]$.

La siguiente pregunta a responder es la siguiente: ¿en qué casos (y cómo) se debe estrechar el cerco establecido por low y hi sobre $A[0..N)$? Estos son en realidad los casos inductivos y están caracterizados por la desigualdad $low + 1 < hi$. Note que bajo esta suposición, existe un número entero mid que satisface $low < mid < hi$. Esto en combinación con la condición C_1 , cuando $N \neq 0$, garantiza que mid es un índice del arreglo $A[0..N)$ y por ende, se puede consultar el valor almacenado allí: es precisamente contra $A[mid]$ que se compara x para decidir cuál de las dos partes $A[low..mid)$ o $A[mid..hi)$ descartar directamente. El cómo estrechar el cerco se responde de manera oportunista: como se ha de descartar una de dos partes, es conveniente que (sin saber a priori cuál) las dos partes sean lo suficientemente grandes como para estrechar el cerco lo más ajustado posible. Es de esta forma como se llega a la conclusión de que es conveniente que mid sea un punto medio (puede haber más de uno, ¿por qué?) entre low y hi .



Algoritmo 2.5.1

1. si $low = hi$, entonces responder negativamente;
2. si $low + 1 = hi$, entonces responder afirmativamente si $A[low] = x$ y negativamente de lo contrario; y
3. si $low + 1 < hi$, entonces sea mid un punto medio entre low y hi y:
 - a) si $A[mid] = x$, entonces responder afirmativamente;
 - b) si $A[mid] < x$, entonces recurrir sobre $A[mid..hi)$; y
 - c) si $A[mid] > x$, entonces recurrir sobre $A[low..mid)$.

Con base en este diseño se propone la siguiente implementación:

Implementación 2.5.1: Búsqueda binaria basada en el Algoritmo 2.5.1

```

1 def binsearch(A, x, low, hi):
2     ans = None
3     if low==hi: ans = False
4     elif low+1==hi: ans = A[low]==x
5     else:

```

```

6     mid = low+((hi-low)>>1)      # mid = (low+hi)//2
7     if A[mid]==x: ans = True
8     elif A[mid]<x: ans = binsearch(A, x, mid, hi)
9     else: ans = binsearch(A, x, low, mid)
10    return ans

```

La función `binsearch` identifica tres casos. Si el espacio de búsqueda es vacío, entonces responde negativamente. Si el espacio de búsqueda ha sido reducido a una sola posición, entonces se responde con base en lo que hay en esa posición: si x está allí, entonces se responde afirmativamente; de lo contrario, negativamente. Si el espacio de búsqueda tiene al menos dos elementos, entonces se calcula la “mitad” mid del subarreglo y se recurre con base en la relación existente entre x y el elemento $A[mid]$, cuando el elemento no se encuentra allí. Esta es una traducción literal Algoritmo 2.5.1 al lenguaje de programación Python.

A continuación se presenta la demostración de que la función `binsearch` es correcta con respecto a su especificación.

Teorema 2.5.1

Para $N = \text{len}(A)$ y $0 \leq low \leq hi \leq N$, el llamado `binsearch(A, x, low, hi)` determina si x está en $A[low..hi]$. En consecuencia, `binsearch(A, x, 0, N)` determina si x está en $A[0..N)$.

Demostración

Se procede por inducción sobre la distancia entre low y hi :

- Caso base ($low = hi$): el espacio de búsqueda es vacío, luego se responde negativamente
- Caso base ($low + 1 = hi$): el espacio de búsqueda tiene una sola posición $A[low]$, entonces se determina directamente si x corresponde a ese valor.
- Caso inductivo ($low + 1 < hi$): el espacio de búsqueda tiene al menos dos posiciones. Por la forma en que se calcula mid , se puede demostrar que: (i) $low < mid < hi$ y (ii) mid es un índice del arreglo A (estas demostraciones se proponen como ejercicio para el lector). Con base en estas observaciones, se tiene que la búsqueda de x en $A[low..mid]$ y $A[mid..hi]$, respectivamente, son subproblemas de la búsqueda en $A[low..hi]$. Note que si $A[mid] < x$, es imposible que x esté en $A[low..mid]$ (¿por qué?). De manera similar, si $A[mid] > x$, es imposible que x esté en $A[mid..hi]$ (¿por

qué?). En cada uno de los casos se recurre sobre la “mitad” que tiene el potencial de contener a x .

En consecuencia, el llamado `binsearch(A, x, 0, N)` resuelve el problema de búsqueda de x en el arreglo A ordenado ascendentemente.

La demostración de que la función `binsearch` termina se propone como ejercicio para el lector.

Teorema 2.5.2

Para $N = \text{len}(A)$ y $0 \leq \text{low} \leq \text{hi} \leq N$, el llamado `binsearch(A, x, low, hi)` termina.

Demostración

Se propone como ejercicio para el lector.

Teorema 2.5.3

Para $N = \text{len}(A)$, el llamado `binsearch(A, x, 0, N)` toma $O(\lceil \log N \rceil)$ tiempo, en donde $\lceil \cdot \rceil$ es la función techo.

Demostración

Ver el Ejemplo 1.4.3.

Una versión iterativa del algoritmo de búsqueda binaria se presenta a continuación. Inicialmente identifica dos casos. Si el arreglo dado es vacío, la respuesta necesariamente es negativa. De lo contrario, iterativamente se estrecha el cerco de la búsqueda tratando de acercar `low` y `hi` hasta que sean números consecutivos. En esta segunda parte, en la cual en realidad se lleva a cabo la búsqueda, note que la inicialización `low, hi = 0, N` hace que las condiciones C_0 y C_1 , vistas como invariantes del ciclo iterativo, sean ciertas trivialmente: inicialmente el espacio de búsqueda es todo el arreglo.

Implementación 2.5.2: Búsqueda binaria iterativa

```
1 def binsearch(A, x):
```



```

2  N,ans = len(A),False
3  if N!=0:
4      low,hi = 0,N
5      #  $C_0 \wedge C_1$ 
6      while low+1!=hi:
7          mid = low+((hi-low)>>1)    # mid = (low+hi)//2
8          if A[mid]<=x: low = mid
9          else: hi = mid
10     ans = A[low]==x
11     return ans

```

La correctitud y el análisis asintótico de la versión iterativa de `binsearch` se proponen como ejercicio para el lector.

Ejercicios

- Simule la ejecución del llamado `binsearch(A, 0, len(A))` para
 $A = [-4, 2, 2, 7, 11, 14, 18, 23, 100]$ y $x = 23$.
 Explique gráficamente qué sucede en cada llamado recurrente en relación con las variables `low`, `hi` y `mid`.
- En la demostración del Teorema 2.5.1, caso inductivo, se hacen las afirmaciones (i) y (ii). Proponga demostraciones para cada una de ellas.
- Demuestre el Teorema 2.5.2.
- Demuestre el Teorema 2.5.3.
- Proponga y demuestre el teorema de corrección del algoritmo de búsqueda binaria iterativa en la Implementación 2.5.2. Demuestre que el algoritmo termina.
- Especifique el problema de búsqueda binaria en un arreglo ordenado *descendentemente*.
- La siguiente lista de problemas tienen como entrada un arreglo de números $A[0..N)$, $N \geq 0$, y un número x .
 - Identificar el primer índice en el que aparece x en $A[0..N)$.
 - Identificar el último índice en el que aparece x en $A[0..N)$.
 - Determinar la cantidad de veces que aparece x en $A[0..N)$.
 En cada uno de los problemas anteriores, la respuesta debe ser, e.g., -1 cuando la búsqueda falla. Con base en esta información y para cada uno de los problemas:
 - Diseñe un algoritmo que resuelva el problema en tiempo $O(\log N)$.
 - Demuestre que el algoritmo es correcto.
 - Justifique por qué el algoritmo propuesto termina.

8. En un arreglo $A[0..N)$ de números, un número x es *mayoritario* si aparece, al menos, $N/2$ veces. Suponiendo que $A[0..N)$ está ordenado ascendentemente:
 - a) Diseñe un algoritmo que determine si x es mayoritario en tiempo $O(\log N)$.
 - b) Demuestre que el algoritmo es correcto.
 - c) Demuestre que el algoritmo termina.
 - d) ¿Cómo puede resolver el problema si el arreglo no necesariamente está ordenado?
9. Un *punto fijo* en un arreglo es un índice que almacena su mismo valor. Por ejemplo, 3 es un punto fijo en $[-2, 5, 1, 3, 0, 0]$.
 - a) Especifique el problema de encontrar un punto fijo en un arreglo $A[0..N)$ ordenado ascendentemente.
 - b) Diseñe un algoritmo que resuelva el problema dado en tiempo $O(\log N)$.
 - c) Demuestre que el algoritmo es correcto y termina.
10. En el ámbito de la estadística, la mediana representa el valor de la variable de posición central en un conjunto de datos ordenados. Por ejemplo, en el conjunto $\{7, 8, 9, 10, 11, 12\}$ la mediana es 9,5. Dado un arreglo de números ordenado ascendentemente, la mediana se puede calcular en tiempo $O(1)$. Un problema interesante, más bien, es determinar la mediana de un conjunto de datos cuando estos están divididos en dos grupos de igual tamaño.

Entrada: Arreglos $A[0..N)$ y $B[0..N)$, $N \geq 0$, de números y ordenados ascendentemente.

Salida: Mediana del conjunto de datos representado por los elementos de $A[0..N)$ y $B[0..N)$.

Diseñe y analice un algoritmo que en tiempo $O(\log N)$ resuelva el problema dado. Justifique su respuesta.

11. Un arreglo de números no repetidos es *bitónico* si está conformado por una secuencia ascendente con al menos dos elementos seguida de una secuencia descendente con al menos dos elementos, o si puede ser rotado circularmente para que cumpla esta condición. Por ejemplo, $[1, 2, 3, 0, -1]$ y $[5, 4, 3, 2]$ son bitónicos, mientras que $[0, 2]$, $[2, 4, 3, 5]$ ni $[1, 2, 1]$ lo son. Suponiendo que los números son distintos, diseñe y analice un algoritmo que determine si una arreglo dado $A[0..N)$ es bitónico en tiempo $O(\log N)$.
12. Considere un arreglo $A[0..N)$ con la propiedad especial de que $A[0] \geq A[1]$ y $A[N-2] \leq A[N]$. Se dice que un índice n de $A[0..N)$ es un *mínimo local* cuando $A[n-1] \geq A[n] \leq A[n+1]$ (necesariamente $0 < n < N-1$). Con las condiciones dadas, el arreglo debe tener al menos un mínimo local. Por ejemplo, en $[7, 2, 2, 4, -1, 5, 6]$ los mínimos locales son 1, 2 y 4. Diseñe y analice un algoritmo que en tiempo $O(\log N)$ encuentre un mínimo local en $A[0..N)$.

13. Considere un arreglo $A[0..N)$ que ha sido rotado circularmente k pasos, con $1 \leq k < N$. Por ejemplo, el arreglo $[3, 7, 10, 1, 2]$ está rotado 3 pasos.
- Diseñe y analice un algoritmo que en tiempo $O(\log N)$ calcule la cantidad de pasos que ha sido rotado $A[0..N)$.
 - Diseñe y analice un algoritmo que en tiempo $O(\log N)$ determine si un número x está en $A[0..N)$.

Notas del capítulo y referencias

El planteamiento en este capítulo de dividir, conquistar y combinar es similar al propuesto por T. Cormen et al. [CLRS22]. J. Kleinberg y É. Tardos [KET06], al igual que en este texto, introducen la técnica con *MergeSort*, y diseñan algoritmos para resolver problemas como los de encontrar pares de puntos más cercanos y la transformada rápida de Fourier. Estas dos referencias son fuentes variadas de ejercicios y problemas, sugeridas para profundizar en el tema.

Otros autores como H. Bhasin [Bha15] y A. Levitin [Lev12] distinguen diferentes versiones de dividir y conquistar. Por ejemplo, incluyen *decrementar* y conquistar, y *transformar* y conquistar. Por decrementar se entiende un caso de dividir y conquistar en el cual cada subproblema resulta de reducir el problema inicial en una cantidad constante y no en un factor constante (e.g., como el algoritmo de ordenamiento en la Sección 2.2 en donde cada subproblema tiene una posición menos que ordenar). Por transformar se entiende que las instancias iniciales son pre-procesadas antes de aplicar dividir y conquistar. Por ejemplo, si se desea determinar si un elemento en un arreglo es único, se puede ordenar el arreglo para luego determinar la unicidad del elemento. Estos textos también son una buena fuente de ejemplos y problemas algorítmicos.

El diseño de ciclos iterativos basado en invariantes es común en la literatura; ver, e.g., [CLRS22]. La disciplina de plantear invariantes a modo de predicados formales proviene de enfoques como los de E. Dijkstra [Dij76], D. Gries [Gri81], E. Cohen [Coh90] y A. Kaldewaij [Kal90]. En Castellano, el uso de predicados para la derivación de algoritmos iterativos y recurrentes es propuesto por J. Bohórquez [Boh06].

La inmensa mayoría de problemas propuestos como ejercicios en esta sección hacen parte del folclor de computación y algoritmos, y como tal se pueden encontrar en varias fuentes como libros, artículos científicos y divulgativos, e internet. El Ejercicio 2b de la Sección 2.1 es un resultado conocido como el Teorema de Gomory y cuya demostración usa ciclos Hamiltonianos. Los ejercicios 11, 12 y 13 de la Sección 2.5 están inspirados por sendos problemas en [Eri19].

Programación dinámica

La técnica de diseño presentada hasta ahora responde a una pregunta: ¿cómo evitar recorrer por completo el espacio de soluciones de un problema? Dividir, conquistar y combinar (Capítulo 2) lo parte en subespacios independientes que se resuelven por separado y cuyas soluciones se combinan al final. Este capítulo presenta una segunda respuesta: comprimir el espacio de soluciones, aprovechando que muchas de sus partes se repiten. Al descomponer un problema en subproblemas es frecuente que una misma subinstancia aparezca en varias ramas del cálculo y se resuelva una y otra vez; la *programación dinámica* organiza el cálculo de manera que eso no suceda. De manera más precisa, la programación dinámica parte de una función recurrente que resuelve un problema algorítmico –de decisión, de conteo o de optimización– y produce un algoritmo que la computa resolviendo cada subinstancia una sola vez.

La apuesta que hace la técnica para lograrlo es emplear memoria para ahorrar tiempo. El cálculo de la función de interés dispone de una memoria auxiliar que funciona como un registro: cada valor que se calcula se escribe allí una sola vez y se consulta después tantas veces como sea necesario. Esa apuesta no siempre se puede hacer y precisar cuándo es posible es el asunto con el que abre el capítulo.

Nota 3.0.1: Sobre el nombre

Richard E. Bellman (1920–1984) fue un matemático aplicado que acuñó, hacia 1950, el término *programación dinámica*.

Conviene aclarar de entrada que ‘programación’ no significa aquí escribir código. El término se usa en el sentido que tiene en ‘programación lineal’ o en ‘programación de una obra’: planificar, decidir por etapas. La técnica es anterior a que programar computadores fuera una actividad corriente y el nombre no alude a ella.

El nombre tampoco se escogió por razones técnicas, sino presupuestas. Bellman trabajaba en la Corporación RAND, financiada por la Fuerza Aérea de los Estados Unidos, en un momento en que el responsable de esa financiación en Washington sentía una hostilidad declarada hacia las palabras ‘investigación’ y ‘matemáticas’. Lo que Bellman necesitaba era un rótulo que ocultara que lo que hacía dentro de RAND eran matemáticas y al que ningún congresista pudiera objetar. El siguiente es un apartado traducido de su autobiografía (ver notas y referencias al final de capítulo) en el cual relata la ‘curiosa’ historia detrás de la intención de este nombre.

Una pregunta interesante es, “¿de dónde viene el nombre ‘programación dinámica’?” Los años 1950s no eran buenos para la investigación en matemáticas. Tuvimos un personaje muy interesante en Washington, llamado Wilson. Él era el Secretario de Defensa, y realmente tenía miedo y odio patológicos de la palabra ‘investigación’. No estoy usando estos adjetivos ligeramente; los estoy usando precisamente. Su cara se descomponía, se ponía roja, y se tornaba violento si alguien usaba el término ‘investigación’ en su presencia. Se podrán imaginar cómo se sentía, entonces, acerca del término ‘matemático’. La Corporación RAND estaba empleada por la Fuerza Aérea, la cual tenía, esencialmente, a Wilson como jefe. Por esto, sentí la necesidad de hacer algo para proteger de Wilson y de la Fuerza Aérea el hecho de que en realidad estaba haciendo matemáticas al interior de la Corporación RAND. ¿Qué título, qué nombre, podía escoger? En primer lugar, estaba interesado en planeación (en inglés, *planning*), en toma de decisiones, en pensar. Pero la palabra ‘planeación’ no era una buena elección por varias razones. Por ello decidí usar el término ‘programación’. Quería lograr la idea de que esto era dinámico, con múltiples escenarios, que dependía del tiempo; entonces se me ocurrió matar dos pájaros con una sola piedra. Usemos una palabra con un significado preciso y absoluto, es decir ‘dinámica’, en el sentido físico clásico. Esta palabra también tiene propiedades interesantes como adjetivo dado que es imposible usarla en un sentido peyorativo. Trate de pensar en una combinación que le haga tomar un significado peyorativo. Es imposible. Luego, pensé que ‘programación dinámica’ era un buen nombre. Era algo que ni siquiera un congresista podría objetar. Y ese fue el término que usé como sombrilla para mis actividades.

El relato tiene, eso sí, una inconsistencia que vale la pena señalar. El Wilson al que Bellman se refiere, Charles Erwin Wilson, fue Secretario de Defensa entre 1953 y 1957, mientras que la escogencia del nombre se sitúa en 1950.

Puede tratarse de otro Charles E. Wilson, que en esos años dirigía la Oficina de Movilización de la Defensa o simplemente de un recuerdo mezclado treinta años después. S. Dreyfus reprodujo y comentó estos pasajes en [Dre02].

Nota 3.0.2: Antes que los algoritmos, el nombre

A diferencia de lo que ocurre con las demás técnicas de este texto, aquí el nombre llegó antes que la mayoría de los algoritmos. La programación dinámica no nació en la algorítmica, sino en el control óptimo y la investigación de operaciones: Bellman llegó a la Corporación RAND en 1949 a trabajar sobre procesos de decisión de múltiples etapas, es decir, sobre problemas en los cuales hay que decidir por etapas y cada decisión condiciona las siguientes. En 1957 publicó el libro que fija la técnica y que enuncia el *principio de optimalidad*, antepasado directo de lo que en la Sección 3.1 se llama propiedad de subestructura óptima. Al año siguiente publicó, además, un algoritmo de caminos mínimos que hoy lleva su nombre junto con el de L. Ford.

Casi todos los algoritmos que este capítulo estudia son posteriores. La programación dinámica se incorporó a la algorítmica más tarde, cuando la disciplina empezó a organizar sus resultados por técnica de diseño y no por problema. Varios algoritmos que habían sido publicados de manera independiente quedaron reconocidos como instancias de una misma idea (ver las notas al final del capítulo).

Este capítulo presenta las ideas principales que respaldan la programación dinámica, incluyendo técnicas de implementación, una metodología para su uso y varios ejemplos, en dominios como la comparación de secuencias, la optimización combinatoria y los caminos mínimos en grafos.

3.1. Ideas clave

La programación dinámica es una técnica que puede ser útil para resolver problemas algorítmicos. Sin embargo, no en todos los problemas algorítmicos tiene sentido usarla. Aquellos problemas que son susceptibles de abordar usando la técnica exhiben ciertas propiedades en común que relacionan la solución de instancias con subinstancias.

Nota 3.1.1: Propiedades necesarias

Considere un problema algorítmico y suponga fijada una manera de descomponer cada una de sus instancias en subinstancias del mismo problema. Las

dos propiedades siguientes son necesarias para resolver el problema algorítmico con programación dinámica:

- *Propiedad de subestructura*: la solución de una instancia se puede calcular a partir de las soluciones de una cantidad acotada de subinstancias estrictamente menores.
- *Propiedad de solapamiento*: al desplegar la descomposición, una misma subinstancia se requiere desde varios lugares distintos. Dicho de otro modo, la cantidad de subinstancias diferentes es mucho menor que la cantidad de veces que se necesitan.

Conviene subrayar que estas dos propiedades no se predicán de un problema aislado sino de un problema *junto con* una manera de descomponerlo. Un mismo problema admite descomposiciones distintas y puede ocurrir que unas exhiban las propiedades y otras no. Por eso la Metodología 3.3.1 comienza por escoger una *función objetivo*: esa elección es la que fija la descomposición y, con ella, la que decide si la técnica es aplicable. En varias de las secciones de este capítulo la dificultad principal del diseño consiste, precisamente, en dar con la descomposición adecuada.

Toda descomposición debe cumplir además una condición para que tenga sentido hablar de subinstancias: debe existir una medida concreta y bien definida —el tamaño de un arreglo, la cantidad de elementos disponibles, la longitud de un prefijo— que decrezca estrictamente al pasar de una instancia a sus subinstancias. Sin esa medida, la descomposición puede no terminar y, en consecuencia, no define nada. Cuál es la medida apropiada depende del problema.

Un ejemplo conocido de la propiedad de subestructura es el problema de ordenamiento de un arreglo de números (Problema 2.2.1). Descomponga $A[0..N]$ en las dos mitades $A[0..n]$ y $A[n..N]$, con n la mitad de N . El arreglo ordenado se obtiene mezclando las dos mitades ordenadas, de modo que hay subestructura, y la medida que decrece es la longitud del subarreglo. Lo que no hay es solapamiento: las dos mitades no comparten ningún índice y lo mismo ocurre al desplegar la descomposición, así que ninguna subinstancia se necesita dos veces. Un algoritmo que resuelva cada subinstancia apenas la encuentre no repite trabajo, y esa es la razón por la cual el ordenamiento se resuelve con dividir, conquistar y combinar, y no con programación dinámica.

La propiedad de subestructura se puede especializar dependiendo de la naturaleza del problema que se desea resolver (e.g., decisión, conteo u optimización). El caso de los problemas de optimización es el más frecuente en este capítulo y tiene nombre propio.

Definición 3.1.1: Subestructura óptima

Un problema de optimización, junto con una descomposición fijada, tiene la *propiedad de subestructura óptima* si y solo si se cumplen las dos condiciones siguientes:

- *Descomposición*: para cada instancia existe una solución óptima cuyas partes son soluciones óptimas de las subinstancias correspondientes.
- *Composición*: al combinar soluciones óptimas de las subinstancias se obtiene una solución de la instancia, cuyo valor es el que la descomposición predice.

Nota 3.1.2: Las dos mitades

Las dos condiciones de la Definición 3.1.1 son independientes y ambas hacen falta. La de descomposición garantiza que la recurrencia no se le escapa el óptimo; la de composición, que todo lo que la recurrencia construye es una solución legítima.

Esa separación explica la forma que tienen todas las demostraciones de correctitud de este capítulo. Cuando se demuestra que una función objetivo ϕ calcula el óptimo, el argumento se parte invariablemente en dos desigualdades y cada una verifica una de las dos mitades: la que acota por composición exhibe soluciones construidas a partir de las subinstancias, y la que acota por descomposición toma una solución óptima cualquiera y la parte. El lector que llegue a la Sección 3.5, a la Sección 3.7 o a la Sección 3.8 reconocerá el patrón en cada una.

En la mitad de descomposición el cuantificador importa. En algunos problemas la propiedad vale en forma universal: *toda* solución óptima se descompone en soluciones óptimas de sus subinstancias, como sucede en el Ejemplo 3.1.1. En otros vale únicamente en forma existencial y, entonces, una demostración de correctitud no puede partir de una solución óptima cualquiera: debe exhibir la adecuada o transformar una cualquiera hasta obtenerla. La demostración del Teorema 3.7.1, en la Sección 3.7, es un ejemplo de lo segundo. Enunciar la propiedad en forma universal sería, por lo tanto, demasiado fuerte.

Esta propiedad es la versión algorítmica del *principio de optimalidad* que R. Bellman enunció en 1957 (Nota 3.0.2) y los dos nombres se usan indistintamente en la literatura. La propiedad reaparece en el Capítulo 4: es una

condición necesaria, aunque no suficiente, para que una estrategia voraz produzca soluciones óptimas.

Ejemplo 3.1.1

Considere el problema de determinar la ruta más corta de un punto A a un punto B en una ciudad. Note que si una ruta óptima (porque puede haber más de una) de A a B pasa por un punto C , entonces necesariamente en esa ruta la subruta de A a C también debe ser óptima. De lo contrario, la ruta de A a C no sería óptima: se podría llegar a C “más rápido” de otra forma y, en consecuencia, a B más rápido usando otra ruta. Pero esto riñe con la suposición de que la ruta de A a B que pasa por C es óptima; i.e., no puede haber una ruta mejor.

Observe que el argumento no escogió una ruta óptima en particular sino que partió de una cualquiera. Este problema exhibe, entonces, la forma universal de la propiedad. Para este problema, ¿pueden existir solapamientos entre subproblemas?

A simple vista pareciera que la propiedad de subestructura es inocua por su sencillez y obviedad; sin embargo, no todos los problemas cuentan con esta propiedad. Basta con cambiar ‘más corta’ por ‘más larga’ en el ejemplo anterior para que deje de valer.

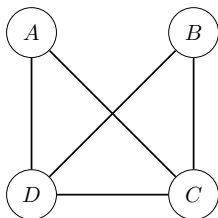


Figura 1. Una ciudad con cuatro sitios y cinco tramos. Entre A y B no hay un tramo directo.

Ejemplo 3.1.2

Considere el problema de determinar la ruta *simple* más larga de un punto A a un punto B en una ciudad. Simple se refiere a que no se visita un sitio más

de una vez; sin esa restricción el problema estaría indefinido, pues bastaría con dar vueltas indefinidamente. La longitud de una ruta es la cantidad de tramos que la componen.

Considere la instancia de la Figura 1. La ruta simple más larga de A a B tiene longitud 3 y una de ellas es $A \rightarrow C \rightarrow D \rightarrow B$. Esa ruta pasa por C , de manera que el argumento del Ejemplo 3.1.1 exigiría que su tramo inicial, de A a C , fuera una ruta simple más larga de A a C . No lo es: ese tramo tiene longitud 1, mientras que la ruta simple más larga de A a C es $A \rightarrow D \rightarrow B \rightarrow C$, de longitud 3. La propiedad de subestructura óptima falla.

La razón de fondo es que las partes óptimas no se dejan recomponer. La ruta simple más larga de A a C es $A \rightarrow D \rightarrow B \rightarrow C$ y la ruta simple más larga de C a B es $C \rightarrow A \rightarrow D \rightarrow B$; al concatenarlas usando C como punto “pivote” se obtiene

$$A \rightarrow D \rightarrow B \rightarrow C \rightarrow A \rightarrow D \rightarrow B,$$

que no es una ruta simple. Ser simple es una restricción global: no se puede verificar mirando las partes por separado y, por eso, las soluciones óptimas de las subinstancias no se dejan combinar.

Note que las dos mitades de la Definición 3.1.1 fallan en este problema y que cada uno de los dos párrafos anteriores refuta una: el primero, la de descomposición; el segundo, la de composición.

Falta ver la propiedad de solapamiento en acción, pues hasta aquí solo ha aparecido un problema que no la tiene. El ejemplo más sencillo es el de los números de Fibonacci, tema de la Sección 3.2: como $fib(n)$ se define en términos de $fib(n-1)$ y $fib(n-2)$, y el cálculo de $fib(n-1)$ vuelve a necesitar $fib(n-2)$, ese mismo valor se requiere desde dos lugares distintos. Al desplegar la recurrencia hay apenas n subinstancias diferentes, mientras que la cantidad de veces que se piden crece exponencialmente. Esa desproporción es la propiedad de solapamiento y es lo que la programación dinámica aprovecha.

La siguiente tabla resume los problemas mencionados en esta sección, cada uno con la descomposición que se discutió para él.

Problema	Subestructura	Solapamiento
Ordenamiento de un arreglo	sí	no
Números de Fibonacci	sí	sí
Ruta más corta	sí	sí
Ruta simple más larga	no	—

Únicamente la segunda y la tercera fila corresponden a problemas que se dejan abordar con programación dinámica. La primera se resuelve con dividir, conquistar

y combinar: sin solapamiento no hay nada que reusar y la memoria auxiliar sería un gasto sin contrapartida. La última no se deja abordar con ninguna de las dos, pues sin subestructura no hay siquiera una recurrencia que plantear; su casilla de solapamiento queda vacía porque no tiene sentido preguntar por ella.

Ejercicios

1. Justificando su respuesta, responda la pregunta formulada al final del Ejemplo 3.1.1.
2. Considere el problema de determinar si dada una colección de números $A[0..N]$ y un número x , es posible escoger algunos elementos de A cuya suma es x . Determine si este problema exhibe las propiedades de subestructura y solapamiento. Justifique e ilustre su respuesta con un ejemplo.
3. Considere el problema de buscar un valor en un arreglo ordenado, junto con la descomposición que descarta la mitad del arreglo en cada paso. Determine cuál de las dos propiedades exhibe y cuál no, y explique por qué eso lo convierte en un candidato para dividir, conquistar y combinar y no para programación dinámica.
4. En la Sección 3.4 se resuelve un mismo problema de dos maneras distintas: con dividir, conquistar y combinar y con programación dinámica. Después de leer esa sección, identifique la descomposición que usa cada una de las dos y explique por qué solo la segunda exhibe la propiedad de solapamiento. Concluya en qué sentido las dos propiedades dependen de la descomposición escogida y no únicamente del problema.
5. Suponga que se cuenta con el registro de precios de una acción a lo largo de N días, un precio por día. Se desea determinar la longitud de la mejor “corrida” de la acción durante los N días: es decir, la mayor cantidad de días (no necesariamente consecutivos) en los cuales la acción aumentó de precio. ¿Exhibe este problema alguna de las propiedades de subestructura o solapamiento? Justifique e ilustre su respuesta con un ejemplo.
6. Un empresario desea hacer una gira por varias ciudades conduciendo su vehículo (debe iniciar y terminar en la ciudad de origen, visitando cada una de las demás ciudades exactamente una vez). Para cada trayecto cuenta con la cantidad de dinero que cuesta el desplazamiento. Suponiendo que existe un camino directo entre cualquier par de ciudades, indique si determinar el menor costo posible de la gira exhibe alguna de las propiedades de subestructura o solapamiento. Justifique e ilustre su respuesta con un ejemplo.
7. En el Ejemplo 3.1.2 se exhibe una instancia en la cual la propiedad de subestructura óptima falla para el problema de la ruta simple más larga. Exhiba una

instancia con cuatro sitios en la cual la propiedad sí se cumpla y explique por qué un solo ejemplo favorable no permite concluir que el problema la tiene.

3.2. Cálculo de los números de Fibonacci

Se deja de lado, por un momento, el proceso de plantear una función recurrente que resuelva un problema con las propiedades de subestructura y solapamiento. El objetivo de esta sección es presentar las dos principales formas, *memorización* y *tabulación*, de instrumentar la programación dinámica. Para ilustrarlas, se parte de una recurrencia bien conocida: la función que calcula los números de Fibonacci.

Definición 3.2.1: Función de Fibonacci

Para $n \in \mathbb{N}$, la definición de la función *fib* es la siguiente:

$$fib(n) = \begin{cases} 0 & , \text{ si } n = 0, \\ 1 & , \text{ si } n = 1, \\ fib(n-2) + fib(n-1) & , \text{ si } n \geq 2. \end{cases}$$

El problema que se desea resolver es el siguiente.

Problema 3.2.1: Cálculo de la función de Fibonacci

Entrada: Un número entero N , $N \geq 0$.

Salida: $fib(N)$.

El Problema 3.2.1 no es de decisión, ni de conteo, ni de optimización: es la evaluación de una función que ya viene dada. Eso es deliberado. La Sección 3.3 se ocupará de cómo llegar a la función recurrente a partir de un problema; aquí la función se recibe hecha para poder concentrarse en cómo calcularla.

Vale la pena anotar que *fib* sí proviene de un problema de conteo. Leonardo de Pisa, conocido como Fibonacci, lo plantea en su *Liber Abaci* de 1202: una pareja de conejos recién nacida tarda un mes en madurar y a partir de entonces produce una pareja nueva cada mes; si ninguno muere, ¿cuántas parejas hay al cabo de n meses? La respuesta es exactamente $fib(n)$ con la definición anterior. La sucesión, sin embargo, había sido estudiada siglos antes en la India, en relación con el conteo de patrones métricos en la poesía sánscrita.

Hay dos casos base: para $N = 0$ o $N = 1$, se tiene que $\text{fib}(N) = N$. El caso recurrente se plantea para cualquier otro valor de N recurriendo sobre los dos valores inmediatamente anteriores, y por eso son necesarios los dos casos base.

La descomposición está fijada por la propia definición de *fib*: la instancia n se descompone en las subinstancias $n - 1$ y $n - 2$. Con esa descomposición, el Problema 3.2.1 tiene las dos propiedades de la Nota 3.1.1. Hay subestructura, pues el valor de una instancia se calcula a partir del de dos subinstancias y la medida que decrece estrictamente es el propio n . Y hay solapamiento, pues dos llamados recurrentes distintos terminan pidiendo la misma subinstancia: en el cálculo de $\text{fib}(4)$ se recalcula $\text{fib}(2)$, como se aprecia en la Figura 2. Note que la Definición 3.1.1 no interviene aquí, pues no se trata de un problema de optimización y no hay soluciones que comparar.

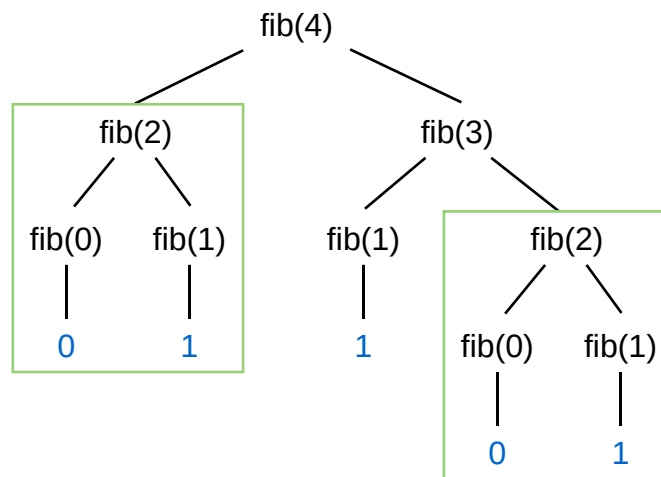


Figura 2. Árbol de llamados recurrentes en el cálculo de $\text{fib}(4)$. El subárbol correspondiente a $\text{fib}(2)$ aparece dos veces.

Una implementación directa de esta función en el lenguaje de programación Python 3 se presenta a continuación.

Algoritmo 3.2.1: Cálculo de la función fib

```

1 def fib(n):
2     ans = None
3     if n<=1: ans = n
4     else: ans = fib(n-2)+fib(n-1)

```

```
5 return ans
```

La función `fib` se usa a continuación con algunos ejemplos:

```
1 print('fib(0):', fib(0))
2 print('fib(1):', fib(1))
3 print('fib(2):', fib(2))
4 print('fib(10):', fib(10))
5 print('fib(15):', fib(15))
```

El resultado de esta ejecución es el siguiente:

```
1 fib(0): 0
2 fib(1): 1
3 fib(2): 1
4 fib(10): 55
5 fib(15): 610
```

Si la implementación de la función `fib` es tan sencilla y básicamente resulta de transcribir la definición matemática como código, ¿por qué hay que preocuparse por usar programación dinámica? La respuesta es que este algoritmo, para valores de N apenas moderados, no termina en ningún plazo razonable. Los llamados recurrentes se solapan y, en consecuencia, hay valores que se calculan una y otra vez.

Se puede ser preciso al respecto. El llamado `fib(n)` hace exactamente $2 \text{fib}(n+1) - 1$ llamados a `fib` (Ejercicio 2). Para $n = 100$ son alrededor de $1,15 \times 10^{21}$ llamados; aun suponiendo, con enorme optimismo, mil millones de llamados por segundo, el cálculo tardaría del orden de treinta mil años. No se trata de tener paciencia. La complejidad temporal de `fib(n)` es, entonces, $O\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right)$, expresión que resulta de resolver la ecuación característica $X^2 - X - 1 = 0$ asociada a la recurrencia (los detalles de una demostración que soporte esta afirmación se proponen como ejercicio para el lector). Es decir, la complejidad temporal de `fib` es exponencial, lo cual no es una buena noticia si el objetivo es contar con un algoritmo eficiente para calcular `fib`.

La forma en que resulta útil la programación dinámica es precisamente almacenando el resultado de los cálculos, ahorrando tiempo de cómputo cuando sea necesario visitar una y otra vez dichas instancias en cada uno de los subproblemas.

Nota 3.2.1: Memorización y tabulación

Hay dos formas usualmente empleadas para diseñar un algoritmo con programación dinámica y que permiten implementar eficientemente la función recurrente asociada a su solución:

Memorización: se calcula la función objetivo *por demanda* de modo tal que los valores intermedios, resultado de los llamados recurrentes, se van calculando en la medida que sea necesario. Generalmente, resultan algoritmos recurrentes muy parecidos a la función objetivo. Hay una memoria compartida que se usa: para una instancia específica del problema se consulta si ha sido resuelta antes (i.e., el valor está en la memoria compartida), entonces se usa el valor registrado en la memoria; de lo contrario, se calcula el valor correspondiente a dicha instancia, ya bien sea directa o recurrentemente, y se almacena en la memoria compartida para que pueda ser usado posteriormente de ser necesario.

Tabulación: se calcula la función objetivo *exhaustivamente* considerando todos los casos posibles para implementar la función dada con base en los parámetros de interés. La memoria compartida se va “llenando” incrementalmente con todos estos casos hasta obtener el valor deseado. Generalmente, resultan algoritmos iterativos.

En lo que resta de la sección se ilustra cómo usar memorización y tabulación para obtener una implementación eficiente de la función *fib*.

3.2.1. Memorización. La programación dinámica con memorización es conocida en inglés como programación dinámica *top-down*. Básicamente, se extiende la función a implementar con una memoria compartida que puede ser leída y modificada en los llamados recurrentes. Esto, específicamente, puede hacerse con una variable global o aumentando la lista de parámetros de la función con una variable que es pasada por referencia. Una de las ventajas de las implementaciones con memorización es que la estructura del código resultante es básicamente la misma que la de una transcripción de la función objetivo.

La idea general es la siguiente:

- Si el valor que se desea calcular está en la memoria compartida (i.e., ha sido calculado previamente), este valor se usa.
- De lo contrario, se calcula directa o recurrentemente el valor deseado y se almacena en la memoria compartida. En las invocaciones recurrentes para los subproblemas también se hace uso de la memoria compartida de modo tal que los cálculos intermedios queden almacenados en ella.

La implementación de *fib* con memorización se presenta en el Algoritmo 3.2.2 con la función `fib_memo`; un diccionario representa la memoria compartida y es identificado como `mem`.

Algoritmo 3.2.2: Cálculo de fib con memorización

```

1 def fib_memo(n, mem):
2     ans = None
3     if n in mem: ans = mem[n]    # if the value is available, use it!
4     else:         # otherwise, it needs to be computed
5         if n <= 1: ans = n
6         else: ans = fib_memo(n-2, mem) + fib_memo(n-1, mem)
7         mem[n] = ans             # store the value in the shared memory
8     return ans

```

Cuando el valor que se quiere calcular está disponible en `mem`, el algoritmo lo retorna. De lo contrario, el valor se calcula (incluyendo los casos base) y, una vez se cuenta con este valor, se almacena antes de retornarlo. Note que los llamados recurrentes también hacen uso de la memoria compartida: dado que `mem` se pasa por referencia, todos los cálculos que se hagan en los llamados recurrentes quedan almacenados en `mem`.

Es importante observar que la función `fib_memo` se comporta correctamente siempre y cuando se respeten algunas condiciones sobre sus parámetros. En particular, se debe garantizar que `mem` no contiene “basura” en el sentido de que los valores almacenados allí son correctos. De lo contrario, no habría garantía acerca de los cálculos realizados ni del valor retornado. Por ello, es importante contar con condiciones de representación que apoyen el diseño del algoritmo.

$$C_0 : (\forall k \mid k \in \mathbb{N} : k \in \text{mem} \Rightarrow \text{mem}[k] = \text{fib}(k)).$$

La condición C_0 indica que si k es una de las llaves del diccionario, necesariamente dicha llave está asociada al valor $\text{fib}(k)$; es decir, lo que está almacenado en el diccionario (suponiendo que solamente hay números naturales como llaves) son valores correctos de Fibonacci.

Conviene distinguir esta condición de una precondition. La condición C_0 se predica de la memoria compartida y debe valer tanto al entrar como al salir de cada llamado: es la que permite reusar sin verificar lo que está almacenado. Que n sea un número natural, en cambio, es una precondition sobre el parámetro: restringe cómo se puede invocar la función, no cambia durante la ejecución y no dice nada sobre `mem`.

Teorema 3.2.1

Sea $n \in \mathbb{N}$ y sea `mem` un diccionario que satisface la condición C_0 . Entonces el llamado `fib_memo(n, mem)` es tal que:

1. calcula $fib(n)$; y
2. al terminar su ejecución `mem` sigue satisfaciendo C_0 .

Demostración

Se procede por inducción fuerte sobre n , con análisis de casos sobre el cuerpo de `fib_memo`:

- Caso $n \in mem$: no se modifica `mem`, por lo cual C_0 sigue siendo cierto. Como C_0 se supone y $n \in mem$, entonces $mem[n] = fib(n)$. Luego, `fib_memo(n, mem)` calcula $fib(n)$.
- Caso $n \notin mem$: si $n = 0$ o $n = 1$, entonces `ans` toma el valor n , que es $fib(n)$, y la única modificación a `mem` agrega la llave n asociada a ese valor, de modo que C_0 se preserva. Si $n \geq 2$, los dos llamados recurrentes son sobre $n - 2$ y $n - 1$, ambos estrictamente menores que n . Por hipótesis inductiva, el primero calcula $fib(n - 2)$ y deja `mem` satisfaciendo C_0 ; en consecuencia el segundo se hace bajo las suposiciones del enunciado y calcula $fib(n - 1)$, dejando de nuevo C_0 cierto. Luego `ans` toma el valor $fib(n - 2) + fib(n - 1)$, que es $fib(n)$ por definición, y aumentar `mem` con $mem[n] = ans$ preserva C_0 .

Resta por hacer explícito cómo `fib_memo` resuelve el problema planteado inicialmente.

Teorema 3.2.2

El llamado `fib_memo(N, {})`, en donde `{}` representa el diccionario vacío, calcula $fib(N)$.

Demostración

La demostración es un corolario del Teorema 3.2.1 y se propone como ejercicio al lector.

A continuación se muestra el resultado de ejecución de `fib_memo`, incluyendo el cálculo para $n = 100$, y el contenido de la memoria compartida después de este llamado. Note que todos los valores hasta 100 han sido calculados y almacenados en la memoria compartida. Una ventaja práctica de la memorización es que la memoria compartida puede ser calculada incrementalmente: por ejemplo, hacer el llamado

para calcular `fib_memo(15, mem)` y posteriormente hacer el llamado para calcular `fib_memo(100, mem)` de tal forma que en el segundo llamado se cuente con algunos valores previamente (y correctamente) calculados en `mem`.

```

1 print('fib(0):', fib_memo(0, dict()))
2 print('fib(1):', fib_memo(1, dict()))
3 print('fib(2):', fib_memo(2, dict()))
4 print('fib(10):', fib_memo(10, dict()))
5 print('fib(15):', fib_memo(15, dict()))
6
7 mem = dict()
8 print('fib(100):', fib_memo(100, mem))
9 print(mem)

```

El resultado de esta ejecución es el siguiente:

```

1 fib(0): 0
2 fib(1): 1
3 fib(2): 1
4 fib(10): 55
5 fib(15): 610
6 fib(100): 354224848179261915075
7 {0: 0, 1: 1, 2: 1, 3: 2, 4: 3, 5: 5, 6: 8, 7: 13, 8: 21, 9: 34, 10: 55,
8  11: 89, 12: 144, 13: 233, 14: 377, 15: 610, 16: 987, 17: 1597,
9  ...
10 98: 135301852344706746049, 99: 218922995834555169026,
11 100: 354224848179261915075}

```

Finalmente, se establece el orden de la complejidad temporal de `fib_memo`.

Teorema 3.2.3

Sea $n \in \mathbb{N}$. Si el acceso y modificación de `mem` con `mem[n]` toma tiempo $O(\psi(n))$, entonces el llamado `fib_memo(n, mem)` toma tiempo $O(n\psi(n))$.

Demostración

Se propone como ejercicio al lector.

Si el acceso y modificación de diccionarios es de orden constante, entonces la complejidad temporal de `fib_memo` es de orden lineal en n . Su complejidad espacial también lo es: al terminar, la memoria compartida tiene una llave por cada valor entre 0 y n , a lo cual hay que sumar la pila de llamados recurrentes, que alcanza

profundidad n . Es importante resaltar que se está suponiendo algo que en la práctica no es necesariamente cierto: operar números grandes es de orden constante. En realidad, como la función fib crece rápidamente, es fácil encontrar valores de n no muy grandes para los cuales $fib(n)$ sobrepasa la cota de 2^{63} para números enteros en un procesador de 64 bits. En el caso particular de `fib_memo`, Python “esconde” del usuario esta complicación dado que sus enteros básicos son de precisión arbitraria. Este no necesariamente es el caso en otros lenguajes de programación, y deben usarse librerías especiales o implementar las operaciones de enteros grandes partiendo de cero. Con cualquiera de estas opciones, las operaciones aritméticas dejan de ser de orden constante en el peor de los casos.

3.2.2. Tabulación. Al igual que la memorización, la tabulación es una alternativa para diseñar algoritmos con programación dinámica. Esta técnica es conocida en inglés como *bottom-up*. Aunque sirven el mismo propósito, los algoritmos con tabulación son intrínsecamente distintos a los que resultan con memorización. Primero, los algoritmos con tabulación son iterativos por naturaleza. Segundo, con memorización los valores de la función objetivo se calculan por demanda, mientras que con tabulación se pueden calcular más valores intermedios de los que son necesarios para calcular la función para el objetivo final. Tercero, en algunos casos las tabulaciones pueden ser optimizadas para ahorrar espacio. Esta última característica hace que la tabulación sea preferida, en muchas situaciones, sobre la memorización.

A modo de ejemplo, se diseñará un algoritmo por tabulación que permita calcular la función fib . Esto se hará con la ayuda de una variable que almacene el cálculo parcial de los valores de la función hasta obtener el valor deseado.

$$P_0 : (\forall i \mid 0 \leq i < n : tab[i] = fib(i)).$$

$$P_1 : 0 \leq n \leq N + 1.$$

La variable de almacenamiento es el arreglo $tab[0..N + 1]$ que permite guardar valores de fib . En particular, $tab[0..n)$ almacena los primeros n valores de fib . La variable n indica qué valores se han calculado; note que cuando $n = N + 1$, el valor deseado está en $tab[N]$. Visualmente, estos invariantes se pueden representar como se muestra en el *diagrama de necesidades* de la Figura 3.

En una tabulación, el diagrama de necesidades indica cómo se inicializa la memoria adicional (en este caso $tab[0] = 0$ y $tab[1] = 1$) y cómo los valores que se calculan recurrentemente (i.e., $tab[n]$ para $n \geq 2$) se pueden determinar a partir de valores previamente calculados en la memoria (en este caso, $tab[n - 2]$ y $tab[n - 1]$). Note que de acuerdo con el invariante P_0 , los valores de fib están calculados correctamente en $tab[0..n)$, y n varía entre 0 y $N + 1$, inclusive.

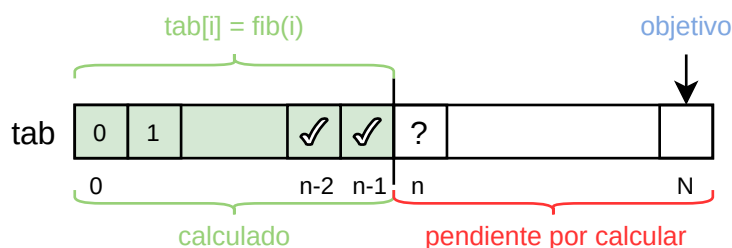


Figura 3. Diagrama de necesidades para la tabulación de *fib*. Las posiciones 0 y 1 de *tab* se inicializan directamente; cada valor posterior se obtiene de los dos inmediatamente anteriores.

Con base en estos invariantes y en el diagrama de necesidades, se propone la función *fib_tab* en el Algoritmo 3.2.3 para calcular iterativamente el valor de la función *fib*.

Algoritmo 3.2.3: Cálculo de *fib* con tabulación

```

1 def fib_tab(N):
2     ans = None
3     if N<=1: ans = N
4     else:
5         tab,n = [ None for _ in range(N+1) ],2
6         tab[0],tab[1] = 0,1
7         #  $P_0 \wedge P_1$ 
8         while n!=N+1:
9             tab[n],n = tab[n-2]+tab[n-1],n+1
10        ans = tab[N]
11    return ans

```

La función *fib_tab* determina el valor de retorno distinguiendo dos casos. Si $N \leq 1$, que corresponde a los casos base, el valor se calcula directamente. De lo contrario, se crea la memoria auxiliar en *tab*[0..N] inicializada con valores indefinidos; excepto por *tab*[0] y *tab*[1] que son 0 y 1, respectivamente, dado que *fib*(0) = 0 y *fib*(1) = 1. Para cualquier valor a partir de 2, el valor *tab*[n] se calcula sumando los dos valores inmediatamente anteriores en *tab*: por P_0 , estos valores coinciden con *fib*(n-2) y *fib*(n-1), luego se esperaría que *tab*[n] corresponda a *fib*(n) cuando se actualice la tabulación.

Para demostrar que el algoritmo iterativo de tabulación *fib_tab* funciona, es necesario demostrar que el ciclo satisface los invariantes P_0 y P_1 .

Teorema 3.2.4

El ciclo en `fib_tab` satisface los invariantes P_0 y P_1 . Es decir,

1. P_0 y P_1 son ciertos antes de la (posible) primera iteración del ciclo.
2. P_0 y P_1 son ciertos antes de una iteración del ciclo, entonces son ciertos después de una iteración del ciclo.

Demostración

Las propiedades se demostrarán para P_0 ; las demostraciones para P_1 se proponen como ejercicios para el lector.

1. Antes de la primera iteración del ciclo se tiene $n = 2$, de modo que P_0 exige únicamente que $tab[0] = fib(0)$ y que $tab[1] = fib(1)$; es decir, que $tab[0] = 0$ y $tab[1] = 1$. Eso es exactamente lo que hace la inicialización, luego P_0 es cierto antes de la primera iteración del ciclo.
2. Suponga que P_0 es cierto; se desea demostrar que P_0 es cierto después de una iteración del ciclo (es decir, se supone que P_0 es cierto para n y se demuestra que es cierto para $n + 1$). Note que al suponer que P_0 es cierto antes de una iteración del ciclo, se está suponiendo que $tab[0..n]$ tiene los primeros n valores de fib . En el cuerpo del ciclo se modifican $tab[n]$ y n con los valores $tab[n - 2] + tab[n - 1]$ y $n + 1$, respectivamente; como la asignación es simultánea, los lados derechos se evalúan antes de modificar nada y, por lo tanto, $tab[n - 2]$ y $tab[n - 1]$ son los valores que cubre la suposición. Además, el ciclo solo se ejecuta con $n \geq 2$, así que vale $fib(n) = fib(n - 2) + fib(n - 1)$. Se concluye que $tab[n] = fib(n)$ y, como n incrementa en una unidad, es cierto que:

$$(\forall i \mid 0 \leq i < n + 1 : tab[i] = fib(i)).$$

Con base en el Teorema 3.2.4 es fácil demostrar que la función `fib_tab` es correcta con respecto a la especificación del Problema 3.2.1.

Teorema 3.2.5

El llamado `fib_tab(N)` calcula $fib(N)$.

Demostración

Recuerde que $N \in \mathbb{N}$, como se indica en la formulación del problema. La demostración de terminación del algoritmo se propone como ejercicio al lector. Si $N = 0$ o $N = 1$, entonces el algoritmo responde 0 o 1, respectivamente. De lo contrario (i.e., $N \geq 2$), suponga que la ejecución del ciclo termina para cualquier $N \geq 2$. Note que esto sucede cuando $n = N + 1$. En este caso, el invariante P_0 es equivalente a:

$$(\forall i \mid 0 \leq i < N + 1 : \text{tab}[i] = \text{fib}(i)).$$

En particular, $\text{tab}[N] = \text{fib}(N)$. Es decir, en el caso $N \geq 2$, `fib_tab(N)` responde con el valor de $\text{fib}(N)$.

En cualquiera de los casos `fib_tab(N)` calcula el valor de $\text{fib}(N)$.

Los cálculos de las complejidades temporal y espacial de `fib_tab` se proponen como ejercicios al lector (¿cuáles son estas complejidades?).

Hay una versión iterativa para calcular *fib* con tabulación que toma tiempo $O(N)$ y espacio $O(1)$. Diseñar este algoritmo se propone como ejercicio al lector (note que en el caso recurrente solo depende de dos valores).

3.2.3. Comparación. Las dos técnicas presentadas en esta sección calculan lo mismo y con la misma complejidad temporal, pero no son intercambiables. La siguiente tabla resume sus diferencias.

	Memorización	Tabulación
En inglés	<i>top-down</i>	<i>bottom-up</i>
Forma del algoritmo	recurrente	iterativo
Valores que calcula	solo los necesarios	todos los del rango
Relación con la definición	transcripción casi literal	exige un orden de llenado
Memoria compartida	diccionario	arreglo o matriz
Reducción del espacio	difícil	con frecuencia posible

Las dos primeras filas explican por qué la memorización suele ser el primer diseño que se intenta: se obtiene casi copiando la definición recurrente. La tercera señala su ventaja propia, que es no calcular lo que no se necesita; hay problemas en los cuales el rango de la función es enorme pero solo una fracción pequeña de sus valores interviene, y allí la tabulación desperdiciaría todo el trabajo restante.

La última fila es la que más pesa en este capítulo. Una vez fijado el orden de llenado de una tabulación, es posible preguntarse qué parte de la tabla sigue haciendo falta y descartar el resto; con memorización eso es mucho más difícil, porque los valores se piden en un orden que depende de la instancia y no se sabe cuándo deja de necesitarse uno. El ejercicio que acaba de proponerse –calcular *fib* en espacio $O(1)$ – es el caso más simple de esa idea, que reaparece en la Sección 3.4,

en la Sección 3.6, en la Sección 3.7 y en la Sección 3.8, cada vez con una forma distinta. Esa es la razón por la cual la reducción del espacio es un paso de la Metodología 3.3.1.

Ejercicios

1. Al estudiar la función `fib`, se afirmó que su complejidad temporal es

$$O\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right),$$

en donde n es valor que se desea calcular. También se indicó que esta expresión resulta de resolver la ecuación de diferencia $X^2 - X - 1 = 0$ con variable X .

- a) Investigue sobre ecuaciones de recurrencia y explique de dónde resulta la ecuación dada.
 - b) Investigue sobre técnicas para resolver ecuaciones de recurrencia y aplique una de ellas para demostrar que la complejidad temporal de `fib(n)` es $O\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right)$.
2. Demuestre que el llamado `fib(n)` hace exactamente $2\text{fib}(n+1) - 1$ llamados a `fib`. Concluya que la cantidad de llamados es impar para todo n y calcule cuántos se hacen para $n = 100$.
 3. Demuestre que el llamado `fib_memo(n, dict())` hace exactamente $2n - 1$ llamados a `fib_memo` cuando $n \geq 1$. Compare esta cantidad con la del ejercicio anterior.
 4. Demuestre el Teorema 3.2.2.
 5. Demuestre el Teorema 3.2.3. Enuncie y demuestre además la afirmación correspondiente sobre la complejidad espacial de `fib_memo`, teniendo en cuenta tanto la memoria compartida como la pila de llamados recurrentes.
 6. En el Teorema 3.2.2 se concluye que el llamado `fib_memo(N, {})`, en donde `{}` representa el diccionario vacío, calcula $\text{fib}(N)$. Proponga (justificando su respuesta) otra forma de invocar `fib_memo` para que calcule $\text{fib}(N)$.
 7. Demuestre que el ciclo en `fib_tab` satisface el invariante P_1 .
 8. Demuestre que `fib_tab` termina.
 9. Para $N \in \mathbb{N}$, demuestre:
 - a) La complejidad temporal de `fib_tab(N)` es $O(N)$.
 - b) La complejidad espacial de `fib_tab(N)` es $O(N)$.
 10. Diseñe un algoritmo iterativo que use la técnica de programación dinámica con tabulación que permita calcular la función fib en tiempo $O(N)$ y espacio

$O(1)$. Formule invariantes y demuestre que el algoritmo es correcto. ¿Cuál es el diagrama de necesidades?

11. Un *coeficiente binomial* es un número entero positivo que corresponde a la expansión polinomial de la potencia de un binomio. Para $n, k \in \mathbb{N}$,

$$\binom{n}{k}$$

es el coeficiente del término x^k en la expansión de $(1+x)^n$. El valor numérico de la expresión $\binom{n}{k}$, leída “de n se toman k ”, corresponde a la cantidad de subconjuntos de k elementos de un conjunto de n elementos (si $k > n$, este valor es 0). Los coeficientes binomiales satisfacen la siguiente recurrencia:

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}.$$

Con base en esta definición recurrente:

- Explique por qué la programación dinámica es útil para resolver el problema de calcular coeficientes binomiales.
- Diseñe un algoritmo recurrente que calcule $\binom{n}{k}$ usando memorización.
- 🌈 Diseñe un algoritmo iterativo que calcule $\binom{n}{k}$ usando tabulación.

3.3. Una metodología

Con base en las ideas de la Sección 3.1 y en el ejemplo desarrollado en la Sección 3.2, se propone una metodología para diseñar algoritmos con programación dinámica a partir de la especificación de un problema. No debe entenderse como una receta infalible, sino como una lista de las preguntas que conviene responder y en qué orden.

Se anuncia de una vez cuál de los pasos concentra la dificultad. No es la implementación, que suele ser breve, ni el análisis, que suele ser inmediato: es el planteamiento de la función objetivo. Encontrar qué generalizar y con cuántos parámetros es lo que decide si la técnica funciona, y es también lo que la Sección 3.1 anticipó al señalar que las propiedades de subestructura y solapamiento se predicen de un problema junto con una descomposición. Escoger la función objetivo es escoger esa descomposición.

Metodología 3.3.1: Programación dinámica

Especificación: especificar el problema, indicando claramente su entrada y su salida. De esa lectura debe quedar claro si lo que se desea es decidir, contar u optimizar, pues de ello depende cuál de las propiedades de la Sección 3.1 hay que verificar: la subestructura general de la Nota 3.1.1 en

los dos primeros casos y la subestructura óptima de la Definición 3.1.1 en el tercero.

Función objetivo: formular una función objetivo ϕ que generalice la salida del problema o que, en su defecto, permita calcularla. Comúnmente se obtiene al generalizar los parámetros del problema que hacen parte de la salida de la especificación, y puede involucrar uno o más parámetros dependiendo del caso. Es importante que en este punto ϕ cuente con una descripción clara en palabras y *no* con una definición formal. La razón es que esa descripción es la especificación de ϕ y la definición recurrente que vendrá después es su diseño: tenerlas por separado es lo que permite demostrar que la segunda cumple la primera. Si se escribe primero la recurrencia, no queda nada contra qué contrastarla.

Reformulación de la especificación: verificar que la salida del problema se puede reescribir en términos de ϕ , de manera directa o indirecta. En algunos problemas basta con evaluar ϕ en los parámetros de la instancia; en otros hace falta además una cuantificación sobre sus valores. Si la salida no se deja reescribir de ninguna de las dos maneras, hay que volver al paso anterior y proponer otra función.

Planteamiento recurrente: presentar una definición recurrente de ϕ . Esta definición es la base de la programación dinámica y, como tal, debe ser completa, cubriendo todas las instancias, y no ser ambigua. Debe además decrecer en una medida bien fundada, en el sentido de la Sección 3.1, para que el cálculo termine.

Correctitud de la función objetivo: demostrar que la definición recurrente calcula lo que la descripción en palabras dice. Este paso es el que verifica que el problema tiene subestructura con respecto a la descomposición escogida, y su forma habitual se discute en la subsección siguiente.

¿Memorización o tabulación?: decidir cómo se instrumenta la recurrencia. La tabla de la Sección 3.2 resume las diferencias entre las dos opciones; en la práctica, conviene memorizar cuando solo hace falta una fracción pequeña de los valores de ϕ o cuando la recurrencia es difícil de ordenar, y tabular en los demás casos, sobre todo si se aspira a reducir el espacio. Dependiendo de la elección:

- Con memorización, determinar qué estructura de datos representa la memoria compartida y formular las *condiciones de representación* que debe satisfacer, como se hizo con C_0 en la Sección 3.2. Conviene además estimar la profundidad de la recurrencia, que en algunos lenguajes de programación puede provocar errores de ejecución.

- Con tabulación, diseñar un *diagrama de necesidades* que muestre de qué celdas depende cada celda, como el de la Figura 3. El diagrama fija el orden de llenado y de él se leen los invariantes de los ciclos.

Reducción del espacio (opcional): si se escogió tabular, mirar el diagrama de necesidades y preguntarse qué parte de la tabulación se puede descartar sin perder lo que hará falta después. Formular los invariantes de la versión reducida, que típicamente describen una memoria en la cual conviven valores del presente y del pasado.

Reconstrucción de una solución óptima (opcional): si el problema pide exhibir una solución y no solo su valor, decidir cómo recuperarla. La tabulación guarda, además del óptimo, el rastro de las decisiones que lo produjeron y recorrerla en sentido contrario permite reconstruirlas. Tenga presente que este paso puede reñir con el anterior.

Implementación: instanciar el Algoritmo 3.3.1 si se escogió memorización o escribir los ciclos que dicta el diagrama de necesidades si se escogió tabulación, y demostrar que satisfacen los invariantes formulados.

Análisis: establecer las complejidades temporal y espacial. Con tabulación, el tiempo suele ser el tamaño de la tabla multiplicado por el costo de llenar una celda; con memorización, la cantidad de llaves distintas multiplicada por ese mismo costo. Verifique que la cota resultante sea polinomial en el tamaño de la entrada y no solamente en los valores numéricos que aparecen en ella.

Los pasos anteriores se ilustrarán con cada uno de los ejemplos que se presentan en el resto del capítulo. De hecho, la Sección 3.2 ya siguió varios de ellos, aunque sin nombrarlos.

Paso	En la Sección 3.2
Especificación	Problema 3.2.1
Función objetivo	fib , que viene dada
Reformulación	la salida es $fib(N)$
Planteamiento recurrente	Definición 3.2.1
Correctitud	inmediata: la recurrencia es la función
¿Memorización o tabulación?	las dos, a modo de comparación
Reducción del espacio	se propone como ejercicio
Reconstrucción	no aplica
Implementación	algoritmos 3.2.2 y 3.2.3
Análisis	Teorema 3.2.3 y ejercicios

La fila de correctitud merece un comentario. En la Sección 3.2 no hubo nada que demostrar porque la función objetivo *era* la definición recurrente: no había una descripción en palabras separada de la fórmula. Ese es el caso excepcional y por eso esa sección sirve para presentar las técnicas de instrumentación, pero no la metodología completa. En todas las secciones siguientes las dos cosas son distintas y la demostración es indispensable.

3.3.1. El esquema general. Los algoritmos con memorización de este capítulo son todos el mismo algoritmo con distintos rellenos. Conviene, entonces, dejarlo escrito una vez.

Algoritmo 3.3.1: Esquema general de memorización

Para una función objetivo ϕ y una memoria compartida mem indexada por los parámetros p de ϕ , el cálculo de $\phi(p)$ procede así:

1. Si p es una llave de mem , retornar $mem[p]$.
2. De lo contrario:
 - a) si p corresponde a un caso base, calcular el valor directamente;
 - b) de lo contrario, invocar recurrentemente el esquema sobre cada una de las subinstancias que exige la definición de ϕ y combinar los resultados según esa definición;
 - c) almacenar el valor obtenido en $mem[p]$ y retornarlo.

Tres observaciones sobre el Algoritmo 3.3.1.

La estructura es la de ϕ : Quitando el paso 1 y el paso 2(c), lo que queda es la transcripción literal de la definición recurrente. Esa es la ventaja de la memorización y la razón por la cual conviene ensayarla primero: el algoritmo se obtiene casi sin decisiones de diseño.

La memoria nunca se corrige: Un valor almacenado en el paso 2(c) no vuelve a modificarse. Por eso basta con una condición de representación que diga que todo lo almacenado es correcto y por eso el paso 1 puede confiar en lo que encuentra sin verificar nada. Si la definición de ϕ es ambigua o incompleta, esta confianza se pierde y el esquema deja de funcionar.

La profundidad es la de la recurrencia: El esquema consume pila proporcional a la longitud de la cadena más larga de subinstancias, aunque la cantidad de llaves distintas sea pequeña. Esa es una diferencia real con la tabulación, que es iterativa, y es la responsable de los errores de ejecución que se mencionan en el paso correspondiente de la metodología.

3.3.2. El patrón de las dos desigualdades. El paso de correctitud de la función objetivo es el que exige un argumento propio en cada problema. Lo que no cambia de un problema a otro es la forma del argumento y vale la pena describirla una vez.

Sea ϕ la definición recurrente propuesta y sea L el valor que la descripción en palabras dice que ϕ debe calcular. Lo que hay que demostrar es “ $\phi = L$ ”, y eso se hace por inducción sobre la medida que decrece, separando la igualdad en dos desigualdades. Cada una corresponde a una de las dos mitades de la Definición 3.1.1:

Composición: cada alternativa que la recurrencia considera corresponde a una solución legítima de la instancia. En un problema de minimización esto da “ $\phi \geq L$ ” y en uno de maximización “ $\phi \leq L$ ”. El argumento consiste en construir, a partir de las soluciones óptimas de las subinstancias, una solución de la instancia, y verificar que es factible y que su valor es el que la recurrencia predice. Es la mitad “obvia” y la que se olvida con más frecuencia.

Descomposición: ninguna solución se le escapa a la recurrencia. Da la desigualdad contraria. El argumento consiste en tomar una solución óptima de la instancia, partirla según la descomposición escogida y reconocer sus pedazos como soluciones de las subinstancias, para luego invocar la hipótesis inductiva. Es la mitad “trabajosa”, y es donde el cuantificador de la Definición 3.1.1 se vuelve delicado: si la propiedad solo vale en forma existencial, no se puede partir de una solución óptima cualquiera sino que hay que transformarla primero.

Reconocer este patrón ahorra trabajo. Al leer las demostraciones de las secciones siguientes conviene identificar, en cada una, cuál de las dos mitades se está verificando: la Sección 3.5, la Sección 3.7 y la Sección 3.8 demuestran las dos desigualdades de manera explícita, y en la Sección 3.7 se ve con claridad el caso en el cual hay que transformar una solución óptima antes de poder partirla.

Ejercicios

1. El problema de multiplicación de una secuencia de matrices consiste en minimizar la cantidad de operaciones básicas (sumas y multiplicaciones escalares, básicamente) para multiplicar $N \geq 0$ matrices. Este algoritmo es uno de los ejemplos clásicos de programación dinámica.
 - a) Investigue sobre este problema y formule su especificación.
 - b) Investigue sobre su solución con programación dinámica y, paso a paso, identifique el uso en ella de la metodología propuesta en esta sección. Indique claramente en qué consiste cada uno de los pasos y, en caso tal de que alguno de los pasos no esté presente, justifique esta situación.
2. El problema del corte de un lingote de oro (en inglés conocido como *rod cutting*) consiste en maximizar la ganancia que se puede obtener de cortar verticalmente el lingote en varios pedazos (para venderlos individualmente).

Entrada: La longitud $N \geq 0$ de un lingote y una tabla $T[1..N]$ especificando la ganancia a obtener por cada tamaño vendido.

Salida: Máxima ganancia que se puede obtener vendiendo el lingote de tamaño N , posiblemente en varios pedazos, en relación con la tabla $T[1..N]$.

Considere la siguiente función objetivo, para $0 \leq n \leq N$:

$\phi(n)$: “Máxima ganancia que se puede obtener vendiendo el lingote de tamaño n , posiblemente en varios pedazos, en relación con la tabla $T[1..N]$ ”.

Como definición de ϕ , se propone la siguiente función recurrente para $0 \leq n \leq N$:

$$\phi(n) = \begin{cases} 0 & , \text{ si } n = 0, \\ (\uparrow i \mid 1 \leq i \leq n : T[i] + \phi(n - i)) & , \text{ si } n \neq 0. \end{cases}$$

El caso base indica que no hay ganancia alguna cuando no hay nada que vender. El caso recurrente indica que se escoge aquella opción que resulta de cortar el lingote en un pedazo de tamaño i y lo mejor que se pueda hacer con el tamaño restante $n - i$ (¿por qué esto cubre todos los casos?).

Con base en este planteamiento, siga la metodología propuesta en esta sección para diseñar:

- a) Una demostración de que ϕ calcula lo que su descripción dice, siguiendo el patrón de las dos desigualdades. Ilustre con un ejemplo la propiedad de solapamiento.
- b) Un algoritmo con memorización para ϕ , instanciando el Algoritmo 3.3.1.

- c) Un algoritmo con tabulación para ϕ (¿se puede hacer reducción de la tabulación?)
- d) Un algoritmo que, además del valor óptimo, retorne los tamaños de los pedazos en que conviene cortar el lingote.

En cada caso debe resultar código en Python ejecutable.

3. En la tabla de esta sección, la fila de correctitud dice que en la Sección 3.2 no hubo nada que demostrar. Proponga para *fib* una descripción en palabras que no sea la propia definición recurrente –por ejemplo, la que resulta del problema de los conejos– y demuestre, con el patrón de las dos desigualdades, que la Definición 3.2.1 la calcula.

3.4. Suma máxima de un subarreglo

El problema del subarreglo de suma máxima consiste en calcular la máxima suma posible de un subarreglo en un arreglo de números dado.

Problema 3.4.1: Subarreglo de suma máxima

Entrada: Un arreglo $A[0..N)$, con $N \geq 0$, de números.

Salida: Suma máxima de un subarreglo de $A[0..N)$.

Este ha sido un problema estudiado exhaustivamente desde su formulación en 1977 por Ulf Grenander (inicialmente se planteó para una matriz y no para un arreglo unidimensional). En el procesamiento de imágenes, por ejemplo, este problema tiene aplicación en la identificación de áreas brillantes en mapas de bits. También tiene aplicaciones en biología computacional y en finanzas.

Ejemplo 3.4.1

Considere el arreglo $A = [-1, 3, -2, 1, 4, -2, 1, 0]$. El subarreglo de suma máxima es $A[1..5) = [3, -2, 1, 4]$ con suma 6.

Este problema se puede resolver trivialmente para algunas instancias. Por ejemplo, si el arreglo dado únicamente tiene números no negativos, la respuesta es la suma de sus elementos. También, si todos sus elementos son números negativos, la respuesta es 0, pues el arreglo vacío es subarreglo de cualquier arreglo y tiene suma 0. El caso general se puede resolver exhaustiva pero ineficientemente, como se presenta en el Algoritmo 3.4.1.

Algoritmo 3.4.1: Fuerza bruta

1. Para cada pareja (i, j) que satisface $0 \leq i \leq j \leq N$, determinar la suma de $A[i..j]$.
2. Retornar aquella suma que sea máxima.

La complejidad temporal del Algoritmo 3.4.1 es $O(N^3)$ pues calcular todas las parejas (i, j) toma tiempo $O(N^2)$ y para cada una de ellas la suma de $A[i..j]$ toma tiempo $O(N)$. Esta complejidad se puede reducir a $O(N^2)$ con un pre-proceso que toma tiempo y espacio $O(N)$. En particular, se pueden calcular las sumas de los prefijos (o sufijos) de $A[0..N]$ de tal forma que la suma de cada subarreglo $A[i..j]$ se obtiene en tiempo $O(1)$, reduciendo de $O(N^3)$ a $O(N^2)$ el tiempo que toma la solución global.

Aplicando la técnica dividir, conquistar y combinar se puede obtener una solución más eficiente (o menos ineficiente).

Algoritmo 3.4.2: Dividir, conquistar y combinar

Suponga que low y hi son tales que $0 \leq low \leq hi \leq N$. El objetivo de este algoritmo es calcular la suma máxima de un subarreglo en $A[low..hi]$.

1. Si $low = hi$, entonces retornar $-\infty$.
2. Si $low + 1 = hi$, entonces retornar $A[low]$.
3. De lo contrario:
 - a) sea mid un punto medio entre low y hi ,
 - b) recurrir sobre $A[low..mid]$ para determinar la máxima suma de un subarreglo de $A[low..mid]$,
 - c) recurrir sobre $A[mid..hi]$ para determinar la máxima suma de un subarreglo de $A[mid..hi]$,
 - d) determinar la suma máxima de un subarreglo de $A[low..hi]$ que incluye el valor en mid (i.e., que cruza de izquierda a derecha), y
 - e) retornar el máximo valor obtenido en los pasos (3b),(3c),(3d).

El rango vacío del primer paso retorna $-\infty$, que es la identidad del máximo, porque sobre un rango vacío no hay ningún subarreglo no vacío que considerar. Si se desea incluir el subarreglo vacío entre las opciones, basta con maximizar con 0 el valor que retorna el algoritmo.

Suponiendo que el Algoritmo 3.4.2 es correcto (es un ejercicio para el lector), su llamado con $low = 0$ y $hi = N$ resuelve el problema del subarreglo de suma máxima en $A[0..N]$. Además, si el paso (3d) se calcula en tiempo lineal en función

de $hi - low$, con peor caso $O(N)$, entonces la cantidad de operaciones básicas que el algoritmo realiza para resolver el problema está dada por la recurrencia $T(N)$ definida en su caso recurrente por

$$T(N) = 2T(N/2) + O(N).$$

Es decir, bajo la suposición hecha, el Algoritmo 3.4.2 toma tiempo $O(N \log N)$ en resolver el problema de suma máxima de un subarreglo. Su complejidad espacial es $O(\log N)$, que corresponde a la profundidad de la recurrencia; la del Algoritmo 3.4.1 es $O(1)$ o bien $O(N)$ en la versión con sumas de prefijos.

Si se cuenta con un algoritmo relativamente eficiente que resuelve el problema sin usar programación dinámica, ¿qué hace este problema en este capítulo? Pues bien, lo que sucede es que se puede diseñar una solución que toma tiempo $O(N)$ si se usa programación dinámica. Ese es el objetivo de lo que resta de esta sección: diseñar dicha solución.

Nota 3.4.1: El algoritmo de Kadane

Los tres algoritmos que esta sección presenta reproducen, en orden, la historia del problema. Grenander lo planteó en 1977, en el contexto de la estimación de máxima verosimilitud de patrones en imágenes digitalizadas y dio para él una solución $O(N^2)$ con sumas de prefijos. Michael Shamos oyó del problema y en una sola noche produjo la solución $O(N \log N)$ por dividir, conquistar y combinar. Poco después, Shamos expuso el problema y su historia en un seminario en Carnegie Mellon; entre los asistentes estaba el estadístico Jay Kadane, quien diseñó la solución lineal en menos de un minuto. Por eso esa solución recibe el nombre de *algoritmo de Kadane* y es el diseño al cual se llega al final de esta sección.

La anécdota se conoce sobre todo por una columna de J. Bentley de 1984, escrita unos cinco años después del seminario, lo cual explica que a veces se le atribuya esa fecha. El propio Kadane relató los hechos en 2023 (ver las notas al final del capítulo).

Que el algoritmo sea asintóticamente óptimo no es una conjetura sino un hecho demostrable. Cualquier algoritmo que resuelva el problema debe leer los N elementos del arreglo: si alguno no se lee, se puede modificar ese elemento hasta cambiar la respuesta sin que el algoritmo se entere. Luego $\Omega(N)$ es una cota inferior y el algoritmo de Kadane la alcanza.

El mismo algoritmo se ha obtenido por caminos muy distintos del que sigue esta sección. D. Gries lo derivó en 1982 aplicando la estrategia de desarrollo de programas a partir de invariantes y R. Bird lo obtuvo en 1989 por manipulación algebraica del algoritmo de fuerza bruta.

El caso de la suma máxima de un subarreglo en un arreglo vacío es trivial, como se ha explicado anteriormente. Igualmente, considerar el arreglo vacío como uno de los posibles subarreglos (aún si el arreglo dado no es vacío) resulta en restringir la salida de la solución a números no negativos. Los casos interesantes se presentan, en realidad, al fijar la atención en aquellos subarreglos que tienen al menos un elemento. El desarrollo que se presenta a continuación se enfoca, entonces, en determinar la suma máxima de un subarreglo no vacío, dando por descontado el caso en el cual $A[0..N)$ tiene tamaño $N = 0$. Posteriormente, se presentará la solución para el caso más general en el cual el arreglo puede ser vacío.

Un arreglo de suma máxima (no vacío) termina en alguno de los índices de $A[0..N)$. Es decir, para algún índice $0 \leq n < N$, un subarreglo de suma máxima debe terminar en n (i.e., incluyendo a $A[n]$ como último elemento de la suma de elementos contiguos). Si se contara con la información de las mejores sumas que terminan en cada uno de los índices $0 \leq n < N$, entonces el problema de determinar la mejor suma de un subarreglo (no vacío) en $A[0..N)$ estaría resuelto: bastaría con maximizar sobre todos estos valores. Esta es la estrategia de diseño que se adopta para resolver el problema en cuestión. Se sigue la Metodología 3.3.1 para diseñar una solución de programación dinámica con tabulación; el paso de especificación ya quedó cumplido con el Problema 3.4.1.

Función objetivo. Para $0 \leq n < N$, se define:

$\phi(n)$: “suma máxima de un subarreglo de $A[0..N)$ que termina en n .”

La función ϕ tiene como dominio los índices de A y co-dominio a todos los números. Una forma alternativa (y equivalente) de interpretar ϕ es la siguiente: $\phi(n)$ es la suma máxima de un sufijo de $A[0..n)$.

Reformulación de la especificación. Note que, con base en ϕ , se puede especificar la salida del problema (restringido) como la siguiente maximatoria:

$$(\uparrow n \mid 0 \leq n < N : \phi(n)).$$

Si por convención se adopta que esta maximatoria es 0 cuando $N = 0$, entonces esta expresión también abarca el caso en el cual el arreglo dado es vacío. Es más, si también se desea considerar el subarreglo vacío como opción entre los subarreglos de A , basta con extender la maximatoria de la siguiente manera (¿por qué?):

$$(\uparrow n \mid 0 \leq n < N : \phi(n)) \uparrow 0.$$

En cualquier caso, es evidente que la función objetivo permite reformular el problema de interés.

Planteamiento recurrente. El objetivo es obtener una definición recurrente para ϕ . Siguiendo los principios de dividir, conquistar y combinar, el primer paso es identificar aquellos casos que se pueden resolver directamente sin necesidad de

recurrir (i.e., conquistar). Dada la definición de la función objetivo, ese caso corresponde a $\phi(0)$: ¿cuál es la suma máxima de un subarreglo de $A[0..N]$ que termina en 0? Solo $A[0..0]$ satisface esta condición. Es decir, $\phi(0) = A[0]$. ¿Qué sucede cuando $n \neq 0$? Para responder esta pregunta, suponga que $A[k..n]$ es un arreglo de suma máxima que inicia en algún $0 \leq k \leq n$ y termina en n . Es decir, el objetivo es que $\phi(n)$ tenga como valor la suma de los elementos de este subarreglo. Si $k = n$, entonces esta suma máxima es $A[n]$. Si $k \neq n$ (i.e., $k < n$), entonces esta suma máxima tiene la forma $A[k] + \dots + A[n-1] + A[n]$ o, de manera equivalente, $\phi(n-1) + A[n]$ (¿por qué? Ayuda: pensar en la propiedad de la subestructura óptima).

Con base en el análisis anterior, se propone la siguiente definición para ϕ , con $0 \leq n < N$ y $N \neq 0$:

$$\phi(n) = \begin{cases} A[0] & , \text{ si } n = 0, \\ A[n] \uparrow (A[n] + \phi(n-1)) & , \text{ si } n \neq 0. \end{cases}$$

El razonamiento anterior es una *derivación*: sirve para descubrir la recurrencia, no para justificarla. En él se usó ϕ con su significado pretendido –“la suma máxima que termina en n ”– antes de que hubiera una definición que lo garantizara, lo cual es legítimo mientras se esté explorando pero no como demostración. La justificación viene ahora, y sigue el patrón de las dos desigualdades de la Sección 3.3.

Correctitud de la función objetivo. La correctitud de la definición recurrente de ϕ se presenta como el Teorema 3.4.1.

Teorema 3.4.1

Sea $0 \leq n < N$. Si $N \neq 0$, entonces $\phi(n)$ es la suma máxima de un subarreglo de $A[0..N]$ que termina en n .

Demostración

Sea $L(n)$ la suma máxima de un subarreglo no vacío de $A[0..N]$ que termina en n ; es decir, el máximo de las sumas de los subarreglos de la forma $A[k..n]$ con $0 \leq k \leq n$. El objetivo es demostrar que $\phi(n) = L(n)$ y se procede por inducción sobre n .

Caso base $n = 0$: el único subarreglo no vacío que termina en 0 es $A[0..0]$, cuya suma es $A[0]$. Luego $L(0) = A[0] = \phi(0)$.

Caso inductivo $n \neq 0$: suponga que $\phi(n-1) = L(n-1)$. Se demuestran las dos desigualdades por separado.

- *Composición:* $\phi(n) \leq L(n)$. Basta ver que los dos argumentos del máximo en la definición de $\phi(n)$ son sumas de subarreglos que terminan en n . El primero, $A[n]$, es la suma de $A[n..n]$, luego $A[n] \leq L(n)$. Para el segundo, sea $A[k..n-1]$ un subarreglo de suma máxima que termina en $n-1$, de modo que su suma es $L(n-1)$; entonces $A[k..n]$ es un subarreglo que termina en n y su suma es $L(n-1) + A[n]$, de donde

$$A[n] + \phi(n-1) = A[n] + L(n-1) \leq L(n),$$

usando la hipótesis inductiva. Como los dos argumentos no exceden a $L(n)$, tampoco lo hace su máximo.

- *Descomposición:* hay que ver que $\phi(n) \geq L(n)$. Tome un subarreglo $A[k..n]$ de suma máxima que termine en n , con $0 \leq k \leq n$. Si $k = n$, entonces $L(n) = A[n] \leq \phi(n)$, pues $A[n]$ es uno de los dos argumentos del máximo. Si $k < n$, entonces $A[k..n-1]$ es un subarreglo no vacío que termina en $n-1$ y, por lo tanto, su suma no excede a $L(n-1)$. Entonces se tiene:

$$\begin{aligned} L(n) &= (A[k] + \cdots + A[n-1]) + A[n] && \text{(aritmética)} \\ &\leq L(n-1) + A[n] && \text{(observación)} \\ &= \phi(n-1) + A[n] && \text{(hipótesis inductiva)} \\ &\leq \phi(n) && \text{(uno de los argumentos).} \end{aligned}$$

En cualquier caso, $\phi(n) = L(n)$.

Con base en el planteamiento recurrente de la función objetivo, es fácil ver con ejemplos que existen solapamientos entre diferentes instancias de la solución. Se propone al lector, como ejercicio, encontrar instancias para las cuales esto pasa y explicar en qué consisten los solapamientos con esos casos en particular.

¿Memorización o tabulación? Se prefiere, dado que uno de los objetivos del diseño de la solución es ahorrar espacio, implementar la técnica de programación dinámica para la función objetivo con tabulación.

De acuerdo con la definición recurrente de ϕ , se propone el diagrama de necesidades de la Figura 4. En el diagrama se usa la memoria de tabulación $tab[0..N)$

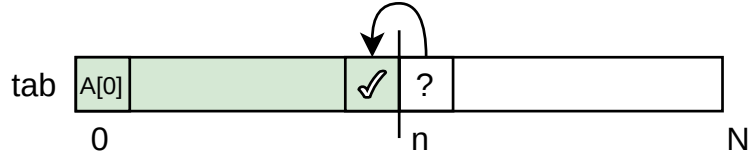


Figura 4. Diagrama de necesidades para la tabulación de ϕ en el problema del subarreglo de suma máxima. El valor $tab[n]$ depende únicamente de $tab[n-1]$.

para almacenar los valores de la función y una variable n para indicar el índice de la tabla que se debe calcular en un momento dado. Esta tabla en la posición 0 es inicializada con $A[0]$, como lo indica la definición de ϕ . Para calcular el valor para $n \geq 1$ (i.e., en el caso recurrente) en la tabla se depende directamente del valor inmediatamente anterior (esto sucede porque $\phi(n)$ depende de $\phi(n-1)$). Esto quiere decir que el valor en $tab[n-1]$ debe calcularse antes de intentar calcular $tab[n]$. Estas observaciones son importantes para diseñar el algoritmo que llena la tabla con los valores de ϕ .

El diseño del algoritmo iterativo para llenar la memoria de tabulación se basa en los siguientes invariantes:

$$P_0 : (\forall i \mid 0 \leq i < n : tab[i] = \phi(i)).$$

$$P_1 : 0 \leq n \leq N.$$

El invariante P_0 indica que el subarreglo $tab[0..n)$ tiene, en orden, los primeros n valores de ϕ : el valor en la i -ésima posición de tab almacena el valor $\phi(i)$. El segundo invariante indica en qué rango toma valores la variable n . Note que se pudo restringir a 1 como límite inferior para n , pero no es necesario. Con estos invariantes la estrategia está clara: llenar tab de izquierda a derecha. Con esta tabla completamente llena es posible, entonces, resolver el problema de determinar la suma máxima de los subarreglos de $A[0..N)$ al calcular el valor máximo en tab . Esto último se hará con una segunda pasada sobre tab , usando el siguiente invariante adicional.

$$P_2 : ans = (\uparrow i \mid 0 \leq i < n : \phi(i)) \uparrow 0.$$

Implementación. Con base en este diseño, se propone el Algoritmo 3.4.3.

Algoritmo 3.4.3: Tabulación para ϕ

```

1 def mss(A):
2     ans, N = 0, len(A)
3     if N != 0:
4         tab = [ None for _ in range(N) ]

```

```

5     tab[0] = A[0]
6     #  $P_0 \wedge P_1$ 
7     for n in range(1, N):
8         tab[n] = max(A[n], tab[n-1]+A[n])
9         #  $P_0 \wedge P_1 \wedge P_2$ 
10        for n in range(N):
11            ans = max(ans, tab[n])
12    return ans

```

El caso en el cual el arreglo dado es vacío se trata de manera especial con ayuda de una instrucción condicional.

Análisis. Resta demostrar que `mss` es correcta con respecto a la especificación dada y determinar sus complejidades temporal y espacial.

Teorema 3.4.2

El llamado `mss(A)`:

1. Calcula la suma máxima de un subarreglo de $A[0..N)$, incluyendo el subarreglo vacío entre las opciones.
2. Toma tiempo $O(N)$.
3. Toma espacio $O(N)$.

Demostración

Se propone como ejercicio al lector.

Reducción del espacio. En el diagrama de necesidades se puede observar que para el cálculo de `tab[n]` solo es necesario contar con el valor precalculado en `tab[n - 1]`. Es decir, de manera más general, para los casos recurrentes solo es necesario contar con un valor pre-calculado y no con un prefijo completo de `tab`. Con base en esta observación, se propone el invariante P_3 que introduce la variable entera `prev` para reemplazar el arreglo `tab`:

$$P_3 : \text{prev} = \phi(n - 1).$$

La variable `prev` representa la mejor suma de $A[0..N)$ que termina en la posición inmediatamente anterior a n . Por ello, la idea es usar este valor en el cuerpo del ciclo iterativo e ignorar todos los valores para índices inferiores a $n - 1$. Además, es necesario incluir el cálculo del valor máximo de ϕ dentro del cuerpo del ciclo principal puesto que sería imposible recuperarlo de otra forma habiendo reducido

el espacio de la tabulación (no hay arreglo que recorrer para calcular el máximo). Esta estrategia resulta en la función `mss_opt` que se presenta en el Algoritmo 3.4.4.

Algoritmo 3.4.4: Reducción del espacio de la tabulación

```

1 def mss_opt(A):
2     ans, N = 0, len(A)
3     if N!=0:
4         prev, ans = A[0], max(ans, A[0])
5         #  $P_1 \wedge P_2 \wedge P_3$ 
6         for n in range(1, N):
7             prev = max(A[n], prev+A[n])
8             ans = max(ans, prev)
9     return ans

```

Si se demuestra correcto, el algoritmo optimizado resolvería el problema de interés en tiempo $O(N)$ y en espacio $O(1)$.

Teorema 3.4.3

El llamado `mss_opt(A)`:

1. Calcula la suma máxima de un subarreglo de $A[0..N)$, incluyendo el subarreglo vacío entre las opciones.
2. Toma tiempo $O(N)$.
3. Toma espacio $O(1)$.

Demostración

Se propone como ejercicio al lector.

Finalmente, el Algoritmo de Kadane se puede obtener del Algoritmo 3.4.4 bajo ciertos supuestos o acuerdos. La observación clave es notar que las asignaciones a las variables `prev` y `ans` en las líneas 4 y 7-8 son básicamente las mismas: `prev` se actualiza con el siguiente “mejor” y `ans` lleva registro del mejor encontrado entre los primeros n valores de la función. Con esa observación se puede iniciar el ciclo en $n = 0$, si se acepta que `curr` vale 0 al entrar en él. Ese valor inicial admite una lectura precisa: es $\phi(-1)$, la suma máxima de un subarreglo que termina inmediatamente antes del arreglo, que no puede ser otro que el subarreglo vacío. Con esa convención, la primera iteración calcula $A[0] \uparrow (A[0] + 0) = A[0]$, que es justamente $\phi(0)$. La

transformación del Algoritmo 3.4.4 bajo las observaciones anteriores resulta en el Algoritmo de Kadane, el cual se presenta en el Algoritmo 3.4.5.

Algoritmo 3.4.5: Algoritmo de Kadane

```

1 def kadane(A):
2     N, curr, ans = len(A), 0, 0
3     for n in range(N):
4         curr = max(A[n], curr+A[n])
5         ans = max(ans, curr)
6     return ans

```

Al igual que `mss_opt`, la función `kadane` resuelve el problema de suma máxima de un subarreglo en un arreglo usando tiempo lineal y espacio constante. El diseño de los invariantes para el ciclo en `kadane` y su corrección se proponen como ejercicio para el lector.

Ejercicios

1. Diseñe una función `mss_bf` en Python que implemente el diseño del Algoritmo 3.4.1. Corra la implementación con arreglos de números generados aleatoriamente de tamaños 10, 50, 100 y 250. ¿Cuál es la diferencia en tiempo entre las ejecuciones?
2. Después de presentar el Algoritmo 3.4.1, se propone una mejora para disminuir su complejidad de $O(N^3)$ a $O(N^2)$. Diseñe una función `mss_bf_opt` en Python que implemente este diseño optimizado. Corra la implementación con arreglos de números generados aleatoriamente de tamaños 10, 100, 1000 y 5000. ¿Cuál es la diferencia en tiempo entre las ejecuciones?
3. Diseñe una función `mss_dcc` en Python que implemente el diseño del Algoritmo 3.4.2. Corra la implementación con arreglos de números generados aleatoriamente de tamaños 10, 100, 1000, 10000 y 100000. ¿Cuál es la diferencia en tiempo entre las ejecuciones?
4. En la reformulación de la especificación con base en ϕ , se afirma que la expresión

$$(\uparrow n \mid 0 \leq n < N : \phi(n)) \uparrow 0.$$

permite considerar el subarreglo vacío como opción entre los subarreglos de A . Proponga un ejemplo de un arreglo $A[0..N)$ en el cual el resultado de la maximatoria sobre n sea negativo.

5. En el planteamiento recurrente de ϕ , se supone que $A[k..n]$ es un arreglo de suma máxima que inicia en algún $0 \leq k \leq n$ y termina en n . En el caso $k < n$

se indica que la suma $A[k] + \dots + A[n-1] + A[n]$ es igual a $\phi(n-1) + A[n]$. Ilustre esta afirmación con un ejemplo y elabore una justificación de por qué la igualdad es cierta (no es necesaria una demostración).

6. Elabore un ejemplo en el cual se exhiba la propiedad de solapamiento para ϕ . Explique brevemente su respuesta.
7. Diseñe un algoritmo con memorización para la función ϕ , incluyendo los predicados correspondientes a las condiciones que debe satisfacer el algoritmo. Demuestre que el algoritmo es correcto con respecto a la especificación dada, y estime las complejidades temporal y espacial.
8. Demuestre el Teorema 3.4.2.
9. Demuestre el Teorema 3.4.3.
10. Proponga invariantes para el Algoritmo de Kadane (Algoritmo 3.4.5) y demuestre que es correcto. También determine las complejidades temporal y espacial del algoritmo.
11. Modifique el Algoritmo de Kadane (Algoritmo 3.4.5) para que no se tenga en cuenta el arreglo vacío entre los subarreglos de A al calcular la suma máxima. Explique brevemente por qué funciona el algoritmo propuesto.
12. Modifique cualquiera de los algoritmos propuestos en esta sección que toman tiempo lineal para que, además de calcular la suma máxima de un subarreglo de $A[0..N]$, retorne k, n tales que el subarreglo $A[k..n]$ es de suma máxima (la complejidad del algoritmo resultante también debe ser lineal en función del tamaño del arreglo dado).

3.5. Subsecuencia creciente más larga

El problema de la subsecuencia creciente más larga consiste en determinar la longitud de la subsecuencia creciente de mayor tamaño en un arreglo de números dado. Antes de especificarlo es necesario precisar la noción de subsecuencia, que no debe confundirse con la de subarreglo usada en la Sección 3.4.

Definición 3.5.1: Subsecuencia

Sea $A[0..N]$ un arreglo de números. Una *subsecuencia* de $A[0..N]$ de longitud k , con $0 \leq k \leq N$, es una sucesión

$$A[i_0], A[i_1], \dots, A[i_{k-1}]$$

determinada por índices $0 \leq i_0 < i_1 < \dots < i_{k-1} < N$. La subsecuencia de longitud 0 se llama *vacía*. Cuando $k \neq 0$ se dice que la subsecuencia *termina* en i_{k-1} .

Una subsecuencia es *creciente* si y solo si

$$A[i_0] < A[i_1] < \cdots < A[i_{k-1}];$$

las subsecuencias de longitud 0 y 1 son crecientes por vacuidad.

Note que una subsecuencia queda determinada por un conjunto de índices y no por sus valores: dos subsecuencias distintas pueden tener los mismos elementos. El problema que se desea resolver es el siguiente.

Problema 3.5.1: Subsecuencia creciente más larga

Entrada: Un arreglo $A[0..N]$, con $N \geq 0$, de números.

Salida: Longitud de una subsecuencia creciente más larga de $A[0..N]$.

Se habla de *una* subsecuencia creciente más larga, y no de *la*, porque puede haber varias; lo que es único es su longitud.

Ejemplo 3.5.1

Considere el arreglo $A = [3, 1, 4, 1, 5, 9, 2, 6]$. La subsecuencia determinada por los índices 0, 2, 4, 5 es 3, 4, 5, 9 y es creciente de longitud 4. También lo son 1, 4, 5, 9 (índices 1, 2, 4, 5) y 1, 4, 5, 6 (índices 1, 2, 4, 7). Ninguna subsecuencia creciente de A tiene longitud 5, de modo que la respuesta para esta instancia es 4.

En contraste, la subsecuencia 3, 1, 4 (índices 0, 1, 2) no es creciente, y la sucesión 4, 3 no es subsecuencia de A porque sus índices no aparecen en orden.

Nota 3.5.1: Subsecuencias y subarreglos

Todo subarreglo de $A[0..N]$ es una subsecuencia –la que resulta de tomar índices consecutivos– pero no toda subsecuencia es un subarreglo. La diferencia se paga en la cantidad de objetos que hay que considerar: $A[0..N]$ tiene $N(N+1)/2 + 1$ subarreglos y 2^N subsecuencias, una por cada subconjunto de índices. Un algoritmo de fuerza bruta que enumere las subsecuencias para quedarse con la más larga que sea creciente toma, entonces, tiempo exponencial en el tamaño del arreglo.

La diferencia también se paga en el diseño, y esto es lo que hace que este problema no sea una variación menor del de la Sección 3.4. Allí, un

subarreglo de suma máxima que termina en n o bien es $A[n..n]$ o bien extiende un subarreglo que termina en $n - 1$: la contigüidad obliga a que el elemento anterior sea exactamente $A[n - 1]$ y por eso $\phi(n)$ dependía únicamente de $\phi(n - 1)$. En una subsecuencia creciente que termina en n , el elemento anterior a $A[n]$ puede ser cualquier $A[k]$ con $k < n$ y $A[k] < A[n]$. No hay un único candidato sino tantos como índices haya a la izquierda de n .

A diferencia del Problema 3.2.1, este es un problema de optimización, de modo que la propiedad que hay que verificar es la subestructura óptima de la Definición 3.1.1 y no solamente la subestructura general. El Problema 3.5.1 la exhibe, junto con la de solapamiento, así que se sigue la Metodología 3.3.1; el paso de especificación ya quedó cumplido con el enunciado del problema.

Función objetivo. Para $0 \leq n < N$, se define:

$\phi(n)$: “longitud de una subsecuencia creciente más larga de $A[0..N)$ que termina en n .”

La función ϕ tiene como dominio los índices de A y como co-dominio los enteros positivos. Note que $\phi(n) \geq 1$ para cualquier índice n , pues la subsecuencia determinada por el único índice n es creciente y termina en n .

Reformulación de la especificación. Toda subsecuencia creciente no vacía de $A[0..N)$ termina en alguno de sus índices. Luego, la salida del problema se puede escribir como la maximatoria:

$$(\uparrow n \mid 0 \leq n < N : \phi(n)) \uparrow 0.$$

El 0 adicional cubre el caso en el cual el arreglo dado es vacío, pues allí la maximatoria se toma sobre un rango vacío. A diferencia de lo que sucede en la Sección 3.4, ese es el *único* caso en el que el 0 interviene: como $\phi(n) \geq 1$ siempre, la maximatoria nunca es negativa.

Vale la pena señalar una diferencia con el problema del morral. Allí, la función objetivo evaluada en los parámetros de la instancia, $\phi(N, X)$, era ya la salida del problema. Aquí la función objetivo es apenas un ingrediente: la maximatoria externa es indispensable y no se puede prescindir de ella. Esta es la misma situación de la Sección 3.4 y es la razón por la cual la salida se reformula en lugar de simplemente evaluarse.

Planteamiento recurrente. Sea $0 \leq n < N$ y considere una subsecuencia creciente más larga de $A[0..N)$ que termina en n ; sea k el índice del elemento inmediatamente anterior a $A[n]$ en dicha subsecuencia, si es que existe. Hay dos posibilidades. Si la subsecuencia tiene longitud 1, entonces no hay tal k y la subsecuencia es simplemente $A[n]$. Si tiene longitud mayor, entonces $k < n$ y, por ser creciente, $A[k] < A[n]$; además, al quitar $A[n]$ queda una subsecuencia creciente

que termina en k , la cual debe ser lo más larga posible (de lo contrario, se podría alargar la original –¿por qué?). Es la propiedad de subestructura óptima.

Ahora bien, no se sabe de antemano cuál de los índices a la izquierda de n juega el papel de k . Se apuesta a todos los caballos: se consideran todos los k admisibles y se prefiere el que produzca la mayor longitud. Con base en este análisis se propone la siguiente definición para ϕ , con $0 \leq n < N$:

$$\phi(n) = 1 + (\uparrow k \mid 0 \leq k < n \wedge A[k] < A[n] : \phi(k)).$$

Nota 3.5.2: Una recurrencia sin caso base

La definición de ϕ consta de una sola ecuación y no distingue casos. Esto puede sorprender, pues todas las funciones objetivo vistas hasta ahora tienen al menos un caso base explícito. Lo que sucede es que el caso base está escondido en la convención de que una maximatoria sobre un rango vacío vale 0, la misma convención que permitió obtener el Algoritmo de Kadane en la Sección 3.4. En efecto, cuando $n = 0$ el rango es vacío y se obtiene $\phi(0) = 1 + 0 = 1$, que es la respuesta correcta.

Lo mismo ocurre, y esto es lo interesante, cuando $n \neq 0$ pero ningún índice a la izquierda de n satisface $A[k] < A[n]$: el rango vuelve a ser vacío y $\phi(n) = 1$. Es decir, una única ecuación cubre a la vez el caso base y un caso recurrente degenerado. Escribir $\phi(0) = 1$ como caso aparte no sería incorrecto, pero sí redundante.

La recurrencia tampoco es ambigua ni incompleta: para cada n el lado derecho está definido en términos de valores $\phi(k)$ con $k < n$, y la definición se puede leer como una definición por inducción fuerte sobre n .

Correctitud de la función objetivo. Como siempre, la observación intuitiva debe ser respaldada por un razonamiento formal.

Teorema 3.5.1

Sea $0 \leq n < N$. Entonces $\phi(n)$ es la longitud de una subsecuencia creciente más larga de $A[0..N)$ que termina en n .

Demostración

Sea $L(n)$ la longitud de una subsecuencia creciente más larga de $A[0..N)$ que termina en n ; note que $L(n) \geq 1$ y que el objetivo es demostrar $\phi(n) = L(n)$.

Se procede por inducción fuerte sobre n , suponiendo $\phi(k) = L(k)$ para todo $0 \leq k < n$. Se demuestran las dos desigualdades por separado.

- *Descomposición:* $L(n) \leq \phi(n)$. Sea $A[i_0], \dots, A[i_{m-1}]$ una subsecuencia creciente de longitud $m = L(n)$ que termina en n (i.e., $i_{m-1} = n$). Si $m = 1$, entonces $L(n) = 1 \leq \phi(n)$ dado que la maximatoria en la definición de ϕ nunca es negativa. Si $m \geq 2$, sea $k = i_{m-2}$. Entonces $k < n$ y $A[k] < A[n]$ por ser creciente la subsecuencia, luego k está en el rango de la maximatoria. Además, $A[i_0], \dots, A[i_{m-2}]$ es una subsecuencia creciente de longitud $m - 1$ que termina en k , de donde $L(k) \geq m - 1$. Entonces se tiene:

$$\begin{aligned}
 \phi(n) &\geq 1 + \phi(k) && (k \text{ está en el rango}) \\
 &= 1 + L(k) && (\text{hipótesis inductiva}) \\
 &\geq 1 + (m - 1) && (\text{observación anterior}) \\
 &= L(n) && (\text{aritmética}).
 \end{aligned}$$

- *Composición:* $\phi(n) \leq L(n)$. Si el rango de la maximatoria es vacío, entonces $\phi(n) = 1 \leq L(n)$. De lo contrario, como el rango es finito y no vacío, la maximatoria se alcanza en algún k con $k < n$ y $A[k] < A[n]$; es decir, $\phi(n) = 1 + \phi(k)$. Por hipótesis inductiva, $\phi(k) = L(k)$ y existe una subsecuencia creciente de longitud $L(k)$ que termina en k . Al extenderla con $A[n]$ se obtiene una subsecuencia de longitud $L(k) + 1$: es subsecuencia porque $k < n$ y es creciente porque $A[k] < A[n]$. Como termina en n , se concluye que $L(n) \geq L(k) + 1 = \phi(n)$.

En cualquier caso, $\phi(n) = L(n)$.

La propiedad de solapamiento es evidente en esta formulación: el cálculo de $\phi(n)$ invoca $\phi(k)$ para varios índices k y cada uno de esos llamados invoca a su vez índices menores. Encontrar instancias concretas en las cuales un mismo $\phi(k)$ se requiere desde dos llamados distintos se propone como ejercicio al lector.

¿Memorización o tabulación? Se opta por una tabulación. El diagrama de necesidades correspondiente se presenta en la Figura 5 y tiene una forma que no había aparecido antes en este capítulo. En el diagrama se usa la memoria de tabulación $tab[0..N]$ para almacenar los valores de la función y una variable n para indicar el índice de la tabla que se debe calcular en un momento dado. Hasta ahora, cada celda dependía de una cantidad fija de celdas previamente calculadas: una en la Sección 3.4 y dos en la Sección 3.6. Aquí, en cambio, $tab[n]$ puede depender de cualquiera de las n celdas que lo preceden. Esto tiene dos consecuencias: llenar la

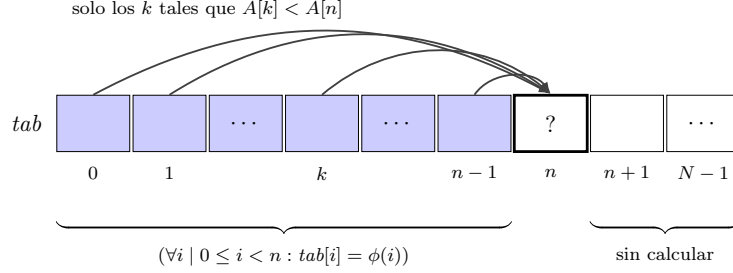


Figura 5. Diagrama de necesidades para la tabulación de ϕ en el problema de la subsecuencia creciente más larga. El valor $tab[n]$ depende de todo el prefijo $tab[0..n)$ y no de una cantidad acotada de celdas; cuáles de ellas intervienen efectivamente depende de los valores de A y no solo del índice n .

celda n ya no toma tiempo constante sino tiempo $O(n)$ y la reducción del espacio deja de ser posible, como se discute más adelante.

La estrategia sigue siendo llenar tab de izquierda a derecha. Los invariantes del ciclo principal son:

$$\begin{aligned} P_0 : & \quad (\forall i \mid 0 \leq i < n : tab[i] = \phi(i)). \\ P_1 : & \quad 0 \leq n \leq N. \\ P_2 : & \quad ans = (\uparrow i \mid 0 \leq i < n : \phi(i)) \uparrow 0. \end{aligned}$$

Los invariantes P_0 y P_1 son los mismos de la Sección 3.4: el subarreglo $tab[0..n)$ tiene, en orden, los primeros n valores de ϕ y n toma valores entre 0 y N . El invariante P_2 acumula la maximatoria de la reformulación sobre los valores ya calculados; a diferencia de lo hecho en la Sección 3.4, aquí conviene calcularla en el mismo ciclo y no en una segunda pasada, pues el ciclo ya no es de tiempo constante por iteración y una pasada adicional no cambia el orden del algoritmo.

El cálculo de $tab[n]$ requiere, a su vez, un ciclo que recorra los índices a la izquierda de n . Es el primer ciclo anidado del capítulo, y como tal necesita sus propios invariantes. Para un n fijo, y con k la variable del ciclo interno:

$$\begin{aligned} Q_0 : & \quad tab[n] = 1 + (\uparrow i \mid 0 \leq i < k \wedge A[i] < A[n] : \phi(i)). \\ Q_1 : & \quad 0 \leq k \leq n. \end{aligned}$$

El invariante Q_0 indica que $tab[n]$ contiene, en todo momento, el valor que tendría $\phi(n)$ si el rango de su maximatoria se restringiera a los índices menores que k . Cuando $k = 0$ el rango es vacío y $tab[n] = 1$, que es precisamente el valor con el cual se inicializa la celda; cuando $k = n$ el rango es el de la definición de ϕ y, por lo tanto, $tab[n] = \phi(n)$. Con base en este diseño se propone el Algoritmo 3.5.1.

Implementación. En el Algoritmo 3.5.1 se presenta una tabulación para la función objetivo. Note las finalidades de los dos ciclos y cómo los invariantes permiten entender cada una de ellas.

Algoritmo 3.5.1: Tabulación para ϕ

```

1 def lis(A):
2     N, ans = len(A), 0
3     tab = [ None for _ in range(N) ]
4     #  $P_0 \wedge P_1 \wedge P_2$ 
5     for n in range(N):
6         tab[n] = 1
7         #  $Q_0 \wedge Q_1$ 
8         for k in range(n):
9             if A[k] < A[n]: tab[n] = max(tab[n], tab[k]+1)
10        ans = max(ans, tab[n])
11    return ans

```

A diferencia de las funciones `mss` y `mss_opt` de la Sección 3.4, la función `lis` no requiere tratar de manera especial el caso en el cual el arreglo dado es vacío. Cuando $N = 0$, ninguno de los dos ciclos se ejecuta y se retorna el valor inicial de `ans`, que es 0: exactamente lo que indica la reformulación de la especificación.

A continuación se ilustra el uso de la función con algunas instancias:

```

1 >>> print(lis([3, 1, 4, 1, 5, 9, 2, 6]))
2 4
3 >>> print(lis([]))
4 0
5 >>> print(lis([7, 7, 7]))
6 1
7 >>> print(lis([5, 4, 3, 2, 1]))
8 1
9 >>> print(lis([1, 2, 3, 4, 5]))
10 5

```

La tercera instancia es un buen recordatorio de que la subsecuencia debe ser creciente en sentido estricto: en `[7, 7, 7]` ninguna subsecuencia de longitud 2 lo es.

Análisis. El Teorema 3.5.2 cierra el diseño de la solución del problema con el criterio de correctitud y de análisis asintótico.

Teorema 3.5.2

El llamado $\text{lis}(A)$, con $\text{len}(A) = N$:

1. Calcula la longitud de una subsecuencia creciente más larga de $A[0..N)$.
2. Toma tiempo $O(N^2)$.
3. Usa espacio $O(N)$.

Demostración

Se propone como ejercicio al lector.

Reducción del espacio. El paso opcional de la Metodología 3.3.1 invita a explorar si la memoria de tabulación se puede reducir. En las dos secciones anteriores la respuesta fue afirmativa y en ambos casos el argumento se leyó directamente en el diagrama de necesidades: bastaba con conservar una celda en la Sección 3.4 y una columna en la Sección 3.6. Aquí la respuesta es negativa, y conviene entender por qué.

Nota 3.5.3: Por qué la tabulación no se puede reducir

Una reducción del tipo de las secciones anteriores consiste en conservar solo una ventana de tamaño fijo de la tabulación. Para que eso funcione debe existir una cota w , independiente de la instancia, tal que $\phi(n)$ solo dependa de los valores $\phi(k)$ con $n - k \leq w$. En este problema tal cota no existe.

Considere, para $N \geq 4$, el arreglo

$$A = [1, 2, 3, \underbrace{0, 0, \dots, 0}_{N-4}, 4].$$

Se tiene $\phi(0) = 1$, $\phi(1) = 2$ y $\phi(2) = 3$; cada una de las posiciones ocupadas por un 0 tiene $\phi(n) = 1$; y para la última posición, $\phi(N-1) = 1 + 3 = 4$, valor que se alcanza únicamente en el índice 2. Es decir, el cálculo de la última celda necesita el valor de una celda que está a distancia $N-3$, la cual puede hacerse tan grande como se quiera.

Conviene precisar qué es exactamente lo que falla, porque es fácil equivocarse de criterio. Lo que impide la reducción no es que las dependencias dependan de los datos de la instancia. En este problema dependen de ellos —el rango de la maximatoria está restringido por la condición $A[k] < A[n]$ — pero eso, por sí solo, no sería obstáculo. La Sección 3.7 presenta un problema en el cual cuáles celdas se consultan también depende de los datos y allí la reducción del espacio funciona sin dificultad.

Lo que decide el asunto es si la ventana está acotada. En la Sección 3.4 cada celda depende de una celda anterior, en la Sección 3.6 de dos y en la Sección 3.7 de tres, siempre vecinas y siempre a distancia acotada, sin importar cuál de ellas intervenga en cada caso concreto. Aquí la ventana es todo el prefijo.

Esto no significa que el espacio $O(N)$ sea desperdiciado. Significa que $tab[0..N)$ es, en este problema, información que el algoritmo realmente necesita conservar completa.

Nota 3.5.4: Una solución en $O(N \log N)$

La cota $O(N^2)$ del Algoritmo 3.5.1 no es la mejor posible. Existe un algoritmo que resuelve el Problema 3.5.1 en tiempo $O(N \log N)$ y espacio $O(N)$, basado en una idea distinta: en lugar de calcular para cada índice la longitud de la mejor subsecuencia que termina allí, se mantiene, para cada longitud ℓ , el menor valor con el cual puede terminar una subsecuencia creciente de longitud ℓ dentro del prefijo procesado. Ese registro es siempre una sucesión creciente, lo cual permite actualizarlo con una búsqueda binaria por cada elemento del arreglo.

Ese algoritmo suele describirse como una combinación de una decisión voraz con una búsqueda binaria y no como programación dinámica en el sentido de este capítulo: no computa una función recurrente evitando recalculer subinstancias. Su diseño se propone como ejercicio.

Vale la pena anotar, por último, que este problema ha recibido mucha más atención de la que su enunciado sugiere, por razones que poco tienen que ver con la algorítmica. En las notas del final del capítulo se dan referencias, incluyendo un libro dedicado exclusivamente a él.

Ejercicios

1. Demuestre que un arreglo $A[0..N]$ tiene exactamente 2^N subsecuencias y $N(N+1)/2 + 1$ subarreglos. ¿Para qué valores de N coinciden estas dos cantidades?
2. En el planteamiento recurrente de ϕ se afirma que, al quitar $A[n]$ de una subsecuencia creciente más larga que termina en n , la subsecuencia que queda debe ser lo más larga posible entre las que terminan en k . Justifique esta afirmación.
3. Elabore un ejemplo en el cual se exhiba la propiedad de solapamiento para ϕ . Explique brevemente su respuesta.
4. Demuestre que el ciclo interno del Algoritmo 3.5.1 satisface los invariantes Q_0 y Q_1 .
5. Demuestre el Teorema 3.5.2.
6. Una subsecuencia es *no decreciente* si cada uno de sus elementos es menor o igual que el siguiente.
 - a) Especifique el problema de determinar la longitud de una subsecuencia no decreciente más larga de $A[0..N]$.
 - b) Modifique la definición de ϕ y el Algoritmo 3.5.1 para resolverlo. ¿Qué responde su algoritmo para el arreglo $[7, 7, 7]$?
7. Considere ahora el problema de determinar la longitud de una subsecuencia *decreciente* más larga de $A[0..N]$.
 - a) Resuélvalo modificando la definición de ϕ .
 - b) Resuélvalo sin modificar el Algoritmo 3.5.1, es decir, transformando solo el arreglo de entrada. Justifique que la transformación es correcta.
8. Diseñe un algoritmo con memorización para la función ϕ instanciando el Algoritmo 3.3.1, e incluya las condiciones de representación que debe satisfacer la memoria compartida. Demuestre que el algoritmo es correcto y estime sus complejidades temporal y espacial.
9. Modifique el Algoritmo 3.5.1 para que, además de calcular la longitud de una subsecuencia creciente más larga de $A[0..N]$, retorne los índices que la determinan. La complejidad temporal del algoritmo resultante debe seguir siendo $O(N^2)$. ¿Cuál es su complejidad espacial?
10. En la Nota 3.5.3 se exhibe, para cada $N \geq 4$, un arreglo en el cual el cálculo de $\phi(N-1)$ requiere el valor de $\phi(2)$. Construya una familia de arreglos en la cual el cálculo de $\phi(N-1)$ requiera los valores de $\phi(k)$ para *todos* los índices $k < N-1$.
11. Modifique el Algoritmo 3.5.1 para que, en lugar de la longitud de una subsecuencia creciente más larga, calcule la *cantidad* de subsecuencias crecientes de esa longitud. Formule los invariantes correspondientes y justifique el diseño.

12. Resuelva el Ejercicio 4 de la Sección 3.1, en el cual se desea determinar la mayor cantidad de días, no necesariamente consecutivos, en los cuales el precio de una acción aumentó a lo largo de N días. Explique en qué sentido ese problema es una instancia del Problema 3.5.1.
13. 🌈 Diseñe el algoritmo de tiempo $O(N \log N)$ esbozado en la Nota 3.5.4. Formule los invariantes del ciclo, demuestre que el registro que se mantiene es siempre una sucesión creciente y concluya que el algoritmo es correcto.

3.6. El problema del morral

El problema del morral es un problema de optimización combinatoria.

Nota 3.6.1: El nombre del problema

El nombre del problema del morral (en inglés, *knapsack* o *KS*) se deriva de una analogía en la cual se desea escoger de una colección de elementos para cargar en un morral que tiene una restricción de peso (o volumen). El objetivo es escoger dicha combinación de elementos maximizando su valor total sin exceder la capacidad del morral.

Este es un problema que surge naturalmente en situaciones de asignación de recursos en las cuales quienes toman las decisiones deben escoger proyectos o tareas indivisibles bajo una restricción presupuestal o de tiempo.

De forma general, dada una colección de elementos, cada uno con un peso y un valor asociados, el problema consiste en determinar cuál es la suma máxima de valores que se puede obtener al tomar algunos de los elementos sin que su suma exceda una restricción de peso global.

Problema 3.6.1: Problema del morral

Entrada: Arreglos $V[0..N)$ y $W[0..N)$, con $N \geq 0$, de valores (números no negativos) y de pesos (números enteros no negativos), y un número entero $X \geq 0$.

Salida: Suma máxima de elementos con valores en $V[0..N)$ cuyos pesos en $W[0..N)$ no exceden X .

La condición de salida del problema supone que el valor y el peso del i -ésimo elemento están dados por $V[i]$ y $W[i]$, respectivamente. La naturaleza combinatoria del problema del morral se revela al notar que se requiere una exploración sobre los

subconjuntos de índices de $V[0..N]$. Para cualquier $S \subseteq \{0, \dots, N-1\}$, se definen las funciones *value* y *weight*:

$$\text{value}(S) = (+i \mid i \in S : V[i]) \quad \text{y} \quad \text{weight}(S) = (+i \mid i \in S : W[i]).$$

La expresión $\text{value}(S)$ representa el valor total de los elementos indexados por S , mientras que $\text{weight}(S)$ el peso total de los elementos indexados por S . Así, la salida del problema del morral corresponde a la expresión:

$$(\uparrow S \mid S \subseteq \{0, \dots, N-1\} \wedge \text{weight}(S) \leq X : \text{value}(S)).$$

Como hay 2^N subconjuntos S de $\{0, \dots, N-1\}$ (¿por qué?), se dice que el problema del morral es de naturaleza combinatoria, pues debe considerar todos aquellos subconjuntos S para determinar la mejor combinación posible bajo la restricción dada. Debe ser claro, entonces, que un algoritmo de fuerza bruta para resolver el problema del morral tomará tiempo exponencial en la cantidad de elementos entre los cuales se elige.

Ejemplo 3.6.1

Considere 4 elementos con valores y pesos dados por, respectivamente:

$$V = [4, 6, 5, 1] \quad \text{y} \quad W = [3, 5, 5, 2].$$

- Con $X = 15$, la mejor opción es llevar todos los elementos, obteniendo como valor total 16 y peso total 15.
- Con $X = 11$, el valor máximo es 11. Esto resulta de dos configuraciones cuyo peso total es 10: llevar los elementos 0, 1, 3 o los elementos 1, 2.
- Con $X = 5$, la mejor opción es llevar el elemento 1, con valor total 6 y peso total 5.
- Con $X = 1$, la mejor (y única) opción es no echar nada al morral.

Este es un problema de optimización, de modo que la propiedad que hay que verificar es la subestructura óptima de la Definición 3.1.1. El problema del morral la exhibe, junto con la de solapamiento (¿por qué?), así que se seguirá la Metodología 3.3.1; el paso de especificación ya quedó cumplido con el Problema 3.6.1.

Función objetivo. Hay dos variables que permiten generalizar la salida del problema: N y X . Por una parte, la cantidad de elementos disponibles se puede hacer variar para considerar diferentes prefijos de $V[0..N]$ y $W[0..N]$ (también se pueden hacer variar los sufijos, resultando en un planteamiento similar). Por otra parte, el peso disponible en el morral cambiaría cada vez que se decide escoger un elemento. Con base en estas observaciones, se propone la siguiente función objetivo

para $0 \leq n \leq N$ y $0 \leq x \leq X$:

$\phi(n, x)$: “suma máxima de elementos con valores en $V[0..n)$
cuyos pesos en $W[0..n)$ no exceden x .”

Reformulación de la especificación. El objetivo del problema es calcular $\phi(N, X)$.

Planteamiento recurrente. La variable n se toma como “pivote”: para razonar por casos, estos se estructuran primero con base en condiciones sobre n y, de ser necesario, luego se dividen en subcasos con base en condiciones sobre la segunda variable x . Es importante tener en cuenta que $\phi(n, x)$ debe ser definida para todos los valores de n y x que satisfacen $0 \leq n \leq N$ y $0 \leq x \leq X$.

Se procede por casos sobre n y x :

- No hay elementos para escoger (i.e., $n = 0$). Luego, sin importar la capacidad x del morral, lo máximo (y único) que se puede obtener es 0.
- Hay al menos un elemento para escoger (i.e., $n > 0$). Entonces están las opciones de que el elemento $n-1$ (este es el n -ésimo elemento) pese más que la capacidad x del morral o que no:
 - si $W[n-1] > x$, no hay más opción que ignorar ese elemento y escoger lo mejor posible con los elementos que restan, sin modificar la capacidad del morral; es decir, se recurre con $n-1$ y x .
 - si $W[n-1] \leq x$, entonces están las opciones de adicionar el elemento al morral o no. Como el objetivo es tomar la mejor decisión, se apuesta a los dos caballos. No escogerlo resulta en la misma situación del caso anterior (i.e., se recurre con $n-1$ y x). Escogerlo tiene el efecto de disminuir la capacidad del morral a $x - W[n-1]$ y de aumentar el beneficio en $V[n-1]$ unidades; es decir, se recurre con $n-1$ y $x - W[n-1]$, y se acumula $V[n-1]$ beneficio.

En cada uno de los casos recurrentes se respeta las condiciones $0 \leq n \leq N$ y $0 \leq x \leq X$ (¿por qué?), lo cual es clave para contar con una buena definición de ϕ . La naturaleza exponencial de la búsqueda de “la” mejor elección de elementos se ve en el segundo caso recurrente: allí se apuesta a llevar y a no llevar el elemento, es decir, se consideran todos los subconjuntos de índices pendientes sin acumular el n -ésimo elemento y todos los subconjuntos de índices pendientes acumulando el n -ésimo elemento.

La definición formal de ϕ se presenta para $0 \leq n \leq N$ y $0 \leq x \leq X$:

$$\phi(n, x) = \begin{cases} 0 & , \text{ si } n = 0, \\ \phi(n-1, x) & , \text{ si } n \neq 0 \wedge W[n-1] > x, \\ \phi(n-1, x) \uparrow \\ \quad (\phi(n-1, x - W[n-1]) + V[n-1]) & , \text{ si } n \neq 0 \wedge W[n-1] \leq x. \end{cases}$$

Correctitud de la función objetivo. Es necesario demostrar que la definición de ϕ permite resolver el problema del morral.

Teorema 3.6.1

Si $0 \leq n \leq N$ y $0 \leq x \leq X$, entonces $\phi(n, x)$ es la suma máxima de elementos con valores en $V[0..n]$ cuyos pesos en $W[0..n]$ no exceden x .

Demostración

Diga que un conjunto $S \subseteq \{0, \dots, n-1\}$ es *admisibile para* (n, x) si $\text{weight}(S) \leq x$, y sea

$$L(n, x) = (\uparrow S \mid S \text{ es admisibile para } (n, x) : \text{value}(S)).$$

Note que $L(n, x)$ siempre existe, pues $\emptyset$ es admisibile para cualquier (n, x) y hay una cantidad finita de conjuntos. El objetivo es demostrar que $\phi(n, x) = L(n, x)$, y se procede por inducción sobre n .

Caso base $n = 0$: el único subconjunto de $\emptyset$ es $\emptyset$, cuyo valor es 0. Luego $L(0, x) = 0 = \phi(0, x)$ para cualquier x .

Caso inductivo $n \neq 0$: suponga que $\phi(n-1, y) = L(n-1, y)$ para todo $0 \leq y \leq X$. Se distinguen los dos casos de la definición de ϕ y, en cada uno, se demuestran las dos desigualdades.

- Caso $W[n-1] > x$. En este caso el elemento $n-1$ no cabe en el morral, de modo que ningún conjunto admisibile para (n, x) lo contiene: los conjuntos admisibles para (n, x) son exactamente los admisibles para $(n-1, x)$. Luego $L(n, x) = L(n-1, x)$ y, por hipótesis inductiva, $L(n, x) = \phi(n-1, x) = \phi(n, x)$.
- Caso $W[n-1] \leq x$. Se demuestran las dos desigualdades por separado.
 - *Composición*: $\phi(n, x) \leq L(n, x)$. Basta ver que los dos argumentos del máximo son valores de conjuntos admisibles para (n, x) . Por hipótesis inductiva, $\phi(n-1, x) = L(n-1, x) = \text{value}(S_1)$ para algún S_1 admisibile para $(n-1, x)$; como $S_1 \subseteq \{0, \dots, n-2\}$, también es admisibile para (n, x) . De nuevo por hipótesis inductiva, $\phi(n-1, x - W[n-1]) = \text{value}(S_2)$ para algún S_2 admisibile para $(n-1, x - W[n-1])$; entonces $S_2 \cup \{n-1\}$ es admisibile para (n, x) , pues su peso es $\text{weight}(S_2) + W[n-1] \leq x$, y su valor es $\text{value}(S_2) + V[n-1]$. Como los dos argumentos son valores de conjuntos admisibles, ninguno excede a $L(n, x)$ y tampoco lo hace su máximo.

- *Descomposición:* $\phi(n, x) \geq L(n, x)$. Tome S admisible para (n, x) con $value(S) = L(n, x)$. Si $n-1 \notin S$, entonces S es admisible para $(n-1, x)$ y, por hipótesis inductiva, $L(n, x) = value(S) \leq L(n-1, x) = \phi(n-1, x) \leq \phi(n, x)$. Si $n-1 \in S$, entonces $S \setminus \{n-1\}$ es admisible para $(n-1, x-W[n-1])$, pues su peso es $weight(S) - W[n-1] \leq x - W[n-1]$. Entonces se tiene:

$$\begin{aligned}
L(n, x) &= value(S \setminus \{n-1\}) + V[n-1] && (n-1 \in S) \\
&\leq L(n-1, x - W[n-1]) + V[n-1] && (\text{admisibilidad}) \\
&= \phi(n-1, x - W[n-1]) + V[n-1] && (\text{hipótesis inductiva}) \\
&\leq \phi(n, x) && (\text{uno de los argumentos}).
\end{aligned}$$

En cualquier caso, $\phi(n, x) = L(n, x)$.

Note que la demostración nunca necesitó preguntarse cuándo la solución óptima es vacía. La pregunta parece natural –si no se lleva nada, ¿qué se puede concluir?– pero conduce a un callejón: que $\emptyset$ sea óptimo no implica que no haya elementos ni que todos pesen demasiado, porque puede haber elementos que quepan y valgan 0. Separar el argumento en las dos mitades evita el problema, porque en ninguna de las dos hace falta caracterizar la solución óptima: en la de composición se construyen conjuntos admisibles y en la de descomposición se parte uno cualquiera.

La demostración puede parecer larga, pero su estructura es la misma que la del Teorema 3.4.1 y la del Teorema 3.5.1. Lo único que la alarga es que hay dos casos en la definición de ϕ y que, en el segundo, el máximo tiene dos argumentos. Vale la pena señalar que lo que se demuestra es que ϕ calcula el *valor* óptimo y no que identifique una configuración que lo alcance; puede haber más de una, como lo muestra el Ejemplo 3.6.1 con $X = 11$.

¿Memorización o tabulación? Se diseñará una tabulación para implementar eficientemente ϕ con base en el diagrama de necesidades de la Figura 6. La tabulación se implementa con una matriz $tab[0..N][0..X]$ con $N + 1$ columnas y $X + 1$ filas. El objetivo es calcular el valor correspondiente a $tab[N][X]$ dado que el problema se resuelve con $\phi(N, X)$. Cuando $n = 0$, sin importar el valor de x , el valor $\phi(n, x)$ es 0. Por ello, la primera columna de la matriz tiene únicamente el valor 0. Para calcular $\phi(n, x)$, en el primer caso recurrente se depende de $\phi(n-1, x)$ y en el segundo caso recurrente además de $\phi(n-1, x - W[n-1])$. Por ello, en la matriz se identifican dos celdas de la columna $n-1$: cada una corresponde a cada uno de los llamados recurrentes, para los dos posibles valores de la capacidad restante en el morral. Estas observaciones indican que se puede llenar la tabulación por columnas,

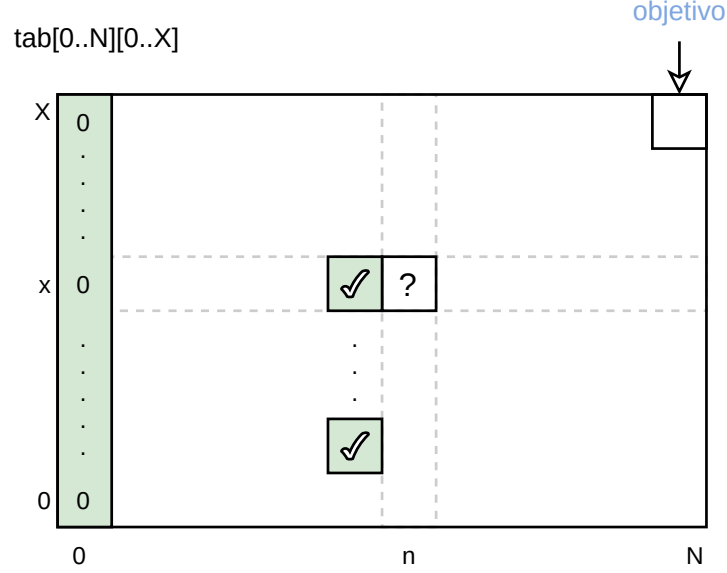


Figura 6. Diagrama de necesidades para la tabulación de ϕ en el problema del morral. El valor $tab[n][x]$ depende de dos celdas de la columna $n - 1$, una por cada llamado recurrente.

comenzando por la columna $n = 1$ (esta no es la única forma de hacerlo, ¿cierto?). Esta decisión se ve reflejada en los siguientes invariantes:

$$P_0 : (\forall i, j \mid 0 \leq i < n \wedge 0 \leq j \leq X : tab[i][j] = \phi(i, j)).$$

$$P_1 : (\forall j \mid 0 \leq j < x : tab[n][j] = \phi(n, j)).$$

$$P_2 : 0 \leq n \leq N + 1.$$

$$P_3 : 0 \leq x \leq X + 1.$$

El invariante P_0 indica que todas las columnas a la izquierda de la columna n han sido llenadas con los valores correspondientes de ϕ . El invariante P_1 especifica que los valores en la columna n , por debajo de la fila x , corresponden a los valores de $\phi(n, \cdot)$. Los invariantes P_2 y P_3 son de apoyo: el primero indica que los valores de n varían entre 0 y $N + 1$, inclusive; el segundo, que los valores de x varían entre 0 y $X + 1$, inclusive. Los dos invariantes de apoyo sugieren una forma de llenar la tabla: por columnas, de izquierda a derecha, hasta llenar la columna N . Visualmente, esta decisión se puede incluir en el diagrama de necesidades, como lo muestra la Figura 7.

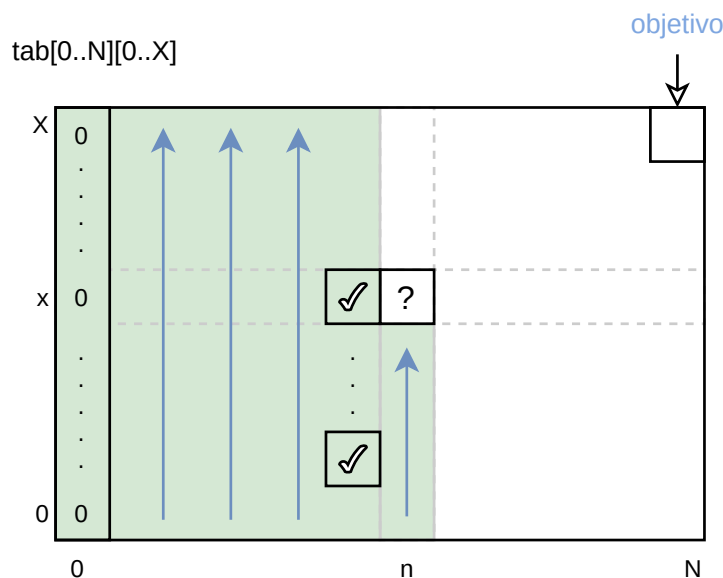


Figura 7. Orden de llenado de la tabulación en el problema del morral: por columnas, de izquierda a derecha.

Para implementar el algoritmo de tabulación, basta con seguir los invariantes y tener clara la imagen mental de ellos en el diagrama de necesidades. La idea es entonces, una vez creada la tabla e inicializada su primera columna, iniciar a llenarla desde la columna $n = 1$. Esto se hará como con las máquinas de escribir: se llena una columna, se procesa la siguiente. Intencionalmente se permite que la variable x llegue a $X + 1$, aunque esta fila no haga parte de la tabla: con $x = X + 1$ se sabe que se ha llenado la columna actual y se procede a iniciar la siguiente. Algo similar sucede con n : se habrá llenado la tabla por completo cuando se tenga $n = N + 1$.

Implementación. El Algoritmo 3.6.1 presenta la tabulación diseñada para la función ϕ .

Algoritmo 3.6.1: Tabulación para el problema del morral

```

1 def ks_tab(V, W, X):
2     N = len(V)
3     tab = [ [ None for _ in range(X+1) ] for _ in range(N+1) ]
4     for x in range(X+1): tab[0][x] = 0
5     n,x = 1,0
6     #  $P_0 \wedge P_1 \wedge P_2 \wedge P_3$ 
7     while n!=N+1:
8         if x==X+1: n,x = n+1,0

```



```

9     else:
10         if x < W[n-1]: tab[n][x] = tab[n-1][x]
11         else: tab[n][x] = max(tab[n-1][x], tab[n-1][x-W[n-1]]+V[n-1])
12         x += 1
13     return tab[N][X]

```

Análisis. La demostración de la corrección del algoritmo es rutina con base en los invariantes dados y la especificación del problema. La complejidad espacial está dominada por el tamaño de la tabla, y la temporal por su creación y actualización. Antes de introducir formalmente estos resultados, se presenta un ejemplo de la ejecución del algoritmo.

```

1 >>> V = [4, 6, 5, 1]
2 >>> W = [3, 5, 5, 2]
3 >>> print(ks_tab(V, W, 15))
4 16
5 >>> print(ks_tab(V, W, 11))
6 11
7 >>> print(ks_tab(V, W, 5))
8 6
9 >>> print(ks_tab(V, W, 1))
10 0

```

Teorema 3.6.2

El llamado `ks_tab(V, W, X)`, con $\text{len}(V) = \text{len}(W) = N$:

1. Calcula la suma máxima de elementos con valores en $V[0..N)$ cuyos pesos en $W[0..N)$ no exceden X .
2. Toma tiempo $O(NX)$.
3. Usa espacio $O(NX)$.

Demostración

Se propone como ejercicio al lector.

La complejidad temporal del algoritmo propuesto es polinomial en función de N y X . Sin embargo, es importante tener en cuenta que la cota X *no* es polinomial en función de N ; a esto se le llama tiempo pseudo-polinomial. Por ejemplo, si $X = 2^{32}$, basta con 32 bits para representar X . Sin embargo, el algoritmo propuesto usa 2^{32}

filas para construir y llenar la tabla. Por ello, es usual indicar que la complejidad temporal (y espacial) de la función `ks_tab` es $O(N \cdot 2^{\text{bits en } X})$.

3.6.1. Primera optimización. El diagrama de necesidades sugiere que el diseño de la tabulación se puede optimizar. En realidad, el cálculo de la columna $n \geq 1$ solo depende de valores en la columna $n - 1$. Además, al iniciar con la columna $n = 1$, la columna $n = 0$ está completamente inicializada. Esta observación permite hacer una optimización importante en el espacio usado por la tabulación.

El diagrama de necesidades de la Figura 8 reduce la tabulación a dos columnas: una para el presente (que corresponde a la columna n) y otra para el pasado inmediatamente anterior (que corresponde a la columna $n - 1$). La noción de “pre-

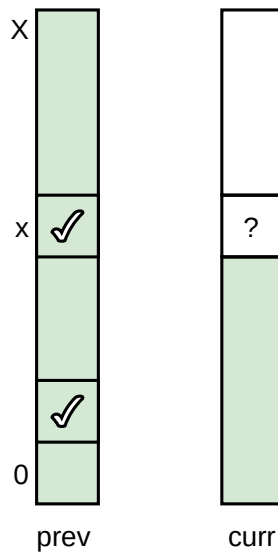


Figura 8. Diagrama de necesidades de la primera optimización: la tabulación se reduce a la columna del presente y la del pasado inmediato.

sente” se representa con el arreglo *curr* y la de “pasado inmediato” con *prev*. Un reto surge ante la pregunta de qué hacer cuando *curr* haya sido llenado. Note que en el caso en el cual exista la necesidad de seguir procesando la tabulación (i.e., $n \neq N + 1$), este arreglo pasará a ser el pasado inmediato y no habrá uso para los valores almacenados en *prev*. Por ello, y para evitar la creación de memoria adicional y copias innecesarias, se puede decidir reutilizar *prev* para el rol de nuevo presente. Para facilitar esta decisión en el código final, se prefiere que la tabulación sea una matriz de dos columnas, cada una de ellas de tamaño $X + 1$, y usar dos

variables (abusando la notación, *prev* y *curr*) que alternen opuestamente con los valores 0 y 1: cuando *prev* sea 0, necesariamente *curr* es 1, y viceversa.

$$Q_0 : (\forall j \mid 0 \leq j \leq X : \text{tab}[\text{prev}][j] = \phi(n-1, j)).$$

$$Q_1 : (\forall j \mid 0 \leq j < x : \text{tab}[\text{curr}][j] = \phi(n, j)).$$

$$Q_2 : 0 \leq \text{prev} \leq 1.$$

$$Q_3 : 0 \leq \text{curr} \leq 1.$$

$$Q_4 : \text{prev} + \text{curr} = 1.$$

$$P_2 : 0 \leq n \leq N + 1.$$

$$P_3 : 0 \leq x \leq X + 1.$$

La matriz para la tabulación es $\text{tab}[0..2][0..X]$. El invariante Q_0 indica que $\text{tab}[\text{prev}]$ contiene todos los valores del pasado inmediatamente anterior, mientras que el arreglo $\text{tab}[\text{curr}][0..x)$ los del presente (aún faltan por procesar valores del presente en $\text{tab}[\text{curr}][x..X]$), de acuerdo con Q_1 . Los invariantes Q_2 y Q_3 especifican que los valores de *prev* y *curr* solo pueden ser 0 o 1, y el invariante Q_4 que los valores de estas variables no debe coincidir. Los invariantes P_2 y P_3 corresponden a los de la especificación del algoritmo sin optimizar. Con base en este diseño se presenta la función `ks_tab_opt1` en el Algoritmo 3.6.2.

Algoritmo 3.6.2: Primera optimización del espacio

```

1 def ks_tab_opt1(V, W, X):
2     N = len(V)
3     tab = [ [ 0 for _ in range(X+1) ] for _ in range(2) ]
4     n,x,prev,curr = 1,0,0,1
5     # Q0 ∧ Q1 ∧ Q2 ∧ Q3 ∧ Q4 ∧ P2 ∧ P3
6     while n!=N+1:
7         if x==X+1: n,x,prev,curr = n+1,0,1-prev,1-curr
8         else:
9             tab[curr][x] = tab[prev][x]
10            if W[n-1]<=x:
11                tab[curr][x] = max(tab[curr][x], tab[prev][x-W[n-1]]+V[n-1])
12            x += 1
13     return tab[prev][X]
```

Además de los cambios derivados de la nueva forma de hacer la tabulación, la función `ks_tab_opt1` incluye una actualización en la inicialización de la tabla: en el momento de su creación, todos los valores en ella son 0. Esto evita tener que procesar la primera columna después de su creación (es decir, se está ahorrando una pasada por una columna completa). Inicialmente, *prev* es asignada el índice

0 y *curr* el índice 1. Es decir, el pasado está registrado en la columna 0 de *tab* y el presente en la columna 1. El cuerpo del ciclo tiene dos partes: una de cambio de columna (línea 7) y otra de cambio de fila (líneas 8-12). El cambio de columna se hace de manera similar al algoritmo sin optimización, incrementando *n* en una unidad y asignando 0 a *x*. Las variables *prev* y *curr* invierten sus valores (¿por qué la resta funciona?). Se pudo optar por intercambiar directamente los valores de estas dos variables apelando a la sustitución “simultánea” ofrecida por Python. Sin embargo, esto no es posible en lenguajes de programación imperativos como C, C++ o Java. Por esto, se prefiere hacer el “complemento” con 1, dado que esta operación aritmética se puede implementar con una asignación individual sin la necesidad de una variable intermedia/temporal. El cuerpo del ciclo se encarga de encontrar el mejor valor posible para $\phi(n, x)$ de acuerdo con su definición recurrente. Observe el valor que se retorna está al final de la columna *prev* y no en la columna *curr* (¿por qué?).

Se ilustra el uso de la función `ks_tab_opt1` con algunas instancias del problema:

```

1 >>> V = [4, 6, 5, 1]
2 >>> W = [3, 5, 5, 2]
3 >>> print(ks_tab_opt1(V, W, 15))
4 16
5 >>> print(ks_tab_opt1(V, W, 11))
6 11
7 >>> print(ks_tab_opt1(V, W, 5))
8 6
9 >>> print(ks_tab_opt1(V, W, 1))
10 0

```

El diseño de la tabulación para `ks_tab_opt1` permite reducir el espacio de la tabulación a $O(2X) = O(X)$, mientras que el tiempo de ejecución se mantiene en $O(NX)$.

Teorema 3.6.3

El llamado `ks_tab_opt1(V, W, X)`, con $\text{len}(V) = \text{len}(W) = N$:

1. Calcula la suma máxima de elementos con valores en $V[0..N)$ cuyos pesos en $W[0..N)$ no exceden X .
2. Toma tiempo $O(NX)$.
3. Usa espacio $O(X)$.

Demostración

Se propone como ejercicio al lector.

3.6.2. Segunda optimización. El espacio de la tabulación puede ser reducido aún más. Note que los valores que se utilizan para actualizar la columna n (i.e., la del presente) están en un espacio acotado dentro de la columna $n - 1$ (i.e., la del pasado inmediatamente anterior). Si de alguna forma se pudiera consolidar la información de los dos momentos en el tiempo en una sola columna, esto permitiría reducir el espacio de $2X$ a X posiciones de memoria. La observación clave es que para tabular $\phi(n, x)$, solo se requieren valores de $\phi(n - 1, y)$, con $0 \leq y \leq x$. Es decir, en la tabulación deseada se podría “partir” esa columna de tal forma que la parte inferior al índice x tuviera solo valores del pasado inmediato y en la parte superior de x solo valores del presente. El diagrama de necesidades de la Figura 9 resume visualmente estas observaciones. Como la parte inferior de $tab[0..X]$ debe

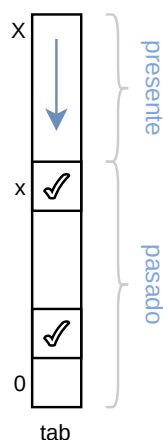


Figura 9. Diagrama de necesidades de la segunda optimización: una única columna en la cual la parte inferior al índice x tiene valores del pasado inmediato y la parte superior valores del presente.

tener disponibles los valores del pasado, necesariamente la forma de llenarla debe iniciar por su parte superior. Los siguientes invariantes hacen explícita esta y las

observaciones anteriores.

$$R_0 : (\forall j \mid 0 \leq j \leq x : \text{tab}[j] = \phi(n-1, j)).$$

$$R_1 : (\forall j \mid x < j \leq X : \text{tab}[j] = \phi(n, j)).$$

$$P_2 : 0 \leq n \leq N+1.$$

$$P_3 : -1 \leq x \leq X.$$

El invariante R_0 especifica que el subarreglo $\text{tab}[0..x]$ tiene los valores del pasado inmediato (i.e., desde $\phi(n-1, 0)$ hasta $\phi(n-1, x)$). El invariante R_1 , a su vez, especifica que el subarreglo $\text{tab}[x+1..X]$ tiene los valores del presente (i.e., desde $\phi(n, x+1)$ hasta $\phi(n, X)$). Los invariantes P_2 y P_3 son los mismos de las dos versiones anteriores de los algoritmos de tabulación de ϕ .

Algoritmo 3.6.3: Segunda optimización del espacio

```

1 def ks_tab_opt2(V, W, X):
2     N = len(V)
3     tab = [ 0 for _ in range(X+1) ]
4     n, x = 1, X
5     #  $R_0 \wedge R_1 \wedge P_2 \wedge P_3$ 
6     while n != N+1:
7         if x == -1: n, x = n+1, X
8         else:
9             if W[n-1] <= x:
10                 tab[x] = max(tab[x], tab[x-W[n-1]]+V[n-1])
11             x -= 1
12     return tab[X]
```

Inicialmente, $\text{tab}[0..X]$ es creado con valores 0. A partir de la columna $n = 1$ y hasta la columna $n = N$, se hace una de dos cosas. O bien se detecta que se ha procesado una nueva columna (i.e., $x = -1$) y se avanza a la (posible) siguiente columna para comenzar el proceso desde la fila X . De lo contrario, el valor en $\text{tab}[x]$ es $\phi(n-1, x)$. Si se tiene capacidad disponible en el morral (i.e., $W[n-1] \leq x$) y hay mayor beneficio en llevar el elemento a no llevarlo, entonces se actualiza $\text{tab}[x]$ con $\phi(n-1, x - W[n-1]) + V[n-1]$.

Teorema 3.6.4

El llamado $\text{ks_tab_opt2}(V, W, X)$, con $\text{len}(V) = \text{len}(W) = N$:

1. Calcula la suma máxima de elementos con valores en $V[0..N)$ cuyos pesos en $W[0..N)$ no exceden X .

2. Toma tiempo $O(NX)$.
3. Usa espacio $O(X)$.

Demostración

Se propone como ejercicio al lector.

Asintóticamente, los algoritmos 3.6.2 y 3.6.3 usan la misma cantidad de espacio. Sin embargo, el espacio se reduce exactamente a la mitad en `ks_tab_opt2` y, además, el código es más sucinto y fácil de seguir. Por estas razones debería preferirse esta función a `ks_tab_opt1`.

Ejercicios

1. Demuestre que la cantidad de subconjuntos de un conjunto de N elementos es 2^N .
2. Considere los 4 elementos con pesos y valores dados en el Ejemplo 3.6.1. Enumere los 16 conjuntos de índices correspondientes a esta instancia del problema del morral, y a cada uno de ellos asocie el peso y valor total. Verifique que la conclusión es correcta para cada X en dicho ejemplo.
3. Justifique con un ejemplo que el problema del morral exhibe las propiedades de subestructura óptima y solapamiento.
4. Demuestre que la definición recurrente de $\phi(n, x)$, para $0 \leq n \leq N$ y $0 \leq x \leq X$, en cualquier llamado recurrente mantiene los valores de n y x en estos rangos.
5. Una demostración tentadora del Teorema 3.6.1 empieza por caracterizar la solución óptima S y observa que, si $S = \emptyset$, entonces no hay elementos para escoger (i.e., $n = 0$) o todos los elementos disponibles pesan más que x (i.e., $W[i] > x$ para $0 \leq i < n$). Exhiba una instancia del problema del morral en la cual $\emptyset$ sea una solución óptima sin que se cumpla ninguna de las dos condiciones, y explique por qué esa caracterización no se puede usar como punto de partida.
6. En la tabulación propuesta para ϕ en `ks_tab` se opta por llenar `tab[0..N][0..X]` por columnas. Diseñe una tabulación para ϕ que resulte de llenar `tab` por filas. Bajo esta decisión de llenar la matriz, ¿es posible optimizar el espacio de la tabulación? Justifique su respuesta.
7. Demuestre el Teorema 3.6.2.

8. Diseñe un algoritmo con memorización para ϕ . Demuestre que es correcto con respecto a la especificación hecha. ¿Cuáles son sus complejidades temporal y espacial? Justifique su respuesta.
9. Investigue acerca de tiempo pseudo-polinomial y explique su relación con algoritmos numéricos. En particular, explique su relación con el algoritmo común para determinar si un número es primo.
10. Demuestre el Teorema 3.6.3.
11. En la función `ks_tab_opt1`, los papeles de presente y pasado se intercambian con la asignación `prev, curr = 1-prev, 1-curr`. Demuestre que si a es un número entero que satisface $0 \leq a \leq 1$, entonces $0 \leq 1 - a \leq 1$, y explique por qué eso es lo que garantiza que `prev` y `curr` sigan siendo índices válidos de la tabla.
12. En la función `ks_tab_opt1` se retorna el valor `tab[prev][X]`. Explique por qué sería incorrecto retornar el valor `tab[curr][X]`.
13. Demuestre el Teorema 3.6.4.
14. En el planteamiento de la función objetivo, la cantidad de elementos disponibles se hizo variar para considerar diferentes prefijos de los índices de $V[0..N]$. Plantee una definición alternativa a ϕ en la cual se hagan variar los diferentes sufijos de esos índices.
15. Hay una variante del problema del morral en la cual de cada elemento hay una cantidad ilimitada de copias (y se permite llevar cuantas copias sean deseadas de cada elemento).
 - a) Especifique esta variante del problema del morral.
 - b) Diseñe una solución con tabulación para el problema especificado usando la Metodología 3.3.1. Si es posible reducir el espacio de la tabulación, redúzcalo al máximo.
16. La siguiente especificación corresponde al problema, comúnmente denominado, suma exacta de un subconjunto (en inglés, *Subset Sum*):

Entrada: Arreglo $A[0..N]$, con $N \geq 0$, de números y un número X .

Salida: ¿Hay un subconjunto de elementos de $A[0..N]$ cuya suma es X ?

- a) En la literatura se indica que el problema de la suma exacta de un subconjunto es una instancia particular del problema del morral. Justifique por qué esta afirmación es cierta e ilústre su respuesta con ejemplos.
 - b) Diseñe un algoritmo con tabulación que permita resolver el problema de suma exacta de un subconjunto.
-

3.7. Subsecuencia común más larga

El problema de la subsecuencia común más larga consiste en determinar cuánto se parecen dos arreglos, midiendo ese parecido por la mayor cantidad de elementos que aparecen en ambos, en el mismo orden y no necesariamente de manera contigua. La noción de subsecuencia es la de la Definición 3.5.1; lo nuevo es que ahora hay dos arreglos y se busca algo que sea subsecuencia de los dos a la vez.

Definición 3.7.1: Subsecuencia común

Sean $A[0..M)$ y $B[0..N)$ arreglos. Una sucesión s es una *subsecuencia común* de A y B si y solo si s es subsecuencia de $A[0..M)$ y s es subsecuencia de $B[0..N)$, en el sentido de la Definición 3.5.1. La subsecuencia vacía es común a cualquier par de arreglos, de modo que siempre existe al menos una subsecuencia común.

Note que una subsecuencia común está determinada por *dos* conjuntos de índices, uno en cada arreglo, y que esos dos conjuntos pueden ser distintos entre sí.

Problema 3.7.1: Subsecuencia común más larga

Entrada: Arreglos $A[0..M)$ y $B[0..N)$, con $M \geq 0$ y $N \geq 0$.

Salida: Longitud de una subsecuencia común más larga de $A[0..M)$ y $B[0..N)$.

Ejemplo 3.7.1

Considere los arreglos de caracteres $A = \text{ALGORITMO}$ y $B = \text{LOGARITMO}$. La sucesión LGRITMO es una subsecuencia común: en A la determinan los índices 1, 2, 4, 5, 6, 7, 8 y en B los índices 0, 2, 4, 5, 6, 7, 8. Tiene longitud 7 y es una subsecuencia común más larga.

No es la única: LORITMO también es común a A y B y también tiene longitud 7. Como es usual en los problemas de optimización, lo que es único es el valor óptimo y no la configuración que lo alcanza.

Nota 3.7.1: Dónde aparece este problema

Este problema es el corazón de las herramientas que comparan archivos de texto línea por línea, como el programa `diff` y los sistemas de control de versiones: las líneas que pertenecen a una subsecuencia común más larga son las que se reportan como no modificadas, y todas las demás como insertadas o eliminadas. En biología computacional aparece al comparar secuencias de ADN o de proteínas, en donde el parecido entre dos secuencias es evidencia de parentesco evolutivo. Variantes cercanas del problema, como la distancia de edición, se usan en correctores ortográficos y en motores de búsqueda.

Una solución de fuerza bruta consiste en enumerar las 2^M subsecuencias de A y verificar, para cada una, si es también subsecuencia de B . Esa verificación se puede hacer en tiempo $O(N)$, de modo que el costo total es $O(2^M \cdot N)$: exponencial, como en los demás problemas de este capítulo. El objetivo es obtener una solución polinomial con programación dinámica.

Este es un problema de optimización, de modo que la propiedad que hay que verificar es la subestructura óptima de la Definición 3.1.1. Se sigue entonces la Metodología 3.3.1, cuyo paso de especificación ya quedó cumplido con el Problema 3.7.1.

Función objetivo. Como en el problema del morral, hay dos parámetros que se pueden hacer variar y en este caso son los tamaños de los dos arreglos. Para $0 \leq m \leq M$ y $0 \leq n \leq N$, se define:

$\phi(m, n)$: “longitud de una subsecuencia común más larga de $A[0..m)$ y $B[0..n)$.”

Es decir, se generaliza el problema a todos los pares de prefijos de los arreglos dados.

Reformulación de la especificación. El objetivo del problema es calcular $\phi(M, N)$. A diferencia de lo que ocurre en la Sección 3.4 y en la Sección 3.5, aquí no hace falta ninguna maximatoria externa: la función objetivo evaluada en los parámetros de la instancia es ya la salida del problema, como sucedía en la Sección 3.6.

Planteamiento recurrente. Se procede por casos sobre m y n . Si alguno de los dos prefijos es vacío no hay nada que compartir y la única subsecuencia común es la vacía; por lo tanto $\phi(m, n) = 0$ cuando $m = 0$ o $n = 0$.

Suponga entonces que $m \neq 0$ y $n \neq 0$, y fije la atención en los últimos elementos de los dos prefijos, $A[m-1]$ y $B[n-1]$. Hay dos posibilidades:

- Si $A[m-1] = B[n-1]$, conviene emparejar esos dos elementos y resolver el problema para los prefijos que quedan al quitarlos; es decir, $\phi(m, n) = 1 + \phi(m-1, n-1)$. Que esto sea correcto no es evidente: podría ocurrir que alguna subsecuencia común más larga no emparejara $A[m-1]$ con $B[n-1]$. Lo que se

demuestra más adelante es que, si eso ocurre, siempre se puede modificar esa subsecuencia para que sí lo haga, sin cambiar su longitud.

- Si $A[m-1] \neq B[n-1]$, entonces esos dos elementos no se pueden emparejar entre sí y, en consecuencia, al menos uno de los dos sobra. Como no se sabe cuál, se apuesta a los dos caballos: se descarta $A[m-1]$ y se recurre con $m-1$ y n , se descarta $B[n-1]$ y se recurre con m y $n-1$, y se prefiere la opción que produzca la mayor longitud.

En los tres casos los llamados recurrentes respetan las condiciones $0 \leq m \leq M$ y $0 \leq n \leq N$. La definición formal de ϕ se presenta para $0 \leq m \leq M$ y $0 \leq n \leq N$:

$$\phi(m, n) = \begin{cases} 0 & , \text{ si } m = 0 \vee n = 0, \\ 1 + \phi(m-1, n-1) & , \text{ si } m, n \neq 0 \wedge A[m-1] = B[n-1], \\ \phi(m-1, n) \uparrow \phi(m, n-1) & , \text{ si } m, n \neq 0 \wedge A[m-1] \neq B[n-1]. \end{cases}$$

Observe que el segundo caso tiene un solo llamado recurrente mientras que el tercero tiene dos. Es en el tercero donde se esconde el crecimiento exponencial que la programación dinámica va a evitar.

Correctitud de la función objetivo. El Teorema 3.7.1 presenta el criterio de correctitud de la función ϕ .

Teorema 3.7.1

Si $0 \leq m \leq M$ y $0 \leq n \leq N$, entonces $\phi(m, n)$ es la longitud de una subsecuencia común más larga de $A[0..m]$ y $B[0..n]$.

Demostración

Sea $L(m, n)$ la longitud de una subsecuencia común más larga de $A[0..m]$ y $B[0..n]$. Se procede por inducción sobre $m+n$, suponiendo que $\phi(i, j) = L(i, j)$ para todo par (i, j) con $i+j < m+n$.

Si $m = 0$ o $n = 0$, la única subsecuencia común es la vacía, luego $L(m, n) = 0 = \phi(m, n)$. Suponga entonces que $m \neq 0$ y $n \neq 0$, y sea s una subsecuencia común de longitud $k = L(m, n)$. Se distinguen dos casos.

- Caso $A[m-1] = B[n-1]$. Se procede por doble desigualdad.

Composición: $L(m, n) \geq 1 + L(m-1, n-1)$. Al tomar una subsecuencia común más larga de $A[0..m-1]$ y $B[0..n-1]$ y extenderla con el elemento $A[m-1]$ se obtiene una subsecuencia común de $A[0..m]$ y $B[0..n]$, pues $A[m-1] = B[n-1]$ y los índices usados son menores que $m-1$ y $n-1$, respectivamente.

Descomposición: $L(m, n) \leq 1 + L(m - 1, n - 1)$. Note que $k \geq 1$, ya que el elemento $A[m - 1]$ por sí solo es una subsecuencia común. Sean i y j los índices en A y en B del último elemento de s . Si $i \neq m - 1$ o $j \neq n - 1$, considere la sucesión s' que resulta de quitarle a s su último elemento y agregarle $A[m - 1]$: como los índices que s usa antes de su último elemento son menores que $i \leq m - 1$ y que $j \leq n - 1$, la sucesión s' es una subsecuencia común de la misma longitud k cuyo último elemento está emparejado en los índices $m - 1$ y $n - 1$. Sin pérdida de generalidad se puede suponer, entonces, que $i = m - 1$ y $j = n - 1$. Al quitarle a s su último elemento queda una subsecuencia común de $A[0..m - 1)$ y $B[0..n - 1)$ de longitud $k - 1$, de donde $L(m - 1, n - 1) \geq k - 1$.

De las dos desigualdades y de la hipótesis inductiva se concluye $\phi(m, n) = 1 + \phi(m - 1, n - 1) = 1 + L(m - 1, n - 1) = L(m, n)$.

- Caso $A[m - 1] \neq B[n - 1]$. Se procede por doble desigualdad.

Composición: $L(m, n) \geq L(m - 1, n)$ y $L(m, n) \geq L(m, n - 1)$, pues toda subsecuencia común de un par de prefijos lo es también de cualquier par de prefijos más largos.

Descomposición: $L(m, n) \leq L(m - 1, n) \uparrow L(m, n - 1)$. Si $k = 0$ la desigualdad es inmediata. De lo contrario, sean i y j los índices en A y en B del último elemento de s ; se tiene $A[i] = B[j]$ y, como $A[m - 1] \neq B[n - 1]$, no puede suceder a la vez que $i = m - 1$ y $j = n - 1$. Si $i < m - 1$, entonces s es subsecuencia común de $A[0..m - 1)$ y $B[0..n)$, luego $k \leq L(m - 1, n)$. Si $j < n - 1$, un argumento simétrico da $k \leq L(m, n - 1)$.

De las dos desigualdades y de la hipótesis inductiva se concluye:

$$\begin{aligned} \phi(m, n) &= \phi(m - 1, n) \uparrow \phi(m, n - 1) \\ &= L(m - 1, n) \uparrow L(m, n - 1) \\ &= L(m, n). \end{aligned}$$

Note que las desigualdades de esta demostración se establecen entre L y combinaciones de L , y solo al final se invoca la hipótesis inductiva para pasar a ϕ ; en las secciones anteriores se establecen directamente entre ϕ y L . Las dos formas caben en el patrón de la Sección 3.3 y cada una resulta más cómoda según el problema, pero conviene reconocerlas como la misma cosa.

La propiedad de solapamiento se observa con facilidad: $\phi(m - 1, n)$ y $\phi(m, n - 1)$ invocan ambos a $\phi(m - 1, n - 1)$. Es exactamente la misma situación del Problema 3.2.1, con dos llamados que confluyen en un tercero: esto hace crecer exponencialmente una implementación que se limite a transcribir la definición de ϕ .

3.7.1. Tabulación. El siguiente paso es determinar si se usa memorización o tabulación.

¿Memorización o tabulación? Se diseñará una tabulación con base en el diagrama de necesidades de la Figura 10. La tabulación se implementa con una

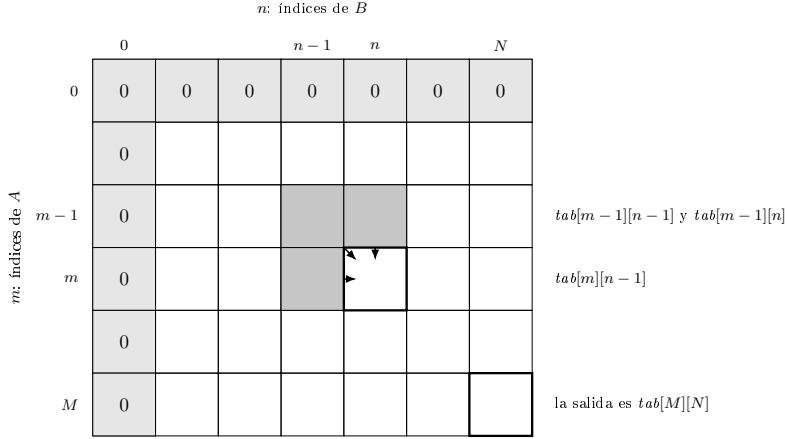


Figura 10. Diagrama de necesidades para la tabulación de ϕ en el problema de la subsecuencia común más larga. La primera fila y la primera columna corresponden a los casos base. El valor $tab[m][n]$ depende de las tres celdas vecinas que lo preceden: la diagonal, la superior y la izquierda.

matriz $tab[0..M][0..N]$ con $M+1$ filas y $N+1$ columnas. El objetivo es calcular el valor correspondiente a $tab[M][N]$, dado que el problema se resuelve con $\phi(M, N)$. La primera fila y la primera columna corresponden al caso base y tienen únicamente el valor 0. Para calcular $tab[m][n]$ en los casos recurrentes se depende de $tab[m-1][n-1]$, de $tab[m-1][n]$ y de $tab[m][n-1]$; cuál de ellos se usa depende de si $A[m-1]$ y $B[n-1]$ coinciden o no.

Las tres celdas de las que depende $tab[m][n]$ están en la fila $m-1$ o a la izquierda en la fila m . Esto indica que se puede llenar la tabulación por filas, de arriba hacia abajo y, dentro de cada fila, de izquierda a derecha. Esta decisión se ve reflejada en los siguientes invariantes:

$$P_0 : (\forall i, j \mid 0 \leq i < m \wedge 0 \leq j \leq N : tab[i][j] = \phi(i, j)).$$

$$P_1 : (\forall j \mid 0 \leq j < n : tab[m][j] = \phi(m, j)).$$

$$P_2 : 0 \leq m \leq M+1.$$

$$P_3 : 0 \leq n \leq N+1.$$

El invariante P_0 indica que todas las filas por encima de la fila m han sido llenadas con los valores correspondientes de ϕ . El invariante P_1 especifica que los valores en la fila m , a la izquierda de la columna n , corresponden a los valores de ϕ .

Los invariantes P_2 y P_3 son de apoyo y, como en la Sección 3.6, permiten que las variables tomen un valor por fuera del rango de la tabla para señalar que se terminó de llenar una fila o la tabla completa. Con base en este diseño se presenta la función `lcs_tab` en el Algoritmo 3.7.1.

Implementación. El Algoritmo 3.7.1 presenta una implementación de la tabulación diseñada para ϕ .

Algoritmo 3.7.1: Tabulación para ϕ

```

1 def lcs_tab(A, B):
2     M,N = len(A),len(B)
3     tab = [ [ None for _ in range(N+1) ] for _ in range(M+1) ]
4     for n in range(N+1): tab[0][n] = 0
5     m,n = 1,0
6     #  $P_0 \wedge P_1 \wedge P_2 \wedge P_3$ 
7     while m!=M+1:
8         if n==N+1: m,n = m+1,0
9         else:
10            if n==0: tab[m][n] = 0
11            elif A[m-1]==B[n-1]: tab[m][n] = tab[m-1][n-1]+1
12            else: tab[m][n] = max(tab[m-1][n], tab[m][n-1])
13            n += 1
14     return tab[M][N]
```

La estructura de `lcs_tab` es la misma de `ks_tab` en la Sección 3.6, con los papeles de filas y columnas intercambiados. El cuerpo del ciclo tiene dos partes: una de cambio de fila, cuando ya se llenó la fila actual, y otra que calcula una celda distinguiendo los tres casos de la definición de ϕ . Note que la primera columna se trata dentro del ciclo, con la condición `n==0`, mientras que la primera fila se inicializa antes de entrar en él.

Se ilustra el uso de la función con algunas instancias:

```

1 >>> print(lcs_tab('ALGORITMO', 'LOGARITMO'))
2 7
3 >>> print(lcs_tab('ABC', 'ABC'))
4 3
5 >>> print(lcs_tab('ABC', 'XYZ'))
6 0
7 >>> print(lcs_tab('', 'ABC'))
8 0
```

Análisis. La correctitud y el análisis asintótico del Algoritmo 3.7.1 se enuncian en el Teorema 3.7.2.

Teorema 3.7.2

El llamado `lcs_tab(A, B)`, con $\text{len}(A) = M$ y $\text{len}(B) = N$:

1. Calcula la longitud de una subsecuencia común más larga de $A[0..M)$ y $B[0..N)$.
2. Toma tiempo $O(MN)$.
3. Usa espacio $O(MN)$.

Demostración

Se propone como ejercicio al lector.

A diferencia de lo que ocurre en la Sección 3.6, esta complejidad es genuinamente polinomial en el tamaño de la entrada: M y N son las longitudes de los arreglos y no valores numéricos que aparecen en ellos, de modo que no hay aquí nada parecido al tiempo pseudo-polinomial.

3.7.2. Reducción del espacio. El diagrama de necesidades muestra que el cálculo de la fila m solo depende de valores en la fila $m - 1$ y de valores ya calculados en la fila m misma. Como en la Sección 3.6, esto sugiere que no hace falta conservar la tabla completa. Ahora bien, la reducción no es idéntica a la de aquella sección y conviene entender por qué.

Nota 3.7.2: Por qué no funciona el recorrido en reversa

En la Sección 3.6 el espacio se redujo a un solo arreglo recorriéndolo en sentido contrario al del llenado. El truco funciona allí porque el cálculo de una celda depende únicamente de celdas del pasado inmediato: al recorrer en reversa, las posiciones que aún no se han sobrescrito son exactamente las que se necesitan.

Aquí ese truco falla, porque $\phi(m, n)$ depende también de $\phi(m, n - 1)$, que es un valor del *presente*. Recorrer la fila de derecha a izquierda dejaría ese valor sin calcular. Y recorrerla de izquierda a derecha, sobre un único arreglo, destruye el valor $\phi(m - 1, n - 1)$ justo un paso antes de necesitarlo: al escribir en la posición $n - 1$ se pierde el dato que hará falta al procesar la posición n .

La solución es conservar ese único valor en una variable auxiliar antes de sobrescribirlo. Es un patrón que vale la pena reconocer: cuando una reducción

de espacio falla por un solo dato, casi siempre basta con una variable adicional en lugar de renunciar a la reducción.

Vale la pena observar, además, que en este problema *cuáles* de las tres celdas vecinas se consultan depende de los datos: si $A[m-1] = B[n-1]$ se usa la diagonal y si no, las otras dos. Eso no impide la reducción, y es justamente el punto que señala la Nota 3.5.3: lo que decide el asunto no es si las dependencias dependen de la instancia sino si la ventana está acotada. Aquí lo está, en tres celdas vecinas, aunque no siempre sean las mismas tres.

Con base en esa observación se propone el diagrama de necesidades de la Figura 11, en el cual la tabulación es un único arreglo $row[0..N]$ acompañado de una variable $prev$. Los invariantes del ciclo interno, para una fila m fija, hacen explícitas

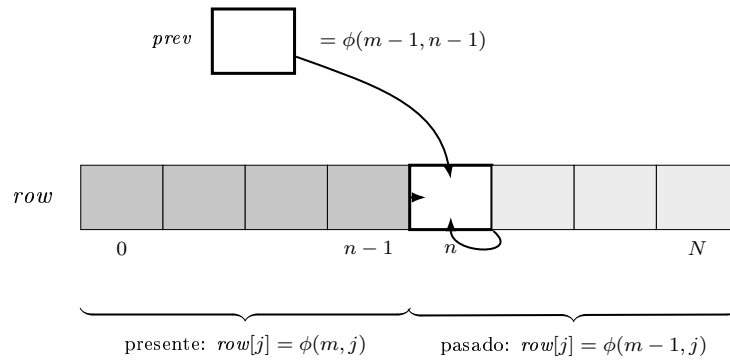


Figura 11. Diagrama de necesidades de la reducción del espacio en el problema de la subsecuencia común más larga. El arreglo row tiene, a la izquierda de la posición n , valores del presente y, de la posición n en adelante, valores del pasado inmediato. La variable $prev$ conserva el valor $\phi(m-1, n-1)$, que ya fue sobrescrito en row .

estas observaciones:

$$R_0 : (\forall j \mid 0 \leq j < n : row[j] = \phi(m, j)).$$

$$R_1 : (\forall j \mid n \leq j \leq N : row[j] = \phi(m-1, j)).$$

$$R_2 : prev = \phi(m-1, n-1).$$

$$R_3 : 1 \leq n \leq N+1.$$

El invariante R_0 especifica que el subarreglo $row[0..n)$ tiene los valores del presente y el invariante R_1 que el subarreglo $row[n..N]$ tiene los del pasado inmediato. En particular, $row[n]$ es $\phi(m-1, n)$ y $row[n-1]$ es $\phi(m, n-1)$: los dos valores que

requiere el tercer caso de la definición de ϕ . El invariante R_2 conserva el tercer valor necesario, $\phi(m-1, n-1)$, que ya no está disponible en *row*. El resultado es la función `lcs_tab_opt` del Algoritmo 3.7.2.

Algoritmo 3.7.2: Reducción del espacio de la tabulación

```

1 def lcs_tab_opt(A, B):
2     M, N = len(A), len(B)
3     row = [ 0 for _ in range(N+1) ]
4     m = 1
5     while m != M+1:
6         prev, n = 0, 1
7         #  $R_0 \wedge R_1 \wedge R_2 \wedge R_3$ 
8         while n != N+1:
9             curr = row[n]
10            if A[m-1] == B[n-1]: row[n] = prev+1
11            else: row[n] = max(row[n], row[n-1])
12            prev, n = curr, n+1
13        m += 1
14    return row[N]
```

La variable *curr* guarda el valor de *row[n]* antes de sobrescribirlo, es decir, guarda $\phi(m-1, n)$. Al final de la iteración ese valor pasa a *prev*, que en la iteración siguiente –en la cual la variable *n* vale uno más– contendrá exactamente $\phi(m-1, n-1)$, como lo exige R_2 . La asignación inicial *prev, n* = 0, 1 establece R_2 al comenzar cada fila, pues $\phi(m-1, 0) = 0$ para cualquier *m*.

Teorema 3.7.3

El llamado `lcs_tab_opt(A, B)`, con $\text{len}(A) = M$ y $\text{len}(B) = N$:

1. Calcula la longitud de una subsecuencia común más larga de $A[0..M)$ y $B[0..N)$.
2. Toma tiempo $O(MN)$.
3. Usa espacio $O(N)$.

Demostración

Se propone como ejercicio al lector.

Como el espacio utilizado es $O(N)$ y el problema es simétrico en sus dos argumentos, conviene invocar la función con el arreglo más corto en el segundo parámetro, obteniendo espacio $O(\min(M, N))$.

3.7.3. Reconstrucción de una solución óptima. Hasta este punto, todos los algoritmos del capítulo calculan el *valor* de una solución óptima y ninguno exhibe una solución que lo alcance. Para el problema del morral se obtiene el mejor beneficio posible pero no qué elementos llevar; para el problema del agente viajero, el costo del mejor circuito pero no el circuito. Esta sección cierra esa brecha, y lo hace aquí porque es donde la reconstrucción entra en conflicto con la reducción del espacio que acaba de hacerse.

Problema 3.7.2: Reconstrucción de una subsecuencia común más larga

Entrada: Arreglos $A[0..M]$ y $B[0..N]$, con $M \geq 0$ y $N \geq 0$.

Salida: Una subsecuencia común más larga de $A[0..M]$ y $B[0..N]$.

La idea es que la tabulación contiene, además del valor óptimo, el rastro de las decisiones que lo produjeron. Basta con recorrerla en sentido contrario, desde la celda $tab[M][N]$ hacia la celda $tab[0][0]$, preguntando en cada paso cuál de los tres casos de la definición de ϕ fue el que se aplicó:

- Si $A[m-1] = B[n-1]$, se aplicó el segundo caso. Ese elemento hace parte de la subsecuencia que se está reconstruyendo y el recorrido continúa en $tab[m-1][n-1]$.
- Si $A[m-1] \neq B[n-1]$, se aplicó el tercer caso y el valor proviene del mayor entre $tab[m-1][n]$ y $tab[m][n-1]$. El recorrido continúa en aquella de las dos celdas que alcanza el máximo; si las dos lo alcanzan, cualquiera de ellas sirve.
- El recorrido termina al llegar a la fila 0 o a la columna 0, pues allí la subsecuencia común es vacía.

Como los elementos se descubren de derecha a izquierda, se obtienen en orden inverso y hay que invertirlos al final. El Algoritmo 3.7.3 implementa esta estrategia. Recibe la tabulación completa como parámetro, la cual se obtiene de `lcs_tab` cambiando su última línea por `return tab`.

Algoritmo 3.7.3: Reconstrucción de una solución óptima

```

1 def lcs_rec(A, B, tab):
2     ans, m, n = [], len(A), len(B)
3     while m != 0 and n != 0:
```

```

4     if A[m-1]==B[n-1]: ans,m,n = ans+[A[m-1]],m-1,n-1
5     elif tab[m-1][n]>=tab[m][n-1]: m -= 1
6     else: n -= 1
7     ans.reverse()
8     return ans

```

Se ilustra el uso de la función con la instancia del Ejemplo 3.7.1:

```

1 >>> A,B = 'ALGORITMO', 'LOGARITMO'
2 >>> print(''.join(lcs_rec(A, B, lcs_tab(A, B))))
3 LGRITMO

```

Teorema 3.7.4

Sea tab la tabulación construida por el Algoritmo 3.7.1 para los arreglos $A[0..M]$ y $B[0..N]$. El llamado $lcs_rec(A, B, tab)$:

1. Calcula una subsecuencia común más larga de $A[0..M]$ y $B[0..N]$.
2. Toma tiempo $O(M + N)$.
3. Usa espacio $O(M + N)$, sin contar el de la tabulación.

Demostración

Se propone como ejercicio al lector.

El costo de la reconstrucción es despreciable frente al de construir la tabla: en cada iteración del ciclo disminuye m , disminuye n o disminuyen ambos, de modo que a lo sumo se ejecutan $M + N$ iteraciones. Lo que no es despreciable es el espacio.

Nota 3.7.3: La tensión entre reducir y reconstruir

El Algoritmo 3.7.3 necesita la tabulación completa y por eso no se puede combinar con el Algoritmo 3.7.2: la reducción del espacio conserva el valor óptimo pero destruye el rastro de las decisiones que lo produjeron. Es decir, para este problema se puede calcular el valor óptimo en espacio $O(\min(M, N))$, o exhibir una solución óptima en espacio $O(MN)$, pero los dos algoritmos presentados no permiten hacer ambas cosas a la vez.

Esta tensión conviene tenerla presente cada vez que se reduzca el espacio de una tabulación: la pregunta pertinente no es solamente si el valor óptimo se puede seguir calculando, sino también qué se pierde con la información que

se descarta. No es, eso sí, inevitable: en la Sección 3.8 se reconstruyen las soluciones óptimas sin costo adicional de espacio, porque allí el rastro de las decisiones cabe en un solo dato por celda.

Vale la pena señalar que la tensión sí se puede resolver para este problema en particular. Existe un algoritmo, debido a D. Hirschberg, que reconstruye una subsecuencia común más larga en tiempo $O(MN)$ y espacio $O(\min(M, N))$, combinando la programación dinámica de esta sección con la técnica de dividir, conquistar y combinar. La idea es calcular, con el algoritmo de espacio reducido, por dónde cruza una solución óptima la fila central de la tabla, y recurrir sobre las dos mitades que quedan. En las notas del final del capítulo se indican referencias.

Ejercicios

1. Demuestre que si $A[0..M)$ y $B[0..N)$ no tienen ningún elemento en común, entonces todas las entradas de la tabulación son 0.
2. Demuestre que ϕ es monótona en cada uno de sus argumentos; es decir, que $\phi(m-1, n) \leq \phi(m, n)$ y $\phi(m, n-1) \leq \phi(m, n)$ siempre que los índices estén en rango. Concluya que dos celdas vecinas de la tabulación difieren en 0 o en 1.
3. Elabore un ejemplo en el cual se exhiba la propiedad de solapamiento para ϕ . Explique brevemente su respuesta.
4. Demuestre el Teorema 3.7.2.
5. Demuestre el Teorema 3.7.3. En particular, demuestre que el ciclo interno satisface los invariantes R_0 , R_1 , R_2 y R_3 .
6. Demuestre el Teorema 3.7.4.
7. Diseñe un algoritmo con memorización para ϕ instanciando el Algoritmo 3.3.1, e incluya las condiciones de representación que debe satisfacer la memoria compartida. ¿Cuáles son sus complejidades temporal y espacial?
8. En el Ejemplo 3.7.1 se exhiben dos subsecuencias comunes más largas de ALGORITMO y LOGARITMO.
 - a) ¿Cuál de las dos retorna el Algoritmo 3.7.3? Explique por qué.
 - b) Modifique la línea 5 del algoritmo cambiando $\geq$ por $>$. ¿Cambia la longitud de la respuesta? ¿Cambia la subsecuencia retornada?
9. Modifique el Algoritmo 3.7.1 para que, además de la longitud, calcule la *cantidad* de subsecuencias comunes más largas distintas que hay. Cuidado: dos conjuntos de índices distintos pueden determinar la misma sucesión de valores; decida explícitamente cuál de las dos cantidades está calculando.

10. El problema de la *subcadena* común más larga pide la mayor longitud de un subarreglo que aparezca en $A[0..M)$ y en $B[0..N)$; es decir, la versión contigua de este problema.
 - a) Especifique el problema.
 - b) Explique por qué la definición de ϕ de esta sección no lo resuelve, e ilustre su explicación con un ejemplo.
 - c) Diseñe una función objetivo y una definición recurrente apropiadas, en el espíritu de la función objetivo de la Sección 3.4, y proponga una tabulación.
 11. La *distancia de edición* entre dos arreglos es la mínima cantidad de operaciones de inserción, eliminación y sustitución de un elemento que se requieren para transformar el primero en el segundo.
 - a) Especifique el problema de calcular la distancia de edición.
 - b) Siga la Metodología 3.3.1 para diseñar una solución con tabulación. Debe resultar código en Python ejecutable.
 - c) ¿Se puede reducir el espacio de la tabulación? ¿Se puede reconstruir una secuencia de operaciones de costo mínimo?
 12. Suponga que $A[0..N)$ es un arreglo cuyos elementos son todos distintos y sea $B[0..N)$ el arreglo que resulta de ordenar A de manera creciente. Demuestre que una subsecuencia común más larga de A y B es una subsecuencia creciente más larga de A (Sección 3.5). ¿Qué complejidad temporal tiene el algoritmo que resulta y cómo se compara con la del Algoritmo 3.5.1? ¿Qué sucede si A tiene elementos repetidos?
 13. Generalice el problema a tres arreglos $A[0..M)$, $B[0..N)$ y $C[0..K)$. Proponga una función objetivo y su definición recurrente, y determine las complejidades temporal y espacial de la tabulación correspondiente. ¿Cuántos casos tiene la definición recurrente?
 14. 🌈 Diseñe el algoritmo de Hirschberg esbozado en la Nota 3.7.3, que reconstruye una subsecuencia común más larga en tiempo $O(MN)$ y espacio $O(\min(M, N))$. Justifique la cota de espacio y explique por qué la cota de tiempo se mantiene a pesar de la recurrencia.
-

3.8. Caminos mínimos entre todos los pares

El problema de caminos mínimos entre todos los pares consiste en calcular, para cada par de vértices de un grafo dirigido y pesado, el costo del camino más barato que los une. Es el problema con el cual se abrió la Sección 3.1 –allí se usó la ruta más corta entre dos puntos de una ciudad para ilustrar la propiedad de subestructura óptima– y esta sección lo resuelve en su forma general.

Nota 3.8.1: Grafos dirigidos y pesados

Un *grafo* G es una pareja (V, E) de *vértices* V y de *arcos* $E \subseteq V \times V$ sobre los vértices. Si $(u, v) \in E$ se dice que hay un arco entre u y v . El grafo puede ser *dirigido* o *no-dirigido*. En un grafo dirigido los arcos tienen dirección, es decir, un origen y un destino. En un grafo no-dirigido los arcos no tienen dirección, únicamente conectan dos vértices. En este texto, mientras no se haga la aclaración, los grafos se suponen no-dirigidos; en esta sección, en cambio, se suponen dirigidos.

Un *camino* de u a v en G es una sucesión de vértices $u = x_0, x_1, \dots, x_\ell = v$, con $\ell \geq 0$, tal que $(x_i, x_{i+1}) \in E$ para $0 \leq i < \ell$. Los vértices $x_1, \dots, x_{\ell-1}$ se llaman *intermedios*. El camino es *simple* si no repite vértices. Un *ciclo* es un camino de un vértice a sí mismo con $\ell \geq 1$. Note que para cualquier vértice u existe el camino de u a u con $\ell = 0$, llamado *camino vacío*.

Dada una función de peso $w : E \rightarrow \mathbb{R}$, el *costo* de un camino es la suma de los pesos de los arcos que lo componen; el camino vacío tiene costo 0.

Las relaciones de amistad o de seguimiento en redes sociales se pueden modelar como grafos, al igual que sistemas biológicos como redes de co-expresión genética o redes de interacción entre proteínas.

A diferencia de lo que sucede en otras secciones de este capítulo, los pesos aquí pueden ser negativos. Esa libertad tiene un precio y es la única hipótesis que hay que imponer sobre la entrada.

Problema 3.8.1: Caminos mínimos entre todos los pares

Entrada: Un grafo dirigido $G = (V, E)$, con $N = |V| \geq 0$ vértices y función de peso $w : E \rightarrow \mathbb{R}$, sin ciclos de peso negativo.

Salida: El costo mínimo de un camino de u a v , para cada par de vértices $u, v \in V$.

Nota 3.8.2: ¿Por qué se excluyen los ciclos de peso negativo?

Suponga que en G hay un ciclo de peso negativo y que dicho ciclo es alcanzable desde u y alcanza a v . Entonces no existe un camino de costo mínimo de u a v : dado cualquier camino, se puede obtener otro más barato recorriendo el ciclo una vez más y así indefinidamente. El ínfimo de los costos es $-\infty$ y el problema

queda mal planteado. Por eso la hipótesis aparece en la especificación y no como un detalle técnico.

Al excluirlos se gana algo más que buen planteamiento. Si no hay ciclos de peso negativo, entonces todo ciclo tiene costo no negativo y, en consecuencia, quitarle a un camino los ciclos que contenga no aumenta su costo. Luego, si existe algún camino de u a v , entonces existe uno *simple* de costo mínimo. Como en un grafo de N vértices hay una cantidad finita de caminos simples, el mínimo se alcanza y el problema tiene solución. La demostración de esta afirmación se propone como ejercicio al lector.

Vale la pena contrastar esta situación con la del Ejemplo 3.1.2, en donde exigir que los caminos fueran simples destruía la propiedad de subestructura óptima. Aquí la simplicidad de los caminos no es una restricción impuesta desde afuera sino una consecuencia de la hipótesis sobre los pesos y por eso no causa el mismo problema.

Se supondrá que $V = \{0, 1, \dots, N - 1\}$ y que el grafo se representa con una matriz de adyacencia $w[0..N][0..N]$ tal que $w[u][v]$ es el peso del arco (u, v) cuando este existe, $w[u][u] = 0$ y $w[u][v] = +\infty$ cuando no hay arco de u a v . Note que esta matriz *no* es necesariamente simétrica, pues el grafo es dirigido.

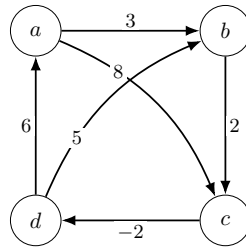


Figura 12. Un grafo dirigido y pesado con cuatro vértices y un arco de peso negativo. No tiene ciclos de peso negativo.

Ejemplo 3.8.1

Considere el grafo dirigido con 4 vértices de la Figura 12, en el cual el arco de c a d tiene peso negativo. El grafo no tiene ciclos de peso negativo: el único ciclo es $b \rightarrow c \rightarrow d \rightarrow b$, de costo $2 + (-2) + 5 = 5$. La matriz de costos mínimos entre todos los pares es la siguiente:

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
<i>a</i>	0	3	5	3
<i>b</i>	6	0	2	0
<i>c</i>	4	3	0	-2
<i>d</i>	6	5	7	0

Observe que el costo mínimo de *a* a *c* es 5 y no 8: conviene ignorar el arco directo y pasar por *b*. Del mismo modo, el costo mínimo de *c* a *a* es 4, alcanzado por el camino $c \rightarrow d \rightarrow a$ que aprovecha el arco de peso negativo.

Nota 3.8.3: ¿Por qué no basta con un algoritmo voraz?

El problema de caminos mínimos desde un único origen y el algoritmo voraz de Dijkstra que lo resuelve son el tema de la Sección 4.8. Ejecutar ese algoritmo una vez por cada vértice resolvería el problema de esta sección y en muchos casos sería la mejor opción disponible.

Sin embargo, la correctitud del algoritmo de Dijkstra depende de que los pesos sean no negativos: su decisión voraz consiste en dar por definitiva la distancia del vértice más cercano aún no procesado y un arco de peso negativo puede invalidar esa decisión más adelante. Con pesos arbitrarios, entonces, la estrategia deja de servir. El algoritmo de esta sección no impone esa restricción y esa es precisamente su ventaja. En los ejercicios se explora una tercera vía, debida a D. Johnson, que reformula los pesos para poder usar Dijkstra incluso cuando hay arcos negativos.

Nota 3.8.4: Sobre el nombre del algoritmo

El algoritmo que se obtiene al final de esta sección se conoce universalmente como *algoritmo de Floyd–Warshall*, por R. Floyd y S. Warshall, quienes lo publicaron de manera independiente en 1962. El registro histórico, sin embargo, es más generoso: B. Roy había descrito el mismo esquema en 1959, tres años antes. Por esa razón, en este texto se prefiere llamarlo *algoritmo de Roy–Floyd–Warshall*, con los tres nombres en orden cronológico.

Conviene precisar en qué consistió el aporte de cada uno. Roy y Warshall se ocuparon de la *clausura transitiva* de un grafo, es decir, de determinar qué vértices son alcanzables desde cuáles; Floyd fue quien observó que el mismo esquema de recurrencia sirve para calcular distancias mínimas y no solamente alcanzabilidad. No se trata del mismo problema, sino de la misma

idea aplicada a dos problemas distintos. En los ejercicios y en las notas del final del capítulo se explora hasta dónde llega esa idea.

Una advertencia práctica para el lector: en la literatura, en los cursos y en internet, el algoritmo aparece casi siempre bajo el nombre de Floyd–Warshall. Al buscar material complementario conviene usar ese nombre.

Este es un problema de optimización, de modo que la propiedad que hay que verificar es la subestructura óptima de la Definición 3.1.1. El problema la exhibe, junto con la de solapamiento, así que se sigue la Metodología 3.3.1; el paso de especificación ya quedó cumplido con el Problema 3.8.1.

Función objetivo. La salida del problema tiene dos parámetros, los vértices u y v , pero hacerlos variar no basta: es lo que ya hace la especificación y no produce subproblemas más simples. Hace falta un tercer parámetro que mida el progreso y la elección acertada –la de Roy, Warshall y Floyd– consiste en restringir cuáles vértices se permiten como intermedios. Para $0 \leq k \leq N$ y $u, v \in V$, se define:

$\phi(k, u, v)$: “costo mínimo de un camino de u a v cuyos vértices intermedios están todos en $\{0, \dots, k-1\}$.”

Es decir, k cuenta cuántos vértices están habilitados para ser usados de paso, comenzando por el vértice 0. Cuando $k = 0$ no se permite ningún vértice intermedio y cuando $k = N$ se permiten todos. Note que la restricción recae sobre los vértices *intermedios*: los extremos u y v son siempre libres.

Reformulación de la especificación. El objetivo del problema es calcular $\phi(N, u, v)$ para cada par de vértices $u, v \in V$, pues con $k = N$ la restricción sobre los vértices intermedios desaparece.

Planteamiento recurrente. El caso base es $k = 0$: sin vértices intermedios, el único camino posible de u a v es el arco directo, cuyo costo es $w[u][v]$ o el camino vacío cuando $u = v$, cuyo costo es 0. La convención adoptada para la matriz de adyacencia cubre las tres situaciones a la vez.

Suponga ahora que $k \neq 0$ y considere un camino de costo mínimo de u a v con vértices intermedios en $\{0, \dots, k-1\}$. Por la Nota 3.8.2 se puede suponer que es simple. La pregunta que organiza el análisis es si ese camino usa o no el vértice $k-1$, que es el que se acaba de habilitar:

- Si no lo usa, entonces sus vértices intermedios están en $\{0, \dots, k-2\}$ y su costo es $\phi(k-1, u, v)$.
- Si lo usa, entonces lo usa exactamente una vez, por ser simple. Eso permite partir el camino en dos: el tramo de u a $k-1$ y el tramo de $k-1$ a v . Ninguno de los dos tramos usa $k-1$ como vértice intermedio, de modo que ambos tienen sus intermedios en $\{0, \dots, k-2\}$, y cada uno debe ser de costo mínimo –de lo

contrario el camino completo no lo sería—. Su costo es entonces $\phi(k-1, u, k-1) + \phi(k-1, k-1, v)$.

Como no se sabe de antemano cuál de las dos posibilidades se da, se prefiere la más barata. La definición formal de ϕ se presenta para $0 \leq k \leq N$ y $u, v \in V$:

$$\phi(k, u, v) = \begin{cases} w[u][v] & , \text{ si } k = 0, \\ \phi(k-1, u, v) \downarrow \\ (\phi(k-1, u, k-1) + \phi(k-1, k-1, v)) & , \text{ si } k \neq 0. \end{cases}$$

La definición es notablemente más simple que la del problema del morral o la del agente viajero: un solo caso recurrente, sin condiciones sobre los datos. Toda la dificultad está en haber encontrado el tercer parámetro.

Correctitud de la función objetivo. La correctitud de la función objetivo se presenta en el Teorema 3.8.1.

Teorema 3.8.1

Sea G un grafo dirigido sin ciclos de peso negativo. Si $0 \leq k \leq N$ y $u, v \in V$, entonces $\phi(k, u, v)$ es el costo mínimo de un camino de u a v cuyos vértices intermedios están todos en $\{0, \dots, k-1\}$.

Demostración

Sea $L(k, u, v)$ el costo mínimo de un camino de u a v con vértices intermedios en $\{0, \dots, k-1\}$, con el convenio de que $L(k, u, v) = +\infty$ cuando no existe tal camino. Se procede por inducción sobre k .

Caso $k = 0$: los caminos de u a v sin vértices intermedios son el arco (u, v) , si existe, y el camino vacío cuando $u = v$. Por la convención adoptada para la matriz de adyacencia, $L(0, u, v) = w[u][v] = \phi(0, u, v)$.

Caso $k \neq 0$: suponga que $\phi(k-1, x, y) = L(k-1, x, y)$ para cualesquiera $x, y \in V$. Se demuestran las dos desigualdades. Note que, por tratarse de un problema de minimización, la mitad de composición produce la desigualdad $\phi \geq L$ y la de descomposición la contraria; en las secciones anteriores, que son de maximización, los papeles están invertidos.

- *Composición:* $\phi(k, u, v) \geq L(k, u, v)$. Basta ver que $L(k, u, v)$ no excede a ninguno de los dos argumentos del mínimo. Si alguno de ellos vale $+\infty$, la desigualdad correspondiente es inmediata, así que se pueden suponer finitos.

Para el primero: por hipótesis inductiva, $\phi(k-1, u, v) = L(k-1, u, v)$ es el costo de un camino de u a v con intermedios en $\{0, \dots, k-2\}$, que también los tiene en $\{0, \dots, k-1\}$; luego $L(k, u, v) \leq \phi(k-1, u, v)$.

Para el segundo: de nuevo por hipótesis inductiva, hay un camino de u a $k-1$ de costo $\phi(k-1, u, k-1)$ y otro de $k-1$ a v de costo $\phi(k-1, k-1, v)$, ambos con intermedios en $\{0, \dots, k-2\}$. Al concatenarlos se obtiene una caminata de u a v cuyos vértices interiores están en $\{0, \dots, k-1\}$, aunque puede repetir vértices. Quitarle los ciclos produce un camino simple de u a v cuyo costo no es mayor –pues ningún ciclo tiene costo negativo– y cuyos vértices intermedios son un subconjunto de los anteriores. Ese camino es admisible, de donde $L(k, u, v) \leq \phi(k-1, u, k-1) + \phi(k-1, k-1, v)$.

- *Descomposición:* $\phi(k, u, v) \leq L(k, u, v)$. Si no hay camino admisible, entonces se tiene $L(k, u, v) = +\infty$ y la desigualdad es inmediata. De lo contrario, sea P un camino simple de u a v de costo $L(k, u, v)$ con intermedios en $\{0, \dots, k-1\}$, el cual existe por la Nota 3.8.2. Si $k-1$ no es intermedio de P , entonces P tiene sus intermedios en $\{0, \dots, k-2\}$ y, por hipótesis inductiva, $\phi(k-1, u, v) \leq \text{cost}(P) = L(k, u, v)$; como $\phi(k, u, v) \leq \phi(k-1, u, v)$, se concluye. Si $k-1$ sí es intermedio de P , entonces aparece exactamente una vez por ser P simple, y P se parte en P_1 , de u a $k-1$, y P_2 , de $k-1$ a v . Ninguno de los dos tiene a $k-1$ como intermedio, luego ambos tienen sus intermedios en $\{0, \dots, k-2\}$ y, por hipótesis inductiva,

$$\phi(k-1, u, k-1) + \phi(k-1, k-1, v) \leq \text{cost}(P_1) + \text{cost}(P_2) = L(k, u, v),$$

de donde $\phi(k, u, v) \leq L(k, u, v)$.

El solapamiento es masivo en esta formulación: el cálculo de la capa k consulta N^2 veces la fila y la columna $k-1$ de la capa anterior. Es, de hecho, el problema de este capítulo en el cual una implementación que se limite a transcribir la definición de ϕ resulta más catastrófica.

3.8.1. Tabulación. Como se explica a continuación, la tabulación es una opción conveniente para computar ϕ .

¿Memorización o tabulación? La decisión es más fácil aquí que en las secciones anteriores. La ventaja propia de la memorización es calcular solo los valores que hacen falta, pero en este problema hacen falta casi todos: la salida pide $\phi(N, u, v)$ para *cada* par de vértices y eso arrastra prácticamente los $N \cdot N \cdot (N+1)$ estados de la función. No hay nada que ahorrar, y a cambio habría que pagar la profundidad de la recurrencia y el costo de un diccionario con esa cantidad de llaves. Se opta, entonces, por una tabulación.

De acuerdo con la definición recurrente de ϕ , se propone el diagrama de necesidades de la Figura 13. La tabulación es tridimensional: hay $N + 1$ capas, una por

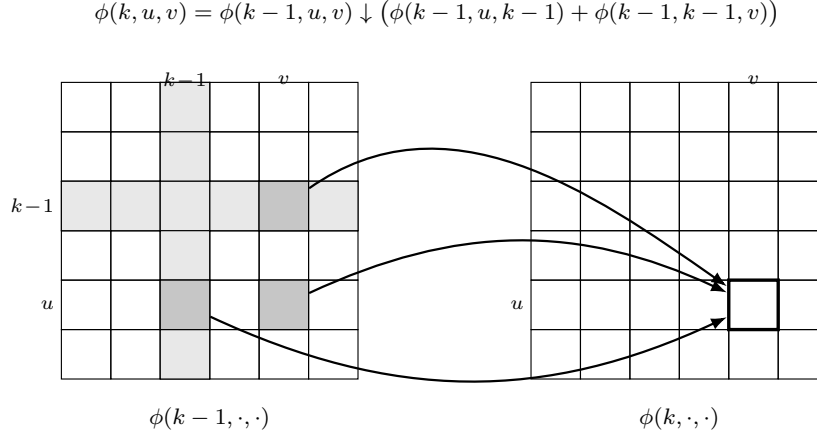


Figura 13. Diagrama de necesidades para la tabulación de ϕ en el problema de caminos mínimos entre todos los pares. La celda (u, v) de la capa k depende de tres celdas de la capa $k-1$: la misma celda (u, v) , la celda $(u, k-1)$ de la columna $k-1$ y la celda $(k-1, v)$ de la fila $k-1$.

cada valor de k , y cada capa es una matriz de $N \times N$ celdas indexada por parejas de vértices. La capa 0 es la matriz de adyacencia. Cada capa posterior se calcula por completo a partir de la anterior, lo cual sugiere llenar la tabulación capa por capa. Los invariantes del ciclo principal son:

$$P_0 : (\forall i, u, v \mid 0 \leq i < k \wedge u, v \in V : \text{tab}[i][u][v] = \phi(i, u, v)).$$

$$P_1 : 0 \leq k \leq N + 1.$$

El invariante P_0 indica que todas las capas anteriores a la capa k han sido llenadas con los valores correspondientes de ϕ . Los invariantes de los dos ciclos internos, que recorren los pares de vértices dentro de una capa, se proponen como ejercicio al lector. El diseño resulta en la función `apsp_tab` del Algoritmo 3.8.1.

Implementación. En la implementación se distinguen claramente la creación de la tabulación, sin valores, un primer paso para establecer los casos base y, finalmente, el proceso de calcular completamente la recurrencia en la tabla de la tabulación.

Algoritmo 3.8.1: Tabulación para ϕ

```

1 INF = float('inf')
2
3 def apsp_tab(N, w):
4     tab = [ [ [ None for _ in range(N) ] for _ in range(N) ]
5             for _ in range(N+1) ]
6     for u in range(N):
7         for v in range(N): tab[0][u][v] = w[u][v]
8     #  $P_0 \wedge P_1$ 
9     for k in range(1, N+1):
10        for u in range(N):
11            for v in range(N):
12                tab[k][u][v] = min(tab[k-1][u][v],
13                                   tab[k-1][u][k-1]+tab[k-1][k-1][v])
14    return tab[N]

```

La variable k recorre los valores de 1 a N y en la iteración k el vértice que se habilita como intermedio es $k-1$. Vale la pena insistir en esta correspondencia, pues es la fuente de errores más común al implementar el algoritmo. El valor de retorno es la última capa, que corresponde a $\phi(N, \cdot, \cdot)$.

Se ilustra el uso de la función con el grafo del Ejemplo 3.8.1:

```

1 >>> w = [ [ 0, 3, 8, INF ],
2 ...      [ INF, 0, 2, INF ],
3 ...      [ INF, INF, 0, -2 ],
4 ...      [ 6, 5, INF, 0 ] ]
5 >>> for row in apsp_tab(4, w): print(row)
6 [0, 3, 5, 3]
7 [6, 0, 2, 0]
8 [4, 3, 0, -2]
9 [6, 5, 7, 0]

```

Análisis. El análisis del algoritmo que tabula a ϕ se presenta en el Teorema 3.8.2.

Teorema 3.8.2

El llamado `apsp_tab(N, w)`:

1. Calcula el costo mínimo de un camino de u a v , para cada par de vértices $u, v \in V$.
2. Toma tiempo $O(N^3)$.
3. Usa espacio $O(N^3)$.

Demostración

Se propone como ejercicio al lector.

3.8.2. Reducción del espacio. El espacio $O(N^3)$ es escandaloso: guarda $N+1$ matrices cuando la respuesta es una sola. Como cada capa depende únicamente de la anterior, el argumento de la Sección 3.6 sugiere conservar dos capas y alternarlas, lo cual daría espacio $O(N^2)$. Resulta, sin embargo, que se puede hacer algo mejor y más simple: una sola matriz, sobreescrita en el lugar. Que esto sea correcto no es evidente, porque al sobreescribir se destruyen valores de la capa $k-1$ que todavía se van a necesitar. El siguiente resultado explica por qué no importa.

Teorema 3.8.3

Sea $1 \leq k \leq N$ y sean $u, v \in V$. Entonces

$$\phi(k, u, k-1) = \phi(k-1, u, k-1),$$

$$\phi(k, k-1, v) = \phi(k-1, k-1, v).$$

Es decir, la fila $k-1$ y la columna $k-1$ no cambian al pasar de la capa $k-1$ a la capa k .

Demostración

Observe primero que $\phi(k-1, k-1, k-1) = 0$. En efecto, el camino vacío de $k-1$ a $k-1$ tiene costo 0 y es admisible, y cualquier otro camino de $k-1$ a $k-1$ es un ciclo, cuyo costo es no negativo por hipótesis sobre G . Escriba $a = \phi(k-1, u, k-1)$ y $b = \phi(k-1, k-1, k-1)$ para abreviar; se acaba de observar que $b = 0$. Entonces se tiene:

$$\begin{aligned} \phi(k, u, k-1) &= a \downarrow (a + b) && \text{(definición de } \phi) \\ &= a \downarrow (a + 0) && (b = 0) \\ &= a \downarrow a && \text{(aritmética)} \\ &= a && \text{(idempotencia del mínimo).} \end{aligned}$$

La demostración de la segunda igualdad es análoga y se propone como ejercicio al lector.

La consecuencia práctica del Teorema 3.8.3 es inmediata. Los únicos valores de la capa $k-1$ que se consultan al calcular la capa k , además de la propia celda (u, v) , están en la fila $k-1$ y en la columna $k-1$. Si esas dos no cambian, entonces da lo

mismo leerlas de la capa $k-1$ o de la capa k ya parcialmente escrita. Se puede, por lo tanto, trabajar sobre una única matriz. El diagrama de necesidades correspondiente se presenta en la Figura 14. Los invariantes de los dos ciclos internos, para un valor

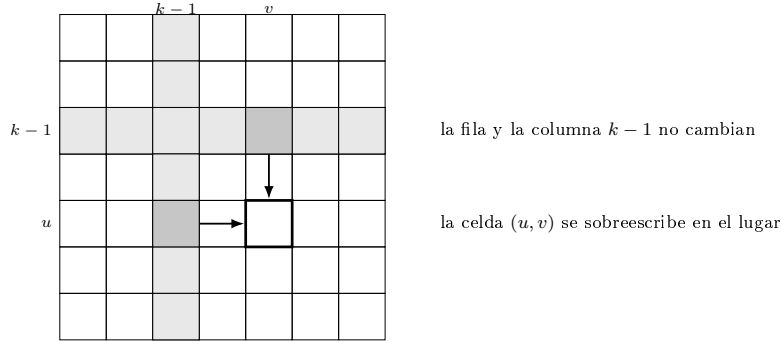


Figura 14. Diagrama de necesidades de la reducción del espacio en el problema de caminos mínimos entre todos los pares. Los valores que se consultan están en la cruz que forman la fila $k-1$ y la columna $k-1$, y por el Teorema 3.8.3 esa cruz no cambia al pasar de una capa a la siguiente.

fijo de k , describen una matriz que mezcla presente y pasado:

$$Q_0 : (\forall i, j \mid 0 \leq i < u \wedge j \in V : tab[i][j] = \phi(k, i, j)).$$

$$Q_1 : (\forall j \mid 0 \leq j < v : tab[u][j] = \phi(k, u, j)).$$

$$Q_2 : (\forall j \mid v \leq j < N : tab[u][j] = \phi(k-1, u, j)).$$

$$Q_3 : (\forall i, j \mid u < i < N \wedge j \in V : tab[i][j] = \phi(k-1, i, j)).$$

Las filas anteriores a la u ya tienen valores del presente (Q_0), las posteriores todavía tienen valores del pasado (Q_3) y la fila u está partida en dos por la columna v (Q_1 y Q_2). Sin el Teorema 3.8.3 esta mezcla sería fatal; con él, es inofensiva. El resultado es la función `apsp` del Algoritmo 3.8.2.

Algoritmo 3.8.2: Algoritmo de Roy–Floyd–Warshall

```

1 def apsp(N, w):
2     tab = [ [ w[u][v] for v in range(N) ] for u in range(N) ]
3     for k in range(1, N+1):
4         #  $Q_0 \wedge Q_1 \wedge Q_2 \wedge Q_3$ 
5         for u in range(N):
6             for v in range(N):

```

```

7         tab[u][v] = min(tab[u][v], tab[u][k-1]+tab[k-1][v])
8     return tab

```

Son seis líneas de código y esa brevedad explica buena parte de la popularidad del algoritmo. Note que la matriz de adyacencia se copia en lugar de modificarse: es una cortesía hacia quien invoca la función, que probablemente no espera que su grafo quede alterado.

Teorema 3.8.4

El llamado `apsp(N, w)`:

1. Calcula el costo mínimo de un camino de u a v , para cada par de vértices $u, v \in V$.
2. Toma tiempo $O(N^3)$.
3. Usa espacio $O(N^2)$.

Demostración

Se propone como ejercicio al lector.

Vale la pena registrar que esta es la tercera forma de reducción del espacio que aparece en el capítulo y que las tres son distintas. En la Sección 3.6 se conservó una sola columna recorriéndola en reversa; en la Sección 3.7, una sola fila más una variable auxiliar; aquí, la tabulación completa de una capa, sobreescrita en el lugar. Lo que las tres tienen en común es el método: mirar el diagrama de necesidades y preguntarse qué se puede descartar sin perder lo que hará falta.

3.8.3. Reconstrucción de una solución óptima. Como en la Sección 3.7, el algoritmo calcula el costo de los caminos óptimos pero no los exhibe. La reconstrucción es aquí particularmente elegante y no cuesta más que una matriz adicional.

La idea es mantener, junto con los costos, una matriz $nxt[0..N][0..N]$ tal que $nxt[u][v]$ es el vértice que sigue a u en un camino de costo mínimo de u a v . Inicialmente, cuando los únicos caminos disponibles son los arcos directos, $nxt[u][v]$ es v si el arco (u, v) existe y no está definido en caso contrario. Cada vez que la actualización de $tab[u][v]$ mejora el costo pasando por el vértice $k - 1$, el camino de u a v empieza como el camino de u a $k - 1$; por lo tanto, $nxt[u][v]$ debe tomar el valor de $nxt[u][k - 1]$. Con esa matriz, recuperar un camino consiste en seguir los apuntadores desde u hasta llegar a v .

Algoritmo 3.8.3: Reconstrucción de los caminos óptimos

```

1 def apsp_wit(N, w):
2     tab = [ [ w[u][v] for v in range(N) ] for u in range(N) ]
3     nxt = [ [ v if w[u][v]!=INF else None for v in range(N) ]
4             for u in range(N) ]
5     for k in range(1, N+1):
6         for u in range(N):
7             for v in range(N):
8                 if tab[u][k-1]+tab[k-1][v] < tab[u][v]:
9                     tab[u][v] = tab[u][k-1]+tab[k-1][v]
10                    nxt[u][v] = nxt[u][k-1]
11     return tab,nxt
12
13 def path(u, v, nxt):
14     ans = []
15     if nxt[u][v]!=None:
16         ans = [u]
17         while u!=v:
18             u = nxt[u][v]
19             ans.append(u)
20     return ans

```

El llamado `path(u, v, nxt)` retorna la lista vacía cuando no hay camino de u a v y de lo contrario la sucesión de vértices de un camino de costo mínimo, incluyendo los dos extremos. Con el grafo del Ejemplo 3.8.1, en el cual los vértices a, b, c, d se identifican con 0, 1, 2, 3:

```

1 >>> tab,nxt = apsp_wit(4, w)
2 >>> print(path(0, 2, nxt), tab[0][2])
3 [0, 1, 2] 5
4 >>> print(path(2, 0, nxt), tab[2][0])
5 [2, 3, 0] 4

```

Teorema 3.8.5

El llamado `apsp_wit(N, w)` toma tiempo $O(N^3)$ y espacio $O(N^2)$. Si `tab` y `nxt` son los valores que retorna, entonces el llamado `path(u, v, nxt)` retorna un camino de u a v de costo `tab[u][v]` y toma tiempo $O(N)$.

Demostración

Se propone como ejercicio al lector.

A diferencia de lo que sucede en la Nota 3.7.3, aquí no hay tensión entre reducir el espacio y reconstruir una solución: la matriz *nxt* ocupa $O(N^2)$, que es exactamente lo que ya ocupa la tabulación reducida. La razón de fondo es que el rastro de las decisiones se puede resumir en un solo vértice por par, mientras que en el problema de la subsecuencia común más larga hacía falta la tabla entera.

Nota 3.8.5: Detección de ciclos de peso negativo

La especificación del Problema 3.8.1 excluye los grafos con ciclos de peso negativo, pero nada impide ejecutar el Algoritmo 3.8.2 sobre uno de ellos. Los valores que se obtienen ya no corresponden a ϕ , pues ϕ no está bien definida en ese caso. Sin embargo, la diagonal de la matriz resultante permite detectar la situación: hay un ciclo de peso negativo en G si y solo si $tab[u][u] < 0$ para algún vértice u .

Esto convierte al algoritmo en una herramienta de verificación, además de una de cálculo, y es la manera usual de comprobar que la hipótesis de la especificación se cumple antes de confiar en el resultado. La demostración de esta afirmación se propone como ejercicio al lector.

Ejercicios

1. Demuestre la afirmación de la Nota 3.8.2: si G no tiene ciclos de peso negativo y existe algún camino de u a v , entonces existe un camino simple de u a v de costo mínimo.
2. Formule los invariantes de los dos ciclos internos del Algoritmo 3.8.1 y demuestre el Teorema 3.8.2.
3. Complete la demostración del Teorema 3.8.3 con la igualdad $\phi(k, k-1, v) = \phi(k-1, k-1, v)$.
4. Demuestre el Teorema 3.8.4. En particular, demuestre que los ciclos internos satisfacen los invariantes Q_0 , Q_1 , Q_2 y Q_3 .
5. Demuestre el Teorema 3.8.5.
6. Demuestre la afirmación de la Nota 3.8.5.
7. En el Algoritmo 3.8.2 el ciclo sobre k es el más externo de los tres. Explique con un ejemplo concreto por qué el algoritmo deja de ser correcto si se coloca

- como el más interno, es decir, si se recorre **for** $u \dots$ **for** $v \dots$ **for** k . ¿Qué invariante deja de cumplirse?
8. La *clausura transitiva* de un grafo dirigido $G = (V, E)$ es la relación que indica, para cada par de vértices u, v , si existe algún camino de u a v . Este es el problema que resolvieron Roy y Warshall (Nota 3.8.4).
 - a) Especifique el problema.
 - b) Adapte la función objetivo ϕ de esta sección reemplazando los costos por valores Booleanos. ¿En qué se convierten el mínimo y la suma?
 - c) Escriba el algoritmo resultante. ¿Cuántas líneas de código lo separan del Algoritmo 3.8.2?
 9. Suponga ahora que el peso de un arco representa la capacidad de un canal de comunicación y que la capacidad de un camino es la del arco de menor capacidad que lo compone. El problema de capacidad máxima pide, para cada par de vértices, la mayor capacidad posible de un camino que los une.
 - a) Especifique el problema y adapte la función objetivo.
 - b) Escriba el algoritmo resultante.
 10. Compare las definiciones recurrentes de esta sección y de los dos ejercicios anteriores. En los tres casos la recurrencia tiene la misma forma, con dos operaciones que cambian de un problema a otro. Identifique esas dos operaciones en cada caso y determine cuál es el elemento neutro de cada una. Investigue qué es un *semianillo* y explique en qué sentido los tres problemas son instancias de un mismo problema algebraico.
 11. Investigue sobre el algoritmo de Bellman–Ford, que resuelve el problema de caminos mínimos desde un único origen admitiendo pesos negativos.
 - a) Identifique en él la metodología de la Sección 3.3: ¿cuál es su función objetivo y cuál su definición recurrente?
 - b) ¿Cuál es su complejidad temporal? Compárela con la de ejecutarlo una vez por cada vértice del grafo frente a la del Algoritmo 3.8.2. ¿Para qué grafos conviene cada opción?
 12. ¿Qué sucede si el grafo de entrada es no-dirigido? En particular, explique qué le ocurre a la hipótesis de la especificación cuando hay una arista de peso negativo, y concluya para qué clase de grafos no-dirigidos tiene sentido el problema.
 13. 🌈 Investigue sobre el algoritmo de Johnson, mencionado en la Nota 3.8.3, que resuelve el problema de esta sección con pesos negativos ejecutando el algoritmo de Dijkstra una vez por cada vértice, después de reformular los pesos.
 - a) Explique en qué consiste la reformulación de los pesos y por qué preserva los caminos de costo mínimo.
 - b) Determine su complejidad temporal y compárela con la del Algoritmo 3.8.2. ¿Para qué grafos es preferible?
-

3.9. El problema del agente viajero

El problema del agente viajero es un problema de optimización combinatoria en grafos. A diferencia del problema de la Sección 3.8, aquí no se busca el costo de un camino entre dos vértices sino el de un recorrido que pase por todos ellos.

Nota 3.9.1: Grafos completos y circuitos

Las nociones de grafo, de arco y de camino son las de la Nota 3.8.1. A diferencia de lo que sucede en la Sección 3.8, en esta sección los grafos se suponen no-dirigidos.

Se dice que un grafo es *completo* cuando hay un arco entre cualquier par de vértices (distintos). Un *circuito* en un grafo es un camino que visita exactamente una vez cada uno de sus vértices y al finalizar regresa al vértice de origen.

Nota 3.9.2: El problema del agente viajero

El problema del agente viajero (en inglés, *travelling salesman problem* o *TSP*) se ocupa de responder la siguiente pregunta: dada una colección de ciudades y las distancias entre cada par de ellas, ¿cuál es la ruta más corta posible que visita cada ciudad exactamente una vez y al finalizar regresa a la ciudad origen?

K. Menger parece ser el primer matemático en ocuparse del problema, en Viena hacia 1930; la formulación moderna se le atribuye a H. Whitney, a partir de una charla que dio en Princeton en 1934. Desde entonces ha sido estudiado exhaustivamente en las áreas de optimización y teoría de la complejidad, hasta el punto de ser uno de los problemas más trabajados de la disciplina.

De forma general, el sistema de ciudades y rutas en el problema del agente viajero se puede abstraer en un grafo en el cual las ciudades se modelan como vértices y los caminos entre ciudades como arcos. En este problema se menciona una noción de medida para las rutas. Para incluir este elemento en el modelado del problema, se puede optar por tener una función de distancia sobre los arcos del grafo. Note que en el problema del agente viajero se desea encontrar un circuito de costo mínimo en un grafo completo con respecto a una función de distancia sobre sus arcos.

Problema 3.9.1: El problema del agente viajero

Entrada: Un grafo $G = (V, E)$ completo, con $|V| \geq 3$ vértices y función de peso sobre los arcos $w : E \rightarrow \mathbb{R}_{\geq 0}$.

Salida: El costo mínimo (con respecto a w) de un circuito en G .

La condición $|V| \geq 3$ no es un tecnicismo. Como se verá enseguida, un circuito está compuesto por exactamente $|V|$ arcos distintos y un grafo completo con dos vértices tiene un solo arco; con menos de tres vértices, entonces, no hay circuitos y el problema no tiene sentido.

Note además que la salida es el *costo* de un circuito óptimo y no el circuito mismo. Como en la Sección 3.7 y en la Sección 3.8, exhibir una solución óptima es un problema distinto, que se enuncia por separado.

Problema 3.9.2: Reconstrucción de un circuito de costo mínimo

Entrada: Un grafo $G = (V, E)$ completo, con $|V| \geq 3$ vértices y función de peso sobre los arcos $w : E \rightarrow \mathbb{R}_{\geq 0}$.

Salida: Un circuito de costo mínimo (con respecto a w) en G .

El Problema 3.9.2 se propone como ejercicio al final de la sección, una vez resuelto el Problema 3.9.1.

La salida del Problema 3.9.1 se puede precisar matemáticamente y exhibir así, entre otras cosas, su estructura combinatoria. La primera observación importante es que en un circuito *no* es importante identificar cuál es el vértice de “inicio”, pues cualquiera de los vértices en dicho circuito puede ser considerado de inicio (¿por qué?). Entonces, para el modelado del problema es natural volcar el interés en los arcos del grafo. Suponga que se cuenta con un predicado *circuit* que indica, para cualquier $C \subseteq E$, si C es un circuito en G . Es decir, $circuit(G, C)$ (o, simplemente, $circuit(C)$ cuando G se entiende del contexto) es cierto únicamente cuando C es un circuito en G . Además, se define la función *cost* para cualquier subconjunto $C \subseteq E$ y función de peso $w : E \rightarrow \mathbb{R}_{\geq 0}$ de la siguiente manera:

$$cost(C) = (+e \mid e \in C : w(e)).$$

Es decir, $cost(C)$ es la suma de los pesos de los arcos en C . Con base en estas definiciones, se formula la salida del problema del agente viajero de la siguiente manera:

$$(\downarrow C \mid C \subseteq E \wedge circuit(C) : cost(C)).$$

Esta fórmula expresa que se desea calcular el mínimo costo entre todos los circuitos en G . Note que la cantidad de posibles circuitos está acotada por $2^{|E|}$, la cantidad de

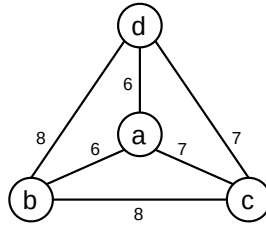
subconjuntos de E . Más precisamente, está acotada por la cantidad de subconjuntos de E de tamaño $|V|$, dado que un circuito tiene exactamente $|V|$ arcos; es decir, por

$$\binom{|E|}{|V|},$$

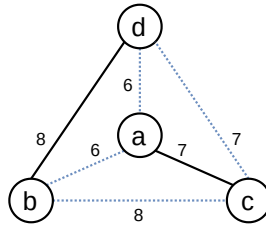
que denota la cantidad de subconjuntos de tamaño $|V|$ en un conjunto de tamaño $|E|$. En esta observación yace la naturaleza combinatoria del problema del agente viajero.

Ejemplo 3.9.1

A continuación se presenta un grafo completo con 4 vértices a, b, c, d .



El circuito $a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$ es óptimo y tiene costo 27:



3.9.1. Diseño de una función objetivo. Necesariamente hay un circuito de costo mínimo cuando G tiene al menos tres vértices; suponga que $C \subseteq E$ es un circuito óptimo de G en este sentido. Sin pérdida de generalidad, también suponga que

$$C = \{(u_0, u_1), (u_1, u_2), \dots, (u_{N-1}, u_0)\},$$

en donde $N = |V|$ y $V = \{u_0, u_1, u_2, \dots, u_{N-1}\}$. Observe que el circuito

$$u_0, u_1, u_2, \dots, u_{N-1}, u_0$$

es óptimo si y solo si el camino de u_0 a u_{N-1} en este circuito es óptimo, entre todos los caminos posibles de u_0 a u_{N-1} que incluyen los demás vértices de G (de lo contrario, no sería de costo mínimo ¿por qué?). Esta es la propiedad de la

subestructura óptima que se “explotará” para formular una función objetivo que ayude a resolver el problema del agente viajero usando programación dinámica.

Función objetivo. Dado que en un circuito no importa desde dónde se inicie el recorrido, se fija arbitrariamente un vértice $s \in V$ como punto de partida. Se usa el hecho de que G tiene al menos tres vértices; de lo contrario el problema es trivial, ya que dicho circuito no existe.

Para $X \subseteq V$ y $u \in X$:

$\phi(u, X)$: “mínimo costo de un camino (simple) que inicia en s , visita cada uno de los vértices en X y termina en u .”

Basada en la propiedad de la subestructura óptima, la observación importante para aceptar la formulación de ϕ como (potencialmente) útil es que todo circuito óptimo se puede construir con base en rutas óptimas que visiten todos los vértices del grafo dado.

Reformulación de la especificación. El objetivo es entonces calcular el valor de la siguiente expresión:

$$(\downarrow u \mid u \in V \setminus \{s\} : \phi(u, V \setminus \{s\}) + w(u, s)).$$

Entre todos los posibles caminos que inician en s , y que incluyen todos los demás vértices del grafo y terminan en u , se desea calcular aquel camino que sumando la ruta de regreso directa de u a s sea de costo mínimo. Note que cuando $V = \{s\}$ la minimatoria se toma sobre un rango vacío y, en consecuencia, su valor es $+\infty$: la identidad del mínimo. Esto refleja correctamente que un grafo con un único vértice no tiene circuitos.

El caso $|V| = 2$ merece una advertencia y es la razón por la cual la especificación exige $|V| \geq 3$. Si $V = \{s, u\}$, la expresión anterior se reduce a $\phi(u, \{u\}) + w(u, s) = 2w(s, u)$, que es un valor finito. Pero ese valor no corresponde a ningún circuito: el recorrido $s \rightarrow u \rightarrow s$ usa dos veces el único arco disponible, mientras que un circuito consta de $|V|$ arcos *distintos*. La reformulación, entonces, solo es fiel a la especificación cuando el grafo tiene al menos tres vértices.

Planteamiento recurrente. Una ruta de costo mínimo de s a cualquier otro vértice debe estar conformada por subrutas óptimas. Es decir, cada ruta que inicie en s en dicho camino, también debe ser de costo mínimo.

La definición formal de ϕ se presenta para $X \subseteq V \setminus \{s\}$ y $u \in V \setminus \{s\}$:

$$\phi(u, X) = \begin{cases} +\infty & , \text{ si } u \notin X, \\ w(s, u) & , \text{ si } X = \{u\}, \\ (\downarrow v \mid v \in X \setminus \{u\} : \phi(v, X \setminus \{u\}) + w(v, u)) & , \text{ si } u \in X \wedge |X| \geq 2. \end{cases}$$

Se consideran tres casos para definir ϕ . Hay dos opciones entre u y X : que u sea elemento de X o no y, cuando lo es, que sea el único o no. Es imposible que haya un camino que visite todos los elementos de X y que termine en u cuando $u \notin X$; la forma de expresar que esto es imposible es con el valor $+\infty$ dado que es la identidad del mínimo en los reales. Si u es el único elemento de X , solo hay un camino de s a u : el camino directo entre s y u , el cual tiene costo $w(s, u)$. Cuando en X hay otros elementos en adición a u , se apuesta por todos los caballos: se prefiere aquél vértice $v \in X \setminus \{u\}$ que en dicho camino conecte directamente con u y para el cual el camino desde s permita construir un camino hasta u de costo mínimo.

Correctitud de la función objetivo. El Teorema 3.9.1 presenta la propiedad de correctitud del diseño propuesto para ϕ .

Teorema 3.9.1

Sean $s \in V$ y $X \subseteq V \setminus \{s\}$. Si $u \in X$, entonces $\phi(u, X)$ es el costo mínimo de un camino (simple) que inicia en s , visita cada uno de los vértices en X y termina en u .

Demostración

Sea $L(u, X)$ el costo mínimo de un camino simple que inicia en s , visita cada uno de los vértices de X y termina en u , con el convenio de que $L(u, X) = +\infty$ cuando no existe tal camino. El objetivo es demostrar que $\phi(u, X) = L(u, X)$, y se procede por inducción sobre $|X|$.

Si $u \notin X$ no hay ningún camino que termine en u y visite exactamente los vértices de X , luego $L(u, X) = +\infty = \phi(u, X)$. Si $X = \{u\}$, el único camino que inicia en s , visita únicamente a u y termina en u es el arco (s, u) , de costo $w(s, u)$; luego $L(u, \{u\}) = w(s, u) = \phi(u, \{u\})$.

Suponga entonces que $u \in X$ y $|X| \geq 2$, y que $\phi(v, Y) = L(v, Y)$ para todo Y con $|Y| < |X|$ y todo v . Escriba $Y = X \setminus \{u\}$ y note que $|Y| = |X| - 1$. Se demuestran las dos desigualdades.

- *Composición:* $\phi(u, X) \geq L(u, X)$. Basta ver que $L(u, X)$ no excede a ninguno de los valores sobre los cuales se toma la minimatoria. Sea $v \in Y$ y suponga que $\phi(v, Y)$ es finito, pues de lo contrario la desigualdad correspondiente es inmediata. Por hipótesis inductiva hay un camino simple P que inicia en s , visita exactamente los vértices de Y , termina en v y tiene costo $\phi(v, Y)$. Como $u \notin Y$, el camino que resulta de extender P con el arco (v, u) sigue siendo simple, inicia en s , visita exactamente

los vértices de X y termina en u ; su costo es $\phi(v, Y) + w(v, u)$. Luego $L(u, X) \leq \phi(v, Y) + w(v, u)$ para cada $v \in Y$.

- *Descomposición:* $\phi(u, X) \leq L(u, X)$. Si no hay camino admisible la desigualdad es inmediata. De lo contrario, sea P un camino simple de costo $L(u, X)$ que inicia en s , visita exactamente los vértices de X y termina en u . Como $|X| \geq 2$, el vértice que precede a u en P es algún $v \in Y$. Al quitarle a P su último arco queda un camino simple que inicia en s , visita exactamente los vértices de Y y termina en v , de modo que su costo no es menor que $L(v, Y)$. Entonces se tiene:

$$\begin{aligned}
 L(u, X) &= L(u, X) - w(v, u) + w(v, u) && \text{(aritmética)} \\
 &\geq L(v, Y) + w(v, u) && \text{(observación anterior)} \\
 &= \phi(v, Y) + w(v, u) && \text{(hipótesis inductiva)} \\
 &\geq \phi(u, X) && (v \in Y).
 \end{aligned}$$

En cualquier caso, $\phi(u, X) = L(u, X)$.

¿Memorización o tabulación? La función ϕ es distinta a las que se han usado para resolver los problemas anteriores. En particular, tiene como parámetro un conjunto. Como tal, esta definición se puede implementar directamente en un lenguaje de programación (usando conjuntos disponibles en sus librerías o implementando la estructura de datos correspondiente). No resulta fácil pensar cómo tabular cuando en una de las dos dimensiones hay un conjunto. Tampoco resulta fácil pensar cómo implementar una memorización eficiente cuando el acceso a los datos depende de una colección y que esta colección puede “mutar” entre los distintos llamados recurrentes. En realidad, estas son noticias *parcialmente* malas. La noticia parcialmente buena es que bajo ciertas suposiciones acerca de la cantidad de vértices en el grafo, los conjuntos se pueden representar con números naturales y las operaciones básicas sobre ellos (al menos las que se requieren para calcular la función objetivo) se pueden realizar en tiempo constante. Entonces, antes de proponer una solución con programación dinámica, se estudia cómo representar conjuntos y algunas de sus operaciones con números naturales.

3.9.2. Especificación de conjuntos con máscaras de bits. El propósito de esta sección es mostrar cómo, bajo ciertas suposiciones, se pueden usar máscaras y operaciones de bits para representar conjuntos y operaciones sobre ellos. En particular, para un conjunto de N elementos, es deseable contar con las siguientes operaciones:

- Representación del conjunto completo o *universal*, junto con todos sus subconjuntos.
- Sustraer un elemento de un conjunto.
- Consultar si un elemento hace parte de un conjunto.

- Determinar si un conjunto es el conjunto unitario determinado por un elemento dado.

Estando disponible y de manera eficiente, la funcionalidad enumerada sería suficiente para implementar un algoritmo con programación dinámica que permita calcular la función ϕ eficientemente. Al menos, esta es la intención.

Las máscaras de bits (i.e., secuencias de bits) ofrecen una posibilidad si la cantidad de elementos N está acotada: un conjunto universal con N elementos se puede representar con N bits. Es más, cualquier conjunto con a lo sumo N elementos se puede también representar con una secuencia de bits de tamaño N . Sin pérdida de generalidad, se puede suponer que hay un orden entre los elementos del conjunto universal que se quiere representar. Bajo esta suposición, se puede asumir que el conjunto a codificar es $U = \{0, 1, \dots, N - 1\}$ de tal forma que 0 identifica el primer elemento de dicho conjunto, 1 el segundo y así, con $N - 1$ representando el último elemento en este orden arbitrario. Una secuencia de bits de tamaño N se puede, entonces, usar como función característica para representar exactamente un subconjunto de U : el n -ésimo bit, yendo del menos significativo al más significativo, indica si el n -ésimo elemento de U hace parte del subconjunto, para $0 \leq n < N$.

Ejemplo 3.9.2

Considere el conjunto $\{a, b, c, d\}$ de etiquetas. Suponga que las etiquetas tienen el siguiente orden:

$$a < b < c < d.$$

Entonces:

- La secuencia 1111 representa el conjunto $\{a, b, c, d\}$.
- La secuencia 1010 representa el conjunto $\{b, d\}$.
- La secuencia 0100 representa el conjunto unitario $\{c\}$.
- La secuencia 0000 representa el conjunto vacío $\emptyset$.

Entonces, en el problema del agente viajero se puede suponer que el conjunto de vértices $V = \{v_0, v_1, \dots, v_{N-1}\}$ y sus subconjuntos, se pueden representar con máscaras de bits de tamaño N (fijando algún orden sobre V).

En un lenguaje de programación, como Python, se puede construir rápidamente un conjunto universo para cualquier $N \in \mathbb{N}$ haciendo corrimiento de bits y una resta (ojo, no es la única manera de hacerlo). A continuación se presenta la función `universe` que dado dicho N , retorna el número entero cuya representación en base 2 es una secuencia de exactamente N unos:

```
1 def universe(N): return (1<<N)-1
```

Por ejemplo, `universe(3)` es 7 dado que su representación en base 2 es 111. La expresión `1<<N` calcula el número natural 2^N . Al restarle 1, se “apaga” el único bit existente y todos los 0 a su derecha de “encienden”, resultando en una secuencia de N unos.

El siguiente reto es determinar si un bit específico está encendido o apagado en una máscara de bits. Suponiendo que se cuenta con una máscara de N bits, para cualquier $0 \leq n < N$, se puede identificar si el n -ésimo bit está encendido con una disyunción bit a bit.

```
1 def is_elt(n, X): return (X|(1<<n))==X
```

El llamado `is_elt(n, X)` es cierto cuando el n -ésimo bit del código binario del número natural X es 1, suponiendo que n es una posición en la máscara de bits correspondiente a X .

De una manera muy similar, se puede apagar el n -ésimo bit en una máscara de bits.

```
1 def remove_elt(n, X): return X-(1<<n) if is_elt(n, X) else X
```

Si ese bit está encendido, entonces se apaga; de lo contrario, la máscara no se modifica.

Finalmente, determinar si una máscara representa un conjunto unitario dado es fácil de calcular: en la máscara correspondiente al conjunto unitario $\{n\}$ únicamente hay un bit encendido, exactamente el n -ésimo.

```
1 def singleton(n, X): return X==(1<<n)
```

Estas cuatro operaciones, basadas en comparación y aritmética de bits, son eficientes en la práctica cuando N no supera el tamaño de la palabra en un procesador. Por ejemplo, en procesadores de 32 bits estas operaciones se pueden implementar directa y eficientemente siempre y cuando $N \leq 32$. Lo mismo sucede con procesadores de 64 bits, etc. De lo contrario, se deberá incurrir en procedimientos adicionales que difícilmente serán de orden constante.

3.9.3. Diseño de un algoritmo con memorización. Bajo la suposición de que se pueden representar los subconjuntos de V con máscaras de bits, se obtiene casi directamente un algoritmo para implementar ϕ con memorización. Concretamente, se supondrá que G tiene N vértices $V = \{0, 1, \dots, N-1\}$ y que su representación es una matriz de adyacencia $w[0..N][0..N]$ tal que para cualquier $u, v \in V$, se tiene $w[u][v] = w[v][u] = w(u, v)$ y $w[u][u] = 0$. Es decir, los vértices se identifican con los primeros N números naturales y la función de peso entre vértices se especifica con una matriz cuadrada y simétrica en donde cada entrada indica el peso del arco entre los vértices correspondientes; esta matriz es 0 en su diagonal.

El Algoritmo 3.9.1 presenta las funciones `phi_memo` y `tsp` que resuelven el problema del agente viajero para un grafo de N vértices y función w de peso en los arcos. La función principal es `tsp`, la cual recibe estos dos parámetros (se pudo también optar por tener estos dos parámetros como variables globales, simplificando –pero oscureciendo en este caso en particular– el código). La función `phi_memo` es la solución por memorización que implementa la función ϕ . Note que el vértice 0 juega el papel del vértice s en la especificación de la función ϕ .

Implementación.

Algoritmo 3.9.1: Memorización para el agente viajero

```

1 INF = float('inf')
2
3 def phi_memo(N, w, u, X, mem):
4     ans, key = None, (u, X)
5     if key in mem: ans = mem[key]
6     else:
7         if not(is_elt(u, X)): ans = INF
8         elif singleton(u, X): ans = w[0][u]
9         else:
10            ans, Y = INF, remove_elt(u, X)
11            for v in range(1, N):
12                if is_elt(v, Y):
13                    ans = min(ans, phi_memo(N, w, v, Y, mem) + w[v][u])
14            mem[key] = ans
15     return ans
16
17 def tsp(N, w):
18     ans = INF
19     X = remove_elt(0, universe(N))
20     mem = dict()
21     for u in range(1, N):
22         ans = min(ans, phi_memo(N, w, u, X, mem) + w[u][0])
23     return ans

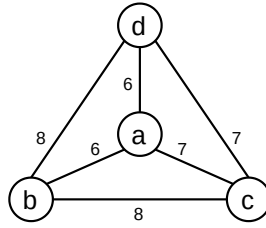
```

Un llamado `tsp(N, w)` inicialmente construye en X el conjunto de todos los vértices excepto el vértice de partida, es decir, el conjunto $\{1, \dots, N-1\}$. Iniciando con la memoria vacía, se itera sobre cada uno de los vértices en este conjunto tratando de encontrar aquella combinación que permita construir un circuito de costo mínimo que inicie en 0 y termine en 0, agotando todos los demás vértices en X . Esto corresponde, tal cual, con la reformulación de la especificación en la Sección 3.9.1.

El llamado `phi_memo(N, w, u, X, mem)` calcula la función $\phi(u, X)$ para el grafo de N vértices con función de peso $w[0..N][0..N]$, usando la memorización en el diccionario `mem`. Las llaves del diccionario son parejas en donde el primer argumento corresponde a un identificador de vértice y el segundo a una máscara de bits (representando un conjunto de vértices). Si el valor asociado a la llave (u, X) está presente en el diccionario, se usa directamente como valor de retorno. De lo contrario, se calcula el valor correspondiente a esta llave y se almacena en el diccionario antes de retornarlo. Note que se calcula este valor con base en cada uno de los tres casos que hacen parte de la definición recurrente de ϕ .

Ejemplo 3.9.3

A continuación se presenta el grafo completo con 4 vértices a, b, c, d del Ejemplo 3.9.1.



Este es un grafo con $N = 4$ vértices y función de peso w dada por la siguiente matriz de adyacencia:

```

1 w = [ [ 0, 6, 7, 6 ],
2       [ 6, 0, 8, 8 ],
3       [ 7, 8, 0, 7 ],
4       [ 6, 8, 7, 0 ] ]

```

El valor calculado por la función `tsp` en este caso es 27:

```

1 >>> tsp(4, w)
2 27

```

Análisis. La correctitud de `tsp` depende directamente de la correctitud de `phi_memo`, bajo las siguientes condiciones:

$$C_0 : (\forall u, X \mid (u, X) \in \text{mem} : \text{mem}[(u, X)] = \phi(u, X)).$$

$$C_1 : (\forall u, X \mid (u, X) \in \text{mem} : 1 \leq u < N \wedge 0 \leq X < 2^N).$$

Las condiciones C_0 y C_1 son condiciones de representación, respectivamente, de la memorización y de conjuntos con máscaras de bits. La condición C_0 indica que los valores almacenados en la memoria compartida son correctos en relación con ϕ . La condición C_1 indica que los valores utilizados como índice de la memoria son los

esperados: un vértice distinto al de inicio y una máscara de bits para el conjunto de N vértices. La correctitud de `phi_memo` consiste, básicamente, en garantizar que estas dos condiciones se mantienen a lo largo de todos los llamados recurrentes.

Teorema 3.9.2

Para $V = \{0, 1, \dots, N - 1\}$ y función de peso $w[0..N][0..N]$, el llamado `phi_memo(N, w, u, X, mem)` con $u \in V \setminus \{0\}$ y $0 \leq X < 2^N$:

1. Preserva las condiciones C_0 y C_1 , y
2. Calcula $\phi(u, X)$.

Demostración

Se propone como ejercicio para el lector.

El diccionario `mem` tiene a lo sumo $N \cdot 2^N$ llaves y el valor asociado a cada una de ellas se calcula exactamente una vez. Suponiendo que el acceso al diccionario y las operaciones sobre la máscara de bits son de orden constante, ese cálculo toma tiempo $O(N)$ por las líneas de código 11-13. Luego, el trabajo total realizado por todos los llamados a `phi_memo` es $O(N^2 \cdot 2^N)$. Como el ciclo en `tsp` comparte una única memoria `mem`, este trabajo total no se multiplica por la cantidad de iteraciones y la complejidad temporal del llamado `tsp(N, w)` es también $O(N^2 \cdot 2^N)$. La complejidad espacial es $O(N \cdot 2^N)$, dominada por la cantidad de llaves del diccionario. Los detalles de estas afirmaciones se proponen como ejercicio para el lector.

Teorema 3.9.3

El llamado `tsp(N, w)` toma tiempo $O(N^2 \cdot 2^N)$ y espacio $O(N \cdot 2^N)$.

Demostración

Se propone como ejercicio para el lector.

Nota 3.9.3: El algoritmo de Bellman–Held–Karp

El algoritmo desarrollado en esta sección se conoce como *algoritmo de Held–Karp* o, con más justicia, como *algoritmo de Bellman–Held–Karp*. Fue publicado en 1962 de manera independiente por R. Bellman, por un lado, y por M. Held y R. Karp, por el otro.

Que sea de Bellman cierra un arco que el capítulo abrió en la Nota 3.0.1. La Nota 3.0.2 señala que en programación dinámica el nombre llegó antes que casi todos los algoritmos; este es la excepción, pues lo diseñó quien puso el nombre, doce años después de haberlo escogido.

Vale la pena anotar, por último, que la cota $O(N^2 \cdot 2^N)$ sigue siendo, más de sesenta años después, la mejor conocida para el caso general del problema. Bajar la base de la exponencial por debajo de 2 es un problema abierto célebre, que ha resistido todos los intentos.

Ejercicios

1. Encuentre 5 ejemplos cotidianos de grafos, y explique cuáles son los vértices y arcos en cada uno de ellos. En cada caso, explique si el grafo es dirigido o no-dirigido, y si es completo. Si hay una noción de peso, distancia o importancia, ¿cuál es?
2. Suponga que se cuenta con un circuito en un grafo $G = (V, E)$. Explique por qué cada uno de los vértices en dicho circuito puede ser considerado como vértice inicial del circuito.
3. Proponga una definición formal para el predicado $circuit(C)$ relativa a un grafo $G = (V, E)$ tal que $C \subseteq E$. Si prefiere, puede tratar $circuit$ como una función Booleana $circuit : 2^E \rightarrow \mathbb{B}$, en donde 2^E denota la colección de subconjuntos de E .
4. Explique por qué la cantidad máxima de circuitos en un grafo completo $G = (V, E)$ está acotada por $\binom{|E|}{|V|}$. ¿Son circuitos todos los subconjuntos de E de tamaño $|V|$? ¿Por qué?
5. Enumere todos los circuitos del grafo en el Ejemplo 3.9.1 sin distinguir entre rotaciones (¿cuántos hay?). Justifique que el circuito de costo mínimo allí es 27.
6. En la definición recurrente de ϕ se incluye como caso base $\phi(u, X) = +\infty$ cuando $u \notin X$. En realidad, este caso se incluye por completitud de ϕ dado que $X \subseteq V \setminus \{s\}$ y $u \in V \setminus \{s\}$. Explique por qué este caso no es necesario para calcular la salida del problema. En particular, justifique cómo ϕ se puede definir, sin introducir inconsistencias matemáticas, únicamente para aquellos casos en los cuales $u \in X$.

7. Es posible obtener una versión distinta de la función `remove_elt` usando exclusivamente operaciones sobre bits (sin operaciones aritméticas). Investigue acerca de esta operación y proponga una versión alternativa de `remove_elt`.
 8. ¿Cómo se puede iterar sobre todos los subconjuntos de un conjunto de tamaño N con máscaras de bits? Diseñe un algoritmo que itere sobre todas las posibles máscaras de 2 bits y calcule la tabla de verdad para la disyunción y conjunción.
 9. Demuestre el Teorema 3.9.2.
 10. Demuestre el Teorema 3.9.3.
 11. Resuelva el Problema 3.9.2 extendiendo el Algoritmo 3.9.1 para que, además del costo de un circuito óptimo, retorne un circuito que lo alcance. ¿Cuánto espacio adicional requiere? Compare su respuesta con lo que sucede en la Sección 3.7 y en la Sección 3.8 (Nota 3.7.3).
 12. Diseñe un algoritmo de programación dinámica con tabulación para ϕ y un algoritmo iterativo para resolver el problema del agente viajero. Estime su complejidad temporal y espacial.
-

Notas del capítulo y referencias

La noción de programación dinámica cobra un aire más general cuando se considera más allá de la computación y del estudio de algoritmos eficientes para implementar cierto tipo de funciones recurrentes. Para el estudio de fenómenos y problemas económicos, como originalmente nace, es un método recursivo para resolver problemas de decisión secuenciales. Se le conoce también como *inducción reversa* (en inglés, *backward induction*), y es empleada para encontrar reglas de decisión óptimas en juegos, equilibrios perfectos de juegos multi-agente y equilibrios competitivos en modelos económicos dinámicos. El texto autobiográfico de R. Bellman [Bel84] incluye apartes de los orígenes de la programación dinámica en este contexto más general, al igual que algunos trabajos más recientes en Economía como el de J. Rust [Rus]. El aparte textual que se presenta en la Nota 3.0.1 es tomado de [Bel84, p. 159]; la traducción al castellano es responsabilidad del autor. Las dos obras que menciona la Nota 3.0.2 son el libro de 1957 en el cual Bellman fija la técnica y enuncia el principio de optimalidad [Bel57], y el artículo del año siguiente en el cual aparece el algoritmo de caminos mínimos que hoy lleva su nombre junto con el de L. Ford [Bel58].

Las propiedades de subestructura y de solapamiento se plantean casi siempre para estudiar qué problemas son candidatos a resolverse con programación dinámica, y lo mismo sucede con las dos técnicas de implementación, la memorización (en inglés, *top-down*) y la tabulación (en inglés, *bottom-up*), que se comparan en la Sección 3.2. Estos pares de temas se encuentran explicados, de manera similar a la

seguida en este capítulo, en [CLRS22] y [KET06]. La metodología propuesta en la Sección 3.3 es original, pero de ninguna forma alejada a otras propuestas como la encontrada en [Eri19]. En el caso de este texto, el enfoque se basa en la definición de una función objetivo que posteriormente se define formalmente de manera recurrente y en separar esas dos cosas para que la segunda se pueda demostrar correcta respecto de la primera.

Los problemas escogidos para ilustrar el uso de la programación dinámica también son extensamente conocidos en la comunidad de algoritmos. Posiblemente, el enfoque de anotar los diseños de memorización con condiciones y de tabulación con invariantes explícitos no sea tan popular. En este capítulo se ha hecho el esfuerzo de exponer las principales propiedades de los problemas y los algoritmos usando una notación matemática sencilla. El uso de invariantes à la Dijkstra & Scholten también es planteado por J. Bohórquez [Boh06]. El algoritmo que implementa la reducción a una sola columna en la Sección 3.6 está inspirado en observaciones hechas en [Eri19].

La mayoría de los algoritmos presentados calcula el valor de una solución óptima y no una solución que lo alcance. La reconstrucción de una solución óptima se desarrolla en la Sección 3.7 y en la Sección 3.8, y se propone como ejercicio en las demás secciones. Se trata de una técnica general y no de un truco propio de un problema en particular: la tabulación guarda, además del valor óptimo, el rastro de las decisiones que lo produjeron y recorrerla en sentido contrario permite recuperarla. Los textos citados en estas notas la presentan bajo nombres como *traceback* o *backtracking of the table*, que no deben confundirse con la técnica de reintento, tema de otro capítulo de este texto.

La historia del problema de la Sección 3.4, resumida en la Nota 3.4.1, se conoce sobre todo por una columna de J. Bentley [Ben84], en la cual aparecen tanto la anécdota como los tres algoritmos. J. Kadane publicó su propia versión de los hechos más de cuarenta años después [Kad23], y allí señala, además, que el algoritmo que Bentley reconstruyó en su columna no coincide exactamente con el que él tenía en mente. El mismo algoritmo lineal se ha obtenido por caminos independientes del seguido en este capítulo: D. Gries [Gri82] lo derivó aplicando la estrategia estándar de desarrollo de programas a partir de invariantes —el mismo estilo que se sigue aquí— y R. Bird [Bir89] lo obtuvo por manipulación algebraica del algoritmo de fuerza bruta.

El problema de la subsecuencia creciente más larga de la Sección 3.5 es, entre los que se estudian en este capítulo, el que más atención ha recibido por fuera de la algorítmica. La pregunta por la longitud típica de la subsecuencia creciente más larga de una permutación aleatoria se conoce como el problema de Ulam y ha resultado tener conexiones profundas con áreas en apariencia distantes: permutaciones aleatorias, matrices aleatorias, tableaux de Young y modelos de crecimiento.

Tanto así que el problema cuenta con un libro dedicado exclusivamente a él, el de D. Romik [Rom15], en donde se desarrollan el teorema de Vershik–Kerov–Logan–Shepp sobre la forma límite, el teorema de Baik–Deift–Johansson y la distribución de Tracy–Widom. Nada de eso hace falta para leer la Sección 3.5, pero sí ilustra bien que un problema algorítmico sencillo de enunciar puede esconder matemáticas de gran riqueza. El algoritmo de tiempo $O(N \log N)$ mencionado en la Nota 3.5.4 se remonta al trabajo de C. Schensted [Sch61], en donde aparece como subproducto de una correspondencia entre permutaciones y tableaux; su presentación moderna en términos del solitario ‘paciencia’ (en inglés, *patience sorting*) se encuentra en el primer capítulo de [Rom15].

El problema de la subsecuencia común más larga de la Sección 3.7, en cambio, es conocido sobre todo por sus aplicaciones. El algoritmo de espacio lineal que reconstruye una solución óptima, mencionado en la Nota 3.7.3, se debe a D. Hirschberg [Hir75] y combina la programación dinámica de esa sección con dividir, conquistar y combinar. La variante conocida como distancia de edición, propuesta como ejercicio en la misma sección, fue estudiada por R. Wagner y M. Fischer [WF74]; la solución que allí se presenta es esencialmente la tabulación de la Sección 3.7 con un caso adicional. Vale la pena anotar que, para ninguno de los dos problemas, se conoce un algoritmo cuyo tiempo sea sustancialmente mejor que el cuadrático y que hay evidencia de que tal algoritmo no existe: bajo hipótesis estándar de la teoría de la complejidad, no hay soluciones en tiempo $O((MN)^{1-\epsilon})$ para ningún $\epsilon > 0$.

El algoritmo de la Sección 3.8 tiene una historia enredada que vale la pena contar completa. B. Roy lo describió en 1959 [Roy59] y S. Warshall en 1962 [War62], ambos para el problema de la clausura transitiva; ese mismo año, R. Floyd [Flo62] publicó la versión para caminos mínimos que es la que se estudia aquí. Ni siquiera la formulación con tres ciclos anidados, que es la que todo el mundo memoriza, es de alguno de los tres: se debe a P. Ingerman [Ing62], también de 1962. El nombre usual, algoritmo de Floyd–Warshall, omite a Roy a pesar de haber sido el primero, y por eso en este texto se prefiere Roy–Floyd–Warshall, como se explica en la Nota 3.8.4. A. Schrijver documenta en detalle las diversas atribuciones [Sch04, Sec. 8.6g].

Los últimos ejercicios de esa sección apuntan a algo más profundo que una curiosidad histórica. La misma recurrencia resuelve tres problemas distintos, a saber, alcanzabilidad, distancias mínimas y capacidad máxima, cambiando únicamente las dos operaciones que combinan los valores. Y va más lejos: S. Kleene había usado ese mismo esquema en 1956 [Kle56] para convertir un autómata finito en una expresión regular, con la unión y la concatenación en el papel de esas dos operaciones. Lo que las cuatro instancias comparten es una estructura algebraica, la de *semianillo*, y el problema general de calcular esta clase de recurrencias sobre un semianillo arbitrario se conoce como el *problema algebraico de caminos*. Es un buen ejemplo de que la

programación dinámica no es un catálogo de trucos sino una manera de reconocer estructura: cuatro problemas que nadie agruparía a primera vista resultan ser el mismo cálculo.

El problema de la Sección 3.9 es, probablemente, el problema de optimización combinatoria más estudiado que existe. K. Menger fue el primer matemático en ocuparse de él, hacia 1930, y la formulación moderna se le atribuye a H. Whitney a partir de una charla que dio en Princeton en 1934. El algoritmo que la sección desarrolla fue publicado en 1962 de manera independiente por R. Bellman [Bel62], por un lado, y por M. Held y R. Karp [HK62], por el otro; de ahí el nombre de algoritmo de Bellman–Held–Karp que se usa en la Nota 3.9.3. Su cota de $O(N^2 \cdot 2^N)$ sigue siendo, más de sesenta años después, la mejor conocida para el caso general del problema. Obtener una solución exacta en tiempo $O(c^N)$ con $c < 2$ es un problema abierto célebre.

Los textos de T. Cormen et al. [CLRS22], J. Kleinberg y É. Tardos [KET06], J. Erickson [Eri19], H. Bhasin [Bha15] y A. Levitin [Lev12] son fuentes extensas de ejemplos de soluciones con programación dinámica y ejercicios para ser resueltos con esta técnica. Textos de entrenamiento diseñados para programación competitiva como los de S. Skiena [Ski08] y S. Halim et al. [HHE20] contienen ejemplos complementarios, además de ‘trucos’ de implementación, al igual que páginas de internet asociadas a programación competitiva y algoritmos.

Además de las técnicas de reducción de espacio vistas para tabulaciones, existen técnicas para la reducción del tiempo de cómputo para algunos problemas que admiten programación dinámica como solución. Entre ellas se encuentran las basadas en monotonías y desigualdades propuestas por D. Knuth en [Knu88]. Existen otras basadas en envolvimientos convexos, y en dividir y conquistar; se sugiere al lector [Bei13] para un recuento más detallado y ejemplos.

Algoritmos voraces

Las dos técnicas de diseño presentadas hasta ahora responden, cada una a su manera, a una misma pregunta: ¿cómo evitar recorrer por completo el espacio de soluciones de un problema? Dividir, conquistar y combinar lo parte en subespacios independientes que se resuelven por separado; la programación dinámica lo comprime, aprovechando que muchas subinstancias se repiten. Ambas respuestas conservan una característica común: el espacio de soluciones se sigue explorando, solo que de manera organizada. Este capítulo presenta una tercera respuesta y es la más radical de todas: no explorar, sino ir a la fija. Un algoritmo voraz recorre un único camino del espacio de soluciones, aquel que le dicta un criterio de selección fijado de antemano y nunca se desvía de él, para resolver –generalmente– un problema de optimización. De manera más precisa, un *algoritmo voraz* (en inglés, *greedy algorithm*) construye una solución tomando una secuencia de decisiones, cada una de las cuales optimiza un criterio local e inmediato, sin considerar sus consecuencias a largo plazo y sin reconsiderarla después. Lo notable, cuando la técnica funciona, es que esa sucesión de elecciones miopes resulta ser una solución globalmente óptima.

En la práctica, la optimización local rara vez conduce a un óptimo global. Es decir, en el espacio de los algoritmos de optimización, los voraces son realmente excepciones. Entonces, ¿por qué estudiar este tipo de algoritmos si raramente han de funcionar? La razón principal es que cuando los algoritmos voraces existen, estos son elegantes en su diseño, sencillos de programar y eficientes computacionalmente. Hay una frase anónima que resume bien esta situación, casi paradójica:

Los algoritmos voraces no funcionan, pero cuando sí, lo hacen muy bien.

–Autor desconocido.

De esa excepcionalidad se desprende el rasgo que distingue a este capítulo de los anteriores. El diseño de un algoritmo voraz debe estar *siempre* acompañado de una demostración de su corrección, i.e., de que logra resolver el problema de optimización planteado. No se trata de una precaución metodológica, sino de una

necesidad: para casi cualquier problema de optimización se puede proponer un criterio de selección plausible, escribir en pocas líneas el algoritmo voraz correspondiente y ejecutarlo sin que nada falle; el programa responderá y responderá –muy posiblemente– mal. Un algoritmo voraz sin demostración no es un algoritmo con una laguna teórica, es una conjetura.

Nota 4.0.1: ¿Dónde está la dificultad?

Vale la pena contrastar este capítulo con el Capítulo 3. En programación dinámica, el trabajo difícil está en *encontrar* la función objetivo y su definición recurrente; una vez planteadas, la demostración de corrección suele ser una inducción rutinaria y la implementación una transcripción. En los algoritmos voraces sucede exactamente lo contrario: el algoritmo es casi siempre inmediato –ordenar y recorrer, o escoger repetidamente el mínimo o máximo– y todo el esfuerzo intelectual está en demostrar que el criterio escogido es el correcto. El lector que mida el avance por la cantidad de código escrito tendrá la impresión de que este capítulo es más fácil que el anterior.

Ahora bien, estas demostraciones no se inventan desde cero cada vez: siguen un puñado de patrones que se repiten con notable regularidad. Identificarlos, nombrarlos y saber cuál ensayar en cada situación es el objetivo central del capítulo y es también la razón por la cual los ejemplos se presentan en el orden en que se presentan. Se estudiarán tres patrones: el *argumento de intercambio*, que transforma una solución óptima cualquiera en otra que contiene la elección voraz; el *argumento de inversión*, su variante para problemas cuya solución es un ordenamiento; y el argumento de que *el voraz va adelante*, que compara paso a paso lo construido por el algoritmo contra cualquier otra solución. Los tres se introducen sobre ejemplos concretos y se sistematizan en la Sección 4.3.

Es importante aprender a plantear este tipo de demostraciones, más allá de solo tener familiaridad con ellas o inclusive de entenderlas. Las demostraciones de *optimización local*, como se llamarán en el desarrollo del capítulo, intrínsecamente incluyen el diseño del algoritmo voraz correspondiente: cómo elegir el “siguiente” elemento que permitirá, finalmente, contar con una solución óptima (i.e., globalmente óptima). Este patrón ayudará, además, a encontrar un argumento formal –a veces no tan evidente– para justificar por qué un algoritmo voraz funciona.

Nota 4.0.2: Sobre el nombre

En la literatura en inglés esta técnica se conoce como *greedy*, adjetivo que describe a quien toma todo lo que puede en el momento sin pensar en lo

que vendrá después. En castellano circulan las traducciones *voraz*, *ávido* y, con menos fortuna, *glotón*. En este texto se prefiere *voraz* por ser la más extendida, pero conviene no perder de vista lo que el nombre describe: no la calidad del resultado, sino la actitud del algoritmo frente a cada decisión. Elegir lo mejor *ahora* es una descripción del procedimiento; que eso produzca lo mejor *al final* es precisamente lo que hay que demostrar.

Nota 4.0.3: Antes que el nombre, los algoritmos

Como ocurre con el reintento, los algoritmos voraces son bastante más antiguos que la idea de estudiarlos como una técnica de diseño. En 1926, Otakar Borůvka publicó un algoritmo para construir redes eléctricas de costo mínimo que hoy se reconoce como el primer algoritmo para el problema del árbol de cubrimiento mínimo. A ese le siguieron los de Joseph Kruskal (1956) y Robert Prim (1957) para el mismo problema —este último redescubriendo un algoritmo que Vojtěch Jarník había publicado en 1930—, el de David Huffman (1952) para códigos de longitud variable y el de Edsger Dijkstra (1959) para caminos más cortos. Todos ellos son anteriores a cualquier intento de explicar qué tienen en común. Ese intento llegó con la teoría de matroides, en particular con el trabajo de Jack Edmonds a comienzos de la década de 1970, que caracteriza una clase de problemas en los cuales la estrategia voraz es óptima. La caracterización es elegante pero parcial: varios de los problemas de este capítulo se resuelven vorazmente sin caer dentro de ella (ver las notas al final del capítulo).

Este capítulo presenta las ideas principales que respaldan los algoritmos voraces, una metodología para su uso y varios ejemplos, en dominios como el agendamiento óptimo, las redes de costo mínimo y los caminos más cortos.

4.1. Ideas clave

Considere el problema de pagar una cantidad exacta con la menor cantidad posible de monedas, disponiendo de una cantidad ilimitada de monedas de cada denominación. Con las denominaciones de 1, 5, 10 y 25 unidades, y una cantidad de 30 unidades por pagar, la respuesta es inmediata: una moneda de 25 y una de 5, dos monedas en total. Y el razonamiento que produce esa respuesta también lo es: se toma la moneda más grande que quepa, se descuenta y se repite con lo que falte. Este razonamiento es un algoritmo voraz, y se escribe en tres líneas.

Algoritmo 4.1.1

```
1 def coins(D, M, X, m):
2     # D[0..M] holds denominations in decreasing order
3     # and D[M-1] == 1
4     ans = None
5     if X == 0: ans = 0
6     else: ans = X // D[m] + coins(D, M, X % D[m], m + 1)
7     return ans
```

La suposición de que $D[M - 1] = 1$ garantiza que el llamado termina siempre con $X = 0$ (¿por qué?), de modo que la función está bien definida. Y funciona: con $D = [25, 10, 5, 1]$ la función responde el mínimo para cualquier cantidad.

Ejemplo 4.1.1

Con las denominaciones usuales, la función responde lo esperado:

```
1 >>> coins([25, 10, 5, 1], 4, 30, 0)
2 2
3 >>> coins([25, 10, 5, 1], 4, 63, 0)
4 6
```

Ahora considere las denominaciones 1, 3 y 4, y la cantidad 6:

```
1 >>> coins([4, 3, 1], 3, 6, 0)
2 3
```

La respuesta corresponde a las monedas $4 + 1 + 1$. Pero $6 = 3 + 3$ se paga con dos monedas, y por lo tanto la respuesta correcta es 2.

El Ejemplo 4.1.1 merece una mirada cuidadosa del detalle porque contiene en miniatura todo lo que hay que entender sobre esta técnica. El algoritmo no cambió; el código es exactamente el mismo. No hubo error de programación, ni de índices ni un caso base mal planteado. Tampoco hubo una ejecución fallida: la función terminó, retornó un número y ese número es la cantidad de monedas de un pago legítimo de 6 unidades. Lo único que cambió fue el conjunto de denominaciones y con ese cambio desapareció la corrección.

Nota 4.1.1: Fallar en silencio

Fallar en silencio es una forma muy incómoda de fallar que puede tener un algoritmo. Un algoritmo de reintento mal podado se demora; uno de programación dinámica con una recurrencia equivocada suele producir disparates visibles. Un algoritmo voraz con un criterio equivocado produce respuestas plausibles, del tipo correcto, frecuentemente cercanas al óptimo y algunas veces iguales a él. Ninguna cantidad de pruebas con casos pequeños distingue con seguridad un criterio correcto de uno que no lo es –con $D = [4, 3, 1]$, las cantidades $0, \dots, 5$ dan la respuesta correcta y solo la sexta falla. Por eso la demostración no es opcional.

El propósito de esta sección es precisar qué hay que demostrar. Se introduce el vocabulario mínimo para hablar de problemas de optimización y de la manera en que un algoritmo voraz construye sus soluciones, se plantea un esquema general que todos los algoritmos del capítulo instancian y se aíslan las dos propiedades de las cuales depende que ese esquema sea correcto.

4.1.1. Problemas de optimización. Los problemas que se resuelven con algoritmos voraces son, casi siempre, problemas de optimización. La estructura común a todos ellos es la siguiente.

Definición 4.1.1: Problema de optimización

Un *problema de optimización* está dado por un conjunto $\mathcal{F}$ finito y no vacío de *soluciones factibles*, una *función objetivo* $c : \mathcal{F} \rightarrow \mathbb{R}$ y un *sentido*, que es maximizar o minimizar. Una solución factible $s \in \mathcal{F}$ es *óptima* si $c(s) \geq c(s')$ para toda $s' \in \mathcal{F}$ cuando el sentido es maximizar, o $c(s) \leq c(s')$ para toda $s' \in \mathcal{F}$, cuando es minimizar.

Los tres elementos de la Definición 4.1.1 deben leerse en la especificación del problema y conviene practicar esa lectura desde ahora. En el problema de las monedas, $\mathcal{F}$ es el conjunto de las colecciones de monedas que suman exactamente la cantidad pedida, c cuenta cuántas monedas tiene la colección y el sentido es minimizar. Note que la factibilidad –sumar exactamente la cantidad– y la optimalidad –usar pocas monedas– son requisitos de naturaleza distinta: el primero es una condición que se cumple o no se cumple, mientras el segundo es una comparación entre las que sí la cumplen. Perder de vista esa distinción es la fuente más común de confusión al plantear un problema de optimización.

Nota 4.1.2: Sobre la unicidad

La Definición 4.1.1 habla de *una* solución óptima, no de *la* solución óptima, y la diferencia importa. Puede haber varias soluciones factibles distintas con el mismo valor óptimo de la función objetivo; lo único que necesariamente es único es ese valor. Todos los teoremas de este capítulo están enunciados de manera de admitir esa multiplicidad y varias de sus demostraciones dependen precisamente de poder reemplazar una solución óptima por otra igual de buena.

4.1.2. Candidatos, decisiones y criterio de selección. Un algoritmo voraz no examina $\mathcal{F}$: lo construye por partes. Para hacerlo necesita que las soluciones factibles se puedan describir como colecciones de piezas más simples.

Definición 4.1.2: Candidatos y soluciones parciales

Considere un problema de optimización con soluciones factibles $\mathcal{F}$. Un *conjunto de candidatos* para el problema es un conjunto finito C tal que $\mathcal{F} \subseteq \mathcal{P}(C)$. Un subconjunto $S \subseteq C$ es una *solución parcial* sii $S \subseteq S'$ para alguna $S' \in \mathcal{F}$; es decir, sii S todavía se puede extender hasta una solución factible.

Con este vocabulario, la tarea de un algoritmo voraz se describe en una frase: partir de la solución parcial vacía y agregarle candidatos, uno a la vez, hasta obtener una solución factible. Lo que caracteriza a la técnica es cómo se decide cuál candidato agregar.

Definición 4.1.3: Criterio de selección

Un *criterio de selección* sobre un conjunto de candidatos C es un preorden total $\preceq$ sobre C . Un candidato $c \in C'$ es *óptimo* en $C' \subseteq C$ según el criterio sii $c \preceq c'$ para todo $c' \in C'$.

Que el criterio sea un preorden *total* garantiza que siempre haya un candidato óptimo entre los disponibles; que sea un preorden y no un orden admite que haya empates y por lo tanto varios candidatos óptimos a la vez. Ninguna de las dos cosas es un detalle técnico. La primera es lo que hace que el algoritmo nunca se quede sin saber qué hacer. La segunda obliga a que las demostraciones de este capítulo funcionen para *cualquiera* de los candidatos óptimos, no para uno escogido a conveniencia.

Ejemplo 4.1.2

Los criterios de selección de las secciones de este capítulo, enunciados de manera informal:

- En el problema de las monedas, el candidato es una moneda y el criterio prefiere la de mayor denominación.
- En el agendamiento de actividades (Sección 4.2), el candidato es una actividad y el criterio prefiere la que termine más temprano.
- En el cubrimiento de intervalos (Sección 4.4), el candidato es un intervalo y el criterio prefiere, entre los que cubren el extremo pendiente, el que se extienda más lejos.
- En el árbol de cubrimiento mínimo (Sección 4.7), el candidato es un arco y el criterio prefiere el de menor peso.

Observe que en los cuatro casos el criterio se enuncia en una línea y no menciona el objetivo global del problema.

4.1.3. El esquema general. Con los ingredientes anteriores, el esquema que instancian todos los algoritmos voraces del capítulo es el siguiente.

Algoritmo 4.1.2: Esquema general de un algoritmo voraz

Para un conjunto de candidatos C y un criterio de selección $\preceq$:

1. $S = \emptyset$.
2. Mientras $S \notin \mathcal{F}$ y $C \neq \emptyset$:
 - a) escoger $c \in C$ óptimo según $\preceq$ y removerlo de C ;
 - b) si $S \cup \{c\}$ es una solución parcial, entonces $S = S \cup \{c\}$.
3. Si $S \in \mathcal{F}$, retornar S ; de lo contrario, no hay solución.

Tres observaciones sobre el Algoritmo 4.1.2, cada una de las cuales lo separa de las técnicas ya estudiadas.

No hay retroceso: Un candidato removido de C en el paso 2(a) no vuelve a considerarse y un candidato agregado a S en el paso 2(b) no vuelve a retirarse. Comparado con el Capítulo 5, donde toda decisión es provisional y puede deshacerse, aquí toda decisión es definitiva.

No hay exploración: En cada iteración se examina un único candidato, el que dicta el criterio. La programación dinámica presentada en el Capítulo 3, en la misma situación, evaluaría todas las alternativas y se quedaría con la mejor; el algoritmo voraz apuesta a una y sigue.

El ciclo termina: Cada iteración remueve un elemento de C , que es finito. Por eso el esquema siempre termina, incluso cuando el criterio de selección es completamente equivocado. Esta es justamente la razón por la cual un criterio equivocado no se manifiesta como una ejecución fallida sino como una respuesta incorrecta (o falla en silencio).

Nota 4.1.3: Criterios estáticos y dinámicos

El paso 2(a) del Algoritmo 4.1.2 pide el candidato óptimo entre los que quedan, y la manera de conseguirlo depende de si el criterio cambia o no durante la ejecución.

Cuando $\preceq$ no depende de S –el peso de un arco, la denominación de una moneda, el tiempo de finalización de una actividad– se dice que el criterio es *estático*. En ese caso el paso 2(a) no requiere estructura alguna: basta ordenar C una sola vez al comienzo y recorrerlo en orden. Esta es la razón por la cual tantos algoritmos voraces tienen la forma *ordenar y recorrer*, y también la razón por la cual su complejidad temporal suele quedar dominada por el $O(|C| \log |C|)$ del ordenamiento.

Cuando $\preceq$ sí depende de la solución parcial construida hasta el momento, el criterio es *dinámico* y hay que recalcularlo, o al menos actualizarlo, en cada iteración. Ordenar de antemano ya no sirve y se necesita una estructura de datos que entregue el mínimo y admita modificaciones, típicamente una cola de prioridad. Los algoritmos de las últimas secciones del capítulo son de este tipo y su análisis es, por eso, más interesante.

4.1.4. Las dos propiedades. Falta lo esencial: ¿bajo qué condiciones el Algoritmo 4.1.2 produce una solución óptima y no simplemente una solución factible? Son dos condiciones y conviene mantenerlas separadas porque fallan de maneras distintas.

La primera dice que comprometerse con la elección voraz no cierra la puerta a la optimalidad.

Definición 4.1.4: Propiedad de elección voraz

Un criterio de selección $\preceq$ satisface la *propiedad de elección voraz* para un problema de optimización sii, para toda instancia con al menos una solución factible no vacía y para todo candidato c óptimo según $\preceq$ entre los disponibles, existe una solución óptima que contiene a c .

La segunda dice que, una vez tomada esa decisión, lo que queda es un problema de la misma naturaleza y basta resolverlo bien.

Definición 4.1.5: Subestructura óptima

Un problema de optimización tiene *subestructura óptima* con respecto a un conjunto de candidatos C sii, para todo candidato $c \in C$ que hace parte de alguna solución óptima, se cumple que $S \cup \{c\}$ es una solución óptima del problema siempre que S sea una solución óptima del problema que resulta de comprometerse con c .

Esta es la Definición 3.1.1 del Capítulo 3, particularizada a la descomposición propia de este capítulo: la que consiste en comprometerse con un candidato, de modo que la única subinstancia es el problema que queda después del compromiso. De las dos mitades que tiene aquella definición, aquí se enuncia únicamente la de composición, que es la que necesita la demostración del Teorema 4.1.1 por ser una inducción hacia adelante. El papel de la otra mitad, la de descomposición, lo cumple en este contexto la propiedad de elección voraz de la Definición 4.1.4: es ella la que garantiza que existe una solución óptima que contiene el candidato escogido.

Estas dos propiedades son exactamente lo que hace falta y esa es la razón de ser de todo el aparato anterior.

Teorema 4.1.1

Suponga que el criterio de selección $\preceq$ satisface la propiedad de elección voraz y que el problema tiene subestructura óptima, ambas con respecto al conjunto de candidatos C . Entonces el Algoritmo 4.1.2 retorna una solución óptima siempre que exista alguna solución factible.

Demostración

Se procede por inducción sobre $|C|$.

Si $C = \emptyset$, la única solución parcial es $\emptyset$ y, por suposición, existe alguna solución factible; luego $\emptyset \in \mathcal{F}$ y es la única, por lo que es óptima. El algoritmo retorna $\emptyset$.

Si $C \neq \emptyset$, considérese la primera iteración del ciclo. Si $\emptyset \in \mathcal{F}$ el ciclo no se ejecuta y el argumento del caso anterior aplica. De lo contrario, sea c el candidato escogido en el paso 2(a). Por la propiedad de elección voraz existe una solución óptima S^* con $c \in S^*$; en particular $\{c\}$ es una solución

parcial y el paso 2(b) efectivamente agrega c a S . Las iteraciones restantes del ciclo son una ejecución del mismo esquema sobre el problema que resulta de comprometerse con c , con conjunto de candidatos $C \setminus \{c\}$ y el mismo criterio de selección. Como $|C \setminus \{c\}| < |C|$ y ese problema tiene solución factible —la propia S^* lo atestigua— la hipótesis inductiva garantiza que dichas iteraciones producen una solución óptima S para él. Por la subestructura óptima, $S \cup \{c\}$ es una solución óptima del problema original, que es lo que el algoritmo retorna.

Nota 4.1.4: Optimización local y global

El Teorema 4.1.1 explica la organización que siguen todas las secciones de este capítulo. En cada una se demuestran dos teoremas:

- un *teorema de optimización local*, que enuncia la propiedad de elección voraz para el criterio propuesto, y es siempre la parte más retadora;
- un *teorema de optimización global*, que concluye que el algoritmo es correcto, y que no es más que la instancia del Teorema 4.1.1 para el problema en cuestión.

La subestructura óptima no suele enunciarse por separado porque en los problemas de este capítulo es evidente: comprometerse con una decisión deja un problema del mismo tipo sobre menos candidatos. Conviene, de todos modos, verificarla y no darla por descontada.

4.1.5. ¿Por qué falla? Con el vocabulario ya disponible se puede decir con precisión qué pasó en el Ejemplo 4.1.1. El problema de las monedas tiene subestructura óptima y la tiene con cualquier conjunto de denominaciones: si una solución óptima para pagar X usa una moneda de denominación d , entonces lo que resta de esa solución es necesariamente una solución óptima para pagar $X - d$ (¿por qué?). La propiedad que falla es la otra. Con $D = [4, 3, 1]$ y $X = 6$, el candidato óptimo según el criterio es la moneda de 4; la única solución óptima es $\{3, 3\}$ y no contiene ninguna moneda de 4. Es decir, el criterio no satisface la propiedad de elección voraz.

Nota 4.1.5: Un contraejemplo basta

Note la asimetría entre demostrar y refutar. Demostrar la propiedad de elección voraz exige un argumento que cubra todas las instancias del problema.

Refutarla exige exhibir una sola instancia, típicamente una diminuta: el contraejemplo de las monedas usa tres denominaciones y la cantidad 6. De ahí una recomendación práctica que este texto insiste en repetir: antes de intentar demostrar que un criterio funciona, dedíquele un rato honesto a intentar romperlo con instancias pequeñas. Es mucho más rápido y en la mayoría de los casos –recuerde que los algoritmos voraces son la excepción– va a funcionar. El Ejercicio 5 propone hacerlo de manera sistemática.

Nota 4.1.6: Voraces y programación dinámica

Las dos técnicas comparten la exigencia de subestructura óptima y se separan exactamente en la otra propiedad. La programación dinámica no supone nada parecido a la elección voraz: en cada paso considera todas las decisiones disponibles, resuelve el subproblema que resulta de cada una y se queda con la mejor. Por eso siempre funciona –el problema de las monedas con $D = [4, 3, 1]$ se resuelve sin dificultad con las técnicas del Capítulo 3– y por eso cuesta más.

La relación entre ambas se puede resumir así: un algoritmo voraz es una solución de programación dinámica en la cual se ha demostrado que no hace falta evaluar todas las alternativas porque siempre gana la misma. Esa demostración es el teorema de optimización local y el premio por conseguirla es pasar de una tabla a un recorrido.

4.1.6. Análisis. El análisis de un algoritmo voraz es directo y contrasta con el enfoque que tuvo el del capítulo anterior.

Teorema 4.1.2

Suponga que verificar si $S \cup \{c\}$ es una solución parcial toma tiempo $O(f)$ y que escoger el candidato óptimo entre los disponibles toma tiempo $O(g)$. Entonces el Algoritmo 4.1.2 toma tiempo $O(|C| \cdot (f + g))$. Si además el criterio de selección es estático, se puede ordenar C al comienzo y obtener tiempo $O(|C| \log |C| + |C| \cdot f)$.

Demostración

Se propone como ejercicio al lector.

No hay aquí nada parecido al factor $|V|$ que domina el costo de un algoritmo de reintento y que ningún análisis asintótico logra acotar con precisión. El ciclo del Algoritmo 4.1.2 ejecuta a lo sumo $|C|$ iteraciones siempre, para toda instancia, sin importar el criterio. Los algoritmos voraces son eficientes por construcción; lo que hay que demostrar no es que sean rápidos, sino que sean correctos.

Ejercicios

1. Demuestre el Teorema 4.1.2.
2. Justifique las siguientes afirmaciones hechas en esta sección:
 - a) El Algoritmo 4.1.1 termina siempre, bajo la suposición de que $D[M - 1] = 1$.
 - b) El problema de las monedas tiene subestructura óptima con cualquier conjunto de denominaciones.
 - c) El conjunto vacío es una solución parcial de todo problema de optimización con al menos una solución factible.
3. En la Definición 4.1.3 se pide que el criterio de selección sea un preorden *total*. Explique qué le ocurre al Algoritmo 4.1.2 si se admiten criterios que sean órdenes parciales, y por qué la demostración del Teorema 4.1.1 deja de funcionar.
4. El paso 2(b) del Algoritmo 4.1.2 descarta el candidato escogido cuando no puede agregarse a S .
 - a) Dé un ejemplo de un problema de esta sección o de la siguiente en el cual ese descarte efectivamente ocurra.
 - b) ¿Qué pasaría si, en lugar de descartarlo, el algoritmo lo devolviera a C ?
5. Implemente en Python el Algoritmo 4.1.1 y una función que calcule por fuerza bruta la mínima cantidad de monedas.
 - a) Escriba un programa que genere conjuntos de denominaciones al azar (siempre incluyendo la denominación 1) y cantidades pequeñas, y reporte la primera discrepancia entre las dos funciones.
 - b) ¿Qué tan pequeño es el menor contraejemplo que encuentra?
 - c) Repita el experimento restringiéndose a denominaciones de la forma $1, b, b^2, b^3, \dots$ para un $b \geq 2$ fijo. ¿Qué observa?
6. Demuestre que si las denominaciones son $1, b, b^2, \dots, b^{M-1}$ para algún $b \geq 2$, entonces el criterio de escoger la moneda de mayor denominación que quepa satisface la propiedad de elección voraz. *Ayuda:* demuestre primero que ninguna solución óptima puede usar b o más monedas de una misma denominación distinta de la mayor.

7. Considere el problema de escoger un subconjunto de peso máximo de un arreglo $A[0..N)$ de números, sujeto a la restricción de que no se pueden escoger dos posiciones consecutivas.
 - a) Identifique el conjunto de candidatos, las soluciones factibles y la función objetivo, según la Definición 4.1.1.
 - b) Proponga un criterio de selección y refútelo con un contraejemplo.
 - c) Proponga un segundo criterio y refútelo también.
8. 🌐 La Definición 4.1.2 exige que las soluciones factibles sean subconjuntos de un conjunto de candidatos, lo cual deja por fuera los problemas cuya solución es un ordenamiento, como el de la Sección 4.5. Proponga una manera de acomodarlos dentro del marco de esta sección y discuta si el Teorema 4.1.1 sigue siendo aplicable.

4.2. Agendamiento de actividades

Considere el problema de agendar procedimientos quirúrgicos en la sala de cirugías de un hospital o clínica. Cada cirugía tiene un horario, con horas de inicio y finalización. Suponiendo que entre más cirugías se hagan, mayor será el beneficio para los pacientes (acá nada tiene que ver el dinero, ¿cierto?), surge la siguiente pregunta: ¿cómo maximizar la cantidad de cirugías a realizar sin que haya conflicto de horario entre ellas?

De una forma más general, esta pregunta es formulada en el problema del agendamiento de actividades.

Problema 4.2.1: Agendamiento de actividades

- Entrada:** Un arreglo $A[0..N)$, $N \geq 0$, de parejas de números (s_n, e_n) tales que $0 \leq s_n < e_n$ indican el tiempo de inicio y finalización de la actividad $0 \leq n < N$, respectivamente.
- Salida:** Máxima cantidad de actividades en $A[0..N)$ que se pueden agendar sin conflicto.

Como convención para cualquier n , con $0 \leq n < N$, las expresiones $A[n][0]$ y $A[n][1]$ denotan, respectivamente, el tiempo de inicio y finalización de la actividad n : es decir, $s_n = A[n][0]$ y $e_n = A[n][1]$. Note que todos los tiempos se expresan con cantidades no negativas. Es necesario precisar qué significa actividades *compatibles* y en *conflicto* para entender completamente la especificación del problema de agendamiento de actividades.

Nota 4.2.1

Por convención, se entenderá que cada actividad (s, e) representa el intervalo cerrado-abierto de tiempo $[s..e)$. De esta forma, dos actividades (s_i, e_i) y (s_j, e_j) son *compatibles* sii $[s_i..e_i) \cap [s_j..e_j) = \emptyset$; de lo contrario, se dice que están en *conflicto*.

El primer paso es leer la especificación con el vocabulario de la Sección 4.1. Los tres elementos de la Definición 4.1.1 están todos en el enunciado del Problema 4.2.1:

- las *soluciones factibles* son los subconjuntos de actividades de $A[0..N)$ que no tienen conflictos entre sí, llamados *agendamientos*;
- la *función objetivo* de un agendamiento es su tamaño; y
- el *sentido* es maximizar.

El conjunto de candidatos, en el sentido de la Definición 4.1.2, son las N actividades, y una solución parcial es simplemente un agendamiento –cualquier conjunto sin conflictos se puede extender hasta uno maximal, en particular hasta uno de tamaño máximo o no.

Nota 4.2.2: Maximal y máximo

Conviene distinguir dos adjetivos que se parecen y no significan lo mismo. Un agendamiento es *maximal* sii no se le puede agregar ninguna actividad más sin producir un conflicto; es *máximo* sii ningún otro agendamiento tiene más actividades que él.

Todo agendamiento máximo es maximal, pero no al contrario. Con las tres actividades $(0, 10)$, $(1, 3)$ y $(4, 6)$, el agendamiento $\{(0, 10)\}$ es maximal –las otras dos están en conflicto con ella– y sin embargo $\{(1, 3), (4, 6)\}$ es más grande. El Problema 4.2.1 pide un agendamiento **máximo** y esa es la palabra que aparecerá en los enunciados de esta sección. La distinción no es una sutileza: encontrar un agendamiento maximal es trivial y encontrar uno máximo es el problema.

Ejemplo 4.2.1

Se ilustra cada cirugía como un rectángulo cuya coordenada izquierda en el eje horizontal denota el tiempo de inicio y la derecha el tiempo de finalización.



Las actividades que tienen su fondo resaltado hacen parte de un agendamiento máximo, sin conflictos entre ellas. Dada esta entrada, la salida es 6 para el problema de agendamiento.

El problema de agendamiento se puede resolver exhaustivamente enumerando todos los subconjuntos de actividades y, entre aquellos que no tienen conflictos, determinar el tamaño máximo posible. Es más, este problema exhibe las propiedades de la subestructura óptima y solapamiento (¿por qué?), y por ello es candidato a ser resuelto con programación dinámica (ver Ejercicio 15). El enfoque voraz es distinto al de una exploración exhaustiva o al de utilizar programación dinámica, pues evita apostarle a ‘todos los caballos’ y en cambio apuesta solo a uno de ellos, a uno que ganará (o que no perderá).

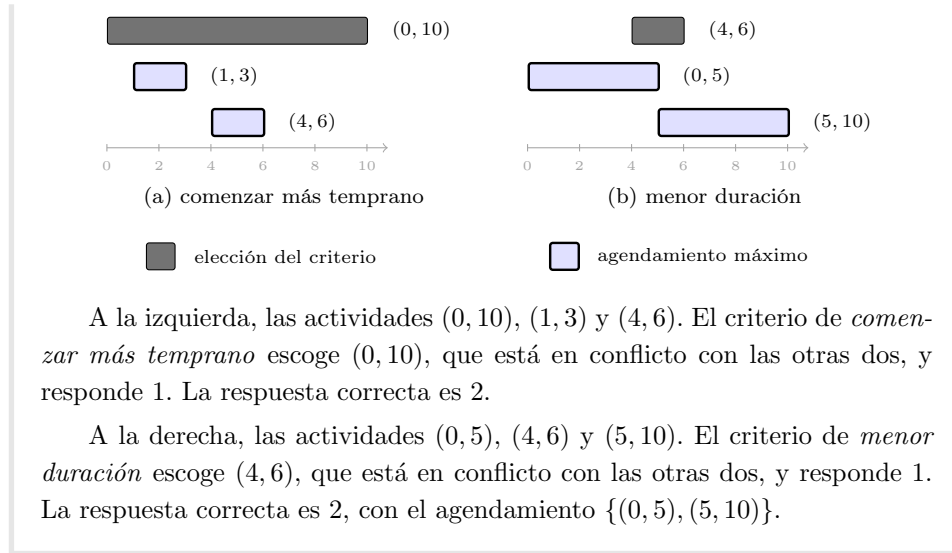
4.2.1. Escoger un criterio. Fijados los candidatos, lo único que falta para tener un algoritmo es un criterio de selección. Y aquí conviene resistir la tentación de proponer el primero que se le ocurra a uno y ponerse a demostrar que funciona, porque lo más probable –según se advirtió en la Nota 4.1.5– es que no funcione. La disciplina que este texto recomienda es la contraria: proponer varios criterios plausibles e intentar romperlos todos. Los que sobrevivan son los candidatos a demostración.

Para el agendamiento de actividades hay al menos cuatro criterios que se le ocurren a cualquiera:

1. escoger la actividad que *comienza* más temprano;
2. escoger la actividad de menor *duración*;
3. escoger la actividad que está en conflicto con la menor *cantidad* de las restantes;
4. escoger la actividad que *termina* más temprano.

En los cuatro casos se entiende que, tras escoger una actividad, se descartan las que estén en conflicto con ella y se repite el procedimiento. Los dos primeros criterios se rompen con tres actividades.

Ejemplo 4.2.2: Dos criterios que no funcionan



Note lo pequeños que son los contraejemplos y lo rápido que se encuentran: tres actividades bastan en ambos casos y en ambos la estrategia falla por la misma razón, que es haber escogido una actividad que consume demasiado tiempo del recurso.

El tercer criterio es más resistente y no se rompe con instancias tan pequeñas; encontrar un contraejemplo requiere algo de trabajo y se propone en el Ejercicio 9. Que un criterio sobreviva a los primeros intentos de refutación es información útil, pero no es una demostración: la única manera de saber si funciona es demostrarlo.

El cuarto criterio es el que sobrevive y la intuición que lo respalda es la siguiente: entre más extenso sea el rango que queda libre para agendar cirugías, más cirugías se podrán agendar. Suponga que se cuenta con un conjunto X de cirugías por agendar. La pregunta clave es, ¿cuál agendar primero? Bajo la intuición anterior, se puede elegir una cirugía $a \in X$ que termine lo más pronto posible dado que esto permite que, para una posterior elección, haya más tiempo para agendar otras cirugías bajo la misma estrategia.

Algoritmo 4.2.1

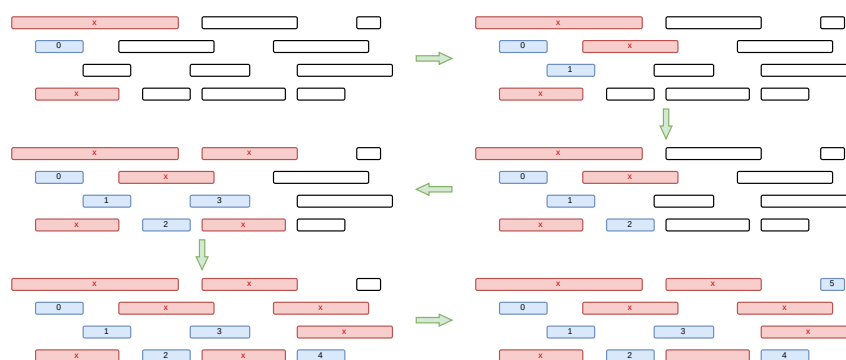
El Algoritmo 4.1.2 instanciado de la siguiente manera:

- el conjunto de candidatos C es el de las actividades en $A[0..N]$;
- el criterio de selección es $a \preceq b$ sii a no termina después de b ; y
- $S \cup \{a\}$ es una solución parcial sii a es compatible con todas las actividades en S .

Desenrollado, el Algoritmo 4.2.1 hace lo siguiente: escoge una actividad que termine lo más pronto posible, la agenda, descarta todas las que estén en conflicto con ella y repite el procedimiento con las que queden, hasta que no quede ninguna.

Ejemplo 4.2.3

El siguiente diagrama muestra, paso a paso, cómo el Algoritmo 4.2.1 selecciona un agendamiento máximo a partir del conjunto de actividades en el Ejemplo 4.2.1.



Cada elección hecha por el algoritmo se indica con un número consecutivo y aquellas actividades que van siendo descartadas porque tienen conflictos con al menos una de las actividades seleccionadas se marcan con una 'x'.

Lo que resta de la sección es demostrar que el Algoritmo 4.2.1 es correcto. De acuerdo con el Teorema 4.1.1, hay que establecer dos cosas: que el criterio satisface la propiedad de elección voraz y que el problema tiene subestructura óptima.

4.2.2. Optimización local: un argumento de intercambio. El teorema de optimización local enuncia, para este problema particular, la propiedad de elección voraz de la Definición 4.1.4.

Teorema 4.2.1: Optimización local

Sea X un conjunto finito y no vacío de actividades. Si $a \in X$ es tal que su tiempo de finalización es mínimo en X , entonces a hace parte de un agendamiento máximo de actividades de X .

Demostración

Suponga que $Y \subseteq X$ es un agendamiento máximo. Como $X \neq \emptyset$, necesariamente $Y \neq \emptyset$ (¿por qué?). Se procede por casos:

- Si $a \in Y$, entonces la conclusión del teorema es correcta.
- Si $a \notin Y$, entonces sea b aquella actividad en Y que tiene tiempo de finalización mínimo. Necesariamente $b \neq a$, porque de lo contrario $a \in Y$ (una contradicción). Considere el conjunto $Z = (Y \setminus \{b\}) \cup \{a\}$. Como $b \in Y$ y $a \notin Y$, se tiene que $|Z| = |Y|$. Además note que Z no tiene conflictos porque el tiempo de finalización de b no puede ser menor que el de a (a tiene tiempo de finalización mínimo entre todas las actividades de X , luego también de Y): al remover b de Y , las actividades que están en $Y \setminus \{b\}$ no tienen conflicto y como a no termina después de b , Z no tiene conflictos (por suposición, en Y no hay conflictos). Luego, Z es un agendamiento de actividades de X con $|Z| = |Y|$ que contiene a a ; como Y es máximo, Z también lo es, que es lo que se deseaba.

Nota 4.2.3: El argumento de intercambio

La demostración del Teorema 4.2.1 es el primer ejemplar de un patrón que reaparecerá varias veces en este capítulo y que conviene reconocer desde ahora. Se llama *argumento de intercambio* y su forma es siempre la misma:

1. se parte de una solución óptima *cualquiera* Y ;
2. si Y ya contiene la elección voraz a , no hay nada que hacer;
3. de lo contrario, se identifica en Y un elemento b que ocupa el “lugar” de a , se construye Z intercambiando b por a , y se demuestra que Z sigue siendo factible y que no es peor que Y .

Lo que hace funcionar al paso 3 es siempre alguna propiedad extrema (i.e., de minimalidad o maximalidad) de la elección voraz. Aquí es que a no termina después de b , lo cual garantiza que ninguna actividad que era compatible con b deja de serlo con a . Al diseñar un argumento de intercambio, la pregunta que hay que hacerse es: *¿qué le permite a la elección voraz reemplazar a cualquier otra sin causar daño?*

4.2.3. Optimización global. Con la propiedad de elección voraz demostrada, falta verificar la subestructura óptima. En este problema es directa.

Teorema 4.2.2: Subestructura óptima

Sea X un conjunto finito de actividades y $a \in X$ una actividad que hace parte de algún agendamiento máximo de X . Sea X_a el conjunto de las actividades de X compatibles con a . Si S es un agendamiento máximo de X_a , entonces $S \cup \{a\}$ es un agendamiento máximo de X .

Demostración

El conjunto $S \cup \{a\}$ es un agendamiento de X porque S no tiene conflictos y todas sus actividades son compatibles con a , por definición de X_a . Falta ver que es máximo.

Sea Y un agendamiento máximo de X que contiene a a ; existe por suposición. Entonces $Y \setminus \{a\}$ es un agendamiento de X_a (¿por qué?) y, como S es máximo en X_a , se tiene $|Y| - 1 = |Y \setminus \{a\}| \leq |S|$. Luego $|Y| \leq |S| + 1 = |S \cup \{a\}|$. Como Y es máximo en X y $S \cup \{a\}$ es un agendamiento de X , también $|S \cup \{a\}| \leq |Y|$. En conclusión $|S \cup \{a\}| = |Y|$ y $S \cup \{a\}$ es máximo.

Teorema 4.2.3: Optimización global

Sea X un conjunto finito de actividades. El Algoritmo 4.2.1 construye un agendamiento máximo para X .

Demostración

El criterio de selección satisface la propiedad de elección voraz por el Teorema 4.2.1 y el problema tiene subestructura óptima por el Teorema 4.2.2. El conjunto vacío es siempre un agendamiento, de modo que hay al menos una solución factible. Por el Teorema 4.1.1, el Algoritmo 4.2.1 retorna un agendamiento máximo.

Vale la pena detenerse en lo corta que resultó esta demostración. Todo el trabajo está en el Teorema 4.2.1, que es donde se decide si el criterio sirve o no; el Teorema 4.1.1 se encarga del resto de una vez y para siempre. Esta es exactamente la división de labores que anunció la Nota 4.1.4 y que se repetirá en todas las secciones del capítulo.

4.2.4. Otra ruta: el voraz va adelante. La demostración de la subsección anterior no es la única posible y conviene conocer una alternativa porque hay problemas para los cuales es la única que se consigue. En lugar de apoyarse en el

Teorema 4.1.1, se compara directamente lo que construye el algoritmo contra un agendamiento arbitrario y se demuestra que el primero *va adelante* del segundo en todo momento.

Teorema 4.2.4: El voraz va adelante

Sea X un conjunto finito de actividades, sean $a_0, \dots, a_{n-1}$ las actividades que selecciona el Algoritmo 4.2.1 y sean $b_0, \dots, b_{m-1}$ las de un agendamiento cualquiera de X , ambas colecciones ordenadas ascendentemente por tiempo de finalización. Entonces, para todo $0 \leq k < n \wedge m$, la actividad a_k no termina después de b_k .

Demostración

Se procede por inducción sobre k .

Si $k = 0$, la actividad a_0 es una de tiempo de finalización mínimo en todo X y $b_0 \in X$; luego a_0 no termina después de b_0 .

Si $k > 0$, por la hipótesis inductiva a_{k-1} no termina después de b_{k-1} . Como $b_0, \dots, b_{m-1}$ no tiene conflictos y está ordenado por tiempo de finalización, la actividad b_k comienza después de que b_{k-1} termina (¿por qué?) y, por lo tanto, después de que a_{k-1} termina. Es decir, b_k es compatible con todas las actividades en $a_0, \dots, a_{k-1}$ y, en consecuencia, estaba disponible cuando el algoritmo escogió a_k . Como el algoritmo escoge una actividad de tiempo de finalización mínimo entre las disponibles, a_k no termina después de b_k .

Demostración: Segunda demostración del Teorema 4.2.3

El Algoritmo 4.2.1 construye un agendamiento (¿por qué?), de modo que basta ver que $n \geq m$ para cualquier agendamiento $b_0, \dots, b_{m-1}$ de X .

Hacia una contradicción, suponga que $m > n$. Entonces la actividad b_n existe y, por el Teorema 4.2.4 con $k = n - 1$, la actividad a_{n-1} no termina después de b_{n-1} . Como b_n comienza después de que b_{n-1} termina, la actividad b_n es compatible con todas las actividades en $a_0, \dots, a_{n-1}$; es decir, seguía disponible cuando el algoritmo se detuvo. Esto contradice que el algoritmo se detiene únicamente cuando no quedan candidatos disponibles. Luego $n \geq m$.

Nota 4.2.4: ¿Cuál usar?

Los dos argumentos demuestran lo mismo y ninguno es superior al otro en general. El de intercambio es más corto cuando se consigue, porque delega la inducción en el Teorema 4.1.1, y es el que se ensaya primero. Depende de poder exhibir la solución óptima como algo que se puede modificar localmente sin dañar. El de *el voraz va adelante* es más laborioso, pero más flexible: no exige intercambiar nada, solo comparar. Es la herramienta indicada cuando la solución del problema es una secuencia y lo que se quiere comparar es el progreso paso a paso. Se volverá sobre él en la Sección 4.8, donde el argumento de intercambio no está disponible.

4.2.5. Una implementación eficiente. Hasta este punto, el diseño y el análisis del algoritmo voraz se ha hecho con base en una abstracción del problema original, suponiendo que la colección de actividades es un conjunto. De acuerdo con la especificación del problema, la colección de actividades es un arreglo $A[0..N]$ de parejas. Por ello, aún hace falta algo más de trabajo para llegar a tener un algoritmo que opere sobre arreglos, con la estrategia voraz diseñada.

El criterio de selección de este problema es *estático* en el sentido de la Nota 4.1.3: el tiempo de finalización de una actividad no cambia con lo que se haya agendado antes. Aplica entonces la receta de *ordenar y recorrer*, y lo único que falta es evitar hacer un cálculo costoso, en el caso recurrente, de aquellas actividades que son compatibles con la actividad seleccionada.

Suponga que $A[0..N]$ está ordenado ascendentemente por el tiempo de finalización de las actividades. Note que en este caso, si se ha hecho el mejor agendamiento posible para $A[0..n]$, con $0 \leq n \leq N$, las actividades candidatas para extender este mejor agendamiento parcial están en $A[n..N]$. Esto es porque el tiempo de finalización de estas actividades no es menor que el tiempo de finalización de la actividad $n - 1$ (y, por ende, de ninguna actividad antes de ella). Es decir, si se tiene registro de qué actividades han sido exploradas (e.g., $A[0..n]$) y el tiempo de finalización de la última tarea seleccionada allí, entonces se puede recurrir sobre las demás actividades en $A[n..N]$, en orden: se descarta $A[n]$ cuando tiene conflicto con alguna de las ya seleccionadas en $A[0..n]$ (esto se hace con base en el tiempo de finalización de la última actividad seleccionada) o se selecciona porque termina más temprano entre todas aquellas actividades en $A[n..N]$ que no generan conflicto con la selección hecha hasta ese punto.

La siguiente función objetivo, para $0 \leq n \leq N$ y $t \in \mathbb{R}_{\geq 0}$, ayuda a plasmar matemáticamente la idea elaborada en el párrafo anterior:

$\phi(n, t)$: “máxima cantidad de actividades compatibles en $A[n..N]$
que no tienen conflicto con el intervalo $[0..t]$.”

Para resolver el problema de agendamiento de actividades, el objetivo es calcular $\phi(0, 0)$ suponiendo que ninguna actividad tiene tiempos expresados con números negativos, como está estipulado en la especificación del problema.

La definición recurrente de ϕ se establece tomando a n como pivote, y seleccionando o descartando actividades en relación con el intervalo $[0..t)$. Formalmente, para $0 \leq n \leq N$ y $t \in \mathbb{R}_{\geq 0}$, se propone la siguiente definición recurrente de la función objetivo:

$$\phi(n, t) = \begin{cases} 0 & , \text{ si } n = N, \\ \phi(n+1, t) & , \text{ si } n \neq N \wedge A[n][0] < t, \\ 1 + \phi(n+1, A[n][1]) & , \text{ si } n \neq N \wedge A[n][0] \geq t. \end{cases}$$

El caso base corresponde a la situación en la cual no hay actividades para agendar; el agendamiento óptimo tiene tamaño 0. El caso inductivo se divide en dos, siempre suponiendo que hay al menos una actividad que potencialmente puede ser agendada (i.e., $n \neq N$). Si hay conflicto entre la actividad n y el intervalo “protegido” (i.e., $A[n][0] < t$), entonces esta se ignora y se recurre con el resto de las actividades. De lo contrario, si no hay conflicto entre la actividad n y el intervalo protegido (i.e., $A[n][0] \geq t$), entonces se selecciona la actividad y se recurre con las demás que están pendientes por explorar, actualizando a $[0..A[n][1])$ el intervalo protegido (i.e., con el cual se quiere evitar un conflicto).

Nota 4.2.5: Dónde quedó el esquema

Los dos pasos del Algoritmo 4.1.2 siguen ahí, aunque ya no se vean. El paso 2(a) –escoger el candidato óptimo entre los disponibles– lo realiza el ordenamiento previo junto con el recorrido de izquierda a derecha: la actividad n es siempre, entre las que quedan, una de tiempo de finalización mínimo. El paso 2(b) –verificar si el candidato se puede agregar– es la comparación $A[n][0] \geq t$, y el parámetro t es todo el estado que hace falta mantener para hacerla en tiempo constante.

Teorema 4.2.5

Suponga que las actividades en $A[0..N)$ están ordenadas ascendentemente por tiempo de finalización (i.e., $A[i][1] \leq A[j][1]$, para $0 \leq i < j < N$). Entonces:

1. Para $0 \leq n \leq N$ y $t \in \mathbb{R}_{\geq 0}$, el valor de $\phi(n, t)$ es la máxima cantidad de actividades compatibles en $A[n..N)$ que no tienen conflicto con el intervalo $[0..t)$.

2. El valor de $\phi(0, 0)$ es la máxima cantidad de actividades en $A[0..N)$ que se pueden agendar sin conflicto.

Demostración

Se propone como ejercicio al lector.

Con base en la definición de ϕ , es claro cómo diseñar un algoritmo recurrente para su implementación. A continuación se presenta el Algoritmo 4.2.2, como la implementación de la función **act** que permite calcular ϕ .

Algoritmo 4.2.2

```

1 def act(A, N, n, t):
2     ans = None
3     if n==N: ans = 0
4     else:
5         if A[n][0]<t: ans = act(A, N, n+1, t)
6         else: ans = 1 + act(A, N, n+1, A[n][1])
7     return ans

```

A continuación se presenta un ejemplo con un llamado a la función **act** para un conjunto de actividades que, a escala, representan a aquellas en el Ejemplo 4.2.1.

Ejemplo 4.2.4

Considere el siguiente arreglo de parejas que identifica las actividades del Ejemplo 4.2.1:

```

1 A = [ (0, 14), (16, 24), (29, 31),
2       (2, 6), (9, 17), (22, 30),
3       (6, 10), (15, 20), (24, 32),
4       (2, 9), (11, 15), (16, 23), (24, 28) ]

```

Después de ordenar el arreglo ascendentemente por la segunda componente de las parejas, se obtiene:

```

1 >>> A.sort(key = lambda x: x[1])
2 >>> A
3 [ (2, 6), (2, 9), (6, 10), (0, 14), (11, 15),
4   (9, 17), (15, 20), (16, 23), (16, 24), (24, 28),

```

```
5 (22, 30), (29, 31), (24, 32) ]
```

Con base en la versión ordenada del arreglo, se tiene:

```
1 >>> act(A, len(A), 0, 0)
```

```
2 6
```

El Teorema 4.2.6 presenta los resultados de correctitud de la función `act`, junto con sus complejidades temporal y espacial.

Teorema 4.2.6

Si las actividades en $A[0..N)$ están ordenadas ascendentemente por tiempo de finalización, entonces el llamado `act(A, N, 0, 0)` calcula $\phi(0,0)$, y toma tiempo y espacio (sin considerar la pila de ejecución para la recurrencia) $O(N)$ y $O(1)$, respectivamente. Si es necesario ordenar el arreglo, entonces la complejidad temporal es $O(N \log N)$.

Demostración

Note que la función `act` es una transcripción de la función ϕ , agregando los parámetros `A` y `N` que nunca son modificados. El tiempo de ejecución de `act` es linealmente proporcional a la cantidad de elementos en $A[0..N)$, i.e., es $O(N)$. El espacio es $O(1)$ sin tener en cuenta el espacio requerido para una pila de recurrencia sin optimizaciones al código. Teniendo en cuenta el espacio de la pila, el espacio es $O(N)$ debido a los llamados recurrentes.

El resultado coincide con lo que anticipa el Teorema 4.1.2 para un criterio estático: verificar la factibilidad de un candidato toma tiempo constante, de modo que el costo total es $O(N \log N + N) = O(N \log N)$ y queda dominado por el ordenamiento.

Ejercicios

1. Diseñe un algoritmo de búsqueda exhaustiva que enumere todos los subconjuntos de actividades en $A[0..N)$ para resolver el problema de agendamiento.
2. Enumere todos los agendamientos máximos del conjunto de actividades presentado en el Ejemplo 4.2.1.
3. Explique por qué el problema de agendamiento de actividades exhibe las propiedades de subestructura óptima y solapamiento. Ilustre la explicación con un ejemplo.
4. De acuerdo con la Nota 4.2.2, todo agendamiento máximo es maximal pero no al contrario.
 - a) Demuestre la primera afirmación.
 - b) Diseñe un algoritmo que, en tiempo $O(N)$ sobre un arreglo ya ordenado, produzca un agendamiento maximal.
 - c) ¿Produce el Algoritmo 4.2.1 un agendamiento maximal? Justifique.
5. Demuestre que cada una de las siguientes afirmaciones hechas en la demostración del Teorema 4.2.1 es cierta:
 - a) Si $X \neq \emptyset$, entonces $Y \neq \emptyset$.
 - b) Por suposición, Y no tiene conflictos.
 - c) El conjunto Z no tiene conflictos.
6. En la demostración del Teorema 4.2.2 se afirma que $Y \setminus \{a\}$ es un agendamiento de X_a . Demuestre que esta afirmación es correcta.
7. En la demostración del Teorema 4.2.4 se afirma que la actividad b_k comienza después de que b_{k-1} termina. Justifique por qué esta afirmación es cierta y señale exactamente dónde se usa que $b_0, \dots, b_{m-1}$ no tiene conflictos.
8. El Teorema 4.2.4 está enunciado para un agendamiento cualquiera, no necesariamente máximo.
 - a) ¿Se usa esa generalidad en la segunda demostración del Teorema 4.2.3?
 - b) ¿Seguiría siendo cierto el enunciado si las dos colecciones se ordenaran ascendentemente por tiempo de *inicio* en lugar de por tiempo de finalización?
9. 🌈 Considere el criterio de escoger la actividad que está en conflicto con la menor cantidad de las restantes. Construya un contraejemplo que demuestre que no satisface la propiedad de elección voraz. *Ayuda:* no existe ninguno con menos de 11 actividades; conviene pensar en una actividad “central” con pocos conflictos cuya selección arruine el agendamiento.

10. En el planteamiento recurrente de ϕ , el caso en el cual $n \neq N \wedge A[n][0] \geq t$ indica que la actividad n se considera como parte del agendamiento óptimo que se está construyendo. ¿Por qué no hay conflicto si $A[n][0]$ puede ser igual a t ?
11. Justifique con un ejemplo, por qué la suposición de ordenamiento de $A[0..N)$ es necesaria para que el Teorema 4.2.5 sea cierto.
12. Considere la siguiente función objetivo para el problema de agendamiento de actividades, con $0 \leq n \leq N$ y $t \in \mathbb{R}_{\geq 0} \cup \{+\infty\}$:

$$\bar{\phi}(n, t) : \quad \text{“máxima cantidad de actividades compatibles en } A[0..n) \\ \text{que no tienen conflicto con el intervalo } [t.. + \infty).”$$

Para resolver el problema de agendamiento de actividades, el objetivo es calcular $\bar{\phi}(N, +\infty)$. Proponga un planteamiento recurrente para $\bar{\phi}$, y diseñe un algoritmo voraz que la calcule en tiempo $O(N)$ y espacio $O(1)$.

13. Diseñe un algoritmo voraz que resuelva en tiempo $O(N)$ el problema de agendamiento bajo la suposición de que el arreglo $A[0..N)$ está ordenado:
 - a) Ascendentemente por tiempo inicio de las actividades.
 - b) Descendentemente por tiempo de inicio de las actividades.
14. La estrategia voraz diseñada en esta sección está basada en la idea de escoger aquella actividad que termine lo más pronto posible entre aquellas que no generan conflicto, para posteriormente recurrir. Para cada una de las siguientes estrategias voraces, o bien demuestre que sirve como regla de optimización local para resolver el problema de agendamiento de actividades (si hay empates, se debería poder escoger cualquiera de las posibilidades) o describa un pequeño contraejemplo que ilustre por qué no sirve:
 - a) Escoja aquella actividad a que *termina* lo más tarde posible, descarte las actividades que están en conflicto con a y recurra.
 - b) Escoja aquella actividad a que *comienza* lo más tarde posible, descarte las actividades que están en conflicto con a y recurra.
 - c) Si no hay conflicto, escoja todas las actividades; de lo contrario, descarte la actividad con la *mayor duración* y recurra.
 - d) Si no hay conflicto, escoja todas las actividades; de lo contrario, descarte la actividad con la *mayor cantidad de conflictos* y recurra.
15. Diseñe un algoritmo con programación dinámica para calcular ϕ , sin suponer el principio de optimalidad local, y que resuelva el problema de agendamiento de actividades, suponiendo que los tiempos de las actividades en $A[0..N)$ son números naturales.
16. Diseñe un algoritmo iterativo que, en tiempo $O(N)$ y espacio $O(1)$, permita calcular la función ϕ .

17. Suponga ahora que cada actividad n tiene además un *valor* $V[n] > 0$ y que el objetivo es maximizar la suma de los valores de las actividades agendadas, en lugar de su cantidad.
- Demuestre que el criterio de esta sección –escoger la que termine más temprano– deja de satisfacer la propiedad de elección voraz.
 - Refute también el criterio de escoger la actividad de mayor valor.
 - Explique por qué el Teorema 4.2.2 sí sigue siendo cierto para esta variante, y qué dice eso sobre la técnica que conviene usar.

4.3. Una metodología

Con base en las ideas de la Sección 4.1 y en el ejemplo desarrollado en la Sección 4.2, se propone una metodología para diseñar algoritmos voraces. Es deliberadamente análoga a la Metodología 3.3.1. Como aquella, no debe entenderse como una receta infalible, sino como una lista de las preguntas que conviene responder y en qué orden.

Hay, eso sí, una diferencia importante con las metodologías de los otros capítulos y conviene anunciarla de una vez. Allá, si se siguen los pasos con cuidado, se obtiene un algoritmo correcto. Acá no: hay un paso, el de *optimización local*, en el cual el diseño puede sencillamente fracasar, y entonces no hay más remedio que volver atrás y empezar de nuevo con otro criterio, o abandonar la técnica. Esa posibilidad de fracaso no es un defecto de la metodología, sino la naturaleza del asunto: los algoritmos voraces son la excepción y no la regla.

Metodología 4.3.1: Algoritmos voraces

Especificación: especificar el problema, indicando claramente su entrada y su salida, y reconocer en el enunciado los tres elementos de la Definición 4.1.1: cuáles son las soluciones factibles, cuál es la función objetivo y cuál es el sentido. Vale la pena insistir en separar la factibilidad de la optimalidad, pues confundirlas es la fuente más común de especificaciones equivocadas. Si de esta lectura no resulta un problema de optimización, la técnica de este capítulo no aplica.

Candidatos: determinar el conjunto C de candidatos y qué es una solución parcial, en el sentido de la Definición 4.1.2. Un candidato es aquello con lo que el algoritmo se compromete de una sola vez, de modo que este paso fija la granularidad de las decisiones. Casi nunca es único: el mismo problema admite descomposiciones distintas y no todas conducen a un criterio que funcione.

Criterio de selección: proponer *varios* criterios de selección, en el sentido de la Definición 4.1.3, y no solo uno. Cada uno debe ser un preorden total sobre C y debe poder evaluarse sin conocer la respuesta al problema. La heurística usual para inventarlos es preguntarse qué recurso escaso consume cada candidato –tiempo, capacidad, presupuesto– y ordenar los candidatos por cuánto consumen o aportan, o por la razón entre ambas cantidades.

Refutación: antes de intentar demostrar nada, intentar romper cada criterio propuesto con instancias pequeñas. Este paso es económico, rápido y casi siempre exitoso; la Nota 4.1.5 explica por qué conviene hacerlo primero. Un buen procedimiento es implementar el criterio y una solución de fuerza bruta, y compararlas sobre instancias aleatorias diminutas. Tenga presente que este paso nunca concluye afirmativamente: que un criterio sobreviva no demuestra nada, solo indica cuál merece el esfuerzo del paso siguiente.

Optimización local: demostrar que el criterio sobreviviente satisface la propiedad de elección voraz de la Definición 4.1.4. Es el paso difícil, el único que puede fracasar y aquel en el cual reside toda la corrección del algoritmo. La subsección siguiente cataloga los patrones de demostración disponibles.

Optimización global: verificar que el problema tiene subestructura óptima, en el sentido de la Definición 4.1.5, y concluir con el Teorema 4.1.1. La verificación suele ser rutinaria –comprometerse con un candidato deja un problema del mismo tipo sobre menos candidatos– pero no debe omitirse. Si el paso anterior se hizo con el patrón de *el voraz va adelante*, la conclusión global se obtiene directamente de la comparación y no hace falta invocar el teorema.

Función objetivo: proponer una función objetivo ϕ y definirla recurrentemente, tal y como se hace en el Capítulo 3. Este paso no aporta corrección –ya se tiene– pero convierte la estrategia abstracta en algo que se transcribe mecánicamente a código y, de paso, hace explícita la relación entre las dos técnicas.

Implementación: instanciar el Algoritmo 4.1.2 y escoger las estructuras de datos. Aquí decide todo la Nota 4.1.3: si el criterio es estático, basta ordenar C una vez y recorrerlo; si es dinámico, se necesita una estructura de datos auxiliar como una cola de prioridad, por ejemplo. Considere por separado qué estructura hace falta para verificar la factibilidad de un candidato, que puede ser distinta de la que entrega el mínimo o el máximo.

Análisis: establecer las complejidades temporal y espacial con el Teorema 4.1.2, identificando el costo $O(f)$ de verificar la factibilidad y el costo $O(g)$ de escoger el candidato óptimo. En la mayoría de los casos el resultado queda dominado por el ordenamiento inicial o por las operaciones sobre la estructura de datos auxiliar.

Los pasos anteriores se ilustrarán con cada uno de los ejemplos que se presentan en el resto del capítulo. De hecho, la Sección 4.2 ya los siguió todos, aunque sin nombrarlos.

Paso	En la Sección 4.2
Especificación	Agendamientos, su tamaño, maximizar
Candidatos	Las N actividades
Criterio de selección	Cuatro propuestas
Refutación	Dos rotas con tres actividades
Optimización local	Teorema 4.2.1, por intercambio
Optimización global	Teoremas 4.2.2 y 4.2.3
Función objetivo	$\phi(n, t)$ y su definición recurrente
Implementación	Ordenar por tiempo de finalización y recorrer
Análisis	$O(N \log N)$, dominado por el ordenamiento

4.3.1. Tres patrones de demostración. El paso de optimización local es el que concentra la dificultad y no existe un procedimiento que lo resuelva siempre. Lo que sí existe es un repertorio pequeño de formas de argumentar que cubre la gran mayoría de los casos. Esta subsección las describe y dice cuándo conviene ensayar cada una.

Argumento de intercambio. Es el patrón por defecto y el primero que hay que ensayar. Se parte de una solución óptima *arbitraria* Y y se demuestra que se puede transformar en otra, igual de buena, que contiene la elección voraz c . Si $c \in Y$ no hay nada que hacer; de lo contrario, se identifica en Y el elemento b al cual c puede sustituir, se define $Z = (Y \setminus \{b\}) \cup \{c\}$, y se demuestra que Z sigue siendo factible y que su valor objetivo no es peor que el de Y .

Lo que hace funcionar el intercambio es siempre alguna propiedad óptima de c : que termine lo más temprano posible, que pese lo menos posible, que se extienda lo más lejos posible. Al diseñar el argumento, la pregunta a responder es la que plantea la Nota 4.2.3: *¿qué le permite a la elección voraz reemplazar a cualquier otra sin causar daño?* Aparece en la Sección 4.2, en la Sección 4.6 y en la Sección 4.7.

Argumento de inversión. Es la variante del intercambio para problemas cuya solución no es un subconjunto sino un *ordenamiento* de todos los candidatos. Allí no hay nada que escoger —todos los candidatos van en la solución— y lo único

que se decide es en qué orden, de modo que la propiedad de elección voraz hay que reformularla.

La forma del argumento es la siguiente. Se define qué es una *inversión*: una pareja de elementos que aparecen en un orden contrario al que dicta el criterio. Se demuestra que intercambiar los dos elementos de una inversión mejora estrictamente el valor objetivo. Se concluye que una solución óptima no puede tener inversiones y que, por lo tanto, está ordenada según el criterio. El intercambio ocurre aquí entre dos elementos de la misma solución, y no entre una solución y un candidato externo; por lo demás, el espíritu es idéntico. Aparece en la Sección 4.5.

El voraz va adelante. Cuando ninguna de las dos formas anteriores se deja construir, queda comparar directamente lo que el algoritmo produce contra una solución arbitraria, decisión por decisión. Se ordenan ambas soluciones de acuerdo con el orden en que se tomaron las decisiones, se define una medida de progreso tras k decisiones y se demuestra, por inducción sobre k , que la medida del algoritmo voraz nunca es peor que la de la otra solución. De ahí se concluye que la solución voraz es al menos tan buena.

La ventaja de este patrón es que no exige modificar la solución óptima, sino únicamente compararla; por eso funciona en situaciones en las cuales el intercambio no está disponible. La desventaja es que hay que inventar la medida de progreso, que no siempre es evidente, y que la inducción es más laboriosa. Aparece en la Sección 4.2, como demostración alternativa y en la Sección 4.8 donde es la única disponible.

La siguiente tabla resume el repertorio.

Patrón	¿Qué se compara?	La solución es
Intercambio	Una óptima con ella misma modificada	un subconjunto
Inversión	Una óptima con ella misma reordenada	un ordenamiento
Va adelante	La voraz con una óptima, paso a paso	una secuencia

Nota 4.3.1: Cuando ninguno funciona

Si los tres patrones se resisten, hay tres explicaciones posibles y conviene descartarlas en este orden.

La primera, y con mucho la más probable, es que el criterio de selección sea sencillamente incorrecto y que el contraejemplo exista pero no se haya encontrado. Antes de insistir en la demostración, vuelva al paso de refutación y busque con más empeño o con instancias más grandes.

La segunda es que el criterio sea correcto pero requiera un argumento distinto. El repertorio de esta subsección es el usual, no el exhaustivo; en

particular, la teoría de matroides ofrece un cuarto camino, que caracteriza de manera abstracta una clase de problemas en los cuales toda estrategia voraz funciona (ver las notas al final del capítulo).

La tercera es que el problema sencillamente no admita una solución voraz. Esto no es una derrota: si en el paso de optimización global se verificó que hay subestructura óptima, entonces el problema es candidato para las técnicas del Capítulo 3 y el trabajo hecho hasta ahora –los candidatos, las decisiones, la función objetivo– se aprovecha íntegro. Si ni siquiera hay subestructura óptima, queda el Capítulo 5.

Ejercicios

1. Considere el problema de determinar la mínima cantidad de salas necesarias para realizar todas las actividades de un arreglo $A[0..N)$, de manera que dos actividades en conflicto nunca se programen en la misma sala.
 - a) Siga, paso a paso, la metodología propuesta en esta sección, indicando claramente en qué consiste cada uno de los pasos. Debe resultar código en Python ejecutable.
 - b) Para el paso de *criterio de selección*, proponga al menos tres criterios y refute los que pueda.
 - c) Identifique cuál de los tres patrones usó en el paso de *optimización local* y explique por qué los otros dos no se acomodan bien a este problema.
2. Considere el problema de llevar la mayor cantidad posible de objetos en un morral de capacidad W , dado un arreglo $P[0..N)$ con los pesos de los objetos. Note que interesa la *cantidad* de objetos y no su valor.
 - a) Aplique la metodología completa.
 - b) Compare el resultado con el problema del morral del Capítulo 3 y explique, en términos de la Definición 4.1.4, por qué aquel no admite solución voraz y este sí.
3. Para cada uno de los siguientes problemas, indique cuál de los tres patrones de demostración ensayaría primero y justifique brevemente su elección. No es necesario construir las demostraciones.
 - a) Dado un arreglo de números, ordenarlos de manera que la concatenación de sus representaciones decimales sea el número más grande posible.
 - b) Dado un conjunto de puntos sobre una recta, cubrirlos todos con la menor cantidad posible de intervalos de longitud fija L .
 - c) Dado un arreglo de tareas, cada una con una duración y una fecha límite, ordenarlas de manera que se minimice el máximo retraso.

4. La metodología de esta sección y la de la Sección 3.3 comparten varios pasos.
 - a) Empareje los pasos que se corresponden entre las dos.
 - b) Identifique los pasos que existen en una y no en la otra, y explique en cada caso por qué.
5. El paso de *refutación* se describe como económico y casi siempre exitoso. Escriba un programa que, dado un problema de optimización sobre arreglos pequeños de números, un criterio de selección y una solución de fuerza bruta, busque el contraejemplo más pequeño mediante enumeración exhaustiva de instancias. Úselo sobre alguno de los criterios refutados en la Sección 4.2.
6. 🌐 La Nota 4.3.1 afirma que si hay subestructura óptima pero el criterio falla, el problema es candidato para programación dinámica. Formule y demuestre esa afirmación con precisión, indicando qué hipótesis adicionales hace falta suponer sobre el conjunto de candidatos para que el algoritmo de programación dinámica resultante sea eficiente.

4.4. Cubrimiento de intervalos

Suponga que hay un evento de interés público a ser cubierto por una agencia de noticias. El evento cuenta con una hora de inicio y una de finalización. Además, la agencia de noticias conoce la disponibilidad de horarios de periodistas que podrían cubrir el evento, en donde cada periodista compromete una franja de trabajo determinada por una hora de inicio y otra de finalización. Con base en este contexto: ¿cuál es la mínima cantidad de periodistas que debe desplegar la agencia de noticias para cubrir la totalidad del evento?

Esta pregunta se puede formular a modo de problema algorítmico de la siguiente manera.

Problema 4.4.1: Cubrimiento mínimo de intervalos

Entrada: Un arreglo $A[0..N)$, $N \geq 0$, de parejas de números (s_n, e_n) tales que $s_n < e_n$ indican la hora de inicio y finalización que puede cubrir el periodista n , $0 \leq n < N$, y S y E que indican el tiempo de inicio y finalización del evento, $S < E$.

Salida: Mínima cantidad de periodistas con turnos en $A[0..N)$ requeridos para cubrir la totalidad del evento que inicia en S y termina en E .

Al igual que con el agendamiento de actividades (Sección 4.2), como convención para cualquier n , con $0 \leq n < N$, las expresiones $A[n][0]$ y $A[n][1]$ denotan, respectivamente, el tiempo de inicio y finalización del turno que puede asumir el

periodista n : es decir, $s_n = A[n][0]$ y $e_n = A[n][1]$. Note que todos los tiempos pueden incluir valores negativos.

Nota 4.4.1

Por convención, se entenderá que cada turno (s, e) representa el intervalo cerrado-abierto de números reales $[s..e)$. De esta forma, el evento a cubrir corresponde al intervalo $[S..E)$.

Al problema de cubrimiento de intervalos, como se plantea anteriormente, se le pueden asociar algunas suposiciones. Estas harán más fácil su abordaje y permitirán enfocar los esfuerzos en resolver la esencia del problema mismo, sin que estas cambien su naturaleza ni su complejidad intrínsecas (ver Ejercicio 3). Las suposiciones son las siguientes:

1. Los turnos en $A[0..N)$ cubren completamente el evento, es decir, los intervalos asociados a los turnos de los periodistas son un *cubrimiento* del intervalo determinado por el evento:

$$[S..E) \subseteq (\cup n \mid 0 \leq n < N : [s_n..e_n)).$$

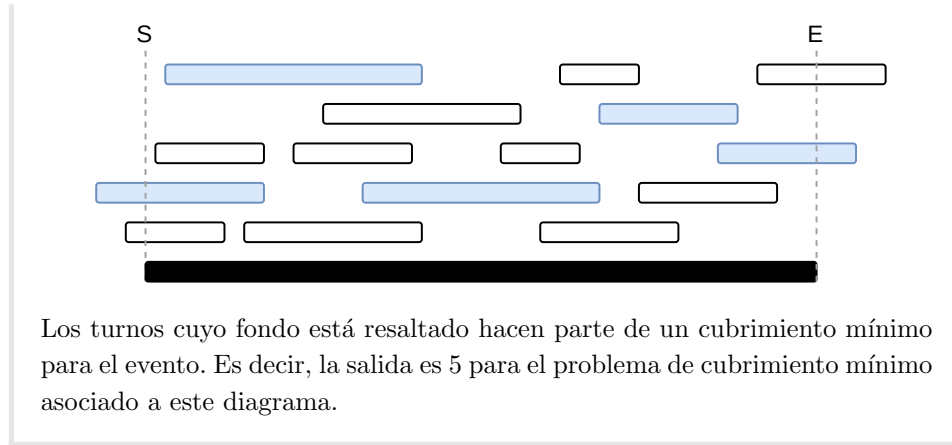
2. Los turnos asociados a los periodistas están relacionados con el evento; es decir, para cualquier $0 \leq n < N$ se cumple:

$$[s_n..e_n) \cap [S..E) \neq \emptyset.$$

La primera condición establece que los turnos de los periodistas son suficientes para cubrir la totalidad del evento. Si esta condición no se cumple, el problema especificado no tiene solución. La segunda condición establece que los turnos de los periodistas están relacionados con el evento. Es decir, que cada uno puede aportar a cubrir alguna porción del evento. Esta condición garantiza que ningún turno a ser considerado puede terminar antes del inicio del evento ni iniciar después de la finalización del evento; es decir, este tipo de turnos se pueden ignorar tranquilamente.

Ejemplo 4.4.1

Se ilustra cada turno como un rectángulo cuya coordenada izquierda en el eje horizontal denota el tiempo de inicio y la derecha el tiempo de finalización; hay 15 turnos en el diagrama. El rectángulo cuyo fondo es negro representa el evento, con tiempos S y E de inicio y finalización, respectivamente.



Leído con el vocabulario de la Sección 4.1, los tres elementos de la Definición 4.1.1 están en el enunciado del Problema 4.4.1: las *soluciones factibles* son los subconjuntos de turnos que cubren $[S..E)$, la *función objetivo* de uno de ellos es su tamaño y el *sentido* es minimizar. El conjunto de candidatos son los N turnos.

Al igual que con el problema de agendamiento de actividades (Sección 4.2), el problema de cubrimiento mínimo se puede resolver exhaustivamente enumerando todos los subconjuntos de turnos y, entre aquellos que cubran $[S..E)$, identificar el tamaño mínimo posible. Es más, este problema también exhibe las propiedades de la subestructura óptima y solapamiento (¿por qué?), y por ello es candidato a ser resuelto con programación dinámica (ver Ejercicio 13).

4.4.1. Abstracción y estrategia voraz. Cualquier estrategia voraz que se plantee para resolver este problema debe considerar al menos dos aspectos. Primero, el de optimización: minimizar la cantidad de turnos. Segundo, garantizar que los turnos seleccionados cubran el evento completamente. Note que el primer aspecto, el de optimización, no tiene mucha gracia sin el segundo aspecto (¿por qué?).

Considere la situación general en la cual un conjunto de intervalos cerrados-abiertos A cubre completamente un intervalo $[s..e)$, con $s < e$. El objetivo es identificar un subconjunto de A , lo más pequeño posible, que cubra el mismo intervalo. Note que alguno de los intervalos en A debe incluir el extremo s (Suposición 1). Cualquier intervalo $[s_i..e_i)$ que incluya a s debe satisfacer $s_i \leq s$ (Suposición 2). Luego, alguno de estos intervalos debe ser parte de una solución óptima.

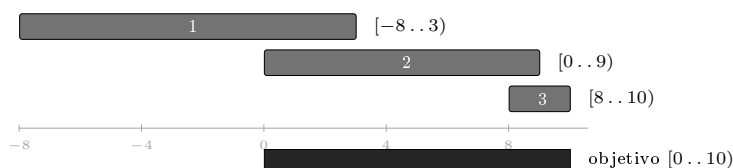
La pregunta es cuál y se responde con ayuda del criterio de selección. Note que apenas se hizo una observación que restringe severamente el conjunto de candidatos elegibles: solo tiene sentido escoger entre los intervalos que incluyen a s . Los tres criterios siguientes respetan esa restricción y difieren en cómo desempatan entre ellos. Escoger, entre los intervalos que incluyen a s , aquel que:

1. comienza más temprano;
2. es más largo; o
3. termina más tarde.

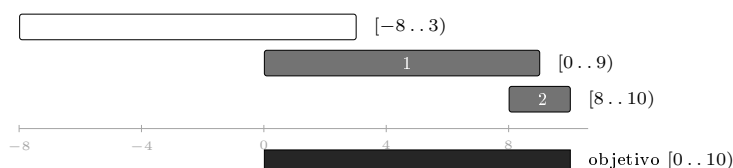
Siguiendo la recomendación de la Nota 4.1.5, conviene intentar romperlos antes de intentar demostrar alguno. Los dos primeros caen con la misma instancia y con solo tres intervalos.

Ejemplo 4.4.2: Dos criterios que no funcionan

(a) y (b): *comienza más temprano o el más largo*, 3 intervalos



(c): *termina más tarde*, 2 intervalos



Se trata de cubrir $[0..10]$ con los intervalos $[-8..3]$, $[0..9]$ y $[8..10]$.

Los dos primeros criterios escogen $[-8..3]$: es el que comienza más temprano entre los que incluyen a 0 y también el más largo de los dos. Hecha esa elección, queda por cubrir $[3..10]$ y hacen falta los otros dos intervalos, para un total de tres.

El tercer criterio escoge $[0..9]$, que es el que termina más tarde. Queda por cubrir $[9..10]$, que se resuelve con $[8..10]$, para un total de dos.

El contraejemplo señala con precisión qué mide cada criterio. La longitud de un intervalo y su extremo inferior son irrelevantes: lo único que importa es hasta dónde alcanza *hacia la derecha*, porque es lo único que reduce el subproblema que queda por resolver. Entre más se extienda el intervalo a elegir, más pequeño será ese subproblema. Es decir, la intuición sugiere que entre todos los intervalos que incluyen a s , es más conveniente seleccionar uno de los que termine más tarde (puede haber más de uno). Esta será la apuesta de una estrategia voraz para resolver el problema de cubrimiento mínimo de intervalos. Así, un algoritmo voraz utilizaría este principio de optimización local incluyendo el extremo menor del intervalo y

cubriendo lo que más se pueda después de este, cada vez resultando un subproblema más pequeño por resolver (i.e., un subintervalo más pequeño por cubrir), hasta agotarlo completamente.

Nota 4.4.2: Un criterio dinámico

Hay una diferencia importante con el criterio de la Sección 4.2 que conviene notar desde ahora. Allá, el criterio –el tiempo de finalización de una actividad– no dependía de nada de lo que el algoritmo hubiera hecho antes. Acá sí: cuál es el mejor intervalo depende de s y s cambia con cada selección. Se trata, en los términos de la Nota 4.1.3, del primer criterio *dinámico* del capítulo.

Note además que la restricción de elegibilidad –incluir a s – cumple aquí el papel que en la Sección 4.2 cumplía la verificación de factibilidad. En este problema, esa verificación sería vacía: por la Suposición 1, cualquier subconjunto de A se puede extender hasta un cubrimiento, de modo que ningún candidato es rechazable por infactible. Todo el trabajo lo hace el criterio.

A continuación se presenta la formalización y la demostración de este principio de optimización local para el problema de cubrimiento mínimo de intervalos.

Teorema 4.4.1: Optimización local

Sea A un conjunto finito de intervalos que cubren $[s..e)$. Si $[s_n..e_n) \in A$ es tal que incluye a s y e_n es máximo entre todos los intervalos en A que incluyen a s , entonces $[s_n..e_n)$ hace parte de un conjunto mínimo de intervalos en A que cubre $[s..e)$.

Demostración

Suponga que $Y \subseteq A$ es un conjunto mínimo de intervalos que cubren $[s..e)$. Como $s < e$, necesariamente $A \neq \emptyset$ y $Y \neq \emptyset$ (¿por qué?). Se procede por casos:

- Si $[s_n..e_n) \in Y$, entonces la conclusión del teorema es correcta.
- Si $[s_n..e_n) \notin Y$, entonces hay un intervalo, por ejemplo $[s_k..e_k) \in Y$, con $k \neq n$, que incluye a s . En efecto, $s \in [s..e)$ porque $s < e$ y Y cubre $[s..e)$. Considere el conjunto $Z = (Y \setminus \{[s_k..e_k)\}) \cup \{[s_n..e_n)\}$. Como $k \neq n$, necesariamente $|Z| = |Y|$, concluyendo que Z cumple con el requerimiento de minimalidad (dado que Y es mínimo). Falta ver que Z también cubre $[s..e)$, para lo cual basta ver que $[s_n..e_n)$ aporta al menos tanto como

$[s_k..e_k)$ dentro de $[s..e)$. Como ambos incluyen a s , se tiene $s_k \leq s$ y $s_n \leq s$; y como e_n es máximo en A con respecto a todos aquellos intervalos que incluyen a s , se tiene $e_k \leq e_n$ en particular. Luego:

$$[s_k..e_k) \cap [s..e) = [s..e_k \downarrow e) \subseteq [s..e_n \downarrow e) = [s_n..e_n) \cap [s..e).$$

Es decir, todo punto de $[s..e)$ que cubría $[s_k..e_k)$ lo cubre también $[s_n..e_n)$ y por lo tanto Z es un cubrimiento de $[s..e)$.

En conclusión, $[s_n..e_n)$ hace parte de un conjunto mínimo de intervalos en A que cubre $[s..e)$.

Nota 4.4.3: Otra vez el intercambio

La demostración del Teorema 4.4.1 es, paso por paso, la misma de la Nota 4.2.3: se parte de una solución óptima arbitraria, se identifica en ella el elemento al cual la elección voraz puede sustituir y se verifica que el reemplazo no hace daño. Lo único que cambia es la propiedad de optimalidad que hace legítimo el reemplazo. Allá era que la actividad escogida no termina después de ninguna otra; acá es que el intervalo escogido no termina antes que ningún otro de los que incluyen a s .

La estrategia planteada en el Teorema 4.4.1 constituye el principio de diseño para obtener una solución voraz al problema de cubrimiento mínimo de intervalos, como lo plantea recurrentemente el Algoritmo 4.4.1.

Algoritmo 4.4.1

Para un conjunto (finito) de intervalos cerrados-abiertos A y un intervalo $[s..e)$ cubierto por A :

1. Si $s \geq e$, entonces no seleccionar nada.
2. De lo contrario, sea $[s_n..e_n)$ un intervalo en A que incluye a s y con e_n máximo entre todos los intervalos en A que incluyen a s :
 - a) seleccionar $[s_n..e_n)$ como parte del cubrimiento mínimo y
 - b) recurrir con los intervalos en A para cubrir el intervalo $[e_n..e)$.

El Algoritmo 4.4.1 considera un caso base y uno inductivo para determinar un cubrimiento mínimo de $[s..e)$ con intervalos en A . El caso base está determinado por la condición $s \geq e$ y corresponde a la situación en la cual el intervalo a ser cubierto es vacío. Por ende, no debe hacerse nada. El caso inductivo, supone que hay algo que cubrir y se usa el Teorema 4.4.1 directamente. Al recurrir, se selecciona

a $[s_n..e_n)$ para cubrir el extremo de $[s..e)$ que incluye a s dado que e_n es máximo entre todos ellos y se recurre para cubrir el subintervalo $[e_n..e)$, con los mismos intervalos disponibles en A , teniendo así en cuenta la contribución hecha por el intervalo $[s_n..e_n)$ al cubrimiento de $[s..e)$. Note que no es necesario hacer casos sobre A (si es vacío o no); basta con hacer casos sobre la relación entre s y e (¿por qué?).

4.4.2. Optimización global. Con la propiedad de elección voraz ya demostrada, falta verificar la subestructura óptima para poder concluir.

Teorema 4.4.2: Subestructura óptima

Sea A un conjunto finito de intervalos que cubren $[s..e)$, con $s < e$, y sea $[s_n..e_n) \in A$ un intervalo que incluye a s y que hace parte de algún cubrimiento mínimo de $[s..e)$. Si $S \subseteq A$ es un cubrimiento mínimo de $[e_n..e)$, entonces $S \cup \{[s_n..e_n)\}$ es un cubrimiento mínimo de $[s..e)$.

Demostración

El conjunto $S \cup \{[s_n..e_n)\}$ cubre $[s..e)$: el intervalo $[s_n..e_n)$ cubre $[s..e_n)$ porque $s_n \leq s < e_n$ y S cubre lo que resta. Falta ver que es mínimo.

Sea Y un cubrimiento mínimo de $[s..e)$ con $[s_n..e_n) \in Y$; existe por suposición. Entonces $Y \setminus \{[s_n..e_n)\}$ cubre $[e_n..e)$ (¿por qué?) y, como S es mínimo, se tiene $|Y| - 1 \leq |S|$. Luego:

$$|S \cup \{[s_n..e_n)\}| \leq |S| + 1 \leq |Y|.$$

Como Y es mínimo y $S \cup \{[s_n..e_n)\}$ es un cubrimiento de $[s..e)$, las dos desigualdades son igualdades y $S \cup \{[s_n..e_n)\}$ es mínimo.

Teorema 4.4.3: Optimización global

Sea A un conjunto (finito) de intervalos cerrado-abiertos que cubren el intervalo $[s..e)$. El Algoritmo 4.4.1 identifica un cubrimiento (mínimo) para $[s..e)$ con intervalos en A .

Demostración

El criterio de selección satisface la propiedad de elección voraz por el Teorema 4.4.1 y el problema tiene subestructura óptima por el Teorema 4.4.2.

El conjunto A cubre $[s..e)$ por suposición, de modo que hay al menos una solución factible. Por el Teorema 4.1.1, el Algoritmo 4.4.1 identifica un cubrimiento mínimo.

Nota 4.4.4: La misma demostración dos veces

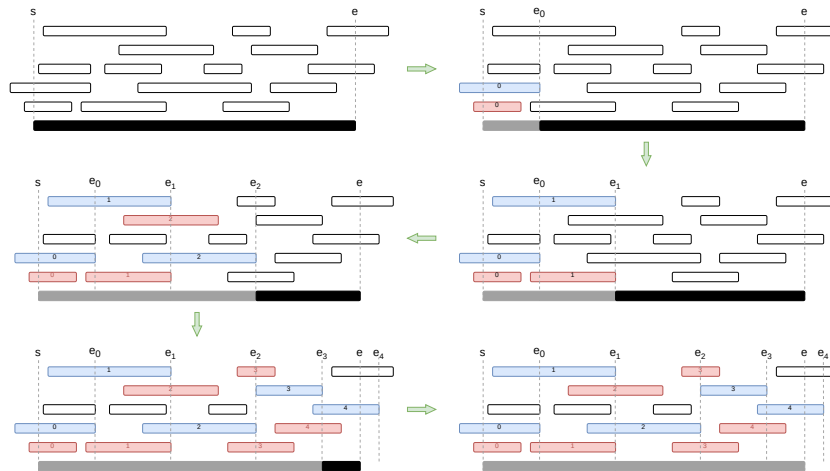
La demostración del Teorema 4.4.3 y la del Teorema 4.2.3 son, salvo por los nombres, idénticas: se invoca el teorema local, se invoca la subestructura óptima y se concluye con el Teorema 4.1.1. Eso no es una coincidencia ni una economía de la exposición, sino exactamente lo que la Nota 4.1.4 anunció: una vez demostrado el teorema de optimización local, el paso global ya está hecho de una vez y para siempre.

Vale la pena notar un detalle de la correspondencia con el Algoritmo 4.1.2. Allí, el candidato escogido se remueve de C en cada iteración, mientras que el Algoritmo 4.4.1 recurre con todos los intervalos de A . La diferencia es inofensiva: como $[s_n..e_n)$ no incluye a e_n , ese intervalo no vuelve a ser elegible en ninguna llamada posterior, y removerlo o no da lo mismo (ver Ejercicio 5).

El Ejercicio 4 propone una tercera demostración, esta vez con el patrón de *el voraz va adelante* de la Sección 4.2, que no necesita ninguno de los dos teoremas anteriores.

Ejemplo 4.4.3

El siguiente diagrama muestra, paso a paso, cómo el Algoritmo 4.4.1 selecciona un cubrimiento mínimo para el intervalo $[s..e)$, como en el Ejemplo 4.4.1.



En el primer paso, correspondiente al caso recurrente, se identifican dos intervalos que incluyen a s (con etiqueta 0); se escoge el intervalo que termina más tarde (color azul) y se descarta el otro (color rojo). Al recurrir, el intervalo a ser cubierto corresponde a $[e_0..e)$, lo cual se indica en la barra inferior con color negro (el color gris representa la parte del intervalo que ha sido cubierta). En la medida en que avanza el algoritmo, los intervalos seleccionados van siendo etiquetados con el paso correspondiente, y coloreados dependiendo de si son seleccionados o no. Además, el intervalo a cubrir se va reduciendo hasta ser cubierto completamente. Esta situación se observa en el último paso, que se resuelve con el caso base del algoritmo, dado que $e_4 > e$. Observe que hay intervalos que nunca son considerados (los que no tienen ningún color de relleno).

4.4.3. Una implementación eficiente. Una implementación directa del Algoritmo 4.4.1 resultaría en tiempo cuadrático en el tamaño de A (i.e., $O(N^2)$). Observe que para cada extremo inferior de s , se debe determinar la colección de intervalos en A que contienen a s y, entre ellos, seleccionar uno de los que tenga mayor extremo superior, lo cual tomaría tiempo $O(N)$. Como hay, en el peor de los casos, N extremos inferiores, entonces se deberían hacer $O(N^2)$ operaciones básicas en total.

Para una implementación eficiente se propone ordenar la colección de intervalos en $A[0..N)$ ascendentemente por los extremos inferiores, desempatando arbitrariamente cuando haya empates. Suponiendo, tal y como lo propone la estrategia voraz, ir cubriendo el intervalo objetivo $[S..E)$ del extremo inferior al superior, la situación general que se tiene es la siguiente: cubrir $[s..E)$ (con $s \geq S$) con intervalos en $A[n..N)$ (con $0 \leq n \leq N$) sabiendo que el mejor intervalo en $A[0..n)$ que incluye a s termina en a (con $a \geq s$). Si $n = N$, entonces ya se han agotado todos los intervalos en A y no hay nada más que hacer. De lo contrario, el intervalo $A[n] = (s_n, e_n)$ debe ser considerado y se dan, principalmente, dos opciones: $s \in [s_n..e_n)$ o $s \notin [s_n..e_n)$. Con la primera opción, necesariamente el intervalo $A[n]$ se debe considerar cuando $e_n > a$; de lo contrario no aportaría nada mejor y se podría descartar. Con la segunda opción, o bien el intervalo $A[n]$ termina antes de s (i.e., $e_n \leq s$) o inicia después de s (i.e., $s_n > s$). Como A está ordenado ascendentemente por tiempo de inicio, en la primera situación se puede ignorar el intervalo y avanzar. En la segunda situación, por la misma razón, se concluye que el mejor cubrimiento para s termina en a y $A[n..N)$ debe ser usado para cubrir $[a..E)$. Con este diseño, la mayoría de los intervalos se visitará una vez y como máximo dos veces. Es decir, la complejidad es lineal en N , pagando el “precio” de ordenarlo previamente (lo cual se puede hacer en $O(N \log N)$).

Nota 4.4.5: Dinámico sin cola de prioridad

La Nota 4.1.3 advierte que un criterio dinámico obliga, en general, a mantener una estructura que entregue el mejor candidato y admita modificaciones. El diseño anterior muestra que no siempre hace falta: acá basta un ordenamiento hecho una sola vez al comienzo y un recorrido de izquierda a derecha.

Lo que lo permite es una propiedad particular de este problema. El parámetro del que depende el criterio –el extremo s por cubrir– solo puede crecer, y los intervalos están ordenados por su extremo inferior. En consecuencia, los candidatos elegibles se recorren en el mismo orden en que van apareciendo y basta con acumular en a el mejor visto hasta el momento. Cuando esa monotonía no está disponible, no queda más remedio que usar una cola de prioridad, como ocurrirá en la Sección 4.7 y en la Sección 4.8.

La siguiente función objetivo, para $0 \leq n \leq N$, y $s, a \in \mathbb{R}$ con $s \geq S$ y $a \geq S$, plasma matemáticamente la idea elaborada en el párrafo anterior.

$\phi(n, s, a)$: “mínima cantidad de intervalos en $A[n..N)$ necesarios para cubrir $[s..E)$ con un cubrimiento de s que se extiende hasta a .”

Para resolver el problema de cubrimiento mínimo de intervalos, el objetivo es entonces calcular $\phi(0, S, S)$ bajo las condiciones establecidas en el enunciado del problema y las dos suposiciones hechas previamente.

La definición recurrente de ϕ se establece tomando a n como pivote, y seleccionando o descartando intervalos en $A[n..N)$ dependiendo de la relación de $A[n]$ con el intervalo $[s..E)$ a ser cubierto. Formalmente, para $0 \leq n \leq N$, y $s, a \in \mathbb{R}$ con $s \geq S$ y $a \geq S$, se propone la siguiente definición recurrente de la función objetivo:

$$\phi(n, s, a) = \begin{cases} 0 & , \text{ si } n = N, \\ \phi(n+1, s, a \uparrow A[n][1]) & , \text{ si } n \neq N \wedge A[n][0] \leq s, \\ 1 + \phi(n+1, a, a \uparrow A[n][1]) & , \text{ si } n \neq N \wedge A[n][0] > s. \end{cases}$$

El caso base corresponde a la situación en la cual no hay intervalos por considerar. El caso inductivo consta de dos subcasos. El primero de ellos, especifica la situación en la cual el intervalo $A[n]$ puede contribuir al cubrimiento de s : si mejora lo que se ha encontrado anteriormente, entonces se recurre avanzando en los intervalos y actualizando el valor del mejor cubrimiento hasta ahora encontrado para s ; de lo contrario, el intervalo $A[n]$ se ignora y se avanza. El segundo de ellos, especifica la situación en la cual el intervalo $A[n]$ (y los intervalos de allí en adelante) no puede contribuir al cubrimiento de s : entonces, se ha encontrado el mejor intervalo que cubre a s y se avanza tratando de encontrar el mejor cubrimiento para $[a..e)$ con

intervalos en $A[n+1..N]$ con un cubrimiento de a que se extiende hasta $a \uparrow A[n][1]$ (porque $A[n]$ puede ser parte de los intervalos que cubren a a).

Teorema 4.4.4

Suponga que los intervalos en $A[0..N]$ están ordenados ascendentemente por el extremo inferior (i.e., $A[i][0] \leq A[j][0]$, para $0 \leq i < j < N$). Entonces:

1. Para $0 \leq n \leq N$ y $s, e, a \in \mathbb{R}$ con $a \geq s$, el valor de $\phi(n, s, a)$ es la mínima cantidad de intervalos en $A[n..N]$ necesarios para cubrir $[s..E]$ con un cubrimiento de s que se extiende hasta a .
2. El valor de $\phi(0, S, S)$ es la mínima cantidad de intervalos en $A[0..N]$ que cubren $[S..E]$.

Demostración

Se propone como ejercicio al lector.

Con base en la definición de ϕ , es claro cómo diseñar un algoritmo recurrente para su implementación. A continuación se presenta el Algoritmo 4.4.2, como la implementación de la función `mic` que permite calcular ϕ .

Algoritmo 4.4.2

```

1 def mic(A, N, E, n, s, a):
2     ans = None
3     if n==N: ans = 0
4     else:
5         if A[n][0]<=s: ans = mic(A, N, E, n+1, s, max(a, A[n][1]))
6         else: ans = 1 + mic(A, N, E, n+1, a, max(a, A[n][1]))
7     return ans

```

A continuación se presenta un ejemplo con un llamado a la función `mic` para un conjunto de intervalos que, a escala, corresponden al Ejemplo 4.4.1.

Ejemplo 4.4.4

El objetivo es cubrir el intervalo $[0..34]$ con el siguiente arreglo de parejas que especifica los intervalos del Ejemplo 4.4.1:

```

1 A = [ (1, 14), (21, 25), (31, 37),

```

```

2      (9, 19), (23, 30),
3      (1, 6), (7, 13), (18, 22), (29, 36),
4      (-3, 6), (11, 23), (25, 32),
5      (-1, 4), (5, 14), (20, 27) ]

```

Después de ordenar el arreglo ascendentemente por la primera componente de las parejas, se obtiene:

```

1 >>> A.sort()
2 >>> A
3 [ (-3, 6), (-1, 4), (1, 6), (1, 14), (5, 14), (7, 13), (9, 19),
4   (11, 23), (18, 22), (20, 27), (21, 25), (23, 30), (25, 32),
5   (29, 36), (31, 37) ]

```

Con base en la versión ordenada del arreglo, se tiene:

```

1 >>> mic(A, len(A), 34, 0, 0, 0)
2 5

```

El Teorema 4.4.5 presenta los resultados de correctitud de la función `mic`, junto con sus complejidades temporal y espacial.

Teorema 4.4.5

Si los intervalos en $A[0..N)$ están ordenados ascendentemente por el extremo inferior, entonces el llamado `mic(A, N, E, 0, S, S)` calcula $\phi(0, S, S)$, y toma tiempo y espacio $O(N)$ y $O(1)$, respectivamente. Si es necesario ordenar el arreglo, entonces la complejidad temporal es $O(N \log N)$.

Demostración

Note que la función `mic` es una transcripción de la función ϕ , agregando los parámetros A , N y E que nunca son modificados. El tiempo de ejecución de `mic` es linealmente proporcional a la cantidad de elementos en $A[0..N)$, i.e., es $O(N)$. El espacio es $O(1)$ sin tener en cuenta el espacio de la pila de la recurrencia. Finalmente, si es necesario ordenar el arreglo, se puede usar un algoritmo como *MergeSort* (Sección 2.4) con tiempo de ejecución $O(N \log N)$.

Ejercicios

1. Encuentre otro cubrimiento mínimo en el Ejemplo 4.4.1.
2. Ilustre, paso a paso, la ejecución de la función `mic` en el Ejemplo 4.4.4.
3. Para abordar el diseño de una solución algorítmica del problema de cubrimiento de intervalos se hicieron dos suposiciones, indicando que estas no cambiarían la naturaleza misma del problema. Una de ellas, la segunda, indica que se puede suponer que en $A[0..N)$ solo hay intervalos que pueden contribuir a cubrir $[S..E)$. Diseñe un algoritmo que en tiempo $O(N)$ preprocese a $A[0..N)$ e identifique cuáles de sus intervalos se pueden descartar por no cumplir con esta suposición. ¿Afectaría esta solución la complejidad definitiva del algoritmo diseñado para resolver el problema de cubrimiento mínimo de intervalos?
4. Desarrolle una demostración alternativa del Teorema 4.4.3 siguiendo el patrón de *el voraz va adelante*, tal y como se hace en el Teorema 4.2.4. *Ayuda:* sean $I_0, \dots, I_{n-1}$ los intervalos que selecciona el Algoritmo 4.4.1 y $J_0, \dots, J_{m-1}$ los de un cubrimiento cualquiera de $[s..e)$, ambos ordenados ascendentemente por extremo superior. Demuestre por inducción que el extremo superior de I_k no es menor que el de J_k .
5. La Nota 4.4.4 afirma que da lo mismo recurrir con A o con $A \setminus \{[s_n..e_n)\}$ en el paso inductivo del Algoritmo 4.4.1. Demuestre esa afirmación y explique qué pasaría si los intervalos fueran cerrados en ambos extremos.
6. Considere el criterio de escoger, entre los intervalos que incluyen a s , aquel que *cubra la mayor porción todavía no cubierta* del intervalo objetivo.
 - a) Demuestre que este criterio coincide con el de la Sección 4.4 cuando el algoritmo avanza de izquierda a derecha.
 - b) ¿Sigue coincidiendo si se permite escoger intervalos que no incluyan a s ? Justifique con un ejemplo.
7. Suponga que se obtiene la función $\hat{\phi}$ a partir de ϕ modificando el segundo caso inductivo de $1 + \phi(n+1, a, a \uparrow A[n][1])$ a $1 + \phi(n, a, a)$, manteniendo la misma condición. Explique cuál es el sentido de esta reformulación de la recurrencia, y explique si el algoritmo asociado serviría o no para resolver el problema del cubrimiento de intervalos.
8. Diseñe una versión iterativa de la función `mic`.
9. Modifique la función `mic` para que retorne una colección mínima de intervalos para cubrir $[S..E)$.
10. Modifique la función `mic` para que, además de calcular la cantidad mínima de intervalos, identifique cuándo dicha colección no existe, es decir, cuando no se cumple la suposición de que $A[0..N)$ necesariamente cubre $[S..E)$.

11. Suponga que la condición del caso base de la función ϕ se actualiza de $n = N$ a $n = N \vee s \geq E$:
 - a) ¿Qué efecto tiene esto en la correctitud del algoritmo?
 - b) ¿Cambia la complejidad temporal del algoritmo?
 - c) ¿En qué casos sería útil esta nueva versión de la condición del caso base?
 En cada caso, justifique brevemente su respuesta.
12. La estrategia voraz planteada en esta sección aborda el problema con un algoritmo que incrementalmente cubre el intervalo $[S..E)$ desde su extremo inferior. Diseñe un algoritmo cuya estrategia sea abordar el problema incrementalmente cubriendo el intervalo desde su extremo superior. Especialmente, plantee el teorema de optimización local y una función recurrente que resuelva el problema de acuerdo con la estrategia planteada.
13. Diseñe un algoritmo usando la técnica de programación dinámica que resuelva el problema del cubrimiento mínimo de intervalos. Calcule sus complejidades temporal y espacial, y compárelas con las complejidades del algoritmo voraz diseñado en esta sección.

4.5. Tiempo en el sistema

Es común ver personas esperando a ser atendidas en una sucursal bancaria, en una fila, con un turno para realizar una consignación, un retiro o un reclamo, o para abrir/cerrar una cuenta. Los bancos saben cuánto tiempo toma, aproximadamente, cada servicio, en donde este tiempo puede ser distinto entre diferentes tipos de servicio. Un objetivo deseable es organizar los turnos de los clientes para minimizar el tiempo total que tardan todos en esperar y recibir el servicio. Este problema se conoce como el *tiempo en el sistema* y su propósito es identificar una organización óptima de turnos de espera. Tiene una gran cantidad de aplicaciones. Por ejemplo, para acceder a un archivo compartido en un servidor o en la nube, o para agendar procesos en un procesador.

Para plantear el problema, dado su carácter general, a continuación se abstrae el concepto de persona como agente.

Problema 4.5.1: Tiempo en el sistema

Entrada: Un arreglo $A[0..N)$, $N \geq 0$, de números positivos en donde $A[n]$ indica el tiempo que toma servir al agente n , con $0 \leq n < N$.

Salida: Mínimo tiempo alcanzable en el sistema al servir a los agentes con tiempos de servicio en $A[0..N)$.

Observe que si se minimiza el tiempo en que tardan todos los agentes en ser atendidos, directamente se está minimizando el tiempo promedio de atención porque la cantidad de agentes se supone fija.

Una forma de resolver el problema es por medio de una exploración exhaustiva de todas las posibilidades y quedarse con uno de los ordenamientos que tenga mínimo tiempo en el sistema, como se observa en el Ejemplo 4.5.1.

Ejemplo 4.5.1

Considere que hay tres agentes con tiempos $A = [4, 8, 3]$ de tal forma que el agente n es servido en tiempo $A[n]$, con $0 \leq n < 3$. Los turnos se pueden ordenar de 6 formas distintas, con los siguientes tiempos (totales) en el sistema:

$$0, 1, 2 : 4 + (4 + 8) + (4 + 8 + 3) = 31$$

$$0, 2, 1 : 4 + (4 + 3) + (4 + 3 + 8) = 26$$

$$1, 0, 2 : 8 + (8 + 4) + (8 + 4 + 3) = 35$$

$$1, 2, 0 : 8 + (8 + 3) + (8 + 3 + 4) = 34$$

$$2, 0, 1 : 3 + (3 + 4) + (3 + 4 + 8) = 25$$

$$2, 1, 0 : 3 + (3 + 8) + (3 + 8 + 4) = 29.$$

Observe que el tiempo total en el sistema se calcula tomando el tiempo que cada agente toma en ser atendido y el tiempo que espera antes de ser atendido. En el primer ordenamiento, el agente 0 es servido de primero (4 unidades de tiempo), el agente 1 es servido de segundo (4 unidades de tiempo esperando y 8 siendo servido) y el agente 2 es servido de último (4+8 unidades de tiempo esperando y 3 siendo servido). Entre todas las posibilidades, la mejor corresponde al quinto ordenamiento que resulta en 25 unidades de tiempo en el sistema: primero el agente 2, luego el agente 0 y finalmente el agente 1.

El tamaño del espacio de búsqueda de un algoritmo que exhaustivamente genere todos los ordenamientos de N agentes está acotado por las $N!$ permutaciones de $A[0..N]$; esto, sin calcular aún el tiempo en el sistema para cada ordenamiento. Suponiendo que el cálculo del tiempo en el sistema se puede hacer en tiempo $O(N)$ para cada ordenamiento, se tiene que un algoritmo exhaustivo (i.e., de fuerza bruta) toma tiempo $O(N \cdot N!) = O((N+1)!)$ en resolver este problema. La generación sistemática de las permutaciones de un arreglo, que es justamente lo que tal algoritmo necesitaría, se estudia en la Sección 5.2 del Capítulo 5.

Para un problema cuya solución es un ordenamiento, un criterio de selección es sencillamente una manera de ordenar los candidatos, y aquí solo hay dos que

resulten naturales: atender primero a los agentes de menor tiempo de servicio, o atender primero a los de mayor tiempo de servicio. El Ejemplo 4.5.1 refuta el segundo sin necesidad de buscar más lejos. El ordenamiento 1, 2, 0, que atiende de primero al agente más lento, produce 34 unidades de tiempo en el sistema, contra las 25 del ordenamiento 2, 0, 1. La razón es clara: el tiempo de servicio del agente que se atiende de primero lo esperan *todos* los demás, de modo que conviene que sea lo más pequeño posible.

El Ejemplo 4.5.1 sugiere entonces que un mejor ordenamiento para la atención de agentes es uno en el cual se atiendan primero aquellos agentes cuyo tiempo de servicio sea menor. Uno que primero sirva al agente con menor tiempo de servicio, luego al agente con segundo menor tiempo de servicio y así hasta que, finalmente, sirva al agente con mayor tiempo de servicio. Es decir, intuitivamente, un ordenamiento es óptimo cuando los agentes que toman menos tiempo se atienden primero. Dicho de otra forma, uno en el cual los tiempos están ordenados ascendentemente.

Antes de continuar, se define una notación que permite referirse directamente al tiempo del sistema asociado a un arreglo de números.

Nota 4.5.1

Por convención, para cualquier arreglo de números $A[0..N)$, $N \geq 0$, la expresión $\sigma(A)$ se define como:

$$\begin{aligned}\sigma(A) &= N \cdot A[0] + (N - 1) \cdot A[1] + \cdots + 1 \cdot A[N - 1] \\ &= (+n \mid 0 \leq n < N : (N - n) \cdot A[n]).\end{aligned}$$

Note que $\sigma(A)$ representa el tiempo en el sistema asociado a los tiempos de acuerdo con el ordenamiento en $A[0..N)$.

4.5.1. Ordenar en lugar de escoger. Una motivación para abordar este problema es que el esquema que se plantea como estrategia voraz para su solución es distinto al que comparten los problemas de agendamiento de actividades (Sección 4.2) y de cubrimiento de intervalos (Sección 4.4). Conviene precisar en qué consiste esa diferencia antes de continuar.

Leído con el vocabulario de la Sección 4.1, las *soluciones factibles* de este problema son todos los ordenamientos posibles de los N tiempos de servicio, la *función objetivo* de uno de ellos es su tiempo en el sistema —es decir, σ — y el *sentido* es minimizar. Hasta ahí, nada nuevo.

La diferencia está en el conjunto de candidatos. En los dos problemas anteriores, el algoritmo tenía que decidir cuáles candidatos entraban en la solución y cuáles no, y por eso la Definición 4.1.2 describe una solución factible como un subconjunto del conjunto de candidatos. Acá no hay nada que escoger: todos los agentes van a

ser atendidos, todos los tiempos de servicio aparecen en toda solución factible y, en consecuencia, la verificación de factibilidad del Algoritmo 4.1.2 resulta vacía, pues cualquier ordenamiento es factible. Lo único que se decide es en qué orden.

Esto obliga a reformular la propiedad de elección voraz de la Definición 4.1.4, que habla de un candidato que hace parte de alguna solución óptima y que aquí no dice nada, dado que todos los candidatos hacen parte de todas las soluciones. La reformulación adecuada, tal y como se anticipó en la Sección 4.3, se hace en términos de *inversiones* y da lugar al segundo de los tres patrones de demostración del capítulo.

4.5.2. Optimización local: un argumento de inversión. Para continuar, es necesario recordar la noción de inversión de un arreglo (Ejercicio 6, Sección 2.4). Una *inversión* en un arreglo $A[0..N)$, $N \geq 0$, es una pareja de índices $0 \leq i < j < N$ tales que $A[i] > A[j]$. Por ejemplo, en el arreglo $[4, 8, 3]$ del Ejemplo 4.5.1 hay dos inversiones: $(0, 2)$ y $(1, 2)$.

Teorema 4.5.1: Optimización local

Sea $A[0..N)$, $N \geq 0$, un arreglo de números. Si en $A[0..N)$ hay una inversión con testigos $0 \leq i < j < N$, entonces el arreglo que resulta de intercambiar los valores en i y j tiene un tiempo de sistema estrictamente menor que el de $A[0..N)$.

Demostración

Suponga que $A[0..N)$ tiene una inversión con testigos $0 \leq i < j < N$. También suponga que $B[0..N)$ es una copia de $A[0..N)$ que resulta de intercambiar los valores en los índices i y j . Formalmente, $B[i] = A[j]$, $B[j] = A[i]$ y $B[k] = A[k]$ para cualquier $0 \leq k < N$ con $k \neq i$ y $k \neq j$. El objetivo es demostrar $\sigma(A) > \sigma(B)$. Basta con demostrar $\sigma(A) - \sigma(B) > 0$. Por la Nota 4.5.1, tanto $\sigma(A)$ como $\sigma(B)$ son sumas de los términos $(N - k) \cdot A[k]$ y $(N - k) \cdot B[k]$, respectivamente, para $0 \leq k < N$. Como $A[k] = B[k]$ para todo k distinto de i y de j , todos esos términos se cancelan en la resta y solo

sobreviven los correspondientes a i y a j :

$$\begin{aligned}
 \sigma(A) - \sigma(B) &= (N - i) A[i] + (N - j) A[j] \\
 &\quad - (N - i) B[i] - (N - j) B[j] \\
 &= (N - i) A[i] + (N - j) A[j] \\
 &\quad - (N - i) A[j] - (N - j) A[i] \\
 &= [(N - i) - (N - j)] (A[i] - A[j]) \\
 &= (j - i) (A[i] - A[j]) \\
 &> 0.
 \end{aligned}$$

El último paso es válido porque $j > i$ y, por suposición, (i, j) es una inversión (i.e., $A[i] > A[j]$). En conclusión, $\sigma(B) < \sigma(A)$.

Nota 4.5.2: El argumento de inversión

La demostración del Teorema 4.5.1 es el ejemplar del segundo patrón del capítulo. Comparada con el argumento de intercambio de la Nota 4.2.3, la diferencia es dónde ocurre el intercambio: allá se cambiaba un elemento de una solución óptima por un candidato que estaba *fuera* de ella; acá se intercambian dos elementos que ya están *dentro* de la misma solución.

La consecuencia es que la conclusión también cambia de forma. Un argumento de intercambio concluye que existe una solución óptima que contiene la elección voraz; un argumento de inversión concluye que ninguna solución óptima puede tener inversiones y que, por lo tanto, toda solución óptima está ordenada según el criterio. Esa es la reformulación de la propiedad de elección voraz que exige un problema cuya solución es un ordenamiento.

4.5.3. Optimización global. La pregunta de qué ordenamiento es el mejor posible se puede responder, gracias al Teorema 4.5.1, de forma directa: aquel en donde no haya inversiones. Esto es porque si existe una inversión en un ordenamiento, entonces el tiempo en el sistema asociado disminuye necesariamente al intercambiar los elementos en dicha inversión. Como la cantidad de elementos en $A[0..N)$ es finita, necesariamente el proceso de disminuir la cantidad de inversiones allí presentes acabará eventualmente con un ordenamiento ascendente (más precisamente, no decreciente) de sus valores; note que intercambiar los valores de una inversión reduce estrictamente la cantidad de inversiones del arreglo (¿por qué?).

Teorema 4.5.2: Optimización global

Un ordenamiento de los tiempos de servicio minimiza el tiempo en el sistema si, y solo si, es no decreciente.

Demostración

Sea $A[0..N]$, $N \geq 0$, un arreglo de números que registra un ordenamiento de los tiempos de servicio. Se demuestran las dos implicaciones.

- Suponga que $A[0..N]$ minimiza el tiempo en el sistema y, hacia una contradicción, que no está ordenado no decrecientemente. Entonces, hay una inversión (i, j) con $0 \leq i < j < N$ y, por el Teorema 4.5.1, el arreglo que resulta de intercambiar los valores en i y j tiene un tiempo en el sistema estrictamente menor. Esto contradice la minimalidad de $A[0..N]$. Luego, $A[0..N]$ está ordenado no decrecientemente.
- Recíprocamente, suponga que $A[0..N]$ está ordenado no decrecientemente. Dos ordenamientos no decrecientes de los mismos tiempos de servicio coinciden posición a posición (*¿por qué?*) y, por ende, tienen el mismo tiempo en el sistema. Por la implicación anterior, algún ordenamiento que minimiza el tiempo en el sistema es no decreciente; como todos los ordenamientos no decrecientes tienen el mismo tiempo en el sistema, $A[0..N]$ también lo minimiza.

Nota 4.5.3

El Teorema 4.5.2 no afirma que el ordenamiento óptimo sea único: cuando hay tiempos de servicio repetidos, hay varios ordenamientos *de los agentes* que producen el mismo arreglo de tiempos y, por lo tanto, el mismo tiempo en el sistema. Lo que sí es único es el arreglo de tiempos resultante. El Ejercicio 1 de esta sección ilustra la situación.

4.5.4. Una implementación eficiente. El criterio de selección de este problema es *estático* en el sentido de la Nota 4.1.3: el tiempo de servicio de un agente no cambia con lo que se haya decidido antes. Aplica entonces la receta de *ordenar y recorrer*. De hecho, aquí el ordenamiento no es una preparación para el algoritmo voraz sino que *es* el algoritmo voraz. Lo único que queda por hacer es evaluar la función objetivo sobre el arreglo ya ordenado.

Entendiendo que un ordenamiento no decreciente de $A[0..N]$ minimiza el tiempo en el sistema asociado, solo queda pendiente diseñar un algoritmo que calcule eficientemente este tiempo (para cualquier ordenamiento). Considere la siguiente

función objetivo, con $0 \leq n \leq N$:

$\phi(n)$: “tiempo en el sistema asociado al ordenamiento en $A[0..n)$.”

Se entiende que el ordenamiento en $A[0..n)$ corresponde a los primeros n agentes en ser atendidos. La definición recurrente de ϕ , para $0 \leq n \leq N$, consta de un caso base y uno inductivo, como se propone a continuación:

$$\phi(n) = \begin{cases} 0 & , \text{ si } n = 0, \\ \phi(n-1) + (N - (n-1)) \cdot A[n-1] & , \text{ si } n \neq 0. \end{cases}$$

El caso base corresponde a la situación en la cual no hay agentes por servir y el tiempo en el sistema es 0. El caso inductivo, el tiempo de servicio es el tiempo de servicio correspondiente a todos los agentes que se han atendido previamente en adición al tiempo que toma servir al n -ésimo agente, acumulando este último tiempo para los agentes que hace falta atender (i.e., $N - n$).

Observe que $\phi(N)$ debe determinar el tiempo en el sistema para los N agentes de acuerdo con el ordenamiento establecido en $A[0..N)$. Entonces, debe ser cierto que $\phi(N) = \sigma(A)$, con $A[0..N)$.

Teorema 4.5.3

$$\phi(N) = \sigma(A).$$

Demostración

Note que N está fijo, tanto en la definición de σ como en la de ϕ , de modo que la inducción no puede hacerse sobre el tamaño del arreglo. Se demuestra, por inducción sobre n , que para todo $0 \leq n \leq N$:

$$\phi(n) = (+k \mid 0 \leq k < n : (N - k) \cdot A[k]).$$

Si $n = 0$, ambos lados valen 0: el izquierdo por definición de ϕ y el derecho por ser una suma vacía. Si $n \neq 0$, por definición de ϕ y por la Hipótesis Inductiva:

$$\begin{aligned} \phi(n) &= \phi(n-1) + (N - (n-1)) \cdot A[n-1] \\ &= (+k \mid 0 \leq k < n-1 : (N - k) \cdot A[k]) \\ &\quad + (N - (n-1)) \cdot A[n-1] \\ &= (+k \mid 0 \leq k < n : (N - k) \cdot A[k]). \end{aligned}$$

Instanciando la igualdad anterior con $n = N$ y comparándola con la Nota 4.5.1, se concluye $\phi(N) = \sigma(A)$.

Lo que resta es definir una función recurrente que implemente la función ϕ , como lo propone el Algoritmo 4.5.1.

Algoritmo 4.5.1

```
1 def sch(A, N, n):
2     ans = None
3     if n==0: ans = 0
4     else: ans = (N-(n-1))*A[n-1] + sch(A, N, n-1)
5     return ans
```

A continuación se presenta un ejemplo en el cual se usa la función `sch` con los tiempos en el Ejemplo 4.5.1.

Ejemplo 4.5.2

El objetivo es determinar el mejor tiempo de servicio (i.e., el mínimo) con los tiempos registrados en $A = [4, 8, 3]$ (Ejemplo 4.5.1).

```
1 A = [ 4, 8, 3 ]
```

Después de ordenar el arreglo ascendentemente, se obtiene:

```
1 >>> A.sort()
2 >>> A
3 [3, 4, 8]
```

Luego, usando la función `sch` se obtiene:

```
1 >>> sch(A, len(A), len(A))
2 25
```

El Teorema 4.5.4 presenta la correctitud de la función `sch`, junto con sus complejidades temporal y espacial.

Teorema 4.5.4

Si los valores en $A[0..N)$ están ordenados no decrecientemente, entonces el llamado `sch(A, N, N)` calcula $\phi(N)$, y toma tiempo y espacio $O(N)$ y $O(1)$, respectivamente. Si es necesario ordenar el arreglo, entonces la complejidad temporal es $O(N \log N)$.

Demostración

Se propone como ejercicio al lector.

El resultado coincide con lo que anticipa el Teorema 4.1.2 para un criterio estático. Note, sin embargo, que acá el ordenamiento no es un costo accesorio que se paga para poder aplicar la estrategia voraz, como ocurría en las dos secciones anteriores: es la estrategia voraz misma. Por eso el $O(N \log N)$ es, en este problema, insuperable con este enfoque.

Ejercicios

1. Suponga que hay 4 agentes en el sistema con tiempos 4, 1, 8 y 8, respectivamente. Enumere los tiempos en el sistema asociados a todos los ordenamientos de estos tiempos. ¿Cuál es el mejor?
2. Demuestre el Teorema 4.5.4.
3. Proponga una versión iterativa para `sch`.
4. Diseñe una función objetivo ψ , como alternativa a ϕ , de acuerdo con la siguiente especificación, con $0 \leq n \leq N$,

$\psi(n)$: “tiempo en el sistema asociado al ordenamiento en $A[n..N]$.”

con el objetivo de calcular $\psi(0)$ de tal manera que $\phi(N) = \psi(0)$. Justifique brevemente su diseño.

5. Suponga que, al contrario de lo que se busca en el problema del tiempo en el sistema, se desea calcular el máximo tiempo posible.
 - a) Proponga una estrategia voraz.
 - b) Enuncie y demuestre un teorema de optimización local.
 - c) Enuncie y demuestre un teorema de optimización global.
 - d) Proponga y defina recurrentemente una función objetivo ϕ .
 - e) Diseñe una función `schdual` en el lenguaje de programación Python que calcule ϕ .
 - f) Enuncie un teorema de correctitud para `schdual`, y establezca las complejidades temporal y espacial de la función.
 - g) Demuestre el teorema de correctitud de `schdual`.
6. La Nota 4.5.2 sostiene que el argumento de inversión es una variante del argumento de intercambio.
 - a) Explique por qué la Definición 4.1.4 no dice nada útil sobre este problema.
 - b) Proponga una manera de acomodar los problemas de ordenamiento dentro del marco de la Sección 4.1, de modo que el Teorema 4.1.1 sea aplicable.

Ayuda: tome como candidato la decisión de *cuál agente se atiende a continuación*.

7. 🌈 Suponga ahora que cada agente n tiene además un *peso* $W[n] > 0$, que mide su importancia, y que el objetivo es minimizar la suma de los productos entre el peso de cada agente y el instante en que termina de ser atendido.
 - a) Demuestre, con dos agentes, que atender primero al de menor tiempo de servicio deja de ser óptimo.
 - b) Proponga un criterio de selección para esta variante. *Ayuda:* el tiempo de servicio y el peso deben combinarse en una sola cantidad.
 - c) Enuncie y demuestre el teorema de optimización local correspondiente, siguiendo el patrón de inversión de esta sección.
 - d) Verifique que el problema original es el caso particular en el cual todos los pesos valen 1.

4.6. El morral fraccionario

El problema del morral, presentado en la Sección 3.6, pide escoger elementos de una colección –cada uno con un peso y un valor– de manera que el valor total sea máximo sin que el peso total exceda una capacidad dada. Es un problema de optimización, tiene subestructura óptima y su enunciado se parece bastante a los de este capítulo. Es razonable preguntarse si admite una solución voraz.

La respuesta es negativa y conviene ver por qué antes de continuar. El criterio de selección más plausible para este problema es la *densidad* de un elemento, es decir, el valor que aporta por unidad de peso; llevar primero los elementos de mayor densidad parece la manera natural de aprovechar una capacidad escasa.

Ejemplo 4.6.1: El morral resiste

Considere dos elementos con valores y pesos dados por, respectivamente:

$$V = [6, 10] \quad \text{y} \quad W = [3, 7],$$

y una capacidad $X = 7$. Las densidades son $6/3 = 2$ y $10/7 \approx 1,43$, de modo que el criterio prefiere el elemento 0. Al llevarlo quedan 4 unidades de capacidad, en las cuales el elemento 1 no cabe. El valor total es 6.

Sin embargo, llevar únicamente el elemento 1 produce un valor de 10.

Nota 4.6.1: Qué es lo que falla

El Ejemplo 4.6.1 no refuta la idea de que la densidad sea la medida correcta; lo que refuta es que llevar primero el elemento más denso deje siempre abierta la puerta a una solución óptima. Y la razón por la cual la cierra es la *indivisibilidad*: al llevar el elemento 0 quedan 4 unidades de capacidad que ya no sirven para nada, porque no hay manera de llevar cuatro séptimas partes del elemento 1.

Esa observación sugiere un experimento. ¿Qué pasa si se permite partir los elementos? El problema que resulta ya no es el mismo –nadie ha pedido resolverlo– pero es más fácil, y lo que se aprenda de él no será tiempo perdido: la Sección 5.7 lo utilizará para resolver el problema original.

Problema 4.6.1: El morral fraccionario

Entrada: Arreglos $V[0..N]$ y $W[0..N]$, con $N \geq 0$, de valores y pesos (números positivos), y un número $X \geq 0$.

Salida: Máximo valor de $(+n \mid 0 \leq n < N : x_n \cdot V[n])$ entre las asignaciones $x_0, \dots, x_{N-1}$ con $x_n \in [0..1]$ que satisfacen $(+n \mid 0 \leq n < N : x_n \cdot W[n]) \leq X$.

El número x_n indica qué fracción del elemento n se lleva: $x_n = 1$ significa llevarlo entero, $x_n = 0$ no llevarlo y $x_n = 1/3$ llevar una tercera parte, que pesa $W[n]/3$ y aporta $V[n]/3$. El Problema 4.6.1 con la restricción adicional $x_n \in \{0, 1\}$ es exactamente el problema del morral de la Sección 3.6; por eso se dice que es una *relajación* suya.

Leído con el vocabulario de la Sección 4.1, las *soluciones factibles* son las asignaciones que respetan la capacidad, la *función objetivo* de una de ellas es el valor total que produce y el *sentido* es maximizar. El conjunto de candidatos son los N elementos.

Nota 4.6.2: Asignar en lugar de escoger

Este problema tiene, respecto del marco de la Sección 4.1, una particularidad parecida a la de la Sección 4.5 aunque de otra naturaleza. Allá una solución factible era un ordenamiento de los candidatos; acá es una *asignación* que le da a cada candidato un número en $[0..1]$ y no un subconjunto como pide la Definición 4.1.2.

La adaptación es directa y no vale la pena formalizarla en el cuerpo del texto: donde las definiciones dicen que una solución *contiene* un candidato,

aquí hay que leer que le *asigna un valor determinado*. Con esa lectura, la propiedad de elección voraz de la Definición 4.1.4 afirma que existe una solución óptima que le asigna al candidato preferido por el criterio exactamente lo que le asigna el algoritmo voraz y la subestructura óptima dice lo de siempre. Los subconjuntos son el caso particular en el cual las asignaciones solo pueden valer 0 o 1.

4.6.1. Escoger un criterio. Hay tres criterios de selección que parecen naturales: llevar primero el elemento de mayor *valor*, el de menor *peso* o el de mayor *densidad* $V[n]/W[n]$. Una sola instancia con tres elementos basta para descartar los dos primeros.

Ejemplo 4.6.2: Dos criterios que no funcionan

Considere tres elementos con valores y pesos dados por, respectivamente:

$$V = [1, 10, 20] \quad \text{y} \quad W = [1, 5, 100],$$

y una capacidad $X = 5$. Las densidades son 1, 2 y 0,2.

El criterio de *mayor valor* prefiere el elemento 2, del cual cabe una vigésima parte; el valor total es 1.

El criterio de *menor peso* prefiere el elemento 0, que se lleva entero y aporta 1; con las 4 unidades de capacidad restantes se llevan cuatro quintas partes del elemento 1, que aportan 8. El valor total es 9.

El criterio de *mayor densidad* prefiere el elemento 1, que se lleva entero, agota la capacidad y aporta 10. Ninguna otra asignación produce más (¿por qué?).

Los dos criterios refutados fallan por la misma razón y es instructiva: ambos miran una sola de las dos cantidades que definen a un elemento. El valor sin el peso no dice cuánta capacidad cuesta obtenerlo; el peso sin el valor no dice qué se obtiene a cambio. La densidad es la única de las tres que las combina y es la que sobrevive.

De aquí en adelante se supone que los elementos están ordenados descendientemente por densidad, es decir, que:

$$\frac{V[0]}{W[0]} \geq \frac{V[1]}{W[1]} \geq \dots \geq \frac{V[N-1]}{W[N-1]}.$$

Como los pesos son positivos, la densidad está bien definida y este ordenamiento siempre existe. Note que el criterio es *estático* en el sentido de la Nota 4.1.3: la densidad de un elemento no depende de lo que se haya llevado antes. Aplica entonces la receta de *ordenar y recorrer*.

Algoritmo 4.6.1

Para elementos ordenados descendientemente por densidad y una capacidad X : llevar los elementos enteros, en ese orden, mientras quepan; del primero que no quepa, llevar la fracción que alcance; no llevar nada de los restantes.

4.6.2. Optimización local. Lo que el Algoritmo 4.6.1 le asigna al primer elemento es la mayor fracción de él que cabe en el morral, que es 1 si el elemento entero cabe y $X/W[0]$ si no.

Teorema 4.6.1: Optimización local

Suponga $N \geq 1$, que los elementos están ordenados descendientemente por densidad y que $g = 1 \downarrow (X/W[0])$. Entonces, hay una solución óptima $y_0, \dots, y_{N-1}$ del Problema 4.6.1 con $y_0 = g$.

Demostración

Escríbase $d_n = V[n]/W[n]$ para la densidad del elemento n , de modo que $d_0 \geq d_n$ para todo n , y sea $y_0, \dots, y_{N-1}$ una solución óptima cualquiera. Note primero que $y_0 \leq g$: por un lado $y_0 \leq 1$ por ser una solución factible y por otro $y_0 \cdot W[0] \leq X$ por la restricción de capacidad, es decir, $y_0 \leq X/W[0]$.

Si $y_0 = g$, no hay nada que hacer. Suponga entonces $y_0 < g$ y considere $\delta = (g - y_0) \cdot W[0] > 0$, que es el peso que hace falta liberar para poder subir y_0 hasta g .

El primer paso es verificar que ese peso está disponible entre los demás elementos, i.e., que:

$$(+n \mid 1 \leq n < N : y_n \cdot W[n]) \geq \delta.$$

En efecto, $g \cdot W[0] \leq X$ (¿por qué?), y por lo tanto:

$$\delta = g \cdot W[0] - y_0 \cdot W[0] \leq X - y_0 \cdot W[0].$$

Ahora bien, si la solución y dejara capacidad sin usar podría aumentarse y_0 —lo cual es posible porque $y_0 < g \leq 1$ — y su valor crecería, contradiciendo que es óptima. Luego la capacidad está agotada, el peso dedicado a los elementos $1, \dots, N-1$ es exactamente $X - y_0 \cdot W[0]$ y este no es menor que δ .

El segundo paso es el intercambio. Constrúyase la asignación z tomando $z_0 = g$ y reduciendo los demás y_n , en cualquier orden, hasta haber liberado un peso total de exactamente δ ; el paso anterior garantiza que esto es posible.

Sea $\Delta_n \geq 0$ la reducción aplicada a y_n , de manera que:

$$(+n \mid 1 \leq n < N : \Delta_n \cdot W[n]) = \delta.$$

La asignación z es factible: cada z_n sigue en $[0..1]$ y el peso total no cambió, pues se agregaron δ unidades al elemento 0 y se liberaron δ de los demás.

Falta comparar los valores. Lo que se gana es $(g - y_0) \cdot V[0] = \delta \cdot d_0$ y lo que se pierde es:

$$\begin{aligned} (+n \mid 1 \leq n < N : \Delta_n \cdot V[n]) &= (+n \mid 1 \leq n < N : \Delta_n W[n] \cdot d_n) \\ &\leq (+n \mid 1 \leq n < N : \Delta_n W[n] \cdot d_0) \\ &= \delta \cdot d_0. \end{aligned}$$

Es decir, el valor de z no es menor que el de y . Como y es óptima, z también lo es y $z_0 = g$.

Nota 4.6.3: Densidad y capacidad

El Teorema 4.6.1 es otro argumento de intercambio de los descritos en la Nota 4.2.3, y su propiedad óptima es que ninguna otra densidad supera a d_0 : por eso el peso que se le quita a cualquier otro elemento vale, medido en valor, a lo sumo lo que ese mismo peso vale puesto en el elemento 0.

Compare esta demostración con el Ejemplo 4.6.1, en el cual la misma idea fracasaba. La diferencia está en el paso en el cual se construye z reduciendo los y_n hasta liberar *exactamente* δ . Eso es lo que la divisibilidad permite y la indivisibilidad prohíbe: en el problema original solo se pueden liberar elementos completos y ningún peso intermedio.

4.6.3. Optimización global. Como en las secciones anteriores, una vez demostrada la propiedad de elección voraz solo falta verificar la subestructura óptima.

Teorema 4.6.2: Subestructura óptima

Suponga $N \geq 1$ y sea $g = 1 \downarrow (X/W[0])$. Suponga además que hay una solución óptima del Problema 4.6.1 que le asigna g al elemento 0. Si $z_1, \dots, z_{N-1}$ es una solución óptima del problema sobre los elementos $1, \dots, N-1$ con capacidad $X - g \cdot W[0]$, entonces $g, z_1, \dots, z_{N-1}$ es una solución óptima del problema original.

Demostración

La asignación $g, z_1, \dots, z_{N-1}$ es factible porque el peso que dedica a los elementos $1, \dots, N-1$ no excede $X - g \cdot W[0]$.

Sea y una solución óptima del problema original con $y_0 = g$, que existe por suposición. Entonces $y_1, \dots, y_{N-1}$ es una solución factible del problema sobre los elementos $1, \dots, N-1$ con capacidad $X - g \cdot W[0]$ y, como z es óptima allí, el valor de $y_1, \dots, y_{N-1}$ no supera el de $z_1, \dots, z_{N-1}$. Sumando $g \cdot V[0]$ a ambos lados se concluye que el valor de y no supera el de $g, z_1, \dots, z_{N-1}$. Como y es óptima, los dos valores coinciden.

Teorema 4.6.3: Optimización global

Si los elementos están ordenados descendientemente por densidad, entonces el Algoritmo 4.6.1 resuelve el Problema 4.6.1.

Demostración

Se procede por inducción sobre N .

Si $N = 0$ no hay nada que llevar y la única asignación, que es la vacía, es óptima.

Si $N \geq 1$, el Teorema 4.6.1 garantiza que hay una solución óptima que le asigna al elemento 0 exactamente $g = 1 \downarrow (X/W[0])$, que es lo que le asigna el algoritmo. Lo que resta por resolver es el mismo problema sobre los elementos $1, \dots, N-1$ con capacidad $X - g \cdot W[0]$, cuyos elementos siguen ordenados descendientemente por densidad; por la Hipótesis Inductiva, el algoritmo lo resuelve de manera óptima. Por el Teorema 4.6.2, lo que produce en total es una solución óptima del problema original.

Nota 4.6.4: La misma inducción

La demostración anterior es, paso por paso, la del Teorema 4.1.1: optimización local para comprometerse con la primera decisión, hipótesis inductiva para el resto y subestructura óptima para pegar las dos partes. No se invocó aquel teorema directamente únicamente porque su enunciado habla de subconjuntos de candidatos y aquí las soluciones son asignaciones, según se advirtió en la Nota 4.6.2. El Ejercicio 7 propone enunciarlo en la generalidad necesaria.

4.6.4. Una implementación eficiente. Considere la siguiente función objetivo, para $0 \leq n \leq N$ y $x \geq 0$:

$\phi(n, x)$: “máximo valor que se puede obtener con los elementos en $V[n..N]$ y $W[n..N]$, permitiendo fracciones, con una capacidad de x .”

Para resolver el problema del morral fraccionario, el objetivo es calcular $\phi(0, X)$ sobre los elementos ya ordenados por densidad.

La definición recurrente de ϕ se establece tomando a n como pivote y preguntando si el elemento n cabe entero en la capacidad disponible:

$$\phi(n, x) = \begin{cases} 0 & , \text{ si } n = N, \\ V[n] + \phi(n+1, x - W[n]) & , \text{ si } n \neq N \wedge W[n] \leq x, \\ x \cdot V[n]/W[n] & , \text{ si } n \neq N \wedge W[n] > x. \end{cases}$$

El caso base corresponde a la situación en la cual no quedan elementos por considerar. El primer caso inductivo corresponde a la situación en la cual el elemento n cabe entero: se lleva completo y se recurre con la capacidad reducida. El segundo caso inductivo corresponde a la situación en la cual el elemento n no cabe: se lleva la fracción $x/W[n]$, que aporta $x \cdot V[n]/W[n]$, con lo cual la capacidad queda agotada y no hace falta recurrir.

Nota 4.6.5: Dos maneras de terminar

Vale la pena notar que la recurrencia tiene dos maneras de terminar y que corresponden a las dos maneras en las que el Algoritmo 4.6.1 se detiene: porque se acabaron los elementos o porque se acabó la capacidad. La segunda no aparece como un caso base sino escondida en el tercer caso, que no recurre. Esta es también la razón por la cual el algoritmo lleva a lo sumo un elemento partido (¿por qué?).

Algoritmo 4.6.2

```

1 def frac(V, W, N, X, n):
2     # items sorted in decreasing order of density V[i]/W[i]
3     ans = None
4     if n==N: ans = 0
5     else:
6         if W[n]<=X: ans = V[n] + frac(V, W, N, X-W[n], n+1)
```

```

7     else: ans = X*(V[n]/W[n])
8     return ans

```

Ejemplo 4.6.3

Considere los elementos del Ejemplo 4.6.2:

```

1 V = [1, 10, 20]
2 W = [1, 5, 100]

```

Después de ordenarlos descendientemente por densidad, se obtiene:

```

1 >>> P = sorted(zip(V, W), key = lambda p: -p[0]/p[1])
2 >>> V = [p[0] for p in P]
3 >>> W = [p[1] for p in P]
4 >>> V, W
5 ([10, 1, 20], [5, 1, 100])

```

Con base en la versión ordenada y con una capacidad de 5, se lleva únicamente el primer elemento:

```

1 >>> frac(V, W, len(V), 5, 0)
2 10.0

```

Con una capacidad de 7, en cambio, se llevan el primero y el segundo enteros, y una centésima parte del tercero:

```

1 >>> frac(V, W, len(V), 7, 0)
2 11.2

```

Teorema 4.6.4

Si los elementos están ordenados descendientemente por densidad, entonces el llamado `frac(V, W, N, X, 0)` calcula $\phi(0, X)$, y toma tiempo y espacio (sin considerar la pila de ejecución para la recurrencia) $O(N)$ y $O(1)$, respectivamente. Si es necesario ordenar los elementos, entonces la complejidad temporal es $O(N \log N)$.

Demostración

Se propone como ejercicio al lector.

4.6.5. Una cota para el problema original. El morral fraccionario no fue el problema que se quiso resolver, sino una relajación del que sí interesaba. Lo que se gana con la relajación es lo siguiente.

Teorema 4.6.5

Para cualesquiera arreglos $V[0..N)$ y $W[0..N)$ de números positivos y cualquier capacidad $X \geq 0$, el valor óptimo del Problema 4.6.1 no es menor que el valor óptimo del problema del morral de la Sección 3.6 sobre los mismos datos.

Demostración

Sea $S \subseteq \{0, \dots, N-1\}$ una solución óptima del problema del morral. La asignación que toma $x_n = 1$ para $n \in S$ y $x_n = 0$ para $n \notin S$ es una solución factible del Problema 4.6.1 –tiene el mismo peso total, que no excede X – y produce el mismo valor. Luego el valor óptimo del problema relajado no puede ser menor.

Es decir, en tiempo $O(N \log N)$ se obtiene un número que acota por encima la respuesta de un problema que el Capítulo 3 tuvo que resolver por tabulación. Contar con una cota superior que se calcule rápido es exactamente lo que necesita la técnica de ramificación y poda, y en eso la empleará la Sección 5.7.

Ejercicios

1. Demuestre el Teorema 4.6.4.
2. Justifique las siguientes afirmaciones hechas en esta sección:
 - a) En el Ejemplo 4.6.2, ninguna asignación produce un valor mayor que 10.
 - b) El Algoritmo 4.6.1 lleva a lo sumo un elemento partido.
 - c) En la demostración del Teorema 4.6.1 se cumple $g \cdot W[0] \leq X$, sea cual sea de los dos valores el que alcanza el mínimo.
3. En el enunciado del Problema 4.6.1 se exige que los pesos sean positivos.
 - a) ¿Qué parte de esta sección deja de funcionar si se permite que algún peso sea 0?
 - b) Proponga una manera de tratar esos elementos y justifique que la solución resultante sigue siendo óptima.
4. Modifique la función `frac` para que, además del valor óptimo, retorne las fracciones $x_0, \dots, x_{N-1}$ que lo alcanzan.

5. Diseñe una versión iterativa de la función `frac`, en tiempo $O(N)$ y espacio $O(1)$ sobre elementos ya ordenados.
6. Suponga que el criterio de la densidad produce empates, es decir, que hay elementos $i \neq j$ con $V[i]/W[i] = V[j]/W[j]$.
 - a) ¿Afecta esto la corrección del Algoritmo 4.6.1?
 - b) ¿En qué paso de la demostración del Teorema 4.6.1 se usa que la desigualdad entre densidades no es estricta?
7. La Nota 4.6.2 sostiene que las definiciones de la Sección 4.1 se adaptan a este problema reemplazando “la solución contiene al candidato” por “la solución le asigna al candidato un valor determinado”.
 - a) Escriba las versiones generalizadas de la Definición 4.1.2, la Definición 4.1.4 y la Definición 4.1.5.
 - b) Enuncie y demuestre la versión correspondiente del Teorema 4.1.1.
 - c) Verifique que las definiciones originales son el caso particular en el cual las asignaciones solo toman los valores 0 y 1.
8. Considere la variante del morral fraccionario en la cual de cada elemento hay una cantidad ilimitada de copias.
 - a) Especifique el problema.
 - b) Proponga un criterio de selección, y enuncie y demuestre el teorema de optimización local correspondiente.
 - c) Diseñe un algoritmo que lo resuelva en tiempo $O(N)$, sin necesidad de ordenar.
9. 🌈 El Teorema 4.6.5 da una cota superior para el problema del morral, y se quiere saber qué tan buena es. Sean B el valor óptimo del morral fraccionario, O el del morral y V^* el mayor de los valores en $V[0..N)$.
 - a) Demuestre que $B < O + V^*$.
 - b) Concluya que llevar, en orden de densidad, los elementos enteros que quepan produce una solución del problema del morral cuyo valor es mayor que $O - V^*$.
 - c) Construya una familia de instancias en las cuales la diferencia entre B y O se acerque tanto como se quiera a V^* .

4.7. Árboles de cubrimiento mínimo

Identificar un árbol de cubrimiento mínimo es un problema de optimización sobre grafos. Dado un grafo no dirigido, conexo y con peso en sus arcos, el problema consiste en encontrar un subgrafo que cumpla con dos propiedades específicas: una estructural, de ser conexo y sin ciclos, y otra de optimalidad, en cuanto que tenga la menor suma posible de los pesos de sus arcos. Este problema, conocido en

inglés como *minimum spanning tree* (MST), ha sido ampliamente estudiado y se conocen soluciones algorítmicas *voraces* eficientes, además relativamente fáciles de implementar.

Antes de especificar el problema algorítmico es indispensable introducir formalmente algunos conceptos y términos útiles para su definición. En particular, se precisan las nociones de árbol de cubrimiento y de minimalidad en este contexto a partir de nociones de grafos que comúnmente se conocen.

Definición 4.7.1

Un *árbol* es un grafo no dirigido que es conexo y acíclico. Dado un grafo no dirigido $G = (V, E)$, se dice que $T = (V_T, E_T)$ es un *árbol de cubrimiento* de G si:

$$T \text{ es un árbol,} \quad V_T = V \quad \text{y} \quad E_T \subseteq E.$$

Dada además una función de peso $w : E \rightarrow \mathbb{R}$, se escribe $w(T)$ para la suma de los pesos de los arcos en E_T . Se dice que T es un *árbol de cubrimiento mínimo* de G (con respecto a w) si ningún otro árbol de cubrimiento T' de G satisface $w(T') < w(T)$.

Un árbol de cubrimiento mínimo de un grafo es un subgrafo que conecta todos los vértices, sin ciclos y lo menos pesado posible. Un escenario común en el cual se requieren árboles de cubrimiento mínimo es en el diseño de redes de comunicaciones. Considere diferentes ubicaciones que deben ser conectadas y que el costo de conectar dos ubicaciones es proporcional a su distancia. El objetivo es conectar todas las ubicaciones usando el presupuesto más modesto posible. La solución deberá ser un árbol de cubrimiento mínimo.

Problema 4.7.1: Árbol de cubrimiento mínimo

Entrada: Un grafo $G = (V, E)$ conexo con función de peso $w : E \rightarrow \mathbb{R}$.

Salida: Un árbol $T = (V, E_T)$ de cubrimiento mínimo de G (con respecto a w).

Nota 4.7.1: Sobre el signo de los pesos

Ni la Definición 4.7.1 ni el Problema 4.7.1 imponen restricción alguna sobre el signo de los pesos y es deliberado: *ningún* resultado de esta sección la

necesita. El teorema del corte y los dos algoritmos que se presentan más adelante funcionan igual con pesos negativos.

En las aplicaciones habituales –distancias, costos, capacidades– los pesos son positivos y hay enunciados que sí dependen de ello. El Ejercicio 1 es uno y por eso agrega la hipótesis explícitamente en lugar de heredarla del problema.

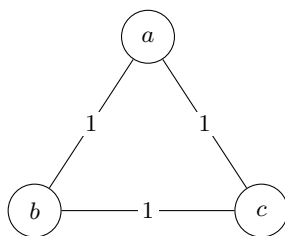
Vale la pena retener la observación porque el contraste con la Sección 4.8 es marcado. Allá la hipótesis sobre el signo de los pesos no es un adorno de la especificación sino el corazón de la demostración y el algoritmo deja de funcionar sin ella.

Leído con el vocabulario de la Sección 4.1, los tres elementos de la Definición 4.1.1 están en el enunciado del Problema 4.7.1: las *soluciones factibles* son los árboles de cubrimiento de G , la *función objetivo* de uno de ellos es la suma de los pesos de sus arcos y el *sentido* es minimizar. El conjunto de candidatos son los arcos en E y una solución factible es –identificando T con E_T – un subconjunto de ellos.

Una aproximación de fuerza bruta para resolver el problema consiste en enumerar todos los subconjuntos de E y determinar, entre aquellos que representen un árbol de cubrimiento de G , uno de suma mínima. Como hay un total de $\Theta(2^{|E|})$ subconjuntos de arcos de E , necesariamente este enfoque no es viable en la práctica. Aún observando que un árbol con vértices V tiene $|V| - 1$ arcos (Ejercicio 2), la enumeración de los subconjuntos de E de tamaño $|V| - 1$ sigue siendo exponencial en tamaño. Es decir, los enfoques de fuerza bruta no son una opción si se desea considerar grafos con, al menos, centenares de arcos.

Ejemplo 4.7.1

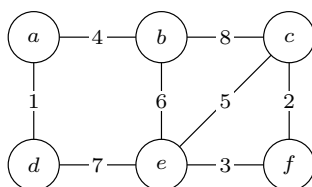
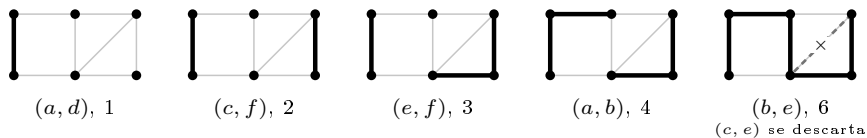
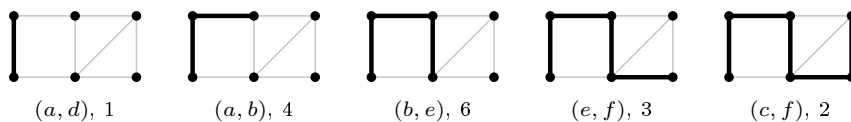
El MST de un grafo dado no es necesariamente único. Por ejemplo, considere el grafo de tres vértices que se dibuja a continuación:



Este grafo tiene tres árboles de cubrimiento mínimo, dados por los siguientes conjuntos de arcos, cada uno con suma de pesos 2: $\{(a, b), (b, c)\}$, $\{(a, b), (a, c)\}$ y $\{(a, c), (b, c)\}$.

Ejemplo 4.7.2: Kruskal y Prim sobre el mismo grafo

Los dos algoritmos voraces de esta sección construyen el mismo árbol de cubrimiento mínimo del grafo que se dibuja arriba, pero lo hacen en órdenes distintos. En cada panel, los vértices ocupan las mismas posiciones que en el grafo de referencia y los arcos ya seleccionados se resaltan.

**Kruskal****Prim**

El algoritmo de Kruskal escoge los arcos en orden ascendente de peso, sin importar dónde estén: durante buena parte de la ejecución lo construido no es un árbol sino un bosque de pedazos sueltos y el arco (c, e) se descarta porque cerraría un ciclo. El algoritmo de Prim, en cambio, mantiene siempre un único árbol que crece desde el vértice a y por eso los pesos que escoge no aparecen en orden ascendente.

Los dos terminan con el mismo conjunto de cinco arcos, de peso total 16. Que coincidan no es casualidad: en este grafo los pesos son distintos par a par y el MST resulta ser único (Ejercicio 26).

Nota 4.7.2: Un problema, dos criterios

Esta sección se aparta de la organización de las anteriores y conviene anunciarlo. En cada una de ellas se propuso un criterio de selección, se demostró para él un teorema de optimización local y se concluyó con uno de optimización

global. Aquí se presentan *dos* algoritmos voraces distintos –el de Kruskal y el de Prim– que resuelven el mismo problema con criterios de selección distintos.

Lo notable es que no hacen falta dos teoremas de optimización local. Los dos criterios son casos particulares de un mismo resultado, el teorema del corte y por eso el desarrollo tiene la forma de un tronco común –el de esta subsección y las dos siguientes– del que cuelgan dos ramas simétricas. Al llegar a ellas, cada una aportará su criterio de selección, su implementación y su análisis, mientras que la corrección estará ya demostrada.

4.7.1. El esquema genérico. La idea general para construir un MST de $G = (V, E)$ con función de peso w es la de siempre: partir del conjunto vacío de arcos y agregar arcos, uno a la vez, cuidando de no arruinar la posibilidad de llegar a una solución óptima. Lo primero es precisar en qué consiste ese cuidado.

Definición 4.7.2

Un conjunto de arcos $A \subseteq E$ es *factible* sii $A \subseteq E_T$ para algún MST $T = (V, E_T)$ de G . Un arco $e \in E \setminus A$ es *seguro* para un conjunto factible A sii $A \cup \{e\}$ es factible.

Nota 4.7.3: Factible no es lo mismo que ser parte de un MST

La Definición 4.7.2 exige que *todo* A esté contenido en un mismo MST y no basta con pedir que cada uno de sus arcos, por separado, haga parte de alguno.

El Ejemplo 4.7.1 lo muestra. Allí cada uno de los tres arcos hace parte de algún MST del triángulo, pero el conjunto formado por los tres no está contenido en ninguno: tiene un ciclo y ningún árbol lo tiene. La condición débil se satisface y la fuerte no.

La distinción importa porque lo que hay que preservar al construir la solución es la condición fuerte. Un algoritmo que solo garantice la débil puede terminar con un conjunto de arcos que no es un árbol.

Note, por último, que la factibilidad de un conjunto de arcos es más exigente que la de una solución. Un árbol de cubrimiento cualquiera es una solución factible del Problema 4.7.1, pero sus arcos forman un conjunto factible únicamente cuando el árbol es de peso mínimo.

Con esta noción, el esquema es inmediato: mantener a A factible y extenderlo con arcos seguros hasta que sea un árbol de cubrimiento.

Algoritmo 4.7.1: Esquema genérico para construir un MST

1. $A = \emptyset$.
2. Mientras $|A| < |V| - 1$:
 - a) encontrar un arco $e \in E \setminus A$ que sea seguro para A ;
 - b) extender A con e .
3. Retornar A .

Nota 4.7.4: Por qué esto no es un algoritmo voraz

Conviene ser claro sobre qué es y qué no es el Algoritmo 4.7.1. Ser seguro *no* es un criterio de selección en el sentido de la Definición 4.1.3: la Definición 4.7.2 está formulada en términos de los MST de G , que son exactamente lo que se está tratando de calcular. Un algoritmo no puede consultarla.

El Algoritmo 4.7.1 es, entonces, una plantilla de corrección y no un algoritmo. Su utilidad es la de fijar de una vez la obligación que cada algoritmo concreto debe cumplir: exhibir un criterio de selección *evaluable* y demostrar que los arcos que escoge son seguros. Los algoritmos voraces de esta sección son los de Kruskal y Prim; el esquema es lo que tienen en común.

4.7.2. Optimización local: el teorema del corte. Las siguientes nociones son las que permiten enunciar un criterio suficiente para que un arco sea seguro, sin mencionar los MST de G .

Definición 4.7.3: Cortes y arcos ligeros

Sea $G = (V, E)$ un grafo no dirigido con función de peso w .

- Un *corte* de V es una pareja (S_0, S_1) de subconjuntos no vacíos de V tales que $S_0 \cap S_1 = \emptyset$ y $S_0 \cup S_1 = V$.
- Un arco $e \in E$ *cruza* un corte (S_0, S_1) de V sii uno de los vértices en e está en S_0 y el otro está en S_1 .
- Un corte (S_0, S_1) de V *respeto* un subconjunto de arcos $E' \subseteq E$ sii ningún arco en E' cruza (S_0, S_1) .
- Un arco $e \in E$ es *ligero* para un corte (S_0, S_1) de V sii $w(e)$ es mínimo entre todos los arcos que cruzan (S_0, S_1) (note que puede haber más de un arco ligero para un corte (S_0, S_1) de V).

El resultado que se presenta a continuación es la base matemática sobre la cual funcionan los algoritmos voraces comúnmente utilizados para calcular el MST de un grafo. Es el teorema de optimización local de esta sección y, a diferencia de los de las secciones anteriores, va a servir para dos criterios de selección distintos.

Teorema 4.7.1: Optimización local: el teorema del corte

Sea $A \subseteq E$ factible y (S_0, S_1) un corte de V que respeta a A . Si $(u, v) \in E$ es un arco ligero para (S_0, S_1) , entonces (u, v) es seguro para A .

Demostración

Como A es factible, hay un MST $T = (V, E_T)$ de G con $A \subseteq E_T$. Note además que $(u, v) \notin A$, pues (u, v) cruza un corte que respeta a A . Se procede por casos.

- Si $(u, v) \in E_T$, entonces $A \cup \{(u, v)\} \subseteq E_T$ y el mismo T atestigua que $A \cup \{(u, v)\}$ es factible. Por ende, (u, v) es seguro para A .
- Si $(u, v) \notin E_T$, entonces note que $(V, E_T \cup \{(u, v)\})$ no es un árbol (¿por qué?). Como T es un árbol, hay un camino simple único (¿por qué?) entre u y v en T . Dado que u y v están en “lados” opuestos de (S_0, S_1) , al menos un arco en el camino de u a v cruza este corte. Sea (x, y) dicho arco: observe que (x, y) no está en A porque (S_0, S_1) respeta a A . Como el camino simple entre u y v es único en T , remover (x, y) de E_T desconecta T en dos componentes. Estas dos componentes se pueden volver a unir agregando el arco (u, v) . Es decir, $T' = (V, (E_T \setminus \{(x, y)\}) \cup \{(u, v)\})$ es un árbol de cubrimiento de G .

El objetivo ahora es demostrar que el árbol de cubrimiento T' de G es de costo mínimo. Para ello, note lo siguiente:

$$\begin{aligned}
 w(T') &= (+e \mid e \in (E_T \setminus \{(x, y)\}) \cup \{(u, v)\} : w(e)) \\
 &= (+e \mid e \in E_T : w(e)) + w(u, v) - w(x, y) \\
 &\leq (+e \mid e \in E_T : w(e)) \\
 &= w(T).
 \end{aligned}$$

El paso de la desigualdad es posible porque (u, v) es ligero para (S_0, S_1) y (x, y) también cruza ese corte, de modo que $w(u, v) \leq w(x, y)$. Por otra parte, se sabe que T es un MST de G . Luego $w(T) \leq w(T')$. De las dos desigualdades se concluye

$$w(T) = w(T').$$

Es decir, T' es un MST de G . Finalmente, $A \subseteq E_T \setminus \{(x, y)\}$ porque $(x, y) \notin A$, y por lo tanto $A \cup \{(u, v)\}$ está contenido en los arcos de T' . Es decir, $A \cup \{(u, v)\}$ es factible y (u, v) es seguro para A .

Nota 4.7.5: Otra vez el intercambio

La demostración del Teorema 4.7.1 es un argumento de intercambio de los descritos en la Nota 4.2.3, aunque de una variedad algo más elaborada que las de las secciones anteriores. Se parte de una solución óptima arbitraria que contiene a A , se identifica en ella el arco (x, y) al cual la elección voraz puede sustituir –que no es un arco cualquiera, sino uno que el camino entre u y v obliga a que exista– y se verifica que el reemplazo no aumenta el peso total.

La propiedad óptima que hace legítimo el intercambio es que (u, v) sea ligero: como (x, y) también cruza el corte, su peso no puede ser menor que el de (u, v) . Y el papel del corte que respeta a A es garantizar que el arco sacrificado no pertenezca a A , es decir, que el intercambio no deshaga ninguna decisión ya tomada.

4.7.3. Optimización global. Con el teorema del corte demostrado, la corrección del Algoritmo 4.7.1 genérico es una inducción rutinaria. Note que el enunciado es condicional: afirma que *si* el paso 2(a) siempre encuentra un arco seguro, entonces el resultado es correcto.

Teorema 4.7.2: Optimización global

Sea $G = (V, E)$ conexo con $V \neq \emptyset$. Si en cada iteración el paso 2(a) del Algoritmo 4.7.1 encuentra un arco seguro para A , entonces el algoritmo termina y el conjunto A que retorna es el conjunto de arcos de un MST de G .

Demostración

Se demuestra primero que “ A es factible” es un invariante del ciclo del paso 2.

Antes de la primera iteración, $A = \emptyset$. Como G es conexo y $V \neq \emptyset$, el grafo tiene al menos un árbol de cubrimiento y, como E es finito, tiene al menos uno de peso mínimo; sea $T = (V, E_T)$ uno de ellos. Entonces $\emptyset \subseteq E_T$ y A es factible. Si A es factible al comienzo de una iteración, por suposición el paso

2(a) encuentra un arco $e \in E \setminus A$ seguro para A y el paso 2(b) lo agrega. Por la Definición 4.7.2, $A \cup \{e\}$ es factible.

El ciclo termina: como $e \notin A$, cada iteración incrementa $|A|$ en exactamente 1, de modo que después de a lo sumo $|V| - 1$ iteraciones la guarda del paso 2 deja de cumplirse.

Al terminar el ciclo, A es factible y $|A| = |V| - 1$. Sea $T = (V, E_T)$ un MST de G con $A \subseteq E_T$. Por el Ejercicio 2, $|E_T| = |V| - 1 = |A|$ y un subconjunto de un conjunto finito que tiene su mismo tamaño es el conjunto entero. Luego $A = E_T$ y A es el conjunto de arcos de un MST de G .

Nota 4.7.6: Lo que le falta al esquema

El Teorema 4.7.2 deja pendiente exactamente una cosa y es la hipótesis: que el paso 2(a) *encuentre* un arco seguro cada vez que la guarda se cumple. Nada de lo dicho hasta ahora garantiza que exista.

Esa es, entonces, la obligación que cada concreción del esquema debe cumplir, y son dos cosas distintas:

1. exhibir un criterio de selección evaluable y demostrar, con ayuda del Teorema 4.7.1, que el arco que escoge es seguro; y
2. demostrar que, mientras $|A| < |V| - 1$, ese criterio siempre tiene algo que escoger.

Las dos subsecciones siguientes hacen ese trabajo dos veces, para dos criterios distintos.

Nota 4.7.7: Un problema anormalmente permisivo

Las secciones anteriores dedicaron un esfuerzo apreciable a refutar criterios de selección plausibles y la Nota 4.1.5 insistió en que romperlos es lo primero que hay que intentar. Con los árboles de cubrimiento mínimo ese ejercicio resulta curiosamente infructuoso: casi todo criterio razonable funciona.

Escoger repetidamente el arco más liviano que no cierre un ciclo funciona y es el algoritmo de Kruskal. Escoger el arco más liviano que conecte lo ya construido con lo que falta funciona y es el de Prim. Escoger simultáneamente, para cada componente, su arco incidente más liviano también funciona y es el algoritmo de Borůvka mencionado en la Nota 4.0.3. Incluso funciona proceder al revés, eliminando repetidamente el arco más pesado cuya remoción no desconecte el grafo (Ejercicio 27).

Esa abundancia no es casualidad ni suerte. Los árboles de cubrimiento son el ejemplo canónico de una estructura llamada *matroide* y el resultado central de esa teoría –al que aludía la Nota 4.0.3– caracteriza precisamente las situaciones en las cuales la estrategia voraz es óptima. Las notas al final del capítulo indican dónde estudiarla.

4.7.4. El algoritmo de Kruskal. El primero de los dos algoritmos concreta el esquema con el criterio de selección más simple que se le puede ocurrir a uno: recorrer los arcos de menor a mayor peso y quedarse con los que se pueda.

Algoritmo 4.7.2: Kruskal

El Algoritmo 4.7.1 con el siguiente criterio de selección: entre los arcos que no cierran un ciclo con A , escoger uno de peso mínimo.

A diferencia de ser seguro, esta condición sí se puede evaluar: para saber si un arco cierra un ciclo con A basta con mirar A y el peso de un arco viene dado. La obligación que impone la Nota 4.7.6 son dos resultados y el primero es una aplicación directa del teorema del corte.

Teorema 4.7.3: Optimización local para Kruskal

Sea $A \subseteq E$ factible y sea $(u, v) \in E \setminus A$ un arco de peso mínimo entre los que no cierran un ciclo con A . Entonces, (u, v) es seguro para A .

Demostración

Sea $S \subseteq V$ el conjunto de vértices de la componente conexa de (V, A) que contiene a u . Se verifica que $(S, V \setminus S)$ es un corte de V que respeta a A y para el cual (u, v) es ligero, con lo cual el Teorema 4.7.1 da la conclusión.

- $(S, V \setminus S)$ es un corte. El conjunto S no es vacío porque $u \in S$. Y $V \setminus S$ tampoco lo es porque $v \notin S$: si v estuviera en la componente de u , habría un camino entre u y v en (V, A) y el arco (u, v) cerraría un ciclo con A , contrario a la suposición.
- El corte respeta a A . Ningún arco de A cruza $(S, V \setminus S)$, pues los dos extremos de un arco de A están siempre en la misma componente conexa de (V, A) .
- (u, v) es ligero para el corte. El arco (u, v) cruza el corte, por lo dicho arriba. Sea (x, y) cualquier otro arco que lo cruce. Entonces x y y están

en componentes distintas de (V, A) y, por lo tanto, (x, y) no cierra un ciclo con A . Como (u, v) es de peso mínimo entre los arcos con esa propiedad, se tiene $w(u, v) \leq w(x, y)$.

Note dónde estuvo el trabajo. El criterio de Kruskal no menciona ningún corte, de modo que la demostración tuvo que fabricar uno –el inducido por la componente conexa de u – y verificar sus tres propiedades por separado. Esa construcción es la parte no evidente del argumento y conviene retenerla, porque es lo que permite que un criterio enunciado sobre ciclos se conecte con un teorema enunciado sobre cortes.

Queda la segunda obligación, que es garantizar que el criterio siempre tenga algo que escoger mientras la guarda del paso 2 se cumpla.

Teorema 4.7.4: Existencia

Sea $G = (V, E)$ conexo y $A \subseteq E$ con $|A| < |V| - 1$ tal que (V, A) es acíclico. Entonces, hay al menos un arco en $E \setminus A$ que no cierra un ciclo con A .

Demostración

Como (V, A) es acíclico y $|A| < |V| - 1$, el grafo (V, A) no es conexo (¿por qué?) y tiene al menos dos componentes. Sea S el conjunto de vértices de una de ellas, de modo que $S \neq \emptyset$ y $V \setminus S \neq \emptyset$. Como G es conexo, hay al menos un arco de E con un extremo en S y el otro en $V \setminus S$; ese arco no está en A y sus extremos están en componentes distintas de (V, A) , de modo que no cierra un ciclo.

Con las dos obligaciones cumplidas, la corrección del algoritmo se obtiene invocando el teorema del esquema genérico. El único detalle que falta es notar que la hipótesis de aciclicidad del resultado anterior se cumple siempre, pues todo conjunto factible está contenido en los arcos de un árbol.

Teorema 4.7.5: Optimización global para Kruskal

El Algoritmo 4.7.2 calcula los arcos de un MST de G .

Demostración

Todo conjunto factible es acíclico, pues está contenido en los arcos de un árbol. Por el Teorema 4.7.4, mientras la guarda del paso 2 se cumpla el criterio tiene al menos un arco entre los cuales escoger; y por el Teorema 4.7.3, el que escoge es seguro. Se satisface entonces la hipótesis del Teorema 4.7.2.

Representación del grafo. Para este algoritmo conviene representar el grafo como una *lista de arcos*: un arreglo $X[0..M]$ que contiene la tripleta $(u, v, w(u, v))$ por cada arco $(u, v) \in E$, en cualquier orden. Los vértices se identifican con los enteros en $[0..N]$. Es la representación natural aquí porque el criterio de selección ordena los arcos y los recorre uno por uno, sin preguntar nunca por los vecinos de un vértice.

Note además que el criterio es *estático* en el sentido de la Nota 4.1.3: el peso de un arco no cambia con lo que se haya seleccionado antes. Aplica entonces la receta de *ordenar y recorrer*, y lo único que falta es poder responder rápido si un arco cierra un ciclo.

Un tipo abstracto de datos. Decidir si el arco (u, v) cierra un ciclo con A equivale a decidir si u y v ya están en la misma componente conexa de (V, A) . Y las componentes conexas de un grafo son las clases de equivalencia de la relación “estar conectado con”, de modo que lo que hace falta es poder consultar y modificar una relación de equivalencia sobre los vértices.

Definición 4.7.4: El tipo abstracto disjoint-union

Sea U un conjunto finito. El tipo abstracto de datos *disjoint-union* sobre U mantiene una relación de equivalencia $\sim$ sobre U y ofrece tres operaciones:

- $make(U)$ construye la relación en la cual cada elemento es equivalente únicamente a sí mismo, en tiempo $O(|U|)$;
- $find(x)$, para $x \in U$, retorna el *representante canónico* de la clase de equivalencia de x , en tiempo $O(\log |U|)$;
- $union(x, y)$, para $x, y \in U$, reemplaza $\sim$ por la menor relación de equivalencia que la contiene y en la cual $x \sim y$, en tiempo $O(\log |U|)$.

El representante canónico es un elemento de la clase, el mismo para todos sus miembros; es decir, $find(x) = find(y)$ sii $x \sim y$.

La operación *union* merece un comentario. La “menor relación de equivalencia que contiene a $\sim$ y en la cual $x \sim y$ ” es exactamente la que resulta de fusionar en una sola las clases de x y de y , dejando las demás intactas (Ejercicio 8). Esa es la razón del nombre: el efecto de la operación es la unión de dos clases disjuntas.

Nota 4.7.8: Tipo abstracto y estructura de datos

Conviene separar dos nociones que suelen confundirse.

Un *tipo abstracto de datos* (en inglés, *abstract data type* o ADT) describe *qué* se puede hacer y a qué costo: un conjunto de operaciones, con sus parámetros, su comportamiento y una restricción de complejidad temporal sobre cada una. Nada dice sobre *cómo* se logra. La Definición 4.7.4 es un ADT.

Una *estructura de datos* (en inglés, *data structure* o DS) describe *cómo*: una representación concreta en memoria y unos algoritmos que operan sobre ella. Se dice que una DS *implementa* un ADT cuando ofrece todas sus operaciones, con el comportamiento especificado, dentro de las restricciones de complejidad exigidas.

La separación paga dos veces. Permite analizar el Algoritmo 4.7.2 antes de haber escogido una implementación, contando operaciones del ADT en lugar de instrucciones. Y permite reemplazar la implementación por otra sin revisar nada de lo anterior, siempre que la nueva respete las mismas restricciones.

Una estructura de datos: *dforest*. La estructura que implementa el ADT anterior representa cada clase de equivalencia como un árbol cuya raíz es el representante canónico de la clase; el conjunto de todas ellas es un bosque, de donde viene el nombre *dforest*. Se guarda en dos arreglos: $p[0..N)$ con el padre de cada elemento, donde una raíz es su propio padre, y $sz[0..N)$ con el tamaño del árbol que cada raíz encabeza.

Algoritmo 4.7.3: La estructura dforest

```

1 def dmake(N):
2     return [ list(range(N)), [1]*N ]
3
4 def dfind(D, x):
5     p = D[0]
6     while p[x] != x: x = p[x]
7     return x
8
9 def dunion(D, x, y):
10    p, sz = D[0], D[1]
11    rx, ry = dfind(D, x), dfind(D, y)
12    if rx != ry:
13        if sz[rx] < sz[ry]: rx, ry = ry, rx
14        p[ry] = rx

```

15 $sz[rx] = sz[rx] + sz[ry]$

La función `dfind` sube de padre en padre hasta llegar a la raíz, de modo que su costo es proporcional a la altura del árbol. La función `dunion` cuelga una raíz de la otra y ahí está la única decisión de diseño: cuelga siempre el árbol *más pequeño* del más grande. Esa regla, llamada *unión por tamaño*, es lo que impide que los árboles se vuelvan altos.

Teorema 4.7.6

En un *dforest* construido con `dmake` y modificado únicamente con `dunion`, todo árbol de altura h tiene al menos 2^h elementos.

Demostración

Se procede por inducción sobre la cantidad de llamados a `dunion` ejecutados.

Si no se ha ejecutado ninguno, cada árbol tiene un solo elemento y altura 0, y $1 \geq 2^0$.

Suponga que la propiedad vale antes de un llamado a `dunion` que efectivamente fusiona dos árboles, con raíces r_x y r_y , alturas h_x y h_y y tamaños $s_x \geq s_y$; el código cuelga r_y de r_x . Los demás árboles no cambian. El árbol resultante tiene tamaño $s_x + s_y$ y altura $h_x \uparrow (h_y + 1)$. Hay dos casos:

- Si $h_x \geq h_y + 1$, la altura sigue siendo h_x y el tamaño creció, de modo que $s_x + s_y > s_x \geq 2^{h_x}$ por la Hipótesis Inductiva.
- Si $h_x < h_y + 1$, la altura pasa a ser $h_y + 1$ y, usando $s_x \geq s_y$ y la Hipótesis Inductiva sobre el árbol de r_y :

$$s_x + s_y \geq 2 \cdot s_y \geq 2 \cdot 2^{h_y} = 2^{h_y+1}.$$

Teorema 4.7.7

El Algoritmo 4.7.3 implementa el ADT *disjoint-union* de la Definición 4.7.4 sobre $U = [0..N)$.

Demostración

Que las operaciones se comportan como exige la Definición 4.7.4 se propone como ejercicio al lector (Ejercicio 9). Para las restricciones de complejidad, note que `dmake` construye dos arreglos de tamaño N en tiempo $O(N)$.

Por el Teorema 4.7.6, un árbol de altura h tiene al menos 2^h elementos y, como ningún árbol tiene más de N elementos, se cumple $2^h \leq N$; es decir, $h \leq \log_2 N$. El ciclo de `dfind` ejecuta a lo sumo h iteraciones, cada una en tiempo constante, de modo que `dfind` toma tiempo $O(\log N)$. La función `dunion` hace dos llamados a `dfind` y una cantidad constante de trabajo adicional, de modo que también toma tiempo $O(\log N)$.

Nota 4.7.9: Compresión de caminos

La unión por tamaño no es la única técnica disponible. La *compresión de caminos* consiste en aprovechar cada llamado a `dfind` para reconectar directamente a la raíz todos los elementos que se visitaron en el ascenso, de manera que las consultas siguientes sobre ellos sean inmediatas. Combinada con la unión por tamaño, el costo amortizado de las dos operaciones deja de ser $O(\log N)$ y pasa a ser esencialmente constante; las notas al final del capítulo indican dónde estudiar el resultado preciso, que es notablemente difícil.

Para lo que sigue, sin embargo, la mejora es irrelevante. El Algoritmo 4.7.2 tiene que ordenar los arcos y ese ordenamiento va a dominar el costo total sin importar qué tan rápidas sean las operaciones sobre el *dforest*. Es un ejemplo instructivo de que optimizar la parte más sofisticada de un algoritmo no siempre mejora nada. El Ejercicio 12 propone implementarla y medir.

La implementación. Para llegar a un algoritmo hace falta un último ingrediente. El criterio del Algoritmo 4.7.2 pide, en cada paso, el arco de peso mínimo entre los que no cierran un ciclo, lo cual parece obligar a revisar todos los arcos disponibles cada vez. No es necesario: si $X[0..M)$ contiene los arcos de E ordenados ascendentemente por peso y ya se examinaron los de $X[0..m)$, entonces basta con mirar $X[m]$. En efecto, los arcos de $X[0..m)$ que no fueron seleccionados cierran un ciclo con lo construido y los de $X[m..M)$ no pesan menos que $X[m]$. Luego, si $X[m]$ no cierra un ciclo, es de peso mínimo entre los que no lo cierran.

Escríbanse $N = |V|$ y $M = |E|$. Considere entonces la siguiente función objetivo, definida para $0 \leq m \leq M$, un conjunto de arcos A y un valor D del ADT

disjoint-union sobre V , bajo tres suposiciones:

- Y_0 : $A \subseteq X[0..m)$ y A es factible;
- Y_1 : todo arco de $X[0..m)$ que no está en A cierra un ciclo con A ;
- Y_2 : las clases de D son las componentes conexas de (V, A) .

$\phi(A, m, D)$: “los arcos del árbol de cubrimiento mínimo que resulta de extender A con arcos de $X[m..M)$, según el criterio del Algoritmo 4.7.2.”

Para resolver el problema del árbol de cubrimiento mínimo, el objetivo es calcular $\phi(\emptyset, 0, \text{make}(V))$, llamado en el cual las tres suposiciones se cumplen trivialmente.

La definición recurrente de ϕ se establece tomando a m como pivote y preguntando, por medio del ADT, si el arco $X[m] = (u, v, w)$ cierra un ciclo con A . Para aligerar la notación se escriben $r_u = \text{find}(u)$ y $r_v = \text{find}(v)$ para los representantes canónicos de u y v en D , y $\text{end}(A, m)$ para la condición $m = M \vee |A| = N - 1$:

$$\phi(A, m, D) = \begin{cases} A & , \text{ si } \text{end}(A, m), \\ \phi(A, m+1, D) & , \text{ si } \neg \text{end}(A, m) \wedge r_u = r_v, \\ \phi(A \cup \{X[m]\}, m+1, \text{union}(u, v)) & , \text{ si } \neg \text{end}(A, m) \wedge r_u \neq r_v. \end{cases}$$

El caso base cubre dos situaciones distintas y conviene distinguirlas. La primera, $m = M$, es la habitual: no quedan arcos por examinar. La segunda, $|A| = N - 1$, es una parada temprana: por el Ejercicio 2, un conjunto factible de ese tamaño es ya el conjunto de arcos de un árbol de cubrimiento, de modo que ningún arco restante puede agregarse y recorrer lo que falta de $X[m..M)$ sería trabajo perdido. El primer caso inductivo corresponde a la situación en la cual u y v ya están en la misma componente conexa de (V, A) : el arco cerraría un ciclo, se descarta y se recurre sin modificar nada. El segundo caso inductivo corresponde a la situación en la cual u y v están en componentes distintas: el arco se agrega a A y, para que la suposición Y_2 siga cumpliéndose, hay que fusionar las dos clases correspondientes de D .

Nota 4.7.10: Las suposiciones son los invariantes

Las tres suposiciones sobre los parámetros de ϕ no son decoración. La suposición Y_0 es la que hace correcto al algoritmo y es el invariante del Teorema 4.7.2. La suposición Y_1 es la que permite mirar únicamente a $X[m]$ en lugar de recorrer todos los arcos disponibles. Y la suposición Y_2 es la que hace correcta la consulta al ADT: sin ella, comparar r_u con r_v no diría nada sobre ciclos.

Al transcribir ϕ a una versión iterativa, las tres se convierten en los invariantes del ciclo (Ejercicio 13). Esta correspondencia entre las suposiciones

de una función objetivo y los invariantes del ciclo que la calcula es general y conviene tenerla presente.

Teorema 4.7.8

Sea $G = (V, E)$ conexo con función de peso w y suponga que $X[0..M)$ contiene los arcos de E , con sus pesos, ordenados ascendentemente por w . Entonces $\phi(\emptyset, 0, \text{make}(V))$ es el conjunto de arcos de un MST de G .

Demostración

Basta con verificar que la secuencia de arcos que ϕ agrega a A es una ejecución del Algoritmo 4.7.2, pues entonces el Teorema 4.7.5 da la conclusión.

Las tres suposiciones se cumplen en el llamado inicial y se preservan en cada paso: Y_0 por el Teorema 4.7.3, que garantiza que el arco agregado es seguro; Y_1 porque el arco $X[m]$ se descarta únicamente cuando cierra un ciclo; y Y_2 por el llamado a *union*.

Por Y_2 , la condición $r_u \neq r_v$ equivale a que $X[m]$ no cierre un ciclo con A ; y por Y_1 y el ordenamiento de $X[0..M)$, según se argumentó arriba, ese arco es de peso mínimo entre los que no cierran un ciclo. Es decir, cada arco que ϕ agrega es uno de los que el criterio del Algoritmo 4.7.2 habría escogido.

Las dos maneras de terminar corresponden con la guarda del paso 2 del Algoritmo 4.7.1. Si ϕ se detiene porque $|A| = N - 1$, la guarda dejó de cumplirse y el algoritmo habría terminado también. Y si se detiene porque $m = M$ con $|A| < N - 1$, entonces todo arco de E cierra un ciclo con A y, por el Teorema 4.7.4, el grafo G no era conexo, contrario a la suposición.

Con base en la definición de ϕ , es claro cómo diseñar un algoritmo recurrente para su implementación.

Algoritmo 4.7.4

```

1 def kruskal(X, N, M, A, m, D):
2   # X[0..M) holds edges (u, v, w) sorted by weight
3   ans = None
4   if m==M or len(A)==N-1: ans = A
5   else:
6     u, v, w = X[m]
```

```

7     if dfind(D, u)!=dfind(D, v):
8         A.append(X[m])
9         dunion(D, u, v)
10    ans = kruskal(X, N, M, A, m+1, D)
11    return ans
12
13 def mst_kruskal(X, N, M):
14     X.sort(key = lambda x: x[2])
15     return kruskal(X, N, M, [], 0, dmake(N))

```

Nota 4.7.11: Estado compartido

La definición de ϕ trata a A y a D como valores: el segundo caso inductivo recurre con $A \cup \{X[m]\}$ y con $union(u, v)$, que son objetos nuevos. El Algoritmo 4.7.4, en cambio, modifica en sitio la lista y el *dforest*, y los pasa a la recurrencia tal cual.

Las dos versiones coinciden porque ningún llamado vuelve sobre un estado anterior: la recurrencia avanza siempre de m a $m + 1$ y nunca deshace una decisión. Copiar A en cada paso sería correcto, pero costaría tiempo $O(N)$ por llamado y arruinaría la complejidad que se establece a continuación.

Ejemplo 4.7.3

Considere el grafo del Ejemplo 4.7.2, con los vértices $a, \dots, f$ identificados con los enteros $0, \dots, 5$:

```

1 X = [ (0,1,4), (1,2,8), (0,3,1), (1,4,6),
2       (2,5,2), (3,4,7), (4,5,3), (2,4,5) ]

```

El llamado a `mst_kruskal` produce los cinco arcos del MST, en el orden en el que fueron seleccionados:

```

1 >>> mst_kruskal(X, 6, len(X))
2 [(0, 3, 1), (2, 5, 2), (4, 5, 3), (0, 1, 4), (1, 4, 6)]

```

El resultado coincide con la fila superior del Ejemplo 4.7.2, incluido el descarte del arco $(2, 4, 5)$ —el arco (c, e) de peso 5— que es el único que la recurrencia examina sin agregar.

Teorema 4.7.9

Sea $G = (V, E)$ conexo con función de peso w , $|V| = N$ y $|E| = M$, representado por la lista de arcos $X[0..M)$. El llamado `mst_kruskal(X, N, M)` calcula los arcos de un MST de G y toma tiempo $O(M \log M)$ y espacio (sin considerar la pila de ejecución para la recurrencia) $O(N)$.

Demostración

La corrección es el Teorema 4.7.8: al ordenar $X[0..M)$ ascendentemente por peso, `mst_kruskal` establece la hipótesis que ese teorema requiere y el resto es el cálculo de $\phi(\emptyset, 0, \text{make}(V))$.

El ordenamiento toma tiempo $O(M \log M)$ y el llamado a `dmake` toma tiempo $O(N)$. La recurrencia hace a lo sumo $M + 1$ llamados –menos, si la parada temprana se activa– y cada uno ejecuta a lo sumo tres operaciones del ADT, cada una en tiempo $O(\log N)$ por el Teorema 4.7.7, más una cantidad constante de trabajo adicional; la recurrencia toma entonces tiempo $O(M \log N)$. Como G es conexo se tiene $N - 1 \leq M$ y, por lo tanto, $\log N \in O(\log M)$: el costo de la recurrencia es $O(M \log M)$ y el total también.

El espacio adicional es el de los dos arreglos del *dforest*, que es $O(N)$, más el de la respuesta, que tiene $N - 1$ arcos. Teniendo en cuenta el espacio de la pila, el espacio es $O(M)$ debido a los llamados recurrentes.

El resultado confirma lo que anticipaba la Nota 4.7.9: el costo queda dominado por el ordenamiento y no por la estructura de datos. También ilustra el Teorema 4.1.2 con un caso en el cual la verificación de factibilidad no es gratuita: acá $O(f) = O(\log N)$ y no $O(1)$ como en las secciones anteriores; aun así, el ordenamiento se impone.

4.7.5. El algoritmo de Prim. El segundo algoritmo concreta el esquema con un criterio de selección distinto. En lugar de recorrer todos los arcos del grafo en orden de peso, mantiene un conjunto S de vértices ya conectados y se pregunta, en cada paso, cuál es el arco más liviano que sale de S .

Algoritmo 4.7.5: Prim

El Algoritmo 4.7.1 con el siguiente criterio de selección, a partir de un vértice inicial $r \in V$ y de $S = \{r\}$: escoger un arco ligero para el corte $(S, V \setminus S)$ y agregar a S el extremo del arco que aún no estaba allí.

Este criterio también se puede evaluar, pues solo menciona los pesos de los arcos y el conjunto S , que el algoritmo conoce. Las dos obligaciones de la Nota 4.7.6 son, en este caso, casi inmediatas.

Teorema 4.7.10: Optimización local para Prim

Sea $S \subseteq V$ con $S \neq \emptyset$ y $S \neq V$, y sea $A \subseteq E$ factible tal que todo arco de A tiene sus dos extremos en S . Si (u, v) es un arco ligero para el corte $(S, V \setminus S)$, entonces (u, v) es seguro para A .

Demostración

La pareja $(S, V \setminus S)$ es un corte de V , pues ambas partes son no vacías por suposición. El corte respeta a A , pues ningún arco con sus dos extremos en S puede cruzarlo. Y (u, v) es ligero para él, también por suposición. El Teorema 4.7.1 da la conclusión.

La brevedad de la demostración anterior no debe engañar: es la que tiene que ser, porque el criterio de Prim está enunciado directamente en términos de un corte y no hay nada que construir. Toda la dificultad la absorbió el Teorema 4.7.1.

Queda la segunda obligación, que es garantizar que el criterio siempre tenga algo que escoger. Es igual de directa.

Teorema 4.7.11: Existencia

Sea $G = (V, E)$ conexo y $S \subseteq V$ con $S \neq \emptyset$ y $S \neq V$. Entonces, hay al menos un arco de E que cruza el corte $(S, V \setminus S)$.

Demostración

De lo contrario, ningún camino de G podría ir de un vértice de S a uno de $V \setminus S$ y el grafo no sería conexo.

Con las dos obligaciones cumplidas, la corrección del algoritmo se obtiene invocando el teorema del esquema genérico. Lo único que hay que verificar es que las

hipótesis de los dos resultados anteriores se cumplen en cada iteración y para eso basta observar que el conjunto S crece exactamente en un vértice por cada arco seleccionado.

Teorema 4.7.12: Optimización global para Prim

El Algoritmo 4.7.5 calcula los arcos de un MST de G .

Demostración

Mientras la guarda del paso 2 se cumpla, se tiene $|A| < |V| - 1$ y, como el algoritmo agrega un vértice a S por cada arco que selecciona, se tiene $S \neq V$; por el Teorema 4.7.11 hay entonces algún arco que cruza el corte y el criterio tiene entre cuáles escoger. Todo arco seleccionado tiene sus dos extremos en S al momento de agregarse, de modo que el Teorema 4.7.10 garantiza que es seguro. Se satisface entonces la hipótesis del Teorema 4.7.2.

Nota 4.7.12: El mismo teorema, dos veces

Vale la pena comparar las demostraciones de los dos teoremas de optimización local. Ambas consisten en exhibir un corte que respete a A y para el cual el arco escogido sea ligero, y en las dos el trabajo está en encontrar ese corte.

En Prim el corte es explícito: es $(S, V \setminus S)$, el algoritmo lo mantiene y el criterio está enunciado en términos de él. En Kruskal está oculto: el criterio no lo menciona y la demostración del Teorema 4.7.3 tiene que fabricarlo a partir de la componente conexa del arco escogido. Esa es toda la diferencia entre los dos algoritmos y explica por qué Kruskal construye un bosque mientras Prim construye un árbol: Kruskal usa un corte distinto en cada paso y Prim usa siempre el mismo, que crece.

Representación del grafo. A diferencia de Kruskal, el criterio de Prim pregunta repetidamente por los arcos incidentes a un vértice y no recorre la colección de arcos en un orden fijo. Para este algoritmo conviene entonces representar el grafo con *listas de adyacencia*: un arreglo $G[0..N]$ en el cual $G[u]$ es la lista de las parejas (v, w) tales que (u, v) es un arco de peso w . Los vértices se identifican con los enteros en $[0..N]$ y, como el grafo es no dirigido, cada arco aparece dos veces: la pareja (v, w) en $G[u]$ y la pareja (u, w) en $G[v]$. En total, las listas ocupan espacio $O(N + M)$ y recorrerlas todas toma tiempo $O(N + M)$.

Note que el criterio es *dinámico* en el sentido de la Nota 4.1.3: cuál es el arco más liviano que sale de S depende de S , que cambia en cada paso. No aplica entonces la receta de *ordenar y recorrer* que sirvió en todas las secciones anteriores, y hace falta una estructura que entregue el mínimo y admita modificaciones.

Un tipo abstracto de datos. Antes de buscar esa estructura conviene precisar qué se le va a pedir. La observación que lo permite es la siguiente: en lugar de guardar los arcos que cruzan el corte, basta con guardar, para cada vértice $v \notin S$, el peso del arco más liviano que lo conecta con S . Ese número se llamará la *llave* de v y vale $+\infty$ cuando ningún arco conecta a v con S . El arco ligero del corte es entonces el que corresponde al vértice de llave mínima y al mover un vértice u de $V \setminus S$ a S solo hay que revisar si los arcos incidentes a u mejoran alguna llave.

Definición 4.7.5: El tipo abstracto minheap

Sea U un conjunto finito con $|U| = N$. El tipo abstracto de datos *minheap* sobre U mantiene un subconjunto $Q \subseteq U$ y una llave $key(x) \in \mathbb{R} \cup \{+\infty\}$ para cada $x \in Q$, y ofrece cuatro operaciones:

- *make*(U) construye el estado con $Q = U$ y $key(x) = +\infty$ para todo $x \in U$, en tiempo $O(N)$;
- *empty*() retorna un valor Booleano que indica si $Q = \emptyset$, en tiempo $O(1)$;
- *extract*() remueve de Q un elemento x de llave mínima y retorna la pareja $(x, key(x))$, en tiempo $O(\log N)$;
- *decrease*(x, k) reemplaza $key(x)$ por k y retorna verdadero cuando $x \in Q$ y $k < key(x)$; en cualquier otro caso no modifica nada y retorna falso, en tiempo $O(\log N)$.

Que *extract* retorne la llave junto con el elemento no es un capricho: esa llave es el peso del arco que se va a agregar a la solución y sin ella habría que consultarlo aparte.

Una estructura de datos: *bheap*. La estructura que implementa el ADT anterior es el *montón binario*. Consiste en mantener los elementos de Q en un árbol binario *casi completo* –todos los niveles llenos salvo quizás el último, que se llena de izquierda a derecha– sujeto a la condición de que la llave de cada nodo no supere la de sus hijos. Bajo esa condición, el elemento de llave mínima está siempre en la raíz.

Un árbol de esa forma no necesita apuntadores: se guarda en un arreglo $h[0..K)$ en el cual la raíz es $h[0]$ y los hijos de $h[i]$ son $h[2i + 1]$ y $h[2i + 2]$, cuando esos índices existen. La estructura completa consta de cuatro campos: ese arreglo, un

arreglo $key[0..N)$ con las llaves, un arreglo $pos[0..N)$ que indica en qué posición de h está cada elemento –o -1 si ya no está en Q – y el tamaño K de Q .

Esa correspondencia entre posiciones del arreglo y nodos del árbol es la única aritmética que la estructura necesita, y conviene aislarla en funciones propias para que el resto del código pueda escribirse hablando de padres y de hijos en lugar de sumas y productos.

Nota 4.7.13: ¿Para qué sirve el arreglo de posiciones?

El arreglo $pos[0..N)$ es la parte del diseño que no es evidente y conviene entender por qué está ahí.

La operación $decrease(x, k)$ tiene que modificar la llave de un elemento x dado y para eso necesita saber en qué posición del montón se encuentra. Sin $pos[0..N)$ habría que buscarlo recorriendo $h[0..K)$, lo cual costaría $O(N)$ y haría imposible cumplir la cota de la Definición 4.7.5. Mantenerlo cuesta poco: cada vez que dos elementos intercambian posiciones en h , se intercambian también sus entradas en pos .

Esta es la diferencia entre un montón binario y una simple cola de prioridad de las que solo insertan y extraen. La operación $decrease$ obliga a poder *entrar* al montón por un elemento cualquiera y no solo por la raíz.

Algoritmo 4.7.6: La estructura bheap

```

1 INF = float("inf")
2
3 def bis_root(i): return i==0
4
5 def bparent(i): return (i-1)>>1
6
7 def bleft(i): return 1+(i<<1)
8
9 def bright(i): return (1+i)<<1
10
11 def bswap(H, i, j):
12     h, pos = H[0], H[2]
13     h[i], h[j] = h[j], h[i]
14     pos[h[i]], pos[h[j]] = i, j
15
16 def bup(H, i):
```



```

17 if not(bis_root(i)):
18     h, key, p = H[0], H[1], bparent(i)
19     if key[h[p]]>key[h[i]]:
20         bswap(H, i, p)
21         bup(H, p)
22
23 def bdown(H, i):
24     h, key, K = H[0], H[1], H[3]
25     l, r, b = bleft(i), bright(i), i
26     if l<K and key[h[l]]<key[h[b]]: b = l
27     if r<K and key[h[r]]<key[h[b]]: b = r
28     if b!=i:
29         bswap(H, i, b)
30         bdown(H, b)

```

Las cuatro primeras funciones son la traducción entre las dos maneras de ver la estructura. La función `bis_root` responde si una posición es la raíz; `bparent`, `bleft` y `bright` entregan las posiciones del padre y de los dos hijos. Están escritas con corrimientos de bits $-i \ll 1$ es $2i$ y $(i-1) \gg 1$ es $\lfloor (i-1)/2 \rfloor$ — porque es la manera habitual de multiplicar y dividir por dos, pero lo que importa no es esa economía sino que, una vez definidas, ningún otro lugar del código vuelve a mencionar un índice calculado.

Note que `bparent` solo tiene sentido cuando la posición no es la raíz, razón por la cual `bup` consulta primero a `bis_root` y únicamente después calcula el padre (Ejercicio 17).

Las funciones `bup` y `bdown` restauran la condición del montón cuando una sola llave se ha alterado y las dos son recurrentes. La primera compara la llave del elemento con la de su padre y, si es menor, los intercambia y vuelve a llamarse sobre la posición del padre; se detiene al llegar a la raíz o al encontrar un padre que ya no la supera. La segunda escoge, entre la posición dada y las de sus hijos que existan, la de llave mínima. Si no es la propia, intercambia y vuelve a llamarse sobre ella; se detiene cuando el elemento ya no supera a ninguno de sus hijos. En ambos casos el llamado recurrente es la última instrucción que se ejecuta, de modo que las dos son recursivas de cola y se pueden convertir en ciclos sin esfuerzo (Ejercicio 18). Con ellas, las cuatro operaciones del ADT se escriben de inmediato.

Algoritmo 4.7.7: Las operaciones sobre un bheap

```

1 def bmake(N):
2     return [ list(range(N)), [INF]*N, list(range(N)), N ]
3

```

```

4 def bempty(H):
5     return H[3]==0
6
7 def bextract(H):
8     h, key, pos = H[0], H[1], H[2]
9     u = h[0]
10    H[3] = H[3]-1
11    bswap(H, 0, H[3])
12    pos[u] = -1
13    bdown(H, 0)
14    return u, key[u]
15
16 def bdecrease(H, x, k):
17     key, pos = H[1], H[2]
18     ans = pos[x]>=0 and k<key[x]
19     if ans:
20         key[x] = k
21         bup(H, pos[x])
22     return ans

```

La función `bmake` no tiene que hacer ningún trabajo de reacomodo porque todas las llaves valen $+\infty$ y la condición del montón se cumple trivialmente. La función `bextract` toma la raíz, trae el último elemento a ocupar su lugar y lo hace bajar. La función `bdecrease` baja la llave y hace subir al elemento desde la posición que le indica `pos`.

Teorema 4.7.13

Un montón binario con $K \geq 1$ elementos tiene altura a lo sumo $\log_2 K$.

Demostración

Considere un nodo a profundidad d , es decir, uno al cual se llega desde la raíz siguiendo d arcos. Como los hijos de $h[i]$ están en las posiciones $2i+1$ y $2i+2$, la posición de un nodo a profundidad d es al menos $2^d - 1$ (¿por qué?). Como las posiciones ocupadas son las de $[0..K)$, se tiene $2^d - 1 \leq K - 1$; es decir, $d \leq \log_2 K$.

Teorema 4.7.14

El Algoritmo 4.7.7 implementa el ADT *minheap* de la Definición 4.7.5 sobre $U = [0..N)$.

Demostración

Que las operaciones se comportan como exige la Definición 4.7.5 se propone como ejercicio al lector (Ejercicio 16). Para las restricciones de complejidad:

- **bmake** construye tres arreglos de tamaño N en tiempo $O(N)$;
- **bempty** consulta un campo, en tiempo $O(1)$;
- **bextract** hace una cantidad constante de trabajo y un llamado a **bdown**. Cada llamado a **bdown** hace una cantidad constante de trabajo y, cuando recurre, lo hace sobre la posición de un hijo, es decir, un nivel más abajo. Por el Teorema 4.7.13 la cadena de llamados tiene longitud a lo sumo $\log_2 N$. En total, $O(\log N)$;
- **bdecrease** hace una cantidad constante de trabajo y un llamado a **bup**, que por la misma razón produce una cadena de longitud a lo sumo $\log_2 N$; esta vez cada llamado recurre sobre la posición del padre, un nivel más arriba. En total, $O(\log N)$.

Note, por último, que **bup** y **bdown** consumen espacio de pila proporcional a la longitud de su cadena de llamados, es decir, $O(\log N)$. Es una cantidad despreciable frente a los $O(N)$ que ocupan los tres arreglos de la estructura, y desaparece del todo si se las escribe iterativamente.

Nota 4.7.14: Una estructura más simple

El montón binario no es la única manera de resolver el problema y no siempre es la mejor. La alternativa más simple consiste en guardar las llaves en un arreglo y no hacer ningún esfuerzo por mantenerlas ordenadas: entonces *decrease* toma tiempo $O(1)$, pero *extract* tiene que recorrer el arreglo completo y toma tiempo $O(N)$.

Esa estructura no cumple la Definición 4.7.5, de modo que no implementa el ADT y no puede sustituirse por el Algoritmo 4.7.7 sin más. Pero sería un error descartarla: como se verá en la Sección 4.7 más adelante, combinada con una matriz de pesos produce una versión de Prim que le gana al montón binario cuando el grafo es denso. El Ejercicio 21 propone desarrollarla.

La implementación. Escribáse $N = |V|$. El estado del algoritmo consta de cuatro piezas: el conjunto A de arcos ya seleccionados, el conjunto Q de los vértices que aún no se han extraído, un valor H del ADT *minheap* que asigna una llave a cada elemento de Q y un arreglo $P[0..N)$ que registra, para cada vértice, el vecino que realiza su llave. Como es usual, S denota el conjunto $[0..N) \setminus Q$ de los vértices ya extraídos.

El conjunto Q queda determinado por H , pues es su primera componente según la Definición 4.7.5. Se lleva de todos modos como un parámetro aparte porque es lo que cambia de un llamado al siguiente y porque las suposiciones que vienen a continuación se enuncian en términos suyos.

Considere la siguiente función objetivo, bajo tres suposiciones:

- Z_0 : A es factible y todo arco de A tiene sus dos extremos en S ;
- Z_1 : (S, A) es un árbol;
- Z_2 : para todo $v \in Q$, $key(v) = (\downarrow x \mid x \in S \wedge (x, v) \in E : w(x, v))$ y, cuando ese valor es finito, $P[v]$ es un vértice de S que lo alcanza.

$\phi(A, Q, H, P)$: “los arcos del árbol de cubrimiento mínimo que resulta de extender A con arcos incidentes a S , según el criterio del Algoritmo 4.7.5.”

Para resolver el problema del árbol de cubrimiento mínimo desde un vértice inicial r , el objetivo es calcular $\phi(\emptyset, V, decrease(r, 0), P_0)$, donde $P_0[v] = v$ para todo v . En ese llamado $S = \emptyset$ y las suposiciones Z_0 y Z_1 se cumplen trivialmente; la suposición Z_2 es la única que no y el efecto de haber bajado a 0 la llave de r es precisamente que r sea el primer vértice extraído, con lo cual Z_2 pasa a cumplirse desde entonces.

La definición recurrente de ϕ se establece tomando como pivote el vértice de llave mínima. Escribanse $(u, k) = extract()$ para ese vértice y su llave, H' para el estado del *minheap* después de extraerlo y Q' para $Q \setminus \{u\}$. La relajación de u modifica dos de los cuatro parámetros a la vez, de modo que se escribe:

$$(H'', P') = relax(u, H', P)$$

para la pareja que resulta de aplicar $decrease(v, w)$ a toda pareja (v, w) en $G[u]$, actualizando $P[v] = u$ cada vez que la operación retorne verdadero. Escribáse, por último, a_u para el arco $(P[u], u, k)$, que es el que se agrega a la solución cuando u no es el vértice inicial. Con esa notación:

$$\phi(A, Q, H, P) = \begin{cases} A & , \text{ si } empty(Q), \\ \phi(A, Q', H'', P') & , \text{ si } \neg empty(Q) \wedge P[u] = u, \\ \phi(A \cup \{a_u\}, Q', H'', P') & , \text{ si } \neg empty(Q) \wedge P[u] \neq u. \end{cases}$$

El caso base corresponde a la situación en la cual no quedan vértices por agregar a S . El primer caso inductivo corresponde al vértice inicial: es el único que se extrae sin que ningún arco lo haya conectado con S , lo cual se reconoce en que $P[u]$ sigue valiendo u . El segundo caso inductivo es el habitual: el arco $(P[u], u)$ es un arco ligero del corte, su peso es la llave k con la cual u fue extraído y se agrega a la solución. En ambos casos hay que relajar, para que la suposición Z_2 siga cumpliéndose tras haber crecido S con u .

Nota 4.7.15: ¿Por qué el mínimo de las llaves es un arco ligero?

El criterio del Algoritmo 4.7.5 pide un arco ligero para el corte $(S, V \setminus S)$ y la recurrencia extrae un vértice de llave mínima. Que las dos cosas coincidan es consecuencia de la suposición Z_2 .

En efecto, por Z_2 la llave de cada $v \in Q$ es el peso del arco más liviano entre v y S , de modo que el mínimo de las llaves sobre Q es el peso del arco más liviano que cruza el corte. Si u alcanza ese mínimo, entonces $(P[u], u)$ es un arco de ese peso que cruza el corte, es decir, un arco ligero.

Teorema 4.7.15

Sea $G = (V, E)$ conexo con $|V| = N$, representado por sus listas de adyacencia, y sea $r \in V$. Entonces $\phi(\emptyset, V, \text{decrease}(r, 0), P_0)$ es el conjunto de arcos de un MST de G .

Demostración

Basta con verificar que la secuencia de arcos que ϕ agrega a A es una ejecución del Algoritmo 4.7.5, pues entonces el Teorema 4.7.12 da la conclusión.

Tras la primera extracción se tiene $S = \{r\}$ y las tres suposiciones se cumplen. Se preservan en cada paso: Z_0 porque el arco agregado tiene un extremo en S y el otro es u , que pasa a estar en S ; Z_1 porque ese arco conecta u con el árbol ya construido sin crear ciclos, dado que $u \notin S$ antes de extraerlo; y Z_2 por el llamado a *relax*, que recorre $G[u]$ y por lo tanto revisa todos los arcos incidentes al único vértice que se agregó a S .

Por la Nota 4.7.15, el arco $(P[u], u)$ que se agrega en el segundo caso inductivo es un arco ligero del corte $(S, V \setminus S)$, que es exactamente lo que el criterio del Algoritmo 4.7.5 escoge.

Falta ver que el primer caso inductivo ocurre únicamente en la primera extracción. Como G es conexo y $S \neq \emptyset$ después de ella, el Teorema 4.7.11 garantiza que algún arco cruza el corte; por Z_2 , el vértice extraído tiene entonces llave finita y $P[u] \neq u$.

Con base en la definición de ϕ , es claro cómo diseñar un algoritmo recurrente para su implementación.

Algoritmo 4.7.8

```

1 def prim(G, N, A, H, P):
2     # G[u] is the list of pairs (v, w) for the edges out of u
3     ans = None
4     if bempty(H): ans = A
5     else:
6         u, k = bextract(H)
7         if P[u] != u: A.append((P[u], u, k))
8         for v, w in G[u]:
9             if bdecrease(H, v, w): P[v] = u
10        ans = prim(G, N, A, H, P)
11    return ans
12
13 def mst_prim(G, N, r):
14     H = bmake(N)
15     bdecrease(H, r, 0)
16     return prim(G, N, [], H, list(range(N)))

```

El ciclo de la línea 8 es la operación *relax*: recorre la lista de adyacencia de u y deja que el ADT decida, en cada vecino, si la llave mejora. Note que no hace falta preguntar si v sigue en Q ni comparar llaves a mano, porque `bdecrease` ya hace las dos cosas y reporta el resultado. Como en el Algoritmo 4.7.4, la lista A y el estado H se modifican en sitio y se pasan a la recurrencia tal cual, lo cual es correcto por la razón que da la Nota 4.7.11.

Note también que el conjunto Q , que la definición de ϕ lleva como parámetro, no aparece en el código: vive dentro de H , que es precisamente lo que la estructura de datos oculta. Por eso `bempty` responde sin recibirlo aparte y `bextract` lo reduce en uno como efecto de su llamado.

Ejemplo 4.7.4

Considere el grafo del Ejemplo 4.7.2, con los vértices $a, \dots, f$ identificados con los enteros $0, \dots, 5$. Sus listas de adyacencia se construyen a partir de la lista de arcos:

```

1 >>> N = 6
2 >>> G = [ [] for u in range(N) ]
3 >>> for u, v, w in X: G[u].append((v,w)); G[v].append((u,w))
4 >>> G[4]
5 [(1, 6), (3, 7), (5, 3), (2, 5)]

```

El llamado a `mst_prim` desde el vértice a produce los cinco arcos del MST, en el orden en el que fueron seleccionados y orientados del vértice que ya estaba en S hacia el que se agrega:

```

1 >>> mst_prim(G, N, 0)
2 [(0, 3, 1), (0, 1, 4), (1, 4, 6), (4, 5, 3), (5, 2, 2)]

```

El resultado coincide con la fila inferior del Ejemplo 4.7.2. Compárelo con el del Ejemplo 4.7.3: el conjunto de arcos es el mismo, pero el orden en el que aparecen no lo es y, a diferencia de Kruskal, sus pesos no son ascendentes.

Teorema 4.7.16

Sea $G = (V, E)$ conexo con $|V| = N$ y $|E| = M$, representado por sus listas de adyacencia. El llamado `mst_prim(G, N, r)` calcula los arcos de un MST de G y toma tiempo $O((N + M) \log N)$ y espacio (sin considerar la pila de ejecución para la recurrencia) $O(N)$.

Demostración

La corrección es el Teorema 4.7.15.

El llamado a `bmake` toma tiempo $O(N)$. La recurrencia hace $N + 1$ llamados, pues cada uno extrae un vértice y hay N de ellos. Cada llamado ejecuta una operación *extract*, en tiempo $O(\log N)$ por el Teorema 4.7.14; en total, $O(N \log N)$.

El ciclo de relajación recorre $G[u]$ una vez por cada vértice u y hace una operación *decrease* por cada pareja. Como cada arco aparece dos veces entre todas las listas de adyacencia, la cantidad total de operaciones *decrease* es $2M$ y el tiempo que consumen es $O(M \log N)$. Recorrer las listas cuesta, además, $O(N + M)$.

Sumando, el tiempo total es $O(N + N \log N + M \log N + N + M) = O((N + M) \log N)$.

El espacio adicional es el de los tres arreglos del *bheap* y el del arreglo $P[0..N]$, que es $O(N)$, más el de la respuesta, que tiene $N - 1$ arcos.

Como G es conexo se tiene $N - 1 \leq M$ y, por lo tanto, la cota se puede escribir simplemente como $O(M \log N)$. Vale la pena notar dónde quedó cada término: el $N \log N$ viene de las extracciones y el $M \log N$ de las relajaciones. Es el segundo el que domina y es también el que explica por qué la representación importa tanto como la estructura de datos: con una matriz de pesos las relajaciones serían N^2 en lugar de $2M$, y ninguna cola de prioridad podría arreglarlo.

4.7.6. ¿Cuál de los dos? Los dos algoritmos resuelven el mismo problema. Antes de compararlos conviene separar en qué difieren estructuralmente, que es lo que no depende de ninguna decisión de implementación.

	Kruskal	Prim
Criterio	el arco más liviano sin ciclo	el más liviano que sale de S
Es	estático	dinámico
Corte	uno distinto en cada paso	siempre el mismo, que crece
Construye	un bosque	un árbol

En cuanto al costo, la comparación no es entre dos algoritmos sino entre tres implementaciones, porque Prim admite dos que no son comparables entre sí. La tercera es la que propone el Ejercicio 21, con matriz de pesos y llaves en un arreglo sin ordenar.

Algoritmo	Representación	Estructura	Tiempo
Kruskal	lista de arcos	<i>dforest</i>	$O(M \log N)$
Prim	listas de adyacencia	<i>bheap</i>	$O(M \log N)$
Prim	matriz de pesos	arreglo	$O(N^2)$

La primera fila merece una aclaración: el Teorema 4.7.9 da $O(M \log M)$, pero en un grafo simple se tiene $M \leq N^2$ y por lo tanto $\log M \leq 2 \log N$, de modo que las dos expresiones son la misma. Es decir, las dos primeras implementaciones son asintóticamente equivalentes y la elección entre ellas se decide por otras razones: cuál representación del grafo ya se tiene a la mano o cuál de las dos estructuras de datos resulta más fácil de programar correctamente.

La tercera fila es la interesante. Como M está entre $N - 1$ y $\binom{N}{2}$, cuál gana depende de la densidad del grafo. Para un grafo disperso, con $M \in \Theta(N)$, las dos primeras toman tiempo $O(N \log N)$ y le ganan con holgura al $O(N^2)$ de la tercera.

Para un grafo denso, con $M \in \Theta(N^2)$, las dos primeras toman tiempo $O(N^2 \log N)$ y pierden. El punto de equilibrio está alrededor de $M \in \Theta(N^2/\log N)$.

Conviene, eso sí, no leer la tabla como si las tres columnas fueran independientes. Nada obliga a implementar Kruskal con lista de arcos ni Prim con listas de adyacencia; lo que ocurre es que cada criterio de selección hace natural una representación y esa representación hace natural una estructura de datos. Cambiar una de las tres decisiones obliga a revisar las otras dos y la tercera fila de la tabla es precisamente lo que resulta de cambiarlas todas a la vez.

Ejercicios

1. Sea $G = (V, E)$ un grafo no dirigido y conexo, y $w : E \rightarrow \mathbb{R}_{>0}$ una función de peso para los arcos; note que este ejercicio agrega la hipótesis de positividad, que el Problema 4.7.1 no impone. Demuestre que si $T = (V, E_T)$, con $E_T \subseteq E$, es un subgrafo conexo de G de costo mínimo (con respecto a w), entonces T es un árbol (de cubrimiento mínimo de G). ¿Dónde se usa la positividad? Exhiba un contraejemplo si se permite que algún peso sea 0.
2. Sea T un árbol con $N \geq 1$ vértices. Demuestre que T tiene $N - 1$ arcos.
3. Demuestre que la noción de árbol en la Definición 4.7.1 es equivalente a la siguiente definición:
 Un *árbol* es un grafo en el cual existe exactamente un camino simple (i.e., sin repeticiones) entre cualquier par de vértices.
4. En el caso cuando $(u, v) \notin E_T$, en la demostración del Teorema 4.7.1, se hacen las siguientes afirmaciones. Demuestre que cada una de ellas es cierta:
 - a) El grafo $(V, E_T \cup \{(u, v)\})$ no es un árbol.
 - b) En T hay un camino simple único entre u y v .
 - c) Al menos un arco del camino entre u y v cruza (S_0, S_1) .
5. La Nota 4.7.3 distingue dos condiciones sobre un conjunto de arcos A : la fuerte, que es ser factible, y la débil, que es que cada arco de A haga parte de algún MST.
 - a) Demuestre que la primera implica la segunda.
 - b) Demuestre que un conjunto que satisface la débil puede tener un ciclo y por lo tanto no estar contenido en ningún árbol.
 - c) ¿Es invariante la condición débil bajo el paso 2(b) del Algoritmo 4.7.1? Justifique.
6. El Teorema 4.7.2 supone $V \neq \emptyset$.
 - a) ¿Qué hace el Algoritmo 4.7.1 cuando $|V| = 1$?
 - b) ¿Por qué la demostración necesita la suposición?
7. En el paso 2(a) del Algoritmo 4.7.1 se exige que el arco seguro esté en $E \setminus A$.

- a) ¿Qué parte de la demostración del Teorema 4.7.2 deja de funcionar si se omite esa exigencia?
 - b) Demuestre que la exigencia no le quita generalidad al esquema.
8. En la Definición 4.7.4, la operación $\text{union}(x, y)$ se define como la menor relación de equivalencia que contiene a $\sim$ y en la cual $x \sim y$.
 - a) Demuestre que esa relación existe y es única.
 - b) Demuestre que sus clases son las de $\sim$, salvo que las de x y y aparecen fusionadas en una sola.
9. Demuestre que las funciones del Algoritmo 4.7.3 se comportan como exige la Definición 4.7.4. *Ayuda:* formule primero el invariante que relaciona el contenido de $p[0..N)$ con la relación de equivalencia representada, y verifique que dmake lo establece y dunion lo preserva.
10. La cota del Teorema 4.7.6 es ajustada.
 - a) Construya, para cada $k \geq 0$, una secuencia de llamados a dunion sobre $N = 2^k$ elementos que produzca un árbol de altura exactamente k .
 - b) Concluya que dfind toma tiempo $\Theta(\log N)$ en el peor caso.
11. Suponga que dunion cuelga siempre r_y de r_x , sin comparar tamaños.
 - a) Exhiba una secuencia de llamados que produzca un árbol de altura $N - 1$.
 - b) ¿Qué complejidad temporal tendría entonces el Algoritmo 4.7.4?
12. Implemente la compresión de caminos descrita en la Nota 4.7.9.
 - a) Modifique dfind para que, después de encontrar la raíz, reconecte a ella todos los elementos visitados.
 - b) Verifique experimentalmente que el Algoritmo 4.7.4 no mejora de manera apreciable y explique por qué.
 - c) Diseñe un experimento en el cual la compresión de caminos sí produzca una mejora apreciable.
13. Diseñe una versión iterativa del Algoritmo 4.7.4.
 - a) Formule los invariantes del ciclo y verifique que son las suposiciones Y_0 , Y_1 y Y_2 de la función objetivo.
 - b) Enuncie y demuestre el teorema de correctitud correspondiente.
 - c) Compare el espacio adicional que usan las dos versiones. ¿Cuál es la ventaja de la iterativa y de dónde viene?
14. Modifique el Algoritmo 4.7.4 para que:
 - a) retorne únicamente el peso del MST, en espacio adicional $O(N)$;
 - b) no tenga la parada temprana $|A| = N - 1$. Mida, sobre grafos aleatorios, cuántos llamados ahorra esa condición. ¿Cambia la complejidad asintótica?
 - c) reporte que G no es conexo, en lugar de retornar un bosque.
15. Demuestre la afirmación que se hace en la demostración del Teorema 4.7.4: si (V, A) es acíclico y $|A| < |V| - 1$, entonces (V, A) no es conexo.

16. Demuestre que las funciones del Algoritmo 4.7.7 se comportan como exige la Definición 4.7.5. *Ayuda:* formule primero los invariantes que relacionan los cuatro campos de la estructura con la pareja (Q, key) ; después demuestre, por inducción sobre la longitud de la cadena de llamados, que `bup` y `bdown` restauran la condición del montón cuando una sola llave se ha alterado.
17. Las funciones `bparent`, `bleft` y `bright` del Algoritmo 4.7.6 están escritas con corrimientos de bits.
 - a) Verifique que `bleft(i)` vale $2i + 1$ y que `bright(i)` vale $2i + 2$.
 - b) Demuestre que `bparent` es inversa de las otras dos: el padre del hijo izquierdo de i y el del hijo derecho de i son ambos i .
 - c) ¿Qué retorna `bparent(0)`? ¿Por qué eso no causa problemas en `bup`, y qué ocurriría si el llamado a `bis_root` se hiciera después y no antes?
18. Escriba versiones iterativas de `bup` y de `bdown`.
 - a) Formule el invariante del ciclo de cada una y verifique que corresponde con la hipótesis inductiva de la demostración recurrente.
 - b) ¿Cambia alguna de las cotas del Teorema 4.7.14?
 - c) ¿Qué se gana y qué se pierde con el cambio?
19. Justifique las siguientes afirmaciones hechas sobre el Algoritmo 4.7.6:
 - a) En la demostración del Teorema 4.7.13, la posición de un nodo a profundidad d es al menos $2^d - 1$.
 - b) La función `bmake` no necesita reacomodar nada.
 - c) La función `bextract` supone que $Q \neq \emptyset$. ¿Por qué el Algoritmo 4.7.8 nunca la llama con $Q = \emptyset$?
20. Suponga que se elimina el arreglo `pos[0..N)` del Algoritmo 4.7.6 y que `bdecrease` busca el elemento recorriendo `h[0..K)`.
 - a) ¿Qué complejidad temporal tendría `decrease`?
 - b) ¿Seguiría la estructura cumpliendo la Definición 4.7.5?
 - c) ¿Qué complejidad temporal tendría el Algoritmo 4.7.8?
21. 🌐 Desarrolle la variante de Prim que anuncia la Nota 4.7.14, con el grafo representado por su matriz de pesos $W[0..N)[0..N)$, en la cual $W[u][v]$ es el peso del arco entre u y v y vale INF cuando no hay tal arco.
 - a) Implemente la estructura que guarda las llaves en un arreglo sin ordenar, y demuestre que `extract` toma tiempo $O(N)$ y `decrease` toma tiempo $O(1)$.
 - b) Explique por qué esa estructura no implementa el ADT de la Definición 4.7.5 y por qué, a pesar de eso, sirve.
 - c) Reescriba el Algoritmo 4.7.8 para que use la matriz y esa estructura, y demuestre que toma tiempo $O(N^2)$.
 - d) ¿Por qué usar la matriz de pesos con un *bheap* no produce nada mejor que $O(N^2 \log N)$?

22. Modifique el Algoritmo 4.7.8 para que retorne únicamente el peso del MST, sin construir la lista de arcos. ¿Cuánto espacio adicional ahorra?
23. La Nota 4.7.12 sostiene que la diferencia entre los dos algoritmos está en el corte que usa cada uno.
 - a) Ejecute a mano el Algoritmo 4.7.4 sobre el grafo del Ejemplo 4.7.2 y, para cada arco seleccionado, exhiba el corte que la demostración del Teorema 4.7.3 construye.
 - b) Haga lo mismo con el Algoritmo 4.7.8.
 - c) ¿Coinciden en algún paso?
24. El Algoritmo 4.7.5 depende de un vértice inicial r .
 - a) Demuestre que el peso del árbol que calcula no depende de cuál sea.
 - b) Exhiba un grafo y dos vértices iniciales para los cuales el *conjunto* de arcos sí sea distinto.
25. Compare las tres implementaciones de la Sección 4.7, cuyos tiempos son $O(M \log N)$, $O(M \log N)$ y $O(N^2)$.
 - a) Determine para qué valores de M , en función de N , cada una es preferible.
 - b) Impleméntelas y compárelas experimentalmente sobre grafos aleatorios de densidad creciente. ¿Coincide el punto de equilibrio medido con el previsto?
 - c) Las dos primeras son asintóticamente equivalentes. ¿Lo son también en la práctica?
26. Demuestre que si los pesos de los arcos de G son distintos par a par, entonces G tiene un único árbol de cubrimiento mínimo. *Ayuda:* suponga que hay dos y considere el arco de menor peso que está en uno y no en el otro.
27. 🌈 Considere la siguiente estrategia, que procede al revés que el Algoritmo 4.7.1: comenzar con $A = E$ y remover repetidamente el arco más pesado cuya remoción no desconecte el grafo, hasta que no se pueda más.
 - a) Enuncie el resultado análogo al Teorema 4.7.1 para esta estrategia. *Ayuda:* el papel de los cortes lo cumplen los ciclos.
 - b) Demuéstrelo.
 - c) Concluya que la estrategia calcula un MST de G .

4.8. Caminos más cortos

La última sección del capítulo resuelve un problema de optimización sobre grafos que se parece al de la Sección 4.7 en la forma, pero no en el fondo. Dado un grafo con peso en sus arcos y un vértice de partida, se trata de determinar qué tan lejos está cada uno de los demás vértices, entendiendo la distancia como el peso total del camino más liviano que los conecta.

A diferencia de las secciones anteriores, aquí conviene trabajar con grafos *dirigidos*, pues los caminos más cortos rara vez son simétricos en las aplicaciones: una calle puede ser de un solo sentido, un vuelo puede costar distinto en cada dirección. Un grafo no dirigido es el caso particular en el cual cada arco aparece en ambos sentidos y con el mismo peso.

Definición 4.8.1

Sea $G = (V, E)$ un grafo dirigido con función de peso $w : E \rightarrow \mathbb{R}_{\geq 0}$. Un *camino* de u a v es una secuencia de vértices $u = x_0, x_1, \dots, x_k = v$, con $k \geq 0$, tal que $(x_i, x_{i+1}) \in E$ para todo $0 \leq i < k$. Su *peso* es la suma de los pesos de sus arcos.

La *distancia* de u a v , escrita $\delta(u, v)$, es el mínimo peso entre todos los caminos de u a v ; vale $+\infty$ cuando no hay ninguno.

Note que un camino de u a u de longitud $k = 0$ siempre existe y tiene peso 0, de modo que $\delta(u, u) = 0$ para todo vértice.

Problema 4.8.1: Caminos más cortos desde un origen

Entrada: Un grafo dirigido $G = (V, E)$ con función de peso $w : E \rightarrow \mathbb{R}_{\geq 0}$ y un vértice $s \in V$.

Salida: La distancia $\delta(s, v)$ para todo vértice $v \in V$.

Nota 4.8.1: Tres problemas, no uno

El Problema 4.8.1 es la variante llamada *de origen único*: se fija el vértice de partida y se pregunta por todos los destinos. Hay otras dos de uso corriente. La de *pareja única* fija también el destino y no se conoce ningún algoritmo que la resuelva asintóticamente más rápido que calcular todas las distancias desde el origen. La de *todas las parejas* pregunta por $\delta(u, v)$ para todo par de vértices y se resuelve con programación dinámica; las notas al final del capítulo indican dónde estudiarla.

Nota 4.8.2: Aquí el signo sí importa

La Nota 4.7.1 advirtió que la Sección 4.7 no usaba en ninguna parte la hipótesis sobre el signo de los pesos. Acá es al revés: la no negatividad que exige el Problema 4.8.1 es indispensable y conviene ver por qué antes de empezar.

Hay que separar dos preguntas que se confunden con facilidad, porque no tienen la misma respuesta.

La primera es si el *problema* está bien planteado, y lo que decide eso son los ciclos y no los arcos. Si hay un ciclo de peso total negativo al cual se puede llegar desde s , recorrerlo repetidamente baja el peso sin límite y $\delta(s, v)$ no existe para los vértices afectados. Mientras no haya ciclos negativos, en cambio, la distancia está perfectamente bien definida aunque haya arcos de peso negativo y hay algoritmos que la calculan (Ejercicio 3).

La segunda es si el *algoritmo* de esta sección la calcula, y lo que decide eso son los arcos y no los ciclos. Basta un solo arco de peso negativo para que falle, aunque el grafo no tenga ningún ciclo: el Ejercicio 2 exhibe una instancia de tres vértices y ningún ciclo en la cual reporta una distancia mayor que la correcta.

Es decir, la hipótesis que exige el Problema 4.8.1 es más fuerte de lo que el problema necesita y exactamente lo que el algoritmo requiere. El lugar donde la demostración la usa se señalará cuando aparezca; vale la pena estar atento, porque es un solo paso.

Leído con el vocabulario de la Sección 4.1, este problema tiene una forma algo distinta de los anteriores. Para un vértice v fijo, las *soluciones factibles* son los caminos de s a v , la *función objetivo* de uno de ellos es su peso y el *sentido* es minimizar. Lo que el problema pide, sin embargo, no es un camino, sino su peso, y no para un vértice, sino para todos a la vez. El algoritmo resolverá los $|V|$ problemas de optimización simultáneamente, aprovechando que comparten el origen.

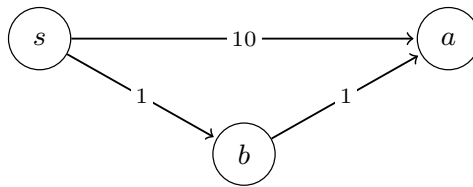
4.8.1. Escoger un criterio. La estrategia voraz mantendrá un conjunto S de vértices cuya distancia desde s ya se conoce con certeza y en cada paso agregará uno más. La pregunta es cuál.

Como en la Sección 4.7, conviene restringir de entrada los candidatos elegibles: solo tiene sentido agregar a S un vértice que esté conectado con S por algún arco. Los tres criterios siguientes respetan esa restricción. Escoger, entre los vértices que no están en S , aquel que:

1. se alcance desde S con la menor cantidad de arcos;
2. esté conectado con S por el arco más liviano; o
3. tenga la menor distancia acumulada desde s .

El segundo criterio merece atención especial, porque es exactamente el del Algoritmo 4.7.5. Si funcionara, el problema de esta sección se resolvería con el algoritmo de la anterior sin cambiarle una línea. No funciona y la misma instancia lo refuta junto con el primero.

Ejemplo 4.8.1: Dos criterios que no funcionan



Se trata de calcular las distancias desde s en el grafo dirigido de tres vértices que se dibuja arriba. Las respuestas correctas son $\delta(s, b) = 1$ y $\delta(s, a) = 2$, esta última por el camino que pasa por b .

El primer criterio agrega a S los vértices a y b de una vez, pues ambos se alcanzan desde s con un solo arco, y le asigna a a la distancia 10. Se queda corto: nunca revisa el camino de dos arcos que la mejora.

El segundo criterio agrega primero a b , cuyo arco desde s pesa 1, y luego agrega a a porque el arco (b, a) pesa 1, que es menos que los 10 del arco (s, a) . Le asigna entonces a a la distancia 1. Se pasa, porque suma mal: el peso del arco no es la distancia al origen.

Los dos errores van en direcciones opuestas y la comparación es instructiva. El primer criterio cuenta arcos y no pesos, de modo que ignora que un camino largo puede ser liviano. El segundo mira el peso del arco pero olvida lo que costó llegar hasta donde ese arco empieza. El criterio correcto es el que combina las dos cosas: lo que importa no es el peso del último arco sino la suma de todo el camino.

Nota 4.8.3: Prim y Dijkstra

El Ejemplo 4.8.1 deja ver que la diferencia entre el algoritmo de esta sección y el de la anterior es de una sola cantidad. Los dos mantienen un conjunto S que crece, los dos le asignan a cada vértice de afuera una llave y los dos extraen repetidamente el de llave mínima. Lo único que cambia es qué mide la llave:

	Prim	Dijkstra
Llave de v	$w(x, v)$	$D[x] + w(x, v)$
Minimiza	la suma de los arcos del árbol	cada camino desde s
Los pesos	pueden ser negativos	deben ser no negativos

Que dos algoritmos tan parecidos resuelvan problemas distintos, y que uno tolere pesos negativos y el otro no, es una buena razón para desconfiar de los parecidos superficiales. El Ejercicio 4 propone precisar la relación.

4.8.2. Optimización local: el voraz va adelante. Fijado el criterio, hay que demostrar que sirve. Aquí el argumento de intercambio no está disponible: una solución es un camino y no hay manera de sustituirle un vértice por otro conservando la factibilidad. Lo que sí se puede hacer es comparar directamente lo que el algoritmo afirma contra un camino arbitrario, que es el tercero de los patrones descritos en la Sección 4.3.

Teorema 4.8.1: Optimización local

Sea $S \subseteq V$ con $s \in S$ y sea D una asignación tal que $D[x] = \delta(s, x)$ para todo $x \in S$. Para $v \notin S$ defínase:

$$D[v] = (\downarrow x \mid x \in S \wedge (x, v) \in E : D[x] + w(x, v)),$$

con la convención de que el mínimo del conjunto vacío es $+\infty$. Si $u \notin S$ es tal que $D[u]$ es mínimo entre los vértices que no están en S , entonces $D[u] = \delta(s, u)$.

Demostración

Se demuestran las dos desigualdades.

Para $\delta(s, u) \leq D[u]$, note que si $D[u] = +\infty$ no hay nada que demostrar. De lo contrario, sea $x \in S$ el vértice que alcanza el mínimo, de modo que $D[u] = D[x] + w(x, u)$. Como $x \in S$ se tiene $D[x] = \delta(s, x)$ y por lo tanto hay un camino de s a x de peso $D[x]$; extendiéndolo con el arco (x, u) se obtiene un camino de s a u de peso $D[u]$. Luego $\delta(s, u) \leq D[u]$.

Para $D[u] \leq \delta(s, u)$, sea P un camino cualquiera de s a u ; basta ver que su peso no es menor que $D[u]$. Como $s \in S$ y $u \notin S$, en algún punto el camino sale de S : sea (x, y) el primer arco de P tal que $x \in S$ y $y \notin S$, y sea P_x el

tramo inicial de P , que va de s a x . Entonces:

$$\begin{aligned} w(P) &\geq w(P_x) + w(x, y) \\ &\geq \delta(s, x) + w(x, y) \\ &= D[x] + w(x, y) \\ &\geq D[y] \\ &\geq D[u]. \end{aligned}$$

La primera desigualdad descarta el tramo de P posterior al arco (x, y) ; la segunda vale porque P_x es un camino de s a x ; la igualdad porque $x \in S$; la tercera por la definición de $D[y]$, que toma el mínimo sobre todos los vértices de S y en particular sobre x ; y la última porque $y \notin S$ y $D[u]$ es mínimo entre los vértices que no están en S .

Nota 4.8.4: ¿Dónde se usa la no negatividad?

Toda la demostración anterior es aritmética elemental, salvo el primer paso: descartar el tramo de P posterior al arco (x, y) y afirmar que el peso no aumenta. Eso vale porque los arcos descartados tienen peso no negativo y es el *único* lugar de la sección donde la hipótesis se usa.

Con un arco de peso negativo el paso falla y el teorema deja de ser cierto, tal y como lo muestra el Ejercicio 2. Vale la pena notar lo poco que hace falta para romper un algoritmo: una desigualdad, en una línea de una demostración.

Note además que lo que se pierde es la corrección de *este* algoritmo y no la del problema. Como advirtió la Nota 4.8.2, con arcos negativos pero sin ciclos negativos las distancias siguen existiendo, y calcularlas exige renunciar a la estrategia voraz, y no a la especificación.

Nota 4.8.5: El voraz va adelante

El Teorema 4.8.1 es el ejemplar del tercero de los patrones catalogados en la Sección 4.3 y el único del capítulo en el cual se usa como demostración principal. La Nota 4.2.4 lo había anunciado.

Reconozca la forma. No se transforma ninguna solución óptima ni se intercambia nada; se toma un camino *arbitrario* P y se demuestra que lo que el algoritmo afirma, $D[u]$, no es peor que su peso. La medida de progreso es la asignación D restringida a S y lo que la cadena de desigualdades establece es que esa medida nunca queda por detrás de ninguna alternativa.

El intercambio no está disponible por una razón de fondo y no por falta de ingenio: en las secciones anteriores una solución era un conjunto o una asignación, objetos a los que se les puede cambiar una pieza dejando el resto intacto. Un camino no admite esa cirugía, pues cambiarle un vértice suele dejar de ser un camino.

4.8.3. Optimización global. El Teorema 4.8.1 justifica una sola extracción, y lo hace bajo una hipótesis fuerte: que D ya coincide con la distancia en todos los vértices de S . Encadenar las extracciones es entonces una inducción sobre el tamaño de S , en la cual esa hipótesis es justamente lo que hay que preservar.

Quedan dos cosas por verificar y ninguna la cubre el teorema anterior. La primera es cómo arranca la inducción, es decir, por qué el primer vértice extraído es el origen y por qué la distancia que se le asigna es la correcta. La segunda es que la hipótesis se preserva en cada paso, que es donde el teorema local hace su trabajo.

Teorema 4.8.2: Optimización global

Sea $G = (V, E)$ dirigido con pesos no negativos y $s \in V$. Repetir la extracción del Teorema 4.8.1, partiendo de $S = \emptyset$ y $D[s] = 0$, hasta que $S = V$, produce $D[v] = \delta(s, v)$ para todo $v \in V$.

Demostración

Se demuestra, por inducción sobre $|S|$, que $D[x] = \delta(s, x)$ para todo $x \in S$.

Si $S = \emptyset$ la afirmación es vacía. La primera extracción escoge a s , pues $D[s] = 0$ y ningún otro vértice tiene llave menor: los pesos son no negativos y por lo tanto ninguna llave lo es. Además $\delta(s, s) = 0$, ya que el camino de longitud cero tiene peso 0 y ningún camino puede pesar menos. Luego $D[s] = \delta(s, s)$ y la afirmación vale para $S = \{s\}$.

Si vale para S con $s \in S$ y $S \neq V$, el Teorema 4.8.1 garantiza que el vértice u que se extrae satisface $D[u] = \delta(s, u)$, de modo que sigue valiendo para $S \cup \{u\}$.

Al terminar se tiene $S = V$ y la afirmación es la conclusión del teorema.

Nota 4.8.6: Sin pasar por el esquema general

La demostración anterior no invoca el Teorema 4.1.1 y no es por descuido. Aquí una solución no es un subconjunto del conjunto de candidatos, como

pide la Definición 4.1.2, sino una asignación de distancias, de modo que el enunciado de aquel teorema no aplica literalmente; es la misma situación de la Nota 4.6.4.

Pero hay algo más y es lo que anticipó la Nota 4.2.4: cuando la optimización local se demuestra con el patrón de *el voraz va adelante*, la conclusión global sale de la propia inducción y no hace falta invocar nada. El teorema local ya comparó lo que el algoritmo produce contra una solución arbitraria; lo único que resta es encadenar las comparaciones.

4.8.4. Una implementación eficiente.

Lo que se hereda. Este algoritmo no necesita ninguna maquinaria nueva. El criterio es *dinámico* en el sentido de la Nota 4.1.3, igual que el de Prim, y las decisiones de la Sección 4.7 se aplican sin cambio alguno:

- el grafo se representa con *listas de adyacencia*, un arreglo $G[0..N]$ en el cual $G[u]$ es la lista de las parejas (v, w) tales que (u, v) es un arco de peso w ; a diferencia del caso no dirigido, ahora cada arco aparece una sola vez;
- las llaves se mantienen en el ADT *minheap* de la Definición 4.7.5, implementado por el Algoritmo 4.7.7.

Lo único que cambia respecto de Prim es el valor con el cual se relaja: donde aquel usaba el peso del arco, este usa la distancia acumulada.

La función objetivo. Escribase $N = |V|$. El estado del algoritmo consta de tres piezas: un arreglo $D[0..N]$ con las distancias ya establecidas, el conjunto Q de los vértices que aún no se han extraído y un valor H del ADT *minheap* que asigna una llave a cada elemento de Q . Como es usual, S denota el conjunto $[0..N] \setminus Q$ de los vértices ya extraídos.

El conjunto Q queda determinado por H , pues es su primera componente según la Definición 4.7.5. Se lleva de todos modos como un parámetro aparte porque es lo que cambia de un llamado al siguiente y porque las suposiciones que vienen a continuación se enuncian en términos suyos.

Considere la siguiente función objetivo, bajo dos suposiciones:

Z_0 : para todo $x \in S$, $D[x] = \delta(s, x)$;

Z_1 : para todo $v \in Q$,

$key(v) = (\downarrow x \mid x \in S \wedge (x, v) \in E : D[x] + w(x, v)).$

$\phi(D, Q, H)$: “el arreglo de distancias que resulta de establecer la de los vértices que aún están en Q .”

Escríbase D_0 para el arreglo que tiene $+\infty$ en todas sus posiciones y H_0 para el estado del *minheap* que resulta de aplicarle $decrease(s, 0)$ al recién construido. Para resolver el problema desde un origen s , el objetivo es calcular $\phi(D_0, V, H_0)$.

En ese llamado $S = \emptyset$ y la suposición Z_0 se cumple trivialmente; la suposición Z_1 es la única que no y el efecto de haber bajado a 0 la llave de s es precisamente que s sea el primer vértice extraído.

La definición recurrente de ϕ se establece tomando como pivote el vértice de llave mínima. Escribanse $(u, k) = extract()$ para ese vértice y su llave, H' para el estado del *minheap* después de extraerlo y $R = relax(u, k, H')$ para el resultado de aplicar $decrease(v, k+w)$ a toda pareja (v, w) en $G[u]$. Se escribe, además, $D[u := k]$ para el arreglo que coincide con D en todas sus posiciones salvo en la posición u , donde vale k :

$$\phi(D, Q, H) = \begin{cases} D & , \text{ si } empty(Q), \\ \phi(D[u := k], Q \setminus \{u\}, R) & , \text{ si } \neg empty(Q). \end{cases}$$

El caso base corresponde a la situación en la cual no quedan vértices por establecer. El caso inductivo registra en $D[u]$ la llave con la cual u fue extraído –que por el Teorema 4.8.1 es $\delta(s, u)$ – y relaja los arcos que salen de u , para que la suposición Z_1 siga cumpliéndose tras haber crecido S .

Nota 4.8.7: Más simple que Prim

Compare esta recurrencia con la de la Sección 4.7 y note que aquí no hay dos casos inductivos, sino uno. Allá el vértice inicial era una excepción, porque no había ningún arco que lo conectara con el árbol y por lo tanto no había arco que agregar a la solución. Acá no hay excepción que hacer: la distancia del origen a sí mismo es 0, que es exactamente la llave con la cual se extrae, y el caso general lo cubre.

Los vértices inalcanzables desde s tampoco necesitan tratamiento especial. Se extraen con llave $+\infty$, precisamente su distancia.

Teorema 4.8.3

Suponga que $G = (V, E)$ es dirigido, que sus pesos son no negativos y que $s \in V$. Entonces $\phi(D_0, V, H_0)[v] = \delta(s, v)$ para todo vértice v .

Demostración

La primera extracción escoge a s , por la razón que da la demostración del Teorema 4.8.2, y tras ella las dos suposiciones se cumplen. Se preservan en cada paso: Z_0 por el Teorema 4.8.1, que garantiza que la llave con la cual se extrae u es $\delta(s, u)$, y Z_1 por el llamado a *relax*, que recorre $G[u]$ y por lo tanto revisa todos los arcos que salen del único vértice que se agregó a S .

La secuencia de extracciones de ϕ es entonces la del Teorema 4.8.2 y su conclusión es la de este teorema.

El algoritmo. Con base en la definición de ϕ , es claro cómo diseñar un algoritmo recurrente para su implementación.

Algoritmo 4.8.1

```

1 def dijkstra(G, N, D, H):
2     # G[u] is the list of pairs (v, w) for the arcs out of u
3     ans = None
4     if bempty(H): ans = D
5     else:
6         u, k = bextract(H)
7         D[u] = k
8         for v, w in G[u]:
9             bdecrease(H, v, k+w)
10        ans = dijkstra(G, N, D, H)
11    return ans
12
13 def sssp(G, N, s):
14     H = bmake(N)
15     bdecrease(H, s, 0)
16    return dijkstra(G, N, [INF]*N, H)

```

El ciclo de la línea 8 es la operación *relax* y es la única línea que distingue a este algoritmo del de Prim: en lugar de ofrecer al ADT el peso w del arco, le ofrece la distancia acumulada $k + w$. Note que no hace falta preguntar si v sigue en Q ni comparar llaves a mano, porque *bdecrease* ya hace las dos cosas.

Note también que el conjunto Q , que la definición de ϕ lleva como parámetro, no aparece en el código: vive dentro de H , que es precisamente lo que la estructura de datos oculta. Por eso *bempty* responde sin recibirlo aparte y *bextract* lo reduce en uno como efecto de su llamado.

Ejemplo 4.8.2

Considere el grafo dirigido con cinco vértices y siete arcos que se describe a continuación, con origen en el vértice 0:

```

1 >>> N = 5
2 >>> A = [ (0,1,4), (0,2,1), (2,1,2), (1,3,5),
3 ...      (2,3,8), (3,4,3), (2,4,12) ]
4 >>> G = [ [] for u in range(N) ]
5 >>> for u, v, w in A: G[u].append((v,w))
6 >>> sssp(G, N, 0)
7 [0, 3, 1, 8, 11]
```

Los vértices se extraen en el orden 0, 2, 1, 3, 4, con llaves 0, 1, 3, 8, 11. Vale la pena seguir lo que le ocurre al vértice 1. Al extraer el origen, su llave baja a 4 por el arco (0, 1). Al extraer el vértice 2, cuya distancia es 1, el arco (2, 1) de peso 2 ofrece la distancia acumulada $1 + 2 = 3$, que mejora la anterior. Es esa la que queda y es la correcta: el camino más corto de 0 a 1 no es el arco directo, sino el que da un rodeo por el vértice 2.

Teorema 4.8.4

Sea $G = (V, E)$ dirigido con pesos no negativos, $|V| = N$ y $|E| = M$, representado por sus listas de adyacencia. El llamado `sssp(G, N, s)` calcula $\delta(s, v)$ para todo $v \in V$, y toma tiempo $O((N + M) \log N)$ y espacio (sin considerar la pila de ejecución para la recurrencia) $O(N)$.

Demostración

La corrección es el Teorema 4.8.2.

El llamado a `bmake` toma tiempo $O(N)$. La recurrencia hace $N + 1$ llamados, pues cada uno extrae un vértice y hay N de ellos. Cada llamado ejecuta una operación *extract*, en tiempo $O(\log N)$ por el Teorema 4.7.14; en total, $O(N \log N)$.

El ciclo de relajación recorre $G[u]$ una vez por cada vértice u y hace una operación *decrease* por cada pareja. Como cada arco aparece una sola vez entre todas las listas de adyacencia, la cantidad total de operaciones *decrease* es M y el tiempo que consumen es $O(M \log N)$. Recorrer las listas cuesta, además, $O(N + M)$.

Sumando, el tiempo total es $O((N + M) \log N)$. El espacio adicional es el de los tres arreglos del *bheap* y el del arreglo $D[0..N)$, que es $O(N)$.

La cota es la misma del Teorema 4.7.16, lo cual no sorprende: los dos algoritmos hacen la misma cantidad de operaciones sobre la misma estructura de datos y difieren únicamente en la aritmética de una línea. Es un buen cierre para el capítulo, porque muestra hasta qué punto la eficiencia de un algoritmo voraz depende de las decisiones de representación y de estructuras de datos, y no del criterio de selección que tanto trabajo costó demostrar.

Ejercicios

1. Justifique las siguientes afirmaciones hechas en esta sección:
 - a) $\delta(u, u) = 0$ para todo vértice u . ¿Dónde se usa que los pesos son no negativos?
 - b) En la demostración del Teorema 4.8.1, el camino P efectivamente tiene un primer arco (x, y) con $x \in S$ y $y \notin S$.
 - c) Los vértices inalcanzables desde s se extraen con llave $+\infty$.
2. Considere el grafo dirigido con vértices s , a y b y arcos (s, a) de peso 2, (s, b) de peso 3 y (b, a) de peso -2 .
 - a) Verifique que no tiene ciclos y que $\delta(s, a) = 1$.
 - b) Ejecute el Algoritmo 4.8.1 sobre él y explique qué responde.
 - c) Identifique el paso exacto de la demostración del Teorema 4.8.1 que falla, según la Nota 4.8.4.
3. 🌈 La Nota 4.8.2 distingue entre los arcos de peso negativo, que este algoritmo no tolera, y los ciclos de peso negativo, que hacen que el problema mismo deje de tener sentido.
 - a) Modifique el Algoritmo 4.8.1 para que verifique, antes de empezar, que todos los pesos son no negativos y reporte un error cuando no lo sean. ¿Cuánto cuesta la verificación?
 - b) Demuestre que si G no tiene ciclos de peso negativo alcanzables desde s , entonces para todo vértice v alcanzable hay un camino de peso $\delta(s, v)$ con a lo sumo $N - 1$ arcos.
 - c) Con base en lo anterior, diseñe un algoritmo que calcule $\delta(s, v)$ para todo v admitiendo pesos negativos y que reporte un error cuando haya un ciclo de peso negativo alcanzable desde s . *Ayuda:* relaje todos los arcos $N - 1$ veces; si una relajación más todavía mejora alguna distancia, hay un ciclo negativo.
 - d) Determine su complejidad temporal y compárela con la del Teorema 4.8.4. ¿Qué se paga por admitir pesos negativos?

4. La Nota 4.8.3 sostiene que Prim y Dijkstra difieren únicamente en qué mide la llave.
 - a) Verifique la afirmación comparando el Algoritmo 4.7.8 con el Algoritmo 4.8.1.
 - b) Explique, en términos de las dos demostraciones de optimización local, por qué Prim tolera pesos negativos y Dijkstra no.
 - c) Construya un grafo no dirigido con pesos no negativos en el cual el árbol de cubrimiento mínimo y el árbol de caminos más cortos desde un vértice sean distintos.
5. El Algoritmo 4.8.1 calcula las distancias pero no los caminos que las alcanzan.
 - a) Modifíquelo para que retorne además un arreglo $P[0..N)$ con el predecesor de cada vértice en un camino más corto desde s , como hace el Algoritmo 4.7.8.
 - b) Escriba una función que, dado ese arreglo y un vértice v , reconstruya el camino.
 - c) Demuestre que los arcos $(P[v], v)$, para v alcanzable y distinto de s , forman un árbol con raíz s .
6. Diseñe una versión iterativa del Algoritmo 4.8.1. Formule los invariantes del ciclo y verifique que son las suposiciones Z_0 y Z_1 de la función objetivo.
7. Suponga que solo interesa $\delta(s, t)$ para un vértice t fijo.
 - a) Modifique el Algoritmo 4.8.1 para que se detenga tan pronto sea posible, y justifique que la respuesta es correcta.
 - b) ¿Cambia la complejidad asintótica en el peor caso?
 - c) Mida, sobre grafos aleatorios, cuántas extracciones ahorra.
8. Suponga que todos los arcos tienen peso 1.
 - a) Demuestre que en ese caso el primero de los tres criterios de esta sección sí funciona.
 - b) Diseñe un algoritmo que resuelva el problema en tiempo $O(N+M)$, sin usar el ADT *minheap*. *Ayuda:* las llaves que se van generando nunca decrecen y difieren en a lo sumo 1.
9. Considere la variante en la cual, entre todos los caminos más cortos de s a v , se pide uno con la menor cantidad de arcos.
 - a) Proponga una manera de resolverla modificando el criterio de selección.
 - b) Enuncie y demuestre el teorema de optimización local correspondiente.
10. Un vértice puede ser extraído con llave $+\infty$ únicamente cuando es inalcanzable desde s .
 - a) Demuéstrelo.
 - b) Modifique el Algoritmo 4.8.1 para que se detenga en cuanto extraiga uno así, y explique por qué es correcto.

11. 🌈 Considere la variante del problema en la cual el peso de un camino no es la suma de los pesos de sus arcos sino el máximo, y se pide minimizarlo; es decir, se busca el camino cuyo arco más pesado sea lo más liviano posible.
- Adapte el Teorema 4.8.1 y demuéstrela.
 - ¿Hace falta suponer que los pesos son no negativos? Justifique con cuidado.
 - Modifique el Algoritmo 4.8.1 para que la resuelva.
 - Relacione la respuesta con el árbol de cubrimiento mínimo de la Sección 4.7.
-

Notas del capítulo y referencias

El planteamiento de los algoritmos voraces en este capítulo es cercano al de T. Cormen et al. [CLRS22] y al de J. Kleinberg y E. Tardos [KET06]. De estos últimos proviene el nombre del tercero de los patrones de demostración de la Sección 4.3: lo que aquí se llama *el voraz va adelante* aparece allá como *greedy stays ahead*, y es también en ese texto donde el argumento de intercambio se presenta como una técnica con nombre propio y no como un truco ocasional. J. Erickson [Eri19] insiste, más que los anteriores, en la fragilidad de los criterios voraces y en la disciplina de refutarlos antes de intentar demostrarlos; las tres referencias son fuentes abundantes de ejercicios. El Ejercicio 14 de la Sección 4.2 es una versión reducida de [Eri19, Problema 4.1].

Los algoritmos de este capítulo son considerablemente más antiguos que la idea de estudiarlos como una familia, tal y como lo anunció la Nota 4.0.3. El primer algoritmo conocido para el árbol de cubrimiento mínimo es el de O. Borůvka [Bor26], publicado en checo en 1926 y motivado por el diseño de una red eléctrica en Moravia. Le siguieron el de V. Jarník [Jar30] —que R. Prim [Pri57] redescubrió veintisiete años después, razón por la cual algunos autores lo llaman algoritmo de Jarník-Prim— y el de J. Kruskal [Kru56]. El algoritmo de caminos más cortos de la Sección 4.8 apareció en un artículo de tres páginas de E. Dijkstra [Dij59], que además contiene un segundo algoritmo para el árbol de cubrimiento mínimo. R. Graham y P. Hell [GH85] escribieron una historia detallada del problema del árbol de cubrimiento mínimo que vale la pena leer, entre otras cosas porque muestra cuántas veces se redescubrió lo mismo.

La explicación de por qué tantos criterios distintos funcionan para el árbol de cubrimiento mínimo, anunciada en la Nota 4.7.7 y en la Nota 4.3.1, está en la teoría de matroides. J. Edmonds [Edm71] demostró que la estrategia voraz es óptima sobre un sistema de independencia si, y solo si, ese sistema es un matroide, y los bosques de un grafo forman uno. J. Oxley [Oxl11] es la referencia estándar sobre la teoría, y B. Korte y J. Vygen [KV18] la presentan desde el punto de vista de la optimización combinatoria, que es el más cercano a los intereses de este

texto. Conviene, eso sí, no sobreestimar el alcance del resultado: de los problemas de este capítulo, únicamente el árbol de cubrimiento mínimo cae dentro de esa caracterización.

La estructura de conjuntos disjuntos de la Sección 4.7 se puede mejorar con la técnica de compresión de caminos que menciona la Nota 4.7.9. R. Tarjan [Tar75] demostró que, combinada con la unión por tamaño, el costo amortizado de cada operación es $O(\alpha(N))$, donde α es la inversa de la función de Ackermann y crece tan despacio que vale a lo sumo 4 para cualquier N que quepa en el universo observable. La demostración es difícil y se encuentra en [CLRS22, Capítulo 19].

El ADT *minheap* de la Definición 4.7.5 admite implementaciones muy distintas del montón binario, que se debe a J. Williams [Wil64]. M. Fredman y R. Tarjan [FT87] introdujeron los *montones de Fibonacci*, en los cuales *decrease* toma tiempo amortizado $O(1)$ y *extract* toma tiempo amortizado $O(\log N)$; sustituyéndolos en los algoritmos de Prim y de Dijkstra, sin cambiarles una línea, el tiempo baja a $O(M + N \log N)$. La constante oculta es grande y en la práctica el montón binario suele ganar, pero el ejercicio de sustitución es instructivo: es la separación entre ADT y estructura de datos rindiendo exactamente lo que la Nota 4.7.8 prometió.

En una dirección distinta, el ADT también se puede implementar sobre un *árbol de segmentos*: un árbol binario completo cuyas hojas son las llaves y en el cual cada nodo interno guarda el mínimo de su subárbol. Las dos operaciones toman tiempo $O(\log N)$ y, a diferencia del montón, se puede consultar el mínimo sobre cualquier subrango, lo cual resulta útil en otros contextos. Su pariente más económico es el *árbol binario indexado* (en inglés, *binary indexed tree*), también llamado *árbol de Fenwick* por P. Fenwick [Fen94], que ocupa un solo arreglo del tamaño del universo y cuyas operaciones son unas pocas líneas de aritmética sobre los bits del índice. Antes de emprender esa última implementación conviene advertir una sutileza. El árbol de Fenwick para mínimos soporta con naturalidad las actualizaciones *monótonas* –bajar una llave, que es exactamente lo que hace *decrease*– pero no las que aumentan un valor, y *extract* necesita subir a $+\infty$ la llave del elemento que remueve. Conseguirlo cuesta un factor logarítmico adicional o exige recurrir al árbol de segmentos, que sí admite actualizaciones arbitrarias. Es un buen ejercicio de diseño, y un buen recordatorio de que una estructura de datos implementa un tipo abstracto solo cuando ofrece *todas* sus operaciones dentro de las cotas exigidas.

La Nota 4.8.1 menciona las otras dos variantes del problema de caminos más cortos. La de todas las parejas se resuelve con el algoritmo de R. Floyd [Flo62], que es un ejercicio clásico de las técnicas del Capítulo 3. Para admitir pesos negativos sin ciclos negativos, como lo pide el Ejercicio 3, está el algoritmo que suele atribuirse a R. Bellman [Bel58] y a L. Ford, que renuncia a la estrategia voraz y paga por ello un factor N . Para la variante de pareja única, cuando se dispone de una estimación de la distancia que falta, el algoritmo A^* de P. Hart, N. Nilsson y B. Raphael [HNR68]

explora una porción mucho menor del grafo; es, en esencia, el algoritmo de la Sección 4.8 con las llaves corregidas por esa estimación.

Quedaron por fuera de este capítulo varios algoritmos voraces importantes. El de D. Huffman [**Huf52**], que construye códigos de longitud variable de redundancia mínima, es quizás el ejemplo más citado de la técnica y su demostración de optimización local es un argumento de intercambio particularmente elegante. También quedaron por fuera los algoritmos voraces que no resuelven un problema sino que lo *aproximan*, como el del cubrimiento de conjuntos; el Ejercicio 9 de la Sección 4.6 es una muestra de ese enfoque, y V. Vazirani [**Vaz01**] es una buena puerta de entrada al tema.

Reintento

Las tres técnicas presentadas hasta ahora responden, cada una a su manera, a una misma pregunta: ¿cómo evitar recorrer por completo el espacio de soluciones de un problema? Dividir, conquistar y combinar lo parte en subespacios independientes; la programación dinámica lo comprime aprovechando que muchas subinstancias se repiten; los algoritmos voraces lo colapsan a un único camino, aquel que dicta el criterio de selección. Hay problemas, sin embargo, para los cuales ninguna de las tres respuestas está disponible. No hay una partición natural que independice los subproblemas, la función objetivo no exhibe subestructura óptima explotable y todo criterio voraz que se proponga admite un contraejemplo. Para ellos queda una última alternativa, que es la que desarrolla este capítulo: recorrer el espacio de soluciones con una exploración sistemática.

La *exploración sistemática* se diseña con base en dos principios fundamentales. El primero es *construir*: las soluciones no se examinan una por una, ya terminadas, sino que se arman por partes, tomando una decisión a la vez. El segundo es *abandonar temprano*: apenas se sabe que lo construido hasta el momento no puede completarse hasta una solución, se deshace la última decisión y se intenta con otra. A esta forma de proceder se le llama *reintento*.

Nota 5.0.1: Sobre el nombre

En la literatura en inglés esta técnica se conoce como *backtracking*. En castellano circulan también los nombres *vuelta atrás* y *retroceso*. En este texto se prefiere *reintento* porque nombra la operación que caracteriza la técnica –intentar de nuevo con una decisión distinta– y no únicamente el desplazamiento que la hace posible. El lector debe reconocer, sin embargo, los otros tres nombres al consultar otras fuentes.

El reintento es la más elemental de las cuatro técnicas de diseño de este texto. También la que peor fama puede tener dado que, en el peor caso, un algoritmo de reintento puede llegar a examinar una cantidad exponencial de soluciones parciales. Esa fama es merecida, pero no necesariamente determinante. Por un lado, para muchos problemas de interés práctico no se conoce –ni se espera conocer, según lo que se estudiará en el Capítulo 7– nada esencialmente mejor. Por otro, la diferencia entre un algoritmo de reintento ingenuo y uno bien diseñado es, en la práctica, la diferencia entre un programa que no termina nunca y uno que responde en milisegundos, aun cuando ambos tengan la misma complejidad asintótica. El diseño de esa diferencia –la *poda*– es el asunto central del capítulo.

Nota 5.0.2: Lehmer, Walker, Golomb y Baumert

El procedimiento que este capítulo describe es mucho más antiguo que su nombre: recorrer sistemáticamente las posibilidades y retroceder ante un callejón sin salida es una idea que aparece en la solución de rompecabezas y laberintos desde el siglo XIX. Se le atribuye al matemático Derrick H. Lehmer haber acuñado, en la década de 1950, el término *backtrack* para nombrarla. En 1960, Robert J. Walker describió la técnica como un método general de enumeración para una clase de problemas combinatorios y, en 1965, Solomon W. Golomb y Leonard D. Baumert publicaron *Backtrack programming*, el artículo que la estableció como una técnica de diseño con derecho propio y que introdujo varias de las ideas de poda que se estudian en este capítulo (ver notas y referencias al final del capítulo).

Este capítulo presenta las ideas principales que respaldan el reintento, una metodología para su uso y varios ejemplos, incluyendo su extensión a problemas de optimización mediante ramificación y poda.

5.1. Ideas clave

Considere un arreglo $A[0..N]$ de números y el objetivo de determinar cuáles de sus subconjuntos de índices suman exactamente un número dado X . Una solución es, entonces, una decisión por cada índice: llevarlo o no llevarlo. Con $N = 3$ hay ocho decisiones posibles y todas caben en una página; con $N = 100$ hay más de 10^{30} y posiblemente no sea buena idea pensar en escribirlas en algún lado. La técnica que se estudia en este capítulo no elimina ese crecimiento, pero sí permite, en muchos casos, no tener que explorar el espacio de búsqueda completamente.

La primera observación es que las ocho soluciones del caso $N = 3$ no son ocho objetos sin relación entre sí: comparten decisiones iniciales. Las cuatro soluciones que llevan el índice 0 coinciden en su primera decisión, y las dos que llevan los

índices 0 y 1 coinciden en las dos primeras. Esta estructura compartida es lo que permite organizar el recorrido como un árbol y, sobre todo, lo que permite descartar muchas soluciones a la vez: al descartar una decisión inicial se descartan de un golpe todas las soluciones que la comparten.

5.1.1. Soluciones parciales. Para hacer precisas estas ideas conviene fijar qué tipo de problema se está resolviendo.

Definición 5.1.1: Problema de búsqueda combinatoria

Un *problema de búsqueda combinatoria* está dado por una longitud $K \geq 0$, una secuencia $C[0..K)$ de conjuntos finitos no vacíos, llamados *conjuntos de candidatos*, y un predicado *sol* sobre las tuplas de $C[0] \times \cdots \times C[K-1]$. El conjunto de *soluciones* del problema es

$$\{s \in C[0] \times \cdots \times C[K-1] \mid \text{sol}(s)\}.$$

Es decir, una solución es una tupla que asigna a cada una de K decisiones un candidato entre los disponibles para esa decisión, de manera que se satisfaga el predicado *sol*. En el problema con el que se abrió la sección, se tiene $K = N$, cada $C[i]$ es el conjunto $\{0, 1\}$ –que corresponde a la decisión de no llevar o llevar el índice i – y *sol*(s) es cierto exactamente cuando la suma de los elementos escogidos por s es X .

Nota 5.1.1: Sobre la longitud fija

La Definición 5.1.1 exige que todas las soluciones tengan la misma longitud K . Esto parece restrictivo –¿cómo representar, por ejemplo, los subconjuntos de $A[0..N)$, que tienen tamaños distintos?–, pero no lo es: basta con tomar una decisión por cada índice, incluyendo la decisión de no usarlo, como se acaba de hacer. Este es un patrón recurrente en el diseño de algoritmos de reintento y vale la pena enunciarlo. Los objetos de tamaño variable se representan como secuencias de decisiones de longitud fija. La representación alternativa, en la cual las soluciones tienen longitudes distintas, es a veces más natural y se explora en el Ejercicio 4.

Las tuplas incompletas, que registran algunas decisiones pero no todas, son el objeto central de la técnica.

Definición 5.1.2: Solución parcial

Sea un problema de búsqueda combinatoria con longitud K y candidatos $C[0..K)$. Una *solución parcial* de longitud k , con $0 \leq k \leq K$, es una tupla $s = \langle s_0, s_1, \dots, s_{k-1} \rangle$ tal que $s_i \in C[i]$ para $0 \leq i < k$. Se escribe $|s| = k$. La solución parcial de longitud 0 se denota $\langle \rangle$ y se llama *vacía*. Una solución parcial de longitud K se llama *completa*.

Si $|s| = k < K$ y $c \in C[k]$, la *extensión inmediata* de s con c es la solución parcial $s \cdot c = \langle s_0, \dots, s_{k-1}, c \rangle$, de longitud $k + 1$. Se dice que t *extiende* a s , y se escribe $s \preceq t$, si s es un prefijo de t .

Note que toda solución es una solución parcial completa, pero no toda solución parcial completa es una solución: para serlo debe además satisfacer *sol*.

Definición 5.1.3: Árbol de reintento

El *árbol de reintento* de un problema de búsqueda combinatoria es el árbol cuyos nodos son las soluciones parciales del problema, cuya raíz es $\langle \rangle$, y en el cual los hijos del nodo s , con $|s| = k < K$, son exactamente las extensiones inmediatas $s \cdot c$ para $c \in C[k]$. Los nodos de longitud K son las hojas del árbol.

El árbol de reintento es a este capítulo lo que el diagrama de necesidades fue al Capítulo 3: no es una estructura de datos que el algoritmo construya en memoria, sino un objeto de diseño que se dibuja en el papel para razonar acerca del algoritmo. De hecho, la característica que define a los algoritmos de esta sección es precisamente que el árbol nunca se construye: se recorre en profundidad y en todo momento el algoritmo tiene en memoria un único camino de la raíz al nodo que está visitando.

5.1.2. Poda. Recorrer el árbol de reintento completo equivale a examinar todas las soluciones parciales completas, es decir, a fuerza bruta. Lo que hace útil a la técnica es poder decidir, en un nodo interno, que ninguna de las hojas de su subárbol es una solución y no seguir descendiendo.

Definición 5.1.4: Predicado de factibilidad

Un predicado *feasible* sobre soluciones parciales es un *predicado de factibilidad* para el problema si y solo si para toda solución t y toda solución parcial s se

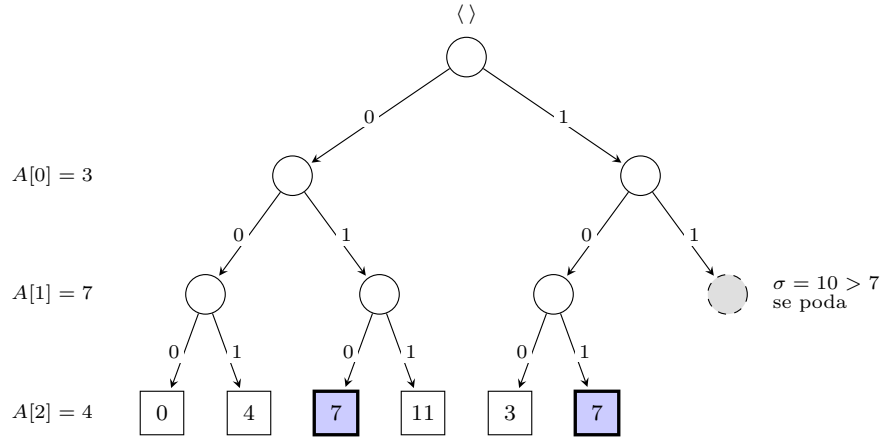


Figura 1. Árbol de reintento para las decisiones sobre $A[0..3)$, con $A = [3, 7, 4]$ y $X = 7$. Cada nivel corresponde a un índice y cada arco a la decisión de no llevarlo (0) o llevarlo (1). Las hojas están etiquetadas con la suma correspondiente y las soluciones aparecen resaltadas.

cumple

$$s \preceq t \implies \text{feasible}(s).$$

Un predicado de factibilidad es, entonces, una condición necesaria para poder llegar a una solución. Por contrapositiva, la definición dice lo que interesa en la práctica para realizar una poda: si $\text{feasible}(s)$ es falso, entonces ninguna extensión de s es una solución y, por tanto, todo el subárbol con raíz en s puede descartarse sin miedo a perder soluciones. A esto último se le llama *podar* el nodo s .

Nota 5.1.2: La poda debe ser conservadora

La Definición 5.1.4 impone una obligación en un solo sentido y esa asimetría es la fuente del error más común al diseñar algoritmos de reintento. El predicado *feasible* debe ser cierto en los prefijos de las soluciones; en cambio, no está obligado a ser falso en ningún otro lugar. Es decir, se le permite equivocarse por exceso de optimismo —dejar pasar soluciones parciales que no llevan a nada, con el único costo de trabajo desperdiciado— pero nunca por exceso de pesimismo, pues podar un nodo que sí tenía soluciones en su subárbol las pierde para siempre.

Dos predicados de factibilidad siempre existen. El primero es $\text{feasible}(s) = \text{true}$, que no poda nada y produce la fuerza bruta. El segundo es $\text{feasible}(s) = \text{“alguna extensión de } s \text{ es solución”}$, que poda todo lo podable pero cuya

evaluación es, en general, tan difícil como el problema original. Todo diseño de un algoritmo de reintento es la búsqueda de un punto intermedio entre estos dos extremos: un predicado que pide mucho y se evalúe barato.

Ejemplo 5.1.1

Considere el problema con el que se abrió la sección, con $A = [3, 7, 4]$ y $X = 7$, y suponga que todos los elementos de A son positivos. Sea $\sigma(s)$ la suma de los elementos escogidos por la solución parcial s . El predicado

$$\text{feasible}(s) : \sigma(s) \leq X$$

es un predicado de factibilidad. En efecto, si $s \preceq t$ y t es solución, entonces $\sigma(t) = X$. Además, como los elementos son positivos y s es un prefijo de t , se tiene $\sigma(s) \leq \sigma(t) = X$.

Con este predicado, el nodo $\langle 1, 1 \rangle$ del árbol de la Figura 1 se poda, pues $\sigma(\langle 1, 1 \rangle) = 3 + 7 = 10 > 7$, y con él se descartan de una vez sus dos hojas. Note que la hipótesis de que los elementos son positivos es indispensable: con $A = [3, 7, -3]$ el predicado ya no es de factibilidad, pues $\langle 1, 1 \rangle$ sí se extiende a la solución $\langle 1, 1, 1 \rangle$. En la Sección 5.5 se estudian predicados de factibilidad que no dependen de esta hipótesis.

5.1.3. El esquema general. Con el vocabulario anterior, el esquema de un algoritmo de reintento se enuncia en pocas líneas. La solución parcial se mantiene en un arreglo s que funciona como pila: se agrega un candidato al final antes de descender y se retira al regresar.

Algoritmo 5.1.1: Esquema general de un algoritmo de reintento

```

1 def backtrack(s):
2     if len(s)==K:
3         if sol(s): report(s)
4     else:
5         for c in C[len(s)]:
6             s.append(c)                # choose
7             if feasible(s): backtrack(s) # explore
8             s.pop()                    # undo

```

El cuerpo del ciclo tiene tres pasos, y esos tres pasos son toda la técnica. Se *elige* un candidato para la decisión siguiente, extendiendo la solución parcial. Se *explora* el subárbol correspondiente, pero únicamente si la extensión sigue siendo

factible. Y se *deshace* la elección, dejando a s como estaba, para poder intentar con el candidato siguiente. La invocación inicial es `backtrack([])`.

Los algoritmos de este capítulo son recurrentes y no iterativos, de modo que el método de diseño por invariantes de ciclo del Capítulo 2 no aplica directamente. Su lugar lo ocupa la especificación del llamado recurrente: lo que debe ser cierto al invocarlo y lo que debe ser cierto al retornar de él. Para el Algoritmo 5.1.1 se propone la siguiente.

Pre : s es una solución parcial.

*Post*₀ : se reportan exactamente las soluciones t tales que $s \preceq t$.

*Post*₁ : el estado es idéntico al que tenía inmediatamente antes de la invocación.

La precondition es deliberadamente débil: no exige *feasible*(s). Esto permite que la invocación inicial `backtrack([])` la satisfaga trivialmente, sin tener que suponer nada acerca de si el problema tiene o no soluciones. La poscondición *Post*₀ es la corrección propiamente dicha, y *Post*₁ es la que hace que el ciclo de la línea 5 pueda intentar con el candidato siguiente sin arrastrar el efecto de los anteriores. Con esta especificación, la corrección del esquema se demuestra por inducción sobre la cantidad de decisiones que faltan por tomar.

Teorema 5.1.1

Sea *feasible* un predicado de factibilidad para el problema. Entonces el Algoritmo 5.1.1 satisface *Post*₀ y *Post*₁ bajo la precondition *Pre*. En particular, el llamado `backtrack([])` reporta exactamente las soluciones del problema, cada una exactamente una vez.

Demostración

Se procede por inducción sobre $K - |s|$, la cantidad de decisiones que faltan por tomar.

- Caso $K - |s| = 0$: entonces $|s| = K$ y s es completa. La única solución parcial que extiende a s es s misma, de modo que las soluciones t con $s \preceq t$ son, o bien ninguna, o bien únicamente s , según si *sol*(s) es falso o cierto. Esto es exactamente lo que hace la línea 3, luego se cumple *Post*₀. Como el algoritmo retorna sin modificar s , también se cumple *Post*₁.
- Caso $K - |s| > 0$: entonces $|s| = k < K$. Sea t una solución con $s \preceq t$ y sea $c = t_k$, el candidato que t escoge para la decisión k ; note que $c \in C[k]$ y que $s \cdot c \preceq t$. Como *feasible* es un predicado de factibilidad y $s \cdot c$ es un prefijo

de la solución t , la Definición 5.1.4 garantiza que $feasible(s \cdot c)$ es cierto. Luego la línea 7 sí invoca $backtrack(s \cdot c)$ y, por hipótesis inductiva, ese llamado reporta a t . Es decir, toda solución que extiende a s es reportada.

Recíprocamente, todo lo que se reporta durante la ejecución lo reporta alguno de los llamados de la línea 7 y, por hipótesis inductiva, es una solución que extiende a $s \cdot c$ para algún $c \in C[k]$; en particular, es una solución que extiende a s . Además, las soluciones que extienden a $s \cdot c$ y las que extienden a $s \cdot c'$ son disjuntas cuando $c \neq c'$, pues difieren en la posición k . Como el ciclo de la línea 5 recorre cada candidato exactamente una vez, ninguna solución se reporta dos veces. Se cumple, entonces, $Post_0$.

Finalmente, para $Post_1$ basta observar que en cada iteración el comando `s.append(c)` de la línea 6 y el `s.pop()` de la línea 8 se corresponden exactamente, y que la invocación intermedia restaura el estado por hipótesis inductiva.

Note que en la demostración anterior la hipótesis de que $feasible$ es un predicado de factibilidad se usa en un solo lugar y se usa en una sola dirección: para garantizar que el algoritmo no deja de descender por un camino que llevaba a una solución. Esta es la razón formal de la asimetría señalada en la Nota 5.1.2.

La poscondición $Post_1$ merece atención aparte, porque en los algoritmos concretos de las secciones siguientes deja de ser evidente.

Nota 5.1.3: El invariante de restauración

La poscondición $Post_1$ se enuncia sobre *todo* el estado que el algoritmo modifique: no solamente la solución parcial s , sino también cualquier estructura auxiliar que se agregue para evaluar $feasible$ eficientemente.

Cuando en las secciones siguientes se agreguen estructuras auxiliares –por ejemplo, los arreglos de diagonales ocupadas en el problema de las N reinas o la suma acumulada en el de la suma exacta– cada una de ellas deberá actualizarse al elegir y restaurarse al deshacer. Omitir una de esas restauraciones es la falla más frecuente en las implementaciones de reintento, y tiene la característica desagradable de producir respuestas incorrectas sin producir errores de ejecución.

5.1.4. Enumerar, contar, optimizar. El Algoritmo 5.1.1 *enumera* las soluciones. Las otras dos tareas que se le piden usualmente a un algoritmo de reintento son pequeñas variaciones sobre el mismo esquema:

Existencia y búsqueda: determinar si hay alguna solución o encontrar una en particular. Basta con detener el recorrido al encontrar la primera, lo cual se

implementa haciendo que **backtrack** retorne un valor Booleano que se propaga hacia arriba.

Conteo: determinar cuántas soluciones hay. Se acumula, por ejemplo, un contador global en lugar de reportar.

Optimización: encontrar la solución que maximiza o minimiza una función objetivo. Este caso admite una forma de poda adicional más poderosa que la de la Definición 5.1.4, porque aprovecha el valor de la mejor solución hallada hasta el momento. Es el tema de la Sección 5.7.

5.1.5. Análisis. El análisis de un algoritmo de reintento se reduce a contar los nodos que efectivamente visita.

Teorema 5.1.2

Sea V el conjunto de nodos del árbol de reintento que el Algoritmo 5.1.1 visita, y suponga que evaluar *feasible* en un nodo toma tiempo $O(f)$ y que $|C[k]| \leq c$ para $0 \leq k < K$. Entonces el algoritmo toma tiempo $O(|V| \cdot c \cdot f)$ y espacio $O(K)$, sin contar el espacio necesario para reportar las soluciones.

Demostración

Se propone como ejercicio al lector.

El factor $|V|$ es el que domina y el que ningún análisis asintótico logra acotar con precisión: en el peor caso, cuando la poda no descarta nada, $|V| = \Theta(c^K)$. Esta cota es genuina y no un artefacto del análisis, pues para cualquier predicado de factibilidad se pueden construir instancias en las cuales no poda nada.

Nota 5.1.4: Sobre el peor caso y la práctica

La observación anterior tiene una consecuencia incómoda: dos algoritmos de reintento para un mismo problema, uno con una poda excelente y otro sin poda alguna, tienen la *misma* complejidad asintótica de peor caso. La herramienta de análisis desarrollada en el Capítulo 1 es, en este capítulo, ciega a la única diferencia que importa.

Esto no invalida el análisis asintótico ni justifica abandonarlo; señala únicamente que en este capítulo debe complementarse. La forma usual de hacerlo es reportar el tamaño $|V|$ del árbol efectivamente recorrido para instancias concretas, que es una medida empírica y no asintótica. Varios de los ejercicios

de este capítulo piden instrumentar los algoritmos con un contador de nodos visitados y comparar; se recomienda al lector no omitirlos, pues son la única manera de percibir el efecto de la poda.

Para cerrar la sección, la siguiente tabla ubica el reintento entre las técnicas de diseño estudiadas, según lo que cada una hace con el espacio de soluciones.

Técnica	Estrategia sobre el espacio de soluciones
Dividir, conquistar y combinar	Partirlo en subespacios independientes
Programación dinámica	Comprimirlo, reusando subinstancias repetidas
Algoritmos voraces	Colapsarlo a un único camino
Reintento	Recorrerlo en profundidad, podando

Ejercicios

- Demuestre el Teorema 5.1.2.
- Demuestre que el árbol de reintento de un problema con longitud K y conjuntos de candidatos de tamaño $c \geq 2$ tiene exactamente $\frac{c^{K+1}-1}{c-1}$ nodos. ¿Cuántos de ellos son hojas? ¿Qué fracción del árbol son hojas cuando $c = 2$ y K es grande?
- Sea *feasible* un predicado de factibilidad para un problema dado.
 - Demuestre que el predicado $feasible'(s) : feasible(s) \wedge true$ también lo es.
 - Sean $feasible_1$ y $feasible_2$ predicados de factibilidad. ¿Lo es $feasible_1 \wedge feasible_2$? ¿Y $feasible_1 \vee feasible_2$? Demuestre o dé un contraejemplo en cada caso.
 - 🌈 Concluya que los predicados de factibilidad para un problema dado forman un retículo bajo el orden $\Rightarrow$ y describa sus elementos extremos.
- La Nota 5.1.1 indica que los objetos de tamaño variable se pueden representar como secuencias de decisiones de longitud fija. Considere la representación alternativa en la cual un subconjunto de índices de $A[0..N)$ se representa directamente como la secuencia creciente de sus elementos, de modo que las soluciones parciales tienen longitudes distintas.
 - Adapte la Definición 5.1.3 a esta representación. ¿Cuáles son ahora los hijos de un nodo? ¿Cuáles son las hojas?
 - Adapte el Algoritmo 5.1.1.
 - Compare la cantidad de nodos de los dos árboles. ¿Cuál representación prefiere y por qué?
- Un predicado de factibilidad *feasible* se dice *perfecto* si para toda solución parcial s se cumple que *feasible*(s) es cierto si y solo si alguna extensión de s es solución.

- a) Demuestre que si se dispone de un predicado perfecto evaluable en tiempo $O(f)$, entonces el Algoritmo 5.1.1 encuentra una solución (o determina que no existe) en tiempo $O(K \cdot c \cdot f)$.
 - b) Explique por qué el numeral anterior *no* demuestra que todo problema de búsqueda combinatoria se resuelva en tiempo polinomial.
6. Considere el problema de generar todas las cadenas de longitud N sobre el alfabeto $\{a, b\}$ que no contienen dos a consecutivas.
- a) Especifique el problema como un problema de búsqueda combinatoria en el sentido de la Definición 5.1.1.
 - b) Proponga un predicado de factibilidad y demuestre que lo es.
 - c) Implemente el algoritmo en el lenguaje de programación Python.
 - d) Instrumente su implementación con un contador de nodos visitados. Compare, para $N = 20$, la cantidad de nodos visitados con y sin poda. ¿Qué sucesión reconoce en la cantidad de soluciones?
7. El Algoritmo 5.1.1 recorre el árbol de reintento en profundidad. Discuta qué ocurriría si se recorriera en amplitud. ¿El algoritmo seguiría siendo correcto? ¿Qué pasaría con el espacio utilizado? ¿Se podría seguir hablando de *deshacer*?
8. 🌈 Escriba una versión iterativa del Algoritmo 5.1.1 que use una pila explícita, proponga los invariantes del ciclo en el estilo del Capítulo 2 y demuestre que el diseño es correcto.

5.2. Generación de objetos combinatorios

Antes de aplicar el reintento a problemas de decisión y de optimización, conviene usarlo para la tarea más simple que admite: generar, uno por uno, todos los objetos de una familia combinatoria. Estos generadores son útiles por sí mismos —para ensayar hipótesis, para verificar por fuerza bruta algoritmos más sofisticados, para construir casos de prueba— y, sobre todo, sirven para fijar el esquema del capítulo en su forma más despojada, sin las complicaciones que introduce una poda interesante.

5.2.1. Subconjuntos. El problema de generación de subconjuntos se especifica a continuación.

Problema 5.2.1: Generación de subconjuntos

Entrada: Un arreglo $A[0..N)$.

Salida: Todos los subconjuntos de elementos de $A[0..N)$.

Siguiendo la Nota 5.1.1, un subconjunto se representa como una secuencia de N decisiones, una por cada índice del arreglo: la decisión i -ésima vale 1 si el elemento $A[i]$ se incluye y 0 si no. Es decir, $K = N$ y $C[i] = \{0, 1\}$ para $0 \leq i < N$. Toda solución parcial completa es una solución, de modo que $sol(s) = true$, y no hay nada que podar: se toma $feasible(s) = true$. El árbol de reintento correspondiente es el de la Figura 1, con N niveles en lugar de tres.

Con estas decisiones, el Algoritmo 5.1.1 se instancia directamente.

Algoritmo 5.2.1: Generación de subconjuntos

```

1 def subsets(A, s):
2     N = len(A)
3     if len(s)==N:
4         print([ A[i] for i in range(N) if s[i]==1 ])
5     else:
6         for c in [0, 1]:
7             s.append(c)
8             subsets(A, s)
9             s.pop()
```

La invocación inicial es `subsets(A, [])`. La línea 4 traduce la secuencia de decisiones al objeto que representa, que es el subconjunto de los elementos cuya decisión fue 1. Note que esa traducción pasa por referencia una extensión de `s` (con 0 o 1) y que luego se elimina, resultando en la versión inicial de `s`. De este modo, $Post_1$ se preserva.

Ejemplo 5.2.1

Considere la siguiente invocación:

```
1 subsets([4,6,5], [])
```

El resultado de esta ejecución es el siguiente:

```

1 []
2 [5]
3 [6]
4 [6, 5]
5 [4]
6 [4, 5]
7 [4, 6]
8 [4, 6, 5]
```

El orden en el cual aparecen los subconjuntos no es arbitrario: es el orden en que el recorrido en profundidad visita las hojas del árbol de reintento, con la decisión 0 explorada antes que la decisión 1. Leyendo las decisiones como dígitos binarios de derecha a izquierda, es el orden $0, 1, \dots, 7$.

Teorema 5.2.1

El llamado `subsets(A, [])`, con $|A| = N$:

1. Reporta exactamente los 2^N subconjuntos de elementos de $A[0..N)$, cada uno exactamente una vez.
2. Visita $2^{N+1} - 1$ nodos del árbol de reintento.
3. Toma tiempo $\Theta(N \cdot 2^N)$.
4. Usa espacio $O(N)$, sin contar el espacio de la salida.

Demostración

Se propone como ejercicio al lector.

El numeral 3 del Teorema 5.2.1 merece un comentario. El factor 2^N es inevitable, pues hay esa cantidad de objetos que reportar; el factor adicional N proviene de construir explícitamente cada subconjunto en la línea 4. Si lo único que se desea es *contar* los subconjuntos o acumular alguna función de ellos que pueda mantenerse incrementalmente, ese factor desaparece y el algoritmo toma tiempo $\Theta(2^N)$.

Nota 5.2.1: Subconjuntos y máscaras de bits

La Sección 3.9 presentó las máscaras de bits como representación de conjuntos. Con ellas, los subconjuntos de $A[0..N)$ se pueden generar sin recurrencia alguna, simplemente recorriendo los enteros de 0 a $2^N - 1$ e interpretando cada uno como el conjunto de índices de sus bits en 1:

```

1 def subsets_mask(A):
2     N = len(A)
3     for m in range(1<<N):
4         print([ A[i] for i in range(N) if (m>>i)&1 ])

```

Esta versión es más corta, más rápida por un factor constante y no consume pila. Cuando se desean todos los subconjuntos sin excepción, debería preferirse.

Su limitación aparece apenas se quiere podar. La generación por máscaras es inherentemente plana: los 2^N enteros se visitan uno por uno y no hay noción de subárbol que descartar, de modo que saber que un prefijo de decisiones no conduce a nada no ahorra ningún trabajo. El reintento, en cambio, paga la sobrecarga de la recurrencia a cambio de conservar la estructura de árbol, que es precisamente lo que la poda explota. Esta tensión –una representación eficiente que no se deja podar contra una menos eficiente que sí– reaparece en la Sección 5.5.

5.2.2. Permutaciones. El problema de generación de permutaciones se especifica a continuación.

Problema 5.2.2: Generación de permutaciones

Entrada: Un arreglo $A[0..N)$ de elementos distintos.

Salida: Todas las permutaciones de $A[0..N)$.

De nuevo hay que identificar qué es una decisión. La elección natural es que la decisión k -ésima determine qué índice de A ocupa la posición k de la permutación. Es decir, $K = N$ y $C[k] = \{0, 1, \dots, N - 1\}$ para todo k . Con esta representación no toda solución parcial completa es una permutación: la secuencia $\langle 0, 0 \rangle$ es una solución parcial legítima según la Definición 5.1.2, pero repite un índice. Entonces

$sol(s) :$ “los elementos de s son distintos par a par”,

y el problema, a diferencia del de los subconjuntos, sí admite poda.

Teorema 5.2.2

El predicado

$feasible(s) :$ “los elementos de s son distintos par a par”

es un predicado de factibilidad para el Problema 5.2.2. Más aún, $feasible$ es perfecto en el sentido del Ejercicio 5.

Demostración

Sean t una solución y s una solución parcial con $s \preceq t$. Como los elementos de t son distintos par a par y los de s son un subconjunto de ellos, los de s también lo son. Luego $feasible(s)$ es cierto. Además, $feasible$ es un predicado de factibilidad.

Para la perfección hay que demostrar, además, que si *feasible(s)* entonces alguna extensión de *s* es solución. Sea $|s| = k \leq N$ y suponga que los *k* elementos de *s* son distintos par a par. Entonces quedan exactamente $N - k$ índices sin usar y basta con ubicarlos, en cualquier orden, en las posiciones $k, k + 1, \dots, N - 1$. La secuencia completa resultante tiene sus *N* elementos distintos par a par y, por lo tanto, es una solución que extiende a *s*.

Evaluar *feasible* recorriendo *s* tomaría tiempo $O(N)$ en cada nodo. Como lo único que cambia entre un nodo y su hijo es un elemento, conviene mantener esa información en una estructura auxiliar: un arreglo *used*[0..*N*] de Booleanos tal que *used*[*i*] indica si el índice *i* ya aparece en la solución parcial. Con él, la factibilidad de extender *s* con el candidato *c* se decide en tiempo constante consultando *used*[*c*].

Esta es la primera vez en el capítulo que aparece estado auxiliar y con él la obligación anunciada en la Nota 5.1.3: el arreglo *used* debe actualizarse al elegir y restaurarse al deshacer, o la poscondición *Post*₁ deja de cumplirse.

Algoritmo 5.2.2: Generación de permutaciones

```

1 def permutations(A, s, used):
2     N = len(A)
3     if len(s)==N:
4         print([ A[i] for i in s ])
5     else:
6         for c in range(N):
7             if not used[c]:
8                 used[c] = True           # choose
9                 s.append(c)
10                permutations(A, s, used)
11                s.pop()                  # undo
12                used[c] = False

```

La invocación inicial es *permutations*(*A*, [], [*False*]**len*(*A*)). Las líneas 8–9 y 11–12 se corresponden en orden inverso, como el llenado y vaciado de una pila; escribirlas así, y no en el mismo orden, es una costumbre que ayuda a no olvidar ninguna restauración cuando el estado auxiliar crece.

Nota 5.2.2: Dónde se evalúa la factibilidad

El Algoritmo 5.1.1 consulta *feasible* después de extender la solución parcial, mientras que el Algoritmo 5.2.2 lo hace antes, en la línea 7. Las dos formas

son correctas y equivalentes. La segunda se prefiere cuando la factibilidad de $s \cdot c$ se puede decidir mirando únicamente el candidato c y el estado auxiliar, pues evita el trabajo de extender y deshacer una solución parcial que se va a descartar de inmediato. La primera es preferible cuando el predicado se expresa más naturalmente sobre la solución parcial ya extendida. En este capítulo se usan ambas, según convenga.

Teorema 5.2.3

El llamado `permutations(A, [], [False]*N)`, con $|A| = N$ y elementos distintos:

1. Reporta exactamente las $N!$ permutaciones de $A[0..N)$, cada una exactamente una vez.
2. Visita $\left(+k \mid 0 \leq k \leq N \frac{N!}{(N-k)!}\right)$ nodos del árbol de reintento.
3. Toma tiempo $\Theta(N \cdot N!)$.
4. Usa espacio $O(N)$, sin contar el espacio de la salida.

Demostración

Se propone como ejercicio al lector.

Vale la pena contrastar los dos generadores de esta sección. El de subconjuntos no poda nada y visita $2^{N+1} - 1$ nodos para producir 2^N objetos: cerca de la mitad de los nodos visitados son hojas útiles. El de permutaciones sí poda y lo hace con un predicado perfecto: por el Teorema 5.2.2, todo nodo que el algoritmo visita conduce a por lo menos una permutación, de modo que no hay un solo nodo de trabajo desperdiciado. Sin la poda, en cambio, el algoritmo recorrería las N^N secuencias de índices para quedarse con $N!$ de ellas; con $N = 10$ eso es la diferencia entre visitar diez mil millones de hojas y visitar tres millones y medio.

Un predicado de factibilidad perfecto es una situación excepcionalmente afortunada y no es la que se encuentra en los problemas de las secciones siguientes. Allí la poda descartará subárboles enormes, pero también dejará pasar soluciones parciales que no llevan a ninguna parte.

Ejercicios

1. Demuestre el Teorema 5.2.1.
 2. Demuestre el Teorema 5.2.3. Para el numeral 2, observe que la cantidad de nodos de longitud k es la cantidad de secuencias de k índices distintos tomados de N .
 3. Demuestre que $\left(+k \mid 0 \leq k \leq N \frac{N!}{(N-k)!} \right) < e \cdot N!$ y concluya que el Algoritmo 5.2.2 visita $\Theta(N!)$ nodos.
 4. Modifique el Algoritmo 5.2.1 para que reporte únicamente los subconjuntos de tamaño exactamente M , con $0 \leq M \leq N$.
 - a) Proponga un predicado de factibilidad no trivial y demuestre que lo es. Ayuda: se puede podar tanto por exceso como por defecto.
 - b) ¿Es perfecto el predicado que propuso?
 - c) Implemente el algoritmo e instrumentelo con un contador de nodos visitados. Compare, para $N = 20$ y $M = 3$, la cantidad de nodos visitados con y sin poda.
 5. El Algoritmo 5.2.2 supone que los elementos de $A[0..N)$ son distintos. Si no lo son, reporta permutaciones repetidas. Modifíquelo para que cada permutación distinta se reporte exactamente una vez, sin generar y luego descartar las repetidas. Ayuda: ordene A previamente.
 6. Considere el problema de generar todas las cadenas bien balanceadas de N paréntesis de apertura y N de cierre.
 - a) Especifique el problema como un problema de búsqueda combinatoria.
 - b) Proponga un predicado de factibilidad y demuestre que lo es.
 - c) Demuestre que su predicado es perfecto.
 - d) Implemente el algoritmo. ¿Cuántas soluciones hay para $N = 1, 2, 3, 4, 5$? Investigue qué sucesión es esa.
 7. 🌈 Un *código de Gray* de N bits es una enumeración de los 2^N subconjuntos de un conjunto de N elementos en la cual dos subconjuntos consecutivos difieren en exactamente un elemento. Diseñe un algoritmo recurrente que genere uno y demuestre que es correcto. ¿Es un algoritmo de reintento?
 8. Use el Algoritmo 5.2.2 para resolver por fuerza bruta el problema del agente viajero de la Sección 3.9. Compare experimentalmente su tiempo de ejecución con el de la solución por memorización de esa sección, para $N = 5, 8, 10, 12$. Explique lo que observa a la luz de las complejidades $\Theta(N \cdot N!)$ y $O(2^N N^2)$.
-

5.3. Una metodología

Con base en los ejemplos de la sección anterior, se propone una metodología para diseñar algoritmos de reintento. Es deliberadamente análoga a las metodologías 3.3.1 y 4.3.1. Como aquella, no debe entenderse como una receta infalible, sino como una lista de las preguntas que conviene responder y en qué orden.

Metodología 5.3.1: Reintento

Especificación: especificar el problema, indicando claramente su entrada y su salida, y si lo que se desea es enumerar todas las soluciones, encontrar una, contarlas u optimizar sobre ellas.

Decisiones: identificar de qué decisiones se compone una solución. Concretamente, determinar la longitud K y los conjuntos de candidatos $C[0..K)$ de la Definición 5.1.1. Esta es la decisión de diseño más importante y casi nunca es única: el mismo problema admite representaciones distintas y con árboles de reintento de tamaños muy diferentes. Una representación desafortunada no necesariamente se compensa después con una buena poda.

Predicado de solución: caracterizar cuándo una solución parcial completa es efectivamente una solución, es decir, definir *sol*.

Predicado de factibilidad: proponer *feasible* y demostrar que lo es, en el sentido de la Definición 5.1.4. La manera usual de obtenerlo es *debilitar* el predicado de solución, ya sea recortando el rango de una cuantificación para que abarque únicamente las decisiones ya tomadas, como en la Sección 5.4, ya sea relajando alguna de sus restricciones, como en la Sección 5.5. Vale la pena averiguar además si es perfecto, pues de serlo el algoritmo no desperdiciará trabajo. Recuerde la Nota 5.1.2: la única obligación es no podar de más.

Estado auxiliar: determinar qué información conviene mantener para que evaluar *feasible* no sea costoso, típicamente en tiempo constante o lineal. Por cada estructura auxiliar que se introduzca, especificar cómo se actualiza al elegir y cómo se restaura al deshacer, de modo que se preserve $Post_1$.

Orden (opcional): escoger en qué orden se toman las decisiones y en qué orden se recorren los candidatos de cada una. Este paso no afecta la corrección –el conjunto de soluciones reportadas es el mismo– pero puede cambiar drásticamente la cantidad de nodos visitados. La heurística usual es tomar primero las decisiones más restringidas y ensayar primero los

candidatos más prometedores, de manera que la poda actúe lo más arriba posible en el árbol.

Implementación: instanciar el Algoritmo 5.1.1 con las decisiones anteriores, verificando que la guarda del algoritmo computa efectivamente *feasible* sobre la solución parcial extendida.

Análisis: acotar el peor caso y, dado lo señalado en la Nota 5.1.4, medir además el tamaño del árbol efectivamente recorrido en instancias concretas.

Nota 5.3.1: ¿Por qué *sol* antes que *feasible*?

Los dos predicados aparecen en pasos consecutivos y en ese orden por una razón que conviene explicitar: no son independientes. La Definición 5.1.4 enuncia la obligación de *feasible* cuantificando sobre las soluciones del problema, de modo que sin una caracterización previa de qué es una solución no hay manera de enunciar esa obligación, mucho menos de descargarla. Un *feasible* propuesto antes que *sol* se propone sin criterio de corrección.

Hay, además, una regularidad que se repite en todos los ejemplos de este capítulo y que vale la pena tener presente:

$$\text{sol}(s) \equiv \text{feasible}(s) \wedge \text{“}s \text{ es completa”}.$$

En la Sección 5.6 es literalmente la definición de *sol* y el Teorema 5.6.3 la lee al revés. En la Sección 5.5 es el Teorema 5.5.3, donde las sumas por sufijo se anulan al llegar al final; en la Sección 5.4 lo pide demostrar el Ejercicio 2.

La regularidad no es automática y por eso hay que verificarla en cada diseño. El predicado trivial $\text{feasible}(s) = \text{true}$ satisface la Definición 5.1.4 y no la cumple en absoluto; entre ese extremo y el otro, cuanto más ajustado sea *feasible*, más cerca estará de coincidir con *sol* sobre las soluciones parciales completas. Cuando la igualdad se alcanza hay una recompensa concreta: el algoritmo no necesita verificar nada en las hojas, porque haber llegado hasta allá por un camino de extensiones factibles es ya haber encontrado una solución.

Esto explica que *sol* parezca desvanecerse del código –con frecuencia no aparece por ninguna parte– y sin embargo siga siendo indispensable en el razonamiento. Es el único puente entre el enunciado del problema, escrito en el lenguaje del dominio, y el predicado que el programa evalúa, escrito en el lenguaje de las decisiones. Saltarse el paso es cómodo hasta que la representación escogida no captura bien el enunciado, y para entonces el error está escondido dentro de *feasible*.

Llama la atención que la lista no incluya un paso dedicado a demostrar que el algoritmo es correcto. La omisión es deliberada: si los pasos anteriores se cumplieron, esa demostración ya está hecha.

Teorema 5.3.1

Sea un problema de búsqueda combinatoria y sea $\mathcal{A}$ un algoritmo tal que

1. *feasible* es un predicado de factibilidad para el problema, en el sentido de la Definición 5.1.4;
2. $\mathcal{A}$ es una instancia del Algoritmo 5.1.1, es decir, difiere de él únicamente en cómo se computan *sol* y *feasible*, y en el estado auxiliar que se haya agregado para computarlos; y
3. en todo nodo, la guarda de $\mathcal{A}$ computa *feasible*($s \cdot c$) y el estado auxiliar satisface la poscondición $Post_1$.

Entonces, $\mathcal{A}$ reporta exactamente las soluciones del problema, cada una exactamente una vez.

Demostración

Por las hipótesis 2 y 3, la ejecución de $\mathcal{A}$ visita los mismos nodos y reporta las mismas soluciones parciales completas que el Algoritmo 5.1.1 con esos predicados: las dos guardas tienen el mismo valor en cada nodo y el estado auxiliar, al restaurarse, no arrastra efectos de una rama a otra. Por la hipótesis 1 se puede aplicar el Teorema 5.1.1 a ese algoritmo, que es lo que se afirma.

Nota 5.3.2: ¿Qué queda por demostrar y qué no?

El Teorema 5.3.1 no dice que no haya nada que demostrar; dice *qué* hay que demostrar y es notable que ninguna de las tres obligaciones se refiera al recorrido del árbol, que es la parte difícil de entender de un algoritmo de reintento. También es la única que la metodología permite dar por resuelta de una vez y para siempre.

Cada hipótesis se descarga en un paso distinto. La primera es exactamente lo que pide el paso de *predicado de factibilidad* y es la única que exige una demostración genuina, usualmente breve. La tercera se descarga en el paso de *estado auxiliar* y es la que suele descuidarse: quien agrega una estructura para acelerar la evaluación de *feasible* contrae dos obligaciones, la de que la guarda siga computando el predicado correcto –lo cual exige enunciar qué significa

que la estructura sea fiel a la solución parcial, como hacen las condiciones R_0 , R_1 y R_2 de la Sección 5.4– y la de restaurarla al deshacer. La segunda se descarga en el paso de *implementación* y suele ser inmediata, pero conviene no darla por sentada porque es justamente la que se rompe en las secciones finales de este capítulo.

En efecto, las tres hipótesis son reales y ninguna sobra. La Sección 5.6 presenta un algoritmo cuyas decisiones no tienen longitud fija y que acumula un conteo en lugar de reportar: incumple la hipótesis 2 y por eso su correctitud se demuestra directamente, por inducción. Las secciones 5.7 y 5.8 mantienen una variable con la mejor solución hallada que deliberadamente *no* se restaura al deshacer: incumplen la hipótesis 3 y por eso hace falta un teorema propio, el Teorema 5.7.2.

Lo que el Teorema 5.3.1 sí garantiza es que en los casos ordinarios –los de las secciones 5.4 y 5.5, y los de la mayoría de los ejercicios– la demostración de correctitud se reduce a verificar una lista y que el esfuerzo puede concentrarse donde de verdad hace falta, que es el análisis.

Estos pasos se ilustrarán con cada uno de los ejemplos que se presentan en el resto del capítulo.

Ejercicios

1. El problema de coloración de un grafo consiste en asignar uno de M colores a cada vértice de un grafo, de manera que dos vértices adyacentes nunca reciban el mismo color.
 - a) Especifique el problema.
 - b) Siga, paso a paso, la metodología propuesta en esta sección, indicando claramente en qué consiste cada uno de los pasos. Debe resultar código en Python ejecutable.
 - c) Para el paso de *orden*, ensaye dos criterios distintos para escoger el vértice siguiente: uno arbitrario y otro que prefiera el vértice con más vecinos ya coloreados. Compare experimentalmente la cantidad de nodos visitados.
 2. Considere de nuevo el problema de generar los subconjuntos de tamaño M del Ejercicio 4 de la Sección 5.2, y la representación alternativa del Ejercicio 4, en la cual una solución parcial es una secuencia creciente de índices.
 - a) Aplique la metodología a cada una de las dos representaciones.
 - b) Cuente, en cada caso, la cantidad de nodos del árbol de reintento.
 - c) Discuta el paso de *decisiones* de la metodología a la luz de esta comparación.
-

5.4. El problema de las N reinas

En el juego de ajedrez, una reina ubicada en una casilla ataca todas las casillas que comparten con ella una fila, una columna o una diagonal, como se ilustra en la Figura 2. El problema que se estudia en esta sección consiste en ubicar N reinas en un tablero de $N \times N$ casillas sin que ninguna ataque a otra.

	0	1	2	3	4
0		×	×	×	
1	×	×	●	×	×
2		×	×	×	
3	×		×		×
4			×		

Figura 2. Casillas atacadas por una reina ubicada en la fila 1 y la columna 2 de un tablero de 5×5 .

Problema 5.4.1: Ubicación de N reinas

Entrada: Un número entero N , $N \geq 1$.

Salida: Todas las formas de ubicar N reinas en un tablero de $N \times N$ casillas, de modo que ningún par de ellas se ataque.

Este problema es el ejemplo canónico del reintento y sirve bien para dos propósitos. El primero es mostrar que la decisión de *representación* de la metodología de la Sección 5.3 puede hacer, por sí sola, la mitad del trabajo: con la representación que se escoge a continuación, dos de las tres formas de ataque quedan descartadas por construcción y no hay que podarlas. El segundo es exhibir, por primera vez en el capítulo, un predicado de factibilidad que *no* es perfecto, con las consecuencias que ello tiene sobre el trabajo desperdiciado.

5.4.1. Decisiones. Una solución es un conjunto de N casillas del tablero, de modo que la representación más directa sería una lista de pares de coordenadas. Esa representación, sin embargo, ignora una restricción que la propia definición del problema impone y que conviene aprovechar desde el principio.

Teorema 5.4.1

En toda solución del Problema 5.4.1 cada fila del tablero contiene exactamente una reina, y lo mismo ocurre con cada columna. En consecuencia, la función que asigna a cada fila la columna de su reina es una permutación de $[0..N)$.

Demostración

Dos reinas ubicadas en una misma fila se atacan, de modo que en una solución cada fila contiene a lo sumo una reina. Como hay N reinas y N filas, cada fila contiene exactamente una. El argumento para las columnas es idéntico.

Sea entonces col la función que asigna a cada fila i la columna de la única reina que hay en ella. Está bien definida por lo anterior. Es inyectiva, pues si $col(i) = col(j)$ con $i \neq j$, las reinas de las filas i y j compartirían columna y se atacarían. Una función inyectiva de $[0..N)$ en sí mismo es una biyección, es decir, una permutación.

El Teorema 5.4.1 indica cómo representar una solución: la decisión k -ésima es la columna que ocupa la reina de la fila k . Es decir, $K = N$ y $C[k] = [0..N)$ para $0 \leq k < N$, exactamente la misma representación que se usó para generar permutaciones en la Sección 5.2.

La ganancia de esta decisión es que dos de las tres formas de ataque desaparecen. Los ataques por fila son imposibles, porque la representación ubica una sola reina por fila. Los ataques por columna se reducen a que la solución parcial repita un valor, que es precisamente la condición ya estudiada. Queda por tratar únicamente el ataque en diagonal.

Teorema 5.4.2

Sean dos reinas ubicadas en las casillas (i, s_i) y (j, s_j) , con $i \neq j$. Las dos reinas comparten una diagonal si y solo si

$$i - s_i = j - s_j \quad \text{o} \quad i + s_i = j + s_j.$$

Demostración

Dos casillas comparten una diagonal si y solo si el desplazamiento vertical entre ellas coincide en magnitud con el horizontal, es decir, si y solo si $|i - j| = |s_i - s_j|$. Ahora bien, para cualesquiera enteros a y b se tiene que $|a| = |b|$ si y solo si $a = b$ o $a = -b$. Tomando $a = i - j$ y $b = s_i - s_j$:

$$\begin{aligned} |i - j| = |s_i - s_j| &\equiv i - j = s_i - s_j \vee i - j = -(s_i - s_j) \\ &\equiv i - s_i = j - s_j \vee i + s_i = j + s_j, \end{aligned}$$

que es lo que se quería demostrar.

El Teorema 5.4.2 dice que las diagonales del tablero se organizan en dos familias, y que cada familia está indexada por una cantidad que es constante a lo largo de

sus diagonales: $i - j$ para las descendentes y $i + j$ para las ascendentes. La Figura 3 muestra ambas.

0	-1	-2	-3	-4
1	0	-1	-2	-3
2	1	0	-1	-2
3	2	1	0	-1
4	3	2	1	0

$i - j$

0	1	2	3	4
1	2	3	4	5
2	3	4	5	6
3	4	5	6	7
4	5	6	7	8

$i + j$

Figura 3. Las dos familias de diagonales de un tablero de 5×5 . Cada casilla está rotulada con el valor que la identifica dentro de su familia; se resaltan la diagonal descendente $i - j = 1$ y la ascendente $i + j = 4$.

Con esto, la condición de que las reinas de las filas i y j no se ataquen adopta una forma notablemente regular: las tres cantidades s_i , $i - s_i$ e $i + s_i$ deben ser distintas para reinas distintas.

$$ok(s, i, j) : s_i \neq s_j \wedge i - s_i \neq j - s_j \wedge i + s_i \neq j + s_j.$$

El predicado de solución exige que eso valga para todo par de reinas.

$$sol(s) : (\forall i, j \mid 0 \leq i < j < N : ok(s, i, j)).$$

5.4.2. Predicado de factibilidad. Una solución parcial describe las reinas de las primeras $|s|$ filas. La condición natural para podar es que esas reinas, las ya ubicadas, no se ataquen entre sí: es decir, el mismo predicado anterior restringido a las decisiones tomadas. Lo único que cambia respecto de sol es el rango de la cuantificación.

$$feasible(s) : (\forall i, j \mid 0 \leq i < j < |s| : ok(s, i, j)).$$

Teorema 5.4.3

El predicado *feasible* es un predicado de factibilidad para el Problema 5.4.1.

Demostración

Sean t una solución y s una solución parcial con $s \preceq t$. Como s es un prefijo de t , se tiene $|s| \leq N$ y $s_i = t_i$ para $0 \leq i < |s|$. El rango de la cuantificación de $feasible(s)$ está contenido en el de $sol(t)$ y el término es el mismo, de modo que $sol(t)$ implica $feasible(s)$.

Nota 5.4.1: Una poda que no es perfecta

El generador de permutaciones de la Sección 5.2 usaba un predicado perfecto: toda solución parcial factible se extendía hasta una solución, y por lo tanto ningún nodo visitado era trabajo perdido. Aquí eso deja de ser cierto, y es la primera vez que ocurre en el capítulo.

El contraejemplo más pequeño e interesante es $N = 4$. La solución parcial $\langle 0 \rangle$ es factible —una sola reina nunca se ataca a sí misma— pero ninguna solución la extiende: como muestra la Figura 4, las dos soluciones del tablero de 4×4 empiezan ubicando la reina de la primera fila en la columna 1 o en la columna 2, nunca en la 0. El algoritmo recorrerá entonces por completo el subárbol de $\langle 0 \rangle$ sin obtener nada.

Esto no es un defecto del predicado que se propuso, sino la situación normal. Detectar que $\langle 0 \rangle$ no lleva a ninguna parte exige, en el fondo, resolver el problema. Lo que un predicado de factibilidad ofrece es una condición necesaria barata, no una respuesta; y el trabajo desperdiciado que deja pasar es exactamente lo que la Nota 5.1.4 anticipaba que el análisis asintótico no logra medir.

5.4.3. Estado auxiliar. Evaluar *feasible* recorriendo la solución parcial completa costaría tiempo $O(|s|)$ en cada nodo. Como al descender solo se agrega una reina, conviene mantener registrado qué columnas y qué diagonales están ocupadas. Por el Teorema 5.4.2 hacen falta tres arreglos de *Booleanos*:

- $used[0..N)$, para las columnas;
- $desc[0..2N - 1)$, para las diagonales descendentes; y
- $asc[0..2N - 1)$, para las ascendentes.

Los tamaños se justifican solos. Para una reina en la casilla (i, c) se tiene $i - c \in [-(N - 1)..N - 1]$, que son $2N - 1$ valores; al sumarles $N - 1$ quedan indexados por $[0..2N - 1)$. Análogamente $i + c \in [0..2N - 2]$, que también son $2N - 1$ valores y no requiere desplazamiento.

Como en el Teorema 3.2.1, conviene enunciar explícitamente qué significa que estos arreglos sean correctos respecto de la solución parcial.

$$\begin{aligned}
 R_0 : \quad & (\forall c \mid 0 \leq c < N : \\
 & \quad \text{used}[c] \equiv (\exists i \mid 0 \leq i < |s| : s_i = c)). \\
 R_1 : \quad & (\forall d \mid 0 \leq d < 2N - 1 : \\
 & \quad \text{desc}[d] \equiv (\exists i \mid 0 \leq i < |s| : i - s_i + N - 1 = d)). \\
 R_2 : \quad & (\forall d \mid 0 \leq d < 2N - 1 : \\
 & \quad \text{asc}[d] \equiv (\exists i \mid 0 \leq i < |s| : i + s_i = d)).
 \end{aligned}$$

Bajo estas condiciones, la factibilidad de extender la solución parcial se decide con tres consultas.

Teorema 5.4.4

Sean s una solución parcial con $\text{feasible}(s)$ y $k = |s| < N$, y suponga que R_0 , R_1 y R_2 son ciertas. Entonces, para todo $c \in C[k]$,

$$\text{feasible}(s \cdot c) \equiv \neg \text{used}[c] \wedge \neg \text{desc}[k - c + N - 1] \wedge \neg \text{asc}[k + c].$$

Demostración

La cuantificación de $\text{feasible}(s \cdot c)$ recorre los pares (i, j) con $0 \leq i < j \leq k$. Los pares con $j < k$ son exactamente los de $\text{feasible}(s)$, que es cierto por hipótesis. Restan los pares de la forma (i, k) con $0 \leq i < k$, para los cuales el término, usando que $s_k = c$, es

$$s_i \neq c \wedge i - s_i \neq k - c \wedge i + s_i \neq k + c.$$

Que esto valga para todo $i < k$ equivale, respectivamente, a que ninguna reina ya ubicada use la columna c , a que ninguna use la diagonal descendente $k - c$ y a que ninguna use la ascendente $k + c$. Por R_0 , R_1 y R_2 , esas tres afirmaciones son $\neg \text{used}[c]$, $\neg \text{desc}[k - c + N - 1]$ y $\neg \text{asc}[k + c]$.

5.4.4. Implementación. Con las decisiones anteriores, el Algoritmo 5.1.1 se instancia como sigue. La guarda de la línea 7 es la del Teorema 5.4.4, y las líneas 8 y 12 mantienen las condiciones R_0 , R_1 y R_2 .

Algoritmo 5.4.1: Ubicación de N reinas

```

1 def queens(N, s, used, desc, asc):
2   if len(s) == N:

```

```

3     print(s)
4     else:
5         k = len(s)
6         for c in range(N):
7             if not used[c] and not desc[k-c+N-1] and not asc[k+c]:
8                 used[c] = desc[k-c+N-1] = asc[k+c] = True    # choose
9                 s.append(c)
10                queens(N, s, used, desc, asc)
11                s.pop()                                     # undo
12                used[c] = desc[k-c+N-1] = asc[k+c] = False

```

La invocación inicial debe partir de la solución parcial vacía y de tres arreglos que registren, correctamente, que no hay ninguna columna ni ninguna diagonal ocupada. Conviene encapsularla.

Algoritmo 5.4.2: Arranque

```

1 def all_queens(N):
2     queens(N, [], [False]*N, [False]*(2*N-1), [False]*(2*N-1))

```

Ejemplo 5.4.1

Considere la siguiente invocación:

```
1 all_queens(4)
```

El resultado de esta ejecución es el siguiente:

```

1 [1, 3, 0, 2]
2 [2, 0, 3, 1]

```

Los dos tableros correspondientes se muestran en la Figura 4. Note que uno es la imagen especular del otro.

Nota 5.4.2: Reportar contra acumular

El Algoritmo 5.4.1 imprime la solución parcial y no la guarda. Si en lugar de imprimir se quisiera acumular las soluciones en una lista, no bastaría con agregar `s` a esa lista: como `s` se modifica en cada paso y se restaura al deshacer,

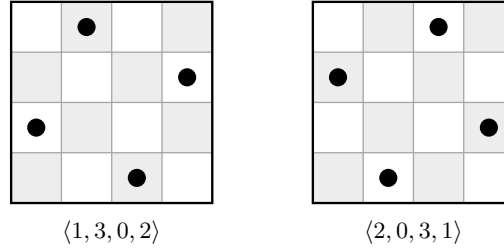


Figura 4. Las dos soluciones del problema de las 4 reinas, con la solución parcial que las representa.

todas las entradas de la lista terminarían apuntando al mismo arreglo, que al final de la ejecución estaría vacío. Hay que agregar una copia, con `s[:]` o equivalente. Este error es una consecuencia directa de la poscondición $Post_1$: el algoritmo promete restaurar el estado, y quien quiera conservar una foto de ese estado debe tomarla por su cuenta.

Teorema 5.4.5

Para $N \geq 1$, el llamado `all_queens(N)` reporta exactamente las soluciones del Problema 5.4.1, cada una exactamente una vez.

Demostración

Se verifican las tres hipótesis del Teorema 5.3.1. La primera es el Teorema 5.4.3. La segunda es inmediata: el Algoritmo 5.4.1 es el Algoritmo 5.1.1 con $C[k] = [0..N)$, con la guarda de la línea 7 evaluada antes de elegir según la Nota 5.2.2, y con el estado auxiliar de las líneas 8 y 12. Para la tercera, que la guarda computa $feasible(s \cdot c)$ es el Teorema 5.4.4, cuyas hipótesis se satisfacen porque las condiciones R_0 , R_1 y R_2 valen inicialmente y se preservan, y que se cumple $Post_1$ se sigue de que las tres asignaciones de la línea 8 se deshacen exactamente en la línea 12. Los detalles se proponen como ejercicio al lector.

Falta observar que lo reportado son soluciones y no meras soluciones parciales completas, lo cual se sigue de que $feasible(s) \equiv sol(s)$ cuando $|s| = N$.

5.4.5. Análisis. El análisis se apoya en una propiedad que parece obvia pero conviene enunciar, porque es la que conecta el algoritmo con el conteo.

Teorema 5.4.6

Todo nodo que visita el Algoritmo 5.4.1 es una solución parcial factible.

Demostración

Se propone como ejercicio al lector. Ayuda: proceda por inducción sobre el orden en que se visitan los nodos. Para el caso base, observe que la cuantificación de $feasible(\langle \rangle)$ tiene rango vacío. Para el paso inductivo, use el Teorema 5.4.4, cuyas hipótesis se satisfacen por hipótesis inductiva, y porque las condiciones R_0 , R_1 y R_2 se preservan.

Teorema 5.4.7

Sea V el conjunto de nodos que visita el Algoritmo 5.4.1. Entonces

$$|V| \leq \left(+k \mid 0 \leq k \leq N : \frac{N!}{(N-k)!} \right) < e \cdot N!,$$

el algoritmo toma tiempo $O(N \cdot N!)$ y usa espacio $O(N)$, sin contar el espacio de la salida.

Demostración

Por el Teorema 5.4.6, todo nodo visitado satisface $feasible$ y, en particular, es una secuencia de valores de $[0..N)$ distintos par a par. Para $0 \leq k \leq N$, la cantidad de tales secuencias de longitud k es $N(N-1) \cdots (N-k+1) = N!/(N-k)!$, pues hay N valores disponibles para la primera posición, $N-1$ para la segunda, y así sucesivamente. Sumando sobre k se obtiene la primera desigualdad.

Para la segunda, sustituyendo $j = N - k$,

$$\begin{aligned} \left(+k \mid 0 \leq k \leq N : \frac{N!}{(N-k)!} \right) &= N! \left(+j \mid 0 \leq j \leq N : \frac{1}{j!} \right) \\ &< N! \left(+j \mid 0 \leq j : \frac{1}{j!} \right) \\ &= e \cdot N!, \end{aligned}$$

que es lo que pide demostrar el Ejercicio 3 de la Sección 5.2.

Para el tiempo basta acotar el trabajo de un nodo, sin contar los llamados recurrentes. En un nodo interno, el ciclo de la línea 6 da N vueltas, y en cada

una se evalúa la guarda en tiempo constante –tres consultas a arreglo, por el Teorema 5.4.4– y se ejecuta una cantidad constante de asignaciones. En una hoja, reportar la solución toma tiempo $O(N)$. En ambos casos el trabajo es $O(N)$ y multiplicando por $|V|$ se obtiene $O(N \cdot N!)$.

Para el espacio, cada invocación aumenta $|s|$ en uno y $|s| \leq N$, de modo que la profundidad de la recurrencia es a lo sumo N y cada marco ocupa espacio constante. Los cuatro arreglos suman $O(N)$ posiciones y se pasan por referencia, sin copiarse.

Vale la pena mirar de cerca qué se usó en esa demostración. De las tres condiciones que impone *feasible* solo se usó una, la de que los valores sean distintos par a par; las dos condiciones sobre las diagonales no intervienen en ninguna parte del conteo. Es decir, la cota del Teorema 5.4.7 es exactamente la cantidad de nodos del árbol de permutaciones del Teorema 5.2.3: la demostración acota el árbol de las reinas por el del generador de permutaciones y renuncia por completo a la poda por diagonales.

Esa renuncia es forzosa y es la ya señalada: la poda por diagonales descarta subárboles enormes sin cambiar el peor caso. La única forma de apreciar su efecto es medir. La siguiente tabla reporta, para varios valores de N , la cantidad de soluciones, la cantidad de nodos que visita el Algoritmo 5.4.1 y la cantidad que visitaría el generador de permutaciones de la Sección 5.2 si se le pidiera enumerar todas las permutaciones para filtrar al final las que no tienen ataques en diagonal.

N	Soluciones	Nodos (reinas)	Nodos (permutaciones)	Razón
4	2	17	65	3.8
6	4	153	1 957	12.8
8	92	2 057	109 601	53.3
10	724	35 539	9 864 101	277.6
12	14 200	856 189	1 302 061 345	1520.8

Las dos columnas centrales crecen exponencialmente, tal como predice el Teorema 5.4.7, pero a ritmos muy distintos: la razón entre ellas se multiplica por un factor cercano a 5 cada vez que N aumenta en 2. Para $N = 12$ la diferencia es la que hay entre recorrer menos de un millón de nodos y recorrer más de mil millones. Ese es, en concreto, el valor de podar temprano en lugar de filtrar al final.

Note ahora que la columna **Nodos (permutaciones)** no es únicamente el costo de una alternativa peor: es, además, el valor exacto de la cota que da el Teorema 5.4.7, por lo que se observó al terminar su demostración. Léida así, la última columna de la tabla mide precisamente cuánto sobreestima esa cota: un factor de 53 para $N = 8$ y de más de 1 500 para $N = 12$, y creciendo. Es la ilustración más concreta que ofrece este capítulo de lo advertido en la Nota 5.1.4: la cota es correcta, es la

mejor que el análisis asintótico permite obtener sin hipótesis adicionales sobre la instancia, pero es inservible para predecir lo que el algoritmo va a hacer.

Nota 5.4.3: Sobre la cantidad de soluciones

La segunda columna de la tabla forma parte de la sucesión 1, 0, 0, 2, 10, 4, 40, 92, 352, 724, ..., que da la cantidad de soluciones del problema de las N reinas. No se conoce una fórmula cerrada para ella, ni un algoritmo esencialmente mejor que la enumeración para calcular sus términos, que es la razón por la cual los valores conocidos llegan apenas hasta N del orden de unas pocas decenas. Llama la atención la irregularidad del comienzo: hay más soluciones para $N = 5$ que para $N = 6$.

Ejercicios

1. Demuestre el Teorema 5.4.6, es decir, que todo nodo visitado por el Algoritmo 5.4.1 es una solución parcial factible. Es el único punto que la demostración del Teorema 5.4.7 da por sentado.
2. Complete la demostración del Teorema 5.4.5. En particular:
 - a) Demuestre que las condiciones R_0 , R_1 y R_2 son ciertas para la invocación inicial.
 - b) Demuestre que el Algoritmo 5.4.1 preserva R_0 , R_1 y R_2 , y que satisface la poscondición $Post_1$.
 - c) Demuestre que si $|s| = N$, entonces $feasible(s) \equiv sol(s)$.
3. Demuestre, sin recurrir al algoritmo, que el Problema 5.4.1 no tiene soluciones para $N = 2$ ni para $N = 3$.
4. La Nota 5.4.1 afirma que $\langle 0 \rangle$ es factible para $N = 4$ pero no se extiende a ninguna solución. Determine cuántos nodos del árbol visita el algoritmo dentro de ese subárbol. ¿Qué fracción del total es?
5. Instrumente el Algoritmo 5.4.1 con un contador de nodos visitados y reproduzca la tabla de esta sección. Agregue las filas correspondientes a $N = 5, 7, 9, 11$ y 13 . ¿Observa alguna diferencia sistemática entre los valores de N pares e impares?
6. Modifique el Algoritmo 5.4.1 para que:
 - a) cuente las soluciones en lugar de reportarlas; y
 - b) reporte únicamente la primera solución que encuentre y termine.
 Para el segundo caso, mida cuántos nodos se visitan y compare con la enumeración completa.

7. El paso de *orden* de la metodología de la Sección 5.3 sugiere ensayar primero los candidatos más prometedores. Modifique el algoritmo para que recorra las columnas desde el centro del tablero hacia los bordes, en lugar de izquierda a derecha, y compare la cantidad de nodos visitados para encontrar la primera solución. ¿Cambia la cantidad de nodos visitados para la enumeración completa? Explique.
8. Considere la variante en la cual el tablero es un toro, es decir, las diagonales se continúan por el borde opuesto.
 - a) Adapte el Teorema 5.4.2 a esta variante.
 - b) Adapte el algoritmo.
 - c) Enumere las soluciones para $N \leq 8$. ¿Para cuáles valores de N hay soluciones?
9. 🌈 El grupo de simetrías del cuadrado, de orden 8, actúa sobre el conjunto de soluciones: rotar o reflejar una solución produce otra solución.
 - a) Verifique que las dos soluciones para $N = 4$ pertenecen a una misma órbita de esta acción.
 - b) Proponga una restricción sobre la primera decisión que permita recorrer solo una parte del árbol sin perder ninguna órbita, y demuestre que es correcta.
 - c) Implemente la mejora y mida el ahorro para $N = 8, 10, 12$.
10. 🌈 Investigue construcciones explícitas que produzcan una solución del Problema 5.4.1 para todo $N \geq 4$, sin recurrir a búsqueda alguna. Implementélas y verifique con el algoritmo de esta sección que lo que producen son, en efecto, soluciones.

5.5. Suma exacta de un subconjunto

El problema de esta sección ya apareció, informalmente, en el Ejemplo 5.1.1, que se usó allí para ilustrar la noción de poda. Aquel ejemplo terminaba con una advertencia: el predicado que se propuso solo era válido suponiendo que todos los elementos del arreglo eran positivos y esa hipótesis era indispensable. El propósito de esta sección es levantarla.

Problema 5.5.1: Suma exacta de un subconjunto

Entrada: Un arreglo $A[0..N)$ de números enteros y un número entero X .

Salida: Todos los subconjuntos de elementos de $A[0..N)$ cuya suma es X .

Este problema ocupa un lugar particular en el texto. Aparece en el Capítulo 3 resuelto por tabulación, se resuelve aquí por reintento y en el Capítulo 7 se demuestra que su versión de decisión es NP-completa. Vale la pena tener presente esa triple aparición mientras se lee lo que sigue.

5.5.1. Decisiones. La representación es la misma de la Sección 5.2: la decisión i -ésima vale 1 si el elemento $A[i]$ se incluye en el subconjunto y 0 si no, de modo que $K = N$ y $C[i] = \{0, 1\}$. Conviene nombrar la suma que una solución parcial acumula.

$$\sigma(s) = (+i \mid 0 \leq i < |s| : s_i \cdot A[i]).$$

Con esta notación, el predicado de solución es inmediato.

$$\text{sol}(s) : \sigma(s) = X.$$

5.5.2. Predicado de factibilidad. El Ejemplo 5.1.1 propuso podar cuando $\sigma(s) > X$, argumentando que agregar elementos positivos solo puede aumentar la suma. Con elementos de signo arbitrario ese argumento se cae: una solución parcial cuya suma ya excede a X puede corregirse más adelante con un elemento negativo y podarla perdería soluciones.

La pregunta correcta no es si $\sigma(s)$ se pasó de X , sino *qué puede llegar a aportar todavía* lo que falta por decidir. Si $|s| = k$, las decisiones pendientes son las de los elementos $A[k..N]$, y cada una de ellas contribuye $A[i]$ o 0. La contribución total, entonces, es a lo sumo la suma de los elementos positivos que quedan y por lo menos la suma de los negativos. Esto sugiere definir, para $0 \leq k \leq N$,

$$\text{pos}(k) = (+i \mid k \leq i < N : A[i] \uparrow 0),$$

$$\text{neg}(k) = (+i \mid k \leq i < N : A[i] \downarrow 0).$$

Teorema 5.5.1

Sea s una solución parcial con $|s| = k$ y sea t una solución parcial completa que extiende a s . Entonces

$$\sigma(s) + \text{neg}(k) \leq \sigma(t) \leq \sigma(s) + \text{pos}(k).$$

Más aún, ambas cotas se alcanzan para alguna extensión de s .

Demostración

Como t extiende a s , se tiene

$$\sigma(t) - \sigma(s) = (+i \mid k \leq i < N : t_i \cdot A[i]).$$

Para cada i se cumple $t_i \in \{0, 1\}$, de modo que $t_i \cdot A[i]$ es 0 o $A[i]$. En cualquiera de los dos casos

$$A[i] \downarrow 0 \leq t_i \cdot A[i] \leq A[i] \uparrow 0,$$

pues $A[i] \downarrow 0$ es el menor y $A[i] \uparrow 0$ el mayor de esos dos valores. Sumando sobre i se obtienen las dos desigualdades del enunciado.

Para ver que las cotas se alcanzan, considere la extensión que toma $t_i = 1$ exactamente cuando $A[i] > 0$: su contribución es $pos(k)$. Análogamente, la que toma $t_i = 1$ exactamente cuando $A[i] < 0$ contribuye $neg(k)$.

El Teorema 5.5.1 dice que las sumas alcanzables desde s están confinadas al intervalo $[\sigma(s) + neg(k), \sigma(s) + pos(k)]$, y que ese intervalo es el más ajustado posible. Si X queda por fuera, no hay nada que buscar en ese subárbol.

Teorema 5.5.2

El predicado

$$feasible(s) : \quad \sigma(s) + neg(|s|) \leq X \leq \sigma(s) + pos(|s|)$$

es un predicado de factibilidad para el Problema 5.5.1.

Demostración

Sean t una solución y s una solución parcial con $s \preceq t$. Por el Teorema 5.5.1, $\sigma(s) + neg(|s|) \leq \sigma(t) \leq \sigma(s) + pos(|s|)$. Como t es solución, $\sigma(t) = X$, y reemplazando se obtiene $feasible(s)$.

Note que en esta demostración no se usó en ningún momento el signo de los elementos de A , que era lo que se quería lograr.

Nota 5.5.1: Comparación con el predicado del Ejemplo 5.1.1

Cuando todos los elementos de A son no negativos se tiene $neg(k) = 0$ para todo k y el predicado de esta sección se reduce a

$$\sigma(s) \leq X \leq \sigma(s) + pos(|s|).$$

La desigualdad de la izquierda es exactamente la poda del Ejemplo 5.1.1. La de la derecha es nueva, y descarta las soluciones parciales que se quedaron cortas sin remedio: aquellas a las que, aun tomando todo lo que queda, no les alcanza para llegar a X .

Es decir, el predicado de esta sección no solo es más general, sino estrictamente más fuerte incluso en el caso positivo. Se podría llamar a las dos mitades *poda por exceso* y *poda por defecto*. La Figura 5 muestra ambas actuando sobre la instancia del Ejemplo 5.1.1 y las tablas del análisis, más adelante en esta sección, muestran que la diferencia entre tener una sola mitad y tener las dos es de casi un orden de magnitud.

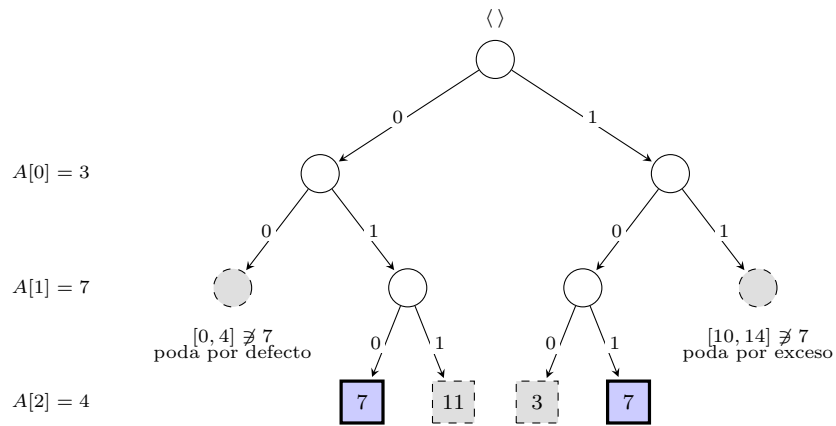


Figura 5. Árbol de reintento para $A = [3, 7, 4]$ y $X = 7$ con el predicado de factibilidad de esta sección. De los quince nodos del árbol completo, el algoritmo visita siete. Compare con la Figura 1, en la cual solo actuaba la poda por exceso.

Nota 5.5.2: Una relajación

El predicado tampoco aquí es perfecto. Para $A = [2, 2]$ y $X = 3$ la solución parcial vacía es factible, pues $0 \leq 3 \leq 4$, pero el problema no tiene soluciones.

La razón es instructiva. El conjunto de sumas alcanzables desde una solución parcial es, en general, un conjunto finito de valores con huecos; en el ejemplo, desde $\langle \rangle$ solo se alcanzan 0, 2 y 4. El predicado no trabaja con ese conjunto —calcularlo es tan difícil como resolver el problema— sino con el intervalo más pequeño que lo contiene, y los huecos se pierden. A esta idea, la de reemplazar un conjunto difícil de describir por otro más simple que lo contiene, se le llama *relajación*, y es una de las maneras más productivas de

obtener predicados de factibilidad. En la Sección 5.7 se usará de nuevo, con otro problema y con otra relajación.

5.5.3. Estado auxiliar. Hacen falta dos cosas. La primera es la suma acumulada $\sigma(s)$, que se mantiene como parámetro. La segunda son los valores de pos y neg , que no dependen de la solución parcial sino únicamente de la entrada y , que por lo tanto, se calculan una sola vez. Ambos satisfacen recurrencias inmediatas: $pos(N) = neg(N) = 0$ y, para $0 \leq k < N$,

$$\begin{aligned} pos(k) &= A[k] \uparrow 0 + pos(k+1), \\ neg(k) &= A[k] \downarrow 0 + neg(k+1). \end{aligned}$$

Calcularlas recorriendo el arreglo de derecha a izquierda es una tabulación en el sentido del Capítulo 3.

Algoritmo 5.5.1: Cotas por sufijo

```

1 def suffix_bounds(A):
2     N = len(A)
3     pos = [0]*(N+1)
4     neg = [0]*(N+1)
5     for k in range(N-1, -1, -1):
6         pos[k] = max(A[k], 0) + pos[k+1]
7         neg[k] = min(A[k], 0) + neg[k+1]
8     return pos, neg

```

Nota 5.5.3: Estado que no hay que restaurar

El estado auxiliar de esta sección contrasta con el de la Sección 5.4 en un punto que vale la pena notar. Allí los tres arreglos se modificaban al elegir y había que restaurarlos explícitamente al deshacer, so pena de perder la poscondición $Post_1$. Aquí no hay nada que restaurar: los arreglos pos y neg nunca se modifican, y la suma acumulada viaja como parámetro, de modo que cada invocación recibe su propia copia y la del invocador queda intacta.

Este es, en general, el recurso más seguro: cuando el estado auxiliar es pequeño, pasarlo por valor elimina de raíz la posibilidad de olvidar una restauración. Cuando es grande –como los $2N - 1$ Booleanos de las diagonales– copiarlo en cada nodo costaría más que mantenerlo, y no queda más remedio que modificar y restaurar.

5.5.4. Implementación. Conviene fortalecer la precondition del llamado recurrente respecto de la que se enunció para el Algoritmo 5.1.1, exigiendo que la solución parcial recibida ya sea factible y que el parámetro `sigma` sea correcto.

Pre : s es una solución parcial, $feasible(s)$ y $sigma = \sigma(s)$.

Fortalecerla tiene una recompensa concreta.

Teorema 5.5.3

Sea s una solución parcial con $|s| = N$. Entonces $feasible(s) \equiv sol(s)$.

Demostración

Las sumas $pos(N)$ y $neg(N)$ son vacías y por lo tanto valen 0. Luego $feasible(s)$ es $\sigma(s) \leq X \leq \sigma(s)$, que equivale a $\sigma(s) = X$, es decir, a $sol(s)$.

Es decir, bajo la precondition fortalecida no hace falta verificar nada en las hojas: toda solución parcial completa que el algoritmo alcance es, necesariamente, una solución.

Algoritmo 5.5.2: Suma exacta de un subconjunto

```

1 def subset_sum(A, X, s, sigma, pos, neg):
2     N = len(A)
3     if len(s) == N:
4         print([ A[i] for i in range(N) if s[i] == 1 ])
5     else:
6         k = len(s)
7         for c in [0, 1]:
8             t = sigma + c * A[k]
9             if t + neg[k+1] <= X <= t + pos[k+1]:
10                 s.append(c)                # choose
11                 subset_sum(A, X, s, t, pos, neg)
12                 s.pop()                    # undo

```

La guarda de la línea 9 es literalmente el predicado de factibilidad evaluado en $s \cdot c$, con t en el papel de $\sigma(s \cdot c)$. La invocación inicial debe calcular las cotas y establecer la precondition, lo cual incluye verificar que la solución parcial vacía sea factible.

Algoritmo 5.5.3: Arranque

```

1 def all_subset_sums(A, X):
2     pos, neg = suffix_bounds(A)
3     if neg[0] <= X <= pos[0]:
4         subset_sum(A, X, [], 0, pos, neg)

```

Ejemplo 5.5.1

Considere las siguientes invocaciones, la primera con la instancia del Ejemplo 5.1.1 y la segunda con la variante de signos mezclados que allí se señaló como problemática:

```

1 all_subset_sums([3,7,4], 7)
2 all_subset_sums([3,7,-3], 7)

```

El resultado de estas ejecuciones es el siguiente:

```

1 [7]
2 [3, 4]
3 [7]
4 [3, 7, -3]

```

La segunda instancia es la que el predicado del Ejemplo 5.1.1 habría manejado mal: allí se observó que $\langle 1, 1 \rangle$, con suma 10, se extiende a la solución $\langle 1, 1, 1 \rangle$, con suma 7. El predicado de esta sección no la poda, porque $neg(2) = -3$ y por lo tanto el intervalo alcanzable desde $\langle 1, 1 \rangle$ es $[7..10]$, que sí contiene a X .

Teorema 5.5.4

El llamado `all_subset_sums(A, X)` reporta exactamente los subconjuntos de elementos de $A[0..N)$ cuya suma es X , cada uno exactamente una vez.

Demostración

Se propone como ejercicio al lector.

5.5.5. Análisis. Como se ha planteado anteriormente, las podas permiten cambiar el valor de la constante que acompaña el comportamiento asintótico del algoritmo de exploración, pero no necesariamente revierte su naturaleza exponencial.

Teorema 5.5.5

El llamado `all_subset_sums(A, X)` con $|A| = N$ visita a lo sumo $2^{N+1} - 1$ nodos, toma tiempo $O(N \cdot 2^N)$ y usa espacio $O(N)$, sin contar el espacio de la salida.

Demostración

Se propone como ejercicio al lector.

Como en las secciones anteriores, la cota de peor caso es indiferente a la calidad de la poda y hay que medir. La siguiente tabla reporta la cantidad promedio de nodos visitados sobre cuarenta instancias aleatorias por fila, con X elegido siempre como la suma de un subconjunto aleatorio, de modo que todas las instancias tienen al menos una solución.

N	Elementos en $[1..100]$		
	Sin poda	Por exceso	Por exceso y defecto
12	8 191	4 808	1 089
16	131 071	88 426	16 208
20	2 097 151	1 102 915	218 810
24	33 554 431	20 442 316	3 406 942

N	Elementos en $[-100..100]$	
	Sin poda	Por exceso y defecto
12	8 191	1 179
16	131 071	17 048
20	2 097 151	208 907
24	33 554 431	3 578 645

Hay dos observaciones que hacer. La primera es que la poda por exceso, por sí sola, es sorprendentemente débil: para $N = 24$ apenas evita recorrer un tercio del árbol. La poda por defecto, agregada, baja el recorrido a una décima parte. La segunda es que la columna de la segunda tabla no es peor que la correspondiente de la primera, pese a que allí no hay poda por exceso posible en el sentido ingenuo: con elementos de ambos signos los intervalos alcanzables son más anchos, pero también lo es el rango de valores de X , y los dos efectos se compensan.

Ninguna de las dos tablas cambia el hecho de que el crecimiento sigue siendo exponencial. Para este problema eso no es un accidente del algoritmo: en el Capítulo 7 se demuestra que su versión de decisión es NP-completa, de modo que no se espera encontrar nada esencialmente mejor.

Nota 5.5.4: Reintento contra tabulación

El Capítulo 3 resuelve este mismo problema por tabulación, con un algoritmo cuya complejidad depende de N y de X , y no de N solamente. Ninguno de los dos domina al otro.

La tabulación es preferible cuando X es pequeño, pues su costo es proporcional a él, y es inaplicable cuando los elementos no son enteros o cuando X es enorme. El reintento es indiferente a la magnitud de X y a la naturaleza de los números –sirve igual con enteros negativos, y con racionales o reales, como se propone verificar en los ejercicios– pero su costo crece con N sin importar cuán pequeña sea la instancia en otros aspectos. Que un problema tenga dos algoritmos incomparables, cada uno mejor en una región distinta del espacio de instancias, es más frecuente de lo que sugiere la costumbre de resumir un algoritmo en una sola cota asintótica.

Ejercicios

1. Demuestre el Teorema 5.5.5.
2. Demuestre el Teorema 5.5.4 verificando las tres hipótesis del Teorema 5.3.1. Note que la segunda no se cumple literalmente, pues el Algoritmo 5.5.2 omite la verificación de *sol* en las hojas; justifique esa omisión con el Teorema 5.5.3 y verifique que el Algoritmo 5.5.3 establece la precondition fortalecida.
3. Demuestre que el Algoritmo 5.5.1 calcula correctamente *pos* y *neg*. Formule los invariantes del ciclo. ¿Cuál es su complejidad temporal?
4. La Nota 5.5.2 exhibe una instancia sin soluciones en la cual el predicado no poda. Construya una instancia *con* soluciones en la cual el algoritmo desperdicie trabajo, y cuantifique cuánto.
5. Demuestre formalmente la afirmación de la Nota 5.5.1: si todos los elementos de A son no negativos, entonces el predicado de esta sección implica el del Ejemplo 5.1.1, y la implicación recíproca es falsa.
6. Reproduzca las dos tablas de esta sección. Agregue una tercera con elementos en $[1..10]$, es decir, con muchos valores repetidos, y explique lo que observa.
7. Una heurística usual para este problema es ordenar A de mayor a menor magnitud antes de comenzar.
 - a) Explique, en términos del paso de *orden* de la metodología de la Sección 5.3, por qué cabría esperar que ayude.
 - b) Mídalo. ¿Ayuda?
8. Modifique el Algoritmo 5.5.2 para que:
 - a) cuente las soluciones en lugar de reportarlas;
 - b) reporte únicamente la primera y termine; y
 - c) reporte una solución de tamaño mínimo.
9. El problema de *partición* consiste en decidir si un arreglo de enteros positivos se puede partir en dos subconjuntos de igual suma. Redúzcalo al Problema 5.5.1 y resuélvalo con el algoritmo de esta sección.
10. El Teorema 5.5.1 y su demostración no usan en ningún momento que los elementos de A sean enteros.
 - a) Verifique que el algoritmo de esta sección funciona sin cambios con elementos racionales.
 - b) Discuta qué ocurre al ejecutarlo sobre números de punto flotante. ¿Sigue siendo correcto? Ayuda: piense en la línea 3 del Algoritmo 5.5.2 y en cómo se comparan dos valores.
11. 🌈 Existe un algoritmo para este problema que toma tiempo del orden de $2^{N/2}$, a costa de usar también ese espacio, y que consiste en partir el arreglo en dos

mitades, enumerar las sumas alcanzables de cada una y combinarlas. Investíguelo, impleméntelo y compare experimentalmente con el algoritmo de esta sección para $N = 24, 28, 32$.

5.6. Conteo de segmentaciones de texto

Considere una cadena de caracteres escrita sin espacios y la pregunta de en cuántas formas distintas puede partirse en palabras. El problema aparece al procesar textos en lenguas que se escriben sin separar las palabras, al interpretar identificadores o direcciones web que las concatenan, y al corregir la salida de sistemas de reconocimiento de escritura o de voz. En todos esos casos, si la cantidad de segmentaciones es mayor a uno es exactamente la señal de que el texto es ambiguo.

Problema 5.6.1: Conteo de segmentaciones

- Entrada:** Una cadena $A[0..N]$ de caracteres, con $N \geq 1$, y un predicado *isWord* tal que, para $0 \leq l < r \leq N$, *isWord*(l, r) indica si $A[l..r]$ es una palabra.
- Salida:** La cantidad de formas distintas de escribir $A[0..N]$ como concatenación de una secuencia de palabras.

El predicado *isWord* se supone dado y correcto; el problema no dice cómo se implementa ni cuánto cuesta evaluarlo. Note que solo se consulta sobre rangos no vacíos, por las razones que se explican más abajo.

El primer paso de la metodología de la Sección 5.3 pide precisar, además de la entrada y la salida, qué se le está pidiendo al algoritmo. Aquí no se pide enumerar las soluciones ni optimizar sobre ellas, sino contarlas: es el modo de *conteo* de los tres que anuncia la Sección 5.1 y es el único problema del capítulo que lo usa. La diferencia con los modos ya vistos no es cosmética y conviene anticiparla, porque condiciona todo lo que sigue.

- A diferencia de la enumeración, no hay que conservar las soluciones ni escribirlas; basta con un acumulador. El algoritmo podrá, por eso, prescindir por completo de la solución parcial.
- A diferencia de la búsqueda, no se puede parar temprano. Cada solución debe alcanzarse para ser contada, de modo que el árbol factible se recorre íntegro y no hay cortocircuito posible.

Conviene fijar formalmente qué se está contando.

Definición 5.6.1: Segmentación

Para $0 \leq l \leq r \leq N$, una *segmentación* de $A[l..r]$ es una secuencia de índices

$$l = p_0 < p_1 < \dots < p_k = r, \quad k \geq 0,$$

tal que $\text{isWord}(p_{t-1}, p_t)$ para todo $1 \leq t \leq k$. Los índices $p_1, \dots, p_{k-1}$ son los *puntos de corte* y cada $A[p_{t-1}..p_t]$ es una de las k palabras de la segmentación. La cadena $A[l..r]$ es *segmentable* si admite por lo menos una segmentación.

Dos segmentaciones son distintas cuando lo son como secuencias de índices. El caso $k = 0$ corresponde a la secuencia vacía de palabras y solo es posible cuando $l = r$.

Nota 5.6.1: Sobre la cadena vacía

Se supone que la cadena vacía es una palabra. La suposición es inocua: una palabra vacía no aporta nada a una concatenación, de modo que suprimir de una secuencia todas sus palabras vacías produce otra secuencia de palabras con la misma concatenación. En consecuencia, contar segmentaciones con palabras no vacías –que es lo que exige la Definición 5.6.1 al pedir cortes estrictamente crecientes– es contar todas las que importan; de otro modo la respuesta sería infinita, pues se podrían intercalar palabras vacías sin límite.

Esta es la razón por la cual isWord solo se consulta sobre rangos no vacíos. La cadena vacía tiene, con esta convención, exactamente una segmentación: la vacía. Ese hecho será el caso base del algoritmo.

Ejemplo 5.6.1

Suponga que las únicas palabras son ES, PARA, PARASOL, SAL, SALTA, SALTAMONTES, SOL, SOLES y MONTES.

- $A = \text{PARASOLES}$, con $N = 9$, tiene tres segmentaciones: $\text{PARA} \cdot \text{SOL} \cdot \text{ES}$, $\text{PARA} \cdot \text{SOLES}$ y $\text{PARASOL} \cdot \text{ES}$.
- $A = \text{PARASOLED}$, con $N = 9$, no tiene ninguna: los únicos prefijos que son palabras son PARA y PARASOL, y ni SOLED ni ED son segmentables.
- $A = \text{SALTAMONTES}$, con $N = 11$, tiene dos: la que no tiene cortes intermedios, pues la cadena completa es una palabra, y $\text{SALTA} \cdot \text{MONTES}$. Note que $\text{SAL} \cdot \dots$ no conduce a ninguna, pese a que SAL sí es una palabra: que un prefijo sea palabra no garantiza que el resto sea segmentable.

5.6.1. Decisiones. El paso siguiente de la metodología pide identificar de qué decisiones se compone una solución. Es, según se advirtió en la Sección 5.3, la decisión de diseño más importante y casi nunca es única. Este problema admite dos representaciones naturales que hay que comparar antes de escoger.

Una decisión por carácter. Una segmentación queda determinada por sus puntos de corte, que forman un subconjunto de las posiciones interiores $1, \dots, N - 1$. Representarla, entonces, es representar ese subconjunto, y la Sección 5.2 ya enseñó cómo: tomar $K = N - 1$ decisiones binarias, donde la decisión i -ésima vale 1 si hay un corte en la posición $i + 1$. Esta representación se ajusta literalmente a la Definición 5.1.1.

Una decisión por palabra. Alternativamente, se puede construir la segmentación de izquierda a derecha decidiendo, en cada paso, *dónde termina la palabra siguiente*. Una solución parcial es entonces una secuencia creciente de cortes

$$s = \langle p_1, p_2, \dots, p_k \rangle, \quad 0 < p_1 < p_2 < \dots < p_k \leq N,$$

con la convención $p_0 = 0$, y extenderla es escoger un j con $p_k < j \leq N$. La cantidad de alternativas de una decisión no es fija y las soluciones parciales tienen longitudes distintas, de modo que esta representación no es la de la Definición 5.1.1, sino la variante que propone explorar el Ejercicio 4.

Las dos son correctas y describen los mismos objetos. Lo que las separa es el tamaño del árbol que producen, y la diferencia se puede establecer con exactitud.

Teorema 5.6.1

Para $0 \leq l \leq N$, sea $f(l)$ la cantidad de segmentaciones de $A[0..l]$. Sea V_p el conjunto de soluciones parciales factibles bajo la representación por palabra y V_c el correspondiente bajo la representación por carácter. Entonces

$$|V_p| = (+l \mid 0 \leq l \leq N : f(l)),$$

$$|V_c| = (+l \mid 0 \leq l < N : f(l) (N - l)).$$

Demostración

Bajo la representación por palabra, una solución parcial factible es una secuencia creciente de cortes cuyos bloques son todas palabras, es decir, exactamente una segmentación del prefijo $A[0..p_k]$; y recíprocamente, toda segmentación de un prefijo es una solución parcial factible. La correspondencia es biyectiva y agrupa por el valor de $p_k = l$, de donde $|V_p| = (+l \mid 0 \leq l \leq N : f(l))$.

Bajo la representación por carácter, una solución parcial factible de longitud k decide las posiciones $1, \dots, k$ y queda determinada por dos datos independientes: la segmentación del prefijo que sus cortes describen, digamos de $A[0..l)$ con $l \leq k$, y la propia k , que registra cuántos caracteres se han recorrido desde el último corte sin cortar. Para cada segmentación de $A[0..l)$ hay una solución parcial factible por cada k con $l \leq k \leq N-1$, esto es, $N-l$ de ellas. Sumando sobre l se obtiene la expresión.

Nota 5.6.2: Por qué se escoge una y no la otra

El Teorema 5.6.1 dice que la representación por carácter visita, por cada solución parcial que la otra visita, hasta N nodos en lugar de uno. La razón es transparente: gasta una decisión por cada posición de la cadena y la mayoría de esas decisiones –las de no cortar– no aportan información alguna ni pueden podarse, porque no cierran ningún bloque. La representación por palabra gasta una decisión por cada palabra de la solución y cada una de ellas se somete de inmediato a la prueba de factibilidad.

Medido sobre textos construidos concatenando palabras de un vocabulario aleatorio, el cociente $|V_c|/|V_p|$ pasa de 5 para $N = 13$ a cerca de 13 para $N = 37$, creciendo de manera aproximadamente lineal, como predice el teorema. En lo que sigue se usa la representación por palabra. La otra no queda descartada como ejercicio: el Ejercicio 3 pide desarrollarla completa y es la mejor manera de comprobar que una representación desafortunada no se compensa después con una buena poda.

Fijada la representación, el predicado de solución dice dos cosas: que todos los bloques son palabras y que el último corte llega al final de la cadena.

$$sol(s) : (\forall t \mid 1 \leq t \leq k : isWord(p_{t-1}, p_t)) \wedge p_k = N.$$

5.6.2. Predicado de factibilidad. Una solución parcial que ya contiene un bloque que no es palabra no tiene arreglo posible, pues los bloques ya decididos no cambian al extender la solución. Eso sugiere quedarse con la primera mitad del predicado de solución.

$$feasible(s) : (\forall t \mid 1 \leq t \leq k : isWord(p_{t-1}, p_t)).$$

Teorema 5.6.2

El predicado *feasible* es un predicado de factibilidad para el Problema 5.6.1.

Demostración

Sean $t = \langle q_1, \dots, q_m \rangle$ una solución y s una solución parcial con $s \preceq t$, digamos $s = \langle q_1, \dots, q_k \rangle$ con $k \leq m$. Los bloques de s son exactamente los primeros k bloques de t y todos ellos son palabras porque $\text{sol}(t)$. Luego $\text{feasible}(s)$.

Teorema 5.6.3

Sea $s = \langle p_1, \dots, p_k \rangle$ una solución parcial con $\text{feasible}(s)$. Entonces $\text{sol}(s)$ si y solo si $p_k = N$.

Demostración

El predicado $\text{sol}(s)$ es la conjunción de $\text{feasible}(s)$ con $p_k = N$. Si la primera se cumple por hipótesis, la conjunción equivale a la segunda.

El Teorema 5.6.3 tendrá una consecuencia agradable sobre la implementación: si el algoritmo solo desciende por soluciones parciales factibles, entonces llegar al final de la cadena es haber encontrado una solución y no hay nada que verificar.

Nota 5.6.3: Una poda que no es perfecta

El predicado no es perfecto en el sentido del Ejercicio 5, y el Ejemplo 5.6.1 ya exhibió el testigo. Para $A = \text{SALTAMONTES}$, la solución parcial $\langle 3 \rangle$ es factible, pues **SAL** es una palabra, pero ninguna solución la extiende: lo que queda, **TAMONTES**, no empieza por ninguna palabra del vocabulario.

Es la misma situación de la Nota 5.4.1 y por la misma razón de fondo: decidir si $\langle 3 \rangle$ lleva a alguna parte exige resolver el problema sobre lo que queda, que es tan difícil como el problema original. Lo que el predicado ofrece es una condición necesaria barata.

5.6.3. Estado auxiliar. El paso siguiente pregunta qué información conviene mantener para que evaluar feasible sea barato. Aquí la respuesta es sorprendentemente escueta. Suponga que $\text{feasible}(s)$ es cierto y que se quiere extender s con un corte j . Los bloques de $s \cdot j$ son los de s , que ya se sabe que son palabras, más el bloque nuevo $A[p_k..j]$. Luego

$$\text{feasible}(s \cdot j) \equiv \text{isWord}(p_k, j),$$

y para decidirlo no hace falta nada de s salvo su último corte.

Nota 5.6.4: El pasado se resume en un índice

Vale la pena detenerse en esto, porque es lo que distingue a este problema de los anteriores y lo que decide el resto del diseño. *Cuáles* palabras se hayan escogido hasta el momento no restringe en nada las que se pueden escoger de ahí en adelante: toda la información del pasado que el algoritmo necesita cabe en un único índice.

Compárese con las secciones anteriores. En el problema de las N reinas, la solución parcial completa determinaba qué columnas y qué diagonales quedaban disponibles, y hubo que mantener tres arreglos para registrarlo, actualizándolos al elegir y restaurándolos al deshacer. En la suma exacta hizo falta arrastrar la suma acumulada, aunque como parámetro y no como estructura modificable. Aquí basta con el último corte, que también viaja como parámetro, de modo que la poscondición $Post_1$ se satisface sin que haya nada que restaurar. Esta observación tiene además una consecuencia que se recoge al final de la sección.

De aquí y del modo de conteo se sigue que el algoritmo no necesita mantener la solución parcial en absoluto. Por un lado, el estado necesario para decidir la factibilidad es un solo índice. Por el otro, lo que se pide es un número, no una lista de soluciones, de modo que tampoco hay que conservar la solución parcial para reportarla. Todo lo que el algoritmo requiere saber en un nodo es dónde terminó la última palabra.

Eso justifica trabajar con la siguiente función objetivo, donde n ocupa el lugar del último corte y $n = 0$ corresponde a la solución parcial vacía.

$\phi(n)$: “la cantidad de segmentaciones de $A[n..N]$.”

El problema original se recupera como el caso particular $\phi(0)$. El dominio incluye $n = N$, es decir, el sufijo vacío. Por la Nota 5.6.1 se tiene $\phi(N) = 1$: es el caso base y es también lo que dice el Teorema 5.6.3 leído sobre esta función.

Definición 5.6.2: Función objetivo

Para $0 \leq n \leq N$,

$$\phi(n) = \begin{cases} 1 & , \text{ si } n = N, \\ (+j \mid n < j \leq N \wedge isWord(n, j) : \phi(j)) & , \text{ si } n < N. \end{cases}$$

La alternativa $j = N$, la que agota el sufijo con una sola palabra, no requiere tratamiento especial: para ella el término es $\phi(N) = 1$, de modo que aporta

una unidad exactamente cuando $isWord(n, N)$, como es de esperarse. Y si ninguna alternativa es factible, la cuantificación tiene rango vacío y $\phi(n) = 0$.

Todo llamado recurrente $\phi(j)$ ocurre con $j > n$, de modo que $N - j < N - n$: la recurrencia siempre avanza hacia el caso base y ϕ está bien definida sobre $0 \leq n \leq N$.

Nota 5.6.5: ¿Dónde quedó el reintento?

La Definición 5.6.2 es, en sí misma, el algoritmo: es ejecutable y cada caso indica exactamente qué calcular. Vista así puede costar reconocer en ella los tres pasos del Algoritmo 5.1.1, que están escondidos en la manera de evaluar la cuantificación.

Los términos se calculan uno tras otro, de izquierda a derecha: se *elige* una alternativa j , se *explora* el sufijo correspondiente, y al retornar se *des-hace* esa decisión para ensayar la siguiente. La poda está en el rango de la cuantificación: las alternativas con $\neg isWord(n, j)$ —que es, por lo visto arriba, exactamente $\neg feasible(s \cdot j)$ — no contribuyen ningún término y no generan llamado recurrente, tal como el Algoritmo 5.2.2 descartaba un candidato antes de extender la solución parcial. En el vocabulario de la Nota 5.2.2, la factibilidad se evalúa antes de elegir.

Lo que no aparece por ninguna parte es el cortocircuito. En una cuantificación existencial, hallar un testigo permite abandonar el resto; en una sumatoria hay que evaluar todos los términos. Es la diferencia entre buscar y contar, anunciada al comienzo de la sección.

5.6.4. Implementación. La transcripción de la Definición 5.6.2 es directa. El predicado se recibe como parámetro, del mismo modo que la memoria compartida se recibe como parámetro en el Algoritmo 3.2.2.

Algoritmo 5.6.1: Conteo de segmentaciones

```

1 def count(A, n, isWord):
2     N = len(A)
3     if n==N: ans = 1
4     else:
5         ans = 0
6         for j in range(n+1, N+1):
7             if isWord(n,j): ans = ans + count(A, j, isWord)
8     return ans

```

Algoritmo 5.6.2: Arranque

```

1 def segmentations(A, isWord):
2     return count(A, 0, isWord)

```

El ciclo de la línea 6 recorre las alternativas y la guarda de la línea 7 es el predicado de factibilidad. No hay ninguna verificación en el caso base, por el Teorema 5.6.3: llegar a $n = N$ por un camino de alternativas factibles es ya haber encontrado una solución, y contarla es sumar uno.

Nota 5.6.6: Sobre el orden

El paso de *orden* de la metodología de la Sección 5.3 es aquí vacío, por una razón que vale la pena hacer explícita: en el modo de conteo no hay nada que ganar reordenando. Las decisiones vienen forzadas de izquierda a derecha por la representación, y el orden en que se recorren las alternativas no puede cambiar la cantidad de nodos visitados, porque todos los nodos factibles se visitan de todas maneras.

Contrástese con la Nota 5.7.5, donde reordenar los candidatos cambiaba el recorrido en un factor de veinte. Allí el orden importaba porque encontrar pronto una buena solución permitía podar el resto; aquí no hay nada análogo. Es la contrapartida de lo observado en la Nota 5.6.5: sin cortocircuito no hay premio por acertar temprano.

Ejemplo 5.6.2

Con el vocabulario del Ejemplo 5.6.1, representado como un conjunto de cadenas:

```

1 WORDS = {'ES', 'PARA', 'PARASOL', 'SAL', 'SALTA',
2         'SALTAMONTES', 'SOL', 'SOLES', 'MONTES'}
3
4 for A in ['PARASOLES', 'PARASOLED', 'SALTAMONTES']:
5     isWord = lambda l, r: A[l:r] in WORDS
6     print(A, segmentations(A, isWord))

```

El resultado de esta ejecución es el siguiente:

```

1 PARASOLES 3
2 PARASOLED 0
3 SALTAMONTES 2

```

5.6.5. El árbol de búsqueda. Los llamados recurrentes que genera una ejecución forman un árbol: cada nodo es un llamado $\phi(n)$ y cada arco es una decisión, es decir, la escogencia de una alternativa j . Las figuras 6 y 7 muestran ese árbol para dos de las entradas del Ejemplo 5.6.1.

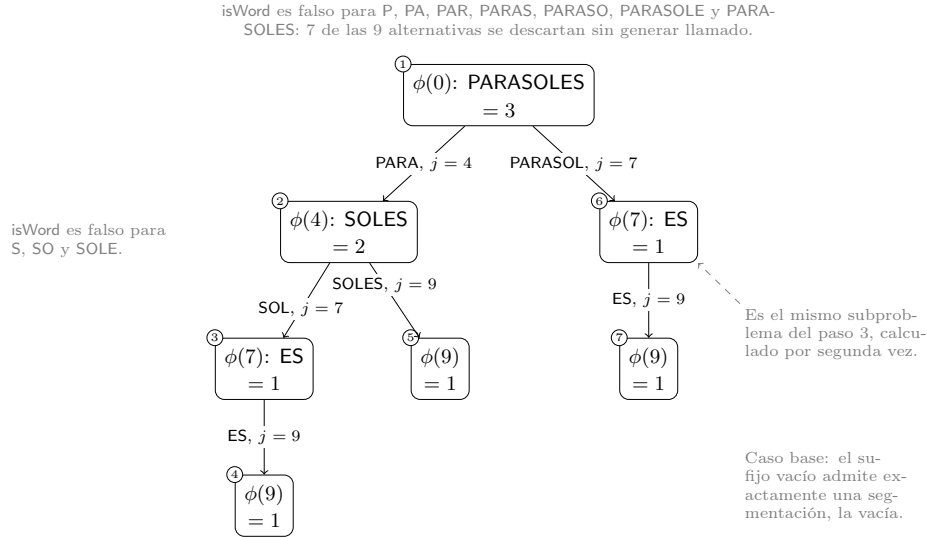


Figura 6. Árbol de búsqueda para $A = \text{PARASOLES}$. Los números indican el orden en que se visitan los nodos. El valor de un nodo interno es la suma de los valores de sus hijos.

En la Figura 6 se aprecian los tres fenómenos que caracterizan al reintento.

Poda: la mayoría de las alternativas se descartan al evaluar *isWord*, sin llegar a generar un llamado: en la raíz, 7 de las 9. El árbol que efectivamente se recorre es mucho menor que el de todas las formas de cortar la cadena.

Retroceso: al retornar del paso 3 con el valor 1, la decisión $j = 7$ queda cerrada y se reintenta con la siguiente alternativa, $j = 9$, cuyo resultado se acumula sobre el anterior.

Repetición: los pasos 3 y 6 resuelven exactamente el mismo subproblema, $\phi(7)$, sin que el segundo aproveche nada de lo hecho por el primero.

Los dos primeros son la estrategia; el tercero es su costo, y de él se ocupa el análisis al final de la sección. Note que las tres hojas del árbol son llamados $\phi(9)$ que retornan 1, y que hay exactamente una por cada segmentación de la cadena. Esto no es casualidad y es lo que hará demostrable el algoritmo: las hojas del árbol de

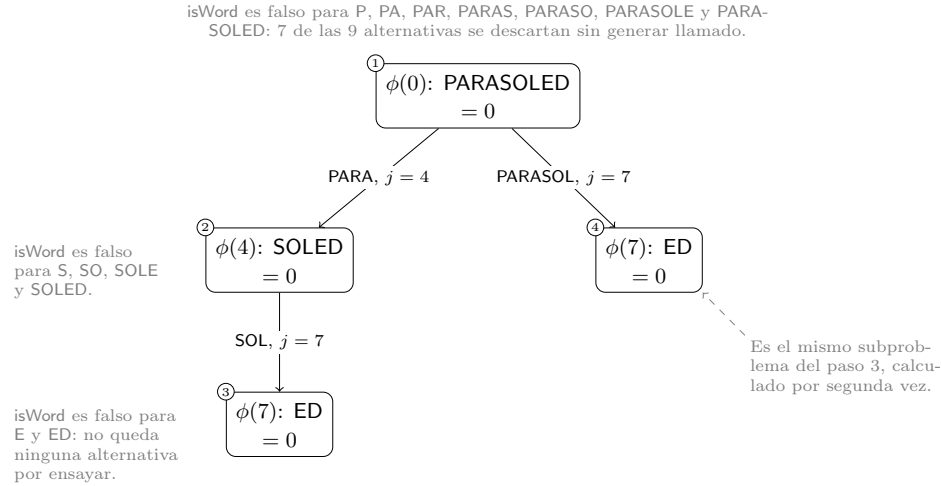


Figura 7. Árbol de búsqueda para $A = \text{PARASOLED}$. Los números indican el orden en que se visitan los nodos.

búsqueda están en biyección con las soluciones y el valor de la raíz es su cantidad. La Figura 7 es el mismo dibujo sin hojas: la cadena no es segmentable y todos los valores son 0.

5.6.6. Verificación de correctitud. El Algoritmo 5.6.1 se aparta del Algoritmo 5.1.1 en dos puntos: las soluciones parciales tienen longitudes distintas y el resultado es un número que se acumula hacia arriba en lugar de una enumeración. El Teorema 5.1.1 no aplica, entonces, directamente, y conviene demostrar la correctitud sin apoyarse en él.

Teorema 5.6.4

Sea n tal que $0 \leq n \leq N$. Entonces $\phi(n)$ es la cantidad de segmentaciones de $A[n..N]$.

Demostración

Sea $S(n)$ el conjunto de segmentaciones de $A[n..N]$. Se procede por inducción fuerte sobre la longitud del sufijo, $m = N - n \geq 0$.

Caso base ($m = 0$, es decir, $n = N$): el sufijo $A[N..N]$ es la cadena vacía. Una segmentación suya es una secuencia $N = p_0 < \dots < p_k = N$, y la

única posible es la de $k = 0$, pues $k \geq 1$ obligaría a $p_0 < p_k$. Luego $|S(N)| = 1 = \phi(N)$.

Caso inductivo ($m \geq 1$, es decir, $n < N$): la hipótesis de inducción fuerte es que $\phi(n') = |S(n')|$ para todo n' con $N - n' < m$. Para cada j con $n < j \leq N$ y $\text{isWord}(n, j)$, sea $S_j(n)$ el conjunto de las segmentaciones de $A[n..N]$ cuya primera palabra termina en j , esto es, aquellas con $p_1 = j$.

Los conjuntos $S_j(n)$ son disjuntos dos a dos, pues una segmentación determina su p_1 . Además cubren a $S(n)$: toda segmentación de $A[n..N]$ tiene $k \geq 1$, porque $n < N$ impide $k = 0$, y por lo tanto tiene un p_1 bien definido, que satisface $n < p_1 \leq N$ y $\text{isWord}(n, p_1)$. Luego

$$|S(n)| = (+j \mid n < j \leq N \wedge \text{isWord}(n, j) : |S_j(n)|).$$

Falta ver que $|S_j(n)| = |S(j)|$. La función que a una segmentación $n < j = p_1 < \dots < p_k = N$ de $S_j(n)$ le asigna $j = p_1 < \dots < p_k = N$, es decir, la que resulta de suprimir el índice inicial, está bien definida –los bloques que quedan son los mismos y siguen siendo palabras– y tiene por inversa la que antepone n a una segmentación de $S(j)$, que es legítima porque $\text{isWord}(n, j)$. Es, entonces, una biyección entre $S_j(n)$ y $S(j)$.

Como $N - j < N - n = m$, la hipótesis de inducción da $|S(j)| = \phi(j)$. Reemplazando en la igualdad anterior se obtiene $|S(n)| = \phi(n)$, que era lo que se quería.

Conviene señalar qué papel juega cada mitad del argumento. Que los $S_j(n)$ cubran a $S(n)$ es lo que garantiza que la enumeración de alternativas es exhaustiva: ninguna segmentación se escapa, porque toda segmentación tiene una primera palabra y esa primera palabra es una de las alternativas consideradas. Que sean *disjuntos* es lo que garantiza que ninguna se cuenta dos veces. Son las dos mitades de la poscondición Post_0 del Algoritmo 5.1.1 –reportar todas las soluciones, cada una exactamente una vez– traducidas al modo de conteo, donde contar de más es tan grave como contar de menos.

Teorema 5.6.5

Para $N \geq 1$, el llamado `segmentations(A, isWord)` resuelve el Problema 5.6.1.

Demostración

Caso particular del Teorema 5.6.4, tomando $n = 0$.

5.6.7. Análisis. Como no se sabe cómo está implementado *isWord*, no se puede saber cuánto cuesta evaluarlo. El análisis se hace, entonces, contando la cantidad de evaluaciones de *isWord* en el peor caso.

Esta medida no deja por fuera nada relevante. Aparte de evaluar *isWord*, atender una alternativa cuesta una cantidad constante de operaciones –la aritmética sobre los índices, la suma y el llamado recurrente– y ningún llamado se genera sin que antes se haya evaluado *isWord*, de modo que la cantidad de llamados no excede en más de uno a la cantidad de evaluaciones. En consecuencia, si *isWord* costara $\Theta(1)$, el tiempo total del algoritmo sería del mismo orden que la cantidad que aquí se cuenta.

El peor caso es fácil de identificar en el modo de conteo y esa es una de sus ventajas para el análisis: como no hay cortocircuito, el recorrido no depende de cuándo se encuentren las soluciones sino únicamente de cuáles alternativas son factibles. Se recorre más árbol cuantas más lo sean, de modo que el peor caso es aquel en que todo subrango de A es una palabra.

Sea $T(m)$ la cantidad de evaluaciones sobre un sufijo de longitud $m = N - n$ en esa instancia. El caso base no evalúa *isWord*, luego $T(0) = 0$. Para $m \geq 1$ se hacen las m evaluaciones $isWord(n, j)$ para $n < j \leq N$, todas ciertas, y cada una genera un llamado recurrente sobre un sufijo de longitud $m - 1, m - 2, \dots, 0$. Es decir,

$$T(m) = (+l \mid 0 \leq l < m : T(l)) + m.$$

Nota 5.6.7: Recurrencias de historia completa

La recurrencia anterior no tiene la forma $T(m) = aT(m/b) + f(m)$ que exige el teorema maestro del Capítulo 2: el llamado no se descompone en una cantidad fija de subproblemas de tamaño proporcional a m , sino en subproblemas de todos los tamaños menores. A las recurrencias de esta forma se les llama *de historia completa*, y aparecen con naturalidad al analizar algoritmos de reintento en los cuales una decisión puede consumir una porción arbitraria de la entrada.

El recurso usual para resolverlas es escribir la misma recurrencia para $m - 1$ y restar, con lo cual la sumatoria desaparece. Para $m \geq 2$:

$$\begin{array}{rcl} T(m) & = & T(0) + \dots + T(m-2) + T(m-1) + m \\ T(m-1) & = & T(0) + \dots + T(m-2) & + (m-1) \end{array}$$

$$T(m) - T(m-1) = T(m-1) + 1$$

Es decir, $T(m) = 2T(m-1) + 1$ con $T(0) = 0$.

Teorema 5.6.6

En el peor caso, el llamado `segmentations(A, isWord)` con $|A| = N$ realiza exactamente $2^N - 1$ evaluaciones de `isWord`, y en esa instancia la respuesta es 2^{N-1} .

Demostración

Resolviendo la recurrencia $T(m) = 2T(m-1) + 1$ con $T(0) = 0$ se obtiene $T(m) = 2^m - 1$, lo cual se demuestra por inducción sobre m y se propone como ejercicio al lector; el resultado es el caso $m = N$. Para la segunda afirmación, si todo subrango es una palabra entonces toda secuencia creciente de cortes es una segmentación, de modo que hay tantas como subconjuntos de las $N - 1$ posiciones interiores.

El segundo enunciado del Teorema 5.6.6 pone el primero en perspectiva. En esa instancia el algoritmo hace algo menos de dos evaluaciones por cada segmentación que cuenta, lo cual es difícil de mejorar *si se insiste en visitar cada solución*. El problema es que no hay ninguna razón para insistir: la salida es un solo número, no una lista, y nada obliga a alcanzar cada solución por separado para saber cuántas hay. Es la enumeración disfrazada de conteo y es evitable.

Nota 5.6.8: La brecha

El costo es exponencial, pero los subproblemas distintos no lo son: ϕ solo se puede llamar sobre $N + 1$ argumentos distintos y `isWord` solo se puede evaluar sobre $\Theta(N^2)$ pares (n, j) distintos. Todo el costo exponencial proviene de recalculiar una y otra vez los mismos sufijos, como ya se veía en la Figura 6: en el árbol de búsqueda, $\phi(j)$ reaparece bajo cada secuencia de decisiones que consuma exactamente los primeros j caracteres.

El lector habrá reconocido la situación. Es la de la Sección 3.1: una función objetivo recurrente cuyos subproblemas se solapan. Y el remedio es el mismo, la memorización, que aquí baja el costo de $\Theta(2^N)$ a $\Theta(N^2)$ evaluaciones de `isWord`; hacerlo se propone como ejercicio al lector. Note que la mejora no es una aceleración marginal sino un cambio de naturaleza: con memorización se puede contar en tiempo polinomial una cantidad de soluciones que es exponencial, cosa imposible para cualquier algoritmo que las visite una por una.

Esta observación merece enunciarse en general, porque no es privativa de este problema. El reintento y la programación dinámica no son técnicas rivales sino consecutivas: la primera produce una función objetivo recurrente que explora el espacio de soluciones y la segunda la hace eficiente cuando esa exploración repite subproblemas. Lo que decide si la segunda es aplicable es precisamente el paso de *estado auxiliar* de la metodología, según se observó en la Nota 5.6.4: que el estado necesario para seguir adelante se resume en pocos parámetros, de modo que la cantidad de subproblemas distintos sea manejable. Cuando ese resumen no existe –como en el problema de las N reinas, donde el estado es el conjunto de columnas y diagonales ya ocupadas– no queda más que recorrer el árbol.

Ejercicios

1. Demuestre, por inducción sobre m , que la solución de la recurrencia $T(m) = 2T(m-1) + 1$ con $T(0) = 0$ es $T(m) = 2^m - 1$.
2. Verifique experimentalmente el Teorema 5.6.6. Instrumente el Algoritmo 5.6.1 con un contador de evaluaciones de *isWord* y ejecútelo, para $1 \leq N \leq 15$, sobre la instancia en la cual todo subrango es una palabra. Verifique también la cantidad de segmentaciones.
3. Desarrolle la representación por carácter descrita en la Sección 5.6, con $K = N - 1$ decisiones binarias.
 - a) Aplíquelo los pasos de la metodología de la Sección 5.3. En particular, formule el predicado de solución, el de factibilidad, y determine cuál es el estado auxiliar necesario. Ayuda: hace falta recordar dónde empieza el bloque que aún está abierto.
 - b) Explique por qué, en esta representación, el Teorema 5.6.3 deja de valer y hay que verificar algo al llegar a una hoja.
 - c) Impleméntela e instrumentela con un contador de nodos visitados. Verifique el Teorema 5.6.1 sobre las tres entradas del Ejemplo 5.6.1.
4. Demuestre que $f(N) = \phi(0)$ y, más generalmente, que la sucesión f del Teorema 5.6.1 satisface una recurrencia análoga a la de la Definición 5.6.2 pero sobre prefijos en lugar de sufijos. ¿Cuál de las dos conviene más para calcular la respuesta?
5. Modifique el Algoritmo 5.6.1 para que:
 - a) decida únicamente si la cadena es segmentable, deteniéndose apenas encuentre la primera segmentación;
 - b) reporte todas las segmentaciones; y

- c) reporte una segmentación con la menor cantidad posible de palabras.
Para el primer caso, explique qué le ocurre a la Nota 5.6.6: ¿sigue siendo vacío el paso de orden?
6. Agregue memorización al Algoritmo 5.6.1, en el estilo del Algoritmo 3.2.2.
- a) Formule la condición de representación de la memoria compartida.
 - b) Demuestre que la versión con memorización es correcta.
 - c) Demuestre que realiza $O(N^2)$ evaluaciones de *isWord*.
 - d) Verifique experimentalmente la mejora sobre la instancia de peor caso del análisis, para $N = 20, 40, 80$.
7. En el Ejemplo 5.6.2 el predicado *isWord* se implementa consultando un conjunto de cadenas.
- a) ¿Cuánto cuesta esa evaluación? Tenga en cuenta que construir $A[l..r)$ toma tiempo proporcional a $r - l$.
 - b) ¿Cómo afecta esto al tiempo total del algoritmo?
8. La Nota 5.6.6 afirma que en el modo de conteo el orden de las alternativas no cambia la cantidad de nodos visitados. Demuéstrelo.
9. Considere la variante en la cual cada palabra tiene un costo y se busca la segmentación de costo mínimo.
- a) Adapte la Definición 5.6.2.
 - b) En el sentido de la Definición 5.7.1, ¿qué papel jugaría aquí una cota optimista? Proponga una.
 - c) ¿Vuelve a tener sentido el paso de orden?
10. 🌈 Una estructura de datos llamada *trie* permite, dado un índice n , recorrer directamente los índices j tales que $A[n..j)$ es una palabra, sin ensayar los demás. Investíguela e implemente con ella una versión del algoritmo que no evalúe *isWord* sobre alternativas que van a fallar. ¿Qué le ocurre al análisis de esta sección?

5.7. Ramificación y poda: el problema del morral

Todas las podas de las secciones anteriores son de la misma naturaleza: descartan una solución parcial porque no puede conducir a *ninguna* solución. Los problemas de optimización admiten una segunda clase de poda, más poderosa, que descarta una solución parcial porque no puede conducir a una solución *mejor que una ya encontrada*. A la combinación de las dos se le llama *ramificación y poda*.

Nota 5.7.1: Sobre el nombre

En inglés la técnica se conoce como *branch and bound*, donde *branch* alude a la ramificación del árbol de reintento y *bound* a la cota que permite descartar ramas enteras. En castellano circulan *ramificación y poda* y *ramificación y acotación*; se prefiere aquí la primera, que es la más difundida, aunque la segunda nombra mejor el ingrediente nuevo, que es la cota.

El problema que se usa para presentarla ya apareció en el Capítulo 3, resuelto por tabulación.

Problema 5.7.1: El morral

Entrada: Dos arreglos $W[0..N)$ y $V[0..N)$ de números enteros positivos, con los pesos y los valores de N elementos, y un número entero positivo C con la capacidad del morral.

Salida: Un subconjunto de elementos en $[0..N)$ de peso total a lo sumo C y de valor total máximo.

5.7.1. Decisiones y factibilidad. La representación es la de la Sección 5.2: la decisión i -ésima vale 1 si el elemento i se lleva y 0 si no, de modo que $K = N$ y $C[i] = \{0, 1\}$. Conviene nombrar el peso y el valor que una solución parcial acumula.

$$\omega(s) = (+i \mid 0 \leq i < |s| : s_i \cdot W[i]),$$

$$\nu(s) = (+i \mid 0 \leq i < |s| : s_i \cdot V[i]).$$

Una solución parcial completa es una solución cuando cabe en el morral y entre todas ellas se busca la de valor máximo.

$$sol(s) : \omega(s) \leq C.$$

Teorema 5.7.1

El predicado $feasible(s) : \omega(s) \leq C$ es un predicado de factibilidad para el Problema 5.7.1, y además es perfecto en el sentido del Ejercicio 5.

Demostración

Sean t una solución y s una solución parcial con $s \preceq t$. Como los pesos son positivos y s es un prefijo de t , se tiene $\omega(s) \leq \omega(t) \leq C$.

Para la perfección, sea s con $\omega(s) \leq C$ y considere la extensión que no lleva ningún elemento más: su peso sigue siendo $\omega(s) \leq C$, de modo que es una solución que extiende a s .

Nota 5.7.2: Una poda perfecta que no sirve

El Teorema 5.7.1 parece una buena noticia y no lo es. En la Sección 5.2 se celebró que un predicado perfecto garantiza que ningún nodo visitado es trabajo perdido; aquí esa garantía es vacía, porque lo que se busca no es *una* solución sino la *mejor*. Que toda solución parcial factible se extienda a alguna solución no dice nada sobre si alguna de esas extensiones vale la pena.

El efecto práctico es que la poda por factibilidad, por sí sola, apenas ayuda: solo descarta las soluciones parciales que ya se pasaron de peso. La tabla del análisis, al final de la sección, muestra que para $N = 20$ eso deja sin recorrer poco más de la mitad del árbol. Para atacar un problema de optimización hace falta otra cosa.

5.7.2. Poda por optimalidad. La idea es llevar registro de la mejor solución encontrada hasta el momento y usar su valor para descartar ramas. Para ello se necesita poder afirmar, en un nodo, que nada de lo que cuelga de él supera ese valor. Eso es lo que hace una cota.

Definición 5.7.1: Cota optimista

Una función B de las soluciones parciales en los números reales es una *cota optimista* para un problema de maximización con función objetivo ν si y solo si, para toda solución t y toda solución parcial s ,

$$s \preceq t \implies \nu(t) \leq B(s).$$

La forma de la definición es deliberadamente la misma de la Definición 5.1.4, y la obligación que impone es igual de asimétrica: la cota debe ser *optimista*, es decir, nunca subestimar lo que se puede alcanzar. Sobreestimar solo cuesta trabajo desperdiciado; subestimar pierde soluciones.

Teorema 5.7.2

Sea B una cota optimista y considere el algoritmo de reintento que mantiene una variable *best* con el valor de la mejor solución hallada hasta el momento,

inicializada con el valor de alguna solución conocida y que poda toda solución parcial s con $B(s) \leq best$. Entonces, al terminar, $best$ es el valor óptimo del problema.

Demostración

Como $best$ solo se actualiza con valores de soluciones efectivamente halladas, en todo momento $best$ es el valor de alguna solución y por lo tanto $best \leq OPT$, donde OPT denota el valor óptimo. Resta demostrar la otra desigualdad.

Sea t^* una solución con $\nu(t^*) = OPT$. Hay dos casos. Si el algoritmo llegó a visitar la hoja t^* , entonces actualizó $best$ con $\nu(t^*)$ o ya tenía un valor mayor, y en cualquier caso el $best$ final es al menos OPT . Si no la visitó, es porque algún prefijo s de t^* fue podado. La poda por factibilidad no pudo ser la causa, pues *feasible* es cierto en todo prefijo de una solución; luego fue la poda por optimalidad, y en ese instante se cumplía $B(s) \leq best$. Como B es una cota optimista y $s \preceq t^*$, se tiene $OPT = \nu(t^*) \leq B(s) \leq best$. Y como $best$ nunca decrece, el valor final también es al menos OPT .

Nota 5.7.3: Estado que sobrevive al deshacer

Hasta ahora, todo el estado que un algoritmo de reintento modificaba al elegir se restauraba al deshacer; era la poscondición $Post_1$ del Algoritmo 5.1.1 y la Nota 5.1.3 insistía en no olvidar ninguna restauración. La variable $best$ rompe esa regla deliberadamente: no se restaura, y no debe restaurarse.

Esa es, de hecho, toda la diferencia entre el reintento y la ramificación y poda. En el reintento las ramas del árbol son independientes: lo que se aprende explorando una no sirve para ninguna otra y por eso el estado puede restaurarse íntegramente. En la ramificación y poda, en cambio, una rama informa a las siguientes y $best$ es el canal por el que viaja esa información. La poscondición correcta para esta sección es

$Post'_1$: al retornar, todo el estado es idéntico al de la invocación,
salvo $best$, que solo puede haber mejorado.

De aquí se sigue también que el orden en que se recorre el árbol deja de ser indiferente. En el reintento puro, cambiar el orden cambia la secuencia en que aparecen las soluciones pero no la cantidad de nodos visitados. Aquí sí la cambia, y mucho: encontrar pronto una buena solución hace que $best$ suba

pronto y que la poda actúe con más fuerza sobre todo lo que falta. Al final de la sección se cuantifica ese efecto.

5.7.3. La cota: relajación fraccionaria. Falta lo esencial, que es exhibir una cota optimista que sea a la vez ajustada y barata de calcular. La Nota 5.5.2 sugirió el camino: reemplazar el problema que falta por resolver por otro más simple que lo contenga.

El problema que falta por resolver, en un nodo s con $|s| = k$, es llenar una capacidad $R = C - \omega(s)$ con los elementos $[k..N]$. La relajación consiste en permitir llevar *fracciones* de un elemento.

Ese problema relajado ya está resuelto: es el Problema 4.6.1 de la Sección 4.6, planteado allí sobre los elementos $[0..N]$ y aquí sobre el sufijo $[k..N]$ con capacidad R .

A diferencia del original, ese problema sí cede ante una estrategia voraz: se llevan los elementos en orden decreciente de *densidad* $V[i]/W[i]$, completos mientras quepan, y del primero que no quepa se lleva la fracción que alcance. Es el Algoritmo 4.6.1, y el Teorema 4.6.3 establece que resuelve el problema siempre que los elementos estén ordenados descendientemente por densidad. La Figura 8 lo ilustra.

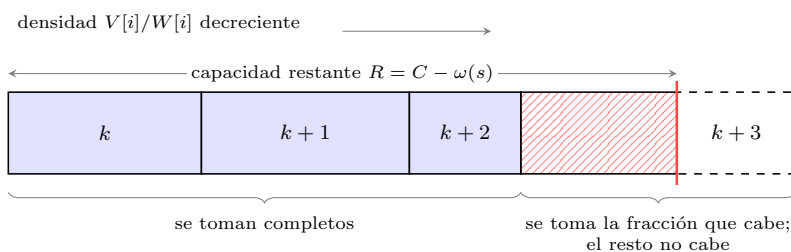


Figura 8. La solución del morral fraccionario para los elementos que faltan por decidir. Es esta cantidad, sumada a $\nu(s)$, la que sirve de cota optimista.

Nota 5.7.4: De dónde viene esta cota

Vale la pena detenerse en lo que este préstamo significa para el libro. El problema del morral es resistente a la estrategia voraz: llevar los elementos por densidad no resuelve el Problema 5.7.1, como lo muestra el Ejemplo 4.6.1. Relajado a fracciones, en cambio, sí cede, y lo hace ante un criterio de selección –la densidad– y un argumento de intercambio. Esa solución voraz de un problema que nadie pidió resolver es precisamente lo que permite podar el

reintento sobre el problema que sí interesa, el mismo que el Capítulo 3 resolvió por tabulación.

Es un patrón general y conviene retenerlo: cuando un problema resiste a una técnica, relajarlo hasta que ceda ante otra suele producir la cota que la primera necesitaba.

Teorema 5.7.3

Sea s una solución parcial con $|s| = k$ y $\omega(s) \leq C$, y sea $F(k, R)$ el valor óptimo del Problema 4.6.1 sobre los elementos $[k..N)$ con capacidad R . Entonces

$$B(s) = \nu(s) + F(k, C - \omega(s))$$

es una cota optimista para el Problema 5.7.1.

Demostración

Sea t una solución con $s \preceq t$. Los valores $t_k, \dots, t_{N-1}$ pertenecen a $\{0, 1\} \subseteq [0..1]$ y satisfacen

$$(+i \mid k \leq i < N : t_i \cdot W[i]) = \omega(t) - \omega(s) \leq C - \omega(s),$$

de modo que constituyen una solución factible del Problema 4.6.1 sobre los elementos $[k..N)$ con capacidad $C - \omega(s)$. Su valor es entonces a lo sumo el óptimo de ese problema, es decir, $\nu(t) - \nu(s) \leq F(k, C - \omega(s))$, que reordenado es $\nu(t) \leq B(s)$.

5.7.4. Implementación. El Teorema 4.6.3 exige que los elementos estén ordenados por densidad decreciente. Conviene ordenarlos una sola vez, antes de comenzar: así, en cualquier nodo, los elementos que faltan por decidir –que son siempre un sufijo– ya están en el orden que la cota necesita, y calcularla es un recorrido lineal.

Algoritmo 5.7.1: Cota optimista

```

1 def bound(W, V, C, k, weight, value):
2   N = len(W)
3   b = value
4   r = C - weight
5   j = k
6   while j < N and W[j] <= r:
```

```

7     b = b + V[j]
8     r = r - W[j]
9     j = j+1
10    if j<N: b = b + V[j]*r/W[j]
11    return b

```

El ciclo de la línea 6 lleva elementos completos mientras quepan y la línea 10 agrega la fracción del primero que no cabe. El algoritmo principal mantiene *best* en una lista de dos posiciones, con el mejor valor y la mejor solución halladas; se pasa por referencia, como la memoria compartida del Algoritmo 3.2.2, para que las mejoras sobrevivan a los retornos.

Algoritmo 5.7.2: El morral por ramificación y poda

```

1 def knapsack(W, V, C, s, weight, value, best):
2     N = len(W)
3     if len(s)==N:
4         if value>best[0]: best[0], best[1] = value, s[:]
5     else:
6         k = len(s)
7         for c in [1, 0]:
8             w = weight + c*W[k]
9             v = value + c*V[k]
10            if w<=C and bound(W, V, C, k+1, w, v)>best[0]:
11                s.append(c) # choose
12                knapsack(W, V, C, s, w, v, best)
13                s.pop() # undo

```

La guarda de la línea 10 reúne las dos podas: la comparación $w \leq C$ es la factibilidad, y la de *bound* contra *best*[0] es la optimalidad. El ciclo de la línea 7 recorre los candidatos en el orden [1, 0], es decir, ensayando primero llevar el elemento; la razón se explica en el análisis.

Algoritmo 5.7.3: Arranque

```

1 def best_knapsack(W, V, C):
2     order = sorted(range(len(W)), key=lambda i: -V[i]/W[i])
3     Ws = [ W[i] for i in order ]
4     Vs = [ V[i] for i in order ]
5     best = [0, None]
6     knapsack(Ws, Vs, C, [], 0, 0, best)

```

```
7 return best[0], [ order[i] for i in range(len(Ws)) if best[1][i]==1 ]
```

El arranque ordena por densidad decreciente, ejecuta la búsqueda y traduce la solución hallada de vuelta a los índices originales. Inicializar `best[0]` en 0 es correcto porque el subconjunto vacío siempre es una solución y su valor es 0.

Ejemplo 5.7.1

```
1 W = [12, 2, 1, 1, 4]
2 V = [4, 2, 2, 1, 10]
3 print(best_knapsack(W, V, 15))
```

El resultado de esta ejecución es el siguiente:

```
1 (15, [4, 2, 1, 3])
```

Es decir, se llevan los elementos 4, 2, 1 y 3, de pesos 4, 1, 2 y 1 y valores 10, 2, 2 y 1: peso total 8 y valor total 15. Note que el elemento 0, el más pesado, se queda por fuera pese a que aún sobra capacidad.

Teorema 5.7.4

El llamado `best_knapsack(W, V, C)` calcula el valor óptimo del Problema 5.7.1 y un subconjunto que lo alcanza.

Demostración

Se propone como ejercicio al lector. Ayuda: combine el Teorema 5.7.2, el Teorema 5.7.3 y el Teorema 4.6.3, y verifique que reordenar los elementos no cambia el conjunto de soluciones del problema.

5.7.5. Análisis. Ordenar cuesta $O(N \log N)$. En cada nodo, evaluar la cota cuesta $O(N)$ en el peor caso, de modo que el trabajo por nodo es $O(N)$.

Teorema 5.7.5

El llamado `best_knapsack(W, V, C)` con $|W| = N$ visita a lo sumo $2^{N+1} - 1$ nodos, toma tiempo $O(N \cdot 2^N)$ y usa espacio $O(N)$.

Demostración

Se propone como ejercicio al lector.

De nuevo la cota de peor caso no distingue entre podar bien y no podar nada, y hay que medir. La siguiente tabla reporta la cantidad promedio de nodos visitados sobre veinte instancias aleatorias por fila, con pesos y valores en $[1..100]$ y capacidad igual a la mitad del peso total.

N	Sin poda	Solo factibilidad	Con cota	Con cota y orden
12	8 191	4 779	172	22
16	131 071	76 754	406	27
20	2 097 151	1 191 407	653	48
24	33 554 431	—	1 248	52
28	536 870 911	—	2 126	53
32	8 589 934 591	—	3 338	74

Las casillas vacías corresponden a mediciones que no se hicieron porque tomaban demasiado tiempo, lo cual ya dice algo: la poda por factibilidad deja un algoritmo que no sirve. La tercera columna confirma lo anunciado en la Nota 5.7.2: para $N = 20$ apenas evita recorrer poco más de la mitad del árbol.

La cuarta columna es la que sorprende. Agregar la cota optimista lleva el recorrido de más de un millón de nodos a menos de setecientos, y a partir de ahí el crecimiento con N es casi lineal. Nada de esto contradice al Teorema 5.7.5: siguen existiendo instancias que obligan a recorrer todo el árbol, pero no son las que uno encuentra al azar.

Nota 5.7.5: Las dos mitades del orden

La última columna difiere de la anterior en el paso de *orden* de la metodología de la Sección 5.3, que tiene dos mitades: en qué orden se toman las decisiones y en qué orden se recorren los candidatos de cada una. Aquí las dos están acopladas.

Ordenar los elementos por densidad decreciente hace que las decisiones importantes se tomen primero. Recorrer los candidatos en el orden $[1, 0]$ hace que la primera hoja que el algoritmo alcanza sea la solución voraz, que suele ser buena, con lo cual *best* arranca alto y la poda muerde desde el principio. Juntas bajan el recorrido de 1 248 nodos a 52 para $N = 24$.

Lo interesante es qué pasa si se aplica solo una. Ordenar por densidad pero recorrer los candidatos en el orden $[0, 1]$ significa empezar descartando el

elemento más prometedor y para $N = 24$ arroja 3 454 nodos: casi el triple que no ordenar nada. Las dos mitades del paso de orden no son independientes, y acertar en una mientras se yerra en la otra puede ser peor que no hacer nada.

Nota 5.7.6: Ramificación y poda contra tabulación

Como en la Nota 5.5.4, conviene comparar con la solución del Capítulo 3. La tabulación resuelve el problema en tiempo $\Theta(N \cdot C)$, que es excelente cuando la capacidad es pequeña y desastroso cuando es grande: con C del orden de 10^{12} la tabla no cabe en ninguna memoria, por pequeño que sea N . La ramificación y poda es indiferente a C , y también a que los pesos y valores sean enteros, pero su peor caso es exponencial en N .

Hay además una diferencia de naturaleza. La tabulación siempre paga su costo completo; la ramificación y poda paga lo que la instancia le exija, y la tabla anterior sugiere que las instancias aleatorias le exigen poco. Esa es también su debilidad: no ofrece ninguna garantía sobre cuánto tardará en una instancia concreta antes de ejecutarla.

Ejercicios

1. Demuestre el Teorema 5.7.5.
2. Demuestre el Teorema 5.7.4.
3. Demuestre que el Algoritmo 5.7.1 calcula $\nu(s) + F(k, C - \omega(s))$ cuando los elementos están ordenados por densidad decreciente. Formule los invariantes del ciclo.
4. El Teorema 5.7.2 poda cuando $B(s) \leq best$.
 - a) Demuestre que con esa condición el algoritmo encuentra *una* solución óptima, pero puede no encontrar todas.
 - b) Modifíquelo para que reporte todas las soluciones óptimas. ¿Qué le ocurre a la cantidad de nodos visitados?
5. Considere la cota optimista trivial $B(s) = \nu(s) + (+i \mid |s| \leq i < N : V[i])$, que supone que todo lo que falta cabe en el morral.
 - a) Demuestre que es una cota optimista.
 - b) Demuestre que nunca es mejor que la del Teorema 5.7.3.
 - c) Mídala. ¿Cuánto se pierde por usarla?
6. Reproduzca la tabla de esta sección y la comparación de la Nota 5.7.5. Agregue una columna con instancias en las cuales el valor de cada elemento es proporcional a su peso. Explique lo que observa.
7. Una variante de la implementación evalúa la cota una sola vez por nodo, antes del ciclo de candidatos, en lugar de una vez por candidato. Implementéla, explique por qué sigue siendo correcta y compare la cantidad de evaluaciones de la cota.
8. El Ejemplo 5.7.1 deja por fuera el elemento más pesado pese a que sobra capacidad. Construya una instancia en la cual la solución óptima deje por fuera el elemento de mayor densidad y explique por qué eso no contradice al Teorema 4.6.3.
9. Adapte la ramificación y poda al problema de la suma exacta de la Sección 5.5, en su versión de optimización: hallar el subconjunto cuya suma se acerque más a X sin pasarse.
 - a) Proponga una cota optimista y demuestre que lo es.
 - b) Implementéla y compare con el algoritmo de la Sección 5.5.
10. 🌈 El recorrido en profundidad no es el único posible. La variante conocida como *el mejor primero* mantiene una cola de prioridad con los nodos vivos y expande siempre aquel cuya cota optimista sea mayor.
 - a) Implementéla.
 - b) Demuestre que el primer nodo completo que expande es óptimo.

- c) Compare la cantidad de nodos visitados y el espacio utilizado con los del algoritmo de esta sección. ¿Cuál prefiere?

5.8. El agente viajero, revisitado

La última sección del capítulo recoge casi todo lo anterior. El problema es el del agente viajero, ya resuelto en el Capítulo 3 por memorización sobre máscaras de bits; la representación es la de las permutaciones de la Sección 5.2; la técnica es la ramificación y poda de la Sección 5.7; y la cota, como allí, proviene de un algoritmo voraz del Capítulo 4, esta vez el del árbol de cubrimiento mínimo.

Problema 5.8.1: El agente viajero

Entrada: Una matriz $D[0..N][0..N]$ de números no negativos, con $D[i][j]$ el costo de ir de la ciudad i a la ciudad j .

Salida: Un recorrido que visite cada una de las N ciudades exactamente una vez y regrese a la de partida, de costo total mínimo.

5.8.1. Decisiones. Un recorrido es un ciclo que pasa por todas las ciudades, de modo que se puede describir dando el orden en que se visitan. Hay una redundancia que conviene eliminar de entrada.

Teorema 5.8.1

Todo recorrido puede describirse comenzando por la ciudad 0, y dos descripciones que difieran solo en la ciudad de partida tienen el mismo costo.

Demostración

Un recorrido es un ciclo, y rotar la secuencia que lo describe no cambia el conjunto de arcos recorridos: los pares de ciudades consecutivas, incluido el que cierra el ciclo, son los mismos. Como el costo es la suma de los costos de esos arcos, no cambia. Rotando hasta que la ciudad 0 quede al principio se obtiene la descripción buscada.

Fijar la ciudad de partida reduce el espacio de búsqueda de $N!$ descripciones a $(N-1)!$ recorridos distintos, sin perder ninguno. Hecho eso, la decisión k -ésima es qué ciudad se visita en el lugar $k+1$ del recorrido: $K = N-1$ y $C[k] = \{1, \dots, N-1\}$

para todo k . Una solución parcial s describe el camino

$$0 \rightarrow s_0 \rightarrow s_1 \rightarrow \cdots \rightarrow s_{|s|-1},$$

y su costo es

$$\gamma(s) = D[0][s_0] + (+i \mid 0 \leq i < |s| - 1 : D[s_i][s_{i+1}]),$$

con $\gamma(\langle \rangle) = 0$. Una solución parcial completa describe un recorrido de costo $\gamma(s) + D[s_{N-2}][0]$.

Como en la Sección 5.2, el predicado de solución exige que las ciudades sean distintas par a par, y el predicado de factibilidad es el mismo, sostenido por un arreglo *used*[0.. N] de Booleanos. Esa poda, por sí sola, deja el algoritmo recorriendo el árbol de permutaciones completo: es la fuerza bruta que el Ejercicio 8 propone comparar con la memorización del Capítulo 3. Lo que sigue es lo que hace la diferencia.

5.8.2. La cota: relajación a un árbol. Se trata de un problema de minimización, de modo que la Definición 5.7.1 se dualiza: una cota optimista es ahora una cota *inferior* sobre el costo de cualquier recorrido que extienda a la solución parcial, y se poda cuando esa cota alcanza o supera el costo del mejor recorrido ya encontrado.

La cota más simple es $\gamma(s)$ misma: como los costos son no negativos, ningún recorrido que extienda a s cuesta menos que el camino ya construido. Es válida y gratuita, pero ignora por completo lo que falta por recorrer.

Para hacer algo mejor conviene mirar qué es, exactamente, lo que falta. Sea $v = s_{|s|-1}$ la última ciudad visitada y sea R el conjunto de ciudades que aún no se visitan. Completar la solución parcial es recorrer un camino que salga de v , pase por todas las ciudades de R y termine en 0. Ese camino es, en particular, un subgrafo conexo y sin ciclos que toca todos los vértices de $R \cup \{v, 0\}$: es un árbol de cubrimiento de ese conjunto. Y de todos los árboles de cubrimiento, ninguno pesa menos que el mínimo.

Teorema 5.8.2

Sea s una solución parcial no vacía, $v = s_{|s|-1}$, y R el conjunto de ciudades que no aparecen en s . Sea $acm(U)$ el peso de un árbol de cubrimiento mínimo del subgrafo inducido por el conjunto de ciudades U . Entonces

$$B(s) = \gamma(s) + acm(R \cup \{v, 0\})$$

es una cota inferior del costo de todo recorrido que extienda a s .

Demostración

Sea t una solución parcial completa que extiende a s y sea c el costo del recorrido que describe. Ese recorrido se compone del camino $0 \rightarrow \dots \rightarrow v$, de costo $\gamma(s)$, y del camino que sale de v , recorre las ciudades de R en algún orden y regresa a 0. Llámese Q a este segundo camino y q a su costo, de modo que $c = \gamma(s) + q$.

El camino Q visita exactamente los vértices de $R \cup \{v, 0\}$, cada uno una vez, y sus arcos lo conectan sin formar ciclos. Es decir, Q es un árbol de cubrimiento del subgrafo inducido por $R \cup \{v, 0\}$. Por definición de árbol de cubrimiento mínimo, $q \geq \text{acm}(R \cup \{v, 0\})$, y por lo tanto $c \geq \gamma(s) + \text{acm}(R \cup \{v, 0\}) = B(s)$.

La Figura 9 muestra por qué la cota es, en efecto, optimista y no exacta: el árbol de cubrimiento mínimo se ramifica, y un camino no puede hacerlo. La relajación consiste precisamente en haber olvidado esa restricción.

Nota 5.8.1: El mismo patrón, otra vez

Vale la pena poner las dos últimas secciones una al lado de la otra. En ambas, el problema resiste a la estrategia voraz. En ambas, relajar una restricción produce un problema que sí cede ante ella: permitir fracciones convierte el morral en un problema voraz, y permitir ramificaciones convierte el recorrido en un árbol de cubrimiento mínimo. Y en ambas, el óptimo del problema relajado es la cota que el reintento necesitaba.

La Nota 5.7.4 enunciaba el patrón; esta sección es la segunda evidencia de que no era una casualidad del morral.

5.8.3. Implementación.**Algoritmo 5.8.1: Cota inferior por árbol de cubrimiento mínimo**

```

1 def bound(D, cost, last, rest):
2     verts = rest + [last, 0]
3     if len(verts)>1:
4         inside = { verts[0] }
5         dist = { v: D[verts[0]][v] for v in verts[1:] }
```

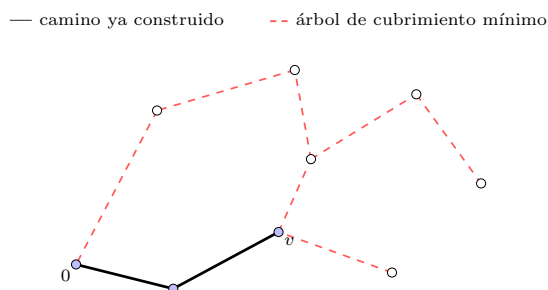


Figura 9. La cota inferior en un nodo del árbol de reintento. El camino ya construido va de 0 a v ; lo que falta por recorrer es un camino de v a 0 que pase por las demás ciudades, y todo camino de esos pesa al menos lo que el árbol de cubrimiento mínimo.

```

6  while dist:
7      u = min(dist, key=dist.get)      # the one closest to the tree
8      cost = cost + dist.pop(u)
9      inside.add(u)
10     for v in dist:
11         if D[u][v]<dist[v]: dist[v] = D[u][v]
12     return cost

```

El Algoritmo 5.8.1 es el algoritmo de Prim del Capítulo 4, en su variante sobre matriz de pesos (Ejercicio 21 de la Sección 4.7), restringido al subgrafo inducido por las ciudades que interesan y acumulando el peso sobre el costo ya recorrido. Esa es la variante que corresponde aquí y no la del montón binario: el grafo del agente viajero es completo, y para grafos densos la versión sobre matriz es la más rápida de las tres que compara la Sección 4.7.

Algoritmo 5.8.2: El agente viajero por ramificación y poda

```

1  def tsp(D, s, used, last, cost, best):
2      N = len(D)
3      if len(s)==N-1:
4          total = cost + D[last][0]
5          if total<best[0]: best[0], best[1] = total, s[:]
6      else:
7          for c in range(1, N):
8              if not used[c]:
9                  rest = [ v for v in range(1,N) if not used[v] and v!=c ]
10                 if bound(D, cost+D[last][c], c, rest)<best[0]:
11                     used[c] = True      # choose

```

```

12         s.append(c)
13         tsp(D, s, used, c, cost+D[last][c], best)
14         s.pop()                                # undo
15         used[c] = False

```

Algoritmo 5.8.3: Arranque

```

1 def best_tour(D):
2     N = len(D)
3     best = [ float('inf'), None ]
4     tsp(D, [], [False]*N, 0, 0, best)
5     return best[0], [0] + best[1]

```

Inicializar `best[0]` en infinito es correcto, aunque no es lo mejor que se puede hacer: cualquier recorrido conocido de antemano sirve como valor inicial y hace que la poda actúe desde el primer nodo. Obtener uno con una heurística barata se propone como ejercicio al lector.

Teorema 5.8.3

El llamado `best_tour(D)` calcula el costo mínimo de un recorrido y un recorrido que lo alcanza.

Demostración

Se propone como ejercicio al lector. Ayuda: combine el Teorema 5.8.1, el Teorema 5.8.2 y la versión para minimización del Teorema 5.7.2.

5.8.4. Análisis. Todo nodo visitado es una solución parcial con ciudades distintas par a par, de modo que el árbol recorrido está contenido en el de las permutaciones de $N - 1$ elementos. Evaluar la cota cuesta $O(N^2)$.

Teorema 5.8.4

El llamado `best_tour(D)` visita a lo sumo $\sum_{k=0}^{N-1} \frac{(N-1)!}{(N-1-k)!}$ nodos, toma tiempo $O(N^2 \cdot N!)$ y usa espacio $O(N)$.

Demostración

Se propone como ejercicio al lector.

La siguiente tabla reporta la cantidad promedio de nodos visitados sobre instancias aleatorias con ciudades ubicadas al azar en un cuadrado y costos iguales a las distancias euclidianas.

N	Recorridos	Solo factibilidad	Cota trivial	Cota por ACM
8	5 040	13 700	3 632	162
10	362 880	986 410	66 298	562
12	39 916 800	108 505 112	1 497 034	4 910
14	6 227 020 800	16 926 797 486	—	8 559
16	1 307 674 368 000	3 554 627 472 076	—	37 338
18	355 687 428 096 000	966 858 672 404 690	—	114 149

La segunda columna es la cantidad de recorridos distintos, $(N-1)!$, y la tercera la cantidad de nodos del árbol de permutaciones, que es lo que recorre el algoritmo si solo se poda por factibilidad; ambas son analíticas y se siguen del Teorema 5.2.3. Las dos últimas son medidas, y las casillas vacías corresponden a mediciones que no se hicieron porque tomaban demasiado tiempo.

La cota trivial, que no mira siquiera lo que falta por recorrer, ya reduce el trabajo en dos órdenes de magnitud. La cota por árbol de cubrimiento mínimo lo reduce en otros tres: para $N = 18$ visita cien mil nodos de un árbol que tiene casi 10^{15} .

Nota 5.8.2: Contra la memorización del Capítulo 3

La solución del Capítulo 3 toma tiempo $O(2^N N^2)$ y espacio $O(2^N N)$. La de esta sección tiene un peor caso mucho peor, $O(N^2 \cdot N!)$, y sin embargo la tabla muestra que sobre instancias aleatorias visita muchos menos nodos de los que la memorización tiene entradas en su tabla. ¿Cuál preferir?

La respuesta depende de qué se tema más. La memorización ofrece una garantía: nunca tardará más de $O(2^N N^2)$, cualquiera que sea la instancia. Lo que cobra por esa garantía es espacio, y lo cobra siempre: para $N = 30$ su tabla tiene más de 3×10^{10} entradas y no cabe en ninguna memoria, por fácil que sea la instancia. La ramificación y poda usa espacio $O(N)$ y suele ser rapidísima, pero no ofrece ninguna garantía: existen instancias que la obligan a recorrerlo todo, y no hay manera barata de saber de antemano si la instancia que se tiene enfrente es una de ellas.

Es el mismo dilema de la Nota 5.5.4 y de la Nota 5.7.6, y aparece una y otra vez al final de este capítulo por una razón de fondo: los problemas que aquí se atacan son, según lo que se estudiará en el Capítulo 7, problemas para los cuales no se espera una solución eficiente. Ante ellos no hay un algoritmo mejor, sino una elección entre garantías distintas.

Ejercicios

1. Demuestre el Teorema 5.8.4.
2. Demuestre el Teorema 5.8.3. En particular, enuncie y demuestre la versión para minimización del Teorema 5.7.2.
3. Demuestre que la cota trivial $B(s) = \gamma(s)$ es una cota inferior válida y que la del Teorema 5.8.2 nunca es menor que ella.
4. El Algoritmo 5.8.3 inicializa `best[0]` en infinito.
 - a) Diseñe una heurística barata que produzca algún recorrido, por ejemplo visitando siempre la ciudad más cercana entre las no visitadas.
 - b) Úsela para inicializar `best` y mida el efecto sobre la cantidad de nodos visitados.
 - c) ¿Sigue siendo correcto el algoritmo si la heurística produce un recorrido cualquiera, no necesariamente bueno?
5. Cuando la matriz D es simétrica, todo recorrido y su inverso tienen el mismo costo, de modo que la tabla de esta sección cuenta cada recorrido dos veces.
 - a) Proponga una restricción sobre las decisiones que elimine esa redundancia y demuestre que no pierde ninguna solución óptima.
 - b) Impleméntela y mida el ahorro.
6. Reproduzca la tabla de esta sección. Agregue una columna con instancias en las cuales los costos son enteros aleatorios sin relación con ninguna geometría, es decir, sin desigualdad triangular. ¿Sigue siendo buena la cota por árbol de cubrimiento mínimo? Explique.
7. El Algoritmo 5.8.2 evalúa la cota una vez por candidato, y cada evaluación reconstruye el conjunto `rest` y ejecuta a Prim desde cero.
 - a) ¿Cuál es el costo total por nodo?
 - b) Proponga e implemente una versión que evite reconstruir `rest` en cada candidato.
8. 🌈 Una cota inferior mejor que la del Teorema 5.8.2 se obtiene observando que el camino que falta no solo cubre $R \cup \{v, 0\}$ sino que además debe salir de v y llegar a 0.

- a) Demuestre que $\gamma(s) + acm(R) + m_v + m_0$ es una cota inferior válida, donde m_v y m_0 son los costos mínimos de un arco entre v y R , y entre 0 y R , respectivamente.
 - b) Compárela experimentalmente con la de esta sección.
9. 🌐 Investigue la cota de Held y Karp, que refina la de esta sección ajustando iterativamente los costos de los arcos sin cambiar el conjunto de recorridos óptimos. Implementela y compare.

Notas del capítulo y referencias

El procedimiento que este capítulo describe es mucho más antiguo que su nombre. Bitner y Reingold [BR75] le atribuyen a Derrick H. Lehmer haber usado por primera vez el término *backtrack*, en la década de 1950, en el contexto de sus trabajos sobre problemas combinatorios en computadores digitales [Leh57]. La primera descripción sistemática de la técnica como método general de enumeración se debe a Walker [Wal60], y su establecimiento como técnica de diseño con derecho propio a Golomb y Baumert [GB65], quienes introdujeron buena parte del vocabulario que aún se usa, incluyendo la idea de podar con condiciones necesarias evaluadas sobre soluciones parciales. El artículo de Bitner y Reingold [BR75] es una buena lectura complementaria: presenta una historia breve de la técnica y varias mejoras de implementación.

La Sección 5.2 aborda la generación de objetos combinatorios por reintento, es decir, de manera recurrente. Hay una alternativa que este texto no desarrolla y que vale la pena conocer: generar los objetos *iterativamente*, mediante algoritmos que calculan el sucesor de un objeto dado en un orden fijado de antemano, o que traducen entre objetos y sus posiciones en ese orden. El texto de Kreher y Stinson [KS98] es una referencia recomendada. Sus capítulos 2 y 3 tratan la generación de subconjuntos, k -subconjuntos, permutaciones y otras familias con esta aproximación, y su capítulo 4 está dedicado a los algoritmos de reintento. La Nota 5.2.1 de este capítulo, que genera subconjuntos recorriendo los enteros de 0 a $2^N - 1$, es el caso más simple de esa familia de técnicas.

El problema de las N reinas de la Sección 5.4 nació como un pasatiempo. En 1848, el compositor de ajedrez Max Bezzel planteó en la *Berliner Schachzeitung* la pregunta por las ocho reinas, y en 1850 Franz Nauck publicó las 92 soluciones y propuso la generalización a N . Por esos mismos años Gauss se ocupó del problema en su correspondencia con Schumacher, sin llegar a resolverlo. Que la lista de Nauck estuviera completa solo quedó demostrado en 1874. La cantidad de soluciones forma la sucesión A000170 de la *On-Line Encyclopedia of Integer Sequences* [OEI]; no se conoce una fórmula cerrada para ella y los valores exactos se han calculado únicamente hasta $N = 27$. El estudio de Bell y Stevens [BS09] recopila lo que

se sabe sobre el problema y sus variantes, incluida la toroidal del Ejercicio 8 de esa sección. Vale la pena mencionar, por su cercanía con el estilo de este texto, que Dijkstra usó este mismo problema para ilustrar lo que llamó programación estructurada [Dij72], con una derivación detallada de un algoritmo de reintento en profundidad.

El problema de la Sección 5.5 y el de la Sección 5.7 son los dos únicos del capítulo que aparecen tres veces en el libro: resueltos por tabulación en el Capítulo 3, por reintento aquí, y demostrados NP-completos en el Capítulo 7. El algoritmo de tiempo $O(2^{N/2})$ que propone el Ejercicio 11 de la Sección 5.5 se debe a Horowitz y Sahni [HS74]; es un buen ejemplo de que reconocer un problema como NP-completo no cierra la puerta a mejoras significativas, aunque no las lleve hasta el terreno polinomial. Para el problema del morral en particular, el tratado de Martello y Toth [MT90] recoge las técnicas exactas y aproximadas conocidas hasta su publicación.

La Sección 5.6 no tiene una referencia obligada: el problema es un ejercicio clásico y su interés en este capítulo es estructural. Su función es exhibir el caso en que las decisiones tienen una cantidad variable de alternativas y, sobre todo, el caso en que el estado necesario para continuar se resume en un solo parámetro. Esa es la condición que hace aplicable la memorización del Capítulo 3, como se discute en la Nota 5.6.8, y el contraste con el problema de las N reinas —donde el estado es el conjunto de columnas y diagonales ocupadas, y no se deja resumir— es lo que da su lugar a la sección.

La ramificación y poda de la Sección 5.7 se le atribuye usualmente a Land y Doig [LD60], quienes la presentaron para programación entera; el nombre *branch and bound*, sin embargo, aparece por primera vez en el trabajo de Little, Murty, Sweeney y Karel [LSMK63], que la aplicó al problema del agente viajero. La reseña de Lawler y Wood [LW66] sistematizó la técnica pocos años después y sigue siendo una lectura provechosa.

La cota por árbol de cubrimiento mínimo de la Sección 5.8 es una versión simplificada de la que Held y Karp introdujeron en un par de artículos [HK70, HK71]. Su cota no usa árboles de cubrimiento sino 1-árboles, y sobre todo la refina ajustando iterativamente los costos de los arcos mediante una transformación que deja invariante el conjunto de recorridos óptimos pero cambia el árbol mínimo; es la que propone investigar el Ejercicio 9 de esa sección. Para el estado actual del problema, el libro de Applegate, Bixby, Chvátal y Cook [ABCC06] documenta el diseño del programa Concorde, con el cual se han resuelto de manera exacta instancias de decenas de miles de ciudades; es una demostración notable de hasta dónde puede llegar la ramificación y poda cuando la cota es lo bastante buena.

Queda un asunto que este capítulo plantea sin resolver. La Nota 5.1.4 señala que el análisis asintótico es ciego a la calidad de la poda y propone medir empíricamente el tamaño del árbol recorrido, que es lo que hacen las tablas de las secciones siguientes. Medir, sin embargo, exige recorrer el árbol, que es precisamente lo que se querría evitar. Knuth [Knu75] propuso un método para *estimar* el tamaño de un árbol de reintento sin recorrerlo, mediante un descenso aleatorio desde la raíz que produce un estimador insesgado. Es una idea sorprendente y de lectura accesible, y permite decidir si vale la pena lanzar una búsqueda antes de lanzarla.

Problemas y algoritmos de decisión

Los computadores son máquinas fascinantes: posiblemente sean uno de los inventos más disruptivos de la humanidad después del lenguaje y la escritura. Desde su invención, han evolucionado tremendamente en relación con su poder de cómputo, y de la diversidad de tareas que pueden hacer o en las cuales pueden asistir. A pesar de esta notoria evolución, el potencial de lo que pueden hacer (o no hacer) no ha cambiado: hay un límite que no podrán rebasar aún teniendo a disposición más memoria, procesadores más veloces y almacenamiento casi ilimitado. Este capítulo resume una historia acerca del significado de la palabra “computabilidad”, de lo que significa computar y de los límites del poder de los computadores. Esta historia se remonta a las etapas más tempranas de las ciencias de la computación (circa 1900) en donde se plantearon preguntas de impacto profundo acerca de la existencia de *algoritmos* o *procedimientos computacionales efectivos* para resolver cierto tipo de problemas.

Este capítulo concluye que, aún en el contexto restringido de los problemas de decisión, hay tareas que no pueden ser resueltas por un algoritmo implantado en un computador, por potente que este último sea: a este tipo de problemas se les denomina *indecidibles*. Un problema de decisión es un problema computacional que, ante una entrada dada, responde afirmativa o negativamente. En el proceso de exhibir la existencia de problemas indecidibles, se usan los lenguajes formales como marco inicial para caracterizar cualquier problema de decisión, y se usan funciones matemáticas (parciales y totales) para definir las nociones de aceptación y decisión asociadas a estos lenguajes. Este preámbulo permite abordar el significado de la computabilidad desde una perspectiva (casi) agnóstica de un modelo de computación específico. Un algoritmo es una función computable, tal y como se postula en la tesis de Church y Turing. La existencia de problemas indecidibles se basa en una demostración no constructiva que concluye que los conjuntos de problemas

de decisión y de algoritmos de decisión es distinto, siendo menor el segundo. Este esfuerzo usa nociones y propiedades de cardinales infinitos.

6.1. Problemas de decisión

Un problema de decisión es un problema algorítmico para el cual el conjunto de posibles respuestas es ‘sí’ o ‘no’. Generalmente, siguen un patrón de especificación.

Problema 6.1.1: Especificación típica de un problema de decisión

Entrada: Un conjunto $X \subseteq U$ y un elemento $x \in U$.

Salida: ¿Está x en X (i.e., $x \in X$)?

Se usa U para identificar un conjunto universo que sirve de referencia de X y que se supone puede ser inferido del contexto del problema. La pregunta de decisión consiste en determinar si un elemento de interés (en este caso x) hace parte del conjunto dado (en este caso X).

A modo de ejemplo, se pueden enunciar algunos problemas de decisión conocidos usando el formato de especificación típica del problema de decisión presentado en el Problema 6.1.1.

Ejemplo 6.1.1

Considere los problemas que se enuncian a continuación, identificando versiones concretas de X , U y x .

Clique: determinar si un grafo (no dirigido) tiene un subgrafo completo de un tamaño $k \in \mathbb{N}$ dado. En este caso, X representa el conjunto de grafos con un subgrafo completo de tamaño k , U el conjunto de grafos (finitos) y x el grafo dado.

Primalidad: determinar si un número es primo. En este caso, X representa el conjunto de números primos, U el conjunto de números naturales y x el número sobre el cual se desea averiguar primalidad.

SAT: determinar si una proposición es satisfacible. En este caso, X corresponde al conjunto de proposiciones satisfacibles, U al conjunto de proposiciones y x a una proposición.

2Partition: determinar si un conjunto de números naturales puede ser dividido exactamente en dos subconjuntos tal que la suma de sus elementos coincida. En este caso, X corresponde al conjunto de conjuntos finitos de números naturales que pueden ser particionados en dos subconjuntos

con igual suma, U el conjunto de todos los conjuntos finitos de números naturales y x el conjunto dado.

La importancia de los problemas de decisión radica en dos fenómenos fundamentales. Primero, los problemas de decisión son, a la vez, suficientemente fáciles de entender, y suficientemente complejos para plasmar la naturaleza de la mayoría de los problemas computacionales y abordar el estudio de la intratabilidad. Segundo, problemas de optimización y de conteo pueden ser reformulados y resueltos a partir de un problema de decisión y una de sus soluciones. La investigación en el área de teoría de complejidad se enfoca típicamente en problemas de decisión, la cual ha sido ampliamente desarrollada en las últimas décadas, permeando diferentes campos del conocimiento dentro y fuera de la ingeniería y las ciencias.

Ejemplo 6.1.2

Considere el Problema 3.9.1 del agente viajero: dado un grafo $G = (V, E)$ completo, con función $w : E \rightarrow \mathbb{R}_{\geq 0}$ de peso sobre los arcos, determinar el costo mínimo (con respecto a w) de un circuito en G .

Este problema de optimización cuenta con el siguiente problema de decisión asociado:

Entrada: un grafo $G = (V, E)$ completo, con función $w : E \rightarrow \mathbb{R}_{\geq 0}$ de peso sobre los arcos y un número real x .

Salida: ¿Existe en G un circuito cuyo costo sea a lo sumo x ?

En la práctica, el problema de optimización original se puede resolver usando como caja negra (e.g., como una función que responde sí o no) un algoritmo que resuelva el problema de decisión asociado aplicando bisección. Note que para un error de precisión razonable (e.g., 10^{-9}), la cantidad de llamados que se hacen a la caja negra está acotada por un polinomio. Como consecuencia, la dificultad computacional intrínseca de resolver el problema de optimización con base en una solución al problema de decisión no es mayor asintóticamente a una que se pueda dar directamente al problema de optimización, aún si esta última es polinomial.

Ejercicios

1. Considere una función $\phi : [0..10^9] \rightarrow \{0, 1\}$ no decreciente y suponga que para cualquier $x \in [0..10^9]$, calcular $\phi(x)$ cuesta tiempo polinomial. Demuestre que un algoritmo de bisección sobre $[0..10^9]$ para determinar el mínimo valor x tal que $\phi(x) = 1$ toma tiempo polinomial.
 2. Investigue acerca de la lista de 21 problemas NP-completos publicada por Richard Karp en el artículo “Reducibility Among Combinatorial Problems” de 1972. Escoja 3 de ellos, preferiblemente entre aquellos que no conozca, y especifíquelos como problemas de decisión siguiendo el formato propuesto en esta sección.
 3. Considere el Problema 3.6.1 del morral. Siguiendo el desarrollo del Ejemplo 6.1.2, reformule el problema de optimización original con base en un problema de decisión asociado. Explique claramente cómo una solución de la versión reformulada sirve para resolver el problema de optimización original.
 4. El problema de satisfacibilidad de una proposición en forma normal conjuntiva (o CNFSAT) se puede formular como un problema de optimización en función de la cantidad de cláusulas. Explique cómo hacer esta formulación.
 5. Suponga que se desea determinar la longitud máxima de un ciclo simple (i.e., sin repetir vértices) en un grafo dirigido. Defina un problema de decisión asociado y resuelva el problema original suponiendo que hay una solución para el problema de decisión. Calcule la complejidad temporal de la solución planteada como función de la complejidad temporal del algoritmo de decisión.
 6. Investigue acerca de los científicos de la computación Stephen Arthur Cook y Leonid Levin, y explique brevemente cuál fue su principal aporte al estudio de problemas intratables.
-

6.2. Un marco universal basado en lenguajes

Los conceptos de clases de lenguajes que se trabajan en este capítulo se definen en términos de *lenguajes formales* y *máquinas de Turing*. Dado que el requisito de tiempo polinomial es suficientemente general, el tratamiento en este capítulo no ahonda en los detalles de las máquinas de Turing. En realidad, cualquier algoritmo ejecutable por una máquina de acceso aleatorio (en inglés, *random access machine*) puede ser simulado por una máquina de Turing de una sola cinta y una sola cabeza con un factor polinomial de costo de cómputo. Por ello, esta sección deja a un lado los detalles de los modelos de computación y se centra en los lenguajes formales.

Definición 6.2.1

Un *alfabeto* es un conjunto finito de símbolos. Un *lenguaje (formal)* L sobre un alfabeto Σ es un conjunto de cadenas formadas con los símbolos de Σ . Se usan los símbolos λ para denotar la *cadena vacía*, $\emptyset$ el *lenguaje vacío* y Σ^* la *colección de todas las cadenas sobre Σ* .

De acuerdo con las convenciones introducidas en la Definición 6.2.1, cualquier lenguaje L sobre un alfabeto Σ es en realidad un subconjunto de Σ^* , i.e., $L \subseteq \Sigma^*$. Una ventaja de los lenguajes es que sobre ellos se pueden definir fácilmente operaciones de interés.

Definición 6.2.2

Sea Σ un alfabeto:

- El *complemento* $\bar{L}$ de un lenguaje $L \subseteq \Sigma^*$ es el conjunto

$$\bar{L} = \Sigma^* \setminus L.$$

- La *concatenación* $L_1 L_2$ de dos lenguajes $L_1, L_2 \subseteq \Sigma^*$ es el conjunto

$$L_1 L_2 = \{s_1 s_2 \mid s_1 \in L_1 \wedge s_2 \in L_2\}.$$

- La *concatenación generalizada* L^k , con $k \geq 0$, de un lenguaje $L \subseteq \Sigma^*$ es el conjunto definido inductivamente, para cualquier $n \in \mathbb{N}$, de la siguiente manera:

$$L^0 = \{\lambda\}$$

$$L^{n+1} = L^n L.$$

Las operaciones de *unión* (i.e., $\cup$), *intersección* (i.e., $\cap$) y *potencia* (i.e., $\mathcal{P}(\cdot)$) son las usuales.

El complemento $\bar{L}$ de un lenguaje L es la colección de las cadenas que no están en L . La concatenación $L_1 L_2$ es el conjunto de cadenas que pueden ser partidas en exactamente dos partes: la primera parte está en L_1 y la segunda en L_2 . La concatenación generalizada L^k es el conjunto de cadenas que se construyen a partir de la concatenación de k cadenas en L . De acuerdo con la definición de conjunto potencia heredada de la teoría de conjuntos, las expresiones $L \subseteq \Sigma^*$ y $L \in \mathcal{P}(\Sigma^*)$ son equivalentes.

Ejemplo 6.2.1

Considere cada una de las siguientes situaciones.

- Si $\Sigma = \{0, 1, \dots, 9\}$ (i.e., el conjunto de los dígitos en base 10), entonces

$$L_1 = \{w \mid w \in \Sigma^* \wedge w \neq \lambda\}$$

es el conjunto de los números naturales (algunos repetidos) en representación decimal y

$$L_2 = \{w \mid w \in L_1 \wedge \text{"}w \text{ termina en } 0\text{"}\}$$

es el conjunto de números naturales múltiplos de 10 en representación decimal.

- Si Σ es el alfabeto, entonces

$$L_1 = \{w \mid w \in \Sigma^* \wedge w \neq \lambda \wedge \text{"}w \text{ está en la base de datos de la RAE"}\}$$

es la colección de palabras del Castellano y

$$L_2 = \overline{L_1}$$

la colección de palabras que no son castizas.

Nota 6.2.1

La inclinación por usar Σ^* para denotar el conjunto de todas las cadenas sobre Σ no es caprichosa. Obedece al hecho de que este conjunto corresponde a la clausura de Kleene de Σ , escrita Σ^* , y definida como:

$$\Sigma^* = \bigcup_{n \in \mathbb{N}} \Sigma^n,$$

es decir, el conjunto de todas las cadenas finitas sobre el alfabeto Σ .

El alfabeto de interés en el desarrollo de esta sección y este capítulo, sin pérdida de generalidad, es el binario (i.e., $\Sigma = \{0, 1\}$). Con ello se supone la existencia de una función inyectiva que identifica cualquier elemento o tupla de elementos (de un universo contable) con una secuencia binaria en $\{0, 1\}^*$.

Nota 6.2.2

Se usa la notación $\langle _ \rangle$ para denotar una *función de codificación* con rango en Σ^* . Por ejemplo, $\langle 5 \rangle$ identifica el código binario que representa a 5, mientras que $\langle \text{"abracadabra"} \rangle$ el código binario que representa la cadena "abracadabra". Esta notación se puede usar sobre otros tipos de estructuras como grafos, matrices, etc.

Es importante anotar que existen varias funciones de codificación. Sin embargo, hacer distinciones no es relevante para el tratamiento que sigue. Además, note que elegir una función de codificación en particular es coherente con la situación que se tiene en los computadores que procesan y calculan volúmenes brutales de información representada en cadenas de bits: cada sistema de archivos puede usar un formato distinto. Tampoco es indispensable que los códigos sean binarios; por ejemplo, pueden ser ternarios o usar cualquier otro alfabeto que garantice que cualquier conversión de códigos binarios al alfabeto de elección esté acotada polinomialmente. A las codificaciones que respetan esta restricción se les denominan *codificaciones razonables*. Por ejemplo, el alfabeto $\{1\}$ (i.e., el unario) no es razonable; ¿por qué?

Finalmente, una bondad de usar lenguajes formales como marco general de representación de problemas computacionales tiene que ver con la facilidad para formular problemas de decisión. Note que, de acuerdo con la especificación típica de un problema de decisión en la Sección 6.1, un problema de decisión es entonces una pregunta sobre la pertenencia o no de una cadena binaria a un lenguaje.

Problema 6.2.1: Especificación de un problema de decisión con lenguajes

Entrada: Un conjunto $L \subseteq \{0, 1\}^*$ y un elemento $x \in \{0, 1\}^*$.

Salida: ¿ $x \in L$?

Gracias al uso de lenguajes formales, y a la transparencia de contar con las máquinas de Turing como modelo de computación abstracto, en las próximas secciones es posible abordar las definiciones y el estudio de las clases **P** y **NP**, los conceptos de decisión, aceptación y completitud, y la técnica de reducción polinomial con algo más que un poco de teoría de conjuntos básica.

Ejercicios

1. Investigue acerca de máquinas de acceso aleatorio (en inglés, *random access machine*) y explique su relación con los computadores modernos.
2. Investigue acerca de máquinas de Turing. Elabore una justificación de la siguiente afirmación: cualquier algoritmo ejecutable por una máquina de acceso aleatorio puede ser simulado por una máquina de Turing de una sola cinta y una sola cabeza con un factor polinomial de costo de cómputo.
3. Explique con ejemplos y justifique por qué el alfabeto unario no es razonable.

6.3. Aceptación y decisión

El objetivo de esta sección es presentar y estudiar las nociones de aceptación y decisión asociadas a los lenguajes formales. Esto se hace con base en funciones parciales y totales, respectivamente, con dominio en las secuencias binarias $\{0, 1\}^*$ y rango en el alfabeto binario $\{0, 1\}$. En esta sección se fija el alfabeto binario $\Sigma = \{0, 1\}$ y, mientras no se precise, una función puede ser total o no (i.e., parcial; ¿por qué?).

Haber adoptado el marco de los lenguajes formales permite expresar concisa y consistentemente la relación entre problemas de decisión y la forma en que estos se pueden resolver.

Definición 6.3.1: Aceptación y rechazo

Sea $A : \Sigma^* \rightarrow \Sigma$ una función. Para $x \in \Sigma^*$, se dice que:

- A *acepta* x sii $A(x) = 1$ y
- A *rechaza* x sii $A(x) = 0$.

El *lenguaje aceptado* por A , denotado $L(A)$, es el conjunto:

$$L(A) = \{x \mid x \in \Sigma^* \wedge A(x) = 1\}.$$

En el marco de los lenguajes formales, la inclinación es por representar una solución a un problema de decisión como una función que, dada una palabra binaria, calcula un valor binario. Por convención, aceptar una cadena se indica con el valor ‘Booleano’ 1 y rechazarla con el 0, similar a lo que se tiene en la lógica Booleana implementada en los computadores de hoy en día. Dada una función A , se asocia a esta el lenguaje $L(A)$ que corresponde exactamente a las cadenas aceptadas por A . Evitar que las funciones sean necesariamente totales es clave, como se verá más

adelante, para caracterizar las nociones de aceptación y decisión asociadas a los problemas de decisión (i.e., a lenguajes binarios).

Ejemplo 6.3.1

Considere las siguientes afirmaciones para una función $A : \Sigma^* \rightarrow \{0, 1\}$:

- Si $A(x) = 1$ para cualquier $x \in \Sigma^*$, entonces $L(A) = \Sigma^*$. Hacia una contradicción, suponga lo contrario: es decir, que $L(A) \neq \Sigma^*$. Entonces, necesariamente $L(A) \subsetneq \Sigma^*$. En consecuencia, hay una cadena $s \in \Sigma^* \setminus L(A)$. Por definición, $A(s) \neq 1$ dado que s no está en $L(A)$ (¿por qué no se puede afirmar que $A(s) = 0$?). Pero esto es una contradicción dado que se ha supuesto que $A(x) = 1$ para cualquier $x \in \Sigma^*$; en particular, para s . Luego, necesariamente $L(A) = \Sigma^*$.
- Si $A(x) = 0$ para cualquier $x \in \Sigma^*$, entonces $L(A) = \emptyset$. La justificación se propone como ejercicio al lector.

En la justificación de la primera parte del Ejemplo 6.3.1, se concluye que $A(s) \neq 1$ cuando $s \notin L(A)$, pero no que $A(s) = 0$. Este es un aspecto aparentemente irrelevante, pero es en realidad extremadamente importante porque no toda función parcial es total. Note que, en general, es posible que $A(s)$ esté indefinido, es decir, que s sea uno de los puntos de indeterminación de A (i.e., para el cual A no tiene asociado ni 0 ni 1). Por eso, sin información que garantice la totalidad de la función A , puede ser un error deducir $A(s) = 0$ cuando $A(s) \neq 1$.

Es usual diseñar un algoritmo con base en otro existente. Por ejemplo, construir nuevos procedimientos en un lenguaje de programación a partir de otros procedimientos existentes en una librería. Uno de estos casos corresponde al de funciones que invierten la salida de otra función.

Definición 6.3.2

Sea $A : \Sigma^* \rightarrow \{0, 1\}$ una función. La función *complementaria* de A , denotada $\overline{A}$, se define para cualquier $x \in \Sigma^*$ como:

- $\overline{A}(x) = 1$ si $A(x) = 0$,
- $\overline{A}(x) = 0$ si $A(x) = 1$ y
- $\overline{A}(x)$ es indeterminada si $A(x)$ es indeterminada.

Cuando A en la Definición 6.3.2 es total, coinciden el complemento del lenguaje determinado por A y el lenguaje determinado por su función complementaria $\overline{A}$.

Teorema 6.3.1

Si $A : \Sigma^* \rightarrow \{0, 1\}$ es una función total, entonces:

$$L(A) = \overline{L(\overline{A})}.$$

Demostración

Sea $x \in \Sigma^*$. El objetivo es demostrar:

$$x \in L(A) \iff x \in \overline{L(\overline{A})}.$$

Se procede por doble implicación.

($\implies$): Se supone $x \in L(A)$ con el objetivo de demostrar que $x \in \overline{L(\overline{A})}$ o, equivalentemente, que $x \notin L(\overline{A})$. Como $x \in L(A)$, se tiene que $A(x) = 1$. Por definición de función complementaria, se tiene que $\overline{A}(x) = 0$. Es decir, $x \notin L(\overline{A})$.

($\impliedby$): Se supone $x \in \overline{L(\overline{A})}$ con el objetivo de demostrar que $x \in L(A)$. Como $x \in \overline{L(\overline{A})}$, entonces $x \notin L(\overline{A})$. O bien $\overline{A}(x) = 0$ o $\overline{A}(x)$ es indeterminado. Por la definición de función complementaria y la suposición de que A es total, no se puede dar que $\overline{A}(x)$ sea indeterminado. Es decir, necesariamente $\overline{A}(x) = 0$. Por definición de $\overline{A}$, se tiene que $A(x) = 1$, como se deseaba.

En conclusión, $L(A) = \overline{L(\overline{A})}$.

El hecho de que una función A determine el lenguaje $L(A)$, no permite concluir que necesariamente A rechaza cualquier cadena $x \notin L(A)$. En la práctica, esto puede suceder cuando un ciclo no termina o cuando hay un error en la ejecución de un programa. Con este preámbulo, y lo desarrollado anteriormente, se motiva la introducción de las nociones de aceptación y decisión asociadas a los lenguajes.

Definición 6.3.3: Aceptación y decisión de un lenguaje

Sea L un lenguaje y $A : \Sigma^* \rightarrow \{0, 1\}$ una función. Se dice que:

- A *acepta* L sii para cualquier $x \in L$ se tiene $A(x) = 1$.
- A *decide* L sii para cualquier $x \in L$ se tiene $A(x) = 1$ y para cualquier $x \notin L$ se tiene $A(x) = 0$.

La Definición 6.3.3 invita a varias observaciones importantes. Primero, toda función de decisión es una de aceptación, pero no necesariamente al contrario. Segundo, si una función A acepta L , independientemente de si A es solo de aceptación o si es de decisión, $L = L(A)$. Tercero, un lenguaje puede ser aceptado por más de una función pero decidido solo por una única función (¿por qué?). Finalmente, cualquier función total necesariamente es una función de decisión; el converso también es cierto, es decir, cualquier función de decisión es total. Esta última observación, en el marco de los lenguajes formales, es especialmente importante porque asocia unívocamente la noción de decisión de un lenguaje a la de totalidad de cierto tipo de funciones sobre lenguajes binarios.

Teorema 6.3.2

Si $A : \Sigma^* \rightarrow \{0, 1\}$ una función, entonces:

$$A \text{ es total} \iff A \text{ decide } L(A).$$

Demostración

Se propone como ejercicio al lector.

Ejercicios

1. Proponga condiciones suficientes para que la siguiente igualdad sea cierta para $A : \Sigma^* \rightarrow \{0, 1\}$:

$$\overline{L(A)} = \{x \mid x \in \Sigma^* \wedge L(A) = 0\}.$$

2. Justifique formalmente la segunda afirmación en el Ejemplo 6.3.1.
3. Sea $A : \Sigma^* \rightarrow \{0, 1\}$. Demuestre o refute:
 - a) $L(A) \cup \overline{L(A)} = \Sigma^*$.
 - b) $L(A) \cup L(\overline{A}) = \Sigma^*$.
4. Presente los detalles de la siguiente afirmación, en el marco de la demostración del Teorema 6.3.1:

Por la definición de función complementaria y la suposición de que A es total, no se puede dar que $\overline{A}(x)$ sea indeterminado.

Sugerencia: procer por contradicción.
5. Formule un problema aceptado exactamente por una única función de aceptación. Es necesario demostrar que dicha función es única.

6. Justifique brevemente por qué toda función de decisión es de aceptación, pero no viceversa.
7. Justifique brevemente por qué si una función A acepta L , independientemente de si A es de decisión o no, $L = L(A)$.
8. Defina un lenguaje L , y dos funciones distintas A_1 y A_2 tales que $L = L(A_1) = L(A_2)$.
9. Demuestre el Teorema 6.3.2.

6.4. Computabilidad

En la práctica, un problema computacional L puede ser resuelto cuando existe un programa o procedimiento que mecánicamente calcule los valores correspondientes a una función A tal que $L = L(A)$. A este tipo de funciones se les denomina *efectivamente computables* o *computables*. La noción de computabilidad no se refiere a eficiencia (en tiempo o espacio), sino más bien a una posibilidad de que exista cierto programa de computador (eficiente o no, en tiempo o espacio) que calcule los valores de una función dada: es una propiedad acerca de las funciones de Σ^* en $\Sigma = \{0, 1\}$. Esta sección presenta una definición de función computable que corresponde a (y se usará como) la noción de algoritmo. Estudia también algunas de sus propiedades, y enuncia la tesis de Church y Turing acerca de lo que significa computar en el marco de los lenguajes formales.

Definición 6.4.1: Funciones computables

Sea $A : \Sigma^* \rightarrow \Sigma$. Se dice que A es *computable* si y solo si existe un programa de computador que cumple las siguientes dos propiedades, para cualquier $x \in \Sigma^*$:

- Si $A(x)$ está definido, entonces el programa termina cuando se invoca con x y retorna el valor de $A(x)$.
- Si $A(x)$ está indefinido, entonces el programa no termina (nunca) cuando se invoca con x y no retorna ningún valor.

Las funciones computables son el análogo formal de la noción intuitiva de algoritmo: una función es computable si existe un algoritmo que permita calcular sus valores. Además, las funciones computables son los objetos de estudio básico en el área denominada teoría de la computabilidad, especialmente porque permiten elaborar en la noción de computabilidad sin necesidad de referirse directamente a un modelo de computación. En realidad, en todos los modelos de computación aceptados actualmente, las correspondientes definiciones de computabilidad coinciden en identificar las mismas colecciones de funciones.

Por su parte, la noción de función computable puede ser más general en cuanto a considerar más parámetros y otro tipo de dominios como, por ejemplo, los números. Sin embargo, para el propósito que anima esta sección, y el capítulo en general, es suficiente enfocarse exclusivamente en funciones con dominio y rango en Σ^* y Σ , respectivamente, sin que los resultados asociados pierdan generalidad.

Como observación final de la noción de computabilidad, note que en la Definición 6.4.1 no se menciona explícitamente lenguaje de programación alguno. Se entiende que este tipo de detalle aporta poco o nada a la noción de computabilidad. Lo que sí es relevante (y clave) es el hecho de aceptar como acuerdo que cualquier algoritmo se define a partir de una cantidad finita de símbolos (en un lenguaje dado) que describen una secuencia de pasos que, por ejemplo, una persona con suficiente tiempo y recursos (como papel y lápiz) puede seguir.

Con ayuda de las funciones computables, la noción de computabilidad puede ser extendida a problemas de decisión.

Definición 6.4.2: Lenguajes computablemente enumerables y computables

Sea $L \subseteq \Sigma^*$. Se dice que:

- L es *computablemente enumerable* si y solo si existe una función computable $A : \Sigma^* \rightarrow \Sigma$ tal que $L(A) = L$.
- L es *computable* si y solo si existe una función total computable $A : \Sigma^* \rightarrow \Sigma$ tal que $L(A) = L$.

Las lenguajes computablemente enumerables y computables coinciden con las nociones de aceptación y decisión en la Definición 6.3.3, respectivamente, cuando las funciones parciales y totales que los caracterizan son computables. De esta forma, un problema de decisión L es computablemente enumerable cuando existe un algoritmo A que retorna 1 al ser invocado con cualquier instancia $x \in L$. Además, si dicho algoritmo retorna 0 al ser invocado con cualquier instancia $x \notin L$, entonces L es computable. Note que, por la definición, cualquier lenguaje computable es computablemente enumerable. Sin embargo, el converso no es necesariamente cierto.

Ejemplo 6.4.1

Considere los siguientes ejemplos:

- Determinar si un número entero hace parte de un arreglo de números enteros. Considere el lenguaje

$$\text{ARRFIND} = \{ \langle A[0..N], x \rangle \mid \text{"}A \text{ es un arreglo y } x \text{ está en } A \text{"} \}.$$

El siguiente algoritmo permite afirmar que ARRFIND es computable. Corresponde a explorar todas las posiciones del arreglo dado para determinar si el número dado está presente o no. Esto se admite porque la igualdad de dos números enteros es computable (¿por qué?).

- Determinar si una fórmula proposicional es una tautología. Considere el lenguaje

$$\text{TAUT} = \{\langle \varphi \rangle \mid \text{"}\varphi \text{ es una proposición tautológica"}\}.$$

El siguiente algoritmo permite afirmar que TAUT es computable. Corresponde a generar la tabla de verdad de φ y si todos sus renglones resultan en verdadero, entonces la proposición es una tautología; de lo contrario, no lo es.

- Determinar si una fórmula de primer orden es un teorema. Considere el lenguaje

$$\text{FOLTHM} = \{\langle \varphi \rangle \mid \text{"}\varphi \text{ es un teorema en lógica de primer orden"}\}.$$

El siguiente algoritmo permite afirmar que FOLTHM es computablemente enumerable. Generar incrementalmente, en un orden arbitrario, las demostraciones en lógica de primer orden. Si aparece la fórmula dada como conclusión de una demostración, entonces dicha fórmula es un teorema de esta lógica. Note que este algoritmo no se detiene si dicha fórmula no es la conclusión de ninguna demostración.

- El problema de correspondencia de Post (ver Ejercicio 8) no es computablemente enumerable. En consecuencia, tampoco es computable.

Nota 6.4.1

Más allá del contexto de los lenguajes formales, los problemas computablemente enumerables o computables reciben los nombres de *semidecidibles* o *decidibles*, respectivamente. Por ejemplo, es un problema semidecidible determinar si una fórmula de primer orden es teorema, mientras que es un problema decidible determinar si una fórmula proposicional es tautología.

Nota 6.4.2

Como lo evidencia el problema de correspondencia de Post en el Ejemplo 6.4.1, hay problemas de decisión que no son ni siquiera semidecidibles (i.e., computablemente enumerables). A este tipo de problemas se les denomina *indecidibles*. La Sección 6.5 presenta un argumento (no constructivo) de la existencia de problemas indecidibles.

Volviendo al dominio de las funciones computables, se enuncian algunas de las operaciones bajo las cuales son cerradas (i.e., bajo las cuales la computabilidad se mantiene).

Teorema 6.4.1

Las siguientes afirmaciones son ciertas acerca de las funciones computables:

1. Son cerradas bajo suma binaria (punto a punto módulo 2).
2. Son cerradas bajo multiplicación (punto a punto).
3. Son cerradas bajo composición (cuando el dominio de, e.g. f , está restringido a Σ en $f \circ g$).
4. Son cerradas bajo mínimo y máximo (punto a punto).

Demostración

Se proponen como ejercicio al lector.

Finalmente, se enuncia la tesis Alonzo Church y Alan Turing acerca de lo que significa computar.

Nota 6.4.3: La tesis de Church y Turing

La tesis de Church y Turing afirma que las funciones computables son exactamente aquellas funciones que pueden ser calculadas mecánicamente con cantidades ilimitadas de tiempo y espacio.

Esta tesis es uno de varios intentos por formalizar la noción de computabilidad. Afirma que las funciones computables son aquellas para las cuales existe un algoritmo (y viceversa). Su carácter no es completamente formal o matemático dado que la noción de computabilidad asociado a las funciones es informal. Por esto, no es un postulado que se pueda demostrar.

Ejercicios

1. Defina brevemente en qué consiste cada uno de los siguientes modelos de computación y enuncie la definición formal de computación correspondiente:
 - a) Máquinas de Turing.
 - b) Cálculo λ (de Alonzo Church).
 - c) Funciones μ -recursivas (de Kurt Gödel).
 - d) Máquinas abstractas (de Emil Post).
 - e) Algoritmos de Markov.
 2. Demuestre que si $A : \Sigma^* \rightarrow \Sigma$ tiene dominio finito, entonces A es computable. Concluya que cualquier $L \subseteq \Sigma^*$ cuyo complemento es finito también es computable.
 3. Demuestre, exhibiendo un algoritmo como testigo, que:
 - a) Una función $A : \Sigma^* \rightarrow \Sigma$ constante es computable.
 - b) El conjunto vacío es computable.
 - c) El conjunto de números naturales es computable.
 - d) El conjunto de números primos es computable.
 4. En el Ejemplo 6.4.1 se afirma que el problema **ARRFIND** es computable. Explique por qué este problema cuando se extiende a números reales deja de ser computable. Ayuda: fijarse en la computabilidad del problema de igualdad de dos números reales.
 5. Demuestre el Teorema 6.4.1.
 6. Sean $L_1, L_2 \subseteq \Sigma^*$ computables. Demuestre que:
 - a) $L_1 \cup L_2$ es computable.
 - b) $L_1 \cap L_2$ es computable.
 7. Sea $A : \Sigma^* \rightarrow \Sigma$. Demuestre que A es computable si y solo si A y su complemento son computablemente enumerables.
 8. Considere el problema de correspondencia de Post mencionado en el Ejemplo 6.4.1.
 - a) Defina claramente en qué consiste este problema e ilustre la definición con un ejemplo.
 - b) Investigue acerca de una demostración de indecidibilidad de este problema y explique sus principales pasos.
 9. Investigue acerca del décimo problema de Hilbert. Explique en qué consiste y cuál es su estado actual.
-

6.5. Indecidibilidad

El principal resultado de la sección es negativo en el sentido en el cual se demuestra que existen funciones que no son computables. Muestra que, en el contexto de lo que significa computar hoy en día, hay funciones que no se pueden mecanizar en un computador. El principal argumento en la demostración es la existencia de una cantidad tan grande de problemas de decisión, en el sentido de la Sección 6.2, que no hay suficientes programas de computador que los puedan resolver mecánicamente. Esto es equivalente a decir que no toda función con dominio en el conjunto de cadenas binarias Σ^* y rango en el conjunto binario Σ puede ser mecanizada en un computador (ver Teorema 6.3.2).

Una advertencia amigable acerca del desarrollo de la sección antes de entrar en materia. Se usarán conceptos y resultados de la teoría de conjuntos como, por ejemplo, cardinales y aritmética cardinal que generalmente no son estudiados en currículos de pregrado de ingeniería y computación. Por ello, la limitante potencial en esta sección es que algunos de los resultados empleados no se demuestran porque las definiciones o técnicas necesarias están fuera del alcance del texto; en consecuencia, esta sección no es autocontenida. Al final del capítulo se presentan referencias para profundizar en estos temas y conocer las demostraciones ausentes, en caso tal de que el lector quiera profundizar en ellos.

6.5.1. Conceptos básicos de cardinales. El plan general para lograr el objetivo de la sección es simple: comparar los tamaños del conjunto de lenguajes y del conjunto de funciones computables para determinar que el primero es mucho más grande que el segundo.

Para razonar sobre el tamaño de conjuntos infinitos es necesario desligar la noción de tamaño de la acción de contar uno a uno, tal y como sucede generalmente cuando los conjuntos son finitos. Por ejemplo, ¿se podrían contar, uno a uno, los números naturales? La respuesta es no: este conjunto no tiene una cota superior y, en consecuencia, cualquier intento de listarlos explícitamente nunca terminaría. En contraste, si se toma cualquier subconjunto finito de los números naturales, por grande que este sea, un proceso de conteo de uno tras otro seguro que terminará (posiblemente en varias generaciones de una familia si el subconjunto es muy grande, pero terminará).

Para referirse al tamaño de conjuntos infinitos, es necesario introducir la noción de cardinal y algunas propiedades asociadas a ella.

Definición 6.5.1: Número cardinal

El *cardinal* de un conjunto X se denota como $\text{card}(X)$. Sean X_1 y X_2 dos conjuntos. Entonces:

- $\text{card}(X_1) = \text{card}(X_2)$ si y solo si existe una función biyectiva entre X_1 y X_2 . En este caso, se dice que X_1 y X_2 son *equipotentes*, escrito $X_1 \sim X_2$.
- $\text{card}(X_1) \leq \text{card}(X_2)$ si y solo si existe una función inyectiva de X_1 a X_2 .

Además, si X_1 y X_2 son disyuntos, se tiene

$$\text{card}(X_1 \cup X_2) = \text{card}(X_1) + \text{card}(X_2).$$

La función $\text{card}(\cdot)$ asocia cualquier conjunto a un *ordinal*; para efectos prácticos, un *cardinal* es un tipo especial de ordinal que permite representar unívocamente el tamaño de un conjunto infinito. Intuitivamente, los ordinales son conjuntos que generalizan la noción de número y que coinciden con los números naturales cuando los objetos de estudio son los conjuntos finitos. De acuerdo con la Definición 6.5.1, para determinar la igualdad de los cardinales (o, equivalentemente, la equipotencia de dos conjuntos) es necesario exhibir una biyección entre los conjuntos en cuestión. Además, se extiende el orden total $\leq$ sobre los números naturales a los cardinales infinitos. Al igual que con conjuntos finitos, el tamaño de una unión disyunta de dos conjuntos es igual a la suma de los tamaños de las dos partes.

Ejemplo 6.5.1

El conjunto de los números naturales y el de los números naturales pares

$$2\mathbb{N} = \{2m \mid m \in \mathbb{N}\}$$

tienen el mismo tamaño (i.e., $\mathbb{N} \sim 2\mathbb{N}$). Esto es cierto porque la función determinada por el mapa

$$n \mapsto 2n$$

es biyectiva. Por una parte, es claramente inyectiva: si $2n_1 = 2n_2$, necesariamente $n_1 = n_2$. Para observar que es sobreyectiva, note que cualquier número en $2\mathbb{N}$ tiene la forma $2n$, cuya preimagen es $n \in \mathbb{N}$. En conclusión, los dos conjuntos tienen el mismo cardinal y son equipotentes.

Note que dado que hay una biyección entre $\mathbb{N}$ y $2\mathbb{N}$, también se tiene que $\text{card}(\mathbb{N}) \leq \text{card}(2\mathbb{N})$ y $\text{card}(2\mathbb{N}) \leq \text{card}(\mathbb{N})$.

El Ejemplo 6.5.1 exhibe una propiedad posiblemente sorprendente de los conjuntos infinitos: aún cuando $2\mathbb{N}$ es un conjunto propio de $\mathbb{N}$, los dos conjuntos tienen la misma cantidad de elementos. Con conjuntos finitos esta situación es imposible porque cualquier conjunto propio de un conjunto finito necesariamente tiene menos elementos. Por ejemplo, el axioma “el todo es mayor que cualquiera de sus partes” de Euclides es inválido para conjuntos infinitos.

Nota 6.5.1

El matemático Richard Dedekind usó la siguiente propiedad para caracterizar conjuntos finitos e infinitos:

Si no hay un conjunto propio de X equipotente a X , entonces X es *finito*. Si hay un conjunto propio de X equipotente a X , entonces X es *infinito*.

Nota 6.5.2

Usualmente, los cardinales infinitos se identifican con los ordinales:

$$\aleph_0, \aleph_1, \aleph_2, \dots$$

El símbolo $\aleph$ fue introducido por el matemático Georg Cantor para representar el tamaño de conjuntos infinitos y proviene del alfabeto hebreo; este símbolo se lee “alef” en Castellano.

Los conjuntos infinitos usualmente se clasifican en dos tipos: aquellos que tienen el mismo tamaño de los números naturales y los demás. A los primeros se les denomina contables, mientras que a los segundos no contables. Históricamente, la distinción entre conjuntos contables y no contables ha permitido entender diferencias profundas entre conjuntos de interés como lo son, por ejemplo, los números racionales y los reales que hacen parte de categorías distintas en relación con el tamaño de los números naturales. El argumento que se desarrolla a lo largo de esta sección es un ejemplo más del uso de esta caracterización, como se terminará de ver en los próximos apartes.

Definición 6.5.2

Sea X un conjunto. Se dice que:

- X es *contable* si y solo si X es finito o $X \sim \mathbb{N}$;
- X es *no contable* de lo contrario.

Se adopta $\text{card}(\mathbb{N}) = \aleph_0$ por convención.

Como consecuencia de la Definición 6.5.2, $\mathbb{N}$ es contable, al igual que cualquiera de sus subconjuntos. Por eso, abusando de la convención, se dice que $\aleph_0$ es el único cardinal infinito contable, además de ser el primer cardinal infinito.

6.5.2. Aritmética cardinal. Los cardinales, como números que son, se pueden operar entre sí. El Teorema 6.5.1 introduce una propiedad de la suma de cardinales que será útil posteriormente.

Teorema 6.5.1: Suma de cardinales infinitos

Sean X_1 y X_2 conjuntos, con X_1 infinito. Si $\text{card}(X_2) \leq \text{card}(X_1)$, entonces $\text{card}(X_1) + \text{card}(X_2) = \text{card}(X_1)$.

La suma de cardinales infinitos puede ser intuitiva si es interpretada, por ejemplo, con ayuda de la unión de conjuntos. Si un conjunto infinito se duplica, el cardinal del conjunto resultante es el mismo. De manera más general, si un conjunto infinito X se extiende con un conjunto que no sobrepasa su cantidad de elementos, el tamaño del conjunto resultante es el mismo de X , como se afirma en el Teorema 6.5.1. A nivel formal, la demostración del Teorema 6.5.1 está fuera del alcance de este texto (se propone como ejercicio al lector interesado en profundizar en el tema o con conocimiento previo acerca de cardinales).

Hay una forma mecánica de construir un conjunto infinito de mayor tamaño que cualquier conjunto infinito dado. Esta corresponde a la operación de construcción del conjunto potencia denotado en este texto como $\mathcal{P}(\cdot)$; este operador denota el conjunto de todos los subconjuntos de un conjunto dado. El Teorema 6.5.2 presenta resultados asociados al conjunto potencia de un conjunto, finito o infinito.

Teorema 6.5.2: Cardinal del conjunto potencia

Para cualquier conjunto X se tiene:

1. $\text{card}(\mathcal{P}(X)) = 2^{\text{card}(X)}$.
2. $\text{card}(X) < \text{card}(\mathcal{P}(X))$.

Al igual que sucede con la demostración del Teorema 6.5.1, la demostración del Teorema 6.5.2 está fuera del alcance de este texto y se propone como ejercicio a lectores interesados en el tema. Intuitivamente, el tamaño del conjunto partes de un conjunto infinito cumple la misma ley de exponenciación para conjuntos finitos. Además, la cantidad de subconjuntos de un conjunto dado X , finito o infinito, es estrictamente mayor que la cantidad de elementos de X .

6.5.3. ¿Cuántos lenguajes hay? Como cada lenguaje identifica un problema algorítmico de decisión, determinar el tamaño del conjunto de lenguajes también permite concluir cuántos problemas de decisión hay. Se establece un resultado clave en el camino hacia este objetivo intermedio al determinar el tamaño de $\{0, 1\}^*$.

Teorema 6.5.3: $\{0, 1\}^*$ es contable

El tamaño de Σ^* es el infinito contable, i.e., $\Sigma^* \sim \mathbb{N}$.

Demostración

Considere los siguientes conjuntos:

$$X_0 = \{w \mid w \in \Sigma^* \wedge \text{"}w \text{ inicia con } 0\text{"}\}$$

$$X_1 = \{w \mid w \in \Sigma^* \wedge \text{"}w \text{ inicia con } 1\text{"}\}.$$

Note que X_0 y X_1 son disyuntos, y junto con $\{\lambda\}$ forman una partición de Σ^* .

Observación 1: $X_0 \sim X_1$. Considere el siguiente mapa para cualquier $x \in \Sigma^*$:

$$x \mapsto \text{"inversión, punto a punto, de los bits de } x\text{"}.$$

Por ejemplo, este mapa convierte la cadena 0001101 en 1110010. Demostrar que este mapa induce una función biyectiva es fácil y se propone como ejercicio al lector.

Observación 2: $X_1 \sim \mathbb{N}$. Considere el siguiente mapa para cualquier $x \in \Sigma^*$:

$$x \mapsto \text{"representación en base 10 de } x\text{"} - 1.$$

Por ejemplo, este mapa convierte la cadena 110 en 5 y la cadena 1 en 0. Demostrar que este mapa induce una función biyectiva entre X_1 y $\mathbb{N}$ es fácil, y se propone como ejercicio al lector.

Con base en estas dos observaciones, se tiene que:

$$\begin{aligned} \text{card}(\Sigma^*) &= \text{card}(X_0 \cup X_1 \cup \{\lambda\}) && \text{(partición de } \Sigma^*) \\ &= \text{card}(X_0) + \text{card}(X_1) + \text{card}(\{\lambda\}) && (X_0, X_1, \{\lambda\} \text{ son disyuntos)} \\ &= \text{card}(X_0) + \text{card}(X_1) + 1 && \text{(definición de card)} \\ &= \text{card}(X_0) + \text{card}(X_1) && \text{(Teorema 6.5.1)} \\ &= \text{card}(X_1) + \text{card}(X_1) && \text{(Observación 1)} \\ &= \text{card}(X_1) && \text{(Teorema 6.5.1)} \\ &= \text{card}(\mathbb{N}) && \text{(Observación 2).} \end{aligned}$$

En conclusión, $\Sigma^* \sim \mathbb{N}$.

El Teorema 6.5.3 establece que no hay más secuencias binarias que números naturales, lo cual puede parecer extraño. Esta observación es consecuencia del resultado anterior. Es más, aún si el tamaño del alfabeto Σ es el infinito contable, la veracidad de este teorema también se mantiene.

Dado que un problema algorítmico es subconjunto de Σ^* , para razonar acerca de la cantidad de lenguajes es necesario razonar acerca del tamaño del conjunto $\mathcal{P}(\Sigma^*)$, es decir, del conjunto partes de Σ^* . Para ello se usan los resultados del Teorema 6.5.2.

Teorema 6.5.4: Cantidad de lenguajes

Hay $2^{\aleph_0}$ lenguajes.

Demostración

Hay tantos problemas algorítmicos como subconjuntos tiene $\mathcal{P}(\Sigma^*)$. Es decir, la cantidad de problemas algorítmicos es $\text{card}(\mathcal{P}(\Sigma^*))$. Observe que:

$$\begin{aligned} \text{card}(\mathcal{P}(\Sigma^*)) &= 2^{\text{card}(\Sigma^*)} && \text{(por el Teorema 6.5.2)} \\ &= 2^{\text{card}(\mathbb{N})} && \text{(por el Teorema 6.5.3)} \\ &= 2^{\aleph_0} && \text{(por el tamaño de } \mathbb{N} \text{).} \end{aligned}$$

Nota 6.5.3

El cardinal $2^{\aleph_0}$ es especial y se identifica con la letra c , dado que corresponde al cardinal del continuo: $2^{\aleph_0} = c = \text{card}(\mathbb{R})$. Por el Teorema 6.5.2 se tiene que $\aleph_0 < c$, es decir, hay (muchos) más números reales que naturales.

De manera aparte, pero como tema complementario, es importante llamar la atención a que c es interesante porque permite plantear uno de los problemas más desafiantes de las matemáticas, la Hipótesis del Continuo de Rienman (HCR):

No existe conjunto alguno cuyo tamaño esté estrictamente entre el tamaño de los naturales y el tamaño de los reales.

Dicho de otra manera, la HCR plantea la imposibilidad de que exista un cardinal entre $\aleph_0$ y $2^{\aleph_0}$. Si la HCR es cierta, entonces se tendría que

$$\aleph_1 = 2^{\aleph_0} = c.$$

6.5.4. ¿Cuántas funciones computables hay? Es necesario fijar un alfabeto de programación para determinar la cantidad de funciones computables. Se supondrá que Π es el alfabeto de símbolos usados para programar una función computable en cualquier lenguaje de programación. De esta forma, se está tomando la dirección de tratar de identificar cada función computable con al menos una cadena de símbolos en Π que representa su codificación en un lenguaje de programación. Esto es lo que sucede en la realidad pues se pueden implementar tantos procedimientos de decisión como secuencias correctas de instrucciones escritas con símbolos en Π haya (en el lenguaje de programación de elección). De esta forma, $\text{card}(\Pi^*)$ impone una cota superior sobre la cantidad de funciones implementables en cualquier lenguaje de programación. Se supone que cada programa es finito y que la cantidad de lenguajes de programación también lo es, lo cual es cierto en la práctica.

Teorema 6.5.5: Cantidad de funciones computables

La cantidad de funciones computables es $\aleph_0$.

Demostración

La cantidad de funciones computables que se pueden programar con el alfabeto Π está acotada por $\text{card}(\Pi^*)$: el tamaño de todas las secuencias sobre Π (un programa tiene una descripción finita, luego debe ser una cadena en Π^*). Siguiendo un razonamiento similar al de la demostración del Teorema 6.5.3, se tiene que $\Pi^* \sim \mathbb{N}$. Luego, la cantidad de funciones computables es, a lo sumo, $\aleph_0$.

Saber que $\aleph_0$ es una cota superior sobre la cantidad de funciones computables es importante. Por una parte, se establece que esta cota se sostiene independientemente de cual sea el alfabeto con que se implementen las soluciones algorítmicas, mientras este alfabeto sea contable. Segundo, es la última pieza que se requiere para demostrar que hay problemas algorítmicos para los cuales no existe programa de computador alguno que los resuelva.

6.5.5. Hay problemas que no se pueden decidir. Este aparte, finalmente, presenta el principal resultado de esta sección. Es decir, demuestra que existen problemas de decisión que no se pueden resolver mecánicamente con un computador, sin importar qué lenguaje de programación se use. Como se mencionó al inicio de la sección, es un resultado negativo porque muestra la imposibilidad de usar los computadores como máquinas universales que resuelvan cualquier problema (de

decisión). Sin embargo, y como se puede comprobar con situaciones diarias, los computadores son extensamente utilizados para resolver problemas y asistir a las personas en la solución de otros tantos. Por ello, a pesar de que teóricamente se establece dicha imposibilidad de solución de problemas mecánicamente, en la práctica cada día se depende más de los computadores.

Teorema 6.5.6

Existe al menos un problema de decisión para el cual no hay una función computable que lo resuelva. Es decir, hay al menos un problema que un computador no puede resolver.

Demostración

Note que hay $2^{\aleph_0}$ lenguajes y $\aleph_0$ funciones computables, por los teoremas 6.5.4 y 6.5.5, respectivamente. Por el Teorema 6.5.2, se tiene que $\aleph_0 < 2^{\aleph_0}$. En consecuencia, hay al menos un problema de decisión para el cual no es posible diseñar un algoritmo que lo decida.

El Teorema 6.5.6 no riñe con el resultado del Teorema 6.3.2. Una cosa es entender que para cada problema hay exactamente una función total que lo decide y otra que dicha función sea pueda mecanizar en un computador. Dicho de otra forma, en un computador no se pueden definir todas las funciones con dominio en Σ^* y rango en Σ . Así es como surge la existencia de problemas indecidibles: son aquellos problemas para los cuales no existe una implementación de su función de decisión asociada. La demostración del Teorema 6.3.2 no es constructiva y por ello no permite identificar concretamente uno de estos lenguajes indecidibles que sirva como testigo para corroborar su conclusión (no por ello deja de ser cierto).

Nota 6.5.4

El Teorema 6.5.6 presenta una realidad abrumadora: en relación con la totalidad de problemas de decisión, son muy pocos (pero muy pocos) aquellos que un computador puede resolver. Aún más sorprendente es que, seguramente, pocas veces los problemas indecidibles surgen en la práctica de la programación. Por ejemplo, en este texto solo se han diseñado algoritmos de decisión implementables.

Ejercicios

1. Demuestre que $\sim$ es una relación de equivalencia. Es decir, reflexiva, simétrica y transitiva.
2. Demuestre que $\leq$ (sobre cardinales infinitos) es un orden parcial. Es decir, reflexiva y transitiva.
3. Use la propiedad de R. Dedekind en la Nota 6.5.1 para concluir que hay una cantidad infinita de números primos. ¿Es esta cantidad contable o no contable?
4. Demuestre que el mapa que invierte punto a punto los bits de las cadenas binarias en la Observación 1 en la demostración del Teorema 6.5.3 induce una función biyectiva entre X_0 y X_1 .
5. Demuestre que el mapa que interpreta cadenas binarias en números en base 10 en la Observación 2 en la demostración del Teorema 6.5.3 induce una función biyectiva entre X_1 y $\mathbb{N}$.
6. Elabore en más detalle la demostración del Teorema 6.5.5. En particular, diseñe una biyección entre Π^* y $\mathbb{N}$. Ayuda: como Π es finito, suponga que tiene $n \geq 1$ elementos. Generalice el razonamiento, pasando de base 2 a base n , de la demostración del Teorema 6.5.3.

Notas del capítulo y referencias

Los problemas de decisión han sido el principal objeto de estudio de la teoría de la computabilidad. Textos como el de M. Davis [Dav85] explican de manera detallada por qué es razonable restringir el estudio de la computabilidad e indecidibilidad a problemas de decisión. Explicaciones similares se pueden encontrar en [CLRS22, Eri19]. Basar el estudio de problemas de decisión en el marco de lenguajes formales es parte de la influencia del texto de Cormen et al. [CLRS22] sobre este texto. Sin embargo, la definición ‘genérica’ de problema de decisión, y las definiciones de aceptación y decisión de lenguajes basada en funciones parciales y totales no es la usual (secciones 6.1 y 6.3). En particular, la relación entre función total y decisión sobre lenguajes (Teorema 6.3.2) es novedosa. Se optó por combinar lenguajes y funciones porque permite abordar el objeto de estudio en cuestión a alto nivel, identificando los conceptos clave y sin perder generalidad. Las funciones parciales y totales son conceptos básicos de matemáticas, y son familiares para un grupo amplio de lectores.

La noción de computabilidad presentada en este capítulo es la usual y tratamientos similares se pueden encontrar en [Dav85, Har12]. Incluir definiciones de

computabilidad a nivel de funciones y de problemas es conveniente dado que permite entender el concepto de manera más general. De igual forma, es importante indicar las relaciones de conjunto computablemente enumerable y computable con las nociones de semidecidibilidad y decidibilidad, que son también comunes en literatura de ciencias de la computación. En la introducción del capítulo se indica que estas definiciones se dan sin necesidad de apelar a un modelo de computación en particular. Esto es posible porque las nociones presentadas coinciden en los principales modelos de computación. Hay tres categorías principales de modelos de computación: modelos secuenciales que incluyen las máquinas de Turing, las máquinas de Post y las máquinas de acceso aleatorio; modelos funcionales que incluyen el cálculo λ , las funciones recursivas generales, los sistemas de reescritura abstractos y la lógica combinatoria; y modelos concurrentes como el modelo de actores, las redes de Petri, la lógica de reescritura y los autómatas celulares. Material adicional de sistemas de reescritura abstractos se puede encontrar en [Ohl02] y de lógica de reescritura en [CDE⁺07]. Los demás modelos cuentan con literatura abundante que puede ser encontrada en textos clásicos o en internet.

El tratamiento que se hace de la existencia de problemas indecidibles o funciones no computables no es el común, ni para pregrados de computación ni de matemáticas. Esto puede ser así por la necesidad de usar aritmética cardinal que no es un tema estudiando, generalmente, en ninguno de estos currículos. La principal justificación para dar este tratamiento en el capítulo tiene que ver con la claridad del argumento (no constructivo): hay al menos una función que no puede ser calculada algorítmicamente. Los resultados acerca de cardinales utilizados en la Sección 6.5, incluyendo su aritmética, pueden ser constatados y profundizados en textos de teoría de conjuntos intuitiva como [Bre06, Hal11] y axiomática como [Kun11]. El texto [Ohl02] de E. Ohlebusch es una buena fuente de problemas indecidibles, como terminación y confluencia, en el marco de los sistemas de reescritura abstractos y de reescritura de términos, y de los problemas indecidibles clásicos de E. Post.

NP-completitud: P vs. NP

Suponga por un momento que alguien recién graduado está en un proceso de selección con miras a una posición de programador en una red social famosa. Esta persona se ha preparado árdidamente para la siguiente entrevista en la cual, como parte del proceso de selección, se valorarán sus conocimientos técnicos y prácticos en algoritmos. Por ello, el recién graduado ha estudiado algoritmos y practicado sus implementaciones, con una y mil variantes, en varios lenguajes de programación.

En la entrevista, se formula la siguiente pregunta: ¿se comprometería a resolver en un par de meses el problema de determinar el tamaño máximo de usuarios de la red social en el cual todos son amigos? La pregunta viene acompañada de un ofrecimiento con un jugoso salario de varios cientos de miles de dólares anuales. Aprovechando sus conocimientos en grafos, el sujeto de la entrevista se da cuenta de que el problema computacional, básicamente, es determinar el tamaño máximo de un subgrafo completo (i.e., la red social es el grafo y el subgrafo es el grupo de amigos).

Problema 7.0.1: Grupo de amigos

Entrada: Un grafo G con N vértices, $N \geq 0$

Salida: El tamaño máximo de un subgrafo completo en G .

Se sabe que hay varios súpercomputadores en la compañía, con capacidades de cómputo inimaginables para un recién graduado; hay un panorama prometedor. Todo parece estar dado para que el recién graduado acepte la pomposa vacante y para que, en un par de meses, el problema haya sido resuelto. Lamentablemente, el recién graduado decide no aceptar la propuesta argumentando que sería irresponsable comprometerse a resolver el problema formulado. Quien lidera el proceso de entrevista agradece al recién graduado por su honestidad, y le ofrece la posición

con un salario mayor y sin tener que comprometerse a resolver el problema de los amigos. Para muchas personas lo usual habría sido que la compañía optara por rechazar al candadito y terminar allí el proceso de selección; ¿qué sucedió?

Hace varias décadas se avanzó significativamente en la tarea de identificar problemas para los cuales no se conocen soluciones *eficientes*, es decir, para los cuales no se conocen algoritmos cuya complejidad temporal sea polinomial (o mejor/menor que polinomial). Como parte de este grupo, fueron identificados problemas de lógica, optimización, combinatoria e inteligencia artificial. Sin embargo, la tarea ha sido muy difícil y, al día de hoy, el proceso se encuentra en el “peor” de dos mundos. Por una parte, para los problemas identificados no se conocen soluciones eficientes. Por otra parte, no hay demostraciones que garanticen que dichas soluciones no existen. Es decir, la tarea de identificación aún se encuentra en una zona gris. Es, además, un área fértil para la investigación, sobretodo por las implicaciones que tienen estos problemas en la sociedad actual. Por ejemplo, el hecho de hacer transacciones en línea seguras depende, básicamente, de no contar con algoritmos que rápidamente puedan descomponer números en factores primos. Los problemas para los cuales no se conocen soluciones que corren en tiempo polinomial son denominados *intratables*, dado que cualquier solución conocida corre en tiempo exponencial (o peor) en función del tamaño de la entrada. Si una red social tiene millones de usuarios, cualquier algoritmo exponencial en su tamaño está lejos de terminar de ejecutarse en decenas de siglos.

En este capítulo se presenta una introducción a la teoría de la NP-completitud, un grupo de problemas cuyo estatus –en relación con la existencia de soluciones eficientes– es desconocido. Alrededor de esta clase de problemas se han obtenido resultados interesantes como, por ejemplo, la factibilidad de poderlos resolver eficientemente bastando con que uno de ellos tenga una solución eficiente. Además, esta exploración permite presentar y explicar una de las preguntas abiertas más interesantes en el campo de la computación (y quizás una de las más importantes de todas las ciencias exactas): la de determinar si el poder de cómputo de los algoritmos *determinísticos* polinomiales es igual al de los algoritmos *no determinísticos* polinomiales. Usualmente, esta pregunta se formula como P vs. NP y es tan importante (y famosa) que quien la resuelva tiene derecho a un premio de un millón de dólares. A pesar de que numerosos intentos han fracasado desde los 1970s en reclamar este premio, se cree que $P \neq NP$.

Para abordar el estudio de la NP-completitud se estudian los problemas dentro del marco de los lenguajes formales. Se limita, sin pérdida de generalidad, el estudio a problemas de decisión y se caracterizan las nociones de decidibilidad e indecidibilidad asociadas a este tipo de problemas. Esta exploración se hace con un lenguaje matemático básico (e.g., conjuntos, cadenas, funciones) y posteriormente, para introducir la noción de indecidibilidad, se utilizan cardinales, contables y no

contables. Es decir, además de estudiar las clases P y NP, y la noción de completitud de NP, se estudia la frontera existente entre lo que puede y no puede resolver mecánicamente un computador. El capítulo culmina con demostraciones que permiten concluir que algunos problemas, como el del grupo de amigos en la entrevista, es en realidad (hasta ahora) intratable.

Desde un punto de vista práctico, encontrarse con un problema NP-completo es una invitación a limitar las expectativas de encontrar un algoritmo eficiente que lo resuelva. En realidad, esto sería equivalente a pretender diseñar un algoritmo eficiente para resolver los problemas clásicos que han sido estudiados por años, y con los cuales han fracasado –hasta ahora– numerosas mentes en el intento. Quizás, eso fue lo que reconoció inmediatamente el recién graduado en su entrevista, y también por ello fue el mensaje de agradecimiento de quien le entrevistaba.

7.1. La clase P

La clase P es la colección de problemas de decisión (i.e., lenguajes) que pueden ser decididos por un algoritmo (i.e., por una función total computable) en tiempo polinomial. Por razones filosóficas, los lenguajes en esta clase son denominados *tratables*. Esta sección presenta la clase P, y con ella formaliza las nociones de problema aceptable y decidible en tiempo polinomial, y estudia algunas de sus propiedades. Se usan indistintamente las nociones de función computable y algoritmo para referirse al mismo objeto de estudio.

Definición 7.1.1: Algoritmo polinomial

Sea $L \subseteq \Sigma^*$ un lenguaje y $A : \Sigma^* \rightarrow \Sigma$ un algoritmo. Se dice que:

- A *acepta* L en tiempo polinomial si y solo si A acepta L y adicionalmente existe un $k \in \mathbb{N}$ tal que, para cualquier $x \in \Sigma^*$, A acepta x en tiempo $O(|x|^k)$.
- A *decide* L en tiempo polinomial si y solo si A decide L y adicionalmente existe un $k \in \mathbb{N}$ tal que, para cualquier $x \in \Sigma^*$, A decide x en tiempo $O(|x|^k)$.

A un algoritmo que toma tiempo polinomial se le denomina *algoritmo polinomial*. Note que estos son los algoritmos cuyo comportamiento, en función de la cantidad de pasos que realizan para resolver un problema, está asintóticamente acotado por un polinomio.

Definición 7.1.2: La clase P

Sea $L \subseteq \Sigma^*$ un lenguaje. Se dice que L está en la clase P si y solo si existe un algoritmo polinomial que decide L . Formalmente,

$$P = \{L \mid L \subseteq \Sigma^* \wedge \text{“hay un algoritmo de decisión polinomial tal que } L = L(A)\text{”}\}.$$

De acuerdo con la Definición 7.1.2, la clase P es una colección de lenguajes, i.e., $P \subseteq \mathcal{P}(\Sigma^*)$. Entonces, para afirmar que un problema L está en P debe demostrarse que L puede ser decidido en tiempo polinomial. Pueden existir varios algoritmos de decisión para L ; es suficiente con que uno de ellos sea polinomial para afirmar que dicho lenguaje está en P.

Ejemplo 7.1.1

Son ejemplos de lenguajes en P:

- Buscar un elemento en un arreglo (si la operación de igualdad se calcula en tiempo polinomial; lo usual es hacerlo en tiempo constante). El lenguaje asociado es:

$$\text{ARRFIND} = \{\langle A[0..N]x \rangle \mid \text{“}A \text{ es un arreglo y } x \text{ está en } A\text{”}\}.$$

- Determinar si un grafo es acíclico. El lenguaje asociado es:

$$\text{ACYCLIC} = \{\langle G \rangle \mid \text{“}G \text{ es un grafo acíclico”}\}.$$

- Determinar si un número es primo. El lenguaje asociado es:

$$\text{PRIME} = \{\langle N \rangle \mid \text{“}N \text{ es un número primo”}\}.$$

Como se mencionó al inicio de la sección, un lenguaje que está en P se denomina tratable. Esto equipara problemas para los cuales hay soluciones que corren en tiempo lineal con otros para los cuales solo haya soluciones que toman tiempo $\Theta(N^{1000})$, en donde N es el tamaño del problema. Los primera clase de algoritmos es útil en la práctica, mientras que claramente la segunda clase no lo es. A favor de esta identificación y categorización de “algoritmo eficiente” hay tres motivos, aún razonables a la luz de las dificultades anteriores:

- Primero, son pocos los problemas en P para los que no se conocen algoritmos acotados por un polinomio con grado razonable (e.g., que un grado no superior a 4 o 5). Esto sucede, principalmente, porque es común que las soluciones sean mejoradas (en relación con su comportamiento temporal asintótico) una vez se establece un primer algoritmo que resuelve un problema de interés.
- Segundo, esta noción de ejecución eficiente es compatible con diferentes modelos de computación, e.g., máquinas de Turing y máquinas de acceso aleatorio. Es

una categorización homogénea y fácilmente interpretable en los modelos de computación más comunes.

- Tercero, la clase P tiene propiedades interesantes en relación con los lenguajes que la conforman. Por ejemplo, es cerrada bajo complemento lo cual quiere decir que si un lenguaje se puede decidir eficientemente, lo mismo se puede hacer con su complemento. Lo mismo sucede con la unión, intersección, concatenación y clausura de Kleene. Esta tercera razón se retoma al final de la sección.

Un resultado importante es que las nociones de decisión y aceptación coinciden en P. Esto no es lo habitual en otras clases de complejidad computacional, lo cual hace de esta observación un resultado a resaltar.

Teorema 7.1.1

El siguiente conjunto es igual a P:

$$\{L \mid L \subseteq \Sigma^* \wedge \text{“hay un algoritmo de aceptación polinomial tal que } L = L(A)\text{”}\}.$$

Demostración

El objetivo es demostrar:

$$P = \{L \mid L \subseteq \Sigma^* \wedge \text{“hay un algoritmo de aceptación polinomial tal que } L = L(A)\text{”}\}.$$

Se procede por doble implicación.

($\Rightarrow$): Si $L \in P$, entonces hay un algoritmo de decisión polinomial A tal que $L(A) = L$ y, para cualquier $x \in \Sigma^* \setminus L$, se tiene que $A(x) = 0$ (por definición de decisión). Luego, A también es un algoritmo de aceptación para L . Es decir, hay un algoritmo polinomial que acepta L (en este caso, el mismo A).

($\Leftarrow$): Si L es aceptado por un algoritmo polinomial, por ejemplo B , entonces $L(B) = L$. Sin embargo, no necesariamente $B(x) = 0$ para cualquier $x \in \Sigma^* \setminus L$ dado que B puede no responder para algunas de estas cadenas (i.e., B puede estar indeterminado para ciertas cadenas binarias). Como B es polinomial, hay un polinomio $p(\cdot)$ tal que para cualquier $x \in \Sigma^*$, B toma tiempo $kp(|x|)$ cuando responde, para alguna constante k . Ahora, considere un algoritmo A que hace lo siguiente al ser invocado con una cadena x (i.e., para cada llamado $A(x)$):

- Hacer el llamado $B(x)$ y espera a que B ejecute $kp(|x|)$ operaciones básicas.
- Si este llamado retorna el valor $y \in \Sigma$, entonces A retorna y ;
- De lo contrario, si no retorna ningún valor, entonces A retorna 0.

Note que A corre en tiempo $O(p)$ y, además de aceptar L , también decide L dado que si $x \notin L$, entonces $A(x) = 0$.

En conclusión, estos dos conjuntos son iguales.

Los lenguajes en P son cerrados bajo algunas operaciones de conjuntos, lo cual tiene una ventaja práctica para la construcción de algoritmos de decisión eficientes. Por ejemplo, si se conocen soluciones polinomiales para dos lenguajes en P , automáticamente se sabe que hay un algoritmo eficiente para resolver la unión de los lenguajes dado que la clase P es cerrada bajo unión. Las propiedades de clausura más notorias sobre P se enuncian en el Teorema 7.1.2.

Teorema 7.1.2

Las siguientes afirmaciones son ciertas acerca de P :

1. Es cerrado bajo complemento; es decir, si $L \in P$, entonces

$$\bar{L} \in P.$$

2. Es cerrado bajo unión; es decir, si $L_1 \in P$ y $L_2 \in P$, entonces

$$(L_1 \cup L_2) \in P.$$

3. Es cerrado bajo intersección; es decir, si $L_1 \in P$ y $L_2 \in P$, entonces

$$(L_1 \cap L_2) \in P.$$

4. Es cerrado bajo concatenación; es decir, si $L_1 \in P$ y $L_2 \in P$, entonces

$$(L_1 \cdot L_2) \in P.$$

5. Es cerrado bajo la clausura de Kleene; es decir, si $L \in P$, entonces

$$L^* \in P.$$

Demostración

Se presenta una demostración de 2 y se proponen las demás demostraciones como ejercicios al lector. Suponga que $L_1 \in P$ y $L_2 \in P$; el objetivo es demostrar que $(L_1 \cup L_2) \in P$. Por las suposiciones, hay algoritmos polinomiales de decisión A_1 y A_2 tales que $L(A_1) = L_1$ y $L(A_2) = L_2$. Considere el siguiente algoritmo para cualquier $x \in \Sigma^*$:

- (i) Invocar $A_1(x)$ y guardar la respuesta.
- (ii) Invocar $A_2(x)$ y guardar la respuesta.

- (iii) Si al menos una de las respuestas obtenidas en (i) y (ii) es 1, entonces retornar 1; de lo contrario, retornar 0.

Note que el algoritmo propuesto retorna 1 si $A_1(x) = 1 \vee A_2(x) = 1$, es decir, si $x \in (L_1 \cup L_2)$. En cualquier otro caso retorna 0. Además, el algoritmo es polinomial dado que consta de 3 pasos y cada uno de ellos toma tiempo polinomial. En conclusión, $(L_1 \cup L_2) \in P$ si L_1 y L_2 están en P.

Finalmente, un llamado a la precaución cuando se componen algoritmos polinomiales. En general, no es cierto que un algoritmo sea polinomial cuando hace una cantidad polinomial de llamadas a subrutinas que toman tiempo polinomial cada una (ver Ejercicio 4). Lo que sí es cierto, es que si la cantidad de llamadas a las subrutinas está acotada por una constante, entonces necesariamente el algoritmo resultante corre en tiempo polinomial.

Ejercicios

1. Verdadero o falso: un algoritmo polinomial puede aceptar un lenguaje indecidible. Justifique su respuesta.
 2. Complete la demostración del Teorema 7.1.2:
 - Demuestre la propiedad de cerramiento bajo complemento (i.e., Teorema 7.1.2.1).
 - Demuestre la propiedad de cerramiento bajo intersección (i.e., Teorema 7.1.2.3).
 - Demuestre la propiedad de cerramiento bajo concatenación (i.e., Teorema 7.1.2.4).
 - Demuestre la propiedad de cerramiento bajo la clausura de Kleene (i.e., Teorema 7.1.2.5).
 3. Demuestre que si un algoritmo hace a lo sumo una cantidad constante de llamados a un algoritmo polinomial (a modo de subrutina) y adicionalmente realiza una cantidad de pasos en tiempo polinomial, entonces el algoritmo corre en tiempo polinomial.
 4. Justifique con un ejemplo por qué si un algoritmo hace a lo sumo una cantidad polinomial de llamados a un algoritmo polinomial (a modo de subrutina), entonces no necesariamente el algoritmo corre en tiempo polinomial. Ayuda: apoyarse en el triángulo de Pascal.
-

7.2. La clase NP

La clase NP es la colección de problemas de decisión que pueden ser verificados en tiempo polinomial. Esta sección presenta la clase NP e introduce el concepto de algoritmo de verificación.

Definición 7.2.1: Algoritmo de verificación

Un *algoritmo de verificación* es una función total computable

$$V : \Sigma^* \times \Sigma^* \rightarrow \Sigma.$$

Se dice que V *verifica* una entrada $x \in \Sigma^*$ si y solo si existe un *certificado* $y \in \Sigma^*$ tal que $V(x, y) = 1$.

Un algoritmo de verificación es una función computable con dos argumentos, en la cual el primer argumento es la cadena de entrada a verificar, y el segundo el certificado o testigo para dicha verificación.

Habiendo definido qué es un algoritmo de verificación, se procede a definir el significado de verificar un lenguaje.

Definición 7.2.2: Verificación de lenguajes

Sea $V : \Sigma^* \times \Sigma^* \rightarrow \Sigma$ un algoritmo de verificación. El *lenguaje verificado* por V es el conjunto:

$$L(V) = \{x \in \Sigma^* \mid \text{“existe un certificado } y \in \Sigma^* \text{ tal que } V(x, y) = 1\text{”}\}.$$

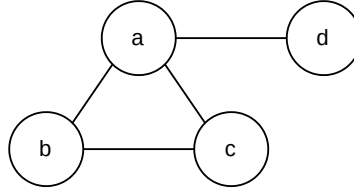
Un algoritmo V verifica un lenguaje L cuando para cualquier $x \in L$ existe un certificado y que el algoritmo puede utilizar para comprobar la pertenencia de x a L . Adicionalmente, no pueden existir certificados con los que V compruebe la pertenencia a L de cadenas que no hacen parte del lenguaje. Es decir, que al afirmar $L = L(V)$ se está entendiendo que no existen certificados y para los cuales $V(x, y) = 1$ cuando $x \notin L$. En el ejemplo de la red social que aparece en la introducción del capítulo, un certificado para comprobar que existe un grupo de 1 000 amigos totalmente conectados es una lista de 1 000 (puede ser una de varias) amigos que realmente están conectados, par a par. Si un grupo así no existe, es imposible contar con dicho certificado.

Ejemplo 7.2.1

Considere el siguiente problema de decisión:

$\text{CLIQUE} = \{ \langle G = (V, E), k \rangle \mid \text{"}G \text{ tiene un subgrafo completo de tamaño } k\text{"} \}$.

Considere el siguiente grafo $G = (V, E)$ con vértices $V = \{a, b, c, d\}$ y arcos $E = \{(a, b), (a, c), (a, d), (b, c)\}$:



Note que:

- $\langle G, 2 \rangle \in \text{CLIQUE}$ dado que $\langle \{a, b\} \rangle$ es un certificado para el cual un algoritmo de verificación A para CLIQUE responde

$$A(\langle G, 2 \rangle, \langle \{a, b\} \rangle) = 1.$$

- $\langle G, 3 \rangle \in \text{CLIQUE}$ dado que $\langle \{a, b, c\} \rangle$ es un certificado para el cual un algoritmo de verificación A para CLIQUE responde

$$A(\langle G, 3 \rangle, \langle \{a, b, c\} \rangle) = 1.$$

- $\langle G, 4 \rangle \notin \text{CLIQUE}$ dado que para cualquier $y \in \Sigma^*$ y algoritmo de verificación A para CLIQUE se tiene que

$$A(\langle G, 4 \rangle, y) = 0 \quad (\text{¿por qué?}).$$

La clase NP es la colección de lenguajes que pueden ser verificados en tiempo polinomial.

Definición 7.2.3: La clase NP

Sea $L \subseteq \Sigma^*$ un lenguaje. Se dice que L está en la clase NP si y solo si existe un algoritmo polinomial que verifica L . Formalmente,

$$\text{NP} = \{ L \mid L \subseteq \Sigma^* \wedge \text{"hay un algoritmo de verificación polinomial tal que } L = L(A)\text{"} \}.$$

De acuerdo con la Definición 7.2.3, la clase NP (al igual que P) es una colección de lenguajes, i.e., $\text{NP} \subseteq \mathcal{P}(\Sigma^*)$. Entonces, para afirmar que un problema L está en NP debe demostrarse que L puede ser verificado en tiempo polinomial. Pueden existir varios algoritmos de verificación para L ; es suficiente con que uno de ellos sea polinomial para afirmar que dicho lenguaje está en NP.

Nota 7.2.1

La sigla NP proviene de una definición alternativa de esta clase de complejidad: de la clase de problema de decisión que pueden ser decididos en tiempo polinomial por un algoritmo en una máquina de Turing *no determinística* (ver Ejercicio 4).

Ejemplo 7.2.2

Son ejemplos de lenguajes en NP:

- El problema de buscar un elemento en un arreglo (si la operación de igualdad se calcula en tiempo polinomial; lo usual es hacerlo en tiempo constante):

$$\text{ARRFIND} = \{ \langle A[0..N], x \rangle \mid \text{"}A \text{ es un arreglo y } x \text{ está en } A\text{"} \}.$$

Un algoritmo de verificación tiene como certificado una posición n de x en $A[0..N]$. Note que la verificación se puede hacer en tiempo polinomial: se debe determinar si $0 \leq n < N$ y $A[n] = x$. Estas dos operaciones toman, a lo sumo, tiempo polinomial.

- El problema de determinar si un grafo es acíclico:

$$\text{ACYCLIC} = \{ \langle G \rangle \mid \text{"}G \text{ es un grafo acíclico"} \}.$$

Un algoritmo de verificación tiene como certificado una lista de vértices. Note que la verificación se puede hacer en tiempo polinomial: se debe determinar si la lista $v_0, \dots, v_{N-1}$ corresponde a todos los vértices de G y si determinan un orden topológico de G . Esta última operación se puede hacer con N búsquedas (en profundidad o amplitud) desde cada uno de los vértices de G (¿cómo?).

- El problema de determinar si un grafo tiene un subgrafo completo de un tamaño dado:

$$\text{CLIQUE} = \{ \langle G = (V, E), k \rangle \mid \text{"}G \text{ tiene un subgrafo completo de tamaño } k\text{"} \}.$$

Un algoritmo de verificación tiene como certificado una lista de vértices. Note que la verificación se puede hacer en tiempo polinomial: se debe determinar si la lista $v_0, \dots, v_{k-1}$ corresponde a algunos vértices de G y si determinan un subgrafo completo de G . Esta última operación se puede hacer en orden $O(k)$ suponiendo que pertenencia al conjunto de arcos se puede hacer en tiempo constante (¿cómo?).

De acuerdo con el Ejemplo 7.2.2, algunos problemas en P también están en NP . En realidad, como se establece en el Teorema 7.2.1, cualquier problema que puede ser decidido en tiempo polinomial también puede ser verificado en tiempo polinomial.

Teorema 7.2.1
$$P \subseteq NP.$$
Demostración

Sea L un lenguaje y suponga que $L \in P$. El objetivo es demostrar que $L \in NP$. Como $L \in P$, entonces hay un algoritmo A que en tiempo polinomial decide $L = L(A)$. Considere el siguiente algoritmo de verificación V con parámetros x (el elemento a determinar si hace parte o no de L) y y (un certificado):

```
1 def V(x, y):  
2   return A(x)
```

Note que V es un algoritmo que corre en tiempo polinomial porque A es polinomial. También note que V verifica L porque:

- Si $V(x, y) = 1$, entonces se tiene que $A(x) = 1$; es decir, V responde afirmativamente cuando $x \in L$.
- Si $V(x, y) = 0$, entonces se tiene que $A(x) = 0$; es decir, V responde negativamente cuando $x \notin L$.

Como V es un algoritmo polinomial de verificación para L , se concluye que $L \in NP$.

El Teorema 7.2.1 resuelve una mitad de la pregunta ¿ P vs. NP ? Establece que para responderla es necesario fijarse en la inclusión de NP en P , es decir, en la pregunta ¿ $NP \subseteq P$? Se cree que esta segunda pregunta es falsa, pero ninguna demostración ha podido ser construida en más de 50 años. Es decir, esta es la parte que ha ocupado muchas horas de muchos científicos y que aún hace falta por resolver, y así definitivamente completar el rompecabezas que dirimiría la relación exacta entre P y NP .

Nota 7.2.2

En la demostración del Teorema 7.2.1 se diseña el algoritmo de verificación V a partir del algoritmo de aceptación A . Note que ante cualquier invocación $V(x, y)$, el certificado y es ignorado completamente por V , lo cual es completamente válido.

Esta observación da pie a una segunda observación, muy importante, acerca de los algoritmos de verificación que se deseen usar como testigos en demostraciones acerca de la pertenencia de lenguajes a la clase **NP**. Como para cualquier algoritmo de verificación V , cadena de entrada x y certificado y , el llamado $V(x, y)$ debe correr en tiempo polinomial, no tiene sentido que el tamaño de y sea extremadamente grande. En particular, para que V sirva como testigo en una demostración de pertenencia a **NP** en donde se use el certificado y , se debe dar que $|y| \in O(|x|^k)$ para alguna constante $k \in \mathbb{N}$. Es decir, en la práctica, el tamaño de los certificados debe ser siempre polinomial en función del tamaño de la entrada para que los algoritmos de verificación puedan ser útiles (para trabajar con la clase **NP**).

Ejercicios

1. Diseñe algoritmos de verificación, respectivamente, para los lenguajes $L = \emptyset$ y $L = \Sigma^*$.
2. En el Ejemplo 7.2.1 se indica que $\{a, b\}$ es un certificado para determinar $\langle G, 2 \rangle \in \text{CLIQUE}$. ¿Cuáles son todos los testigos para $\langle G, 2 \rangle \in \text{CLIQUE}$? ¿Cuáles son todos los testigos para poder afirmar que $\langle G, 0 \rangle$ y $\langle G, 1 \rangle$ están en **CLIQUE**, respectivamente?
3. Suponga que se establece que $\langle G, n \rangle \in \text{CLIQUE}$. ¿Para qué valores de m automáticamente se tiene $\langle G, m \rangle \in \text{CLIQUE}$? Justifique su respuesta.
4. Hay una definición alternativa de la clase **NP** que se basa en algoritmos no determinísticos que corren en máquinas de Turing. Investigue acerca de esta definición, enúnciela y explique por qué es equivalente a la dada en la Definición 7.2.3. El nombre **NP** proviene de, en inglés, *nondeterministic polynomial*.
5. En el Ejemplo 7.2.2 se demuestra que los lenguajes **ARRFIND**, **ACYCLIC** y **CLIQUE** están en **NP**. En el espíritu de la demostración del Teorema 7.2.1, diseñe nuevas demostraciones para la pertenencia de estos tres lenguajes a **NP** en las cuales los algoritmos de verificación ignoren los certificados.

7.3. Reducciones polinomiales

Una estrategia para resolver problemas consiste en transformar el problema de interés en otro para el cual exista una solución. En esta sección, se estudian las reducciones polinomiales como herramienta para transformar, de manera eficiente, un problema de decisión en otro.

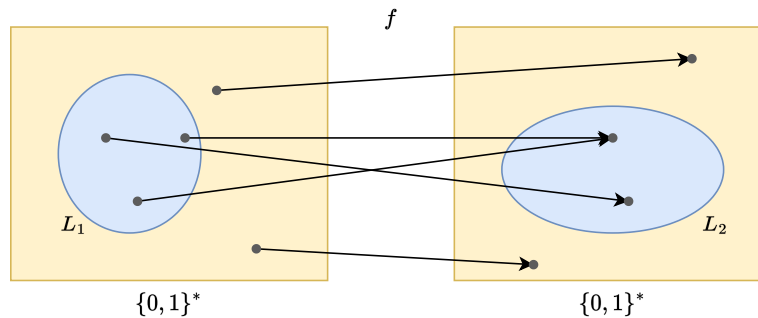
Definición 7.3.1: Reducción polinomial

Sean $L_1 \subseteq \Sigma^*$ y $L_2 \subseteq \Sigma^*$ problemas de decisión. Una *reducción* de L_1 a L_2 es una función $f : \Sigma^* \rightarrow \Sigma^*$ computable tal que para cualquier $x \in \Sigma^*$ se tiene que

$$x \in L_1 \iff f(x) \in L_2.$$

Si f es computable en tiempo polinomial entonces se llama *reducción polinomial*. Se escribe $L_1 \leq_P L_2$ para indicar que hay una reducción polinomial de L_1 a L_2 .

Una reducción polinomial de L_1 a L_2 es un algoritmo polinomial f que transforma instancias de L_1 a instancias de L_2 de tal manera que $x \in L_1$ y $f(x) \in L_2$ coincidan. Note, entonces, que la pregunta de si x pertenece o no a L_1 se puede responder directamente averiguando la respuesta a la pregunta de si $f(x)$ pertenece o no a L_2 . Con una reducción de L_1 a L_2 , como f , nunca será el caso que $x \in L_1$ y $f(x) \notin L_2$ o que $x \notin L_1$ y $f(x) \in L_2$. De manera más general, una reducción preserva pertenencia y no pertenencia de sus imágenes en relación con sus respectivas preimágenes. Una reducción de L_1 a L_2 puede dibujarse de la siguiente manera.



Note que no hay propiedades adicionales que se exijan a las reducciones. Por ejemplo, no deben ser, necesariamente, inyectivas o sobreyectivas en relación con los conjuntos de salida y llegada, como se explora en el Ejercicio 1.

Nota 7.3.1: Reducciones de Karp y Cook

A las reducciones presentadas en la Definición 7.3.1 se les conoce también como *reducciones de Karp* o *reducciones muchos a uno* en honor al científico de la computación Richard M. Karp. Hay otro tipo de reducciones, más generales, denominadas reducciones de Cook en honor al científico de la computación Stephen Cook (también son llamadas reducciones de Turing). Las reducciones de Cook se pueden definir para un grupo de problemas ‘más’ general como lo son los de optimización y construcción.

En este texto se prefieren las reducciones de Karp sobre las de Cook por dos razones principalmente. Primero, estas se formulan exclusivamente para reducciones entre problemas de decisión. Segundo, la clase NP es cerrada bajo reducciones de Karp.

Nota 7.3.2: Richard M. Karp y Stephen Cook

Richard M. Karp y Stephen Cook son reconocidos científicos de la computación.

Richard M. Karp recibió el premio Turing en 1985 por sus contribuciones a la teoría de algoritmos y complejidad. Estas contribuciones incluyen algoritmos eficientes para el problema de corte mínimo o flujo máximo en grafos (junto con Jack Edmonds), la identificación de computabilidad polinomial con la noción de eficiencia algorítmica y por sus contribuciones a la teoría de la NP-completitud. Por ejemplo, en 1972 publicó el artículo “Reducibility Among Combinatorial Problems” en donde demostró la existencia de 21 problemas NP-completos; este es uno de los artículos más citados en el área de algoritmos, computabilidad y complejidad. En 1973 publicó, junto con John Hopcroft, el algoritmo más eficiente hasta la fecha que resuelve el problema de encontrar la máxima cardinalidad de emparejamientos en grafos bipartitos, y en 1987, junto con Michael Rabin, un algoritmo eficiente para búsqueda en cadenas.

Stephen Cook es considerado uno de los padres fundadores de la teoría de complejidad computacional. En el artículo “The Complexity of Theorem Proving Procedures” que apareció en 1971, definió las nociones de reducción polinomial (à la Cook) y de NP-completitud, demostró que el problema de satisfacibilidad proposicional es NP-completo (resultado compartido con L. Levin), y formuló la pregunta de P vs. NP. Recibió el premio Turing en 1982.

Reducir un problema L_1 a otro L_2 significa diseñar un algoritmo que resuelva L_1 suponiendo que existe un algoritmo para resolver L_2 . Este tipo de enfoques se siguen desde, prácticamente, los primeros cursos de algoritmos y programación en donde se pueden hacer llamados desde funciones/procedimientos/módulos a otras funciones/procedimientos/módulos.

Las reducciones polinomiales permiten demostrar formalmente que un problema es al menos tan difícil como otro, módulo un factor de tiempo polinomial. Es decir, si $L_1 \leq_P L_2$, entonces L_1 no es más difícil que L_2 excepto por un factor polinomial. Por esta razón, principalmente, se usa la notación $\leq_P$. Por ejemplo, las reducciones polinomiales son una herramienta poderosa para demostrar que algunos problemas de decisión hacen parte de la clase P.

Teorema 7.3.1

Sean $L_1 \subseteq \Sigma^*$ y $L_2 \subseteq \Sigma^*$. Si $L_1 \leq_P L_2$ y $L_2 \in P$, entonces $L_1 \in P$.

Demostración

Se suponen $L_1 \leq_P L_2$ y $L_2 \in P$, y se demuestra $L_1 \in P$. Es decir, el objetivo es demostrar que existe un algoritmo, por ejemplo, A_1 que decide L_1 en tiempo polinomial. Por la primera suposición, existe una reducción polinomial $f : \Sigma^* \rightarrow \Sigma^*$ de L_1 a L_2 . Por la segunda suposición, hay un algoritmo, por ejemplo, A_2 que decide L_2 en tiempo polinomial. Considere el siguiente algoritmo:

```

1 def A1(x):
2   return A2(f(x))

```

Claramente, A_1 corre en tiempo polinomial (¿por qué?). Basta por demostrar que A_1 decide L_1 y se procede por casos sobre $x \in \Sigma^*$:

($\Rightarrow$): Si $x \in L_1$, entonces $f(x) \in L_2$ dado que f es una reducción. Como A_2 decide L_2 , se tiene que $A_2(f(x)) = 1$. Es decir, $A_1(x) = 1$ cuando $x \in L_1$.

($\Leftarrow$): Si $x \notin L_1$, entonces $f(x) \notin L_2$ dado que f es una reducción. Como A_2 decide L_2 , se tiene que $A_2(f(x)) = 0$. Es decir, $A_1(x) = 0$ cuando $x \notin L_1$.

En conclusión, $L_1 \in P$.

Nota 7.3.3

Las demostraciones que involucran reducciones polinomiales, generalmente siguen un mismo patrón. Para reducir un problema L_1 a un problema L_2 en tiempo polinomial, es necesario hacer tres cosas:

1. Diseñar un algoritmo, for ejemplo f , que en tiempo polinomial transforme cualquier instancia $x \in L_1$ a alguna instancia $y \in L_2$.
2. Demostrar que si $x \in L_1$, entonces $f(x) \in L_2$.
3. Demostrar que si $f(x) \in L_2$, entonces $x \in L_1$.

Diseñar reducciones polinomiales y demostrar que en realidad funcionan es una tarea que demanda experiencia, por lo cual se sugiere al lector practicar.

El tercer paso en la Nota 7.3.3 puede ser confuso para algunos lectores: si ya se sabe que para cualquier $x \in L_1$ se cumple que $f(x) \in L_2$, ¿por qué es necesario este paso? La razón es que aún si se cumplen las condiciones (1) y (2) en la Nota 7.3.3, puede ser el caso que la reducción esté mal diseñada: es posible que $x \notin L_1$ y $f(x) \in L_2$. Es decir, que la reducción diseñada además de enviar instancias “buenas” de L_1 a instancias “buenas” de L_2 , también está enviando instancias “malas” de L_1 a instancias buenas de L_2 . De ser así, no se estaría respetando la definición de reducción.

Ejercicios

1. Sean $L_1 \subseteq \Sigma^*$ y $L_2 \subseteq \Sigma^*$, y $f : \Sigma^* \rightarrow \Sigma^*$ una reducción de L_1 a L_2 . Para cada uno de los siguientes casos, proponga testigos concretos para L_1 , L_2 y f tales que:
 - a) La reducción f es inyectiva.
 - b) La reducción f es sobreyectiva.
 - c) La reducción f es biyectiva.
 - d) La reducción f no es inyectiva.
 - e) La reducción f no es sobreyectiva.
 - f) La reducción f no es biyectiva.

¿En cuáles de los casos anteriores las reducciones son polinomiales?
2. Sean $L_1 \subseteq \Sigma^*$ y $L_2 \subseteq \Sigma^*$, y $f : \Sigma^* \rightarrow \Sigma^*$. Se define $f[L_1]$ como el conjunto imagen de L_1 bajo f , es decir,

$$f[L_1] = \{f(x) \mid x \in L_1\}.$$

Demuestre o refute:

- a) Si $f[L_1] \subseteq L_2$, entonces f es una reducción.

- b) Si f es una reducción, entonces $f[L_1] \subseteq L_2$.
- c) Si $f[L_1] = L_2$, entonces f es una reducción.
- d) Si f es una reducción, entonces $f[L_1] = L_2$.

3. Demuestre que la relación $\leq_P$ es transitiva.
4. Demuestre que, para cualesquiera problemas de decisión L_1 y L_2 , la siguiente equivalencia es cierta:

$$L_1 \leq_P \overline{L_2} \iff \overline{L_1} \leq_P L_2.$$

5. Investigue acerca de los 21 problemas publicados por R. Karp en “Reducibility Among Combinatorial Problems”. Especifique 5 de ellos como problema de decisión.
6. Investigue acerca del problema de máxima cardinalidad de emparejamientos en grafos bipartitos y especifíquelo. Además, formule un ejemplo de una instancia del problema y una solución asociada.
7. Explique qué problema resuelve el algoritmo de Rabin y Karp para cadenas. Dé una breve explicación del algoritmo e ilustre su funcionamiento con un ejemplo sencillo.
8. Considere que está diseñando una calculadora proposicional: dada una fórmula, esta calculadora debe responder si la fórmula dada es satisfacible o no (i.e., si existe una asignación a sus variables proposicionales de tal manera que la proposición resulte verdadera). El hardware de la calculadora solo permite construir proposiciones con base en negación ($\neg$) y disyunción ($\vee$). Sin embargo, la interfaz humano computador debe permitir analizar proposiciones que usen otros conectivos lógicos como equivalencia ($\equiv$), implicación ($\Rightarrow$) y conjunción ($\wedge$).

Como $\{\neg, \vee\}$ es un conjunto suficientemente completo de conectivos (i.e., cualquier proposición se puede expresar en términos de ellos), se entiende que la calculadora tiene futuro. Por ejemplo, la expresión $p \Rightarrow q$ (en donde p y q son variables proposicionales) se puede compilar en la proposición

$$(\neg p \vee q).$$

Algo similar puede hacerse para la equivalencia y la conjunción.

- a) Diseñe un algoritmo recurrente que dada una proposición ϕ la transforme en una proposición lógicamente equivalente, pero que solo usa negaciones y conjunciones, además de las mismas variables proposicionales.
 - b) ¿El algoritmo propuesto es una reducción?
 - c) ¿El algoritmo propuesto es una reducción polinomial?
9. Sean $L_1 \subseteq \Sigma^*$ y $L_2 \subseteq \Sigma^*$. Demuestre que si $L_1 \leq_P L_2$ y $L_2 \in \text{NP}$, entonces $L_1 \in \text{NP}$.

7.4. La clase NPC

La clase NPC es una colección nutrida de problemas prácticos en NP y que son intrínsecamente difíciles en el sentido en el cual no se conocen soluciones polinomiales (i.e., eficientes) para ellos. Una de sus principales propiedades radica en que si se encuentra una solución eficiente para cualquiera de ellos, entonces se tendría que $P = NP$. Sin embargo, se cree (pero no se ha demostrado) que no existen algoritmos eficientes para ninguno de ellos y, en consecuencia, $P \neq NP$. Esta sección presenta la clase NPC, una noción de “ser difícil” para los problemas en NP y un problema que hace parte de NPC.

En el área de la teoría de complejidad computacional, se identifica una colección de problemas que, informalmente, se pueden catalogar como al menos tan difíciles como cualquiera de los problemas más difíciles en NP. A esta clase se le denomina NP-hard.

Definición 7.4.1: NP-hard

Sea $L \subseteq \Sigma^*$. Se dice que L es NP-hard (leído, NP-difícil) si y solo si cualquier $L' \in NP$ es tal que $L' \leq_P L$. Alternativamente,

$$NP\text{-hard} = \{L \subseteq \Sigma^* \mid (\forall L' \in NP) L' \leq_P L\}.$$

Un problema NP-hard es un problema difícil relativamente en comparación con cualquier problema en NP (y, por ende, en P). Note que no necesariamente un problema de estos está obligado a ser parte de NP. Es decir, la definición de esta clase no excluye la posibilidad de que existan problemas difíciles fuera de NP.

Es importante llamar la atención del lector acerca de una dificultad práctica para el uso de la noción NP-hard: para que un problema de decisión sea admitido a esta clase, se debe demostrar que desde *cualquier* problema en NP hay una reducción polinomial a este. Como hay una cantidad infinita (¿contable o no contable?) de problemas en NP, ¿cómo lograr demostrar pertenencia a esta clase? Es más, no es obvio que esta clase no sea vacía. Afortunadamente, Stephen Cook (1971) y Leonid Levin (1973) de manera independiente demostraron que el problema CIRCUIT-SAT es NP-hard, uno de los principales logros de las ciencias de la computación (ver Nota 7.3.2), y que se conoce como el Teorema de Cook y Levin. Este teorema es acerca de la dificultad para resolver el problema de satisfacibilidad de circuitos (combinatorios) Booleanos.

Un *circuito combinatorio Booleano* es una colección no vacía de compuertas lógicas interconectadas por cables, en donde un cable conecta la salida de una compuerta con la entrada de otra, de tal forma que la salida de la primera se convierte en una de las entradas de la segunda. Los cables que no están conectados a la salida

de circuito alguno corresponden a las entradas del circuito, mientras que aquel que no está conectado a ninguna entrada es la salida del circuito (necesariamente hay una única salida). Con base en esta caracterización (de alto nivel) de lo que es un circuito combinatorio Booleano, se puede definir el problema de satisfacibilidad de circuitos: dado un circuito combinatorio construido con compuertas AND, OR y NOT, ¿existe alguna asignación (encendido/apagado) para los cables de entrada del circuito que enciendan el cable de salida? El lenguaje CIRCUIT-SAT es, entonces, el siguiente:

$$\text{CIRCUIT-SAT} = \{\langle C \rangle \mid \text{"}C \text{ es un circuito combinatorio Booleano satisfacible"}\}.$$

Teorema 7.4.1: El Teorema de Cook y Levin

$\text{CIRCUIT-SAT} \in \text{NP-hard}$.

La demostración formal del Teorema 7.4.1 requiere detalles técnicos más allá del alcance de este manuscrito. Además, intencionalmente, se ha hecho un recuento vago del problema dado que no es importante entrar en tantos detalles.

Nota 7.4.1

No incluir una demostración del Teorema de Cook y Levin, no puede opacar su gran importancia. Gracias a la transitividad de las reducciones polinomiales, para demostrar que un problema L es NP-hard, basta con demostrar $\text{CIRCUIT-SAT} \leq_P L$.

El problema de satisfacibilidad de circuitos, CIRCUIT-SAT, surge naturalmente en el área de optimización de hardware: si un circuito siempre resulta apagado, entonces este puede ser eliminado del diseño y sustituido por una única compuerta que siempre resulte apagada. Es fácil ver cómo en la práctica es deseable resolver eficientemente este tipo de problemas.

La clase NPC caracteriza aquellos problemas en NP que son los más difíciles de resolver.

Definición 7.4.2: Problemas NP-complete

Sea $L \subseteq \Sigma^*$. Se dice que L es NP-complete (leído, NP-completo) si y solo si:

1. $L \in \text{NP}$.
2. $L \in \text{NP-hard}$.

La clase NPC es la colección de todos los problemas NP-complete:

$$\text{NPC} = \{L \subseteq \Sigma^* \mid \text{"}L \text{ es NP-complete"}\}.$$

Un problema es NP-complete si está en la intersección entre las clases NP y NP-hard, es decir, que una demostración de pertenencia a la clase NPC requiere dos comprobaciones: una, que el problema está en NP; otra, que el problema está en NP-hard.

Teorema 7.4.2

CIRCUIT-SAT $\in$ NPC.

Demostración

Se propone como ejercicio al lector.

Como se muestra a continuación, la clase NPC suministra la esencia para atacar la pregunta P vs. NP: si existe un problema NP-complete decidible eficientemente, necesariamente cualquier problema en NP puede ser decidido eficientemente.

Teorema 7.4.3

Si $\text{NPC} \cap P \neq \emptyset$, entonces $P = \text{NP}$.

Demostración

Se supone $\text{NPC} \cap P \neq \emptyset$ y se demuestra $P = \text{NP}$. Por el Teorema 7.2.1 se sabe que $P \subseteq \text{NP}$, luego basta con demostrar $P \supseteq \text{NP}$. Como $\text{NPC} \cap P \neq \emptyset$, luego hay un $L \subseteq \Sigma^*$ tal que $L \in \text{NPC}$ y $L \in P$. Note que para cualquier $X \in \text{NP}$ se tiene que $X \leq_P L$ con, por ejemplo, reducción polinomial f como testigo. Como $L \in P$, entonces hay un algoritmo de decisión, por ejemplo, A que decide $L = L(A)$ en tiempo polinomial. Considere el siguiente algoritmo:

```

1 def B(x):
2   return A(f(x))

```

Note que B decide X en tiempo polinomial (los detalles de esta demostración se proponen como ejercicio al lector). Luego, cualquier lenguaje en NP puede ser decidido en tiempo polinomial, es decir, $P \supseteq \text{NP}$. En conclusión, $P = \text{NP}$.

Basándose en el Teorema 7.4.3, durante los últimos 50 años se ha desarrollado una carrera estrepitosa por encontrar algoritmos eficientes para decidir problemas NP-complete. Además del noble interés por mover las fronteras del conocimiento humano, quizás también sea llamativo el premio de US \$1,000,000 ofrecido por el Instituto Clay de Matemáticas para quien resuelva la pregunta P vs. NP. Hasta la fecha, al menos públicamente, no se conoce una respuesta definitiva a esta pregunta, a pesar de que anualmente se publican “demostraciones” equivocadas, en cualquiera de las dos direcciones. En la comunidad científica hay una clara inclinación por creer que $P \neq NP$, pero sin una demostración no se puede dejar en firme dicha intuición.

Nota 7.4.2

Es importante memorizarla la siguiente regla para trabajar con problemas NP-hard (y NP-complete):

Para demostrar que un problema de decisión L es NP-hard, se debe reducir polinomialmente cualquier problema NP-hard conocido a L .

Ejercicios

1. Investigue acerca de un problema de decisión NP-hard que no haga parte de NP. Especifique dicho problema y dé un ejemplo breve de una de sus instancias.
2. ¿El cardinal $\text{card}(\text{NP})$ del conjunto NP es contable o no contable? Justifique su respuesta.
3. Demuestre el Teorema 7.4.2.
4. Suministre una versión formal del problema CIRCUIT-SAT y del problema CNF-SAT de satisfacibilidad de proposiciones en forma normal conjuntiva. Suministre una reducción polinomial del primero al segundo, concluyendo que el segundo es NP-hard.
5. En la demostración del Teorema 7.4.3 se afirma que el algoritmo B decide el problema X en tiempo polinomial. Demuestre formalmente esta afirmación.
6. Demuestre que un problema L es NP-hard si y solo si $P = NP$ cuando L es decidible en tiempo polinomial.
7. Investigue y enuncie los problemas por los cuales actualmente el Instituto Clay de Matemáticas ofrece premios de US \$1,000,000.
8. Un lenguaje L es *completo* para una clase de lenguajes C con respecto a reducciones polinomiales si y solo si $L \in C$ y $L' \leq_P L$ para cualquier $L' \in C$. Demuestre que $\emptyset$ y Σ^* son los únicos lenguajes en P que no son completos para P con respecto a reducciones polinomiales.

Bibliografía

- [ABCC06] David L. Applegate, Robert E. Bixby, Vašek Chvátal, and William J. Cook, *The traveling salesman problem: A computational study*, Princeton Series in Applied Mathematics, Princeton University Press, 2006.
- [BB88] Gilles Brassard and Paul Bratley, *Algorithms: Theory and practice*, Prentice Hall, 1988.
- [Bei13] Wolfgang Bein, *Advanced techniques for dynamic programming*, pp. 41–92, Springer, New York, NY, 2013.
- [Bel57] R. Bellman, *Dynamic programming*, Princeton University Press, Princeton, 1957.
- [Bel58] Richard Bellman, *On a routing problem*, Quarterly of Applied Mathematics **16** (1958), 87–90.
- [Bel62] R. Bellman, *Dynamic programming treatment of the travelling salesman problem*, Journal of the ACM **9** (1962), no. 1, 61–63.
- [Bel84] Richard Bellman, *Eye of the hurricane: an autobiography*, World Scientific, 1984.
- [Ben84] J. Bentley, *Programming pearls: Algorithm design techniques*, Communications of the ACM **27** (1984), no. 9, 865–873.
- [Bha15] Harsh Bhasin, *Algorithms: Design and analysis*, Oxford University Press, 2015.
- [Bir89] R. S. Bird, *Algebraic identities for program calculation*, The Computer Journal **32** (1989), no. 2, 122–126.
- [Boh06] Jaime Bohórquez, *Diseño efectivo de programas correctos*, Escuela Colombiana de Ingeniería, 2006.
- [Bor26] Otakar Borůvka, *O jistém problému minimálním*, Práce Moravské Přírodovědecké Společnosti (1926).
- [BR75] James R. Bitner and Edward M. Reingold, *Backtrack programming techniques*, Communications of the ACM **18** (1975), no. 11, 651–656.
- [Bre06] Joseph Breuer, *Introduction to the theory of sets*, 1st ed., Dover Publications, 2006.

- [BS09] Jordan Bell and Brett Stevens, *A survey of known results and research areas for n -queens*, Discrete Mathematics **309** (2009), no. 1, 1–31.
- [CDE⁺07] Manuel Clavel, Francisco Durán, Steven Eker, Patrick Lincoln, Narciso Martí-Oliet, José Meseguer, and Carolyn Talcott, *All about maude*, 1st ed., Springer, 2007.
- [CLRS22] Thomas Cormen, Charles Leiserson, Ronald Rivest, and Clifford Stein, *Introduction to algorithms*, 4th ed., MIT Press, 2022.
- [Coh90] Edward Cohen, *Programming in the 1990s: an introduction to the calculation of programs*, Springer-Verlag, 1990.
- [Dav85] Martin Davis, *Computability and unsolvability*, Dover Publications, 1985.
- [Dij59] Edsger W. Dijkstra, *A note on two problems in connexion with graphs*, Numerische Mathematik **1** (1959), 269–271.
- [Dij72] ———, *Notes on structured programming*, Structured Programming (Ole-Johan Dahl, Edsger W. Dijkstra, and C. A. R. Hoare, eds.), Academic Press, London, 1972.
- [Dij76] Edsger Wybe Dijkstra, *A discipline of programming*, Prentice-Hall, 1976.
- [Dre02] S. E. Dreyfus, *Richard bellman on the birth of dynamic programming*, Operations Research **50** (2002), no. 1, 48–51.
- [Edm71] Jack Edmonds, *Matroids and the greedy algorithm*, Mathematical Programming **1** (1971), 127–136.
- [Eri19] Jeff Erickson, *Algorithms*, 2019.
- [Fen94] Peter M. Fenwick, *A new data structure for cumulative frequency tables*, Software: Practice and Experience **24** (1994), no. 3, 327–336.
- [Flo62] Robert W. Floyd, *Algorithm 97: Shortest path*, Communications of the ACM **5** (1962), no. 6, 345.
- [FT87] Michael L. Fredman and Robert Endre Tarjan, *Fibonacci heaps and their uses in improved network optimization algorithms*, Journal of the ACM **34** (1987), no. 3, 596–615.
- [GB65] Solomon W. Golomb and Leonard D. Baumert, *Backtrack programming*, Journal of the ACM **12** (1965), no. 4, 516–524.
- [GH85] Ronald L. Graham and Pavol Hell, *On the history of the minimum spanning tree problem*, Annals of the History of Computing **7** (1985), no. 1, 43–57.
- [Gri81] David Gries, *The science of programming*, Springer-Verlag, 1981.
- [Gri82] D. Gries, *A note on a standard strategy for developing loop invariants and loops*, Science of Computer Programming **2** (1982), no. 3, 207–214.
- [Hal11] Paul R. Halmos, *Naive set theory*, 1st ed., Martino Fine Books, 2011.
- [Har12] David Harel, *Algorithmics: The spirit of computing*, 3rd ed., Springer, 2012.
- [HHE20] Steven Halim, Felix Halim, and Suhendry Effendy, *Competitive programming 4: Book I*, LuLu, 2020.
- [Hir75] D. S. Hirschberg, *A linear space algorithm for computing maximal common subsequences*, Communications of the ACM **18** (1975), no. 6, 341–343.

- [HK62] M. Held and R. M. Karp, *A dynamic programming approach to sequencing problems*, Journal of the Society for Industrial and Applied Mathematics **10** (1962), no. 1, 196–210.
- [HK70] Michael Held and Richard M. Karp, *The traveling-salesman problem and minimum spanning trees*, Operations Research **18** (1970), no. 6, 1138–1162.
- [HK71] ———, *The traveling-salesman problem and minimum spanning trees: Part II*, Mathematical Programming **1** (1971), 6–25.
- [HNR68] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael, *A formal basis for the heuristic determination of minimum cost paths*, IEEE Transactions on Systems Science and Cybernetics **4** (1968), no. 2, 100–107.
- [HS74] Ellis Horowitz and Sartaj Sahni, *Computing partitions with applications to the knapsack problem*, Journal of the ACM **21** (1974), no. 2, 277–292.
- [Huf52] David A. Huffman, *A method for the construction of minimum-redundancy codes*, Proceedings of the IRE **40** (1952), no. 9, 1098–1101.
- [Ing62] P. Z. Ingerman, *Algorithm 141: Path matrix*, Communications of the ACM **5** (1962), no. 11, 556.
- [Jar30] Vojtěch Jarník, *O jistém problému minimálním*, Práce Moravské Přírodovědecké Společnosti (1930).
- [Kad23] J. B. Kadane, *Two Kadane algorithms for the maximum sum subarray problem*, Algorithms **16** (2023), no. 11, 519.
- [Kal90] Anne Kaldewaij, *Programming: the derivation of algorithms*, Prentice-Hall, 1990.
- [KET06] Jon Kleinberg and Éva Tardos, *Algorithm design*, Pearson, 2006.
- [Kle56] S. C. Kleene, *Representation of events in nerve nets and finite automata*, Automata Studies (C. E. Shannon and J. McCarthy, eds.), Annals of Mathematics Studies, no. 34, Princeton University Press, Princeton, 1956, pp. 3–41.
- [Knu75] Donald E. Knuth, *Estimating the efficiency of backtrack programs*, Mathematics of Computation **29** (1975), 121–136.
- [Knu88] Donald Knuth, *The art of computer programming: Volume 3 / sorting and searching*, 2nd ed., Addison-Wesley, 1988.
- [Kru56] Joseph B. Kruskal, *On the shortest spanning subtree of a graph and the traveling salesman problem*, Proceedings of the American Mathematical Society **7** (1956), no. 1, 48–50.
- [KS98] Donald L. Kreher and Douglas R. Stinson, *Combinatorial algorithms: Generation, enumeration, and search*, Discrete Mathematics and Its Applications, CRC Press, 1998.
- [Kun11] Kenneth Kunen, *Set theory*, 2nd ed., College Publications, 2011.
- [KV18] Bernhard Korte and Jens Vygen, *Combinatorial optimization: Theory and algorithms*, 6 ed., Springer, 2018.
- [LD60] Ailsa H. Land and Alison G. Doig, *An automatic method of solving discrete programming problems*, Econometrica **28** (1960), no. 3, 497–520.

- [Leh57] Derrick H. Lehmer, *Combinatorial problems with digital computers*, Proceedings of the Fourth Canadian Mathematical Congress, 1957, pp. 160–173.
- [Lev12] Anany Levitin, *Introduction to the design and analysis of algorithms*, 3rd ed., Pearson, 2012.
- [LMSK63] John D. C. Little, Katta G. Murty, Dura W. Sweeney, and Caroline Karel, *An algorithm for the traveling salesman problem*, Operations Research **11** (1963), no. 6, 972–989.
- [LW66] Eugene L. Lawler and David E. Wood, *Branch-and-bound methods: A survey*, Operations Research **14** (1966), no. 4, 699–719.
- [MT90] Silvano Martello and Paolo Toth, *Knapsack problems: Algorithms and computer implementations*, Wiley-Interscience Series in Discrete Mathematics and Optimization, John Wiley and Sons, 1990.
- [OEI] OEIS Foundation Inc., *Sequence A000170: Number of ways of placing n non-attacking queens on an $n \times n$ board*, The On-Line Encyclopedia of Integer Sequences, <https://oeis.org/A000170>.
- [Ohl02] Enno Ohlebusch, *Advanced topics in term rewriting*, 2nd ed., Springer, 2002.
- [Oxl11] James Oxley, *Matroid theory*, 2 ed., Oxford University Press, 2011.
- [Pri57] Robert C. Prim, *Shortest connection networks and some generalizations*, The Bell System Technical Journal **36** (1957), no. 6, 1389–1401.
- [Rom15] D. Romik, *The surprising mathematics of longest increasing subsequences*, Institute of Mathematical Statistics Textbooks, no. 4, Cambridge University Press, New York, 2015.
- [Rou17] Tim Roughgarden, *Algorithms illuminated I: The basics*, 2017.
- [Roy59] B. Roy, *Transitivité et connexité*, Comptes Rendus de l'Académie des Sciences de Paris **249** (1959), 216–218.
- [Rus] John Rust, *Dynamic programming*, pp. 1–26, Palgrave Macmillan, 12.
- [Sch61] C. Schensted, *Longest increasing and decreasing subsequences*, Canadian Journal of Mathematics **13** (1961), 179–191.
- [Sch04] A. Schrijver, *Combinatorial optimization: Polyhedra and efficiency*, Springer, Berlin, 2004.
- [SF13] Robert Sedgewick and Philippe Flajolet, *An introduction to the analysis of algorithms*, 2nd ed., Addison-Wesley Professional, 2013.
- [Ski08] Steven S. Skiena, *The algorithm design manual*, Springer London, 2008.
- [Tar75] Robert Endre Tarjan, *Efficiency of a good but not linear set union algorithm*, Journal of the ACM **22** (1975), no. 2, 215–225.
- [Vaz01] Vijay V. Vazirani, *Approximation algorithms*, Springer, 2001.
- [Wal60] Robert J. Walker, *An enumerative technique for a class of combinatorial problems*, Combinatorial Analysis (Richard E. Bellman and Marshall Hall, Jr., eds.), Proceedings of Symposia in Applied Mathematics, vol. 10, American Mathematical Society, Providence, RI, 1960, pp. 91–94.

-
- [War62] S. Warshall, *A theorem on Boolean matrices*, Journal of the ACM **9** (1962), no. 1, 11–12.
- [WF74] R. A. Wagner and M. J. Fischer, *The string-to-string correction problem*, Journal of the ACM **21** (1974), no. 1, 168–173.
- [Wil64] John W. J. Williams, *Algorithm 232: Heapsort*, Communications of the ACM **7** (1964), no. 6, 347–348.

